Mobility Matters

A Living Survey of Open Problems on Autonomous Agents, Robots and Particles

What is known, what is tight, and what is open, under which assumptions.

Debasish Pattanayak

Version 2026-10-14

AI USE Claude (Anthropic) was used in cataloguing the problems.

How to cite

Debasish Pattanayak. *Mobility Matters: A Living Survey of Open Problems on Autonomous Agents, Robots and Particles.* Version 2026-10-14, 2026. https://drdebmath.github.io/mobility-matters/

BibTeX

```
@misc{Pattanayak2026,
  author = {Debasish Pattanayak},
  title  = {Mobility Matters: A Living Survey of Open Problems
            on Autonomous Agents, Robots and Particles},
  year   = {2026},
  note   = {Version 2026-10-14},
  url    = {https://drdebmath.github.io/mobility-matters/},
}
```

This is a living survey: the website and this book are generated from the same data, and each version is dated. The latest version is at https://drdebmath.github.io/mobility-matters/.

Contents

Acknowledgements v

How to read this atlas 1

I Models and their assumptions 2

1 Notation 4

2 Mobile agents on graphs 5

2.1 How the assumptions fit together . . . 5

2.2 Notation . . . 7

2.3 Assumption axes . . . 9

3 Mobile robots in the plane 16

3.1 How the assumptions fit together . . . 16

3.2 Notation . . . 18

3.3 Assumption axes . . . 18

Which model is stronger . . . 25

4 Search in geometric domains 32

4.1 How the assumptions fit together . . . 32

4.2 Notation . . . 33

4.3 Assumption axes . . . 34

5 Programmable matter 36

5.1 How the assumptions fit together . . . 36

5.2 Notation . . . 37

5.3 Assumption axes . . . 38

II Perspectives 41

6 Why compute by moving 43

7 Symmetry is the adversary 46

8 Finding your way 49

9 From exploration to dispersion, and beyond 52

10 Meeting at one place 55

11 Weak robots in the plane 58

12 From robots to particles 61

13 From the cow path to evacuation 64

14 When agents fail 67

15 What forces the bounds 70

III Mobile agents on graphs 73

16 Exploration 75

16.1 Graph exploration by a single agent . . . 75
16.2 Treasure hunt . . . 93
16.3 Rendezvous . . . 100
16.4 Gathering in graphs . . . 109
16.5 Map construction . . . 115
16.6 Exploration by oblivious robots . . . 121
16.7 Exploration by luminous robots . . . 125
16.8 Exploration of recolourable graphs . . . 152
16.9 Collaborative exploration . . . 168
16.10 Dispersion . . . 180
16.11 Black hole search . . . 206
16.12 Exploration of dynamic graphs by teams . . . 216
16.13 Network decontamination . . . 219

17 Computing with agents 225

17.1 Leader election by agents . . . 225
17.2 Maximal independent set by agents . . . 230
17.3 Minimal dominating set by agents . . . 234
17.4 Triangle and butterfly counting by agents . . . 237
17.5 Minimum spanning tree by agents . . . 242
17.6 BFS tree by agents . . . 245

Claude (Anthropic) was used in cataloguing the problems.

IV Mobile robots in the plane 249

18 Formation problems 251

18.1 Pattern formation . . . 251
18.2 Circle formation . . . 262
18.3 Uniform circle formation . . . 265
18.4 Universal pattern formation . . . 272
18.5 Pattern sequences . . . 275
18.6 Mutual visibility . . . 280
18.7 k-grouping . . . 288
18.8 Gathering . . . 293
18.9 Near-gathering . . . 315
18.10 Partitioning . . . 319
18.11 Convergence . . . 322

19 Symmetry breaking and spreading 328

19.1 Scattering . . . 328
19.2 Leader election in the plane . . . 333
19.3 Separation . . . 337
19.4 Flocking . . . 340
19.5 Uniform deployment on a ring . . . 343

V Search in geometric domains 347

20 Search, evacuation and patrolling 349

20.1 Linear search (cow path) . . . 349
20.2 Evacuation . . . 356
20.3 Chauffeuring a crashed robot . . . 378
20.4 Patrolling . . . 382

VI Programmable matter 388

21 Self-organising particle systems 390

21.1 Leader election in programmable matter . . . 390
21.2 Shape formation in programmable matter . . . 402
21.3 Coating . . . 407
21.4 Compression . . . 411
21.5 Separation of coloured particles . . . 414

Claude (Anthropic) was used in cataloguing the problems.

21.6 Alignment . . . 417
21.7 Bridging . . . 419
21.8 Convex hull formation . . . 422
21.9 Scattering of particles . . . 425
21.10 Energy distribution . . . 428
21.11 Shape formation by hybrid programmable matter . . . 430
21.12 Centralised transformations of modular robots . . . 434

Appendices **439**

A All open questions **440**

B Index of assumptions **457**

C Timeline **473**

Bibliography **489**

Claude (Anthropic) was used in cataloguing the problems.

Acknowledgements

The atlas is maintained by Debasish Pattanayak. It grows with help from readers: everyone whose correction, result or suggested problem is accepted through a GitHub issue is listed here, with the issue that brought it in.

Tools and data

- **Claude (Anthropic)**: AI assistance in searching the literature, reading papers, cataloguing results, drafting text and building the site and the book. Every entry marked verified was checked against the full text of the cited paper; the maintainer reviews the content.
- **Crossref, DataCite, arXiv and OpenAlex**: bibliographic metadata and open access to papers
- **KaTeX and Marked**: mathematics and text rendering on the website
- **LuaLaTeX and the Libertinus fonts**: typesetting of the book

The atlas rests on the work of the authors of the papers it cites; the bibliography lists them all.

How to contribute

Corrections, new results and missing problems are welcome as GitHub issues at `https://github.com/drdebmath/mobility-matters/issues`; every submission is checked against the full text of the paper, and accepted contributions are acknowledged here.

How to read this atlas

The problem space is a **directed acyclic graph**. Independent assumptions, one per axis (the graph or domain, the scheduler, what the entities see, remember and share, the faults), come together to define a problem, and each particular setting appears as a leaf of the graph: a problem solved under one set of assumptions. Results move along the edges: an algorithm for weaker assumptions works under stronger ones, and a lower bound for stronger assumptions holds under weaker ones. Assumption sets also change over time: later work minimises the set a result needs, a stronger assumption subsumes weaker ones, and some assumptions become irrelevant in the presence of better ones. Every setting records the proven lower bounds and what forces them, the paper that introduced it, the current best result, and the gap between them.

282	**234**	**18**	**43**	**208**	**174**
settings	tight bounds	near-optimal	polynomial gaps	unknown	open questions

Status of a bound. Each measure of each variant carries a status, judged against the best proven lower bound for that variant: ● *tight*; ● *near-optimal*; ● *polynomial gap*; ● *unknown*; ● *impossible*. Within a variant, results are listed in order of publication and marked [introductory], [current best], [also], [disputed proof].

Verification. A variant marked VERIFIED has its bounds checked against the papers; DRAFT marks entries with something not yet checked, which its notes name.

Order of the book. Part I opens with the notation (Chapter 1), then introduces the four models and their assumption axes: where each assumption comes from, why the community studies it, and its formal definition. Part II reads across problems and models. The remaining parts give the problems, one part per model and one chapter per family, each problem with its statement, assumption tree and every catalogued setting. The appendices collect the open questions, starting with the most consequential, an alphabetical index of every assumption with its definition, the timeline and the bibliography.

Two orders. The parts arrange problems logically, by model and family; Appendix C lists the same material chronologically.

Part I

Models and their assumptions

What an agent, a robot or a particle can see, remember and do, and what the adversary controls. Each chapter tells how the assumptions of one model fit together, then gives every axis with its origin and formal definitions.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 1

Notation

Each model uses the symbols of its own literature, so that a bound reads the same here as in the papers it comes from. Inside one model a symbol has exactly one meaning. The same letter may mean different things in different models (for example n counts nodes on graphs and robots in the plane); essays that cross models say which model a symbol belongs to. Where a paper writes a different symbol for the same quantity, the table says so under *papers write*. Symbols that a single paper defines for its own result are defined where they are used and listed under that problem.

General conventions

Symbol	Meaning	Papers write
$O, \Omega, \Theta, o, \omega$	asymptotic notation, as n (or the stated parameter) grows; $\tilde{O}$ hides polylogarithmic factors	
log	logarithm to base 2 unless stated	
ε	an arbitrarily small positive real (a constant unless stated)	
κ	a positive constant whose value does not matter for the statement	
t	time: a round, or a point of time in an asynchronous execution	
round	in synchronous settings (FSYNC, SSYNC, synchronous agents), one step in which every activated entity performs one full cycle	
epoch	in asynchronous settings, an interval of time in which every entity completes at least one full cycle; it need not be the shortest such interval. The execution is divided into consecutive epochs, and a bound of X epochs holds for every such division. Programmable matter calls the same notion an *asynchronous round*	

The symbols of each model are listed at the start of its chapter; Appendix B indexes every assumption with its formal definition.

Chapter 2

Mobile agents on graphs

Memory-bounded agents move along the edges of a port-labelled graph, by default with anonymous nodes; what they can see, remember, say and rely on is fixed by its assumption axes.

> **What is optimised**
> Once a task is known to be solvable, the community has mostly asked for it to be **fast**: first the number of rounds (or epochs, or edge traversals), then the **memory per agent** in bits. Problems with hostile nodes or agents, such as black hole search, ask instead for the **smallest number of agents** that still succeeds.

The graph $G = (V, E)$ has n nodes, m edges, maximum degree Δ and diameter D. Nodes are **anonymous** (no identifiers) and, unless assumed otherwise, have **no memory** (no whiteboards). Edges are **port-labelled**: the edges incident to a node v carry distinct labels $1, \dots, \deg(v)$, and the two endpoints of an edge label it independently. An agent at v sees $\deg(v)$ and, after arriving through an edge, the port through which it entered.

There are k agents. Each has a bounded amount of **memory** (persistent across rounds, measured in bits) and runs the same algorithm. In every step an agent (i) exchanges information with whatever agents its communication model allows, (ii) computes, and (iii) either stays or moves through one port. Agents moving in opposite directions along the same edge do not see each other.

Agents know **nothing** about n, m, k, Δ or D unless a result says so.

How the axes work. Every axis is a scale of assumptions. Level 0 is *no assumption*: the environment may be as hostile as the axis allows (a changing, directed graph; asynchronous, anonymous, Byzantine agents that cannot talk). Each higher level adds one assumption. A problem variant is a point on all axes at once, and the catalogue arranges variants as a tree whose root assumes nothing and whose edges each add assumptions. The *standard setting* of the literature (static undirected graph, $k \leq n$, synchronous rounds, local communication, unique identifiers, no faults) sits several steps below the root.

The two standard cost measures are the number of rounds until the problem is solved (time) and the number of bits of memory per agent (memory).

2.1 How the assumptions fit together

Agents that move through a network

The model asks what a team of computing entities can achieve when the only way to act on a network is to walk along its edges. Pelc names the two motivating systems: autonomous robots that start in different places of a terrain or a labyrinth, and software agents, "mobile pieces of software that navigate in a computer network in order to perform maintenance of its components or to collect data distributed in nodes of the network" (Pelc 2019). Since an algorithm depends only on what the entities perceive, remember and how they move, the same model serves both, and there is no central monitor, so the questions belong to distributed computing (Pelc 2019). Later problems kept this double reading:

dispersion was motivated by self-driving electric cars looking for free charging stations with "car-to-car interactions" only (Augustine and Moses Jr. 2018), and black hole search by host attacks on mobile code, a site that destroys incoming agents without a trace (Di Luna et al. 2025b). Each axis of the model fixes one thing that the agents may or may not rely on: the network (dynamics, direction, topology, node identifiers, node memory), the team (count, identifiers, randomness, faults, communication), time, and the start.

Every axis is a freedom of the adversary

Level 0 on each axis is what the adversary controls when nothing is assumed. In the deterministic scenario "the initial positions of the agents are chosen by an adversary which models the worst-case situation" (Pelc 2019); under asynchrony the adversary also chooses when and how fast each move happens (Pelc 2019); in a dynamic network it chooses the edges of every round after seeing the states of the nodes (Kuhn et al. 2010); and with Byzantine agents it controls part of the team (Dieudonné et al. 2014). The remaining axes govern **symmetry**. Two identical agents that run the same deterministic algorithm in an oriented ring and start at the same time keep their distance forever (Pelc 2019): anonymous agents, anonymous nodes, no coins and an adversarial start leave the adversary free to keep the configuration symmetric. Each higher level removes one of these freedoms (labels, node names, coins, a common start, a stable or undirected graph, synchronous rounds, honest agents) and makes the problem easier. This is why results move along the scale: an algorithm proved against a stronger adversary holds against a weaker one, and an impossibility for a weaker adversary holds for a stronger one, as the inclusion hierarchy of time-varying graph classes already makes explicit for dynamics (Casteigts et al. 2012).

How the axes interact

The axes are independent as assumptions but not in their effect. **Identifiers and the start.** Anonymity matters only when the rest of the instance is symmetric: on a line with an odd number of nodes two identical agents meet at the centre, and from asymmetric positions they can still break symmetry (Pelc 2019); from a rooted start, where agents share their position, only labels or coins can decide which agent settles first, which is how Molla and Moses Jr. use randomness in dispersion (Molla and Moses Jr. 2019). **Faults and identifiers.** The two Byzantine levels differ only in whether a faulty agent may forge its label; with known n this is the difference between one good agent sufficing and needing $f + 1$, but with unknown n both need $f + 2$ (Dieudonné et al. 2014) (Bouchard et al. 2016). **Faults and timing.** Under full asynchrony a waiting agent cannot tell a slow agent from a crashed one, so gathering with crash faults is impossible; Pelc restores it by fixing each agent's speed (Pelc 2018). **Dynamics and timing.** In temporal graphs "the distributed computation is by definition synchronous" (Di Luna et al. 2025b), so dynamic settings in the atlas are synchronous. **Communication and the graph.** Global communication does not tell two agents on neighbouring nodes which edge joins them, so edges must still be traversed (Kshemkalyani et al. 2020b). **Node memory and node identifiers.** A whiteboard or a pebble lets an agent recognise a node it has seen, the service that node labels would give (Bender et al. 2002) (Fraigniaud and Ilcinkas 2004); without either, settled agents play that role in dispersion (Kshemkalyani and Ali 2019). **Direction and exploration.** An undirected graph can be explored by depth-first search in fewer than $2|V|$ steps, while some directed graphs need $\Theta(|V|^2)$ (Erlebach et al. 2021), because backtracking along an edge is no longer available (Italiano et al. 2022).

Why the standard setting is where it is

The setting most papers start from is a static, undirected, connected graph with anonymous nodes and locally labelled ports, synchronous rounds, deterministic fault-free agents with distinct labels,

Claude (Anthropic) was used in cataloguing the problems.

communication between co-located agents only, and no node memory. Each choice has a stated reason. Undirected edges capture movement in both directions along a link, and connectivity is necessary for agents in different components to meet at all (Pelc 2019). Anonymous nodes are assumed because labels may be withheld for privacy or security, may be unreadable by weak sensors, and because readable labels would reduce rendezvous to exploration; port numbers stay, because without them an agent may never reach all neighbours of a node of degree at least 3 (Pelc 2019). Distinct agent labels are the cheapest deterministic way to break symmetry (Pelc 2019) (Dessmark et al. 2006); determinism is kept because probabilistic approaches "typically fail to provide absolute guarantees" (Devismes et al. 2026). Local communication matches the physical picture of agents that meet (Augustine and Moses Jr. 2018). Synchronous rounds let agents use waiting as a tool (Pelc 2019). The research programme then removes these assumptions one at a time: asynchrony (De Marco et al. 2006) (Kshemkalyani and Ali 2019), dynamic graphs (Di Luna et al. 2020a) (Agarwalla et al. 2018), directed graphs (Italiano et al. 2022), Byzantine agents (Dieudonné et al. 2014) (Molla et al. 2021), silent agents (Gorain et al. 2022b), and repeated labels (Devismes et al. 2026). The level-0 corner of the model, where none of the assumptions hold, is where most open questions sit.

2.2 Notation

Symbols of this model. Inside the model each symbol has one meaning; *papers write* gives other symbols found in the literature.

Symbol	Meaning	Papers write	Used in
$G = (V, E)$	the graph; for a directed graph E is a set of arcs, written $G = (V, A)$ where both appear		whole model
n	number of nodes, $n = \|V\|$		whole model
m	number of edges (arcs)		whole model
Δ	maximum degree		whole model
Δ_{out}	maximum out-degree of a directed graph	d in some exploration papers	whole model
$\deg(v)$, $\deg_{out}(v)$	degree and out-degree of node v	$\delta(v)$, $\delta_{out}(v)$	whole model
D	diameter of the graph (for a tree explored from a root: its height)		whole model
u, v, w	nodes (local names)		whole model
s	the starting node of a single agent, the root of a rooted team, or the home base of black hole search and decontamination	h (home base), r (root)	whole model
k	number of agents		whole model
ℓ	number of initially occupied nodes; $\ell = 1$ is a rooted start, $\ell = k$ a dispersed start		Dispersion
f	maximum number of faulty agents (crashed or Byzantine)		whole model
N	a known upper bound on n	$\hat{n}$	whole model
$\mathscr{L}$	the set of agent labels in use; $\|\mathscr{L}\|$ distinct labels, $\|\mathscr{L}\| = 1$ anonymous, $\|\mathscr{L}\| = k$ distinct identifiers	ℓ (homonyms)	whole model
L	the largest possible label: labels are integers in $\{1, \dots, L\}$	λ (largest identifier), μ (largest label)	whole model
λ	the smallest label among the agents (for two agents: the smaller label)	l (Pelc 2019)	whole model
τ	delay between the starting times of two agents		whole model

Claude (Anthropic) was used in cataloguing the problems.

Symbol	Meaning	Papers write	Used in
d	distance from the starting node to the target (the treasure), or between the starting nodes of two agents in rendezvous	D for the distance between starting positions (Pelc 2019)	whole model
v^*	the node of the treasure	t	whole model
$e(d)$	number of edges with an endpoint at distance less than d from the start		Treasure hunt
$w(\cdot)$, W	weight of the cheapest path type to the treasure (treasure hunt); W its maximum over the two directions (rendezvous)		Treasure hunt, Rendezvous
h	communication range in hops (h-hop communication)	ℓ (Saxena et al. 2025)	whole model
T	window length in T-interval and T-path connectivity of dynamic graphs		whole model
μ	maximum number of edges missing in a round (μ-bounded 1-interval connected graphs)	ℓ-bounded (Saxena et al. 2025)	whole model
η	number of transient edges of a temporally connected graph		Exploration of dynamic graphs by teams
δ	longest time an edge of a dynamic ring may be absent		Graph exploration by a single agent
p	number of pebbles		whole model
b	number of bits of an agent's memory, when it is a parameter		whole model
β	number of bits of a label written at a node in advance		Graph exploration by a single agent
ϕ	deficiency of a directed graph: the number of arcs whose addition makes it Eulerian	d (Italiano et al. 2022)	Dispersion, Graph exploration by a single agent
ν	number of leaves of a tree	λ	Rendezvous
CR(k)	competitive ratio of a collaborative exploration algorithm with k agents		Collaborative exploration
$X(n)$	time to explore any n-node graph		Dispersion
port_v	port numbering at node v	λ_v	Graph exploration by a single agent
R	running time, in rounds, of a message-passing algorithm simulated by agents		Leader election by agents, Maximal independent set by agents, Minimal dominating set by agents, Minimum spanning tree by agents, BFS tree by agents, Triangle and butterfly counting by agents
K	number of states of a finite-automaton agent		Graph exploration by a single agent, Exploration of recolourable graphs
γ	exponent of the team size in large-team collaborative exploration, $k = Dn^\gamma$		Collaborative exploration
θ	the exponent of Reingold's universal exploration sequences		Treasure hunt
(q, ρ)	type of a path: its largest port q and its length ρ		Treasure hunt

Claude (Anthropic) was used in cataloguing the problems.

Symbol	Meaning	Papers write	Used in
z	number of nodes of non-zero capacity		Dispersion
r	radius of a ball of the grid		Dispersion
c	number of colours (of a robot's light, or of the colours a node can hold)	κ, ℓ	Exploration by luminous robots, Exploration of recolourable graphs
φ	visibility range of a myopic robot, in hops		Exploration by luminous robots
$a \times b$	size of a grid or torus with a rows and b columns		Exploration by luminous robots
$\psi(v)$	current colour of node v	$c(v)$	Exploration of recolourable graphs
c_0	initial colour of the nodes		Exploration of recolourable graphs
π_v	rotor pointer at node v: the port the next departing agent takes		Exploration of recolourable graphs
$N(v)$	the neighbours of node v		Exploration of recolourable graphs
circ(G)	circumference of G: the length of its longest cycle	k	Exploration of recolourable graphs
g	a known upper bound on the circumference		Exploration of recolourable graphs

2.3 Assumption axes

Each axis says where the assumption comes from and why the community studies it, then defines its values. Every axis is a scale. Level 0 assumes nothing; each higher level is a stronger assumption, which makes a problem easier. Values on the same level are alternatives that do not imply each other. An algorithm designed for weaker assumptions works under stronger ones; a lower bound proved under stronger assumptions holds under weaker ones.

Graph dynamics

Graphs that change all the time entered distributed computing as models of mobile, wireless and peer-to-peer networks. Kuhn, Lynch and Oshman introduced T-interval connectivity in a worst-case model where an adversary picks the edges of each round; "for $T = 1$ this means that the graph is connected in every round, but changes arbitrarily between rounds" (Kuhn et al. 2010). Casteigts, Flocchini, Quattrociocchi and Santoro unified the many models of time-varying graphs into a hierarchy of classes: the always-connected class, used earlier by O'Dell and Wattenhofer (as reported there), and recurrent connectivity, in which journeys exist from every time onward, which is level 0 here (Casteigts et al. 2012). They put the reason for the scale in one sentence: in these systems "changes are not anomalies but rather integral part of the nature of the system" (Casteigts et al. 2012). Kuhn et al. show the adversary's strength with a token that must keep moving: in a 1-interval connected graph the adversary can keep it between two nodes forever (Kuhn et al. 2010), which is in effect a single agent that cannot explore. Mobile agents entered the setting with live exploration of 1-interval connected rings (Di Luna et al. 2020a) and dispersion in dynamic rings (Agarwalla et al. 2018); offline exploration of temporal graphs is the centralised counterpart (Erlebach et al. 2021).

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**any dynamics**	A **temporal graph** is a sequence $\mathcal{G} = (G_t)_{t\geq 0}$ with $G_t = (V, E_t)$ on a fixed node set. It is *temporally connected* if for all $u, v \in V$ and every time t there is a journey (a path whose edges appear at non-decreasing times $\geq t$) from u to v. No G_t need be connected.
1	**connected every round**	$\mathcal{G} = (G_t)_{t\geq 0}$ is **1-interval connected** if every snapshot $G_t = (V, E_t)$ is connected. More generally it is T-interval connected if for every t the graph $\left(V, \bigcap_{i=t}^{t+T-1} E_i\right)$ is connected.
2	**static graph**	$G_t = G$ for all t: the edge set never changes.

Edge direction

Most of the mobile-agent literature uses undirected graphs because this "captures the ability of the agents to move in both directions along each link" (Pelc 2019). Directed graphs come from the exploration literature. The earliest paper in the atlas on exploring an unknown directed graph is Deng and Papadimitriou's (Deng and Papadimitriou 1999), continued by Albers and Henzinger (Albers and Henzinger 2000). Bender, Fernández, Ron, Sahai and Vadhan asked what a robot needs to map an unlabelled strongly connected digraph, and showed that one pebble suffices when a bound on n is known (Bender et al. 2002); Fraigniaud and Ilcinkas studied how little memory digraph exploration needs (Fraigniaud and Ilcinkas 2004). Direction removes backtracking: a static undirected graph is explored by depth-first search in fewer than $2|V|$ steps, while some static directed graphs need $\Theta(|V|^2)$ steps (Erlebach et al. 2021). Italiano, Pattanayak and Sharma posed dispersion on directed anonymous graphs because "the existing approaches for undirected graphs rely substantially on visiting a graph edge in both directions", and showed that without strong connectivity dispersion may be unsolvable (Italiano et al. 2022).

Level	Value	Formal definition
0	**directed**	$G = (V, A)$ with $A \subseteq V \times V$; an agent at u may move along $(u, v) \in A$ only, never back along it unless $(v, u) \in A$. G is strongly connected.
1	**undirected**	$G = (V, E)$ with $E \subseteq \binom{V}{2}$; every edge can be traversed in both directions.

Number of agents

Most tasks in this model are posed for a team of any size: gathering an unknown team of anonymous agents (Dieudonné and Pelc 2016), black hole search, where the size of the team is the main measure (Dobrev et al. 2006), and collective exploration by k agents (Fraigniaud et al. 2006a). The cap $k \leq n$ comes from dispersion, where every agent needs a node of its own. Augustine and Moses Jr. posed dispersion with $k = n$, motivated by electric cars that should not queue at one charging station (Augustine and Moses Jr. 2018), and Kshemkalyani and Ali relaxed it to $k \leq n$ (Kshemkalyani and Ali 2019). Molla and Moses Jr. extended dispersion to any k, asking for at most $\lceil k/n \rceil$ agents per node (Molla and Moses Jr. 2019). The extension is not free: with $k > n$ Byzantine dispersion is impossible deterministically whenever $\lceil k/n \rceil > \lceil (k - f)/n \rceil$ (Molla et al. 2021). The axis thus separates tasks where the team must fit into the graph from tasks where the team size is unconstrained.

Level	Value	Formal definition
0	**any** k	The number of agents $k \geq 1$ is arbitrary; $k > n$ is allowed.
1	$k \leq n$	The number of agents satisfies $1 \leq k \leq n$.

Identifiers

Claude (Anthropic) was used in cataloguing the problems.

Pelc's survey names anonymous versus labelled agents as one of the two main pairs of assumptions in rendezvous, and gives the reason: two identical agents running the same deterministic algorithm in an oriented ring, starting together, keep the same distance forever (Pelc 2019). Distinct labels break this symmetry: in the work of Dessmark, Fraigniaud, Kowalski and Pelc each agent knows only its own label and uses it as a parameter of the common algorithm, and rendezvous becomes feasible in every graph at cost polynomial in n, the delay and $\log \lambda$, λ the smaller label; without labels it fails already on two nodes joined by an edge, since both agents always stay or always move together (Dessmark et al. 2006). Without labels the only asymmetry comes from the graph and the starting positions; Dieudonné and Pelc characterised the configurations of anonymous agents that can be gathered (Dieudonné and Pelc 2016). Homonyms, several agents sharing a label, come from Byzantine agreement with homonyms (Delporte-Gallet et al. 2013) and were taken to robots in the plane (Bouzid and Lamani 2011). In graphs, Devismes, Dieudonné and Labourel motivate them by the cost of "generating and maintaining globally unique labels" and by privacy, and ask how many distinct labels gathering really needs (Devismes et al. 2026).

Level	Value	Formal definition
0	**anonymous**	All agents run the same deterministic algorithm on the same input; no agent has a label. Formally the label function is constant: $\mathrm{id}(a) = \mathrm{id}(b)$ for all agents a, b.
1	**homonymous**	Each agent a has a label $\mathrm{id}(a) \in \mathscr{L}$ known to itself (and revealed on meeting), but the labelling need not be injective: $1 \le \lvert\{\mathrm{id}(a)\}\rvert \le k$. With $\lvert\mathscr{L}\rvert$ distinct labels, $\lvert\mathscr{L}\rvert = 1$ is the anonymous case and $\lvert\mathscr{L}\rvert = k$ the eponymous one (Delporte-Gallet et al. 2013).
2	**distinct IDs (eponymous)**	**Distinct identifiers** (eponymous agents): the label function $\mathrm{id} : \{1, \dots, k\} \to \{1, \dots, L\}$ is injective; usually $L = k^{O(1)}$, so a label takes $O(\log k)$ bits.

Randomness

Pelc counts determinism versus randomisation among the two assumptions that most shape the methods for rendezvous: a deterministic algorithm faces adversarial starting positions and is judged by its worst-case cost, while randomised algorithms often use random walks and are judged by expected cost (Pelc 2019). The random-walk view of graph traversal goes back to Aleliunas et al., who connected random walks with universal traversal sequences (Aleliunas et al. 1979). In this model randomness serves mainly to break symmetry: Molla and Moses Jr. let co-located anonymous agents elect a local leader by coin flips, so that dispersion needs $O(\log \Delta)$ bits per agent instead of the $\Omega(\log n)$ bits of deterministic algorithms (Molla and Moses Jr. 2019). Deterministic algorithms are level 0 because they promise a bound for every execution; Devismes, Dieudonné and Labourel note that probabilistic approaches "typically fail to provide absolute guarantees in terms of complexity and/or accuracy" (Devismes et al. 2026). Hence a deterministic algorithm also counts in the randomized setting, and a lower bound against randomized algorithms holds for deterministic ones.

Level	Value	Formal definition
0	**deterministic**	The next state and move of an agent are a function of its current state and what it observes.
1	**private coins**	Each agent additionally has access to independent fair coin flips; guarantees hold with probability 1, in expectation, or with high probability, as stated per result.

Faults

Claude (Anthropic) was used in cataloguing the problems.

The first faults studied for mobile agents sat in the network: Dobrev, Flocchini, Prencipe and Santoro introduced the black hole, a node that destroys every agent that enters it, in rings and then in arbitrary networks (Dobrev et al. 2007) (Dobrev et al. 2006); it models a host that trashes incoming code or crashes without notice (Di Luna et al. 2025b). Faults in the agents, which this axis records, came later. Dieudonné, Pelc and Peleg started the study of gathering with Byzantine agents and defined the two levels used here: a strongly Byzantine agent "can choose an arbitrary port when it moves and it can convey arbitrary information to other agents, while a weakly Byzantine agent can do the same, except changing its label" (Dieudonné et al. 2014). The distinction is about identity and it changes the answer: with known n a single good agent gathers despite f weakly Byzantine agents, but $f+1$ good agents are needed against strongly Byzantine ones, even in a ring of known size, while with unknown n both cases need $f+2$ (Dieudonné et al. 2014); Dieudonné, Pelc and Peleg left the strongly Byzantine upper bounds at $2f+1$ (known n) and $4f+2$ (unknown n), and Bouchard, Dieudonné and Ducourthial closed them (Bouchard et al. 2016) (Pelc 2019). Crash faults are the mildest level; Pelc showed that with fully asynchronous agents gathering with crashes is impossible, since a waiting agent cannot tell a slow agent from a crashed one, and solved it when each agent has a fixed speed (Pelc 2018). Dispersion adopted both Byzantine levels (Molla et al. 2021) and crash faults (Chand et al. 2023b).

Level	Value	Formal definition
0	**any faults**	Up to f agents behave arbitrarily (adversarially) and may **forge** identifiers, i.e. report any label in $\mathscr{L}$.
1	**at most weak Byzantine faults**	Up to f agents behave arbitrarily but always report their own true identifier.
2	**at most crash faults**	Up to f agents may stop forever at an adversarially chosen time; a crashed agent stays at its node (or vanishes, as stated per result) and never acts again.
3	**no faults**	All agents follow the algorithm forever.

Communication

Communication between co-located agents is the classical assumption of rendezvous and gathering, where agents that meet "can exchange all information they currently have" (Pelc 2019). Augustine and Moses Jr. kept it for dispersion, assuming robots "can only communicate with other robots present on their own node", in line with their picture of cars that interact only car to car (Augustine and Moses Jr. 2018). Di Luna, Dobrev, Flocchini and Santoro list the forms used in exploration: face-to-face, wireless (agents on different nodes), and whiteboards (Di Luna et al. 2020a). Kshemkalyani, Molla and Sharma introduced global communication to dispersion to measure "how much global communication helps", and observed that it does not tell two agents on neighbouring nodes which edge joins them, so the graph must still be walked (Kshemkalyani et al. 2020b). For collective tree exploration Fraigniaud, Gąsieniec, Kowalski and Pelc showed that without communication k agents gain nothing over one (Fraigniaud et al. 2006a). Between the two, Italiano, Pattanayak and Sharma studied a **1-hop model**, in which "a robot may also communicate with robots positioned on its 1-hop neighbors", and used it to disperse on directed graphs in $O(kD)$ rounds by broadcasting and gathering information along the graph (Italiano et al. 2022). Saxena, Kaur and Mondal generalised it to **ℓ-hop communication** (h-hop here), "a spectrum between f-2-f ($l=0$) and global communication ($l=D$)", and showed that for balanced dispersion on 1-bounded 1-interval connected graphs agents need 1-hop visibility and 1-hop communication (Saxena et al. 2025); the axis writes h for their ℓ, which the atlas uses for occupied nodes. At level 0, Gorain, Mandal, Mondal and Pandit asked whether talking is needed at all, and dispersed agents that only learn whether they are alone and whether the number of agents at their node changed (Gorain et al. 2022b).

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**no communication**	Agents exchange no messages; an agent at v observes only the number of agents at v (and, where stated, whether that number changed).
1	**local communication**	Agents at the same node at the same time can read each other's entire memory (face-to-face communication). Agents crossing on an edge do not meet.
2	**h-hop communication**	For a fixed $h \geq 1$, stated with each result: at every round an agent at v can read the memory of every agent at a node u with $\mathrm{dist}(v, u) \leq h$ (in the current snapshot G_t for a dynamic graph), in addition to the co-located agents. $h = 1$ is **1-hop communication** (agents at neighbouring nodes communicate); local communication is the case $h = 0$, and global communication the case $h \geq D$. Reading another agent's memory does not tell an agent which port leads to it.
3	**global communication**	At every round every agent can read the memory of every other agent, wherever it is.

Timing

Pelc names synchronous versus asynchronous movement as the second main pair of assumptions for rendezvous: synchronous agents can alternate measured periods of activity and waiting, "a powerful tool", while asynchronous agents move at speeds the adversary varies, so algorithms must force trajectories to overlap (Pelc 2019). The asynchronous model for agents in graphs was introduced by De Marco, Gargano, Kranakis, Krizanc, Pelc and Vaccaro for rendezvous; since a meeting at a node cannot be guaranteed even in the two-node graph, agents may also meet inside an edge (De Marco et al. 2006) (as described in Pelc 2019). Black hole search was posed for asynchronous agents with whiteboards (Dobrev et al. 2006), while dispersion started synchronous and Kshemkalyani and Ali gave its first asynchronous algorithms (Kshemkalyani and Ali 2019). Intermediate schedulers exist: Di Luna et al. brought the semi-synchronous model, "common in the context of mobile agents in continuous spaces", to agents in graphs (Di Luna et al. 2020a). Asynchrony interacts with other axes: it makes gathering with crash faults impossible (Pelc 2018), and dynamic graphs are synchronous by definition (Di Luna et al. 2025b).

Level	Value	Formal definition
0	**asynchronous**	Each agent's Communicate–Compute–Move steps take finite but unbounded, adversarially chosen time. Time is measured in epochs. An **epoch** is an interval of time in which every agent completes at least one full Communicate–Compute–Move step; it need not be the shortest such interval. Time is counted by dividing the execution into consecutive epochs, and a bound of X epochs holds for every such division.
1	**synchronous**	Time proceeds in rounds $t = 1, 2, \ldots$; in every round every agent communicates, computes and moves (at most one edge) simultaneously.

Node memory

Whiteboards are the shared memory of the mobile-agent model: each node keeps a little storage that visiting agents read and write in mutual exclusion, "where the agents communicate by writing on and reading from whiteboards present in each node" (Di Luna et al. 2020a). They are the standard tool in black hole search, posed for asynchronous agents with whiteboards (Dobrev et al. 2006), and in map

Claude (Anthropic) was used in cataloguing the problems.

construction, where identical agents with whiteboards build a map exactly when symmetry can be broken (Das et al. 2007). Their purpose is marking: an agent that can leave information at a node can recognise it later, which is what node labels would otherwise provide (Bender et al. 2002). The cheaper markers are pebbles or tokens, and black hole search showed that whiteboards can be replaced by them (Dobrev et al. 2013) (Flocchini et al. 2012b); for digraphs, whiteboards of $O(\log \Delta_{out})$ bits let an agent with constant memory explore (Fraigniaud and Ilcinkas 2004). Pelc counts the ability to see marks among the assumptions that most change methods, since it "significantly facilitates" rendezvous (Pelc 2019). Level 0 has no node memory, as in dispersion, where settled agents serve as markers instead (Kshemkalyani and Ali 2019).

Level	Value	Formal definition
0	**no whiteboards**	Nodes have no storage: no information can be left at a node except by an agent staying there.
1	**whiteboards**	Each node v has a whiteboard of bounded size that agents at v can read and write, with mutual exclusion.

Node identifiers

Anonymous nodes with locally labelled ports are the standard network of this model, and Pelc gives the reasons. Practically, nodes may withhold their labels "for privacy or security reasons", or the agents' sensors may be too weak to read them; methodologically, readable labels would let agents meet at the smallest label, reducing rendezvous to exploration (Pelc 2019). Port numbers are kept because without them an agent may be unable to visit all neighbours of a node of degree at least 3, and because reading a port needs far less than reading a name (Pelc 2019). The same convention underlies memory-bounded exploration, where a finite automaton explores a graph "whose nodes are unlabeled and whose edge ports are locally labeled" (Fraigniaud et al. 2005a). Labelled nodes belong to the online-exploration tradition, in which the agent recognises nodes and builds a map (Deng and Papadimitriou 1999) (Albers and Henzinger 2000), and to collective exploration, where nodes are typically labelled (Augustine and Moses Jr. 2018) (Fraigniaud et al. 2006a). With labelled nodes many symmetry questions disappear, which is why the axis separates navigation from symmetry breaking.

Level	Value	Formal definition
0	**anonymous nodes**	Nodes have no identifiers; an agent at v sees only $\deg(v)$ and the port numbers $1, \dots, \deg(v)$, which are assigned locally and independently at each endpoint of an edge.
1	**labelled nodes**	Each node v carries a distinct identifier that agents at v can read; hence an agent can recognise a node it has visited.

Topology

Special topologies enter for two reasons: they are the simplest graphs with a given feature, and they are networks that are used. Trees can be explored with termination by a single agent and have a central node or edge that acts as a focal point, so rendezvous was first solved there; rings are "the simplest class" of graphs with cycles (Pelc 2019) (Dessmark et al. 2006). The oriented ring is also the most symmetric instance, where identical agents cannot meet (Pelc 2019). The ring, oriented or not, was the first topology for black hole search (Dobrev et al. 2007), and the oriented ring was the first for live exploration of dynamic graphs (Di Luna et al. 2020a) and, as reported by (Molla et al. 2021), for Byzantine dispersion. Grids model discretised space: the infinite grid is a tool for rendezvous in the plane (Pelc 2019), and dispersion on grids is studied for its own sake (Kshemkalyani et al. 2020c). Hypercubes are interconnection networks on which decontamination was solved before unknown graphs (Flocchini et al. 2008c); the complete graph K_3 is the witness in Pelc's crash impossibility (Pelc 2018); bipartite graphs

Claude (Anthropic) was used in cataloguing the problems.

appear with butterfly counting by agents (Chand et al. 2025a). Level 0 is an arbitrary connected graph, unknown to the agents.

Level	Value	Formal definition
0	**any topology**	G is any connected graph in the class allowed by the other axes.
1	**ring**	G is a cycle C_n (every node of degree 2), or a path when stated.
1	**tree**	G is connected and acyclic: $\|E\| = n - 1$.
1	**grid**	G is a $\sqrt{n} \times \sqrt{n}$ grid graph; whether ports are globally consistent (oriented) is stated per result.
1	**complete graph**	$G = K_n$: every pair of nodes is adjacent.
1	**hypercube**	$G = Q_j$: nodes are the $n = 2^j$ binary strings of length j, adjacent when they differ in one bit.
1	**bipartite**	$V = V_1 \cup V_2$ with $V_1 \cap V_2 = \emptyset$ and every edge joining V_1 to V_2; trees are a special case.

Initial configuration

The starting configuration is the adversary's last choice. Rooted starts come from exploration, where the team leaves a common home: collective tree exploration places all k agents at the root (Fraigniaud et al. 2006a), black hole search starts from a safe home base (Dobrev et al. 2006), and Molla and Moses Jr. call this a rooted graph (Molla and Moses Jr. 2019). Dispersed starts come from rendezvous and gathering, where agents start at distinct nodes (Pelc 2019) (Dieudonné and Pelc 2016), and from black hole search with scattered agents (Di Luna et al. 2025b). A dispersed configuration is also the output of dispersion, so tasks computed by agents after dispersing, such as leader election, start from it (Kshemkalyani et al. 2026) (Sudo et al. 2024). Dispersion itself was posed from arbitrary configurations, robots "placed in an n node graph arbitrarily" (Augustine and Moses Jr. 2018), and rooted dispersion became a separate variant (Molla and Moses Jr. 2019). The two special starts expose different symmetries: co-located agents are indistinguishable by position, so labels or coins must decide who moves (Molla and Moses Jr. 2019); scattered agents have distinct positions that may still look alike in a symmetric graph (Pelc 2019).

Level	Value	Formal definition
0	**any start**	Agents start at arbitrary nodes; $\ell \leq k$ nodes are initially occupied, possibly with several agents each.
1	**rooted start**	All k agents start at the same node ($\ell = 1$). Also called *co-located* (black hole search: all agents at the *home base*) or a *rooted graph* (Molla and Moses Jr. 2019).
1	**dispersed start**	Every initially occupied node holds exactly one agent ($\ell = k$). Also called a *scattered* start, notably in black hole search.

The problems of this model are in Part III: Exploration, Computing with agents.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 3

Mobile robots in the plane

Anonymous point robots in the Euclidean plane repeat Look–Compute–Move cycles; what they agree on, remember, see and rely on is fixed by its assumption axes.

> What is optimised
> The primary question has been **solvability under the fewest assumptions**: which combinations of synchrony, agreement on coordinates, rigidity, multiplicity detection, visibility and memory make a task possible at all. Time (in rounds or epochs) and the number of light colours became the measures later, once the feasibility frontier was mapped.

There are n robots, modelled as points in $\mathbb{R}^2$. They are **anonymous** (no identifiers), **identical** (same algorithm) and **silent** (no messages). Each robot repeats a cycle: **Look** (take a snapshot of the positions of the robots it can see, in its own local coordinate system), **Compute** (run the algorithm on the snapshot) and **Move** (move towards the computed point). Unless assumed otherwise robots are **oblivious**: nothing survives from one cycle to the next.

A **scheduler** decides when robots are active: in FSYNC all robots act in every round; in SSYNC an arbitrary non-empty subset acts in each round, atomically; in ASYNC cycles are arbitrary and a robot may look at others while they move. Time is counted in rounds (FSYNC, SSYNC) or epochs (ASYNC): intervals of time in which every robot completes at least one full cycle, not necessarily the shortest ones.

This is the OBLOT model of Suzuki and Yamashita and of Flocchini, Prencipe and Santoro; **lights are the robots' memory**: a light with a constant number of colours, set by its robot, persists between cycles. A robot may see only its own light (FSTA: finite state), only the others' lights (FCOM: finite communication), or all of them (LUMI); oblivious robots (OBLOT) have none. The number of colours is a measure.

How the axes work. As for graphs, level 0 on every axis assumes nothing, and higher levels are stronger assumptions. A problem variant is a point on all axes; the catalogue arranges variants as a tree rooted at *no assumptions* and as a graph of strengthenings.

3.1 How the assumptions fit together

Weak robots and the least that suffices

Suzuki and Yamashita (Suzuki and Yamashita 1999) modelled a swarm of identical robots that observe each other's positions at unpredictable times and "communicate" only by their moves, with no common coordinate system. Flocchini, Prencipe and Santoro describe the field that grew from this as asking for "the minimal capabilities that the robots should have in order to solve a problem" (Flocchini et al. 2012a), and later work states the goal as identifying "the minimal assumptions on internal capabilities of the robots (e.g., persistent memory, communication) and external conditions of the system (e.g., synchrony, activation scheduler)" (Flocchini et al. 2023). The robots are deliberately weak: anonymous, uniform, disoriented, oblivious and silent, the "weakest and de facto standard model" OBLOT (Flocchini et al. 2023). Each axis of this model is one capability that can be granted on top of that baseline (memory,

agreement, synchrony, detection, sight, identity, randomness), and a result is read as a point in the product of these scales. Impossibilities such as gathering without multiplicity detection (Prencipe 2007) tell which capability is needed; algorithms tell which suffices.

The adversary acts in time, in space and in the frames

Correctness is proved against an adversary, and most axes describe what that adversary controls. The scheduler decides when robots act and, in ASYNC, how long each phase lasts, so that a robot may compute on an outdated snapshot (Prencipe 2007) (Flocchini et al. 2023). Non-rigid moves let it decide where a move ends, subject only to a minimum distance δ (Prencipe 2007) (Flocchini et al. 2016). Disorientation lets it choose each robot's coordinate system, and in some models change it at every activation (Di Luna et al. 2014) (Izumi et al. 2012). Faults let it control some robots outright (Agmon and Peleg 2006), and with randomization the question is what it may know about the coins (Efrima and Peleg 2009). Weakening one of these powers is the usual way to study an axis: SSYNC removes stale snapshots, rigid moves remove the choice of stopping point, a compass removes the choice of frame.

Symmetry is the obstacle

Anonymous, disoriented, deterministic robots in a symmetric configuration receive equivalent snapshots and move symmetrically, so symmetry can persist forever; the formability characterisations are stated in terms of the symmetry of the configuration (Suzuki and Yamashita 1999) (Dieudonné et al. 2010). Several axes are ways to break it. Agreement on chirality or on axes removes reflections and rotations (Dieudonné et al. 2010) (Flocchini et al. 2008a); labels distinguish robots (Bouzid and Lamani 2011) (Asahiro and Yamashita 2025); random bits break ties with probability 1 (Dieudonné and Petit 2009) (Bramas and Tixeuil 2016); a sequential scheduler activates one robot at a time and so elects a leader for free (Flocchini et al. 2026). For gathering, a multiplicity point is itself an asymmetry, which is why multiplicity detection lets $n \geq 3$ robots gather (Cieliebak et al. 2012) while two robots, which cannot create one, need memory, lights or compasses (Suzuki and Yamashita 1999) (Izumi et al. 2012) (Viglietta 2013).

Lights: memory and communication in one register

Persistent memory and explicit communication were both absent from the original model. The light, suggested by Peleg (Peleg 2005) and formalised by Das et al. (Das et al. 2016), gives each robot a constant number of colours that persist and are seen by all, and it removes the difference between SSYNC and ASYNC (Das et al. 2016). Showing only one's own light (FSTA) or only the others' (FCOM) separates remembering from communicating (Flocchini et al. 2016), which turned the memory axis and the scheduler axis into a single landscape of model–scheduler pairs (Flocchini et al. 2019) (Flocchini et al. 2023). Lights also serve the geometric axes: with obstructed visibility they let robots coordinate without collisions (Di Luna et al. 2014), and with two robots they replace persistent memory (Viglietta 2013) (Heriban et al. 2018).

Where the standard setting sits

The usual setting is weak on the axes that concern computability (anonymous, oblivious, disoriented, deterministic, SSYNC or ASYNC, often non-rigid) and idealised on the geometric ones: point robots, transparent, with unlimited visibility and exact sensing (Flocchini et al. 2023) (Bandettini et al. 2011). The idealisations were kept because they make the questions about synchrony, memory and agreement clean, and they were later relaxed one at a time: limited visibility (Ando et al. 1999) (Flocchini et al.

Claude (Anthropic) was used in cataloguing the problems.

2005), fat bodies (Czyzowicz et al. 2009) (Agathangelou et al. 2013), opacity (Di Luna et al. 2014), faults (Agmon and Peleg 2006). In the catalogue every axis starts at its most hostile value, so that the root of each tree is the setting in which nothing is assumed; the standard variant of a problem is then a point well above the root on the geometric axes and close to it on the others.

3.2 Notation

Symbols of this model. Inside the model each symbol has one meaning; *papers write* gives other symbols found in the literature.

Symbol	Meaning	Papers write	Used in
n	number of robots	N in some mutual visibility papers	whole model
f	maximum number of faulty robots (crashed or Byzantine)		whole model
ℓ	number of distinct labels of homonymous robots; $\ell = 1$ anonymous, $\ell = n$ distinct identifiers	h	whole model
c	number of colours of a light		whole model
V	visibility radius (limited visibility)		whole model
δ	minimum distance a robot travels in a non-rigid move before the adversary may stop it		whole model
Δ_0	diameter of the initial configuration	D_E	whole model
k	number of groups (k-grouping; in partitioning the number of fixed points, each receiving n/k robots)	(Bhagat et al. 2021) write k for the group size of partitioning, which is n/k here	k-grouping, Partitioning, Separation
L	length (circumference) of the ring or circle	ℓ	Uniform deployment on a ring
d	the target spacing L/n in uniform deployment		Uniform deployment on a ring
m	number of patterns in a sequence		Pattern sequences
λ	contraction factor of a λ-contracting gathering protocol		Gathering, Near-gathering
r, s	robots (local names)		whole model
g	hop diameter of the initial visibility graph	D_G, D	Gathering, Scattering
ϕ	expected running time function of scattering algorithm SA_ϕ	f	Scattering

3.3 Assumption axes

Each axis says where the assumption comes from and why the community studies it, then defines its values. Every axis is a scale. Level 0 assumes nothing; each higher level is a stronger assumption, which makes a problem easier. Values on the same level are alternatives that do not imply each other. An algorithm designed for weaker assumptions works under stronger ones; a lower bound proved under stronger assumptions holds under weaker ones.

Body

Suzuki and Yamashita's robots are points: each one observes "the positions of all the robots" (Suzuki and Yamashita 1999), several robots may share a point, and nobody blocks anybody's view. Czyzowicz,

Claude (Anthropic) was used in cataloguing the problems.

Gąsieniec and Pelc (Czyzowicz et al. 2009) introduced *fat* robots, closed unit discs that cannot overlap and hide what lies behind them: a robot sees another if some segment between their boundary circles meets no third robot, and a robot that touches another stops and ends its cycle. Gathering then asks for "a configuration for which the union of all discs representing them is connected", in which moreover every robot sees all others, so that all know the task is done (Czyzowicz et al. 2009). Their motivation is realism: "In reality, even very small robots occupy some space", so robots may block each other's view and motion (Czyzowicz et al. 2009); later accounts say the same (Agathangelou et al. 2013). (Czyzowicz et al. 2009) solved three and four oblivious, disoriented robots in ASYNC (non-rigid moves, robots know n) by case analysis and left $n \geq 5$ open ("We were unable to do it"); Agathangelou, Georgiou and Mavronicolas (Agathangelou et al. 2013) answered it for any n in ASYNC, assuming chirality and knowledge of n. Fatness is not weaker in every respect: the common radius gives all robots "a common measure of unit" (Czyzowicz et al. 2009), a unit of length "for free" (Agathangelou et al. 2013) (Bandettini et al. 2011). The catalogue puts fat robots at level 0 because they lose multiplicity points and gain obstruction and collisions; point robots are the setting of almost all other work (Bandettini et al. 2011).

Level	Value	Formal definition
0	**fat**	Each robot is a closed disc of radius 1 (unit disc). Discs may not overlap, so robots cannot share a point and must avoid collisions; a disc hides what lies behind it from other robots. Gathering then asks for a configuration whose union of discs is connected and in which every robot sees all others (Czyzowicz et al. 2009).
1	**point**	Each robot is a point of the plane: several robots may share a point (multiplicity), and a robot hides only what lies exactly behind it on a line of sight.

Scheduler

Suzuki and Yamashita (Suzuki and Yamashita 1999) introduced the semi-synchronous model: robots act "at infinitely many unpredictable time instants", and in each round of a discrete time an arbitrary non-empty set of robots observes and moves instantaneously, every robot being active infinitely often; Prencipe calls it *Atom* (Prencipe 2007) and FSYNC, where every robot acts in every round, is its special case (Das et al. 2016). The asynchronous model (later named CORDA) is due to Flocchini, Prencipe, Santoro and Widmayer (Flocchini et al. 1999), as (Flocchini et al. 2023) and (Prencipe 2007) also state: they assume "asynchronicity within one step", so that "the actual move of a robot may be based on a situation that lies arbitrarily far in the past", and a robot "may be seen an arbitrary number of times by other robots" while moving; they call this "the most radical deviation" from Suzuki and Yamashita's atomic steps (Flocchini et al. 1999). In Prencipe's words, "computations can be made based on obsolete observations" (Prencipe 2007). Prencipe motivates delays inside a cycle as a way "to model also different motorial speeds of the robots" (Prencipe 2007). The schedulers are interesting because they form a hierarchy: rendezvous of two oblivious robots is solvable in FSYNC and not in SSYNC (Suzuki and Yamashita 1999) (Theorem 3.1 and footnote 5), while a problem separating SSYNC from ASYNC for oblivious robots was sought "for more than two decades" and is known only under limited visibility (Flocchini et al. 2023) (Kirkpatrick et al. 2024). Intermediate schedulers (RSYNC, LC-, CM- and Move-atomic ASYNC) were introduced to locate exactly where the power is lost (Flocchini et al. 2023) (Buchin et al. 2022). Sequential schedulers, which activate one robot per round, break symmetry by themselves; Flocchini, Navarra, Pattanayak, Piselli and Santoro note that their relationship with FSYNC "has so far been ignored" and show that the two are orthogonal (Flocchini et al. 2026).

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**ASYNC**	**ASYNC**: each robot's Look, Compute and Move phases take finite but unbounded, adversarially chosen time; a robot may look while others move, and compute on an outdated snapshot. Time is counted in epochs: an **epoch** is an interval of time in which every robot completes at least one full Look–Compute–Move cycle; it need not be the shortest such interval. The execution is divided into consecutive epochs, and a bound of X epochs holds for every such division. In FSYNC and SSYNC time is counted in rounds. Called *Corda* by Prencipe (Prencipe 2007). Intermediate variants restrict interleavings: *LC-atomic* (no Look by another robot between a robot's Look and its Compute), *Move-atomic* (no Look during another robot's Move) and *CM-atomic* (Flocchini et al. 2023).
1	**SSYNC**	**SSYNC**: time is divided into rounds; in each round a non-empty subset of robots, chosen by a fair adversary, executes one atomic Look–Compute–Move cycle. Called *Atom* by Prencipe (Prencipe 2007).
2	**sequential**	**Sequential**: an SSYNC scheduler that activates exactly one robot per round (fairly). Also called a *centralized* scheduler (Terai et al. 2023); a *round-robin* scheduler is a fair special case.
2	**FSYNC**	**FSYNC**: every robot executes one atomic cycle in every round.

Lights (persistent memory)

Suzuki and Yamashita model a robot as "a mobile processor with infinite memory" and call an algorithm *oblivious* when the new position depends only on the current observation, noting that "oblivious algorithms are self-stabilizing by definition" (Suzuki and Yamashita 1999). Obliviousness then became the default, partly because "any algorithm proved correct in this model is also self-stabilizing" (Clement et al. 2010) (Défago et al. 2006); oblivious, silent robots communicate only by moving, "reminiscent of bees in a bee dance" (Flocchini et al. 1999) (Prencipe 2007). Unbounded memory was compared early: two non-oblivious robots can rendezvous in SSYNC and oblivious ones cannot (Suzuki and Yamashita 1999) (Theorems 3.1 and 3.3) (Izumi et al. 2012). Lights were "initially suggested by Peleg" (Das et al. 2016) (Di Luna et al. 2014): his survey proposes giving robots "a small amount (say, O(1) bits) of stable memory" and "marking (at least some of) the robots with colors, flags or visible indicator lights" (Peleg 2005) (see also Efrima and Peleg 2007); Das, Flocchini, Prencipe, Santoro and Yamashita (Das et al. 2016) turned them into the LUMI model, a constant-size register that is persistent and visible to all, so that a robot can remember and communicate a constant number of bits per cycle. Their main results are that LUMI under ASYNC is equivalent to LUMI under SSYNC and strictly stronger than OBLOT under SSYNC (Das et al. 2016). To separate the two roles of a light, Flocchini, Santoro, Viglietta and Yamashita (Flocchini et al. 2016) defined FSTA (a robot sees only its own light: constant memory, no communication) and FCOM (it sees only the others' lights: constant communication, no memory), and showed that FSTA robots with rigid moves rendezvous in SSYNC and FCOM robots even in ASYNC. The question "is it better to remember or to communicate" then became a programme comparing the four models under every scheduler (Flocchini et al. 2019) (Flocchini et al. 2023) (Das et al. 2024).

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**oblivious**	**OBLOT**: a robot's computation depends only on the current snapshot; nothing persists across cycles.
1	**internal light (FSTA)**	**FSTA**: each robot has a light with c colours that only it can see; it persists between cycles (constant persistent memory, no communication). Also called an *internal* light.
1	**external light (FCOM)**	**FCOM**: each robot has a light with c colours that only the others can see (constant communication, no persistent memory of its own). Also called an *external* light.
2	**lights (LUMI)**	**LUMI**: each robot has a light with c colours, set at the end of Compute, visible to all robots including itself; it is both memory and communication. Also called *luminous* robots, or a *full* light.

Agreement on coordinates

Suzuki and Yamashita's robots "initially do not have a common x-y coordinate system", although they share handedness: in every local frame "the positive y-direction is 90 degrees counterclockwise from the positive x-direction" (Suzuki and Yamashita 1999); together with anonymity, this disorientation is what keeps symmetric configurations symmetric. Flocchini, Prencipe, Santoro and Widmayer (Flocchini et al. 1999) (Flocchini et al. 2008a) dropped that common handedness and made agreement a parameter, studying "the role of common knowledge" in pattern formation by asynchronous robots: with the directions and orientations of both axes any pattern can be formed; with one axis direction (with or without its orientation) the problem is solvable when n is odd and in general not when n is even; with no axis it is unsolvable in general (ISAAC version, proofs omitted) (Flocchini et al. 1999). Efrima and Peleg (Efrima and Peleg 2009), "elaborating on" that work, list full compass, half compass, direction-only, axes-only and no compass, and note that without a common orientation robots "do not share the notion of 'clockwise'". Common handedness, *chirality*, became the usual intermediate assumption: Dieudonné, Petit and Villain list sense of direction and chirality among the global properties that decide what a swarm can do, and show that leader election and arbitrary pattern formation are equivalent in ASYNC for $n \geq 4$ with chirality and $n \geq 5$ without (Dieudonné et al. 2010); removing chirality from pattern formation and gathering is a research line of its own (Bramas and Tixeuil 2015) (Cicerone et al. 2019). Partial agreement has also been quantified: Izumi et al. give each robot an "unreliable compass" and find the largest deviation that still lets two oblivious robots gather (Izumi et al. 2012), and agreement on one axis is used for gathering under limited visibility (Poudel and Sharma 2021) (Castenow et al. 2022).

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**disoriented**	Each robot r observes in its own coordinate system, obtained from the global one by an unknown similarity (translation, rotation, uniform scaling, and possibly reflection); systems may differ between robots and, in some models, between cycles. Papers that call robots *disoriented* do not always mean this: Défago and Souissi use the word while assuming chirality (Défago and Souissi 2008). Some papers also let the adversary change a robot's rotation, reflection and unit at every Look (*variable disorientation*) (Flocchini et al. 2023) (Masuzawa et al. 2026).
1	**chirality**	All local coordinate systems have the same handedness (clockwise orientation): no reflection. Also called *handedness*. Prencipe uses *chirality* for agreement on North and East, i.e. a compass (Prencipe 2007).
1	**one axis**	All robots agree on the direction and orientation of one axis (e.g. North), but not necessarily on handedness: two robots may disagree on which side is East. This is how the papers use *one axis agreement* (e.g. robots "agree on only 'left' and 'right'" Bose et al. 2021; robots "do not necessarily have the same handedness" Pagli et al. 2015). One axis and chirality are alternatives on this axis; together they amount to a common compass.
2	**compass**	All robots agree on the directions and orientations of both axes (a common compass); they may still disagree on origin and unit. Called *full compass* by Efrima and Peleg, who also use *half compass* (both axis directions, the positive sense of one axis, no chirality), *direction-only* and *axes-only* (Efrima and Peleg 2009). Agreement on one axis together with common chirality is equivalent: the orientation of the second axis then follows from handedness.

Movement

In CORDA a move may stop before its destination: Prencipe's Assumption A1 gives each robot a constant $\delta_r > 0$ such that it either reaches its destination or travels at least δ_r towards it, so that "it cannot know how far it will go in the current cycle" (Prencipe 2007). The physical motivation is "limits to the robot's motorial capabilities" (Prencipe 2007); the lower bound δ is needed because without it no algorithm could be guaranteed to terminate (Bandettini et al. 2011). Non-rigid moves let the adversary act in space as the scheduler acts in time, and *rigid* moves (every move reaches its destination) are the stronger assumption that many results state explicitly. Flocchini, Santoro, Viglietta and Yamashita (Flocchini et al. 2016) first proved their rendezvous algorithms for rigid moves and then showed that "rigidity of the movements can be traded with knowledge of δ". Heriban, Défago and Tixeuil (Heriban et al. 2018) work explicitly in "the non-rigid ASYNC model", and later landscape papers removed rigidity (and chirality) from the earlier synchronous comparisons of OBLOT, FSTA, FCOM and LUMI, as reported by (Das et al. 2024).

Level	Value	Formal definition
0	**non-rigid**	In a Move phase the adversary may stop a robot before its destination, but only after it has travelled at least $\delta > 0$ (a constant unknown to the robots) or reached the destination.
1	**rigid**	Every Move phase ends at the computed destination.

Claude (Anthropic) was used in cataloguing the problems.

Multiplicity detection

With point robots several robots can share a point, and the snapshot decides whether this is visible. In Prencipe's statement of CORDA robots "can not distinguish whether there is more than one fellow on a given position" (Prencipe 2007). Prencipe observed that every known gathering algorithm used some "extra" assumption, mainly that robots "detect whether a given point on the plane is occupied by one or more robots", and proved gathering impossible without such help (Prencipe 2007). Cieliebak, Flocchini, Prencipe and Santoro (Cieliebak et al. 2012) then showed that with multiplicity detection any $n > 2$ oblivious, disoriented, asynchronous robots gather (abstract read); the idea is to create one multiplicity point and keep it unique (Bandettini et al. 2011). Later work distinguishes a *weak* detector (one robot or more) from a *strong* one (the exact number) (Bouzid et al. 2013) (Pattanayak et al. 2019b); the earliest source of this split was not traced. The level matters: Terai, Wada and Katayama note that strong detection alone lets robots without lights gather, so a model with lights must keep colours from revealing multiplicities (Terai et al. 2023); under sequential schedulers weak detection is necessary and sufficient for gathering (Flocchini et al. 2026).

Level	Value	Formal definition
0	**no multiplicity detection**	A snapshot is a set of points: a point occupied by several robots looks like a single robot.
1	**weak multiplicity**	A snapshot marks each occupied point as holding one robot or more than one.
2	**strong multiplicity**	A snapshot is a multiset: each occupied point comes with the exact number of robots on it.

Visibility

Ando, Oasa, Suzuki and Yamashita (Ando et al. 1999) introduced robots that see only within a fixed radius V; Kirkpatrick et al. call it "the pioneering work on Point Convergence under limited visibility", (Kirkpatrick et al. 2024). Each robot moves towards the centre of the smallest enclosing circle of the robots it sees, but only so far that every robot it sees stays within distance V; the proof assumes points, no obstruction, and phases of negligible duration (so the scheduler is SSYNC although the paper calls the robots asynchronous), with each move reaching its computed destination, at most a small constant away (Ando et al. 1999). Flocchini, Prencipe, Santoro and Widmayer (Flocchini et al. 2005) gathered oblivious robots with limited visibility in ASYNC, with non-rigid moves and without multiplicity detection, when they share "orientation", i.e. agree on the directions and orientations of both axes but not on unit or origin; they conclude that "orientation is at least as powerful as instantaneous movements". Limited visibility makes the model local: the initial visibility graph must be connected (Flocchini et al. 2005) (Lemma 4.7), and known algorithms keep every initially visible pair visible (Ando et al. 1999) (Bandettini et al. 2011) (Kirkpatrick et al. 2024). It also became a tool for separation: Katreniak introduced bounded asynchrony for this problem (Katreniak 2011), and convergence with limited visibility is the known problem that separates SSYNC from ASYNC for oblivious robots (Kirkpatrick et al. 2024) (Flocchini et al. 2023). Das et al. argue that full visibility "is not practically feasible" and compare OBLOT, FSTA, FCOM and LUMI under both visibilities (Das et al. 2024).

Level	Value	Formal definition
0	**limited visibility**	A robot sees exactly the robots within Euclidean distance V of its position, for a fixed radius $V > 0$.
1	**unlimited visibility**	A robot sees all robots (subject to opacity).

Opacity

In Suzuki and Yamashita's model and in CORDA robots are transparent: "three or more collinear agents

Claude (Anthropic) was used in cataloguing the problems.

are assumed to be mutually visible" (Di Luna et al. 2014). Di Luna, Flocchini, Gan Chaudhuri, Santoro and Viglietta (Di Luna et al. 2014) (Di Luna et al. 2017) started "the study of computing when visibility is obstructable" for point robots, since "given three collinear agents, the one in the middle blocks the visibility between the other two". Before them obstruction had been studied for fat robots (Czyzowicz et al. 2009) (Agathangelou et al. 2013) and for robots on a line, according to (Di Luna et al. 2014). The basic problem they posed, Complete (Mutual) Visibility, asks the robots to reach a configuration in which all see each other; they solved it in ASYNC without collisions with a constant number of colours (Di Luna et al. 2014). Opaque robots with lights then became a line of work on time and colours (Vaidyanathan et al. 2015) (Sharma et al. 2021) and on pattern formation (Bose et al. 2021), and Feletti, Mambretti, Mereghetti and Palano compare the computational power of transparent and opaque robots directly (Feletti et al. 2025).

Level	Value	Formal definition
0	**opaque**	Robots are points that obstruct sight: r sees s iff no third robot lies on the open segment $\overline{rs}$.
1	**transparent**	Robots do not obstruct each other's view.

Faults

Agmon and Peleg (Agmon and Peleg 2006) (SODA 2004) brought crash and Byzantine robots into the Look–Compute–Move model, requiring the algorithm "to successfully gather the nonfaulty robots, independently of the behavior of the faulty ones". They observed that most existing algorithms fail with a single crash, gathered $n \geq 3$ robots with one crash in SSYNC (Theorem 3.8), proved that three robots cannot gather, or even converge, in SSYNC if one may be Byzantine (Theorem 4.4; hence also in ASYNC, which their abstract states), and gathered $n \geq 3f + 1$ robots with f Byzantine faults in FSYNC (Theorem 5.10) (Agmon and Peleg 2006). Their robots share handedness, see multiplicities (a Look returns a multiset) and make non-rigid moves of at least a fixed distance (Agmon and Peleg 2006). The motivation is deployment in adverse environments, where a swarm must "ensure coordination in spite of faulty robots" (Clement et al. 2010), and oblivious algorithms are already self-stabilizing, so transient faults come for free and permanent ones are the question (Défago et al. 2006). Défago, Gradinariu, Messika and Raipin-Parvédy (Défago et al. 2006) extended the feasibility study to bounded schedulers and to probabilistic gathering under crash and Byzantine faults. A crashed robot "simply stops moving" and, since correct robots may also stay still, cannot be identified (Efrima and Peleg 2009); crash-tolerant gathering of many faults was obtained with strong multiplicity detection (Bouzid et al. 2013) and with weak multiplicity detection (Pattanayak et al. 2019b).

Level	Value	Formal definition
0	**Byzantine**	Up to f robots move arbitrarily (adversarially); with lights, they may also set arbitrary colours.
1	**crash faults**	Up to f robots stop forever at adversarial times and remain visible at their last position.
2	**no faults**	All robots follow the algorithm forever.

Randomness

The classical model is deterministic: in CORDA the Compute phase runs "a deterministic algorithm" (Prencipe 2007), so a configuration that looks the same to two anonymous, disoriented robots makes them act alike forever. Randomization was introduced to escape such impossibilities: Dieudonné and Petit (Dieudonné and Petit 2009) proved that scattering cannot be solved deterministically in SSYNC and gave a randomized algorithm, and Défago, Gradinariu, Messika and Raipin-Parvédy (Défago et al. 2006) studied probabilistic gathering with and without faults (abstract read). Clément et al. note that work around the impossibilities had focused on additional assumptions, "less attention being shown to the use

Claude (Anthropic) was used in cataloguing the problems.

of randomization", and analyse probabilistic gathering and scattering with Markov chains (Clement et al. 2010). Yamauchi and Yamashita (Yamauchi and Yamashita 2014) formed any pattern in ASYNC without a common coordinate system by randomization, as reported by (Bramas and Tixeuil 2016); Bramas and Tixeuil then treated randomness as a resource, using one random bit per cycle for pattern formation (Bramas and Tixeuil 2016) and proving that $n \log n$ random bits are necessary to scatter n robots (Bramas and Tixeuil 2017). What the adversary may learn about the coin flips (adaptive or not) is part of the model and is fixed per paper (Efrima and Peleg 2009).

Level	Value	Formal definition
0	**deterministic**	The destination computed in a cycle is a function of the snapshot (and the light, if any).
1	**randomized**	Robots may use random bits in Compute; guarantees hold with probability 1 or in expectation, as stated per result.

Identifiers

Suzuki and Yamashita's robots are anonymous: "they all execute the same algorithm and they cannot be distinguished by their appearances" (Suzuki and Yamashita 1999). With disorientation and determinism, anonymity makes symmetry the central obstacle, and the characterisations of solvable instances are stated in terms of the symmetry of the configuration (Suzuki and Yamashita 1999) (Dieudonné et al. 2010). Das, Flocchini, Santoro and Yamashita (Das et al. 2015) compared anonymous and identified (*eponymous*) robots on forming sequences of patterns and found that anonymous robots form only sequences of equal symmetricity while identified robots form nearly all, as summarised by (Bouzid and Lamani 2011). To measure the gap, Bouzid and Lamani (Bouzid and Lamani 2011) imported the homonymous model of Delporte-Gallet et al. (Delporte-Gallet et al. 2013), in which n robots share ℓ labels, into the Suzuki–Yamashita model. Asahiro and Yamashita (Asahiro and Yamashita 2025) recast labels as algorithms: robots running one of ℓ algorithms behave like robots with ℓ shared labels, so the *minimum algorithm size* of a task measures how much identification it needs.

Level	Value	Formal definition
0	**anonymous**	Robots are indistinguishable and run the same algorithm.
1	**homonymous**	Robots carry labels from a set of size ℓ, $1 \le \ell \le n$, which may repeat; equivalently, robots may run one of ℓ algorithms. The minimum ℓ that solves a problem is its *minimum algorithm size* (Asahiro and Yamashita 2025).
2	**distinct IDs (eponymous)**	**Distinct identifiers** (eponymous robots): labels are pairwise distinct ($\ell = n$); whether others can see a robot's label is stated per result.

Which model is stronger

Robot models combine a memory model (OBLOT, FSTA, FCOM, LUMI) with a scheduler. Model X is **stronger** than Y ($X > Y$) if every problem solvable in Y is solvable in X and some problem, the *separator*, is solvable in X but not in Y; $X \equiv Y$ if they solve the same problems; $X \perp Y$ if each solves a problem the other cannot. The schedulers between FSYNC and ASYNC are RSYNC (after an optional fully synchronous prefix, consecutive sets of active robots are disjoint), SSYNC, and three restrictions of ASYNC: LC-atomic (no robot looks between another's Look and Compute), CM-atomic and Move-atomic (no robot looks while another moves) (Flocchini et al. 2023) (Buchin et al. 2022).

A scheduler that allows fewer executions gives at least as much power: FSYNC $\ge$ SSYNC, RSYNC $\ge$ SSYNC, and SSYNC $\ge$ each atomic ASYNC $\ge$ ASYNC. More visible light gives at least as much power

Claude (Anthropic) was used in cataloguing the problems.

under the same scheduler: LUMI $\geq$ FCOM, FSTA $\geq$ OBLOT.

The website shows the same relations as an interactive grid: choosing a model colours every other model as stronger, weaker, equivalent, incomparable or unknown, including relations that follow by transitivity.

Any number of robots

Most results assume chirality and rigid moves; separators use small teams (often $n \leq 4$). Sources: (Das et al. 2016) (Flocchini et al. 2019) (Buchin et al. 2022) (Flocchini et al. 2023) (Das et al. 2021a) (Naito et al. 2025) (Flocchini et al. 2026) (Kirkpatrick et al. 2024).

Model		Model	Separator	Source
OBLOT^{FSYNC}	$>$	OBLOT^{SSYNC}	RDV	Suzuki and Yamashita (1999) Theorem 3.1 (no oblivious algorithm for two robots) and footnote 5 (midpoint algorithm for synchronized robots); see also (Flocchini et al. 2019) Thm 20.4 and (Das et al. 2016) Lemma 3.3; also Flocchini et al. (2019) Suzuki and Yamashita's robots share handedness and each move reaches its computed destination (bounded step length); the impossibility holds a fortiori without these.
LUMI^{ASYNC}	$>$	OBLOT^{SSYNC}	RDV	Das et al. (2016) Thm 3.4 (via Lemma 3.3, Thm 3.3)
LUMI^{ASYNC}	$\equiv$	LUMI^{SSYNC}	*simulation*	Das et al. (2016) Thm 3.6 (via Thm 3.5); also Flocchini et al. (2023)
FCOM^{FSYNC}	$\equiv$	LUMI^{FSYNC}	*simulation*	Flocchini et al. (2019) Thm 2 (Lemma 1, simulation LUbyFCinFSY)
FCOM^{FSYNC}	$>$	FSTA^{FSYNC}	¬IL	Flocchini et al. (2019) Thm 6 (Lemmas 4, 5); also Buchin et al. (2022)
FSTA^{FSYNC}	$>$	OBLOT^{FSYNC}	OSP	Flocchini et al. (2019) Thm 10 (Lemmas 8, 9)
FCOM^{SSYNC}	$\perp$	FSTA^{SSYNC}	¬IL, TAR(d)	Flocchini et al. (2019) Thm 14
FSTA^{SSYNC}	$>$	OBLOT^{SSYNC}	RDV	Flocchini et al. (2019) Thm 15 Uses rendezvous by FSTA robots in SSYNC (Flocchini et al. 2016), which needs each robot's unit of distance to stay fixed.
FCOM^{SSYNC}	$>$	OBLOT^{SSYNC}	RDV	Flocchini et al. (2019) Thm 15
LUMI^{SSYNC}	$>$	FSTA^{SSYNC}	OSP	Flocchini et al. (2019) Thm 17 (Lemma 16)
LUMI^{SSYNC}	$>$	FCOM^{SSYNC}	OSP	Flocchini et al. (2019) Thm 17 (Lemma 16)
LUMI^{FSYNC}	$>$	LUMI^{SSYNC}	CGE	Flocchini et al. (2019) Thm 20.1 (Lemma 19)
FSTA^{FSYNC}	$>$	FSTA^{SSYNC}	CGE	Flocchini et al. (2019) Thm 20.2
FCOM^{FSYNC}	$>$	LUMI^{SSYNC}	CGE	Flocchini et al. (2019) Thm 20.3
OBLOT^{FSYNC}	$\perp$	FCOM^{SSYNC}	SRO, ¬IL	Flocchini et al. (2019) Thm 25 (Lemmas 22, 23)
OBLOT^{FSYNC}	$\perp$	FSTA^{SSYNC}	SRO, TAR(d)	Flocchini et al. (2019) Thm 25
LUMI^{SSYNC}	$\perp$	FSTA^{FSYNC}	¬IL, CGE	Flocchini et al. (2019) Thm 26
FCOM^{SSYNC}	$\perp$	FSTA^{FSYNC}	¬IL, CGE	Flocchini et al. (2019) Thm 26
LUMI^{SSYNC}	$\perp$	OBLOT^{FSYNC}	OSP, CGE*	Flocchini et al. (2019) Thm 28 (answers the Das et al. 2016 question for LUMI^A since LUMI^A ≡ LUMI^S)
FSTA^{FSYNC}	$>$	OBLOT^{SSYNC}	CGE, RDV	Flocchini et al. (2019) Table 3 (Th. 15, 20)
OBLOT^{RSYNC}	$\perp$	FCOM^{SSYNC}	SRO, RDV	Buchin et al. (2022) Thm 3.1
OBLOT^{RSYNC}	$\perp$	FSTA^{SSYNC}	SRO, RDV	Buchin et al. (2022) Thm 3.2
FCOM^{RSYNC}	$>$	FCOM^{SSYNC}	SRO	Buchin et al. (2022) Thm 3.3
FSTA^{RSYNC}	$>$	FSTA^{SSYNC}	SRO	Buchin et al. (2022) Thm 3.4
OBLOT^{RSYNC}	$>$	OBLOT^{SSYNC}	SRO	Buchin et al. (2022) Thm 3.5
LUMI^{SSYNC}	$>$	FSTA^{RSYNC}	CYC	Buchin et al. (2022) Thm 4.1
FSTA^{RSYNC}	$\perp$	FCOM^{SSYNC}	SRO, CYC	Buchin et al. (2022) Thm 4.2
LUMI^{SSYNC}	$\equiv$	LUMI^{RSYNC}	*simulation*	Buchin et al. (2022) Thm 6 (Thm 5, simulation sim-RS-by-S)
FSTA^{FSYNC}	$\perp$	LUMI^{RSYNC}	CYC, CGE	Buchin et al. (2022) Thm 8.1
FSTA^{FSYNC}	$\perp$	FCOM^{RSYNC}	CYC, CGE	Buchin et al. (2022) Thm 8.2

Claude (Anthropic) was used in cataloguing the problems.

Model		Model	Separator	Source
OBLOT^{FSYNC}	$\perp$	LUMI^{RSYNC}	OSP, CGE*	Buchin et al. (2022) Thm 9.1
OBLOT^{FSYNC}	$\perp$	FCOM^{RSYNC}	OSP, CGE*	Buchin et al. (2022) Thm 9.2
OBLOT^{FSYNC}	$\perp$	FSTA^{RSYNC}	OSP, CGE*	Buchin et al. (2022) Thm 9.3; also Naito et al. (2025)
FCOM^{RSYNC}	$\equiv$	LUMI^{RSYNC}	*simulation*	Buchin et al. (2022) Thm 11 (Thm 10: LUMI^S ⊠ FCOM^RS)
FCOM^{RSYNC}	$>$	FSTA^{RSYNC}	CYC	Buchin et al. (2022) Thm 11
FSTA^{RSYNC}	$>$	OBLOT^{RSYNC}	RDV	Buchin et al. (2022) Thm 11 (Lemma 1: RDV $\notin$ OBLOT^RS)
OBLOT^{SSYNC}	$>$	OBLOT^{ASYNC}	MLCv	Flocchini et al. (2023) LIPIcs Thm 6 (arXiv Thm 1); also Kirkpatrick et al. (2024)
OBLOT$^{\text{Move-atomic}}$	$>$	OBLOT^{ASYNC}	TF	Flocchini et al. (2023) LIPIcs Thm 10 (arXiv Thm 2)
OBLOT^{SSYNC}	$>$	OBLOT$^{\text{Move-atomic}}$	MLCv	Flocchini et al. (2023) LIPIcs Thm 11 (arXiv Thm 3)
OBLOT$^{\text{LC-atomic}}$	$\equiv$	OBLOT^{ASYNC}	*simulation*	Flocchini et al. (2023) LIPIcs Thm 11 (arXiv Thm 3)
FCOM^{SSYNC}	$>$	FCOM^{ASYNC}	MLCv	Flocchini et al. (2023) LIPIcs Thm 14 (arXiv Thm 4); also Naito et al. (2025)
FCOM$^{\text{CM-atomic}}$	$\equiv$	FCOM^{ASYNC}	*simulation*	Flocchini et al. (2023) LIPIcs Thm 15 (arXiv Thm 5), simulation SIM
FCOM^{SSYNC}	$\equiv$	FCOM$^{\text{LC-atomic}}$	*simulation*	Flocchini et al. (2023) LIPIcs Thm 17 (arXiv Thm 6)
FCOM$^{\text{LC-atomic}}$	$>$	FCOM$^{\text{CM-atomic}}$	MLCv	Flocchini et al. (2023) LIPIcs Thm 18 (arXiv Thm 7)
FSTA^{SSYNC}	$>$	FSTA^{ASYNC}	MLCv	Flocchini et al. (2023) LIPIcs Thm 21 (arXiv Thm 8)
FSTA$^{\text{Move-atomic}}$	$>$	FSTA^{ASYNC}	TF	Flocchini et al. (2023) LIPIcs Thm 23 (arXiv Thm 9)
FSTA^{SSYNC}	$>$	FSTA$^{\text{Move-atomic}}$	MLCv	Flocchini et al. (2023) LIPIcs Thm 24.2 (arXiv Thm 10.2); also Naito et al. (2025)
FSTA$^{\text{LC-atomic}}$	$\equiv$	FSTA^{ASYNC}	*simulation*	Flocchini et al. (2023) LIPIcs Thm 24.1
FCOM$^{\text{LC-atomic}}$	$\perp$	FSTA$^{\text{Move-atomic}}$	CYC, GCNCL	Flocchini et al. (2023) LIPIcs Thm 28.1 (arXiv Thm 11.1)
FCOM$^{\text{LC-atomic}}$	$\perp$	FSTA^{ASYNC}	CYC, GCNCL	Flocchini et al. (2023) LIPIcs Thm 28.2
FCOM$^{\text{LC-atomic}}$	$>$	OBLOT^{SSYNC}	RDV	Flocchini et al. (2023) LIPIcs Thm 28.3
FCOM^{ASYNC}	$\perp$	FSTA$^{\text{Move-atomic}}$	CYC, GCNCL	Flocchini et al. (2023) LIPIcs Thm 29.1 (arXiv Thm 12.1)
FCOM^{ASYNC}	$\perp$	FSTA^{ASYNC}	CYC, GCNCL	Flocchini et al. (2023) LIPIcs Thm 29.2; also Das et al. (2021a)
FCOM^{ASYNC}	$\perp$	OBLOT^{SSYNC}	MLCv, CYC	Flocchini et al. (2023) LIPIcs Thm 29.3
FCOM^{ASYNC}	$>$	OBLOT$^{\text{Move-atomic}}$	RDV	Flocchini et al. (2023) LIPIcs Thm 29.4 Uses rendezvous by FCOM robots in ASYNC with 12 colours (Flocchini et al. 2016), which needs a fixed unit of distance; under variable disorientation it is impossible (Masuzawa et al. 2026).
FSTA$^{\text{Move-atomic}}$	$\perp$	OBLOT^{SSYNC}	RDV, MLCv	Flocchini et al. (2023) LIPIcs Thm 30.1 (arXiv Thm 13.1) Uses rendezvous by FSTA robots in ASYNC, stated without proof; under variable disorientation it is impossible (Masuzawa et al. 2026).
FSTA$^{\text{Move-atomic}}$	$>$	OBLOT$^{\text{Move-atomic}}$	RDV	Flocchini et al. (2023) LIPIcs Thm 30.2; also Naito et al. (2025) Uses rendezvous by FSTA robots in ASYNC, stated without proof; under variable disorientation it is impossible (Masuzawa et al. 2026).
FSTA^{ASYNC}	$\perp$	OBLOT$^{\text{Move-atomic}}$	RDV, TF	Flocchini et al. (2023) LIPIcs Thm 30.3 Uses rendezvous by FSTA robots in ASYNC, stated without proof; under variable disorientation it is impossible (Masuzawa et al. 2026).
FSTA^{ASYNC}	$\perp$	OBLOT^{SSYNC}	RDV, MLCv	Flocchini et al. (2023) LIPIcs Thm 30.4 Uses rendezvous by FSTA robots in ASYNC, stated without proof; under variable disorientation it is impossible (Masuzawa et al. 2026).
FSTA^{ASYNC}	$>$	OBLOT^{ASYNC}	RDV	Flocchini et al. (2023) LIPIcs Thm 30.5; also Das et al. (2021a), Naito et al. (2025) Uses rendezvous by FSTA robots in ASYNC, stated without proof; under variable disorientation it is impossible (Masuzawa et al. 2026).
OBLOT^{FSYNC}	$\perp$	FSTA^{ASYNC}	SRO, OC	Das et al. (2021a) Thm 1 (Lemmas 1, 2)
FCOM^{ASYNC}	$\perp$	FSTA^{FSYNC}	¬IL, CGE	Das et al. (2021a) Thm 3 (Lemma 3: ¬IL $\in$ FCOM^A)
FCOM^{ASYNC}	$\perp$	FSTA^{SSYNC}	¬IL, TAR(d)	Das et al. (2021a) Thm 4

Claude (Anthropic) was used in cataloguing the problems.

Model		Model	Separator	Source
FCOM^{ASYNC}	$\perp$	OBLOT^{FSYNC}	¬IL, SRO	Das et al. (2021a) Thm 5
FCOM^{SSYNC}	$\perp$	FSTA^{ASYNC}	¬IL, IOP	Das et al. (2021a) Thm 6 (Lemmas 4, 5)
LUMI^{FSYNC}	$>$	FSTA^{FSYNC}	ETE	Naito et al. (2025) Thm 3 (Lemmas 3–5)
LUMI^{FSYNC}	$>$	LUMI^{RSYNC}	ETE	Naito et al. (2025) Thm 3
FSTA^{FSYNC}	$>$	FSTA^{RSYNC}	HET	Naito et al. (2025) Thm 4 (Lemmas 6–10)
LUMI^{RSYNC}	$>$	FSTA^{RSYNC}	HET, TAR(d)*	Naito et al. (2025) Thm 4, Thm 5
FSTA$^{\text{Move-atomic}}$	$\perp$	FCOM^{SSYNC}	VEC, CYC	Naito et al. (2025) Thm 7 (+ CYC from Buchin et al. 2022)
FCOM$^{\text{Move-atomic}}$	$\perp$	FSTA^{SSYNC}	LP–Cv, ZCC	Naito et al. (2025) Thm 8, Thm 9
OBLOT^{FSYNC}	$>$	OBLOT^{RSYNC}	LCM–M	Naito et al. (2025) Thm 11
FSTA^{ASYNC}	$\perp$	OBLOT^{RSYNC}	LCM–M, SRO	Naito et al. (2025) Thm 11 (+ SRO from Flocchini et al. 2019
FCOM^{SSYNC}	$>$	FCOM$^{\text{Move-atomic}}$	RDV	Naito et al. (2025) Table 1 (compiled 'known' relations
OBLOT^{SEQ}	$\perp$	OBLOT^{FSYNC}	UPF*, Gathering (no multiplicity detection)	Flocchini et al. (2026) Thms 1, 2, 3, 4
OBLOT^{SEQ}	$>$	OBLOT^{SSYNC}	UPF*	Flocchini et al. (2026) Sect. 8.1 (Thm 1 + Thms 2, 3)
FCOM^{FSYNC}	$>$	FCOM^{RSYNC}	CGE	Buchin et al. (2022) Thm 7

Two robots

Two robots, variable disorientation, chirality and rigid moves; the 14 models collapse to 13 classes (Kitamura et al. 2025), a preprint.

Model		Model	Separator	Source
LUMI^{FSYNC}	$\equiv$	FCOM^{FSYNC}	*simulation*	Kitamura et al. (2025) Thm 2
LUMI^{FSYNC}	$\equiv$	FSTA^{FSYNC}	*simulation*	Kitamura et al. (2025) Thm 2
FSTA^{FSYNC}	$>$	OBLOT^{FSYNC}	OSP	Kitamura et al. (2025) as for general n (Flocchini et al. 2019)
LUMI^{FSYNC}	$>$	LUMI^{RSYNC}	CGE*	Kitamura et al. (2025) Thm 6
FCOM^{FSYNC}	$>$	FCOM^{RSYNC}	CGE*	Kitamura et al. (2025) Thm 6
FSTA^{FSYNC}	$>$	FSTA^{RSYNC}	CGE*	Kitamura et al. (2025) Thm 6
LUMI^{RSYNC}	$\equiv$	LUMI^{SSYNC}	*simulation*	Kitamura et al. (2025) Thm 1
LUMI^{SSYNC}	$\equiv$	LUMI^{ASYNC}	*simulation*	Kitamura et al. (2025) Thm 1
LUMI^{RSYNC}	$\equiv$	FCOM^{RSYNC}	*simulation*	Kitamura et al. (2025) Thm 3
FCOM^{RSYNC}	$>$	FCOM^{SSYNC}	SRO	Kitamura et al. (2025) Thm 7
FSTA^{RSYNC}	$>$	FSTA^{SSYNC}	SRO	Kitamura et al. (2025) Thm 7
OBLOT^{RSYNC}	$>$	OBLOT^{SSYNC}	SRO	Kitamura et al. (2025) Thm 7
FCOM^{SSYNC}	$\equiv$	FCOM$^{\text{LC-atomic}}$	*simulation*	Kitamura et al. (2025) Thm 5
FCOM^{SSYNC}	$>$	FCOM$^{\text{CM-atomic}}$	MCv	Kitamura et al. (2025) Thm 8
FSTA^{SSYNC}	$>$	FSTA$^{\text{CM-atomic}}$	MCv	Kitamura et al. (2025) Thm 8
OBLOT^{SSYNC}	$>$	OBLOT$^{\text{CM-atomic}}$	MCv	Kitamura et al. (2025) Thm 8
FCOM$^{\text{CM-atomic}}$	$\equiv$	FCOM^{ASYNC}	*simulation*	Kitamura et al. (2025) Thm 4
FSTA$^{\text{CM-atomic}}$	$>$	FSTA^{ASYNC}	DMSD	Kitamura et al. (2025) Thm 9
OBLOT$^{\text{CM-atomic}}$	$>$	OBLOT^{ASYNC}	DMSD	Kitamura et al. (2025) Thm 9
FCOM^{SSYNC}	$\perp$	FSTA^{RSYNC}	SM, RDAM	Kitamura et al. (2025) Thms 10, 11
FCOM^{SSYNC}	$\perp$	FSTA^{SSYNC}	SM, RDAM	Kitamura et al. (2025) Thms 10, 11
FCOM^{SSYNC}	$\perp$	FSTA$^{\text{CM-atomic}}$	SM, RDAM	Kitamura et al. (2025) Thms 10, 11
FCOM^{SSYNC}	$\perp$	FSTA^{ASYNC}	SM, RDAM	Kitamura et al. (2025) Thms 10, 11
FCOM$^{\text{CM-atomic}}$	$\perp$	FSTA^{RSYNC}	SM, RDAM	Kitamura et al. (2025) Thms 10, 11
FCOM$^{\text{CM-atomic}}$	$\perp$	FSTA^{SSYNC}	SM, RDAM	Kitamura et al. (2025) Thms 10, 11
FCOM$^{\text{CM-atomic}}$	$\perp$	FSTA$^{\text{CM-atomic}}$	SM, RDAM	Kitamura et al. (2025) Thms 10, 11
FCOM$^{\text{CM-atomic}}$	$\perp$	FSTA^{ASYNC}	SM, RDAM	Kitamura et al. (2025) Thms 10, 11
FCOM^{ASYNC}	$\perp$	FSTA^{RSYNC}	SM, RDAM	Kitamura et al. (2025) Thms 10, 11
FCOM^{ASYNC}	$\perp$	FSTA^{SSYNC}	SM, RDAM	Kitamura et al. (2025) Thms 10, 11
FCOM^{ASYNC}	$\perp$	FSTA$^{\text{CM-atomic}}$	SM, RDAM	Kitamura et al. (2025) Thms 10, 11
FCOM^{ASYNC}	$\perp$	FSTA^{ASYNC}	SM, RDAM	Kitamura et al. (2025) Thms 10, 11

Claude (Anthropic) was used in cataloguing the problems.

Separating problems

RDV: Rendezvous
Two robots must, within finite time, occupy the same point (not fixed in advance) and stay there. (Suzuki and Yamashita 1999) (Flocchini et al. 2016)

OSP: Oscillating Points
Two robots, from any distance d(t0) > 0, alternately get strictly closer (monotonically) and strictly farther (monotonically) forever. (Das et al. 2016)

¬IL: ¬IL
Three robots in a fixed initial pattern must form a second pattern (a moves 90° clockwise) and then an asymmetric third pattern (b moves 45° clockwise) whose formation requires b to tell a from c. (Flocchini et al. 2019)

TAR(d): Triangle Rotation
Three robots forming triangle ABC (circumcircle C, d known) must form A'B'C' on the same circumcircle with |AA'| = |BB'| = |CC'| = d. (Flocchini et al. 2019)

TAR(d)*: Perpetual Triangle Rotation
TAR(d) repeated forever from each new triangle. (Naito et al. 2025)

CGE: Center of Gravity Expansion
Every robot moves directly from (x,y) to (⌧2x − c_x⌧, ⌧2y − c_y⌧), (c_x, c_y) the initial centre of gravity, then stops. (Flocchini et al. 2019)

CGE*: Perpetual CGE
CGE repeated forever from each new configuration. (Flocchini et al. 2019)

SRO: Shrinking Rotation
Two robots repeatedly move so that the segment either rotates 90° clockwise about its midpoint or rotates 45° and shrinks by $1/\sqrt{2}$, always staying inside the square of the previous step. (Flocchini et al. 2019)

CYC: Cyclic Circles
n ≥ 3 robots: n−1 on a circle, one at the centre; the centre robot must visit, in cyclic order, the 2^{n−1} positions at distances d(i) (d non-invertible) along a fixed radius, returning to the centre between visits. (Buchin et al. 2022)

MLCv: Monotone Line Convergence
CLCv with the distance between the robots never increasing. (Flocchini et al. 2023)

MCv: Monotone Convergence
Two robots converge to a common point without ever increasing their distance (no line constraint). (Kitamura et al. 2025)

TF: Trapezoid Formation
Four robots on a convex quadrilateral with one longest side CD: stay still if it is a trapezoid; otherwise exactly one of a, b (chosen by whether angle $\alpha \geq \pi/4$) moves perpendicular to CD to make ab ∥ CD. (Flocchini et al. 2023)

GCNCL: Get Closer but Not too Close on Line
Two robots approach each other on their segment, never increasing distance, and stop at final distance in [d0/2, d0). (Flocchini et al. 2023)

SM: Single Move
Each of two robots makes exactly one non-null move (attributed to Flocchini et al. 2023 by Naito et al. 2025 and Kitamura et al. 2025, but not in its arXiv or LIPIcs text). (Naito et al. 2025) (Kitamura et al. 2025)

DMSD: Dyadic-Multiple Stopped-Distance
Whenever both of two robots are stopped, their distance is a dyadic-rational multiple of the initial distance. (Kitamura et al. 2025)

RDAM: Rendezvous in one move or Asymmetric Move
RDV1 (meet after exactly one move each) or AM (one robot moves ≤ 1 time and stops, the other moves ≥ 2 times). (Kitamura et al. 2025)

Claude (Anthropic) was used in cataloguing the problems.

ETE: Exponential Times Expansion
n ≥ 3 robots (circle + centre) must form a sequence of 2^{n−1} expansion patterns determined by a non-invertible function (details in paper). (Naito et al. 2025)

HET: Hexagonal Edge Traversal
Two robots on opposite vertices of a regular hexagon must perform a prescribed cyclic 13-step traversal (edge to adjacent vertex, diagonal, return). (Naito et al. 2025)

VEC: Vertical Configuration
SRO-like two-robot rotation problem with an extra 'stay' option and conditions 1–5 (see paper). (Naito et al. 2025)

ZCC: Zig-Zag Collision-free Convergence
Two robots converge collision-free while oscillating along their line (alternately approaching and receding from the start). (Naito et al. 2025)

LP–Cv: Leave the Place – Convergence
Each robot first moves away from the other along their line, then they converge. (Naito et al. 2025)

LCM–M: Least Common Multiple – Movement
Three collinear robots A, B, C with AB:BC = 2:1; B must twice move away from A by d_AB, C once away from A by d_AC. (Naito et al. 2025)

OC: Oscillating Configurations
Four robots must cycle forever through configurations I-II-III-II-I-... (Das et al. 2021a)

IOP: Independent Oscillating Problem
Three collinear robots; each end robot independently oscillates between distance x_i and 2x_i from the middle robot whenever activated. (Das et al. 2021a)

Other settings

- **Limited visibility.** Under FSYNC and SSYNC, every model with full visibility is stronger than the same model with limited visibility, FSYNC is stronger than SSYNC under limited visibility, and a model with limited visibility under FSYNC is incomparable with the same model with full visibility under SSYNC (Das et al. 2024).
- **Opaque robots.** For every memory model and scheduler, opaque robots are weaker than transparent ones; for opaque robots, OBLOT and FSTA are stronger under SSYNC than under ASYNC, and whether LUMI is equally strong under SSYNC and ASYNC is open (Feletti et al. 2025).
- **Robots on graphs.** On graphs embedded in the plane the FSYNC/SSYNC landscape is mapped; one separator uses a crash fault (Sharma et al. 2025).
- **Bounded asynchrony.** With limited visibility and imprecise measurements, oblivious robots are stronger under k-bounded asynchrony (k-ASYNC) than under ASYNC, separated by cohesive convergence; whether k-bounded asynchrony is weaker than SSYNC is open (Kirkpatrick et al. 2024).
- **Sequential schedulers.** Oblivious robots activated one at a time are stronger than under SSYNC and incomparable with FSYNC (Flocchini et al. 2026).

Where the papers disagree

- Rendezvous by FSTA robots in SSYNC (Flocchini et al. 2016) needs each robot's unit of distance to stay fixed; several separations use it, and under variable disorientation it is impossible (Masuzawa et al. 2026).

Claude (Anthropic) was used in cataloguing the problems.

- One paper uses rendezvous by FSTA robots in ASYNC without proof, under variable disorientation where it is impossible; five FSTA-versus-OBLOT relations rest on it (Flocchini et al. 2023) (Masuzawa et al. 2026). $\text{FSTA}^A > \text{OBLOT}^A$ still holds by another separator (Das et al. 2021a).
- The Single Move problem is attributed to (Flocchini et al. 2023) by (Kitamura et al. 2025) and (Naito et al. 2025), but neither version of that paper contains it.
- Simulation costs disagree across papers: for LUMI under SSYNC simulated by FCOM under RSYNC, the colour counts reported for a simulated light with c colours are $O(c \cdot 2^c)$, $64c \cdot 2^c$ and $64c^2$ (Buchin et al. 2022) (Nakajima et al. 2024) (Kitamura et al. 2025).
- Two papers each claim the first separation of SSYNC from ASYNC for oblivious robots, under different assumptions: limited visibility with imprecision (Kirkpatrick et al. 2024), and full visibility (Flocchini et al. 2023).

Open questions

Q1. Is there a problem separating k-bounded asynchrony from SSYNC (Kirkpatrick et al. 2024)?

Q2. The landscape without chirality and rigid moves, especially under ASYNC (Flocchini et al. 2019) (Kitamura et al. 2025).

Q3. Can FSTA robots solve rendezvous in ASYNC with a fixed unit of distance (Flocchini et al. 2016) (Masuzawa et al. 2026)?

Q4. Is LUMI equally strong under SSYNC and ASYNC for opaque robots (Feletti et al. 2025)?

Q5. Which model classes have no problem solvable there but in no strictly weaker model (the "white regions") (Naito et al. 2025)?

Q6. Schedulers beyond the atomicity-defined ones, and robots with energy constraints (Flocchini et al. 2023) (Buchin et al. 2022).

The problems of this model are in Part IV: Formation problems, Symmetry breaking and spreading.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 4

Search in geometric domains

Robots with unit speed move continuously in a known domain (a line, rays, a disk, a polygon) to find a target or an exit whose position is unknown.

What is optimised

The measure is **worst-case time**: for search on unbounded domains, the competitive ratio (time divided by the distance to the target); for evacuation of a bounded domain, the time until the last robot reaches the exit. The questions are exact constants, the price of weaker communication, and the price of faults.

Robots start together (at the origin of a line or rays, or at the centre of a disk or polygon) and move with at most unit speed. A target (or exit) lies at an unknown point; a robot recognises it only when it reaches it. Robots know the domain. In **search** the task ends when the first robot reaches the target; in **evacuation** it ends when the last robot reaches the exit. Robots exchange information either only when they meet (**face-to-face**) or instantly at any distance (**wireless**); some models allow one-way communication.

How the axes work. As in the other models, level 0 assumes nothing and higher levels are stronger assumptions.

4.1 How the assumptions fit together

From the cow path to evacuation

Bellman posed the linear search problem in 1963 and Beck studied it in 1964, with the target placed according to a known probability distribution (Bellman 1963) (Beck 1964) (as reported by Czyzowicz et al. 2021a). Baeza-Yates, Culberson and Rawlins recast it as searching an unbounded domain where each probe costs "proportional to the distance of the next probe position relative to our current position": doubling the turning point (their *linear spiral search*) finds a point at distance d within $9d$ steps, and no algorithm does better up to lower-order terms (Baeza-Yates et al. 1993). Kao, Reif and Tate call it the *cow-path problem* (Kao et al. 1996). Group search sends several robots, and evacuation, posed for a disk by Czyzowicz et al. at DISC 2014, ends only when the last robot reaches the exit (Czyzowicz et al. 2014). Their trade-off is between "spreading sufficiently the robots so that they can find the exit point fast in parallel" and "not to spread them too far" (Czyzowicz et al. 2014). Czyzowicz, Georgiou and Kranakis survey this line of work (Czyzowicz et al. 2019b).

Communication

Once several robots search, what they can tell each other changes the answer. The first evacuation paper already compared robots that talk only when they meet with robots that broadcast at any distance, and showed "a clear separation between the local and the wireless communication models" (Czyzowicz et al. 2014). Later work considered asymmetric senders and receivers, whose evacuation time on the disk lies

strictly between the two (Czyzowicz et al. 2021b) (Georgiou et al. 2022). Communication is therefore the first axis of the model.

Unreliable and unequal robots

Two further axes describe how the robots differ from the ideal. Faults: Czyzowicz et al. introduced crash-faulty searchers that fail to report the target (Czyzowicz et al. 2016b) (as reported by Kupavskii and Welzl 2018) and studied Byzantine searchers that may also lie (Czyzowicz et al. 2021a). Since a lower bound for crash faults is also a lower bound for Byzantine faults, the two form a scale (Kupavskii and Welzl 2018). Speeds: patrolling (Czyzowicz et al. 2011) and two-robot search on the line (Chrobak et al. 2015) (Bampas et al. 2019) gave the robots different maximum speeds, and it turned out that a slow robot can help or be useless depending on the speed ratio (Bampas et al. 2019).

Randomness and the problem axes

Randomness is the fourth axis: Kao, Reif and Tate showed that a randomized cow, whose expected distance is compared with the distance to a goal fixed in advance, does "almost twice as good as is possible with a deterministic algorithm" (Kao et al. 1996). Everything else differs from problem to problem and is kept on the problems as their own axes: the domain (line, m rays, disk, polygons, a fence), the number of robots, what counts as success (first arrival, last arrival, the queen's arrival, fetching a treasure), and how many exits there are.

4.2 Notation

Symbols of this model. Inside the model each symbol has one meaning; *papers write* gives other symbols found in the literature.

Symbol	Meaning	Papers write	Used in
k	number of robots	n	whole model
f	maximum number of faulty robots (crash or Byzantine)		whole model
m	number of rays sharing an endpoint		whole model
e	number of exits on the boundary		Chauffeuring a crashed robot
d	distance from the origin to the target on a line or on rays		whole model
x	position of the target on the line		whole model
R	radius of the disk (results are usually stated for $R = 1$)		whole model
v, v_i	speeds of robots (robot i moves at speed at most v_i)	s, s_i	whole model
α	known arc distance along the boundary (between two exits, or between the treasure and the exit)	d	whole model
r	communication range of wireless robots		whole model
ℓ	length of the fence (segment or circle)		Patrolling
L	known distance between the starting points of two robots		Evacuation
p	index of an ℓ_p norm		Evacuation
w	weight of the second robot's arrival time in weighted group search		Evacuation

Claude (Anthropic) was used in cataloguing the problems.

Symbol	Meaning	Papers write	Used in
σ	number of servants in priority evacuation (the queen and σ servants, $k = \sigma + 1$ robots)	n	Evacuation

4.3 Assumption axes

Each axis says where the assumption comes from and why the community studies it, then defines its values. Every axis is a scale. Level 0 assumes nothing; each higher level is a stronger assumption, which makes a problem easier. Values on the same level are alternatives that do not imply each other. An algorithm designed for weaker assumptions works under stronger ones; a lower bound proved under stronger assumptions holds under weaker ones.

Communication

Czyzowicz et al. defined evacuation from the disk in two communication models, "non-wireless (or local)", where robots exchange information only at the same point at the same moment, and wireless, where a broadcast "is instantly acquired by other robots, independently of their current positions" (Czyzowicz et al. 2014). Their bounds for two and three robots, and for large k, showed "a clear separation between the local and the wireless communication models" (Czyzowicz et al. 2014). The earliest paper in this atlas with mixed abilities is by Czyzowicz et al.: senders transmit wirelessly but receive only face-to-face, and receivers do the opposite; such robots could be cheaper to build, or could be fully wireless robots whose sender or receiver module has failed (Czyzowicz et al. 2021b). For two robots on the disk, Georgiou, Giachoudis and Kranakis showed that the sender/receiver evacuation time lies strictly between the face-to-face and wireless times (Georgiou et al. 2022). Not every setting separates the levels: for two robots of very different speeds on the line, "wireless communication offers no advantage over communication by meeting" (Bampas et al. 2019).

Level	Value	Formal definition
0	**face-to-face**	**Face-to-face**: two robots exchange information only when they are at the same point at the same time.
1	**sender/receiver**	**Sender/receiver**: one robot can only transmit and the other only receive, instantly at any distance.
2	**wireless**	**Wireless**: every robot can send messages to every other robot instantly at any distance.

Faults

Fault-tolerant group search began on the line in 2016. Czyzowicz, Kranakis, Krizanc, Narayanan and Opatrny introduced crash-type faults, robots that "fail to report the target although visiting its location" (Czyzowicz et al. 2016b) (as described by Kupavskii and Welzl 2018). At ISAAC 2016 Czyzowicz et al. studied Byzantine robots, which "can fail to report the target even after reaching it, or ... make malicious claims about having found the target", so that search ends only when the reliable robots have "sufficient verification" (Czyzowicz et al. 2021a). Because a lower bound for crash faults is also a lower bound for Byzantine faults, the two levels form a scale (Kupavskii and Welzl 2018). Unreliable robots had appeared a year earlier in patrolling: at ISAAC 2015 Czyzowicz et al. studied patrollers that "fail to comply with their patrolling duties", so that only visits by reliable robots count (Czyzowicz et al. 2017b). The faults were later carried over to evacuation from the disk (Czyzowicz et al. 2017a). Georgiou, Giachoudis and Kranakis note that "fault tolerant group search has been considered less extensively mainly due to the complexity of the resulting optimization problem" (Georgiou et al. 2022).

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**Byzantine**	Up to f robots may lie: stay silent when they find the target, or claim to have found it where it is not.
1	**crash faults**	Up to f robots may fail to detect or to report the target when they pass over it.
2	**no faults**	Every robot detects and reports the target.

Speeds

In the classical problems all searchers move at unit speed (Baeza-Yates et al. 1993) (Czyzowicz et al. 2014). Czyzowicz, Gąsieniec, Kosowski and Kranakis studied patrolling of a segment and a circle by agents with distinct maximum speeds at ESA 2011 and conjectured that simple partition strategies are optimal (Czyzowicz et al. 2011). The conjecture was later disproved (Kawamura and Kobayashi 2015) (as summarised by Kawamura and Soejima 2020). For search, Chrobak et al. solved two-robot evacuation on the line when the slower robot has at least a third of the faster one's speed (Chrobak et al. 2015), and Bampas et al. covered every speed ratio (Bampas et al. 2019). Czyzowicz et al. motivate such variants by noting that "it is quite natural to consider collaborative tasks in which the participant robots have different capabilities" (Czyzowicz et al. 2018b); disk evacuation with two speeds followed (Lamprou et al. 2020).

Level	Value	Formal definition
0	**distinct speeds**	Each robot i has its own maximum speed v_i, known to all.
1	**equal speeds**	All robots move at most at unit speed.

Randomness

Bellman and Beck posed linear search with a target drawn from a known distribution and the expected time as the measure (Bellman 1963) (Beck 1964) (as reported by Czyzowicz et al. 2021a). In the competitive formulation the target is adversarial, and every deterministic strategy has ratio at least 9 (Theorem 2.1 of Baeza-Yates et al. 1993). Kao, Reif and Tate gave the first randomized algorithm for the cow-path problem on several paths, SmartCow, which picks a random permutation of the paths and a random initial distance; for every goal at distance d fixed in advance its expected distance is at most $c\,d + O(1)$, with $c \approx 4.5911$ on the line, and they proved this optimal on the line (Kao et al. 1996). They note that Gal had given many of these results earlier in a game-theoretic setting (Kao et al. 1996). Randomization thus improves the ratio on the line by about half, which is why the catalogue keeps it as an axis. Most of the group-search and evacuation literature in this atlas is deterministic.

Level	Value	Formal definition
0	**deterministic**	Trajectories are fixed functions of the information received.
1	**randomized**	Trajectories may depend on random choices; the measure is the expected competitive ratio against an oblivious adversary.

The problems of this model are in Part V: Search, evacuation and patrolling.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 5

Programmable matter

Many tiny particles with constant memory occupy nodes of the triangular lattice, move by expanding and contracting, and must self-organise into a shape or elect a leader.

What is optimised

The first question was **what constant-memory, anonymous particles can do at all** (leader election, shapes, coating); then **rounds**, with matching lower bounds where possible, and how weak a scheduler and how little orientation the algorithms can tolerate.

In the **amoebot model** (Derakhshandeh et al. 2014) each particle occupies one node of the infinite triangular lattice (contracted) or two adjacent nodes (expanded). A particle moves by expanding into an empty neighbouring node and contracting into one of its two nodes. Particles are anonymous, have constant memory, and communicate only with neighbours; in the founding announcement each particle is a finite-state machine that reads and writes one flag per incident edge, and a pair of neighbours may combine an expansion and a contraction into a *handover* (Derakhshandeh et al. 2014). An adversarial scheduler activates particles; time is measured in **asynchronous rounds**: intervals of time in which every particle is activated at least once (the epochs of the other models; they need not be the shortest such intervals, and a bound holds for every division of the execution into consecutive rounds). The canonical amoebot model separates the core model from families of assumption variants, including concurrency (Daymude et al. 2023).

How the axes work. As in the other models, level 0 assumes nothing and higher levels are stronger assumptions.

5.1 How the assumptions fit together

Matter that computes

Toffoli and Margolus coined *programmable matter* for matter that changes its physical properties under program control (Toffoli and Margolus 1991) (as reported by Di Luna et al. 2020c, Emek et al. 2019 and Derakhshandeh et al. 2014). Derakhshandeh et al. proposed the amoebot model in a brief announcement at SPAA 2014: particles on the triangular lattice that "have constant memory (in particular, they have no identifiers)", "act asynchronously and make their decision based on local information", and move by expanding and contracting, "which resembles the behavior of amoeba" (Derakhshandeh et al. 2014). The founding papers fixed many details at once: common chirality, one particle active at a time, random bits, constant memory and a structure that stays connected (Derakhshandeh et al. 2014) (Derakhshandeh et al. 2015); the announcement already named compass, initialisation and fault tolerance as dimensions along which the model could vary (Derakhshandeh et al. 2014). Most axes of this model are the history of removing those defaults one by one.

Removing the defaults

Di Luna et al. listed the defaults behind the early shape-formation results (chirality, a sequential schedule and, "more important, randomization") and asked: "What can be done deterministically? Is chirality necessary? ... What happens if the scheduler is not sequential?" (Di Luna et al. 2020c). Their answers, and the leader-election papers that followed (Emek et al. 2019) (Dufoulon et al. 2021) (Chalopin et al. 2026b), gave the axes *orientation*, *scheduler*, *randomness* and *initial shape*. Chalopin, Das and Kokkou observe that every deterministic two-dimensional leader-election algorithm uses at least two extra assumptions from a short menu: randomness, simply connected shape, chirality, movement, a sequential scheduler (Chalopin et al. 2026b). Daymude, Richa and Scheideler then wrote the hierarchy down as the canonical amoebot model, with variants for orientation, memory, concurrency and fairness, because without it "precise systematic comparison of results under the amoebot model is difficult" (Daymude et al. 2023).

Weaker particles, looser structures

Two axes push below the original model. SILBOT particles keep no memory beyond being contracted or expanded, send no messages and act asynchronously with non-atomic actions, so a step depends only on what a particle sees within two hops (D'Angelo et al. 2020); this is the *oblivious* memory level and the *weak* scheduler. The *connectivity* axis records that the requirement that the structure stays connected at every moment is a convention of the literature rather than of the model: Dufoulon, Kutten and Moses Jr. disconnected and reconnected the structure on purpose to elect a leader faster (Dufoulon et al. 2021).

Other mechanisms

Not every problem in this model is about amoebots. Problem-specific axes cover modules on the square grid that rotate or slide under central control (Michail et al. 2019), the hybrid model in which one finite-state agent moves passive tiles in two or three dimensions (Gmyr et al. 2020) (Hinnenthal et al. 2024), and, for leader election, whether particles may move at all (Chalopin et al. 2026b). These axes apply to a single problem each. They are kept on the problem so that the model's own axes stay shared across problems.

5.2 Notation

Symbols of this model. Inside the model each symbol has one meaning; *papers write* gives other symbols found in the literature.

Symbol	Meaning	Papers write	Used in
n	number of particles (modules, tiles)		whole model
D	diameter of the configuration (largest lattice distance between two particles)		whole model
L_{out}, L_{max}	length of the outer boundary and of the longest boundary of the configuration		Leader election in programmable matter
λ	bias towards having more neighbours (stochastic approach)		whole model
γ	bias towards neighbours of the same colour (separation)		Separation of coloured particles
α	compression factor: perimeter at most α times the minimum		Compression

Claude (Anthropic) was used in cataloguing the problems.

Symbol	Meaning	Papers write	Used in
q	number of orientations (alignment)		Alignment
s	number of energy sources		Energy distribution
k	symmetricity of a configuration (a k-symmetric configuration admits up to k leaders)		Leader election in programmable matter

5.3 Assumption axes

Each axis says where the assumption comes from and why the community studies it, then defines its values. Every axis is a scale. Level 0 assumes nothing; each higher level is a stronger assumption, which makes a problem easier. Values on the same level are alternatives that do not imply each other. An algorithm designed for weaker assumptions works under stronger ones; a lower bound proved under stronger assumptions holds under weaker ones.

Orientation

In the geometric amoebot model of Derakhshandeh et al., each particle labels its six ports clockwise from an arbitrary edge, so all particles share chirality but not a direction (Derakhshandeh et al. 2015); the 2014 announcement numbers the edges counter-clockwise and calls this the "no-compass model where particles only share their sense of orientation", as opposed to a full-compass model (Derakhshandeh et al. 2014). Di Luna et al. asked whether chirality is necessary and showed that it is "computationally irrelevant for shape formation" from simply connected configurations (Di Luna et al. 2020c). Emek et al. elected a leader with no orientation at all, using a procedure by which the particles reach common chirality (Emek et al. 2019). Chalopin, Das and Kokkou replaced chirality by agreement on one lattice direction and found that the difficulties are different: a unique stationary leader can then be elected, but explicit termination is impossible (Chalopin et al. 2026b). The canonical amoebot model lists all four levels: assorted, common chirality, common direction, and common direction and chirality (Daymude et al. 2023).

Level	Value	Formal definition
0	**no orientation**	Each particle labels its six incident directions in its own clockwise or counter-clockwise order, starting anywhere.
1	**chirality**	All particles agree on clockwise order, not on a common direction.
1	**common direction**	All particles agree on one lattice direction, not on handedness. Chalopin, Das and Kokkou call this a *common direction* (Chalopin et al. 2026b), but the same words mean agreement on all six directions in other papers (Chalopin et al. 2026a) (Navarra and Piselli 2023b).
2	**common compass**	All particles agree on all six directions.

Scheduler

The early amoebot papers assumed "the standard asynchronous computation model, i.e., only one particle can be active at a time" (Derakhshandeh et al. 2015); in the 2014 announcement, from the set of enabled actions "a single action is arbitrarily chosen and executed" (Derakhshandeh et al. 2014). Di Luna et al. dropped this for a strong scheduler that activates many particles but executes each activation atomically (Di Luna et al. 2020c). Dufoulon, Kutten and Moses Jr. use *strong* for atomic activations and *weaker* for non-atomic ones, under which, they report, deterministic election of a unique leader becomes

Claude (Anthropic) was used in cataloguing the problems.

impossible in some cases (attributed by them to Bazzi and Briones and to Di Luna et al.) (Dufoulon et al. 2021) (Bazzi and Briones 2019). Daymude, Richa and Scheideler observed that "much of the existing literature implicitly assumes amoebot actions are isolated and reliable, reducing analysis to the sequential setting", whereas "real programmable matter systems are concurrent"; their canonical model orders the concurrency variants from asynchronous through synchronous and k-isolated to sequential (Daymude et al. 2023). The fully asynchronous SILBOT model sits at the weak end: all operations are "non-atomic", with possible delays between sensing, computing and acting, so an action may be based on an obsolete observation (D'Angelo et al. 2020).

Level	Value	Formal definition
0	**weak scheduler**	An activation's look, compute and move need not be atomic; concurrent activations may interfere. The scheduler of Bazzi and Briones is described as weak sequential, as non-atomic, and as fair asynchronous by different later papers (Briones et al. 2023) (Dufoulon et al. 2021) (Chalopin et al. 2026b).
1	**strong scheduler**	Each activation is executed atomically; the adversary chooses the order, fairly.
2	**sequential**	At most one particle is active at any time.

Memory

Constant memory is part of the amoebot model from its start: each particle runs a fixed finite program and reads and writes memory shared with its neighbours (Derakhshandeh et al. 2015); in the 2014 announcement the state set and the flag alphabet on each edge are finite (Derakhshandeh et al. 2014). D'Angelo et al. introduced SILBOT at AAMAS 2020, in which particles are "asynchronous, without any direct means of communication (silent) and without memory of past events (oblivious)", to find "the minimal settings for PM under which it is possible to accomplish basic global tasks" (D'Angelo et al. 2020) (as described by Navarra and Piselli 2023b). Each SILBOT particle is "an automaton with two states, CONTRACTED or EXPANDED (they do not have any other form of persistent memory)" (D'Angelo et al. 2020). In exchange, particles see two hops instead of one, because "without communication, 1-hop visibility is not enough for a particle to decide whether it is a LEADER or not" (D'Angelo et al. 2020). The canonical amoebot model lists oblivious, constant, finite and unbounded memory as its memory variants (Daymude et al. 2023). Oblivious particles have since solved leader election, line formation, coating and self-stabilising election with holes (D'Angelo et al. 2020) (Navarra and Piselli 2023b) (Navarra and Piselli 2023a) (Chalopin et al. 2026a).

Level	Value	Formal definition
0	**oblivious**	No persistent state and no messages; a particle decides from its view only.
1	**constant memory**	Each particle has $O(1)$ bits of persistent memory and reads or writes neighbours' memory.

Randomness

The first leader-election algorithm for amoebots used random bits to break symmetry (Derakhshandeh et al. 2015), and Di Luna et al. listed randomization as the "more important" of the assumptions behind the early shape-formation results (Di Luna et al. 2020c). Deterministic algorithms then became a goal of their own: for a simply connected start up to k leaders are elected in a k-symmetric configuration (Di Luna et al. 2020c), and Emek et al. gave the first deterministic election of a unique leader using movement (Emek et al. 2019). A second use of randomness is the stochastic approach of Cannon, Daymude, Randall and Richa, who design a Markov chain whose stationary distribution favours the desired configurations and turn it into a local asynchronous algorithm (Cannon et al. 2016). The design is motivated by statistical

Claude (Anthropic) was used in cataloguing the problems.

physics, and the approach has been applied to separation, alignment and bridging (Cannon et al. 2019) (Kedia et al. 2022) (Arroyo et al. 2018).

Level	Value	Formal definition
0	**deterministic**	No random choices.
1	**randomized**	Particles flip fair coins; guarantees hold with high probability or in expectation.

Initial shape

The first leader-election algorithm for amoebots worked on every boundary of the structure, outer and inner, and its running time depends on the longest boundary (Derakhshandeh et al. 2015) (as summarised by Dufoulon et al. 2021). Several early deterministic algorithms assumed the initial configuration has no holes: Di Luna et al. start from any simply connected configuration (Di Luna et al. 2020c) and Gastineau et al. assume chirality and no holes (Gastineau et al. 2019). Dufoulon, Kutten and Moses Jr. summarise that early deterministic algorithms that did not use movement relied on "assumptions on the initial shape ... (intuitively, that it had no 'holes')" (Dufoulon et al. 2021). Emek et al. removed the assumption by letting particles move (Emek et al. 2019), and Chalopin, Das and Kokkou extended self-stabilising election from simply connected systems to systems with holes (Chalopin et al. 2026a).

Level	Value	Formal definition
0	**holes allowed**	The initial set of occupied nodes induces a connected subgraph; it may enclose empty regions.
1	**no holes**	The initial configuration is connected and encloses no empty node. Also called *contractible* (Briones et al. 2023) or *hole-free* (Gastineau et al. 2019).

Connectivity during execution

The founding papers required the particles to stay connected at all times, "to prevent the particles from drifting apart in an uncontrolled manner like in fluids and because in our case particles communicate only via bonds" (Derakhshandeh et al. 2015). Emek et al. still followed "the common practice" that the shape is connected at all times (Emek et al. 2019). Dufoulon, Kutten and Moses Jr. observed that the model itself does not require this, and gave a leader-election algorithm that disconnects and later reconnects, gaining a large speed-up in time (Dufoulon et al. 2021). In SILBOT, line formation also passes through disconnected configurations and proves that connectivity is recovered (Navarra and Piselli 2023b). In centralised transformations the same distinction separates transformations that may break connectivity from those that may not (Michail et al. 2019).

Level	Value	Formal definition
0	**stays connected**	At every time the occupied nodes induce a connected subgraph.
1	**may disconnect**	Connectivity is required only initially and at the end.

The problems of this model are in Part VI: Self-organising particle systems.

Claude (Anthropic) was used in cataloguing the problems.

Part II

Perspectives

Essays that connect the problems of the atlas across models: where they come from, why the papers say they matter, and how techniques and lower bounds move between them. They assume the models of Part I.

Chapter 6

Why compute by moving

The atlas collects four models that grew out of four different wishes: software that travels through a network, swarms of robots that are as weak as possible, searchers that must find something at an unknown place, and matter that reshapes itself. They share an adversary, locality and symmetry; they differ in what they count.

Problems: Graph exploration by a single agent, Collaborative exploration, Dispersion, Rendezvous, Black hole search, Gathering, Pattern formation, Convergence, Linear search (cow path), Evacuation, Leader election in programmable matter, Shape formation in programmable matter.

Four communities, one abstraction

The four models of the atlas (agents on graphs, robots in the plane, search in geometric domains, programmable matter) were built by different groups for different reasons, and their papers still open with different stories. What they have in common is the decision to abstract from the hardware. Pelc puts it this way for rendezvous: "Since rendezvous algorithms do not depend on the physical nature of the mobile entities executing them, but only on their perception capabilities, memory size, mobility characteristics and on the structure of the environment, we will not distinguish between natural and artificial agents" (Pelc 2019). Each model then fixes those four ingredients differently, and the choice is what makes a problem easy or hard in it.

This essay follows the introductions of the founding papers and surveys: what each community wanted when it started, and what it chose to measure. The other perspectives take single threads (meeting, faults, lower bounds) across the same four models.

Agents on graphs: software that travels

In the graph model the entity is usually a program. Pelc's survey names "software agents, i.e., mobile pieces of software that navigate in a computer network in order to perform maintenance of its components or to collect data distributed in nodes of the network" (Pelc 2019), and later papers keep the same picture of "a software entity endowed with limited memory, capable of traversing a network and visiting nodes" (Kaur et al. 2026a). The oldest problem of the model is older than computer networks: Shannon built a mouse that explores a 5×5 maze using constant memory per vertex (Shannon 1951), as recalled by (Disser et al. 2019). Exploration also mattered to complexity theory. Fraigniaud et al. list "robot navigation, network maintenance, resource discovery, and WWW search" as applications and add that a finite set of finite automata exploring all graphs "would have put USTCON in L" (Fraigniaud et al. 2006c); Disser, Hackfeld and Klimm call exploration "a natural abstraction of a process of computing where nodes correspond to states" (Disser et al. 2019).

Later problems kept the network story but added a reason to move. Black hole search starts from security: a host that "disposes of visiting agents upon their arrival, leaving no observable trace", which can be a virus or simply "the undetectable crash failure of a site" (Di Luna et al. 2025b). Dispersion

was introduced with self-driving electric cars looking for free charging stations, and from the start was placed next to scattering, exploration and load balancing (Augustine and Moses Jr. 2018). What the model counts follows from the picture of a small program on a large network: rounds or moves, the number of agents, and above all the bits of memory an agent carries.

Robots in the plane: how little is enough

The plane model starts from the opposite end: the robots are physical, but the question is how weak they can be. "The study of swarms of mobile robots from an algorithmic perspective was initiated by Suzuki and Yamashita, and the simplicity of their model fostered many research results" (Bramas et al. 2023b), referring to (Suzuki and Yamashita 1999). The applied motivation is deployment where no infrastructure exists: "space, deep sea, or after some natural (or unnatural) disaster", so that the group "must self-organize in the absence of any prior infrastructure (e.g., no global positioning)" (Clement et al. 2010). The research motivation is stated plainly: "The primary motivation in this field is to find the minimum capability required to achieve certain objectives for a system of robots" (Pattanayak et al. 2019b).

Hence the questions are of the form "under which conditions this task can be accomplished by the robots?" (Prencipe 2007). Robots are anonymous, silent and oblivious; what varies is synchrony, agreement on coordinates, multiplicity detection, visibility and, since lights were added, a constant number of persistent colours (Das et al. 2016). Flocchini, Santoro and Wada describe the programme as determining "the impact that internal capabilities (e.g., memory, communication) and external conditions (e.g. synchrony, activation scheduler) have on the solvability of a problem" (Flocchini et al. 2019). The primary measure is therefore feasibility; colours come second, and time (in rounds or epochs) is often not analysed at all.

Search in geometric domains: distance against ignorance

The search model comes from operations research rather than computing. Bellman posed the linear search problem in 1963 and Beck studied it in 1964; Kupavskii and Welzl quote Beck's formulation, "A man in an automobile searches for another man who is located at some point of a certain road" (Kupavskii and Welzl 2018) (Beck 1964). Computer scientists rediscovered it in the late 1980s as the cow-path problem, with the worst-case (competitive) ratio as the measure, and the doubling strategy with ratio 9 is optimal (Kupavskii and Welzl 2018) (Baeza-Yates et al. 1993). Brandt et al. state the method directly: "The standard toolkit to analyze this class of problems is competitive analysis" (Brandt et al. 2020).

The distributed version adds several robots and a communication model. Czyzowicz et al. describe what is abstracted away: in geometric environments "their performance is measured by the geometric distance travelled, most often disregarding their computing, communicating and environment-perceiving activities" (Czyzowicz et al. 2014). Evacuation, which they posed, asks for "a compromise between, on the one hand, spreading sufficiently the robots so that they can find the exit point fast in parallel, and, on the other hand, not to spread them too far" (Czyzowicz et al. 2014). The robots know the domain and each other's trajectories; what they lack is the location of the target. The answers are exact constants, and the open problems are gaps between two constants.

Claude (Anthropic) was used in cataloguing the problems.

Programmable matter: computing inside the material

Programmable matter begins with a vision rather than an application: "a material that can dynamically alter its physical properties in a programmable fashion, controlled either by user input or its own autonomous sensing of its environment" (Daymude et al. 2023). The term is attributed to Toffoli and Margolus by (Di Luna et al. 2020c) and (Dufoulon et al. 2021); Di Luna et al. list smart materials, monitoring and repair, and minimally invasive surgery as envisioned uses (Di Luna et al. 2020c). The amoebot model, introduced at SPAA 2014 (Derakhshandeh et al. 2014), is "motivated by micro- and nano-scale robotic systems with strictly limited computational and locomotive capabilities" (Daymude et al. 2023). Dufoulon, Kutten and Moses note that smart particles "need to be small. Thus, they are also weak in their memory size, computational power, communication ability, movement ability" (Dufoulon et al. 2021).

The first problems were shape formation and leader election, and their link was there from the beginning: "most shape formation algorithms depend on some seed element", and the founding paper asks "whether shape formation can even be solved in circumstances where leader election is not possible" (Derakhshandeh et al. 2015). Later models removed more: SILBOT has no communication and no memory, to find "the minimal settings for PM under which it is possible to accomplish basic global tasks" (Navarra and Piselli 2023b). The measures are rounds (under an adversarial scheduler) and the total number of moves; a centralised line of work asks what transformations are possible at all under given movement mechanisms (Michail et al. 2019).

What they share, and what each one counts

Three ingredients recur. The first is an **adversary**: in a deterministic setting "the initial positions of the agents are chosen by an adversary which models the worst-case situation" (Pelc 2019), and the adversary also picks the port numbers, the activation schedule, the moving edges of a dynamic graph, or the exit of a disk. The second is **locality**: an agent sees only its node and its ports, an oblivious robot sees a snapshot but no history, a particle sees its neighbours, and a searcher recognises the target only when it stands on it. The third is **symmetry**: identical entities in symmetric positions cannot be separated by any deterministic rule. Ta-Shma and Zwick give the standard example, two identical robots in a ring whose ports are all oriented the same way: "they would never meet!" (Ta-Shma and Zwick 2014).

What each model counts follows from its origin. Agents on graphs count rounds, moves, team size and memory bits, because the entity is a program in a network. Robots in the plane first ask whether a task is solvable at all, then how many colours it needs. Search counts distance travelled, as a competitive ratio with exact constants. Programmable matter counts rounds and moves of a very large number of constant-memory particles. Bampas et al. make the contrast explicit within one model: in search, cost is "the time when the first searcher arrives at the target", while in rendezvous it is "the time when the last involved robot reaches the meeting point" (Bampas et al. 2019). The same task, read with a different measure, becomes a different problem, and that is why the atlas keeps the four models apart while linking their problems.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 7

Symmetry is the adversary

Identical agents running the same deterministic program can be kept in lockstep forever. Across graphs, the plane and programmable matter, the papers break this symmetry with identifiers, with randomness, with asymmetric starts, or with the scheduler, and leader election keeps reappearing as the common core.

Problems: Rendezvous, Gathering in graphs, Gathering, Scattering, Dispersion, Leader election in the plane, Pattern formation, Universal pattern formation, Leader election in programmable matter, Leader election by agents.

Why identical agents get stuck

Pelc's survey of deterministic rendezvous names the obstacle directly: "The main problem that has to be solved in order to make deterministic rendezvous possible, both in networks and in the plane, is breaking symmetry" (Pelc 2019). His example is an oriented ring, port 0 clockwise at every node, with two identical agents that start at the same time: whatever one does, the other does too, so the distance between them never changes. The same holds in the empty plane for agents with the same compass, unit and speed (Pelc 2019). Ta-Shma and Zwick make the same point when they give their rendezvous agents labels: without them "the rendezvous problem cannot be solved deterministically" (Ta-Shma and Zwick 2014).

The argument travels to every model in the catalogue. For dispersion, Das, Bose and Sau spell it out: anonymous agents that start on one node are in the same state and run the same algorithm, so "they will always mirror each other's move" and never separate; the same reasoning gives the $\Omega(\log k)$ bits per agent, for k agents, that every deterministic dispersion algorithm needs (Das et al. 2021b) (Augustine and Moses Jr. 2018). In the plane, two oblivious robots cannot gather under a semi-synchronous scheduler (Flocchini et al. 2016) (first shown by Suzuki and Yamashita Suzuki and Yamashita 1999, as the catalogue records it), and scattering robots that may start on a common point has no deterministic solution (Dieudonné and Petit 2009). Clement, Défago, Potop-Butucaru, Izumi and Messika put it generally: because robots are anonymous, "the impossibility results proved in classical distributed computing also hold in robot networks", and flocking, gathering or scattering are impossible without additional assumptions (Clement et al. 2010).

Which configurations are symmetric

Each model has its own notion of symmetry, and the strongest results characterise it exactly. In port-labelled graphs the notion is the *view* of a node, the infinite tree of port sequences seen from it (introduced by Yamashita and Kameda, as recounted by Pelc 2019). Two anonymous agents can meet if and only if their starting nodes have different views (Czyzowicz et al. 2012b), as stated in (Pelc 2019). For larger anonymous teams the asymmetry may come from the agents' positions relative to each other rather than from the graph, which Pelc singles out as the main difficulty solved by Dieudonné and Pelc (Dieudonné and Pelc 2016) (Pelc 2019).

In the plane the notion is geometric. Dieudonné, Levé, Petit and Villain characterise the positions from which disoriented robots can deterministically elect a leader, with and without a common handedness, for every $n > 1$; both characterisations rest on Lyndon words built from the positions (Dieudonné et al. 2013). For fat robots, Gan Chaudhuri and Mukhopadhyaya show that a leader can be elected exactly from asymmetric configurations (Chaudhuri and Mukhopadhyaya 2015).

Leader election as the common core

Dieudonné, Levé, Petit and Villain open their paper with the reason leader election matters: "many problems that are hard to solve become easier to solve with a leader to coordinate the system" (Dieudonné et al. 2013). For robots in the plane the link is an equivalence. Flocchini, Prencipe, Santoro and Widmayer showed that $n \geq 3$ robots that cannot elect a leader cannot form arbitrary patterns (as reported by Dieudonné et al. 2013). Dieudonné, Petit and Villain proved the converse in asynchronous systems for $n \geq 4$ with chirality and $n \geq 5$ without (Dieudonné et al. 2010). Cicerone, Di Stefano and Navarra later gave a full characterisation without chirality, arbitrary pattern formation being solvable exactly from leader configurations, and reported that several earlier proofs, including the arXiv version of (Dieudonné et al. 2010), are not correct (Cicerone et al. 2019).

Programmable matter repeats the pattern. Derakhshandeh et al. motivate leader election by observing that consensus "can easily be solved once the leader election problem can be solved" and that most shape-formation algorithms depend on a seed (Derakhshandeh et al. 2015). Emek, Kutten, Lavi and Moses Jr. call it "an important primitive ... that subsequently breaks symmetry and coordinates the remaining amoebots", and their deterministic algorithm without chirality also makes the particles agree on a handedness (Emek et al. 2019). In the agent-based model on graphs, Kshemkalyani, Kumar, Molla and Sharma note that leader election "can also be seen as a form of symmetry breaking" and build minimum spanning trees, independent sets and dominating sets on the elected leader (Kshemkalyani et al. 2024).

Identifiers, and how many are needed

The first way out is to give agents labels. Pelc lists it first: distinct labels, each known to its owner, make deterministic rendezvous possible in every connected graph, at a cost that grows with the length of the labels (Pelc 2019) (Dessmark et al. 2006). In dispersion the identifiers are what the memory pays for: the $\Omega(\log k)$ lower bound is the cost of storing a distinct identifier (Kshemkalyani et al. 2025a).

Between anonymous and fully labelled lies homonymy, which Delporte-Gallet et al. introduced for Byzantine agreement (Delporte-Gallet et al. 2013). Bouzid and Lamani brought it to robots, recalling that sequences of patterns are formable by labelled robots almost without restriction but by anonymous robots only when all patterns have the same symmetricity (Das et al. 2015); this gap, they write, "illustrates how much anonymity can be a limiting factor" (Bouzid and Lamani 2011). Asahiro and Yamashita treat the number of distinct programs (target functions) as a resource: gathering needs two, and three once a robot may crash (Asahiro and Yamashita 2025). On graphs, Devismes, Dieudonné and Labourel argue that unique labels can be costly to maintain or undesirable for privacy, and that randomised alternatives give no absolute guarantees; they characterise the teams with repeated labels that can gather and gather all of them in time polynomial in the number n of nodes and in $\log \lambda$, where λ is the smallest label (Devismes et al. 2026).

Claude (Anthropic) was used in cataloguing the problems.

Randomness and the scheduler

The second way out is randomness. Clement et al. remark that work on robot impossibilities had focused on extra assumptions rather than on randomisation (Clement et al. 2010). Scattering is the clean case: no deterministic algorithm exists (Dieudonné and Petit 2009), and Bramas and Tixeuil show that, to scatter n robots, $n \log n$ random bits are necessary in total, give a scattering algorithm that uses that many bits and runs in $O(\log \log n)$ rounds without strong multiplicity detection, and prove that no algorithm without strong multiplicity detection runs in constant expected time (Bramas and Tixeuil 2017). For dispersion, coin flips replace identifiers: anonymous randomised agents disperse with $O(\log \Delta)$ bits on graphs of maximum degree Δ (Das et al. 2021b), matching the $\Omega(\log \Delta)$ bound of Molla and Moses Jr. (Molla and Moses Jr. 2019). The randomised amoebot algorithm of Derakhshandeh et al. elects a leader in expected time linear in the longest boundary (Derakhshandeh et al. 2015).

The third way out lives in the scheduler. Under a sequential scheduler, which activates one robot per round, robots can break symmetry and elect a leader. Flocchini, Navarra, Pattanayak, Piselli and Santoro show that this power is "much stronger than the obvious capacity these schedulers offer of breaking symmetry": every pattern except a point becomes formable from every start, which is impossible under the fully synchronous scheduler even for robots with a leader (Flocchini et al. 2026).

What remains open

The cost of breaking symmetry is not settled even in the problems that are otherwise solved. Deterministic dispersion runs in optimal $O(k)$ rounds, but with $O(\log(k + \Delta))$ bits; whether $O(\log k)$ bits, the price of an identifier, suffice is open (Kshemkalyani et al. 2025a). Anonymous randomised agents disperse with optimal memory, but in $\Theta(k^2)$ rounds against an $\Omega(k)$ lower bound (Das et al. 2021b). Whether anonymous randomised agents can disperse from an arbitrary configuration and then stop, rather than stay active forever, is open (Molla and Moses Jr. 2019) (Das et al. 2021b). And faults bring indistinguishability back: weak Byzantine agents can behave exactly like honest ones and occupy nodes, which makes balanced dispersion impossible whenever $\lceil k/n \rceil > \lceil (k - f)/n \rceil$ (Molla et al. 2021).

Claude (Anthropic) was used in cataloguing the problems.

Chapter 8

Finding your way

A single agent in an anonymous graph: how much memory it needs, what a pebble, an advice string or knowledge of n buys, why stopping is harder than exploring, and how treasure hunt asks for exploration only as far as the target. Each line of papers, in its own words, on why the question matters.

Problems: Graph exploration by a single agent, Treasure hunt, Rendezvous, Collaborative exploration.

Why the question is asked

The papers give three reasons, and they explain why the field measures memory rather than time. The first is practical. Fraigniaud, Ilcinkas, Peer, Pelc and Peleg: visiting all nodes "is fundamental in searching for data stored at unknown nodes of a network", traversing all edges is needed "in network maintenance and when looking for defective components", and since "robots and mobile agents are meant to be simple, often small and inexpensive devices", they ask for the least memory that suffices, "regardless of the time of exploration" (Fraigniaud et al. 2005a). They also explain the anonymous model: node labels may be absent or invisible to the agent, but ports must be distinguishable at a node, otherwise not even a star with three leaves can be explored (Fraigniaud et al. 2005a).

The second reason comes from complexity theory. Fraigniaud, Ilcinkas, Rajsbaum and Tixeuil recall that a finite set of finite automata able to explore all graphs "would have put USTCON in L", which motivated "quite a long sequence of studies" until Reingold placed undirected connectivity in log-space (Fraigniaud et al. 2006c) (Reingold 2008). The third is a modelling view stated by Disser, Hackfeld and Klimm: exploration is "a natural abstraction of a process of computing where nodes correspond to states", so a single agent is a single processor and a team of agents is a parallel machine (Disser et al. 2019).

Automata in mazes

The history is told the same way in several introductions. Shannon's maze-solving machine of 1951 is the "first known finite automaton algorithm designed for graph exploration" (Fraigniaud et al. 2005a) (Shannon 1951). Rabin claimed in 1967 that no finite automaton with finitely many pebbles explores all graphs; Budach gave the first formal proof for an automaton without pebbles, phrased for labyrinths (Budach 1978); and Rollik proved that no finite set of cooperating finite automata explores all cubic planar graphs, which implies Rabin's claim because an automaton is at least as strong as a pebble (Rollik 1980). In mazes the picture is kinder: Blum and Kozen showed that a finite automaton with two pebbles, or two cooperating automata, search every two-dimensional maze, because an automaton in a maze "has a compass: it can distinguish N,E,S,W"; on finite planar cubic graphs, by contrast, "no three finite automata together can search all" of them (Theorem 6.1) (Blum and Kozen 1978). The statements about Rabin, Budach and Rollik are as recounted by (Fraigniaud et al. 2005a) and (Disser et al. 2019) (Blum and Kozen also recount Budach's result); those originals were not read for this catalogue.

The modern question starts from those traps. Fraigniaud et al. build, for every K-state automaton, a

planar trap with at most $K + 1$ nodes, and deduce that exploring all graphs of diameter D and maximum degree Δ needs $\Omega(D \log \Delta)$ bits; a depth-first search of depth $D + 1$ shows this is tight (Fraigniaud et al. 2005a). Two resources appear already in this early work and stay in all later work: memory inside the agent, and markers outside it.

Memory, and what knowing n changes

On trees the answer depends on what the agent must do at the end. Perpetual exploration needs and can do with $\Theta(\log \Delta)$ bits (Diks et al. 2002); a memoryless agent that always leaves by the next port after the one it entered also works, as Cohen et al. recall (Cohen et al. 2008). Exploration with return needs $\Theta(\log n)$ bits (Diks et al. 2002) (Ambühl et al. 2011). Exploration with stop sits between $\Omega(\log \log \log n)$ and $O(\log n)$, and whether stopping is cheaper than returning is the open problem left by Diks, Fraigniaud, Kranakis and Pelc (Diks et al. 2002).

Knowledge of n changes what is possible. With an upper bound on n, a random walk with an $O(\log n)$-bit step counter explores any graph with high probability (Aleliunas et al. 1979), and Reingold's log-space universal exploration sequences make this deterministic with $O(\log n)$ bits (Reingold 2008). Disser et al. summarise the single-agent picture as $\Theta(\log n)$ bits, necessary and sufficient (Disser et al. 2019); the lower bound the catalogue records for agents that know n is $\omega((\log n)^{1-\varepsilon})$ bits, and whether the full $\Omega(\log n)$ holds when n is known is listed as open on the exploration page.

Stopping is the hard part

Visiting every node is not the same as knowing that every node has been visited. Without a bound on n and without any way to mark a node, a deterministic agent cannot explore with stop, whatever its memory (Dieudonné and Pelc 2012); already an even cycle cannot be explored with stop without a marker (Fraigniaud et al. 2006c). One pebble that marks the start restores it: depth-first phases of increasing depth find new nodes, and the agent stops after phase $D+1$, using $O(D \log \Delta)$ bits (Fraigniaud et al. 2005b). Fraigniaud, Ilcinkas, Rajsbaum and Tixeuil introduced the reduced-automata technique to prove lower bounds of this kind, and showed $\Omega(\log n)$ bits for exploration with stop and one pebble (Fraigniaud et al. 2006c).

Termination returns in the multi-agent problems. In dispersion, Sudo et al. make all agents stop in the rooted case only with global upper bounds on k and on the number of edges (Sudo et al. 2024), and the only asynchronous dispersion algorithm known in which every agent terminates takes time exponential in the diameter (Kshemkalyani and Ali 2019); the dispersion variants list the open questions.

Help from outside: pebbles, labels and ports

Cohen, Fraigniaud, Ilcinkas, Korman and Peleg view pebbles as the agent's ability "to dynamically label the nodes", and ask what happens if a designer labels the nodes beforehand (Cohen et al. 2008). Their answer is that 2-bit labels (three values) let an automaton with fewer than 25 states explore every graph and stop, and 1-bit labels suffice for graphs of bounded degree, although no finite automaton explores all graphs of maximum degree 3 unaided (Cohen et al. 2008). Ilcinkas asks the dual question for ports: since an adversarial port numbering defeats every finite automaton, how short can the period of a finite automaton be if the designer chooses the ports? Periodic exploration, he writes, is "particularly useful for network maintenance, where every node has to be regularly checked"; a three-state automaton achieves period $4n - 2$ (Ilcinkas 2008), later improved to $3.5n - 2$ against a lower bound of $2n - 2$ (Czyzowicz et al. 2012c).

Claude (Anthropic) was used in cataloguing the problems.

Disser, Hackfeld and Klimm settled the exchange rate between memory and markers: a constant-memory agent needs and can do with $\Theta(\log \log n)$ pebbles, even without knowing n, and the trade-off has two thresholds, no pebbles needed with $\Omega(\log n)$ bits and $\Omega(\log \log n)$ pebbles needed as soon as the memory drops to $O((\log n)^{1-\varepsilon})$ (Disser et al. 2019). At the far end, an automaton with a single state explores every connected graph with two pebbles, and two is the minimum (Pattanayak and Pelc 2024a).

Treasure hunt: exploring only as far as needed

Ta-Shma and Zwick introduced deterministic treasure hunt as "a version of the rendezvous problem in which one of the robots is completely passive" (Ta-Shma and Zwick 2014). They point out that the problem is easy if the treasure is present when the agent starts; the hard case is a treasure placed later, which must still be found within time polynomial in n, and it led them to strongly universal exploration sequences. Whether this is possible without backtracking (when the agent does not see its entry port) is still open in their words: "We do not know if an efficient solution is possible when backtracking is not allowed" (Ta-Shma and Zwick 2014). Xin improved the cover time of these sequences (Xin 2007).

Later papers motivate the problem with rescue. Bouchard, Dieudonné, Labourel and Pelc: "When a miner is lost in a contaminated mine, it may have to be found by a robot, and the length of the robot's trajectory should be as short as possible" (Bouchard et al. 2023a). Cost is then measured against the distance d to the treasure, or $e(d)$, the number of edges near the start, not against the size of the graph; with labelled nodes they reach $O(e(d) \log d)$ against $\Omega(e(d))$ (Bouchard et al. 2023a). Gorain, Mondal, Nayak and Pandit note that depth-first search costs $O(\Delta^d)$ for a treasure at distance d and that urgent searches call for help (Gorain et al. 2022a). Miller and Pelc measure that help as bits of advice from an oracle and give the trade-off between advice and cost (Miller and Pelc 2015); Gorain et al. instead place pebbles once, so that many agents can reuse them (Gorain et al. 2022a). Pattanayak and Pelc drop the assumption that degrees are finite, which they report all earlier treasure hunt and rendezvous algorithms made, motivated by graphs of very large degree such as social networks (Pattanayak and Pelc 2024b).

Further reading

Directed graphs, where the agent cannot walk back, are a separate branch: one pebble and $O(n\Delta_{out} \log n)$ bits suffice against an $\Omega(n \log \Delta_{out})$ lower bound, where Δ_{out} is the maximum out-degree (Fraigniaud and Ilcinkas 2004), and a known bound on n gives polynomial time (Bender et al. 2002). See the directed variants of exploration.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 9

From exploration to dispersion, and beyond

How collaborative exploration asks how much a team speeds up a search, how dispersion arose as the case $k = n$ under weaker assumptions, what dispersion gives back to exploration, and why dispersing first has become the standard opening move for computing graph structures with agents.

Problems: Graph exploration by a single agent, Collaborative exploration, Dispersion, Leader election by agents, Maximal independent set by agents, Minimal dominating set by agents, Minimum spanning tree by agents, BFS tree by agents, Triangle and butterfly counting by agents.

Exploring together

Fraigniaud, Gąsieniec, Kowalski and Pelc posed collective tree exploration: k agents start at the root of an unknown tree and must visit every node as fast as possible (Fraigniaud et al. 2006a). The offline optimum is $\Theta(n/k + D)$, by spreading the agents along an Euler tour of the tree (Disser et al. 2020). Their algorithm is $O(k/\log k)$-competitive, which Disser, Mousset, Noever, Škorić and Steger call "only slightly better than the trivial upper bound of $O(k)$" of a single depth-first traversal (Disser et al. 2020) (Fraigniaud et al. 2006a); without any information sharing, a factor k is unavoidable (Fraigniaud et al. 2006a). Cosson, Massoulié and Viennot frame the problem as "a fundamental problem behind the exploration of an unknown physical environment with a team of agents" and add: "Despite almost two decades of research, we still do not completely understand how much exploration time can decrease with the number of agents" (Cosson et al. 2023).

The lower bound side was opened by Dynia, Łopuszański and Schindelhauer, who showed $\Omega(\log k/\log\log k)$ for $k < \sqrt{n}$ (as reported by Disser et al. 2020); Disser et al. extended it to $k \le n\log^{\kappa} n$ for every constant κ (Disser et al. 2020). The upper bound moved from $O(k/\log k)$ to below $k/\log k$ (Cosson 2024), to $O(\sqrt{k})$ (Cosson and Massoulié 2024), and in a 2026 preprint to $O(\log^2 k)$ (Cosson and Massoulié 2026). These results assume labelled nodes and, except in the whiteboard variants, free communication, so the agents share a map of what has been explored.

Teams with little memory

A second line asks what a team can do with very little memory, and here agents act as markers for each other. Rollik's traps show that no finite team of finite automata explores all graphs (Rollik 1980) (as recounted by Disser et al. 2019). Disser, Hackfeld and Klimm observe that a team of $p + 1$ agents can simulate one agent with p pebbles, one agent walking while the others stand still, and that before their work no algorithm explored undirected graphs with sublogarithmic memory using several agents or pebbles (Disser et al. 2019). Their result closes the question:

> *"Since an additional agent is at least as powerful as a pebble, this implies that $O(\log\log n)$ agents with constant memory can explore any n-vertex graph." (Disser et al. 2019)*

They also show that $\Omega(\log \log n)$ agents are needed even when each has $O((\log n)^{1-\varepsilon})$ bits (Disser et al. 2019). An agent that stays still is a marker; dispersion is built on the same idea.

Dispersion as exploration with k = n

Augustine and Moses Jr. introduced dispersion in 2018: k agents on an n-node graph must reach a configuration with at most one agent per node. Their motivation is self-driving electric cars that look for free charging stations instead of queueing, and they place the problem between scattering, exploration and load balancing from the first page (Augustine and Moses Jr. 2018):

> *"Since any solution to a problem of dispersion acts as a solution to n robot exploration under the same conditions, any possibility results achieved for dispersion apply to n robot exploration." (Augustine and Moses Jr. 2018)*

Lower bounds move the other way, so the $\Omega(D^2)$ bound for n agents on trees (Disser et al. 2020) is a dispersion lower bound. The catalogue records that this bound is existential in D (the hard trees have height about $\sqrt{\log n}$), so whether rooted dispersion on taller trees needs $\Omega(D^2)$ rounds is open.

The assumptions are much weaker than in collaborative exploration. Sudo, Shibata, Nakamura, Kim and Masuzawa single out "the unique computation model": no node identifiers, no memory at the nodes, agents with identifiers that talk only when they share a node (Sudo et al. 2024). The algorithms recover the marker idea: in the depth-first dispersion of Kshemkalyani and Ali, "A robot that docks at a node helps other robots to determine whether they have visited this node before" (Kshemkalyani and Ali 2019). Every later improvement up to time-optimal $O(k)$ rounds with $O(\log(k + \Delta))$ bits, synchronous (Kshemkalyani et al. 2025a) and asynchronous (Pattanayak et al. 2025), is a faster way of using settled agents as markers on anonymous nodes.

What dispersion gives back

Because dispersion with $k = n$ explores, the dispersion results are also the best known results for n agents exploring an anonymous graph with only local communication: $O(n)$ rounds and $O(\log n)$ bits per agent from a rooted start (Kshemkalyani et al. 2025a), an inference the catalogue makes and the paper does not state. With labelled nodes and global communication, n agents explore a tree in D^2 rounds (Disser et al. 2020); whether anonymity and local communication force $\Omega(n)$ rounds on shallow trees is open, and Augustine and Moses Jr. already asked for $o(n)$-round dispersion on trees (Augustine and Moses Jr. 2018).

Dispersion also gave exploration-type questions new resources to measure. Gorain, Mandal and Mondal took up the question of Bouchard, Dieudonné and Pelc whether gathering needs communication (Bouchard et al. 2023b) and asked it for dispersion: their silent agents convey information only by the timing of moves and by counts of co-located agents (Gorain et al. 2022b). Molla and Moses Jr. replaced identifiers by coin flips (Molla and Moses Jr. 2019). Each such variant is, for $k = n$, a statement about collaborative exploration under the same assumptions.

Dispersion as a primitive for computing with agents

Kshemkalyani, Kumar, Molla and Sharma describe the agent-based model by its two differences from message passing: the nodes have no identifiers, computation or storage, and devices can exchange

Claude (Anthropic) was used in cataloguing the problems.

information only when co-located, so a message to a neighbour requires a move (Kshemkalyani et al. 2024). With n agents on n nodes the model corresponds to message passing with one device per node, and their simulation theorem converts any message-passing algorithm that runs in R rounds into an $O(\Delta R \log n)$-round agent algorithm from a dispersed start (plus $O(n \log^2 n)$ rounds otherwise), provided n and Δ are known (Kshemkalyani et al. 2024). Their stated goal is to remove that knowledge, and they note that limited storage "makes it impossible for the relocatable devices to first traverse the graph to learn the topology and then run graph computation as a second step" (Kshemkalyani et al. 2024).

Dispersion is the first step in most of what followed. Chand, Molla and Sivasubramaniam built dominating sets on it: "Our algorithms rely on the ideas used for dispersion", taking "advantage of the fact that most dispersion protocols involve the exploration of G" (Chand et al. 2023a). For triangle counting, Chand, Das and Molla state that a dispersed configuration "is a requirement" (Chand et al. 2026). Kaur, Gorain and Mondal reach maximal independent sets through distance-2 dispersion, which with n agents is the same problem, and elect a leader on the way (Kaur et al. 2026b). Chand and Molla build on time-optimal dispersion (Kshemkalyani et al. 2025a) to obtain, in $O(n)$ rounds, termination after dispersion, a spanning tree, a leader and a minimal dominating set (Chand and Molla 2026).

The progress is quick. Leader election without knowledge went from $O(m)$ rounds with $O(n \log n)$ bits (Kshemkalyani et al. 2024) to $O(n \log^2 n)$ rounds with $O(\log n)$ bits (Kshemkalyani et al. 2025b) and to $O(n)$ rounds (Chand and Molla 2026). Maximal independent sets (Pattanayak et al. 2024) (Kaur et al. 2026b), minimum spanning trees and BFS trees (Chand et al. 2025b), and triangle and butterfly counts (Chand et al. 2026) (Chand et al. 2025a) followed the same route.

Open questions on the boundary

1. **Height versus size.** Is $\Omega(D^2)$ the right bound for n agents on a tree of every height? Exploration theory proves it for one height (Disser et al. 2020); dispersion uses it for all.
2. **Settled agents versus pebbles.** A pebble is a mute marker; a settled agent stores and exchanges $O(\log n)$ bits. How much of the $\Theta(\log \log n)$ pebble bound (Disser et al. 2019) changes when markers can talk?
3. **Anonymity versus labels.** The collaborative-exploration bounds assume labelled nodes and global communication; dispersion assumes neither. Where does anonymity start to cost time?
4. **Lower bounds for agent-based problems.** No time lower bound specific to leader election, independent sets, dominating sets or spanning trees by agents is recorded in the catalogue; the only bounds come from dispersion itself. Whether the $O(n)$ rounds of (Chand and Molla 2026) are optimal for leader election, in particular from a dispersed start where no dispersion is needed, is open.

Further reading

The open questions above are recorded with their variants: the rooted-tree and tree variants of dispersion, and the *anonymous graphs with $k = n$ agents* variant of collaborative exploration.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 10

Meeting at one place

Rendezvous and gathering on graphs, gathering and convergence in the plane, and evacuation through an unknown exit all ask entities to end at one point. The papers give the same reasons for meeting, but each model makes a different part of it hard: symmetry and exploration on graphs, capability in the plane, information in continuous search.

Problems: Rendezvous, Gathering in graphs, Treasure hunt, Gathering, Convergence, Near-gathering, Evacuation, Linear search (cow path).

Why the papers want entities to meet

The reasons given are practical and much the same in every model. Robots exploring a terrain meet "to exchange samples of the ground previously collected by the robots, or exchange information obtained when exploring different parts of the terrain", and software agents meet periodically "to exchange collected data and plan further actions" (Pelc 2019). Bouchard, Dieudonné and Pelc add mobile robots "moving along corridors of a contaminated mine that is not accessible to humans", and give as reasons to gather samples or "to coordinate some future task, such as network maintenance or finding a map of the network" (Bouchard et al. 2023b). In the plane, gathering is called "one of the most fundamental tasks one can ask mobile robots to perform, e.g. to collect acquired data or to update the robots' algorithm" (Bramas et al. 2023b).

The second reason is that meeting is a building block. Gathering "turns out to be an essential prerequisite to achieve more complex cooperative works", so that results on gathering are results on everything built on it (Bouchard et al. 2022); Devismes, Dieudonné and Labourel use almost the same words (Devismes et al. 2026). Clement et al. place gathering among agreement problems: like scattering, it is "similar to the consensus problem in conventional distributed systems", with simple definitions but solvability that "greatly depends on the synchrony of the systems as well as the nature of the faults" (Clement et al. 2010).

On graphs: first break symmetry, then explore

Pelc separates rendezvous settings by whether agents can see something other than the environment, other agents or marks (Pelc 2019). On graphs they usually cannot, and two conventions fix the difficulty. Nodes are anonymous because "if nodes had distinct labels, then agents might explore the graph and meet in the smallest node, hence gathering would reduce to the well-studied task of exploration"; ports are numbered because without them "the adversary could prevent an agent from taking some edge" (Bouchard et al. 2023b). Identical agents may never meet: two of them starting together on a ring with consistent ports run forever in parallel (Ta-Shma and Zwick 2014). So the labels of the agents are the symmetry breaker, and the cost is polynomial in n and the label length (Dessmark et al. 2006) (Ta-Shma and Zwick 2014). In trees the cost is $\Theta(n + \log \lambda)$, where λ is the smaller label (Dessmark et al. 2006). Ta-Shma and Zwick also posed treasure hunt, rendezvous in which one agent never moves, as a tool for this question (Ta-Shma and Zwick 2014).

Without labels the question becomes which initial configurations can meet. Dieudonné and Pelc characterise the configurations of identical agents that can gather, and show that detecting that gathering is complete needs a bound on n (Dieudonné and Pelc 2016). For two identical asynchronous agents, meeting is feasible exactly when their views differ or their starting nodes are joined by a path whose port sequence is a palindrome (Guilbault and Pelc 2011), as summarised in (Pelc 2019). A middle ground is homonymous agents, motivated by cost and privacy: unique labels "can be costly", and agents "may deliberately blur their identity by revealing it only partially" (Devismes et al. 2026). With asynchronous agents the adversary controls speeds and agents may meet inside an edge; polynomial cost was reached only in (Dieudonné et al. 2015), after the ring case (De Marco et al. 2006).

In the plane: first ask whether it is possible

Robots in the plane see each other, so finding the others is not the issue; agreeing on a point is. Prencipe states the research question as "under which conditions this task can be accomplished by the robots?", and shows that without extra assumptions such as multiplicity detection gathering is in general impossible (Prencipe 2007). Suzuki and Yamashita had shown that two oblivious robots cannot meet in SSYNC (Suzuki and Yamashita 1999), and Flocchini et al. explain why two robots matter: rendezvous "separates FSynch from SSynch for oblivious robots", whereas $n \geq 3$ robots gather even in ASYNC with multiplicity detection (Flocchini et al. 2016) (Cieliebak et al. 2012). Izumi et al. note that, for semi-synchronous robots, two-robot gathering is the single pattern that memory makes formable (Izumi et al. 2012).

That made rendezvous the test case for every new capability. Unreliable compasses solve it below a threshold deviation (Izumi et al. 2012). A light with two colours solves it in ASYNC (Das et al. 2016) (Heriban et al. 2018), and three colours suffice when a robot sees only its own light (FSTA, in SSYNC) or only the other's (FCOM, even in ASYNC) (Flocchini et al. 2016), and three colours are also necessary (Otaka et al. 2026). The measure throughout is feasibility and the number of colours, not time.

Convergence: giving up the exact point

Relaxing the goal removes most of the difficulty. Izumi et al. remark in passing that "Two oblivious robots can converge to a point with a naive algorithm that consists of always moving toward the midpoint of their positions" (Izumi et al. 2012). Convergence of n robots was given by Suzuki and Yamashita (Suzuki and Yamashita 1999), with limited visibility by Ando et al. the same year (Ando et al. 1999), and the centre of gravity converges in ASYNC (Cohen and Peleg 2005).

With convergence and with limited visibility, time becomes measurable again, and the bounds are about locality. Under limited visibility in FSYNC, a class of local protocols needs and achieves $\Theta(\Delta_0^2)$ rounds, where Δ_0 is the initial diameter in units of the visibility range (Castenow et al. 2022); near-gathering, in which robots must come close without colliding (Pagli et al. 2015), takes $O(\Delta_0^2)$ epochs for disoriented robots in SSYNC, again tight within that class (Castenow et al. 2022). With one common axis, asynchronous gathering takes $\Theta(\Delta_0)$ epochs (Poudel and Sharma 2021).

In continuous search: meeting at a point nobody chose

Evacuation fixes the meeting point in advance but hides it. Bampas et al. set the three measures side by side: in search the cost is the arrival of the first searcher at a target placed by an adversary; in rendezvous it is the arrival of the last robot, and "the meeting point is arbitrary, i.e., the robots may choose the most convenient one"; in their linear search problem "a pair of robots has to meet at an unknown fixed target

Claude (Anthropic) was used in cataloguing the problems.

point of the environment" and the cost is the arrival of the second (Bampas et al. 2019). Evacuation is this third problem.

Czyzowicz et al., posing evacuation from a disk, describe the tension: robots should spread to find the exit fast but not so far that the walk to it becomes long (Czyzowicz et al. 2014). Communication decides how much the meeting costs. Two wireless robots evacuate in exactly $1 + 2\pi/3 + \sqrt{3} \approx 4.826$ (Czyzowicz et al. 2014); with face-to-face communication the best algorithm takes about 5.6234 (Disser and Schmitt 2019) against a lower bound of about 5.255 (Czyzowicz et al. 2020). On the line, face-to-face robots that must all reach the target cannot beat the single-robot ratio 9 (Bampas et al. 2019) (Chrobak et al. 2015).

One task, three difficulties

Read across the models, meeting is hard for three different reasons. On graphs agents cannot see each other, so meeting needs exploration and symmetry breaking, and the open questions are about cost: labelled rendezvous is polynomial, but the exponent is far from the lower bound (see the rendezvous page). In the plane robots see each other but cannot remember or agree, so the open questions are about which capabilities suffice and how many colours they need. In continuous search robots know the domain and each other's plans but not the target, so the open questions are gaps between constants.

Faults sharpen each version differently: f Byzantine agents on graphs require $f + 1$ good agents, or $f + 2$ when the number n of nodes is unknown (Bouchard et al. 2016); one Byzantine robot among three already prevents gathering in the plane in SSYNC (Agmon and Peleg 2006); and a faulty robot in a disk raises the evacuation time (Czyzowicz et al. 2017a). The perspective *When agents fail* follows that thread.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 11

Weak robots in the plane

Why the theory of robots in the plane starts from oblivious, anonymous, disoriented points; how formation problems became its testbed and the scheduler its adversary; and how a light with a few colours turned into memory, communication and a measure.

Problems: Gathering, Pattern formation, Leader election in the plane, Circle formation, Uniform circle formation, Mutual visibility, Pattern sequences, Universal pattern formation.

Why minimal robots

Since the work of Suzuki and Yamashita (Suzuki and Yamashita 1999), the theory of robots in the plane has asked which *minimal* assumptions let a team solve a task, rather than how a capable robot should solve it (Flocchini et al. 2023). A robot is a point that repeats Look-Compute-Move cycles; it is anonymous, runs the same algorithm as everyone else, has its own coordinate system, and in the standard model OBLOT it is also oblivious and silent (Flocchini et al. 2019). Dieudonné, Petit and Villain state the motive plainly: "The motivation behind such a weak and unrealistic model is the study of the minimal level of ability the robots are required to have in the accomplishment of some basic cooperative tasks in a deterministic way" (Dieudonné et al. 2008).

The introductions also give practical reasons. Many simple robots are cheaper and easier to extend than one strong robot (Bandettini et al. 2011); silent robots can be deployed where communication is impossible or jammed, under water or in a military setting (Di Luna et al. 2014); and robots that do not know n make the system "scalable and open to robot insertions or removals" (Feletti et al. 2023a). Flocchini, Prencipe, Santoro and Viglietta summarise the resulting research programme: the literature has "almost exclusively focused on very simple entities operating in strong adversarial conditions", robots that are anonymous, autonomous, oblivious, disoriented and non-rigid (Flocchini et al. 2017).

Formation problems as the testbed

Almost all computability results in this model concern geometric pattern formation: the robots must reach, and stay in, a configuration similar to a given pattern (Flocchini et al. 2017). As Flocchini et al. recount, Suzuki and Yamashita observed that whether a pattern can be formed depends on the *symmetricity* of the pattern and of the start, so the only patterns that might be formable from every start are a point and a regular polygon (Flocchini et al. 2017). These two became the reference problems. A point is gathering: without multiplicity detection and without extra assumptions, oblivious robots cannot gather (Prencipe 2007), while with weak multiplicity detection $n \geq 3$ asynchronous robots gather from any start (Cieliebak et al. 2012). The regular polygon is uniform circle formation. Défago and Konagaya conjectured that the last step, spreading robots evenly on a circle, has no deterministic finite-time solution; Dieudonné, Petit and Villain proved the conjecture and still formed the circle in SSYNC by another route, for $n \notin \{4, 6, 8\}$ (Dieudonné et al. 2008). In ASYNC the question stayed open until Flocchini, Prencipe, Santoro and Viglietta solved it from every start of distinct points (Flocchini et al. 2017), with the square ($n = 4$) handled in (Mamino and Viglietta 2016); they conclude that for

pattern formation "asynchrony is not a computational handicap" and that chirality and rigidity are "computationally irrelevant" (Flocchini et al. 2017).

Arbitrary patterns connect formation to symmetry breaking. Flocchini, Prencipe, Santoro and Widmayer posed arbitrary pattern formation for asynchronous oblivious robots (Flocchini et al. 2008a), and showed, as Dieudonné, Petit and Villain quote, that forming arbitrary patterns implies leader election for $n \geq 3$ (Dieudonné et al. 2010). The converse holds with chirality for $n \geq 4$ (Dieudonné et al. 2010) and, after a careful re-examination of earlier proofs, without it (Cicerone et al. 2019); which configurations admit a leader is characterised by Dieudonné, Levé, Petit and Villain (Dieudonné et al. 2013). Later formation problems change one physical assumption at a time. Mutual visibility drops the assumption, shared by all earlier models, "that visibility is unobstructed" (Di Luna et al. 2014): opaque robots must reach a position in which no three are collinear (Di Luna et al. 2017). Pattern sequences ask the robots to form patterns one after another (Das et al. 2015).

The scheduler as the adversary

Because robots do nothing but look and move, the main adversary is time. In FSYNC all robots act in every round; in SSYNC a fair adversary picks which robots act in each round; in ASYNC each phase takes a finite but unpredictable time (Das et al. 2016). Asynchrony hurts because robots "can be seen while moving" and compute "on the base of obsolete information" (Cicerone et al. 2019); Flocchini, Santoro, Sudo and Wada name as "one of the strongest debilitating effects of unrestricted asynchrony" that "a robot, when looking, cannot detect if another robot is still or moving" (Flocchini et al. 2023). The adversary also chooses how far a robot moves, subject to a minimum distance δ (non-rigid moves) (Flocchini et al. 2017).

Rendezvous is the separating example: two oblivious robots meet trivially in FSYNC but cannot meet in SSYNC (Flocchini et al. 2016), while $n \geq 3$ robots with multiplicity detection gather even in ASYNC (Flocchini et al. 2016) (Cieliebak et al. 2012). Between SSYNC and ASYNC lie finer schedulers, which make some of the phases atomic (LC-atomic, CM-atomic, M-atomic ASYNC), and below SSYNC lie restricted ones such as RSYNC and the sequential schedulers (Flocchini et al. 2023). Weakening the adversary can be decisive: under any sequential scheduler oblivious robots form every pattern from every start, except a point, which needs weak multiplicity detection (Flocchini et al. 2026).

Lights: a little memory and a little communication

The luminous model, which Das et al. attribute to a suggestion of Peleg, gives each robot a light with a constant number of colours that persists between cycles and is seen by all robots; a robot can therefore both remember and communicate a constant number of bits (Das et al. 2016). Das et al. asked what additional power "would allow the robots to overcome the inherent difficulties and weaknesses of the ASYNC setting". Their answer: with 5 colours ASYNC robots simulate any SSYNC algorithm, two robots with 4 colours meet in ASYNC, and with lights the difference between SSYNC and ASYNC disappears (Das et al. 2016).

Lights then split into two capabilities. Flocchini, Santoro and Wada ask directly, "is it better to remember or to communicate?" (Flocchini et al. 2019). In FSTA the light is seen only by its owner (finite state, silent); in FCOM only by the others (finite communication, oblivious) (Flocchini et al. 2019). Flocchini, Santoro, Viglietta and Yamashita had already shown that rendezvous does not need both: either kind of constant memory suffices under suitable conditions (Flocchini et al. 2016). Under FSYNC, LUMI and FCOM are equivalent and stronger than FSTA, which is stronger than OBLOT; under SSYNC, FCOM and FSTA are incomparable (Flocchini et al. 2019). Under asynchrony, SSYNC is strictly stronger than

Claude (Anthropic) was used in cataloguing the problems.

ASYNC for OBLOT, FCOM and FSTA, and only LUMI removes the difference (Flocchini et al. 2023). These separations mostly assume chirality and rigid moves. For two robots, LUMI, FCOM and FSTA coincide under FSYNC (Kitamura et al. 2025), a preprint.

Colours as a measure

Once lights were known to help, the number of colours became a resource. For two robots with full lights, the count fell from 4 (Das et al. 2016) to 3, optimal for algorithms that look only at colours (Viglietta 2013), and to 2, which is optimal, in ASYNC with non-rigid moves and arbitrary initial colours (Heriban et al. 2018). With external lights (FCOM), 3 colours suffice in SSYNC (Flocchini et al. 2016) and 2 do not (Otaka et al. 2026); with internal lights (FSTA), 3 colours suffice in rigid SSYNC (Flocchini et al. 2016) (Masuzawa et al. 2026). For $n \geq 3$ robots without multiplicity detection, full lights gather with 2 colours in SSYNC (Terai et al. 2023) and with 3 in ASYNC (Nakai et al. 2021). Opaque robots reach mutual visibility with 2 colours in rigid SSYNC, which is optimal (Di Luna et al. 2017).

Colours are then traded against time. For uniform circle formation by opaque luminous robots in ASYNC, the time fell from $O(n)$ epochs (Feletti et al. 2023b) to $O(\log n)$ (Feletti et al. 2023a), a time-colour trade-off was reported (Pattanayak and Sharma 2024) (abstract only), and $O(1)$ epochs with $O(1)$ colours is achievable, which is optimal in both (Feletti et al. 2024). Mutual visibility also takes $O(1)$ epochs with a constant number of colours in ASYNC (Sharma et al. 2021). With lights and a sequential scheduler, robots can form any sequence of patterns up to a length fixed by the number of colours (Feletti et al. 2026).

Open questions

Whether two robots with internal lights (FSTA) can meet in ASYNC at all is open, and in rigid ASYNC the best count for external lights is 6 colours against a lower bound of 3 (Masuzawa et al. 2026) (Otaka et al. 2026). Whether 2 colours suffice for gathering $n \geq 3$ robots in ASYNC is open (Nakai et al. 2021). For uniform circle formation, the exact number of colours for $O(1)$ time, and whether oblivious robots can do it in $O(1)$ epochs, are open (Feletti et al. 2024). Das et al. left open whether ASYNC with lights is stronger than, or incomparable with, FSYNC without lights (Das et al. 2016).

Claude (Anthropic) was used in cataloguing the problems.

Chapter 12

From robots to particles

Programmable matter re-asks the questions of robots in the plane on a lattice, with particles that touch, remember a little and stay connected; SILBOT takes memory and messages away again, and the stochastic approach replaces algorithms by a biased local rule.

Problems: Shape formation in programmable matter, Leader election in programmable matter, Coating, Scattering of particles, Compression, Separation of coloured particles, Alignment, Bridging, Pattern formation, Leader election in the plane.

Programmable matter and the amoebot model

Programmable matter, a term that Di Luna et al. trace to Toffoli and Margolus, is matter that can change its physical properties "in a programmable fashion, based upon user input or autonomous sensing", usually imagined as many small, locally interacting computational particles; the intended uses include smart materials, autonomous monitoring and repair, and minimally invasive surgery (Di Luna et al. 2020c). Navarra and Piselli add that ideas that were "mostly theoretical" in the early 1990s have become realistic as nano-technology advanced (Navarra and Piselli 2023b).

The amoebot model, introduced at SPAA 2014 (Derakhshandeh et al. 2014), is "motivated by micro- and nano-scale robotic systems with strictly limited computational and locomotive capabilities" (Daymude et al. 2023). Particles occupy nodes of the triangular lattice, have constant memory, and move by expanding into an empty neighbouring node and contracting again, like an amoeba (Emek et al. 2019). The system must stay connected, "to prevent the particles from drifting apart in an uncontrolled manner like in fluids and because in our case particles communicate only via bonds" (Derakhshandeh et al. 2015). As results accumulated, the model's assumptions diverged; Daymude, Richa and Scheideler proposed the canonical amoebot model to separate core features from assumption variants and to treat concurrency explicitly, since much earlier work "implicitly assumes amoebot actions are isolated and reliable" (Daymude et al. 2023).

Shape formation is pattern formation on a lattice

Di Luna, Flocchini, Santoro, Viglietta and Yamauchi say it directly: shape formation is the problem "called pattern formation in swarm robotics", and it has been "intensively studied" for autonomous mobile robots (Di Luna et al. 2020c). Derakhshandeh et al. posed shape formation together with leader election, because "most shape formation algorithms depend on some seed element", and asked whether shape formation can be solved when leader election cannot (Derakhshandeh et al. 2015). With chirality, a sequential scheduler and randomization, any shape made of a constant number of triangles can be formed from a well-initialised start (Derakhshandeh et al. 2016).

The deterministic answer echoes Suzuki and Yamashita. Di Luna et al. characterise the shapes that can be formed from any simply connected start: as in the plane, feasibility depends on the symmetry of the start; chirality turns out to be "computationally irrelevant"; randomization lets any shape be formed

once there are enough particles; and $O(n^2)$ moves suffice for n particles, which is optimal (Di Luna et al. 2020c). The comparison with the plane is the one in (Flocchini et al. 2017): there the symmetricity of the start decides which patterns are formable.

Leader election with touching neighbours

Leader election is the primitive behind most amoebot algorithms: a unique leader "breaks symmetry and coordinates the remaining amoebots", and Emek et al. set the goal of an algorithm that "requires as few assumptions as possible" (Emek et al. 2019). Dufoulon, Kutten and Moses recount the history: early deterministic algorithms assumed that the system has no holes, or elected up to six leaders; using movement removed the restriction on the shape but cost at least quadratic time (Dufoulon et al. 2021). Emek et al. elect a unique leader deterministically without chirality, and the particles also agree on a chirality (Emek et al. 2019); with chirality and a strong scheduler, Dufoulon et al. do it in linear time, as fast as the randomized algorithms (Dufoulon et al. 2021) (Daymude et al. 2017).

The contrast with robots in the plane is instructive. Robots that only see can elect a leader only from configurations that are not symmetric (Dieudonné et al. 2013). Particles have constant memory, read their neighbours and stay connected, and these powers break symmetries that positions alone cannot. Particles that do not move form a separate line of work: with chirality on hole-free systems (Gastineau et al. 2019), with holes but up to six leaders (Bazzi and Briones 2019), under an asynchronous adversary (Briones et al. 2023), and self-stabilising (Chalopin et al. 2025).

SILBOT: particles that behave like oblivious robots

SILBOT takes the extra powers away. Navarra and Piselli describe it as "one of the weakest models" for programmable matter: it "includes concurrency and eliminates direct communication among particles as well as local and shared memory", and in exchange particles see two hops instead of one (Navarra and Piselli 2023b). Its aim, introduced by D'Angelo et al. (D'Angelo et al. 2020), is the minimal setting in which basic tasks remain solvable, which is the question Suzuki and Yamashita asked of robots.

The limits of oblivious robots return. In SILBOT up to three leaders are elected on hole-free configurations (D'Angelo et al. 2020). A line cannot be formed with chirality alone; with a full common orientation it takes $O(n^2)$ moves for n particles, which is optimal (Navarra and Piselli 2023b). With chirality, particles coat a convex object with one layer in $O(n^2)$ rounds (Navarra and Piselli 2023a), and particles can scatter so that no two are neighbours (Navarra et al. 2025), a lattice counterpart of scattering in the plane (Dieudonné and Petit 2009).

Bias instead of algorithms

A second approach drops coordination altogether. Cannon, Daymude, Randall and Richa define an energy function whose minima are the desired configurations, design a Markov chain whose stationary distribution favours them, and make each transition probability computable from a particle's neighbourhood, so that the chain becomes a local, asynchronous algorithm; being a Markov chain, it is "inherently robust and oblivious" (Cannon et al. 2016). The motivation is natural systems such as fire ants forming rafts, where each individual "can take cues from its immediate neighbors" without seeing the group (Cannon et al. 2016). For compression, a bias $\lambda > 2 + \sqrt{2}$ towards having more neighbours makes the system compress, while $\lambda < 2.17$ makes it expand (Cannon et al. 2016).

Claude (Anthropic) was used in cataloguing the problems.

The same recipe was extended one ingredient at a time. With two colours and a second bias, one algorithm separates or integrates the colour classes, a behaviour seen in bacteria, social insects and human groups (Cannon et al. 2019). With orientations, particles align or do not for every number $q \geq 2$ of orientations, a variant of the Potts model (Kedia et al. 2022). With terrain, particles build shortcut bridges across a gap, inspired by army ants that balance "the benefit of increased foraging trail efficiency" against the cost of the workers in the bridge (Arroyo et al. 2018). For fire-ant bridges, Oh et al. show that a single bridge forms for large biases, and argue that this "follows from a statistical inevitability, and not from the sophistication of the individuals" (Oh et al. 2024).

Open questions

The stochastic results describe the stationary distribution, not when it is reached: no catalogued paper bounds the mixing time, and for compression the behaviour for $2.17 \leq \lambda \leq 2 + \sqrt{2}$ is not determined (Cannon et al. 2016). For SILBOT, Navarra and Piselli ask whether one common direction suffices for line formation, whether 1-hop visibility suffices, and which other shapes can be formed (Navarra and Piselli 2023b); coating without chirality is open (Navarra and Piselli 2023a). For leader election, linear-time deterministic election without chirality, and under schedulers weaker than the strong one, are open (Dufoulon et al. 2021).

Claude (Anthropic) was used in cataloguing the problems.

Chapter 13

From the cow path to evacuation

The line of work from Bellman's and Beck's linear search problem to group search with faulty robots, evacuation of disks and polygons, priority evacuation, and patrolling by robots with distinct speeds.

Problems: Linear search (cow path), Evacuation, Patrolling.

Linear search and the cow path

Kupavskii and Welzl open their paper with the original question, posed by Bellman in 1963 and studied by Beck, in Beck's words: "A man in an automobile searches for another man who is located at some point of a certain road. [...] How does he search so as to minimize the expected distance traveled?" (Kupavskii and Welzl 2018) (Beck 1964). In that version the searcher knows the distribution of the target. Beck and Newman showed, as reported by (Kupavskii and Welzl 2018), that without knowledge of the distribution the distance cannot be smaller than 9 times the expected distance to the target.

Computer scientists rediscovered the problem as the cow-path problem: a cow at an infinite fence must find the gate, and optimality is the worst-case ratio between the distance travelled and the distance to the gate (Kupavskii and Welzl 2018). Doubling the turning point gives ratio 9 on the line, which is optimal; on m rays, turning distances that grow by the factor $m/(m-1)$ give the optimal ratio $1 + 2m^m/(m-1)^{m-1}$ (Baeza-Yates et al. 1993); randomization lowers the line to about 4.59 (Kao et al. 1996). Searching, Czyzowicz et al. note, "is central to many areas of computer science including data structures, computational geometry, and artificial intelligence" (Czyzowicz et al. 2021a).

Groups, faults and speeds

Czyzowicz et al. proposed parallel versions of the cow path in which k robots search the line and up to f of them are faulty (Kupavskii and Welzl 2018). A crash-faulty robot passes the target without detecting it (Czyzowicz et al. 2016b); a Byzantine robot may also stay silent or announce a target that is not there (Czyzowicz et al. 2021a). For crash faults the optimal ratio is known exactly: proportional schedules (Czyzowicz et al. 2016b) are optimal, as Kupavskii and Welzl proved (Kupavskii and Welzl 2018). For Byzantine faults it is not; already three robots with one liar are open (Czyzowicz et al. 2021a) (Kupavskii and Welzl 2018).

The measure also changed. Bampas et al. compare search, where cost is "the time when the first searcher arrives", with rendezvous, where it is the time when "the last involved robot reaches the meeting point", and study two robots with different speeds that must both reach the target (Bampas et al. 2019). They find optimal strategies for every speed ratio with communication by meeting, and show that wireless communication helps only up to a certain speed ratio (Bampas et al. 2019).

Evacuation: waiting for the last robot

Czyzowicz et al. posed evacuation: k robots start at the centre of a unit disk and must all leave through an exit at an unknown point of the boundary; the cost is the worst-case time until the last robot is out (Czyzowicz et al. 2014). They compare it with the lost-at-sea and cow-path problems and with plane search, and cite Chrobak et al. for the observation that evacuation of uniform agents on a line "is as hard as the cow-path problem" (Czyzowicz et al. 2014) (Chrobak et al. 2015). Two communication models were introduced: wireless, in which a robot that finds the exit tells everyone at once, and face-to-face (local), in which robots exchange information only when they meet (Czyzowicz et al. 2014).

For two wireless robots the answer is exactly $1 + 2\pi/3 + \sqrt{3} \approx 4.826$ (Czyzowicz et al. 2014). Face-to-face, the robots must decide when to look for each other. Czyzowicz et al. obtained a lower bound of $3 + \pi/6 + \sqrt{3} \approx 5.255$ by studying an exit at a vertex of a hexagon, and introduced a "nonintuitive" strategy in which searchers "are forced to detour and meet" (Czyzowicz et al. 2020). Brandt et al. showed that such meetings need not exchange useful information (Brandt et al. 2017), and Disser and Schmitt brought the upper bound to about 5.6234 (Disser and Schmitt 2019); the gap remains open.

What evacuation varies

Each variation changes one ingredient of the original problem. Communication: a sender and a receiver evacuate the disk strictly between the wireless and face-to-face optima (Georgiou et al. 2022), and on the line exactly in ratio $3 + 2\sqrt{2}$ (Czyzowicz et al. 2021b). Faults: with one crash-faulty or Byzantine robot among three, the disk bounds are not tight (Czyzowicz et al. 2017a); on the line, evacuation with $k = 2f + 1$ robots is a sub-problem of Byzantine search, since all robots may have to reach the target (Czyzowicz et al. 2021c). Domain: equilateral triangles and squares (Chuangpishit et al. 2020b) (Georgiou and Jang 2026), m rays (Brandt et al. 2020), and disks with two exits (Pattanayak et al. 2018). Speeds and energy: two robots with different speeds (Lamprou et al. 2020), and the energy spent searching, which Czyzowicz et al. consider "for the first time" (Czyzowicz et al. 2019a).

Priority evacuation

Czyzowicz et al. observe that searchers are usually assumed to have the same capabilities and the same role, yet in practice "only one of the robots has the capability to address an urgent need at the target", such as performing an emergency procedure or closing a breach (Czyzowicz et al. 2018b). Priority evacuation makes this the measure: σ servants help a distinguished robot, the queen, and only the queen's evacuation time counts (Czyzowicz et al. 2018b). For σ servants both bounds tend to 2, but they differ in the $1/\sigma$ term (Czyzowicz et al. 2018b); for one and two servants, the problem was posed in (Czyzowicz et al. 2018a), and the lower bound for one servant was later raised through weighted group search (Georgiou and Wang 2025), leaving a gap between about 4.568 and 4.819.

Patrolling

Patrolling keeps the geometry and the robots with given maximum speeds but replaces a one-time target by a continuing duty: every point must be visited often enough, and the cost is the idle time, the longest time a point is left unvisited (Kawamura and Kobayashi 2015). Kawamura and Kobayashi note that patrolling had been studied mostly through heuristics and experiments, and that theoretical studies "have revealed that there are interesting questions and intricacies even in the simplest settings" (Kawamura and Kobayashi 2015). Czyzowicz et al. studied fences patrolled by agents with distinct

Claude (Anthropic) was used in cataloguing the problems.

speeds (Czyzowicz et al. 2011) and, as Kawamura and Kobayashi report, conjectured that partitioning the fence in proportion to the speeds is always optimal; Kawamura and Kobayashi confirmed this for three agents and disproved it in general (Kawamura and Kobayashi 2015), and Kawamura and Soejima found schedules that patrol a fence $4/3$ times longer than partition and a circle $21/20$ times better than the simple strategies (Kawamura and Soejima 2020). Faults enter as in search: with up to f unreliable robots, the optimal idleness of an Eulerian graph is $(f+1)|E|/k$ (Czyzowicz et al. 2017b). How far an optimal fence schedule can beat partition remains open (Kawamura and Soejima 2020).

Claude (Anthropic) was used in cataloguing the problems.

Chapter 14

When agents fail

A node that destroys visitors, a robot that stops, an agent that lies, a searcher that misses the target: the papers motivate each fault by the hardware or the network, and each kind of fault changes a different measure, from team size to feasibility to an exact constant.

Problems: Black hole search, Gathering in graphs, Gathering, Dispersion, Mutual visibility, Linear search (cow path), Evacuation.

Why the papers expect faults

The papers give two kinds of reasons. The first is hardware and environment. For robots in the plane, faults "can occur due to the unreliable components in the system, which are custom made or manually built", from manufacturing defects, or from "some external factor in the field of deployment" (Pattanayak et al. 2019b); since swarms "are envisioned for applications such as rescue, exploration, and surveillance in hazardous environments, it is natural that robots may experience faults" (Pramanick et al. 2025). The second is scale: "When the number of robots in a swarm increases, the possibility of having one robot crash unpredictably becomes important to tackle" (Bramas et al. 2023b), and on graphs "The scale-up when considering numerous agents is inevitably tied to the occurrence of faults among them" (Bouchard et al. 2022).

The fault models are borrowed from classical distributed computing. A crash fault stops an entity for ever; a Byzantine fault lets it behave arbitrarily, and "the Byzantine fault subsumes the crash fault" (Molla et al. 2020). On graphs a further distinction matters: a weakly Byzantine agent cannot fake its identifier, a strongly Byzantine one can (Molla et al. 2021). Bouzid, Das and Tixeuil import wait-freedom: "an arbitrary and unexpected delay observed at one robot may not prevent other robots from making progress toward the solution", even if $n-1$ of the n robots are delayed or crashed (Bouzid et al. 2013).

A faulty place: the black hole

Black hole search puts the fault in the network instead of the agents. Di Luna et al. motivate it by host attacks on mobile code: a black hole is a site "that disposes of visiting agents upon their arrival, leaving no observable trace of such a destruction", which can be a virus that trashes incoming messages or "the undetectable crash failure of a site" (Di Luna et al. 2025b). Kaur et al. add a node in a cloud that loses essential data (Kaur et al. 2026a). The task is to locate it with at least one agent surviving, so the primary measure is the team size: how many agents the adversary may destroy.

The answers depend on what agents know and how they communicate. Asynchronous agents with whiteboards and no map need $\Delta+1$ agents and $\Theta(n^2)$ moves; with a map two agents suffice and $\Theta(n \log n)$ moves (Dobrev et al. 2006). Tokens can replace whiteboards: $\Delta+2$ agents without a map (Balamohan et al. 2016), two with a map at the same cost (Flocchini et al. 2012b). When the adversary may also remove one edge per round of a ring, three agents are necessary and sufficient, with $\Theta(n^{1.5})$ moves

using pebbles (Di Luna et al. 2025b); in arbitrary dynamic graphs with one missing edge per round, six co-located agents with whiteboards suffice (Kaur et al. 2026a).

Crashes: an entity that stops

A crash is the mildest fault, and in the plane it is often harmless for feasibility. In SSYNC, the correct robots gather despite up to $n - 1$ crashes, wait-free, with only weak multiplicity detection and no agreement on coordinates (Pattanayak et al. 2019b); the first crash-tolerant algorithm, for one crash, is in (Agmon and Peleg 2006). The subtlety is what "gathering" means: in the weak version only correct robots must meet, in the strong version all robots must meet at the crash site; Bramas, Lamani and Tixeuil describe the strong one as one "where correct robots are requested to rescue failed ones", and show that two robots with lights cannot solve it in SSYNC (Bramas et al. 2023b). For mutual visibility with lights one crash fault is tolerated (Aljohani and Sharma 2018).

On graphs a crashed agent "loses all its information immediately, as if the robot has vanished from the network" (Chand et al. 2023b). For dispersion this costs time rather than feasibility: the best crash-tolerant algorithm for k agents runs in $O(\min(m, k\Delta))$ rounds on a graph with m edges and maximum degree Δ (Pattanayak et al. 2023) against the fault-free $O(k)$ (Kshemkalyani et al. 2025a), while on oriented grids crash-tolerant dispersion takes $O(\sqrt{n})$ rounds on n nodes, optimal when $k = \Omega(n)$ (Banerjee et al. 2025). In search, a crashed robot is silent: it visits the target without reporting it (Kupavskii and Welzl 2018). On the line the optimal ratio with f crash faults among k robots is known exactly (Czyzowicz et al. 2016b) (Kupavskii and Welzl 2018); with one crash among three wireless robots in a disk, evacuation takes between about 5.082 and 6.309 (Czyzowicz et al. 2017a).

Byzantine entities: an agent that lies

Byzantine faults are "considered as the worst faults that can occur", and a Byzantine agent "may also convey arbitrary information to the other agents, impersonate the identity of another agent, and so on" (Bouchard et al. 2022). In the plane one Byzantine robot among three already prevents gathering, and even convergence, in SSYNC, while in FSYNC $n \geq 3f + 1$ robots tolerate f Byzantine ones (Agmon and Peleg 2006); for mutual visibility with lights one Byzantine robot among three is likewise fatal (Aljohani and Sharma 2018). On graphs, gathering with f strongly Byzantine agents needs at least $f + 1$ good agents when the number n of nodes is known and $f + 2$ otherwise, as Dieudonné, Pelc and Peleg proved when they posed the problem (Dieudonné et al. 2014), and these team sizes suffice (Bouchard et al. 2016); the algorithms are exponential, and polynomial time is known only for larger teams (Bouchard et al. 2022).

Dispersion was taken to Byzantine agents because "in the real world, where systems must be fault tolerant and errors are the norm instead of the exception, more work must be done" (Molla et al. 2021). On rings it costs $\Theta(n)$ rounds under the assumptions of (Molla et al. 2020); on general graphs the best bound is $\tilde{O}(n^9)$ against $\Omega(n)$, because the algorithms go through Byzantine gathering (Molla et al. 2021); with strong Byzantine agents an arbitrary start becomes polynomial only with a trusted shepherd (Moses Jr. and Redlich 2024). Black hole search with weakly Byzantine companions that survive the black hole needs $2f + 2$ agents with a map (Di Luna et al. 2025a).

In search a Byzantine robot can claim a target that is not there, so "a reliable robot cannot necessarily trust an announcement that the target has been found" (Czyzowicz et al. 2021a). On the line the optimal ratio is known only for some densities f/k (Czyzowicz et al. 2021a); with three robots and one Byzantine, disk evacuation takes between about 5.948 and 6.921 (Czyzowicz et al. 2017a).

Claude (Anthropic) was used in cataloguing the problems.

Reading the fault axis

Each model in the atlas orders faults from most hostile (level 0) to none: strong Byzantine, weak Byzantine, crash, none for agents on graphs; Byzantine, crash, none for robots in the plane and for search. An algorithm proved against Byzantine entities therefore carries over to crashes, and an impossibility for crashes carries over to Byzantine entities. The black hole is not a value on this axis: it is part of the statement of black hole search, whose faults axis concerns the agents (the Byzantine-companion variants sit at weak Byzantine).

The measure that a fault changes differs by model, and that is the clearest lesson of the comparison. Losing agents in a black hole changes the team size. Crashes in the plane rarely change feasibility, but Byzantine robots often do. Crashes on graphs change time by a factor that is not yet understood, and Byzantine agents change it from linear to a high polynomial. In search every fault changes an exact constant, and the gaps between the constants are the open problems.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 15

What forces the bounds

Every lower bound in the atlas names the resource the adversary exploits. Read side by side, a few bottlenecks (distance, symmetry, missing information, an adversary that acts) explain most tight results, and the open entries cluster where no bottleneck beyond distance is known.

Problems: Dispersion, Collaborative exploration, Graph exploration by a single agent, Rendezvous, Treasure hunt, Black hole search, Gathering, Gathering in graphs, Linear search (cow path), Evacuation, Shape formation in programmable matter, Leader election in programmable matter.

Naming the bottleneck

Each lower bound in the atlas carries a `bottleneck`: two or three words for what the adversary uses. Across all problems the commonest names are *distance*, *symmetry breaking* and *symmetry*, then *memory* (colours of lights), *unknown exit*, *adversarial edge changes* and *unknown distance*; a long tail of names (*trap graphs*, *port addressing*, *indistinguishable Byzantine agents*, *timing is the only channel*) covers single arguments. This essay groups them into four kinds and ends with what is left open.

The grouping matters because results move along the assumption axes in opposite directions: an algorithm for a weak setting holds in every stronger one, a lower bound for a strong setting holds in every weaker one. A bound whose bottleneck survives every strengthening (distance, for instance) is inherited by a whole subtree of settings; a bound that depends on an assumption (asynchrony, an unknown exit) disappears as soon as that assumption is lifted.

Distance: something must travel

The plainest argument is that an entity must cover a distance. Kshemkalyani et al. state it for dispersion in one sentence: "Suppose two nodes of a graph are at a distance of $\Omega(k)$, then to disperse from one node to the furthest node takes $\Omega(k)$ time (e.g., line graph)" (Kshemkalyani et al. 2025a). The bound is from (Kshemkalyani and Ali 2019), and after several rounds of algorithms it is matched by $O(k)$ rounds, synchronous and asynchronous (Kshemkalyani et al. 2025a) (Pattanayak et al. 2025). On grids the distance is $\sqrt{k}$ (Kshemkalyani et al. 2020c). For collaborative exploration the offline cost $\Theta(n/k + D)$ is the distance part (Disser et al. 2020).

Programmable matter has the same argument, counted in moves: n particles reshaping a compact configuration into a line need $\Omega(n^2)$ moves in total (Derakhshandeh et al. 2015), as do deterministic shape formation (Di Luna et al. 2020c), universal coating (Derakhshandeh et al. 2017) and centralised transformations by sliding (Michail et al. 2019); universal shape formation needs $\Omega(\sqrt{n})$ rounds (Derakhshandeh et al. 2016). Distance bounds are usually the easy half of a tight result. What took years in dispersion was the algorithm, not the bound.

Symmetry and identity

The second kind of argument is about possibility rather than cost. Identical entities that the adversary keeps in symmetric positions cannot be separated by a deterministic rule: two oblivious robots in SSYNC (Suzuki and Yamashita 1999) (Flocchini et al. 2016), anonymous agents whose views coincide (Dieudonné and Pelc 2016), leader election among disoriented robots in symmetric positions (Dieudonné et al. 2013), arbitrary patterns in ASYNC without a common orientation (Flocchini et al. 2008a), and silent particles that share only chirality (Navarra and Piselli 2023b). Identical points that the robots cannot count produce the same effect: without multiplicity detection gathering is impossible (Prencipe 2007).

Even when identifiers break symmetry, carrying them costs memory. For dispersion of k agents the $\Omega(\log k)$ bits bound holds "since at least one agent needs $\Omega(\log k)$ bits to store its unique ID" (Kshemkalyani et al. 2025a) (Augustine and Moses Jr. 2018). Anonymous rendezvous in graphs of n nodes needs $\Theta(\log n)$ bits (Czyzowicz et al. 2012b), and labelled rendezvous on a ring with simultaneous start takes $\Theta(d \log \lambda)$ rounds, where d is the initial distance between the agents and λ the smaller label: the price of reading a label of $\log \lambda$ bits bit by bit (Dessmark et al. 2006). When agents cannot talk at all, time is the only channel: silent dispersion needs $\Omega(k + \log L)$ rounds (Das et al. 2025b) and $\Omega(\log L + \log \Delta)$ bits (Gorain et al. 2022b), with identifiers at most L and maximum degree Δ. In the plane the same economy appears as colours: two robots with lights need two (Viglietta 2013), with only internal or only external lights three (Otaka et al. 2026).

Information: not knowing where to go

Search problems are bounded by what the searcher does not know. Beck already showed that, without knowing the distribution of the target, the distance travelled cannot be below 9 times the expected distance (as reported by Kupavskii and Welzl 2018), and the cow-path ratio 9 is the competitive form of the same fact (Baeza-Yates et al. 1993); randomization lowers it to about 4.5911 (Kao et al. 1996). Kupavskii and Welzl's bound for m rays, specialised to robots without faults, settled the question of parallel search on m rays "posed by three groups of scientists some 15 to 30 years ago" (Kupavskii and Welzl 2018). An unknown exit gives every evacuation bound on the disk (Czyzowicz et al. 2014), and one-way communication gives the line its exact ratio $3 + 2\sqrt{2}$ (Czyzowicz et al. 2021b).

On graphs the missing information is which port to take. Remembering one port among Δ costs $\lceil \log \Delta \rceil$ bits for perpetual tree exploration (Diks et al. 2002) and $\Omega(\log \Delta)$ bits for dispersion by anonymous randomized agents (Molla and Moses Jr. 2019); treasure hunt with pebbles needs $\Omega(D \log \Delta)$ rounds (Gorain et al. 2022a). Stronger memory bounds come from trap graphs, built so that an agent with too little memory (or too few pebbles) misses part of them: $\Omega(D \log \Delta)$ bits for perpetual exploration (Fraigniaud et al. 2005a) and $\Theta(\log \log n)$ pebbles for a constant-memory agent (Disser et al. 2019). Fraigniaud et al. recall why these bounds were pursued: exploration by finite automata was tied to the complexity of undirected connectivity (Fraigniaud et al. 2006c).

Adversaries that act

Some bounds need an adversary that does more than choose the input. An adversary that removes edges forces three agents and $\Omega(n^{1.5})$ moves for black hole search in a dynamic ring (Di Luna et al. 2025b), makes dispersion with local communication impossible in 1-interval-connected graphs (Kshemkalyani et al. 2020a), and forces $\Omega(n^2)$ steps to explore some temporal graphs even offline, whereas static undirected graphs need fewer than $2n$ (Erlebach et al. 2021). An adversary that delays agents forces $\Theta(n^2)$ moves

Claude (Anthropic) was used in cataloguing the problems.

in black hole search without a map (Dobrev et al. 2006) and an $\Omega(\log^2 k)$ factor in asynchronous collaborative tree exploration (Cosson and Massoulié 2025); one that reveals a tree online forces a competitive ratio of $\Omega(\log k / \log\log k)$ for collaborative exploration (Disser et al. 2020).

Byzantine entities act from inside. Because they are indistinguishable from good ones, gathering on graphs with f Byzantine agents needs $f+1$ good agents (Bouchard et al. 2016), three robots with one Byzantine cannot gather in SSYNC (Agmon and Peleg 2006), dispersion of $k > n$ agents on n nodes into a balanced configuration is impossible whenever $\lceil k/n \rceil > \lceil (k-f)/n \rceil$, because Byzantine agents can occupy nodes exactly like honest ones (Molla et al. 2021), and the best lower bound for search on the line with Byzantine robots, about 5.23, comes from the argument for crash faults (Kupavskii and Welzl 2018). The black hole is the extreme case of an adversary inside the network: an asynchronous team without a map needs $\Delta + 1$ agents, Δ the maximum degree, and it cannot succeed at all if n is unknown or the graph is not 2-connected (Dobrev et al. 2006).

Where the gaps are

At the time of writing, the atlas judges optimality per measure for every variant, and more than two in five of these judgements are *unknown*: no non-trivial lower bound, or no analysis of that measure. Most of them come from two habits of the literature. In the plane, time is often not analysed because the question was feasibility. On graphs, rendezvous and exploration algorithms often leave memory unbounded. Both are open questions that nobody has asked yet rather than hard ones.

The polynomial gaps cluster where the only lower bound is distance. Dispersion on trees with n nodes has $O(n)$ for $k = n$ agents against an $\Omega(D^2)$ bound proved only for trees of height about $\sqrt{\log n}$ (Disser et al. 2020); crash-tolerant dispersion is a factor Δ (the maximum degree) from $\Omega(k)$ (Pattanayak et al. 2023); Byzantine dispersion is $\tilde{O}(n^9)$ against $\Omega(n)$ (Molla et al. 2021). Black hole search with tokens is $O(mn)$, for m edges, against $\Omega(n^2/\Delta)$ (Balamohan et al. 2016). Stopping tree exploration without knowing n uses $O(\log n)$ bits against an $\Omega(\log\log\log n)$ bound (Ambühl et al. 2011) (Diks et al. 2002), and leader election in programmable matter has no proven lower bound, only an informal $\Omega(D)$ remark, D being the diameter of the particle configuration (Dufoulon et al. 2021). Where the bottleneck is symmetry or missing information, results tend to be tight or exact; the gaps that remain in search are between two constants, as in two-robot face-to-face evacuation, about 5.255 against 5.6234 (Czyzowicz et al. 2020) (Disser and Schmitt 2019). The progress map of each problem shows the bottleneck next to every bound, so these patterns can be checked directly.

Claude (Anthropic) was used in cataloguing the problems.

Part III

Mobile agents on graphs

The model and its assumption axes are in Chapter 2.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 16

Exploration

Visit every node of an unknown graph: one agent with as little memory or as few markers as possible, or a team as fast as possible. Treasure hunt and rendezvous are subproblems of exploration; dispersion is derived from collaborative exploration.

Problems in this family:

- Graph exploration by a single agent
- Treasure hunt — subproblem of Graph exploration by a single agent
- Rendezvous — subproblem of Graph exploration by a single agent
- Gathering in graphs — generalisation of Rendezvous
- Map construction — strengthening of Graph exploration by a single agent
- Exploration by oblivious robots — variant of Graph exploration by a single agent
- Exploration by luminous robots — variant of Exploration by oblivious robots
- Exploration of recolourable graphs — variant of Graph exploration by a single agent
- Collaborative exploration
- Dispersion — derived from Collaborative exploration
- Black hole search — variant of Collaborative exploration
- Exploration of dynamic graphs by teams — variant of Collaborative exploration
- Network decontamination

16.1 Graph exploration by a single agent

One agent starts at a node of an unknown anonymous graph and must visit every node; how much memory does it need, which pebbles or knowledge make it possible, and how fast can it be?

First posed in 1951 (Shannon 1951). Shannon's maze-solving mouse Theseus (first prototype 1950) explored a 5×5 grid with constant memory per cell (Disser et al. 2019); the first impossibility theorem for finite automata is Budach's (Budach 1978).

Problem statement and measures

A single deterministic agent is placed at a node s of a connected graph G with n nodes, m edges and maximum degree Δ. Nodes are anonymous (they carry no names the agent can read), and the edges at each node are numbered locally; these numbers are the node's **ports**. The agent sees the degree of its

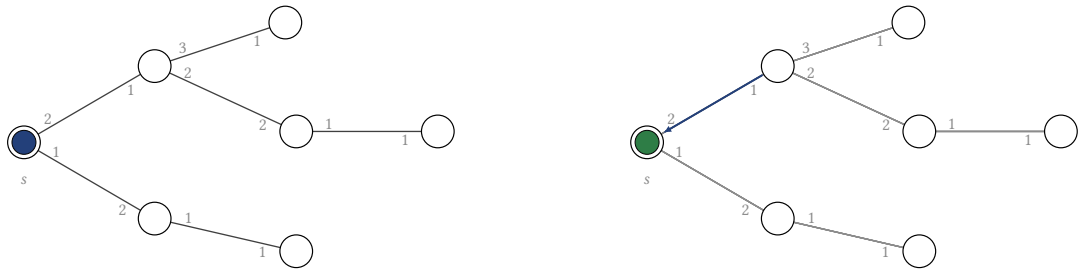
node and the port by which it entered. It must **visit every node of** G. Three termination requirements are standard:

- **perpetual exploration**: every node is eventually visited; the agent never has to stop;
- **exploration with stop**: every node is visited and the agent then stops, anywhere;
- **exploration with return**: every node is visited and the agent stops at s.

Return implies stop, and stop implies perpetual exploration. **Memory** is the number of bits of the agent's state; when the agent carries **pebbles** (objects it can drop on a node and pick up again) their number is reported with the memory. **Time** is the number of moves (edge traversals) until exploration is complete; for periodic exploration it is the period. Unless a variant says otherwise the agent knows nothing about G, not even n.

Exploration is the oldest problem of the mobile-agent literature: it started as maze search by finite automata, and the question of how little memory suffices runs from Budach's impossibility (1978) to Reingold's log-space algorithm (2005). Several agents exploring together form the sibling problem, collaborative exploration.

Measures: **Time**, edge traversals until every node is visited (for periodic exploration: the period); **Memory**, bits of agent memory; pebbles, if any, are counted separately.



(a) Start: one agent at s; ports 1..d at each node

(b) Final: back at s after 2(n-1) = 12 moves; all 7 nodes visited

Figure 16.1: Exploration of a tree by the rule Choose-Next of (Diks et al. 2002): the agent leaves the start node s by port 1 and, entering a node of degree d by port i, leaves by port $(i \bmod d) + 1$. It walks around the tree, each edge once in each direction, and is back at s after $2(n-1)$ moves with every node visited (perpetual exploration repeats this tour). One frame per move.

A first look

Imagine a small cleaning machine put down in a large building with no map, where every room looks the same. We model the building as a **graph**: rooms are **nodes** and corridors are **edges** (each edge joins two nodes), and the machine is an **agent** that walks from node to node along edges; one crossing of one edge is a **move**. n is the number of nodes and m the number of edges; the **degree** of a node is the number of edges at it, and Δ is the largest degree. At each node the edges are numbered $0, 1, \ldots$; these numbers are called **ports**, and the numbering at one node has nothing to do with the numbering at the next. The agent sees only the degree of its current node and the port by which it came in; nodes are **anonymous** (they have no names), so it cannot tell a node it has seen before from a new one. In the basic version (**perpetual exploration**) it must eventually visit every node of every connected graph, without ever having to stop, and the question is how few bits of **memory** (what the agent keeps from one move to the next) make this possible.

Claude (Anthropic) was used in cataloguing the problems.

1. Algorithm: Leave by the next port (trees). (Diks et al. 2002) (Trees, perpetual exploration, p. 83) This is the algorithm Choose-Next of (Diks et al. 2002), for graphs without cycles (trees); here the ports of a node of degree d are numbered $1, \dots, d$.

1. Leave the **start node** (the node where you are placed) by port 1.
2. When you enter a node of degree d by port i, leave it by port $(i \bmod d) + 1$, the next port in cyclic order (at a node of degree 1 this sends you straight back).
3. Repeat forever.

Why it works and what it costs. On a tree this is the rule "keep your right hand on the wall": the agent walks around the tree, and every edge is walked once in each direction before the walk repeats, so all n nodes are visited within $2(n-1)$ moves. The agent only has to remember one port number, which (Diks et al. 2002) count as $O(\log \Delta)$ bits, and they show that $\Omega(\log \Delta)$ bits are needed on trees, so this is optimal. On graphs with cycles the same rule can go round a cycle forever and miss nodes; there the right amount of memory is $\Theta(D \log \Delta)$ bits, with D the diameter (the largest distance between two nodes) (Fraigniaud et al. 2005a).

Why it matters. Choosing the next port from the entry port alone is the basic move of almost every memory-efficient exploration algorithm, and the universal exploration sequences used for general graphs are a generalisation of it.

2. Impossibility: No stopping on a ring without a pebble. (Dieudonné and Pelc 2012) (Exploration with stop, nothing known, p. 84) Ask for more: the agent must visit every node **and then stop**, without knowing n and without a **pebble** (an object it could drop on a node to recognise that node later). Take rings (cycles) of every size, numbered so that at every node port 0 leads clockwise and port 1 counterclockwise. In such a ring every node has degree 2 and every arrival looks the same, so what the agent sees never depends on the size of the ring. A deterministic agent (one with no random choices) therefore makes the same sequence of moves in every such ring. Since it explores and stops in, say, the ring of 3 nodes, it stops after some fixed number T of moves, and it does the same in the ring of $T + 2$ nodes.

Why it holds. In T moves the agent can visit at most $T + 1$ different nodes, so in the ring of $T + 2$ nodes at least one node is never visited: the agent stopped too early. Extra memory does not help, because the problem is not remembering but recognising: the agent can never tell "I am back where I started" from "this is a new node that looks the same". This is why every stopping variant in the catalogue adds a pebble or knowledge of a bound $N \geq n$ (Dieudonné and Pelc 2012).

Why it matters. It shows that the hard part of exploration is often knowing when you are done, a theme that returns in dispersion, rendezvous and gathering.

3. Algorithm: Random walk. (Aleliunas et al. 1979) (Randomized exploration when n is known, p. 87) Allow the agent to flip coins, and tell it a number $N \geq n$.

1. At each node, pick one of the ports uniformly at random and take its edge.
2. Count the moves; stop after $2N^3$ moves.

Why it works and what it costs. (Aleliunas et al. 1979) proved that a random walk visits every node of a connected graph within $2m(n-1)$ moves on average. Since $m \leq N^2/2$ and $n - 1 < N$, this average is below N^3; a walk that runs twice as long as its average cover time fails to cover the graph with probability at most $1/2$ (Markov's inequality), and repeating the walk lowers the failure probability further. The agent needs only a counter, $O(\log N)$ bits. A deterministic agent with the same memory also exists, but its algorithm (Reingold's log-space construction) is much harder (Reingold 2008).

Claude (Anthropic) was used in cataloguing the problems.

Why it matters. The random-walk bound was the starting point for universal traversal and exploration sequences, which turn the random walk into a fixed list of port choices that works on every graph.

What the community started looking at

Exploration began as **maze search by finite automata**. The first question was whether a machine with bounded memory can visit every node at all: Budach showed it cannot for mazes (Budach 1978), Blum and Kozen showed that two pebbles or two automata are enough in mazes (Blum and Kozen 1978), and Rollik showed that every finite team has a planar trap (Rollik 1980).

From there the community turned to **how much memory** an agent needs as a function of the graph: $\Theta(D \log \Delta)$ bits in general (Fraigniaud et al. 2005a), $O(\log n)$ once n is known (Reingold 2008), and the effect of the termination requirement on trees (Diks et al. 2002) (Ambühl et al. 2011). Markers came next: what one pebble buys (Fraigniaud et al. 2006c), and the exact exchange rate between pebbles and memory (Disser et al. 2019). Time was a secondary measure throughout.

Relations to other problems

- **Derived problem: Treasure hunt** (subproblem of). An agent that explores the graph finds any treasure, so every exploration algorithm solves treasure hunt; the question is how much cheaper it can be when the treasure is close, measured against its distance d (or a weight of the path to it) rather than the size of the graph.
- **Derived problem: Rendezvous** (subproblem of). If the agents can break symmetry (for example by their labels), one may wait while the other explores, so exploration solves rendezvous; the work is in doing better than that, or in meeting when neither knows who should wait (Pelc 2019).
- **Derived problem: Map construction** (strengthening of). Building an isomorphic map implies having visited every node, so map construction is exploration with more output; it is used to solve rendezvous and leader election (Das 2019).
- **Derived problem: Exploration by oblivious robots** (variant of). The robots follow the Look–Compute–Move model of the plane, moved to graphs: they see the positions of all robots but remember nothing, so the configuration itself must encode the progress of the exploration.
- **Derived problem: Exploration of recolourable graphs** (variant of). The same task for one agent (visit every node; with return, stop or perpetually), but every node holds a colour from a palette of c colours that the agent at the node reads and, unless the colours are fixed, overwrites; the agent itself may have no memory at all, and in the main model it has no port numbers and sees its neighbours' colours instead. With $c = 1$ the colours carry nothing: an algorithm of the parent problem that uses no markers works here whenever this setting grants its ports, entry port and memory, and a lower bound of this problem proved for any number of colours holds for the parent setting with the same view. The parent's pebbles are movable markers carried by the agent (Pattanayak and Pelc 2024a) (Disser et al. 2019); here the marks stay in the nodes and there is one per node.

Assumptions specific to this problem

Knowledge

Knowing the size of the graph is the classical way to let an exploring walk end. Aleliunas, Karp, Lipton, Lovász and Rackoff proved that a random walk on a connected graph visits every node within $2m(n-1)$

Claude (Anthropic) was used in cataloguing the problems.

expected steps from any start (Theorem 4), so by Markov's inequality $O(mn \log n) \subseteq O(n^3 \log n)$ steps cover it with high probability, and a step counter and the value of n give a randomized explorer that stops (Aleliunas et al. 1979); Reingold's log-space universal exploration sequences do the same deterministically when n is known (Reingold 2008) (as summarised in Disser et al. 2019). Without a bound and without a marker an agent cannot tell a cycle from a longer one, so it can explore perpetually but never stop (Fraigniaud et al. 2006c). Bender, Fernández, Ron, Sahai and Vadhan measured what the bound is worth in directed graphs: one pebble suffices when an upper bound on n is known, while without it $\Theta(\log \log n)$ pebbles are necessary and sufficient, the lower bound holding even for randomized robots (Bender et al. 2002). The top level, a full map, makes static exploration an offline planning task (a known undirected graph is explored by depth-first search in fewer than $2n$ steps); it matters for temporal graphs, where exploration with the whole schedule known in advance was introduced by Michail and Spirakis (Michail and Spirakis 2016) (as reported by Erlebach et al. 2021) and studied by Erlebach, Hoffmann and Kammer (Erlebach et al. 2021).

Level	Value	Formal definition
0	**nothing known**	The agent knows nothing about G: not n, m, Δ or D.
1	**knows a bound on n**	The agent is given an integer $N \geq n$ (an upper bound on the number of nodes) as input.
2	**knows the map**	The agent is given a graph isomorphic to G, with port numbers, and (where stated) its own position on it; for a dynamic graph, the whole sequence (G_t).

Markers

Pebbles come from the tradition of automata in mazes. After Budach proved that no finite automaton explores every finite maze (Budach 1978), Blum and Kozen showed that two pebbles suffice for a finite automaton in a maze, against Shah's conjecture that five were needed, and left open whether one pebble suffices (Blum and Kozen 1978); Hoffmann showed that it does not (as cited by Disser et al. 2019). Cohen, Fraigniaud, Ilcinkas, Korman and Peleg describe this "Rabin's pebble model" as the ability of the robot to label nodes dynamically, at most one unit of label per pebble (Cohen et al. 2008). Bender et al. carried the question to directed graphs and agents with unbounded memory, titling their paper "The power of a pebble": one pebble and a bound on n already give polynomial-time mapping (Bender et al. 2002). In anonymous graphs a marker is what makes stopping possible at all, and by convention a robot asked to stop is given a pebble (Fraigniaud et al. 2006c). The number of pebbles became a resource of its own when Disser, Hackfeld and Klimm showed that $\Theta(\log \log n)$ pebbles are necessary and sufficient for a constant-memory agent on undirected graphs, and that an agent with $O((\log n)^{1-\varepsilon})$ bits already needs $\Omega(\log \log n)$ of them (Disser et al. 2019); this is why one pebble and several pebbles are separate levels.

Level	Value	Formal definition
0	**no markers**	The agent cannot mark nodes.
1	**one pebble**	The agent carries one pebble that it can drop at its current node and later pick up; it sees whether a pebble is at its node.
2	**pebbles**	The agent carries p pebbles (identical or distinguishable, as stated); p is a resource reported with the memory.

Preprocessing

This axis asks what a designer who prepares the graph before the agent arrives can do for it, a question that grew out of the impossibility results of Budach and Rollik for finite automata (Budach 1978) (Rollik 1980). Cohen, Fraigniaud, Ilcinkas, Korman and Peleg let the designer write short labels on the nodes and showed that three label values let a constant-memory robot explore every graph and stop; by Rollik's result two values are the minimum one could hope for (Cohen et al. 2008). They place the

Claude (Anthropic) was used in cataloguing the problems.

model next to pebbles: it "can be considered stronger than Rabin's pebble model since labels are given in a preprocessing stage, but it can also be considered weaker since, once assigned to nodes, the labels cannot be modified", and they cite it as part of a line of work on short information supplied in advance, including the oracle-size measure of Fraigniaud, Ilcinkas and Pelc (Cohen et al. 2008) (Fraigniaud et al. 2006b). Choosing the port numbers is preprocessing that writes nothing on the nodes: Ilcinkas starts from Budach's result that no finite automaton explores all graphs when an adversary sets the local orientation, and asks for the shortest exploration period when the designer sets it, improving the period $10n$ of Dobrev et al. to $4n$ (Ilcinkas 2008). The two forms of preprocessing are kept on the same level because neither is known to simulate the other.

Level	Value	Formal definition
0	**adversarial ports**	Port numbers (and any node labels) are chosen by an adversary.
1	**designer ports**	Before the agent starts, the designer may choose the port numbering $\mathrm{port}_v : \{\text{edges at } v\} \to \{1, \dots, \deg(v)\}$ at every node. Ilcinkas calls a designer's port numbering a *local orientation* (Ilcinkas 2008).
1	**designer labels**	Before the agent starts, the designer may write a label of b bits (a constant) at every node, which the agent reads on arrival.

Termination requirement

The three termination requirements were separated in the memory-bounded exploration literature of the early 2000s. Fraigniaud, Ilcinkas, Peer, Pelc and Peleg list them explicitly, exploration with return, exploration with stop and perpetual exploration, and recall that Diks, Fraigniaud, Kranakis and Pelc showed for trees that exploration with stop needs $\Omega(\log \log \log n)$ bits of memory (Fraigniaud et al. 2005a) (Diks et al. 2002); the same paper shows that exploration with return needs $\Omega(\log n)$ bits (Diks et al. 2002) (as summarised in Disser et al. 2019 and Czyzowicz et al. 2012c), while perpetual exploration of bounded-degree trees is possible with $O(1)$ bits (Fraigniaud et al. 2006c). In arbitrary anonymous graphs the requirement also decides feasibility: a robot cannot recognise that it has closed a cycle, so without a marker or a bound on n it can explore perpetually but never stop (Fraigniaud et al. 2006c). The axis orders the requirements from the hardest (return) to the weakest (perpetual); periodic exploration, which asks every node to be visited infinitely often within a bounded period, is a perpetual task measured by its period (Ilcinkas 2008).

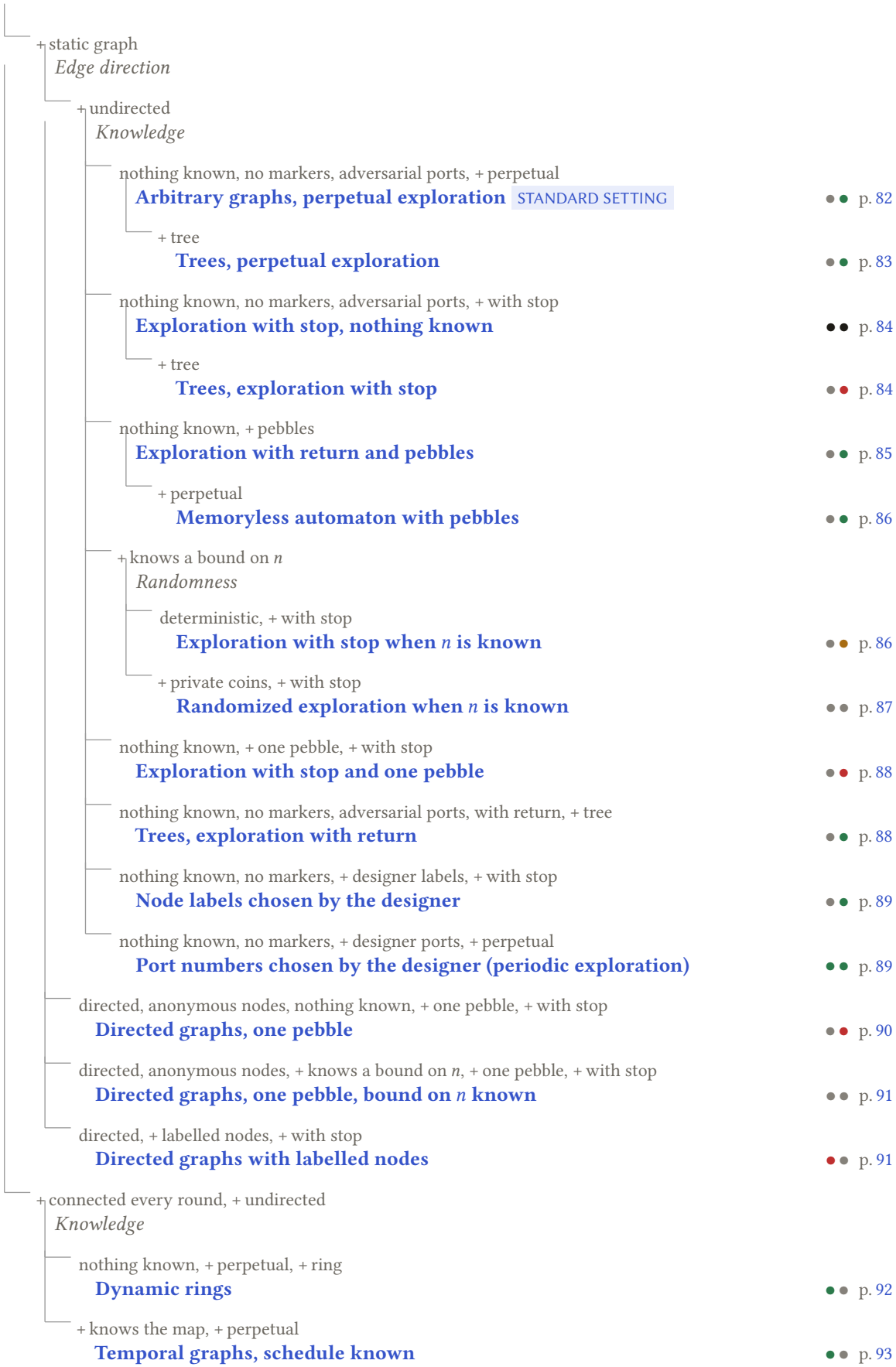
Level	Value	Formal definition
0	**with return**	The agent visits every node and then halts at its start node s.
1	**with stop**	The agent visits every node and then halts, at any node. Also called *terminating exploration*.
2	**perpetual**	Every node is visited at least once (and, for periodic exploration, infinitely often); the agent never needs to halt.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Memory) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Node identifiers → Knowledge → Randomness → Markers → Preprocessing → Termination requirement → Topology.

No assumptions

Claude (Anthropic) was used in cataloguing the problems.



Claude (Anthropic) was used in cataloguing the problems.

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Memory	p.
Arbitrary graphs, perpetual exploration STANDARD SETTING	● unknown — vs —	● tight $O(D \log \Delta)$ vs $\Omega(D \log \Delta)$	82
Trees, perpetual exploration	● unknown — vs —	● tight $O(\log \Delta)$ vs $\lceil \log \Delta \rceil$	83
Exploration with stop, nothing known	● impossible	● impossible	84
Trees, exploration with stop	● unknown — vs —	● polynomial gap $O(\log n)$ vs $\Omega(\log \log \log n)$	84
Exploration with return and pebbles	● unknown $n^{O(1)}$ vs —	● tight $O(1)$ bits, $O(\log \log n)$ pebbles vs $\Omega(\log \log n)$ pebbles	85
Memoryless automaton with pebbles	● unknown — vs —	● tight 0 bits, 2 pebbles vs 2 pebbles	86
Exploration with stop when n is known	● unknown $n^{O(1)}$ vs —	● near-optimal $O(\log n)$ vs $\omega((\log n)^{1-\varepsilon})$	86
Randomized exploration when n is known	● unknown $O(mn)$ vs —	● unknown $O(\log n)$ vs —	87
Exploration with stop and one pebble	● unknown — vs —	● polynomial gap $O(D \log \Delta)$ vs $\Omega(\log n)$	88
Trees, exploration with return	● unknown — vs —	● tight $O(\log n)$ vs $\Omega(\log n)$	88
Node labels chosen by the designer	● unknown $O(m)$ vs —	● tight $O(1)$ vs —	89
Port numbers chosen by the designer (periodic exploration)	● tight $3.5n - 2$ vs $2n - 2$	● tight $O(1)$ vs —	89
Directed graphs, one pebble	● unknown — vs —	● polynomial gap $O(n\Delta_{out} \log n)$ vs $\Omega(n \log \Delta_{out})$	90
Directed graphs, one pebble, bound on n known	● unknown $O(N^2 n^6 \Delta_{out}^2)$ vs —	● unknown — vs —	91
Directed graphs with labelled nodes	● polynomial gap $O(\phi^8 m)$ vs $\Omega(\phi^2 m)$	● unknown — vs —	91
Dynamic rings	● tight $n + \frac{n}{\max\{1,T-1\}}(\delta - 1) + O(\delta)$ vs $n + \frac{n}{\max\{1,T-1\}}(\delta - 1) - O(\delta)$	● unknown — vs —	92
Temporal graphs, schedule known	● tight $O(n^2)$ vs $\Omega(n^2)$	● unknown — vs —	93

16.1.1 Arbitrary graphs, perpetual exploration

Adds nothing known, no markers, adversarial ports, + perpetual STANDARD SETTING VERIFIED

The classical memory question: how many bits must a finite-state agent have to visit every node of every graph of diameter D and degree Δ?

Assumed: graph dynamics: static graph; edge direction: undirected; termination requirement: visit every node; never

Claude (Anthropic) was used in cataloguing the problems.

required to stop (perpetual). **Not assumed:** node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; markers: no markers; preprocessing: ports and nodes as given by an adversary; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known The paper does not study time ("time is not an issue in our approach").	—	● unknown
Memory	$\Omega(D \log \Delta)$ (Fraigniaud et al. 2005a) $\Theta(D \log \Delta)$ bits in the worst case over graphs of diameter D and degree Δ.	$O(D \log \Delta)$ (Fraigniaud et al. 2005a)	● tight

Results.

- Fraigniaud et al. (2005a) [lower bound]: memory $\Omega(D \log \Delta)$. *Bottleneck:* trap graphs. *Holds when:* deterministic agent, no knowledge of the graph; degree $\Delta \geq 3$; the traps are planar; Theorem 4 and Corollary 5: for every K-state agent there is a planar trap of degree Δ and diameter $O(\log K / \log \Delta)$. Corollary 3 gives $\lceil \log n \rceil$ bits directly: for every K-state agent there is a planar trap with at most $K + 1$ nodes (Theorem 2; with loops and multiple edges, or $K + \Delta + 2$ nodes for simple graphs). Earlier: no finite automaton explores all mazes (Budach 1978); no finite team of automata explores all cubic planar graphs (Rollik 1980).

- Fraigniaud et al. (2005a) [current best]: memory $O(D \log \Delta)$. Introductory and current best (Increasing-DFS, Theorem 6 and Corollary 7): depth-first searches of depth $1, 2, 3, \ldots$; no knowledge of D or Δ is needed. The bound is the memory used until exploration is complete; by Theorem 8 any agent that explores all graphs uses unboundedly many states over time.

Open questions.

Q1. What is the time of memory-optimal perpetual exploration, and is there a trade-off between memory and time?

Notes. In (Fraigniaud et al. 2005a) exploring means traversing every edge, not only visiting every node. The model goes back to maze search: Shannon's maze-solving mouse, Budach's theorem that no finite automaton explores all mazes (Budach 1978), Blum and Kozen (Blum and Kozen 1978), and Rollik's result that no finite team of automata explores all cubic planar graphs (Rollik 1980). Blum and Kozen proved that a finite automaton with two pebbles, or two automata together, search every maze, and that no three finite automata together can search all finite planar cubic graphs (Theorem 6.1; also one automaton, Theorem 4.1, and two, Theorem 4.6), so mazes, which come with a compass, are strictly easier than graphs; whether any finite team suffices for planar cubic graphs was left open (Blum and Kozen 1978). Their automata need not halt. This confirms the account of (Fraigniaud et al. 2005a); the reading in (Cohen et al. 2008) that one agent with three pebbles cannot explore all graphs is not a result of (Blum and Kozen 1978). Fraigniaud, Ilcinkas and Pelc showed that a polynomial increase in the number of states strictly enlarges the class of explorable regular graphs (DAM 2008, not in the bibliography yet). Checked against the full text.

16.1.2 Trees, perpetual exploration

Adds + tree **to** Arbitrary graphs, perpetual exploration (p. 82) VERIFIED

On trees, perpetual exploration needs exactly $\Theta(\log \Delta)$ bits: enough to remember a port.

Assumed: graph dynamics: static graph; edge direction: undirected; termination requirement: visit every node; never required to stop (perpetual); topology: tree. **Not assumed:** node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; markers: no markers; preprocessing: ports and nodes as given by an adversary.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not stated in the sources read.	—	● unknown
Memory	$\lceil \log \Delta \rceil$ (Diks et al. 2002) $\Theta(\log \Delta)$ bits.	$O(\log \Delta)$ (Diks et al. 2002)	● tight

Results.

- Diks et al. (2002) [lower bound]: memory $\lceil \log \Delta \rceil$. *Bottleneck:* port addressing. *Holds when:* trees of maximum degree Δ; deterministic.
- Diks et al. (2002) [current best]: memory $O(\log \Delta)$. Introductory and current best. Equivalently, the sequence 1^n is a universal exploration sequence for trees (Koucký 2002): leave by the port after the entry port.

16.1.3 Exploration with stop, nothing known

Adds nothing known, no markers, adversarial ports, + with stop VERIFIED

Without a bound on n and without a way to mark a node, an agent can never be sure it has seen everything.

Assumed: graph dynamics: static graph; edge direction: undirected; termination requirement: explore, then stop anywhere. **Not assumed:** node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; markers: no markers; preprocessing: ports and nodes as given by an adversary; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Dieudonné and Pelc 2012) Feasible with one unmovable token at the start node (Dieudonné and Pelc 2012).	—	● impossible
Memory	*impossible* (Dieudonné and Pelc 2012) Impossible for any memory size.	—	● impossible

Results.

- Dieudonné and Pelc (2012) [lower bound]: time *impossible. Bottleneck:* no way to recognise the start. *Holds when:* deterministic agent, no upper bound on n, no marking of nodes; unbounded memory does not help; even cycles cannot be explored with stop without a way to mark a node (Fraigniaud et al. 2006c).
- Dieudonné and Pelc (2012) [lower bound]: memory *impossible. Bottleneck:* no way to recognise the start. *Holds when:* same conditions as for time.

Notes. The impossibility is why every stopping variant below adds either knowledge (a bound on n) or a way to mark a node (a pebble or a token). Dieudonné and Pelc also study Byzantine tokens: identical unmovable tokens, some of which may be faulty; exploration is feasible in polynomial cost if a bound on the number of tokens is known and at least one is fault-free (Dieudonné and Pelc 2012).

16.1.4 Trees, exploration with stop

Adds + tree **to** Exploration with stop, nothing known (p. 84) VERIFIED

Stopping on a tree without knowing n needs more than constant memory, but how much more is open: $\Omega(\log \log \log n)$ *against* $O(\log n)$.

Assumed: graph dynamics: static graph; edge direction: undirected; termination requirement: explore, then stop anywhere; topology: tree. **Not assumed:** node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; markers: no markers; preprocessing: ports and nodes as given by an adversary.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not studied.	—	● unknown
Memory	$\Omega(\log\log\log n)$ (Diks et al. 2002) $O(\log n)$ versus $\Omega(\log\log\log n)$: exponentially apart.	$O(\log n)$ (Ambühl et al. 2011)	● polynomial gap

Results.

- Diks et al. (2002) [lower bound]: memory $\Omega(\log\log\log n)$. *Bottleneck:* deciding when to stop. *Holds when:* bounded-degree trees built from paths; n unknown; deterministic.
- Ambühl et al. (2011) [current best]: memory $O(\log n)$. Via exploration with return, which implies stop.

Open questions.

Q1. Can exploration with stop on trees use asymptotically less memory than exploration with return? (Open problem of Diks et al. 2002.) Equivalently: close $\Omega(\log\log\log n)$ versus $O(\log n)$.

Notes. If an upper bound N on n is known, $O(\log N)$ bits suffice; if a bound m on the number of nodes of some degree is known, $O(\log\Delta + \log m)$ bits (Diks et al. 2002). "The additional memory is essentially needed to decide when to stop."

16.1.5 Exploration with return and pebbles

Adds nothing known, + pebbles VERIFIED

$\Theta(\log\log n)$ pebbles are necessary and sufficient for a constant-memory agent, which then explores in polynomial time and returns to the start without knowing n.

Assumed: graph dynamics: static graph; edge direction: undirected; markers: several pebbles; their number is a resource. **Not assumed:** node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; preprocessing: ports and nodes as given by an adversary; termination requirement: explore, then stop at the start node; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known Polynomial; the exponent is not tracked.	$n^{O(1)}$ (Disser et al. 2019)	● unknown
Memory	$\Omega(\log\log n)$ pebbles (Disser et al. 2019) $\Theta(\log\log n)$ pebbles with $O(1)$ bits.	$O(1)$ bits, $O(\log\log n)$ pebbles (Disser et al. 2019)	● tight

Results.

- Disser et al. (2019) [lower bound]: memory $\Omega(\log\log n)$ pebbles. *Bottleneck:* trap graphs. *Holds when:* for any agent with $O((\log n)^{1-\varepsilon})$ bits; holds for every termination requirement; deterministic.
- Disser et al. (2019) [current best]: time $n^{O(1)}$, memory $O(1)$ bits, $O(\log\log n)$ pebbles. *Assumes:* distinguishable pebbles; n unknown. Introductory and current best (SODA 2016 for bounded degree, JACM 2019 in general). Since $p+1$ agents can simulate one agent with p pebbles, the same bounds hold for teams: $\Theta(\log\log n)$ constant-memory agents.

Open questions.

Q1. How much memory is needed to encode the agent's transition function? The naive table needs $(\log n)^{O(1)}$ bits (Remark 1 of Disser et al. 2019).

Claude (Anthropic) was used in cataloguing the problems.

Notes. The trade-off reads: $\Omega(\log n)$ bits make pebbles unnecessary; $O((\log n)^{1-\varepsilon})$ bits force $\Omega(\log\log n)$ pebbles; $\Omega(\log\log n)$ pebbles make $O(1)$ bits enough. The traps of (Disser et al. 2019) have $O(K^{2^{5k}})$ nodes for k agents with K states, improving Rollik's tower-type bound (Rollik 1980).

16.1.6 Memoryless automaton with pebbles

Adds + perpetual **to** Exploration with return and pebbles (p. 85) VERIFIED

A deterministic memoryless automaton (one state) explores every connected graph with 2 pebbles, and 2 is the minimum.

Assumed: graph dynamics: static graph; edge direction: undirected; markers: several pebbles; their number is a resource; termination requirement: visit every node; never required to stop (perpetual). **Not assumed:** node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; preprocessing: ports and nodes as given by an adversary; topology: arbitrary connected graph.

Statement for this setting. The agent is a deterministic finite automaton with a single state: its decision depends only on what it observes at the current node (entry port, degree, pebbles present), not on a state. It carries p pebbles labelled $1, \dots, p$, which it can drop and pick up. Exploration is complete when every edge has been traversed at least once.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not studied.	—	● unknown
Memory	2 pebbles (Pattanayak and Pelc 2024a) Two pebbles are necessary and sufficient.	0 bits, 2 pebbles (Pattanayak and Pelc 2024a)	● tight

Results.

- Pattanayak and Pelc (2024a) [lower bound]: memory 2 pebbles. *Bottleneck:* trap graphs. *Holds when:* one pebble is insufficient in general, even if n is known; a memoryless automaton without pebbles cannot explore all graphs.
- Pattanayak and Pelc (2024a) [current best]: memory 0 bits, 2 pebbles. *Assumes:* n and Δ unknown. Introductory and current best. If n is known and the automaton starts at a node with a pebble, one pebble suffices.

Notes. Results from the paper's abstract, supplied by the catalogue owner (a co-author).

16.1.7 Exploration with stop when n is known

Adds deterministic, + with stop VERIFIED

Reingold's log-space universal exploration sequences give an $O(\log n)$-bit agent that explores and stops in polynomial time.

Assumed: graph dynamics: static graph; edge direction: undirected; knowledge: an upper bound on n is known; termination requirement: explore, then stop anywhere. **Not assumed:** node identifiers: anonymous nodes; randomness: deterministic; markers: no markers; preprocessing: ports and nodes as given by an adversary; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known Polynomial; the exponent is not tracked and no non-trivial lower bound is recorded.	$n^{O(1)}$ (Reingold 2008)	● unknown
Memory	$\omega((\log n)^{1-\varepsilon})$ (Disser et al. 2019) $O(\log n)$ versus $\omega((\log n)^{1-\varepsilon})$: a $(\log n)^{\varepsilon}$ factor.	$O(\log n)$ (Reingold 2008)	● near-optimal

Results.

- Disser et al. (2019) [lower bound]: memory $\omega((\log n)^{1-\varepsilon})$. *Bottleneck:* trap graphs. *Holds when:* for every constant $\varepsilon > 0$; the agent may depend on n: it must explore all graphs with at most n nodes; no pebbles; deterministic. Corollary 25 of (Disser et al. 2019): with $O((\log n)^{1-\varepsilon})$ bits an agent needs $\Omega(\log \log n)$ pebbles.

- Reingold (2008) [current best]: time $n^{O(1)}$, memory $O(\log n)$. Introductory and current best. A log-space constructible universal exploration sequence (Koucký 2002) guides a walk through all nodes; the consequence for agents is stated by (Disser et al. 2019).

Open questions.

Q1. Is $\Omega(\log n)$ bits necessary when n is known? The trap arguments (Fraigniaud et al. 2005a) are stated for agents that know nothing.

Notes. Whether the same agent can also return to s (by reversing the sequence) is plausible but not checked against a source.

16.1.8 Randomized exploration when n is known

Adds + private coins, + with stop VERIFIED

A random walk covers any graph in expected $2m(n-1)$ steps; with a step counter and a bound on n it stops after exploring with good probability.

Assumed: graph dynamics: static graph; edge direction: undirected; knowledge: an upper bound on n is known; randomness: randomized (agents have private coins); termination requirement: explore, then stop anywhere. **Not assumed:** node identifiers: anonymous nodes; markers: no markers; preprocessing: ports and nodes as given by an adversary; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound recorded for randomized agents.	$O(mn)$ (Aleliunas et al. 1979)	• unknown
Memory	none known No lower bound recorded for randomized agents.	$O(\log n)$ (Aleliunas et al. 1979)	• unknown

Results.

- Aleliunas et al. (1979) [current best]: time $O(mn)$, memory $O(\log n)$. *Assumes:* expected time; stops correctly with constant probability (Monte Carlo). Expected cover time at most $2m(n-1)$ from any start (Theorem 4). Running the walk for a multiple of that many steps with an $O(\log n)$-bit counter gives the Monte Carlo agent.

Open questions.

Q1. What is the memory needed by a randomized agent that must stop correctly with high probability, as a function of n?

Notes. The same paper proves by the probabilistic method that universal traversal sequences of length $O(n^3 \log n)$ exist for connected Δ-regular graphs, the constant depending on Δ (the proof uses length $2\Delta n(n-1)(\Delta n+2)\lceil \log n \rceil$) (Aleliunas et al. 1979). For directed graphs the cover time can be exponential, while Eulerian digraphs behave like undirected graphs (Aleliunas et al. 1979); Reingold later constructed universal exploration sequences in log space (Reingold 2008).

Claude (Anthropic) was used in cataloguing the problems.

16.1.9 Exploration with stop and one pebble

Adds nothing known, + one pebble, + with stop VERIFIED

One pebble makes stopping possible; the agent then needs $\Omega(\log n)$ bits.

Assumed: graph dynamics: static graph; edge direction: undirected; markers: one pebble (a movable marker); termination requirement: explore, then stop anywhere. **Not assumed:** node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; preprocessing: ports and nodes as given by an adversary; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not studied.	—	● unknown
Memory	$\Omega(\log n)$ (Fraigniaud et al. 2006c) $O(D \log \Delta)$ versus $\Omega(\log n)$: polynomial when D is large.	$O(D \log \Delta)$ (Fraigniaud et al. 2005b)	● polynomial gap

Results.

- Fraigniaud et al. (2006c) [lower bound]: memory $\Omega(\log n)$. *Bottleneck:* reduced automata. *Holds when:* deterministic agent with one pebble that starts on the agent's start node; no knowledge of n; the traps are edge-coloured regular graphs of size $O(K^3)$ for a K-state agent (Theorem 5 of Fraigniaud et al. 2006c).
- Fraigniaud et al. (2005b) [current best]: memory $O(D \log \Delta)$. Introductory and current best (Theorem 3, DFS-with-stop): the pebble marks the start; depth-first phases of depth $1, 2, \ldots$ detect new nodes, and the agent stops after phase $D + 1$. No knowledge of D or Δ. Time is not analysed.

Open questions.

Q1. Is $\Omega(D \log \Delta)$ memory needed for exploration with stop by an agent with one pebble (Fraigniaud et al. 2005b)?

Notes. The same authors show that k non-cooperative agents need $\Omega(\log(n/k))$ bits for perpetual exploration, with traps of size $O(kK)$ (Fraigniaud et al. 2006c). That chapter is inconsistent on tightness: a remark calls the $\Omega(\log n)$ bound tight, citing the SIROCCO paper, while its conclusion says a tight bound for terminating exploration is open. Checked against the full texts.

16.1.10 Trees, exploration with return

Adds nothing known, no markers, adversarial ports, with return, + tree VERIFIED

Returning to the start on a tree takes $\Theta(\log n)$ bits.

Assumed: graph dynamics: static graph; edge direction: undirected; topology: tree. **Not assumed:** node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; markers: no markers; preprocessing: ports and nodes as given by an adversary; termination requirement: explore, then stop at the start node.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not stated in the abstracts read.	—	● unknown
Memory	$\Omega(\log n)$ (Diks et al. 2002) $\Theta(\log n)$ bits.	$O(\log n)$ (Ambühl et al. 2011)	● tight

Results.

- Diks et al. (2002) [lower bound]: memory $\Omega(\log n)$. *Bottleneck:* pigeonhole on start nodes. *Holds when:* every n-node tree; deterministic; holds even if the agent knows the tree up to its start node, so knowing n does not

help.

- Diks et al. (2002) [introductory]: memory $O(\log^2 n)$.
- Ambühl et al. (2011) [current best]: memory $O(\log n)$. Also works when ports are only circularly ordered.

Notes. (Diks et al. 2002) asked for the exact memory of exploration with return; (Ambühl et al. 2011) answered it.

16.1.11 Node labels chosen by the designer

Adds nothing known, no markers, + designer labels, + with stop VERIFIED

With three-valued labels on the nodes, an agent with fewer than 25 states explores every graph and stops after at most $20m$ edge traversals.

Assumed: graph dynamics: static graph; edge direction: undirected; preprocessing: short node labels chosen by the designer; termination requirement: explore, then stop anywhere. **Not assumed:** node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; markers: no markers; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known Only the trivial $\Omega(n)$ is known against $O(m)$.	$O(m)$ (Cohen et al. 2008)	● unknown
Memory	none known Constant memory.	$O(1)$ (Cohen et al. 2008)	● tight

Results.

- Cohen et al. (2008) [current best]: time $O(m)$, memory $O(1)$. *Assumes:* labels of 2 bits (3 values), computed in linear time. Introductory and current best (Theorem 2.1): fewer than 25 states, at most $20m$ edge traversals, stops at a node chosen by the designer. Labelling while exploring gives $O(Dm)$.
- Cohen et al. (2008) [also]: time $O(\Delta^{10}m)$, memory $O(\log \Delta)$. *Assumes:* labels of 1 bit. Theorem 3.1.

Notes. The agent is a Mealy automaton: its output port depends on the input port. Agents with a single state cannot explore all graphs with such labels (Theorems 4.2 and 4.5 of Cohen et al. 2008). Recent work changes the model: zero-memory agents that do not see the entry port but may colour nodes (SIROCCO 2023, OPODIS 2025); these are not catalogued yet. Checked against the full text.

16.1.12 Port numbers chosen by the designer (periodic exploration)

Adds nothing known, no markers, + designer ports, + perpetual DRAFT

If the designer may set the port numbers, a constant-memory agent visits every node periodically with period between $2n-2$ and $3.5n-2$.

Assumed: graph dynamics: static graph; edge direction: undirected; preprocessing: port numbers chosen by the designer; termination requirement: visit every node; never required to stop (perpetual). **Not assumed:** node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; markers: no markers; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	$2n-2$ (Czyzowicz et al. 2012c) $\Theta(n)$; the constant is open: $2n-2$ versus $3.5n-2$ (constant memory), $2.8n-2$ versus $4n-2$ (oblivious).	$3.5n-2$ (Czyzowicz et al. 2012c)	● tight
Memory	none known Constant memory.	$O(1)$ (Czyzowicz et al. 2012c)	● tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Czyzowicz et al. (2012c) [lower bound]: time $2n-2$. *Bottleneck:* every tree edge twice. *Holds when:* any agent: on a tree every edge is traversed in both directions; realised by the star; folklore; for oblivious agents the bound is $2.8n-2$.
- Ilcinkas (2008) [introductory]: time $4n-2$, memory $O(1)$. *Assumes:* three-state agent. Theorem 1; the abstract rounds the period to $4n$.
- Gąsieniec et al. (2008) [also]: time $3.75n-2$, memory $O(1)$. The step between $4n-2$ and $3.5n-2$, as cited by (Czyzowicz et al. 2012c).
- Czyzowicz et al. (2012c) [current best]: time $3.5n-2$, memory $O(1)$. For oblivious agents the same paper gives $(4+\frac{1}{3})n-4$; the best oblivious period is $4n-2$ (Kosowski and Navarra 2012).
- Kosowski and Navarra (2012) [also]: time $4n-2$, memory 0. *Assumes:* oblivious agent (right-hand rule). Figure as reported by (Czyzowicz et al. 2012c).

Open questions.

Q1. Close the constant gaps: $2n-2$ versus $3.5n-2$ for constant memory, $2.8n-2$ versus $4n-2$ for oblivious agents (Czyzowicz et al. 2012c).

Q2. Trade-offs between period and memory: the only lower bound for agents with memory does not depend on memory.

Notes. Without the designer's help, periodic exploration by a finite automaton is impossible (Budach, as cited by Czyzowicz et al. 2012c). These papers began with Dobrev, Jansson, Sadakane and Sung (SIROCCO 2005; period $10n$, as cited by Czyzowicz et al. 2012c). Computing the shortest witness cycle for an oblivious agent is NP-hard. The period is defined differently: Ilcinkas counts returns to a node in the same state through the same port, Czyzowicz et al. returns to a node through the same port. **Unverified:** the oblivious $4n-2$ of (Kosowski and Navarra 2012), known only from (Czyzowicz et al. 2012c).

16.1.13 Directed graphs, one pebble

Adds directed, anonymous nodes, nothing known, + one pebble, + with stop VERIFIED

An agent that cannot walk back along an arc needs $\Omega(n \log \Delta_{out})$ bits even with a pebble; $O(n\Delta_{out} \log n)$ bits suffice.

Assumed: graph dynamics: static graph; markers: one pebble (a movable marker); termination requirement: explore, then stop anywhere. **Not assumed:** edge direction: directed (strongly connected); node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; preprocessing: ports and nodes as given by an adversary; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not stated.	—	● unknown
Memory	$\Omega(n \log \Delta_{out})$ (Fraigniaud and Ilcinkas 2004) Factor $\Delta_{out} \log n / \log \Delta_{out}$.	$O(n\Delta_{out} \log n)$ (Fraigniaud and Ilcinkas 2004)	● polynomial gap

Results.

- Fraigniaud and Ilcinkas (2004) [lower bound]: memory $\Omega(n \log \Delta_{out})$. *Bottleneck:* no backtracking. *Holds when:* strongly connected digraphs, maximum out-degree Δ_{out}; holds even with a linear number of identical pebbles (as cited by Disser et al. 2019).

Claude (Anthropic) was used in cataloguing the problems.

- Fraigniaud and Ilcinkas (2004) [current best]: memory $O(n\Delta_{out} \log n)$. Introductory and current best. With whiteboards instead of a pebble, constant agent memory and $O(\log \Delta_{out})$ bits per node suffice.

Open questions.

Q1. Close $\Omega(n \log \Delta_{out})$ versus $O(n\Delta_{out} \log n)$ bits for exploring digraphs with one pebble.

Notes. Here Δ_{out} is the maximum out-degree (d in Fraigniaud and Ilcinkas 2004). Bender and Slonim showed that two cooperating agents learn any unlabelled digraph in expected polynomial time (FOCS 1994, not yet in the bibliography).

16.1.14 Directed graphs, one pebble, bound on n known

Adds directed, anonymous nodes, + knows a bound on n, + one pebble, + with stop VERIFIED

With a bound $N \geq n$, one pebble lets an agent map any strongly connected digraph in polynomial time.

Assumed: graph dynamics: static graph; knowledge: an upper bound on n is known; markers: one pebble (a movable marker); termination requirement: explore, then stop anywhere. **Not assumed:** edge direction: directed (strongly connected); node identifiers: anonymous nodes; randomness: deterministic; preprocessing: ports and nodes as given by an adversary; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound beyond the trivial $\Omega(m)$.	$O(N^2 n^6 \Delta_{out}^2)$ (Bender et al. 2002)	• unknown
Memory	none known Memory is not bounded in these papers.	—	• unknown

Results.

- Bender et al. (2002) [current best]: time $O(N^2 n^6 \Delta_{out}^2)$. *Assumes:* unbounded memory; all out-degrees padded to Δ_{out} with self-loops; Δ_{out} known. Introductory and current best (Theorem 3.1; N is the paper's $\hat{n}$, Δ_{out} its d). Without a bound on n, for every constant $c > 0$, $c \log \log n$ pebbles give a deterministic polynomial-time algorithm (Theorem 4.1), and $\Omega(\log \log n)$ pebbles are necessary for any algorithm, even randomized, that learns every graph in expected polynomial time with probability greater than $1/2$ (Theorem 4.2).

Open questions.

Q1. Improve the $O(N^2 n^6 \Delta_{out}^2)$ running time (Section 5 of Bender et al. 2002).

Notes. Checked against the full text of (Bender et al. 2002). The agent must output a graph isomorphic to G; it sees out-ports numbered $1, \ldots, \Delta_{out}$ but not in-degrees, and cannot retrace an arc to fetch its pebble. Bender and Slonim had shown that a constant number of pebbles does not suffice for polynomial time when no bound on n is known (as cited in Bender et al. 2002). With labelled nodes a greedy search takes $O(nm)$ moves.

16.1.15 Directed graphs with labelled nodes

Adds directed, + labelled nodes, + with stop DRAFT

An agent that recognises nodes but cannot backtrack must traverse every arc; the cost depends on the deficiency ϕ of the digraph.

Assumed: graph dynamics: static graph; node identifiers: labelled nodes (an agent can recognise a node it has visited); termination requirement: explore, then stop anywhere. **Not assumed:** edge direction: directed (strongly connected);

Claude (Anthropic) was used in cataloguing the problems.

knowledge: nothing known about the graph; randomness: deterministic; markers: no markers; preprocessing: ports and nodes as given by an adversary; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(\phi^2 m)$ (Deng and Papadimitriou 1999) $O(\phi^8 m)$ versus $\Omega(\phi^2 m)$: factor ϕ^6.	$O(\phi^8 m)$ (Fleischer and Trippen 2005)	• polynomial gap
Memory	none known Memory is not bounded in these papers.	—	• unknown

Results.

- Deng and Papadimitriou (1999) [lower bound]: time $\Omega(\phi^2 m)$. *Bottleneck:* unbalanced arcs. *Holds when:* deficiency ϕ: the number of arcs to add to make the digraph Eulerian; deterministic strategies; the argument does not apply to randomized ones; due to Koutsoupias: Corollary 3 of (Deng and Papadimitriou 1999) gives at least $\phi^2/4$ times the number of arcs. Deng and Papadimitriou's own bound (Theorem 1) is about $\phi^2 m/(2\log\phi)$; against randomized strategies they show a ratio of $\Omega(\phi/\log\phi)$ to the optimal tour (Corollary 2).
- Deng and Papadimitriou (1999) [introductory]: time $\phi^{O(\phi)}\,m$. *Assumes:* unbounded memory. Theorem 3: no arc is traversed more than $2^{O(\phi\log\phi)}$ times, without knowing ϕ. Eulerian digraphs ($\phi = 0$): ratio 2 to the optimal tour, which is optimal; deficiency one: no arc traversed more than four times (Theorem 2).
- Albers and Henzinger (2000) [also]: time $\phi^{O(\log\phi)}\,m$. *Assumes:* unbounded memory. First subexponential algorithm; the authors also show that their algorithm needs $\phi^{\Omega(\log\phi)}m$ on some graphs, and that Greedy, Depth-First and Breadth-First need $2^{\Omega(\phi)}m$.
- Fleischer and Trippen (2005) [current best]: time $O(\phi^8 m)$. *Assumes:* unbounded memory. First algorithm polynomial in ϕ; the paper itself was not read, the bound is from the authors' abstract.

Open questions.

Q1. Close $\Omega(\phi^2 m)$ versus $O(\phi^8 m)$ for online exploration of digraphs with deficiency ϕ.

Notes. This is the online-algorithms tradition: nodes are recognisable and the agent builds a map; cost is compared with the optimal tour (the Chinese postman tour), and Deng and Papadimitriou note that measured in absolute terms $\Theta(nm)$ is trivially tight. Checked against the full texts of (Deng and Papadimitriou 1999) and (Albers and Henzinger 2000); the bound of (Fleischer and Trippen 2005) is still from the authors' abstract.

16.1.16 Dynamic rings

Adds nothing known, + perpetual, + ring VERIFIED

On a T-interval-connected ring whose edges may be missing for up to δ steps, the worst-case exploration time is known exactly up to additive terms.

Assumed: graph dynamics: dynamic graph, connected in every round (1-interval connected); edge direction: undirected; termination requirement: visit every node; never required to stop (perpetual); topology: ring or path. **Not assumed:** node identifiers: anonymous nodes; knowledge: nothing known about the graph; randomness: deterministic; markers: no markers; preprocessing: ports and nodes as given by an adversary.

Measure	Lower bound	Best upper bound	Status
Time	$n + \frac{n}{\max\{1,T-1\}}(\delta-1) - O(\delta)$ (Ilcinkas and Wade 2018) Matching up to $\pm\Theta(\delta)$.	$n + \frac{n}{\max\{1,T-1\}}(\delta-1) + O(\delta)$ (Ilcinkas and Wade 2018)	• tight
Memory	none known Memory not studied.	—	• unknown

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Ilcinkas and Wade (2018) [lower bound]: time $n + \frac{n}{\max\{1,T-1\}}(\delta - 1) - O(\delta)$. *Bottleneck:* adversarial edge changes. *Holds when:* underlying graph a ring; the agent does not know the dynamics; T-interval connectivity; δ = longest absence of an edge.
- Ilcinkas and Wade (2018) [current best]: time $n + \frac{n}{\max\{1,T-1\}}(\delta - 1) + O(\delta)$. Introductory and current best. If the agent knows the dynamics, the worst case is $2n - T - \Theta(1)$.

Notes. Multi-agent exploration of dynamic rings and graphs, where the measure is how many agents are needed (Di Luna, Dobrev, Flocchini and Santoro; Gotoh, Flocchini, Masuzawa and Santoro), is not catalogued yet.

16.1.17 Temporal graphs, schedule known

Adds + knows the map, + perpetual VERIFIED

If the whole schedule of an always-connected temporal graph is known, $\Theta(n^2)$ steps are necessary and sufficient in the worst case.

Assumed: graph dynamics: dynamic graph, connected in every round (1-interval connected); edge direction: undirected; knowledge: a complete map is known (for a dynamic graph: the whole schedule); termination requirement: visit every node; never required to stop (perpetual). **Not assumed:** node identifiers: anonymous nodes; randomness: deterministic; markers: no markers; preprocessing: ports and nodes as given by an adversary; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(n^2)$ (Erlebach et al. 2021) $\Theta(n^2)$ in the worst case.	$O(n^2)$ (Erlebach et al. 2021)	• tight
Memory	none known Not a memory question.	—	• unknown

Results.

- Erlebach et al. (2021) [lower bound]: time $\Omega(n^2)$. *Bottleneck:* adversarial edge changes. *Holds when:* an explicit family of always-connected temporal graphs; holds for centralised, offline exploration.
- Erlebach et al. (2021) [current best]: time $O(n^2)$. The n^2 upper bound is attributed by the authors to Michail and Spirakis. They also give $O(n^{1.5}\mathrm{tw}^{1.5}\log n)$ for treewidth tw, $O(n^{1.8}\log n)$ for planar and $2n - 2$ for cycles, and show inapproximability within $O(n^{1-\varepsilon})$.

Open questions.

Q1. Tight bounds when the underlying graph is a grid, has bounded degree or is planar (Erlebach et al. 2021).

Notes. This is a centralised planning problem rather than a distributed one, but it gives the baseline for what any agent could do in a temporal graph.

16.2 Treasure hunt

One agent must find a static treasure at an unknown node; the cost should depend on how far the treasure is, not on the size of the graph.

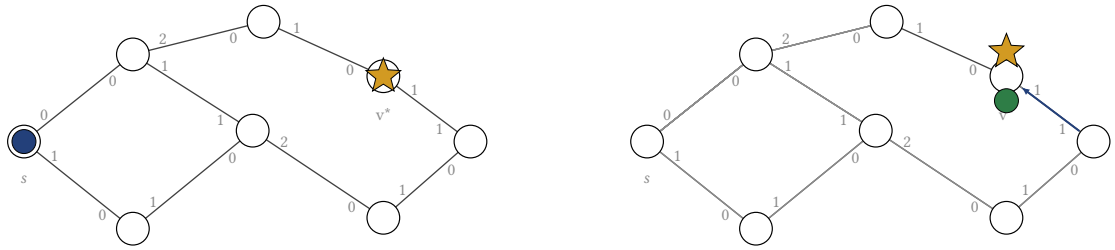
Claude (Anthropic) was used in cataloguing the problems.

First posed in 2007 (Ta-Shma and Zwick 2014). Ta-Shma and Zwick introduced the deterministic treasure hunt problem at SODA 2007, as a variant of rendezvous in which one agent never moves.

Problem statement and measures

A single deterministic agent starts at a node s of an unknown connected graph whose edges carry **ports** (at each node, local numbers of the edges at that node); a **treasure** sits at an unknown node v^* at distance d from s. The agent must reach v^* (it recognises the treasure when it arrives). **Cost** is the number of moves (edge traversals) until the treasure is found, measured against d, against $e(d)$ (the number of edges with an endpoint at distance less than d from s) or against a path weight defined per paper. **Help** counts any external help the agent receives: bits of **advice** from an oracle that knows the instance, or **pebbles** (objects left on nodes) placed by it. Unless a variant says otherwise the agent knows nothing about the graph, d or v^*.

Measures: **Cost**, edge traversals until the treasure is reached; **Help**, bits of advice, or pebbles placed in advance by an oracle.



(a) Start: agent at s, treasure at an unknown node v*

(b) Final: treasure found during the phase N = 8, after 13 moves

Figure 16.2: Treasure hunt by exploration sequences of doubling size (Ta-Shma and Zwick 2014): in the phases $N = 1, 2, 4, 8, \ldots$ the agent follows the sequence for size N from where it stands (entering by port p with term x, it leaves by port $(p + x)$ mod deg; ports are numbered from 0) and stops on the treasure v^*. The short sequences drawn here are illustrative, not actual universal sequences, which are much longer. One frame per move.

A first look

A hiker is lost in a park whose trails form a network, and a rescuer walks from the trailhead looking for her, without a map and without knowing which direction to take. The network is a **graph**: trail junctions are **nodes** and the trails between them are **edges**; the rescuer is a single **agent** that starts at a node s and walks along edges, and one crossing of one edge is a **move**. The graph is unknown and connected, and at each node the edges at that node carry local numbers, its **ports**. The hiker is the **treasure**, at an unknown node v^* at **distance** d from s (the distance is the fewest edges on a path between two nodes). The agent recognises the treasure only when it stands on it. In the basic version the graph is finite, with n nodes, and the agent knows nothing about it; it must reach v^*, and the **cost** is the number of moves it makes.

1. Algorithm: Universal exploration sequences of doubling size. (Ta-Shma and Zwick 2014) (Anonymous finite graphs, p. 98) An **exploration sequence** is a list of numbers $x_1, x_2, \ldots$; following it means: at the i-th move, if you entered the current node (of **degree** deg, the number of edges at it) by port p, leave by port $(p + x_i)$ mod deg. A sequence is **universal for size** N if following it visits every node of

Claude (Anthropic) was used in cataloguing the problems.

every connected graph with at most N nodes, from any start. Such sequences exist and can be computed (Reingold 2008).

1. For $N = 1, 2, 4, 8, \ldots$ (one **phase** per value of N):
2. follow the universal sequence for size N from wherever you are;
3. stop as soon as you stand on the treasure.

Why it works and what it costs. Once $N \geq n$, the sequence for size N visits every node, including v^*, so the search ends during that phase (Ta-Shma and Zwick 2014). Each sequence has length polynomial in N, the lengths grow geometrically, and the loop never goes past $N < 2n$, so the total cost is polynomial in n. It ignores d: a treasure next door may cost as much as one far away. With node labels, a treasure at distance d is found at cost $O(e(d) \log d)$ (Bouchard et al. 2023a), where $e(d)$ is the number of edges with an endpoint at distance less than d from s.

Why it matters. Doubling a guess you do not know (here n) is the standard way to remove knowledge from an algorithm; it reappears in rendezvous and gathering.

2. Lower bound: The treasure is behind the last port. (Bouchard et al. 2023a) (Labelled nodes, locally finite graphs, p. 99) Let s have many neighbours, say 20, and put the treasure on one of them, so $d = 1$. The agent cannot see what is at the other end of an edge without crossing it. An adversary watches the agent and puts the treasure behind whichever of the 20 ports the agent tries last; since every node looks the same from s, the agent cannot avoid this.

Why it holds. The agent walks out and back along at least 19 edges before it tries the last one, so it pays about twice the degree of s. Here $e(1)$, the number of edges with an endpoint at distance less than 1 from s, is exactly the degree of s, and the same idea applied layer by layer gives the general bound $\Omega(e(d))$ of (Bouchard et al. 2023a), which holds even when nodes carry labels. Their algorithm costs $O(e(d) \log d)$, so the gap is a factor $\log d$. Trivially the agent also needs at least d moves.

Why it matters. It shows that the right measure of cost is the number of edges near the start, not the distance, which is why later work states bounds in terms of $e(d)$ or path weights.

What the community started looking at

Ta-Shma and Zwick introduced deterministic treasure hunt as the variant of rendezvous in which one agent never moves (Ta-Shma and Zwick 2014). The question from the start was **cost as a function of how far the treasure is**, not of the size of the graph, which led to strongly universal exploration sequences. Later work asked how much **external help** (advice bits, pebbles) reduces the cost (Miller and Pelc 2015) (Gorain et al. 2022a), and what is possible in graphs with infinite degrees (Pattanayak and Pelc 2024b).

Relations to other problems

- **Subproblem of Graph exploration by a single agent.** An agent that explores the graph finds any treasure, so every exploration algorithm solves treasure hunt; the question is how much cheaper it can be when the treasure is close, measured against its distance d (or a weight of the path to it) rather than the size of the graph.

Claude (Anthropic) was used in cataloguing the problems.

Assumptions specific to this problem

Graph size

Ta-Shma and Zwick posed deterministic treasure hunt in a finite connected graph, as the variant of rendezvous in which one agent never moves (Ta-Shma and Zwick 2014); Pelc notes that rendezvous with arbitrary delay contains this case, since the adversary may wake one agent late enough (Pelc 2019). Infinite graphs make sense once cost is measured locally: Bouchard, Dieudonné, Labourel and Pelc consider connected graphs, finite or countably infinite, with finite degrees, and find a treasure at distance d at cost $O(e(d)\log d)$, refuting a conjecture of Awerbuch, Betke, Rivest and Singh (Bouchard et al. 2023a). Pattanayak and Pelc observed that "all known treasure hunt and rendezvous algorithms rely on the assumption that degrees of all nodes are finite, even when the graph itself may be infinite", removed the assumption, and noted that countable degrees are the limit: with uncountable degrees neither task can be done in finite worst-case time (Pattanayak and Pelc 2024b). The three levels differ in the natural cost measure: a function of n in finite graphs, $e(d)$ in locally finite ones, and a weight $w(s, v^*)$ of the cheapest path type when $e(d)$ may be infinite (Pattanayak and Pelc 2024b).

Level	Value	Formal definition
0	**infinite degrees allowed**	G is finite or countably infinite; a node may have countably infinite degree (ports $1, 2, 3, \dots$).
1	**locally finite**	G is finite or countably infinite, and every node has finite degree.
2	**finite graph**	G has finitely many nodes.

External help

External help measures how much information about the instance buys a fast treasure hunt. Miller and Pelc framed it in the paradigm of algorithms with advice: an oracle that knows the network, the start and the treasure gives the agent a binary string, and the question is the smallest advice that achieves a given cost, against cost $\Theta(m)$ in the worst case without information (Miller and Pelc 2015); among the earlier advice papers they cite tree exploration with advice by Fraigniaud, Ilcinkas and Pelc (Fraigniaud et al. 2008). Their nodes carry labels that the agent reads on arrival (Miller and Pelc 2015). Gorain, Mondal, Nayak and Pandit replaced the string by pebbles that the oracle places on nodes of an anonymous graph, at most one per node, seen only on arrival, arguing that when many agents search independently it is cheaper to put information on the nodes once than to give advice to each agent (Gorain et al. 2022a). They show $O(d \log \Delta)$ pebbles give time $O(d \log \Delta + \log^3 \Delta)$ and that no number of pebbles beats $\Omega(d \log \Delta)$ (Gorain et al. 2022a). The two forms of help sit on the same level because neither simulates the other: advice is global and read at once, pebbles are local and read along the walk.

Level	Value	Formal definition
0	**no help**	No information beyond the agent's own observations.
1	**advice**	An oracle that knows the whole instance gives the agent a binary string before it starts; its length (in bits) is the measure.
1	**oracle pebbles**	An oracle that knows the instance places pebbles on nodes (at most one per node) before the agent starts; their number is the measure.

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Cost, Help) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Graph size → Node identifiers → External help → Topology.

No assumptions

+ static graph, + undirected

Anonymous graphs with possibly infinite degrees ●● p. 97

+ finite graph

Anonymous finite graphs STANDARD SETTING ●● p. 98

+ advice

Treasure hunt with advice ●● p. 98

+ oracle pebbles

Treasure hunt guided by pebbles ●● p. 99

+ locally finite, + labelled nodes

Labelled nodes, locally finite graphs ●● p. 99

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Cost	Help	p.
Anonymous graphs with possibly infinite degrees	● tight $O(w(s,v^*))$ vs $\Omega(w(s,v^*))$	● tight — vs —	97
Anonymous finite graphs STANDARD SETTING	● unknown $O(n^{\theta(1+1/a)})$ vs —	● unknown — vs —	98
Treasure hunt with advice	● tight C vs —	● near-optimal $O(d\log(d\cdot m/C)+\log\log m)$ vs $\Omega(d\log(m/C))$	98
Treasure hunt guided by pebbles	● near-optimal $O(d\log\Delta+\log^3\Delta)$ vs $\Omega(d\log\Delta)$	● unknown $O(d\log\Delta)$ pebbles vs —	99
Labelled nodes, locally finite graphs	● near-optimal $O(e(d)\log d)$ vs $\Omega(e(d))$	● tight — vs —	99

16.2.1 Anonymous graphs with possibly infinite degrees

Adds + static graph, + undirected VERIFIED

A universal algorithm whose cost is optimal among universal algorithms, measured by a weight $w(s,v^*)$ of the cheapest path type to the treasure.

Assumed: graph dynamics: static graph; edge direction: undirected. **Not assumed:** graph size: finite or countably infinite, degrees may be infinite; node identifiers: anonymous nodes; external help: no help; topology: arbitrary connected graph.

Statement for this setting. For a path with largest port q and length ρ, its type is (q,ρ) with value $\rho\, 2^\rho q^\rho$; $w(s,v^*)$ is the value of the cheapest type of a path from s to v^*.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Cost	$\Omega(w(s,v^*))$ (Pattanayak and Pelc 2024b) Optimal order of magnitude among universal algorithms.	$O(w(s,v^*))$ (Pattanayak and Pelc 2024b)	• tight
Help	none known No help is used.	—	• tight

Results.

- Pattanayak and Pelc (2024b) [lower bound]: cost $\Omega(w(s,v^*))$. *Bottleneck:* port addressing. *Holds when:* among universal algorithms (no knowledge of the graph); graphs whose nodes may have countably infinite degree.
- Pattanayak and Pelc (2024b) [current best]: cost $O(w(s,v^*))$. Introductory and current best (Theorem 2.1).

16.2.2 Anonymous finite graphs

Adds + finite graph **to** Anonymous graphs with possibly infinite degrees (p. 97) STANDARD SETTING VERIFIED

If the treasure is present from the start, concatenating universal traversal sequences finds it in polynomial time; if it may appear later, explicit solutions need backtracking and are polynomial with a huge exponent.

Assumed: graph dynamics: static graph; edge direction: undirected; graph size: finite. **Not assumed:** node identifiers: anonymous nodes; external help: no help; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Cost	none known No lower bound in terms of n is recorded.	$O(n^{\theta(1+1/a)})$ (Xin 2007)	• unknown
Help	none known Memory is not studied in these papers.	—	• unknown

Results.

- Ta-Shma and Zwick (2014) [introductory]: cost $O(n^{2\theta})$. *Assumes:* the treasure may appear after the agent starts; backtracking: the agent sees its entry port. Theorem 3.2: an infinite strongly universal exploration sequence with cover time $O(n^{2\theta})$, θ the exponent of Reingold's universal exploration sequences; explicit when those are.
- Xin (2007) [current best]: cost $O(n^{\theta(1+1/a)})$. *Assumes:* the treasure may appear after the agent starts; backtracking: the agent sees its entry port. For any fixed integer $a \gg 1$; with backtracking (the agent sees the entry port).

Open questions.

Q1. Treasure hunt when the treasure may appear after the agent starts and the agent does **not** see its entry port: "We do not know if an efficient solution is possible when backtracking is not allowed" (Ta-Shma and Zwick 2014).

Q2. Do strongly universal traversal sequences exist? They exist if and only if treasure hunt has a solution independent of the start time (Ta-Shma and Zwick 2014).

Notes. For a treasure present when the agent starts, Ta-Shma and Zwick note that following $U_1U_2U_4\ldots$, universal traversal sequences of doubling length, solves the problem in polynomial time (Ta-Shma and Zwick 2014). The bounds above are for the harder case. Checked against the full texts.

16.2.3 Treasure hunt with advice

Adds + advice **to** Anonymous finite graphs (p. 98) VERIFIED

How many bits of advice buy a given cost C: about $d\log(m/C)$.

Claude (Anthropic) was used in cataloguing the problems.

Assumed: graph dynamics: static graph; edge direction: undirected; graph size: finite; external help: advice: bits from an oracle that knows the instance. **Not assumed:** node identifiers: anonymous nodes; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Cost	none known Cost C is the measure to minimise.	C (Miller and Pelc 2015)	• tight
Help	$\Omega(d \log(m/C))$ (Miller and Pelc 2015) A $\log d$ inside the first term on arbitrary graphs.	$O(d \log(d \cdot m/C) + \log\log m)$ (Miller and Pelc 2015)	• near-optimal

Results.

- Miller and Pelc (2015) [lower bound]: help $\Omega(d \log(m/C))$. *Bottleneck:* information. *Holds when:* to achieve cost C; d = distance to the treasure, m = number of edges.
- Miller and Pelc (2015) [current best]: cost C, help $O(d \log(d \cdot m/C) + \log\log m)$. Introductory and current best; on trees $O(d \log(m/C) + \log\log m)$, nearly tight.

Notes. The same paper gives the analogous trade-off for rendezvous.

16.2.4 Treasure hunt guided by pebbles

Adds + oracle pebbles **to** Anonymous finite graphs (p. 98) VERIFIED

An oracle places pebbles (at most one per node); $O(d \log \Delta)$ pebbles let the agent reach the treasure in near-optimal time.

Assumed: graph dynamics: static graph; edge direction: undirected; graph size: finite; external help: pebbles placed by an oracle that knows the instance. **Not assumed:** node identifiers: anonymous nodes; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Cost	$\Omega(d \log \Delta)$ (Gorain et al. 2022a) An additive $\log^3 \Delta$.	$O(d \log \Delta + \log^3 \Delta)$ (Gorain et al. 2022a)	• near-optimal
Help	none known No lower bound on the number of pebbles recorded.	$O(d \log \Delta)$ pebbles (Gorain et al. 2022a)	• unknown

Results.

- Gorain et al. (2022a) [lower bound]: cost $\Omega(d \log \Delta)$. *Bottleneck:* port addressing. *Holds when:* any number of pebbles; d = distance to the treasure.
- Gorain et al. (2022a) [current best]: cost $O(d \log \Delta + \log^3 \Delta)$, help $O(d \log \Delta)$ pebbles. Introductory and current best (SIROCCO 2021 bounds; whether the journal version improves them was not checked).

16.2.5 Labelled nodes, locally finite graphs

Adds + locally finite, + labelled nodes **to** Anonymous graphs with possibly infinite degrees (p. 97) VERIFIED

With node labels, a treasure at distance d is found at cost $O(e(d) \log d)$, against a lower bound of $\Omega(e(d))$.

Assumed: graph dynamics: static graph; edge direction: undirected; graph size: finite or countably infinite, every degree finite; node identifiers: labelled nodes (an agent can recognise a node it has visited). **Not assumed:** external help: no help; topology: arbitrary connected graph.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Cost	$\Omega(e(d))$ (Bouchard et al. 2023a) A $\log d$ factor.	$O(e(d)\log d)$ (Bouchard et al. 2023a)	• near-optimal
Help	none known No help is used.	—	• tight

Results.

- Bouchard et al. (2023a) [lower bound]: cost $\Omega(e(d))$. *Bottleneck:* distance. *Holds when:* $e(d)$: edges with an endpoint at distance below d; holds for every algorithm on some graphs.
- Bouchard et al. (2023a) [current best]: cost $O(e(d)\log d)$. *Assumes:* unbounded memory. Introductory and current best; holds also in the fuel- and rope-restricted models, and refutes a 1999 conjecture of Awerbuch, Betke, Rivest and Singh.

Open questions.

Q1. Remove the $O(\log d)$ factor between $O(e(d)\log d)$ and $\Omega(e(d))$ (Bouchard et al. 2023a).

16.3 Rendezvous

Two agents start at different nodes of an unknown graph and must meet; how long does it take, and how much memory or symmetry breaking does it need?

First posed in 2003 (Dessmark et al. 2006). Deterministic rendezvous of labelled agents in arbitrary graphs was posed by Dessmark, Fraigniaud and Pelc at ESA 2003. Rendezvous itself is older (randomized search games); the catalogue covers the deterministic line (Pelc 2019).

Problem statement and measures

Two mobile agents start at different nodes of an unknown connected port-labelled graph. In the synchronous model they must be at the same node in the same round; in the asynchronous model an adversary controls their speeds and they may also meet inside an edge. Agents are either **labelled** (distinct labels from $\{1,\dots,L\}$; λ is the smaller label) or **anonymous**. They may start at different times; τ is the delay. **Time** is the number of rounds from the start of the later agent (synchronous) or the total number of moves, that is edge traversals (asynchronous cost). **Memory** is bits per agent.

Measures: **Time**, rounds after the later agent starts (synchronous); edge traversals (asynchronous); **Memory**, bits per agent.

A first look

Two friends lose each other in a city with no phones and want to meet at the same spot. The city is a **graph**: street corners are **nodes** and the streets between them are **edges**; the friends are two **agents** that start at different nodes of this unknown connected graph. At each node the edges at that node carry local numbers, its **ports**. Time passes in synchronous **rounds**; in each round an agent stays or makes a **move** along one edge, and the agents **meet** when they are on the same node in the same round (crossing inside an edge does not count). In the basic version each agent knows its own **label**, a distinct integer from $\{1,\dots,L\}$, but not the other's label and nothing about the graph; λ is the smaller of the two labels, n the number of nodes, and the later agent may start τ rounds after the earlier one (the **delay**). The task is to meet, and the cost is the number of rounds from the start of the later agent.

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: agents 1 and 3 at the ends of a tree (n = 4)

(b) Final: agent 3 walks onto the waiting agent 1 in round 21

Figure 16.3: Rendezvous of two labelled agents in a tree, simultaneous start, by the explore-or-wait idea of (Dessmark et al. 2006) (simplified): after one depth-first tour to learn n, each agent reads its label in binary with every bit doubled and 01 appended, and spends one block of $2(n-1)$ rounds per bit, exploring and returning on a 1, waiting at its start node on a 0. Labels 1 and 3 give 11 01 and 11 11 01; in block 3 agent 3 explores while agent 1 waits, and they meet. One frame per round.

1. Algorithm: Explore or wait, bit by bit (trees, simultaneous start). (Dessmark et al. 2006) (Labelled agents in trees, p. 106) If one agent knew it should wait, the other could simply search the graph ("wait for mommy" in Dessmark et al. 2006). Without knowing the other label, use the bits of your own label to take turns. This is a simplified version of their idea, for a tree (no cycles), both agents starting in the same round.

1. Explore the tree by depth-first search and return to your **start node** (the node where you began); you learn n, and this takes $2(n-1)$ rounds. Call $2(n-1)$ rounds a **block**.
2. Write your label in binary, write every bit twice, and append 0 1. For example $5 = 101$ becomes 11 00 11 01.
3. Go through this string one bit per block: on a bit 1, explore the tree and return to your start node; on a bit 0, wait at your start node for the whole block.

Why it works and what it costs. Both agents compute the same block length, so their blocks start and end in the same rounds. Two different labels give strings that differ at some position before the shorter one ends (the final 0 1 marks the end, and no doubled bit looks like it). In that block one agent explores while the other waits at its start node, so the exploring agent walks onto it. The cost is $O(\log \lambda)$ blocks of $O(n)$ rounds, $O(n \log \lambda)$ in all. (Dessmark et al. 2006) reach $O(n + \log \lambda)$ on trees even with a delay τ, which is optimal; on general graphs the best known is $\tilde{O}(n^5 \log \lambda)$ (Ta-Shma and Zwick 2014), against a lower bound of $\Omega(n^2)$.

Why it matters. Turning distinct labels into different move/wait patterns is the basic symmetry-breaking device of deterministic rendezvous.

2. Impossibility: Identical agents in symmetric positions. (Czyzowicz et al. 2012b) (Anonymous agents: memory, p. 107) Take a ring of 6 nodes where at every node port 0 leads clockwise and port 1 counterclockwise. Put two agents without labels (**anonymous**: same algorithm, same initial memory) on opposite nodes, 3 edges apart, starting in the same round. Each agent sees exactly what the other sees: a node of **degree** 2 (two edges at it), the same entry ports, nobody else around. So in every round they make the same choice: both stay, both go clockwise, or both go counterclockwise.

Why it holds. After any number of rounds they are still exactly 3 edges apart, so they never meet, whatever the algorithm and however much **memory** (bits kept between rounds) they have. Identical

Claude (Anthropic) was used in cataloguing the problems.

agents can meet only when their starting positions are not symmetric in this sense, and for those positions $\Theta(\log n)$ bits of memory are exactly what is needed (Czyzowicz et al. 2012b).

Why it matters. It explains why most rendezvous work gives the agents labels, and why for identical agents the question becomes which starting positions can be told apart.

3. Lower bound: Two nodes and the length of a label. (Dessmark et al. 2006) (Labelled agents, arbitrary finite graphs, p. 105) Take the smallest graph: two nodes joined by one edge, one agent at each end, both starting together. Until they meet, an agent sees nothing new (always degree 1, always port 0), so its behaviour in the first t rounds is a string of t choices, stay or move, fixed by its label alone. They meet in a round where exactly one of them moves; if both move they just swap ends.

Why it holds. There are only 2^t strings of t choices. Among $2^t + 1$ labels, two give the same string; give these labels to the two agents and they do the same thing in each of the first t rounds, so they do not meet within t rounds. Hence some pairs of labels need about log (number of labels) rounds, which gives the $\Omega(\log \lambda)$ bound of (Dessmark et al. 2006). Together with the $\Omega(n)$ needed to cover the graph it shows their $O(n + \log \lambda)$ algorithm for trees is optimal.

Why it matters. It shows that the cost of breaking symmetry is the length of a label, a counting argument that recurs in every model with identifiers.

What the community started looking at

Deterministic rendezvous in graphs started with the question of **feasibility and cost polynomial in the graph size and the label length**, for labelled agents that may start at different times (Dessmark et al. 2006). The community then removed the dependence on the delay (Kowalski and Malinowski 2008), made the algorithms constructive (Ta-Shma and Zwick 2014), asked how little **memory** anonymous agents need (Czyzowicz et al. 2012b) (Fraigniaud and Pelc 2013), and moved to asynchronous agents, where even feasibility in every graph took years (De Marco et al. 2006) (Czyzowicz et al. 2012a) (Dieudonné et al. 2015). Pelc's survey records this evolution (Pelc 2019).

Relations to other problems

- **Subproblem of Graph exploration by a single agent.** If the agents can break symmetry (for example by their labels), one may wait while the other explores, so exploration solves rendezvous; the work is in doing better than that, or in meeting when neither knows who should wait (Pelc 2019).
- **Derived problem: Gathering in graphs** (generalisation of). Rendezvous is gathering of two agents; with more agents the questions become which teams can gather, and how many faulty agents a team tolerates.

Assumptions specific to this problem

Graph size

Deterministic rendezvous in graphs was posed for finite graphs, where an agent can afford to explore the whole graph and the cost is a function of n (Dessmark et al. 2006) (Pelc 2019). Bhagat and Pelc argued that this approach "is inefficient for very large graphs, and cannot be used for infinite graphs", and gave the first synchronous algorithm for connected graphs with finite degrees and finitely or countably many nodes, with time polynomial in d, $\log L$ and the number of paths of length d from the starting nodes, a quantity that also bounds rendezvous from below in trees (Bhagat and Pelc 2022). For asynchronous

Claude (Anthropic) was used in cataloguing the problems.

agents, Czyzowicz, Labourel and Pelc had already shown that rendezvous is feasible in every connected countable graph (Czyzowicz et al. 2012a) (as summarised in Pelc 2019); like all earlier work it assumed finite degrees (Pattanayak and Pelc 2024b). Pattanayak and Pelc then allowed countably infinite degrees, where the path count is infinite, and obtained a universal synchronous algorithm with time polynomial in a weight of the path types between the starting nodes and in $\log L$, matched up to a polynomial by a lower bound in the tree whose degrees are all infinite (Pattanayak and Pelc 2024b). The axis thus separates three cost measures: n, the local path count, and path-type weights.

Level	Value	Formal definition
0	**infinite degrees allowed**	G is finite or countably infinite; degrees may be infinite.
1	**locally finite**	G is finite or countably infinite with finite degrees.
2	**finite graph**	G has finitely many nodes.

Start

Dessmark, Fraigniaud, Kowalski and Pelc treated two startup scenarios for deterministic rendezvous: simultaneous startup, and an arbitrary delay τ chosen by the adversary, in which each agent counts rounds from its own wake-up without knowing τ (Dessmark et al. 2006). The delay is the stronger adversary: it can delay one agent long enough that the other must find it while it sleeps, which is treasure hunt (Pelc 2019) (Ta-Shma and Zwick 2014). It changes cost: on a ring simultaneous start gives $\Theta(d \log \lambda)$, while with arbitrary delay $\Omega(n + d \log \lambda)$ is a lower bound even when n is known; and the first polynomial algorithm for arbitrary graphs had cost growing with $\sqrt{\tau}$, leaving open whether the dependence could be removed (Dessmark et al. 2006), which Kowalski and Malinowski and then Ta-Shma and Zwick did (Kowalski and Malinowski 2008) (Ta-Shma and Zwick 2014) (the first as summarised in Pelc 2019). It also changes memory: Fraigniaud and Pelc showed that identical agents need $\Theta(\log n)$ bits in trees with arbitrary delay, even on a line, but only $O(\log \nu + \log \log n)$ bits with simultaneous start in trees with ν leaves, an exponential gap when ν is small (Fraigniaud and Pelc 2013).

Level	Value	Formal definition
0	**any delay**	An adversary wakes the agents at times t_1 and t_2; the delay $\tau = \lvert t_1 - t_2 \rvert$ is unknown.
1	**simultaneous start**	Both agents start in the same round ($\tau = 0$).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Memory) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Graph size → Identifiers → Timing → Start → Topology.

No assumptions

+ static graph, + undirected

Graph size

+ finite graph

Asynchronous anonymous agents ●● p. 104

+ distinct IDs (eponymous)

Asynchronous labelled agents ●● p. 105

+ synchronous

Labelled agents, arbitrary finite graphs STANDARD SETTING ●● p. 105

Claude (Anthropic) was used in cataloguing the problems.

any delay, + tree
Labelled agents in trees ●● p. 106
+ simultaneous start, + ring
Labelled agents in rings, simultaneous start ●● p. 107
anonymous, + synchronous
Anonymous agents: memory ●● p. 107
+ simultaneous start, + tree
Anonymous agents in trees, simultaneous start ●● p. 108
infinite degrees allowed, + distinct IDs (eponymous), + synchronous
Labelled agents, graphs with infinite degrees ●● p. 108

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Memory	p.
Asynchronous anonymous agents	● unknown *finite* vs —	● unknown — vs —	104
Asynchronous labelled agents	● unknown $\mathrm{poly}(n, \log \lambda)$ vs —	● unknown — vs —	105
Labelled agents, arbitrary finite graphs STANDARD SETTING	● polynomial gap $\tilde{O}(n^5 \log \lambda)$ vs $\Omega(n^2)$	● unknown — vs —	105
Labelled agents in trees	● tight $O(n + \log \lambda)$ vs $\Omega(n + \log \lambda)$	● unknown — vs —	106
Labelled agents in rings, simultaneous start	● tight $O(d \log \lambda)$ vs $\Omega(d \log \lambda)$	● unknown — vs —	107
Anonymous agents: memory	● unknown — vs —	● tight $O(\log n)$ vs $\Omega(\log n)$	107
Anonymous agents in trees, simultaneous start	● unknown — vs —	● tight $O(\log \nu + \log \log n)$ vs $\Omega(\log \nu + \log \log n)$	108
Labelled agents, graphs with infinite degrees	● polynomial gap $O((W \log L)^3)$ vs $\Omega(\max(W, \log L))$	● unknown — vs —	108

16.3.1 Asynchronous anonymous agents

Adds + finite graph VERIFIED

Which pairs of starting nodes allow identical asynchronous agents to meet is characterised.

Assumed: graph dynamics: static graph; edge direction: undirected; graph size: finite. **Not assumed:** identifiers: anonymous: all agents are identical; timing: asynchronous; start: arbitrary delay between the two starts; topology: arbitrary connected graph.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known Cost not studied.	*finite* (Guilbault and Pelc 2011)	• unknown
Memory	none known Memory is unbounded.	—	• unknown

Results.

- Guilbault and Pelc (2011) [current best]: time *finite*. Introductory and current best. Feasible iff the views differ or the two nodes are joined by a path whose port sequence is a palindrome (as summarised in Pelc 2019); randomized agents meet with probability 1.

16.3.2 Asynchronous labelled agents

Adds + distinct IDs (eponymous) **to** Asynchronous anonymous agents (p. 104) VERIFIED

An adversary controls speeds; agents may meet inside an edge. Feasible everywhere, and at polynomial cost, but with very large exponents.

Assumed: graph dynamics: static graph; edge direction: undirected; graph size: finite; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers. **Not assumed:** timing: asynchronous; start: arbitrary delay between the two starts; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound recorded for general graphs.	poly($n, \log \lambda$) (Dieudonné et al. 2015)	• unknown
Memory	none known Memory is unbounded.	—	• unknown

Results.

- De Marco et al. (2006) [introductory]: time *finite*. *Assumes:* an upper bound on the size is known. On a ring of known size, cost $O(n \log \lambda)$, which is optimal.
- Czyzowicz et al. (2012a) [also]: time *finite*. Feasible in every connected countable graph without any knowledge; cost at least exponential.
- Dieudonné et al. (2015) [current best]: time poly($n, \log \lambda$). First polynomial cost.

Open questions.

Q1. Find an optimal-cost asynchronous rendezvous algorithm in arbitrary graphs (open problem of Pelc 2019).

16.3.3 Labelled agents, arbitrary finite graphs

Adds + synchronous **to** Asynchronous labelled agents (p. 105) STANDARD SETTING DRAFT

Polynomial in n and $\log \lambda$ and independent of the delay, but far from the known lower bounds.

Assumed: graph dynamics: static graph; edge direction: undirected; graph size: finite; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; timing: synchronous rounds. **Not assumed:** start: arbitrary delay between the two starts; topology: arbitrary connected graph.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(n^2)$ (Dessmark et al. 2006); $\Omega(\log \lambda)$ (Dessmark et al. 2006) $\tilde{O}(n^5 \log \lambda)$ versus $\Omega(n^2 + \log \lambda)$.	$\tilde{O}(n^5 \log \lambda)$ (Ta-Shma and Zwick 2014)	• polynomial gap
Memory	none known Memory is unbounded in these algorithms.	—	• unknown

Results.

- Dessmark et al. (2006) [lower bound]: time $\Omega(n^2)$. *Bottleneck:* exploration. *Holds when:* Theorem 4.4: for all n and all pairs of labels, a regular graph made of two cliques of $n/2$ nodes joined by two swapped edges, ports equal at both ends of each edge; holds even with simultaneous start.
- Dessmark et al. (2006) [lower bound]: time $\Omega(\log \lambda)$. *Bottleneck:* symmetry breaking. *Holds when:* the two-node graph, agents at both ends, simultaneous start: with fewer steps than label bits two labels produce the same move/stay sequence; extends to trees (Theorem 2.3).
- Dessmark et al. (2006) [introductory]: time $O(n^5\sqrt{\tau \log \lambda}\, \log n + n^{10} \log^2 n \log \lambda)$. *Assumes:* memory unbounded; local computation free. Theorem 4.3: polynomial in n, τ and $\log \lambda$. Not constructive in the computational sense: each agent finds, by exhaustive local search, a combinatorial object whose existence is proved by the probabilistic method; only moves are counted.
- Kowalski and Malinowski (2008) [also]: time $O(\log^3 \lambda + n^{15} \log^{12} n)$. *Assumes:* backtracking: the agent learns the port by which it entered. First bound independent of τ (as stated in Pelc 2019); needs backtracking (as stated in Ta-Shma and Zwick 2014).
- Ta-Shma and Zwick (2014) [current best]: time $\tilde{O}(n^5 \log \lambda)$. Independent of τ and needs no backtracking, but not explicit: it relies on universal traversal sequences that are only known to exist. Checked in the paper. (Xin 2007 wrongly says it uses backtracking.)

Open questions.

Q1. Find an optimal-time synchronous rendezvous algorithm for labelled agents in arbitrary graphs (open problem of Pelc 2019).

Notes. ℓ in (Ta-Shma and Zwick 2014) is the length of the shorter label, our $\log \lambda$. Checked against the full text of (Dessmark et al. 2006): there agents know only their own label, learn the entry port and the degree at each node, and do not notice crossing inside an edge. With **simultaneous** start the same paper meets in $O(d\Delta^d \log \lambda)$ rounds in any graph without knowing Δ (Theorem 4.1), optimal when d and Δ are bounded. **Unverified against the original paper:** the bound of (Kowalski and Malinowski 2008) (from Pelc 2019); (Ta-Shma and Zwick 2014) was read in full.

16.3.4 Labelled agents in trees

Adds any delay, + tree **to** Labelled agents, arbitrary finite graphs (p. 105) VERIFIED

In trees, $O(n + \log \lambda)$ even with arbitrary delay, and this cannot be improved.

Assumed: graph dynamics: static graph; edge direction: undirected; graph size: finite; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; timing: synchronous rounds; topology: tree. **Not assumed:** start: arbitrary delay between the two starts.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(n + \log \lambda)$ (Dessmark et al. 2006) $\Theta(n + \log \lambda)$.	$O(n + \log \lambda)$ (Dessmark et al. 2006)	• tight
Memory	none known Memory is unbounded.	—	• unknown

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Dessmark et al. (2006) [lower bound]: time $\Omega(n + \log \lambda)$. *Bottleneck:* distance. *Holds when:* Theorem 2.3: two stars of $n/2 - 1$ leaves whose centres are joined, agents at the centres, for the $\Omega(n)$ part; the two-node argument for the $\Omega(\log \lambda)$ part; holds even with simultaneous start.
- Dessmark et al. (2006) [current best]: time $O(n + \log \lambda)$. *Assumes:* agents know that the graph is a tree, but not its size. Introductory and current best (Theorem 2.2), with arbitrary delay: explore by depth-first search, go to the central node or the central edge, and on a central edge run Procedure Extend-Labels, which meets on K_2 within $2 \log \lambda + 6$ rounds.

Notes. Checked against the full text of (Dessmark et al. 2006).

16.3.5 Labelled agents in rings, simultaneous start

Adds + simultaneous start, + ring **to** Labelled agents, arbitrary finite graphs (p. 105) VERIFIED

With simultaneous start on a ring, rendezvous takes $\Theta(d \log \lambda)$ for initial distance d.

Assumed: graph dynamics: static graph; edge direction: undirected; graph size: finite; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; timing: synchronous rounds; start: simultaneous start; topology: ring or path. **Not assumed:** —.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(d \log \lambda)$ (Dessmark et al. 2006) $\Theta(d \log \lambda)$.	$O(d \log \lambda)$ (Dessmark et al. 2006)	• tight
Memory	none known Memory is unbounded.	—	• unknown

Results.

- Dessmark et al. (2006) [lower bound]: time $\Omega(d \log \lambda)$. *Bottleneck:* symmetry breaking. *Holds when:* rings, for every initial distance d (Theorem 3.1); agents know that the graph is a ring.
- Dessmark et al. (2006) [current best]: time $O(d \log \lambda)$. *Assumes:* agents know that the graph is a ring, but not its size. Introductory and current best (Lemmas 3.1, 3.2 and Theorem 3.1).

Notes. Checked against the full text of (Dessmark et al. 2006). With **arbitrary delay** on an n-node ring, $\Omega(n + d \log \lambda)$ rounds are necessary even when n is known (Proposition 3.1); $O(n \log \lambda)$ suffices for known n (Theorem 3.2) and $O(\lambda\tau + \lambda n^2)$ for unknown n (Theorem 3.3) (Dessmark et al. 2006). The paper announces that a forthcoming paper reaches $O(n \log \lambda)$ with arbitrary delay; as stated in (Pelc 2019), this is (Kowalski and Malinowski 2008), for unknown n.

16.3.6 Anonymous agents: memory

Adds anonymous, + synchronous **to** Asynchronous anonymous agents (p. 104) VERIFIED

Identical agents can meet whenever their positions are not symmetric, and $\Theta(\log n)$ bits are exactly what it takes.

Assumed: graph dynamics: static graph; edge direction: undirected; graph size: finite; timing: synchronous rounds. **Not assumed:** identifiers: anonymous: all agents are identical; start: arbitrary delay between the two starts; topology: arbitrary connected graph.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known Not the focus.	—	● unknown
Memory	$\Omega(\log n)$ (Czyzowicz et al. 2012b) $\Theta(\log n)$ bits.	$O(\log n)$ (Czyzowicz et al. 2012b)	● tight

Results.

- Czyzowicz et al. (2012b) [lower bound]: memory $\Omega(\log n)$. *Bottleneck:* symmetry breaking. *Holds when:* rings; holds even with simultaneous start.
- Czyzowicz et al. (2012b) [current best]: memory $O(\log n)$. Introductory and current best; any delay; also gives the first polynomial-time computation of a quotient graph.

Notes. Rendezvous is feasible exactly when the two starting nodes have different views.

16.3.7 Anonymous agents in trees, simultaneous start

Adds + simultaneous start, + tree **to** Anonymous agents: memory (p. 107) VERIFIED

With simultaneous start, memory drops to $\Theta(\log v + \log\log n)$ for trees with v leaves; with delay it is $\Theta(\log n)$, an exponential gap.

Assumed: graph dynamics: static graph; edge direction: undirected; graph size: finite; timing: synchronous rounds; start: simultaneous start; topology: tree. **Not assumed:** identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Not the focus.	—	● unknown
Memory	$\Omega(\log v + \log\log n)$ (Fraigniaud and Pelc 2013) Matching bounds.	$O(\log v + \log\log n)$ (Fraigniaud and Pelc 2013)	● tight

Results.

- Fraigniaud and Pelc (2013) [lower bound]: memory $\Omega(\log v + \log\log n)$. *Bottleneck:* symmetry breaking. *Holds when:* trees of maximum degree 3; v = number of leaves.
- Fraigniaud and Pelc (2013) [current best]: memory $O(\log v + \log\log n)$. Introductory and current best. With arbitrary delay, $\Omega(\log n)$ even on lines.

16.3.8 Labelled agents, graphs with infinite degrees

Adds infinite degrees allowed, + distinct IDs (eponymous), + synchronous VERIFIED

A universal algorithm works even when nodes have infinite degree; it is within a cubic factor of the lower bound.

Assumed: graph dynamics: static graph; edge direction: undirected; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; timing: synchronous rounds. **Not assumed:** graph size: finite or countably infinite, degrees may be infinite; start: arbitrary delay between the two starts; topology: arbitrary connected graph.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(\max(W, \log L))$ (Pattanayak and Pelc 2024b) Cubic.	$O((W \log L)^3)$ (Pattanayak and Pelc 2024b)	• polynomial gap
Memory	none known Memory is unbounded.	—	• unknown

Results.

- Pattanayak and Pelc (2024b) [lower bound]: time $\Omega(\max(W, \log L))$. *Bottleneck:* port addressing. *Holds when:* $W = \max(w(v_1, v_2), w(v_2, v_1))$, the path weight of the treasure-hunt variant; e.g. the infinite tree with all degrees infinite.
- Pattanayak and Pelc (2024b) [current best]: time $O((W \log L)^3)$. Introductory and current best (Theorem 3.1).

Open questions.

Q1. Find an optimal rendezvous algorithm, "even for finite graphs" (Pattanayak and Pelc 2024b).

16.4 Gathering in graphs

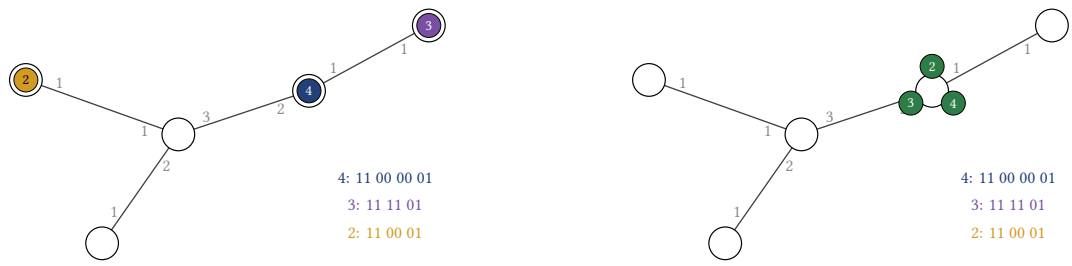
Several agents must all meet at one node of an unknown graph, possibly despite faulty agents.

First posed in 2012 (Dieudonné et al. 2014). Gathering many agents in arbitrary graphs grew out of two-agent rendezvous; the Byzantine version was posed at SODA 2012.

Problem statement and measures

$k \geq 2$ agents start at different nodes of an unknown connected graph and must all be at the same node at the same time, and (with **detection**) know it. **Time** is in rounds; **team** is the number of good agents needed when some agents are **Byzantine** (faulty agents that move arbitrarily and may lie).

Measures: **Time**, rounds until all agents meet; **Team**, good agents needed when f agents are Byzantine.



(a) Start: agents 2, 3, 4 at distinct nodes of a tree

(b) Final: leader 3 walks onto the waiting group of 4 (round 25)

Figure 16.4: Gathering of labelled agents by merging rendezvous (Dessmark et al. 2006): each leader runs the explore-or-wait rendezvous of a tree with its own label (after one tour to learn n, one block of $2(n-1)$ rounds per bit of its doubled label followed by 01: explore on 1, wait at its start node on 0); when groups meet, the largest label stays leader and the others follow it. Colours show the leader each agent follows; one frame per round. Gathering is reached, but without detection.

Claude (Anthropic) was used in cataloguing the problems.

A first look

A group of hikers scattered over a trail network wants to get together at one spot, with no phones and no map. The network is a **graph**: trail junctions are **nodes** and the trails between them are **edges**; the hikers are $k \geq 2$ **agents** that start at different nodes of this unknown connected graph. At each node the edges at that node carry local numbers, its **ports**. Time passes in synchronous **rounds**, and in each round an agent stays or moves along one edge. All agents must be at the same node at the same time and, in the version **with detection**, know that everyone has arrived. In the basic version the agents are **anonymous** (identical: same algorithm, no **labels**, that is, no distinct integers that tell them apart), there are no faults, and n denotes the number of nodes; the question is which starting **configurations** (the sets of nodes on which the agents start) can be gathered, and how fast.

1. Algorithm: Meet in pairs, then follow the largest label. (Labelled agents, p. 112) A standard approach for agents with distinct labels that can read each other's label when they are at the same node. It uses any rendezvous algorithm for two labelled agents (an algorithm that makes two agents meet at one node) that works whatever the delay between their starts, such as the one of (Dessmark et al. 2006).

1. Every agent starts as the **leader** of a **group** containing only itself, and runs the rendezvous algorithm with its own label.
2. When groups meet at a node, everyone there adopts the largest label present; the agent with that label remains a leader and continues its own rendezvous algorithm as if nothing had happened.
3. Every other agent stops being a leader and from now on simply follows its leader's moves.

Why it works and what it costs. A leader's walk is exactly what it would do alone, so any two leaders are guaranteed to meet by the rendezvous algorithm, and at each such meeting one of them stops being a leader. The agent with the largest label is never absorbed, and every other leader meets its group at the latest when it meets that agent, so eventually all k agents move as one group. The time is at most $k-1$ rendezvous times. This gives gathering but not detection: knowing that nobody is missing needs extra knowledge, for example of n; with more than $n/2$ agents and n known, (Molla et al. 2023) gather with detection in $O(n^3)$ rounds.

Why it matters. Reducing gathering to repeated rendezvous with merging is the simplest way to see why distinct labels make gathering solvable.

2. Impossibility: Symmetric configurations of identical agents. (Dieudonné and Pelc 2016) (Anonymous agents, p. 112) Take a ring of 6 nodes, with port 0 leading clockwise and port 1 counterclockwise at every node, and put three identical agents on every second node. Starting together, each agent sees exactly what the others see, so in every round all three make the same choice. Turning the ring by two nodes maps the configuration to itself before the round and after it.

Why it holds. So the agents always stay on three different nodes, two edges apart, and never gather, whatever the algorithm. For identical agents the question is therefore which configurations can be gathered at all; (Dieudonné and Pelc 2016) characterise them and gather every such configuration in time polynomial in n, but detecting that gathering is complete needs an upper bound on n.

Why it matters. It shows that for identical agents the right question is which starting configurations are solvable, not only how fast.

3. Impossibility: Byzantine agents can impersonate the whole team. (Dieudonné et al. 2014) (Byzantine agents, p. 114) Some agents may be **Byzantine**: faulty agents that move arbitrarily and, in the strong version, may lie about their labels. Take $f = 2$ good agents, labelled 0 and 1, and 2 Byzantine ones, on a ring of 4 nodes v_0, v_1, v_2, v_3 with port 0 leading clockwise everywhere. **Run 1:** good agents 0

Claude (Anthropic) was used in cataloguing the problems.

and 1 on v_0, v_1; the two Byzantine agents on v_2, v_3 claim labels 0 and 1 and follow the algorithm honestly. The configuration is symmetric, so when the good agents gather at some node v_i and stop, the Byzantine agents stop at the opposite node v_{i+2}. **Run 2:** the same, except good agent 0 and the Byzantine agent claiming label 0 swap starting nodes.

Why it holds. Every agent sees in run 2 exactly what it saw in run 1, so everyone does the same thing; at the end good agent 1 stands with the Byzantine agent claiming label 0 on one node and good agent 0 stands with the Byzantine agent claiming label 1 on the opposite node, and both groups stop. The good agents are not gathered, so with f strongly Byzantine agents, f good agents cannot gather, even if n is known (Dieudonné et al. 2014) (their Theorem 4.7, for every f, on a ring of $2f$ nodes). With $f+1$ good agents (n known) or $f+2$ (n unknown) gathering becomes possible (Bouchard et al. 2016).

Why it matters. This "two indistinguishable runs" argument is the standard way to prove that faulty agents make a task impossible, and it shows how many good agents a fault-tolerant algorithm must assume.

What the community started looking at

Gathering of many agents in graphs was studied as the natural extension of rendezvous (Pelc 2019): first **which configurations can gather** at all (Dieudonné and Pelc 2016), then **how many faulty agents** a team can tolerate (Dieudonné et al. 2014) (Bouchard et al. 2016), and only then time (Bouchard et al. 2022) (Molla et al. 2023).

Relations to other problems

- **Generalisation of Rendezvous.** Rendezvous is gathering of two agents; with more agents the questions become which teams can gather, and how many faulty agents a team tolerates.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Team) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Faults → Identifiers → Timing → Topology.

No assumptions

+ static graph, + undirected

Faults

+ no faults

Identifiers

anonymous, + synchronous
Anonymous agents STANDARD SETTING ●● p. 112

+ distinct IDs (eponymous), + synchronous
Labelled agents ●● p. 112

+ homonymous, + synchronous
Homonymous agents ●● p. 113

+ at most weak Byzantine faults, + distinct IDs (eponymous), + synchronous
Weakly Byzantine agents ●● p. 113

any faults, + distinct IDs (eponymous), + synchronous
Byzantine agents ●● p. 114

Claude (Anthropic) was used in cataloguing the problems.

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Team	p.
Anonymous agents STANDARD SETTING	● unknown poly(n) vs *impossible*	● unknown — vs —	112
Labelled agents	● unknown $O(n^3)$ vs —	● unknown — vs —	112
Homonymous agents	● unknown poly($n, \log \lambda$) vs —	● unknown — vs —	113
Weakly Byzantine agents	● unknown poly($n, \log L$) vs —	● tight $f+2$ vs $f+2$	113
Byzantine agents	● unknown poly($n, \log \lambda$) vs —	● tight $f+1$ vs $f+1$	114

16.4.1 Anonymous agents

Adds anonymous, + synchronous STANDARD SETTING VERIFIED

Which configurations of identical agents can gather is characterised; detecting that gathering is complete needs a bound on n.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: no faults; timing: synchronous rounds. **Not assumed:** identifiers: anonymous: all agents are identical; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Dieudonné and Pelc 2016) Polynomial; exponent not tracked.	poly(n) (Dieudonné and Pelc 2016)	● unknown
Team	none known Not a team-size question here.	—	● unknown

Results.

- Dieudonné and Pelc (2016) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* gathering with detection by a universal algorithm when no bound on n is known.
- Dieudonné and Pelc (2016) [current best]: time poly(n). *Assumes:* an upper bound on n (for detection). Introductory and current best; without a bound, gathering without detection in time polynomial in the actual size.

16.4.2 Labelled agents

Adds + distinct IDs (eponymous), + synchronous VERIFIED

Labelled agents gather even without talking; with many agents gathering with detection takes $O(n^3)$ rounds.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: no faults; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; timing: synchronous rounds. **Not assumed:** topology: arbitrary connected graph.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound recorded.	$O(n^3)$ (Molla et al. 2023)	• unknown
Team	none known —	—	• unknown

Results.

- Bouchard et al. (2023b) [also]: time *finite*. Gathering with no communication at all.
- Molla et al. (2023) [current best]: time $O(n^3)$. *Assumes:* more than $n/2$ agents; n known. Gathering with detection.

Open questions.

Q1. Gathering without communication when no bound on n is known.

Notes. Two-agent gathering is rendezvous. For Look–Compute–Move robots on rings see (Klasing et al. 2008).

16.4.3 Homonymous agents

Adds + homonymous, + synchronous VERIFIED

When agents may share labels, the teams that can always gather are fully characterised, and gather in time polynomial in n and $\log \lambda$.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: no faults; identifiers: homonymous: agents carry labels, but several may share a label; timing: synchronous rounds. **Not assumed:** topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known —	$\mathrm{poly}(n, \log \lambda)$ (Devismes et al. 2026)	• unknown
Team	none known —	—	• unknown

Results.

- Devismes et al. (2026) [current best]: time $\mathrm{poly}(n, \log \lambda)$. *Assumes:* $O(\log \log \log L)$ bits of common knowledge. Introductory and current best; λ is the smallest label, L the largest.

16.4.4 Weakly Byzantine agents

Adds + at most weak Byzantine faults, + distinct IDs (eponymous), + synchronous VERIFIED

With f weakly Byzantine agents, one good agent suffices when n is known and $f + 2$ are necessary and sufficient when it is not.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: up to f weak Byzantine agents (arbitrary behaviour, cannot fake identifiers); identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; timing: synchronous rounds. **Not assumed:** topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound on time beyond fault-free gathering is recorded.	$\mathrm{poly}(n, \log L)$ (Dieudonné et al. 2014)	• unknown
Team	$f + 2$ (Dieudonné et al. 2014) $f + 2$ for unknown n; 1 for known n.	$f + 2$ (Dieudonné et al. 2014)	• tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Dieudonné et al. (2014) [lower bound]: team $f+2$. *Bottleneck:* indistinguishable Byzantine agents. *Holds when: n* unknown; Theorem 3.11: $f+1$ good agents cannot gather deterministically with termination in some networks.
- Dieudonné et al. (2014) [current best]: time poly$(n, \log L)$, team $f+2$. *Assumes:* f known to all agents. Introductory and current best (Theorem 3.10, unknown n); time polynomial in n and in the length of the largest label of a good agent. With known n, any number of good agents gather within $4n^4 \cdot P(n, \log L)$ rounds (Theorem 3.6; P is the polynomial time bound of the rendezvous algorithm of Ta-Shma and Zwick Ta-Shma and Zwick 2014).

Notes. Checked against the full text of (Dieudonné et al. 2014). Dieudonné, Pelc and Peleg note that $f+2$ is a new threshold for tolerating f Byzantine entities, next to $f+1$ for communication through a cut and $2f+1$ processors for consensus in a complete network.

16.4.5 Byzantine agents

Adds any faults, + distinct IDs (eponymous), + synchronous VERIFIED

With f strongly Byzantine agents, $f+1$ good agents (known n) or $f+2$ (unknown n) are necessary and sufficient.

Assumed: graph dynamics: static graph; edge direction: undirected; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; timing: synchronous rounds. **Not assumed:** faults: up to f strong Byzantine agents (arbitrary behaviour, can fake identifiers); topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known Polynomial only for large teams.	poly$(n, \log \lambda)$ (Bouchard et al. 2022)	● unknown
Team	$f+1$ (Dieudonné et al. 2014); $f+2$ (Dieudonné et al. 2014) $f+1$ / $f+2$.	$f+1$ (Bouchard et al. 2016)	● tight

Results.

- Dieudonné et al. (2014) [lower bound]: team $f+1$. *Bottleneck:* indistinguishable Byzantine agents. *Holds when:* Theorem 4.7: f good agents cannot gather deterministically against f strongly Byzantine agents; holds even in a ring of known size.
- Dieudonné et al. (2014) [lower bound]: team $f+2$. *Bottleneck:* indistinguishable Byzantine agents. *Holds when: n* unknown; inherited from weakly Byzantine agents (Theorem 3.11).
- Dieudonné et al. (2014) [introductory]: team $2f+1$. *Assumes:* f known to all agents. Introductory (Theorems 4.5 and 4.6): $2f+1$ good agents with known n, $4f+2$ with unknown n; deterministic but exponential time. For known n and constant f a randomized algorithm gathers $2f+1$ good agents in polynomial time with probability at least $1-1/n$ (Theorem 4.4).
- Bouchard et al. (2016) [current best]: team $f+1$. $f+2$ when n is unknown; time exponential.
- Bouchard et al. (2022) [also]: time poly$(n, \log \lambda)$. *Assumes:* good agents at least quadratic in f; $O(\log \log \log n)$ bits of global knowledge. First polynomial-time Byzantine gathering. λ is the smallest label of a good agent (the paper's $|l_{min}|$ is $\log \lambda$ here).

Open questions.

Q1. Polynomial-time Byzantine gathering with teams of size close to $f+1$.

Notes. Checked against the full text of (Dieudonné et al. 2014) for its bounds; the agents cannot mark nodes, agents at the same node exchange all information, the adversary may wake agents at different

Claude (Anthropic) was used in cataloguing the problems.

times, and good agents must gather and terminate simultaneously. The paper's results for known n also hold with a known polynomial upper bound on n.

16.5 Map construction

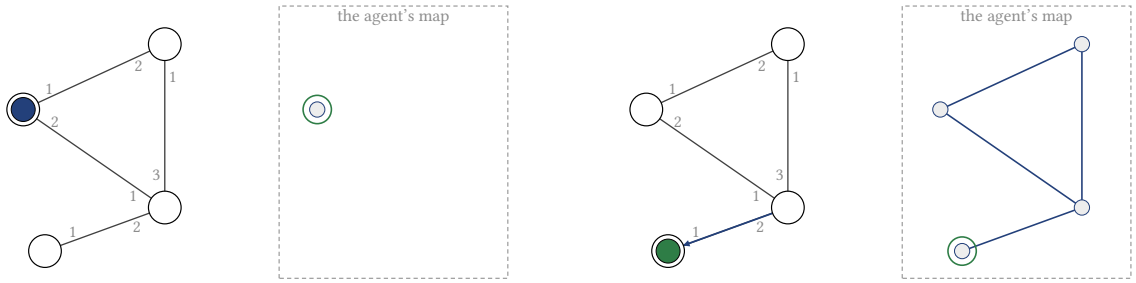
Agents must not only visit every node but output a map of the anonymous graph, up to isomorphism.

First posed in 1991 (Dudek et al. 1991). Dudek, Jenkin, Milios and Wilkes framed robotic exploration as graph construction.

Problem statement and measures

Agents explore an unknown anonymous port-labelled graph and must output a **map**: a port-labelled graph, kept in the agent's memory, that is isomorphic to the explored graph (with the ports), with the agent's current node indicated. **Time** is the number of moves.

Measures: **Cost**, moves until the map is built.



(a) Start: an unknown anonymous graph; the map holds the start node

(b) Final: the last node is new; every port tried, the map is complete

Figure 16.5: Map construction with one movable marker (Dudek et al. 1991) (simplified), a setting of the problem of (Chalopin et al. 2010): the agent crosses the edge at an untried port, drops the marker, steps back and tours the nodes of its map. If it finds the marker on a node already in the map, the edge leads there; otherwise the node is new, and the agent fetches the marker and adds the node and the edge to the map. Left: the unknown graph with its ports; right: the map, with the agent's position circled. One frame per move.

A first look

Imagine an explorer dropped into a maze of identical rooms joined by corridors, with no compass and no way to tell rooms apart; it must draw a correct map of the maze. Here the maze is a **graph** with n **nodes** (the rooms) and m **edges** (the corridors; each edge joins two nodes), and the explorer is an **agent** that walks from node to node along edges; one crossing of one edge is a **move**. The nodes are **anonymous**: they have no names the agent can read. At each node the edges at that node are numbered $1, 2, \ldots$ (these numbers are **ports**), so the agent can choose which edge to take and sees through which port it arrived. One agent starts at some node, its **start node**; it must walk around and finally output a **map**: a graph with port numbers, kept in the agent's memory, that is the same as the real graph up to renaming of the nodes, with the node where the agent stands indicated. We count the moves. In the basic catalogued version the agent cannot leave anything on a node, but it knows upper bounds on n and on the maximum **degree** Δ (the degree of a node is the number of edges at it).

Claude (Anthropic) was used in cataloguing the problems.

1. Algorithm: Drop a marker to recognise a node. (Dudek et al. 1991) (One agent with a marker, p. 120) The agent carries one **marker**, an object that it can put down on a node and pick up again (a simplified description of Dudek et al. 1991).

1. Keep a map of the nodes seen so far; at first it holds only the start node.
2. Choose a node of the map with a port that has not been tried yet. Walk to that node of the graph, following the map, and cross the edge at that port.
3. Put the marker down on the node of the graph just reached, step back across the same edge, and visit every node of the graph that is in the map (walking along edges of the map).
4. If the marker is found on the node of the graph that a node u of the map stands for, the new edge leads to u: record that edge in the map, pick up the marker, and go to step 2.
5. If the marker is not found anywhere, the node is new: cross the edge again, pick up the marker, and add the node and the edge to the map.
6. Stop when every port of every node in the map has been tried.

Why it works and what it costs. Each edge is tried once from each end, so there are at most $2m$ tests. One test costs a walk to the edge plus a tour of the nodes in the map, and both are $O(n)$ moves if the agent walks along a tree inside its map. In total this is $O(mn)$ moves; (Dudek et al. 1991) count the moves precisely: with one marker, at most $7mn - 3n^2$ plus lower-order moves, and fewer with more markers. This is the current best bound recorded in the catalogue for the marker setting.

Why it matters. It showed that a single movable marker turns an unsolvable task into a polynomial one, and it started the study of how much marking or knowledge map construction needs.

2. Impossibility: Graphs that look the same from inside. (Chalopin et al. 2010) (One agent, no marking, p. 119) Take a **ring** (nodes joined in a circle, each node with two edges) and number the ports so that at every node port 1 goes clockwise and port 2 goes counterclockwise. Now compare a ring of 3 nodes with a ring of 6 nodes. An agent without a marker sees the same thing at every node of both rings: two ports, and whichever port it takes, it arrives through the other one. Every walk it can make produces exactly the same observations in the two rings. So it cannot tell the rings apart, and it cannot tell when it has seen every node.

Why it holds. In general, two nodes with the same **view** (everything an agent can observe on walks that start at the node: the ports it can take and the ports through which it arrives) cannot be told apart, and the most an agent can learn is the **quotient graph**, which merges the nodes with the same view into one node (here all nodes have the same view, so the quotient graph is a single node with one edge from the node to itself, port 1 at one end and port 2 at the other). Without a marked node, (Chalopin et al. 2010) note that no algorithm can explore graphs of arbitrary size and stop, so the agent needs extra knowledge (bounds on n and Δ) to stop, and then it builds the quotient graph, which equals the real graph only when all views are distinct.

Why it matters. It explains why every map construction algorithm needs something extra: a marker, a recognisable start node, or knowledge of the size.

3. Algorithm: Recognise a node by walking home. (Chalopin et al. 2010) (One agent, no marking, p. 119) Suppose only the start node s is marked: the agent can recognise s when it stands on it, and nothing else.

1. Build the map breadth-first: for every node u already in the map, store the list of ports of a path from u back to s.
2. To test a port that has not been tried yet, cross its edge and arrive at some node v of the graph.

Claude (Anthropic) was used in cataloguing the problems.

3. For each node u in the map: from v, follow the stored port list of u and see whether the walk ends on the marked node s; then walk back to v.
4. If some list ends at s, then v is u: record the edge in the map. Otherwise v is new, and the agent stores a path home for it (the edge it just crossed followed by the path of the node it came from).
5. Stop when every port has been tested.

Why it works and what it costs. The test is correct because a list of ports, read backwards from s, leads to exactly one node; if it leads home from both u and v, then $u = v$. There are at most $n\Delta$ ports to test, each test tries up to n stored paths, and each path has fewer than n moves, so the cost is $O(n^3\Delta)$ moves, the bound of (Chalopin et al. 2010) (Proposition 9). This needs no knowledge of n, and the output is an exact copy of the graph. Without any marked node the same paper needs known bounds and universal exploration sequences, with $O(N^6\Delta^2 \log N)$ moves, where N is the known upper bound on n.

Why it matters. It shows that a marked start node, which cannot be moved, is already enough, which is much weaker than a marker the agent carries.

What the community started looking at

Map construction started in robotics as **exploration as graph construction** with markers (Dudek et al. 1991); in distributed computing the questions were **feasibility** for anonymous agents (Das et al. 2007) and then the cost for a single agent without marks (Chalopin et al. 2010), as surveyed in (Das 2019).

Relations to other problems

- **Strengthening of Graph exploration by a single agent.** Building an isomorphic map implies having visited every node, so map construction is exploration with more output; it is used to solve rendezvous and leader election (Das 2019).

Assumptions specific to this problem

Agents

Map construction began as a single-robot problem: Dudek, Jenkin, Milios and Wilkes framed robotic exploration as building a graph, with a robot carrying one or more markers (Dudek et al. 1991). Das, Flocchini, Kutten, Nayak and Santoro posed the distributed version: k identical agents, initially dispersed among the nodes and communicating through whiteboards, must each build an identically labelled map (Das et al. 2007) (abstract read). They note that it "is much more difficult than exploration by a single agent, because it involves achieving cooperation among multiple agents", and that it is deterministically unsolvable in some cases (Das et al. 2007). The axis separates the single agent, whose difficulty is recognising nodes it has already seen, from the team, which must in addition break the symmetry between identical agents.

Level	Value	Formal definition
0	**one agent**	A single agent.
1	**several agents**	$k \geq 2$ identical agents.

Knowledge

In an anonymous graph an agent that cannot mark nodes cannot tell a revisited node from a new one by looking at it, so it needs information from outside before it can commit to a map. The standard

Claude (Anthropic) was used in cataloguing the problems.

information is an upper bound on n, which fixes how long a universal sequence must be followed; Chalopin, Das and Kosowski state the trade-off exactly: "in the model where no vertices of the graph are marked, for the task of map construction to be feasible, the agents must know some bound n on the number of nodes of the graph and some bound $d \leq n$ on its degree", and with these bounds they build the map by following universal exploration sequences; without any knowledge they instead assume that the starting node is marked, and then build the map in $O(n^3\Delta)$ moves with $O(n\Delta \log n)$ bits of memory (Chalopin et al. 2010). The clearest measurement of what the bound is worth comes from directed graphs: Bender et al. map any strongly connected digraph with one pebble when an upper bound on n is known, while without it $\Theta(\log \log n)$ pebbles are necessary and sufficient (Bender et al. 2002).

Level	Value	Formal definition
0	**nothing known**	Nothing is known about G.
1	**bound on n**	An upper bound $N \geq n$ is known.

Markers

A marker lets a robot recognise one chosen node, which is exactly what an anonymous graph withholds. Dudek, Jenkin, Milios and Wilkes introduced the robot that carries $k \geq 1$ distinct markers and builds the graph as it explores; they show that without markers the task is unsolvable in general (regular graphs of the same degree look alike everywhere) and that their algorithm makes at most $(5 + 2/k)mn - 3n^2$ plus lower-order moves, that is $O(mn) \subseteq O(n^2\Delta)$ (Dudek et al. 1991). Bender, Fernández, Ron, Sahai and Vadhan showed in "The power of a pebble" that one pebble also suffices to map a directed graph in polynomial time when an upper bound on n is known (Bender et al. 2002). The axis keeps carried markers apart from whiteboards because a marker is a single movable bit, while a whiteboard stores information permanently at every node.

Level	Value	Formal definition
0	**no markers**	No markers.
1	**markers**	The agent carries markers it can drop and pick up.

Whiteboards

Whiteboards, a small memory at every node that visiting agents read and write in mutual exclusion, are the standard communication device in the work of Flocchini, Santoro and coauthors on asynchronous agents (Dobrev et al. 2006) (Flocchini et al. 2012b). Das, Flocchini, Kutten, Nayak and Santoro used them for map construction by k identical agents, which communicate only through these "small public bulletin boards available at each node" (Das et al. 2007) (abstract read). Flocchini, Ilcinkas and Santoro note that whiteboards also "allow to break symmetry among identical agents" (Flocchini et al. 2012b), which is what a team of identical agents needs in order to agree on one labelled map.

Level	Value	Formal definition
0	**no whiteboards**	No whiteboards.
1	**whiteboards**	Each node has a whiteboard.

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Cost) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Timing → Agents → Knowledge → Markers → Whiteboards → Topology.

No assumptions

+ static graph, + undirected

Agents

one agent, + bound on n

One agent, no marking STANDARD SETTING • p. 119

one agent, nothing known, + markers

One agent with a marker • p. 120

+ several agents, + whiteboards

Identical agents with whiteboards • p. 121

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Cost	p.
One agent, no marking STANDARD SETTING	• unknown $O(N^6\Delta^2 \log N)$ vs —	119
One agent with a marker	• unknown $O(mn)$ vs —	120
Identical agents with whiteboards	• unknown *finite* vs *impossible*	121

16.5.1 One agent, no marking

Adds one agent, + bound on n STANDARD SETTING VERIFIED

Knowing upper bounds on n and on the degree, one agent builds the quotient graph of an anonymous graph without marking anything, by following a universal exploration sequence.

Assumed: graph dynamics: static graph; edge direction: undirected; knowledge: an upper bound on n. **Not assumed:** timing: asynchronous; agents: a single agent; markers: no markers; whiteboards: no whiteboards; topology: arbitrary connected graph.

Statement for this setting. The agent knows an upper bound $N \geq n$ (and a bound on the maximum degree; $N - 1$ will do) and may not mark nodes. Following (Chalopin et al. 2010), it must output the **quotient graph** of G with its port numbering, the multigraph obtained by merging nodes with the same view; this is G itself exactly when all nodes have distinct views, and nothing more can be learned, since graphs with the same quotient graph are indistinguishable to the agent. **Time** is the number of edge traversals; local computation is free.

Measure	Lower bound	Best upper bound	Status
Cost	none known No lower bound is proved in (Chalopin et al. 2010).	$O(N^6\Delta^2 \log N)$ (Chalopin et al. 2010)	• unknown

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Chalopin et al. (2010) [current best]: cost $O(N^6\Delta^2 \log N)$. *Assumes:* upper bounds on n and on Δ known. Introductory and current best (Proposition 6): follow a universal exploration sequence $U(2N^2, \Delta)$ of length $O(N^6\Delta^2 \log N)$ from Aleliunas et al., record the signature, and compute the smallest graph with that signature; memory of the same order. The paper writes n and d for the known bounds. Using the routines of Czyzowicz, Kosowski and Pelc (Czyzowicz et al. 2012b) gives $O(n^{10}\Delta^5 \log^2 n)$ time with $O(n^9\Delta^4 \log^2 n)$ memory (as computed in Chalopin et al. 2010).
- Chalopin et al. (2010) [also]: cost $O(N^6\Delta^3 \log N)$. *Assumes:* upper bounds on n and on Δ known. Proposition 7 (class refinement, as in automaton minimisation): slightly more time for much less memory.
- Chalopin et al. (2010) [also]: cost $O(n^3\Delta)$. *Assumes:* the starting node is marked (recognisable); no bound on n needed. Proposition 9: breadth-first construction that recognises a node by walking its path back to the marked home base; the output is an exact copy of G. With a marked start and no knowledge, Proposition 8 also computes a polynomial bound $N \geq n$ with $\Theta(\log n)$ bits, so map construction is possible in polynomial time with optimal memory.

Notes. Checked against the full text of (Chalopin et al. 2010). Which bound needs what: without any marked node the agent must know bounds on n and on the degree (Propositions 6 and 7, $O(N^6\Delta^2 \log N)$ and $O(N^6\Delta^3 \log N)$); with a marked starting node no knowledge is needed (Propositions 8 and 9, $O(n^3\Delta)$ moves and $O(n\Delta \log n)$ bits). The paper writes that "it is not possible to perform exploration with stop in unlabelled graphs of arbitrary size and topology" without a marked node; a marked home base is "a much weaker assumption compared to allowing the agent to have a pebble". The same paper solves rendezvous of two agents with marked home bases, even asynchronously, in $O(n^4\Delta^2)$ moves per agent whenever it is deterministically possible, and detects when it is not (Propositions 10 and 11). The timing axis is not set because a single agent's moves are counted regardless of timing. (Das 2019) summarises that a full map is obtainable only for covering-minimal graphs; in the paper's terms, an exact copy is obtained exactly when all views are distinct. The $O(n^3\Delta)$ bound is the one Disser, Hackfeld and Klimm report (Disser et al. 2019).

16.5.2 One agent with a marker

Adds one agent, nothing known, + markers VERIFIED

An agent with $k \geq 1$ distinct markers it can drop and pick up builds the graph as it explores, in $O(mn)$ moves.

Assumed: graph dynamics: static graph; edge direction: undirected; markers: markers the agent carries. **Not assumed:** timing: asynchronous; agents: a single agent; knowledge: nothing known; whiteboards: no whiteboards; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Cost	none known	$O(mn)$ (Dudek et al. 1991)	● unknown
	No lower bound recorded; (Dudek et al. 1991) reports that experiments needed 25–65% of the bound.		

Results.

- Dudek et al. (1991) [current best]: cost $O(mn)$. *Assumes:* $k \geq 1$ distinct markers; the agent sees the edges at a node in a cyclic order relative to the edge it entered by. Introductory and current best recorded here: at most $(5 + 2/k)mn - 3n^2$ plus lower-order moves (equation (4)); a variant that marks both ends of an unexplored edge makes $(3 + 4/k)mn - n^2$ plus lower-order moves. The bound counts only physical moves; the agent's computation is $O(n^2m + m^2 \log m)$.

Notes. Checked against the full text of (Dudek et al. 1991) (a scan, read through OCR). The agent cannot measure distances and has no compass; without markers the problem is unsolvable in general, because

Claude (Anthropic) was used in cataloguing the problems.

regular graphs of the same degree look the same at every node. For directed graphs, one marker and a bound on n suffice, and without a bound $\Theta(\log \log n)$ pebbles are necessary and sufficient (Bender et al. 2002); Bender et al. also cite the $O(n \cdot m)$ bound of Dudek et al. for one marker.

16.5.3 Identical agents with whiteboards

Adds + several agents, + whiteboards DRAFT

Several identical agents with whiteboards build a common labelled map exactly when symmetry can be broken.

Assumed: graph dynamics: static graph; edge direction: undirected; agents: several identical agents; whiteboards: whiteboards at the nodes. **Not assumed:** timing: asynchronous; knowledge: nothing known; markers: no markers; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Cost	*impossible* (Das et al. 2007) Costs not checked.	*finite* (Das et al. 2007)	• unknown

Results.

- Das et al. (2007) [lower bound]: cost *impossible*. *Bottleneck:* symmetry. *Holds when:* some configurations of identical agents.
- Das et al. (2007) [current best]: cost *finite*. *Assumes:* n and k co-prime. Introductory and current best.

16.6 Exploration by oblivious robots

Oblivious Look–Compute–Move robots with vision must visit every node of a ring or grid and stop; how many robots are needed?

First posed in 2007 (Flocchini et al. 2013). Ring exploration by asynchronous oblivious robots (OPODIS 2007).

Problem statement and measures

k anonymous, oblivious robots run Look–Compute–Move cycles on an anonymous graph: they see where all robots are, compute, and move to a neighbour. They must visit every node and then stop (or, for perpetual exploration, keep visiting). The primary measure is the **number of robots**.

Measures: **Robots**, number of robots needed; **Moves**, total moves.

A first look

Imagine a few identical toy robots on a circular track of n squares. The track is a **ring**: its squares are its n **nodes**, and two squares side by side are **neighbours**. The robots cannot talk, they have no memory at all, and they cannot tell clockwise from counterclockwise, yet together they must visit every node and then stop forever. Each robot works in **Look–Compute–Move cycles**: **Look** (take a **snapshot** showing where all robots are), **Compute** (decide, from that snapshot only, whether to move and where) and **Move** (move to a neighbour). Robots are **oblivious**: they forget everything after each cycle. The nodes are **anonymous** (they have no names), and an **adversary** (an imagined opponent) decides when

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: 5 robots on distinct nodes, n = 8

(b) Walker reaches the group from the other side: every node visited, all stop

Figure 16.6: Exploration of a ring by five oblivious robots (Lamani et al. 2010) (the problem: Flocchini et al. 2013): anonymous robots without memory, on distinct nodes of an anonymous ring with $n = 8$ nodes (co-prime with 5), run Look–Compute–Move cycles, one robot activated per frame here. They first close the gaps into a block of five, then the middle robot forms a tower, which fixes a direction; one end robot, the walker, moves away from the tower round the ring, and when it is back beside the group the configuration means that every node (shaded) has been visited, and no robot moves again. The rules are simplified from (Lamani et al. 2010).

each robot is activated. A robot can tell whether several robots stand on one node (such a node is called a **tower**). The positions of all robots form the **configuration**. In the basic version k robots start on distinct nodes of a ring of n nodes; they must visit every node and then all stop. We count the robots needed.

1. Algorithm: Gather in a block, build a tower, send one walker. (Lamani et al. 2010) (Rings, exploration with stop, p. 124) Five robots, on a ring whose size n is not a multiple of 5 (Lamani et al. 2010). The rule a robot follows depends only on its current snapshot, so each stage must leave a configuration that announces which stage it is.

1. **Block:** while the robots are not on five consecutive nodes (a **block**), robots move closer together, carefully so that no two land on the same node.
2. **Tower:** when the five robots form one block, the middle robot moves onto a neighbour. Now there is exactly one tower, and the configuration is no longer symmetric: it shows a direction.
3. **Walk:** one robot, the **walker**, singled out by its position relative to the tower, moves node by node around the ring, away from the tower.
4. When the walker reaches the other side of the group, the robots form a special pattern (a *tower-chain*); this configuration means the ring has been explored, and nobody moves again.

Why it works and what it costs. The condition that n and 5 share no common factor is what lets the robots break symmetry in the first stage (see the third idea for what goes wrong otherwise). Each stage costs $O(n)$ moves, so the total is $O(n)$, which is optimal, since the walker must cover the ring. For even n, four robots cannot explore deterministically (Lamani et al. 2010), so five is optimal for those rings. The earlier algorithm of (Flocchini et al. 2013) needed 17 or more robots.

Why it matters. It shows the main technique of the area: memoryless robots store their progress in the shape of their own configuration.

2. Impossibility: The final configuration must contain a tower. (Devismes et al. 2012) (Grids, exploration with stop, p. 125) An oblivious robot decides only from its current snapshot. Suppose that the robots stop in a configuration in which every robot is on its own node. That configuration could also

Claude (Anthropic) was used in cataloguing the problems.

be a starting configuration, since robots start on distinct nodes. Started from it, the robots would see snapshots in which they stop, so they would stop at once, after visiting only their own k nodes, fewer than the n nodes of the graph. So every final configuration must contain a tower.

Why it holds. A single robot can never make a tower, so one robot can never explore and stop, on any graph with at least two nodes. With more care (Devismes et al. 2012) show that two robots are not enough on a grid with at least three nodes, and they give an algorithm with three robots, which is optimal (four for the 2×2 grid, five for the 3×3 grid).

Why it matters. This one-line remark is the starting point of most lower bounds for oblivious robots: termination itself must be written in the configuration.

3. Impossibility: Evenly spaced robots stay evenly spaced. (Flocchini et al. 2013) (Rings, exploration with stop, p. 124) Let k divide n, with $k < n$, for example 3 robots on a ring of 6 nodes, one on every second node. Every robot takes exactly the same snapshot, the same in both directions. The adversary activates all robots together and, whenever a robot wants to move but cannot tell its two directions apart, sends every robot clockwise. Then all robots do the same thing: either nobody moves, or every robot moves clockwise and the configuration is again evenly spaced. The robots will take this same snapshot forever, so they either stop where they are, having visited only k nodes, or never stop.

Why it holds. So k robots cannot explore a ring whose size is a multiple of k. (Flocchini et al. 2013) show that exploration is impossible in general whenever n and k share a common factor (the full argument is longer), and solvable when they share none and $k \geq 17$; the five-robot algorithm above later lowered the team size.

Why it matters. Symmetry that robots cannot break is the obstacle behind the number-theoretic conditions that appear throughout the oblivious-robot literature.

What the community started looking at

Robots with no memory must encode progress in their positions, so the first question was **the minimum number of robots** (Flocchini et al. 2013), answered exactly for rings (Lamani et al. 2010) and grids (Devismes et al. 2012).

Relations to other problems

- **Variant of Graph exploration by a single agent.** The robots follow the Look–Compute–Move model of the plane, moved to graphs: they see the positions of all robots but remember nothing, so the configuration itself must encode the progress of the exploration.
- **Derived problem: Exploration by luminous robots** (variant of). The same Look–Compute–Move robots, but each carries a light with a constant number c of colours that acts both as persistent memory (the robot sees its own colour in the next cycle) and as a message (every robot within range reads it); in exchange, robots usually see only within a fixed distance φ. An oblivious algorithm is a luminous one with $c = 1$, so algorithms of the parent problem with myopic vision carry over, and lower bounds proved here for any number of colours hold for oblivious robots too (Ooshita and Tixeuil 2022).

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Robots, Moves) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Identifiers → Timing → Topology.

No assumptions

+ static graph, + undirected

Topology

+ ring

Rings, exploration with stop STANDARD SETTING ●● p. 124

+ grid

Grids, exploration with stop ●● p. 125

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Robots	Moves	p.
Rings, exploration with stop STANDARD SETTING	● tight 5 vs 5	● tight $O(n)$ vs —	124
Grids, exploration with stop	● tight 3 vs 3	● unknown — vs —	125

16.6.1 Rings, exploration with stop

Adds + ring STANDARD SETTING VERIFIED

Five robots explore any ring whose size is co-prime with 5; fewer than five cannot for even n.

Assumed: graph dynamics: static graph; edge direction: undirected; topology: ring or path. **Not assumed:** identifiers: anonymous: all agents are identical; timing: asynchronous.

Measure	Lower bound	Best upper bound	Status
Robots	5 (Lamani et al. 2010); $\Omega(\log n)$ (Flocchini et al. 2013) Five, for n co-prime with 5 and even.	5 (Lamani et al. 2010)	● tight
Moves	none known $O(n)$ moves.	$O(n)$ (Lamani et al. 2010)	● tight

Results.

- Lamani et al. (2010) [lower bound]: robots 5. *Bottleneck:* symmetry. *Holds when:* even n, deterministic.
- Flocchini et al. (2013) [lower bound]: robots $\Omega(\log n)$. *Bottleneck:* symmetry. *Holds when:* for infinitely many n, when the algorithm must work from every configuration.
- Flocchini et al. (2013) [introductory]: robots $O(\log n)$. Solvable for co-prime n and $k \geq 17$; unsolvable in general if n and k are not co-prime.
- Lamani et al. (2010) [current best]: robots 5, moves $O(n)$. *Assumes:* $\gcd(n, 5) = 1$. Optimal moves; 4 robots suffice probabilistically.

Claude (Anthropic) was used in cataloguing the problems.

16.6.2 Grids, exploration with stop

Adds + grid VERIFIED

Three robots explore any grid with at least three nodes (four for 2×2, five for 3×3), which is optimal.

Assumed: graph dynamics: static graph; edge direction: undirected; topology: grid. **Not assumed:** identifiers: anonymous: all agents are identical; timing: asynchronous.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Devismes et al. 2012) Three.	3 (Devismes et al. 2012)	• tight
Moves	none known —	—	• unknown

Results.

- Devismes et al. (2012) [lower bound]: robots 3. *Bottleneck:* symmetry. *Holds when:* grids with at least 3 nodes, deterministic or probabilistic.
- Devismes et al. (2012) [current best]: robots 3. Introductory and current best.

16.7 Exploration by luminous robots

Myopic Look–Compute–Move robots with a light of a few colours must visit every node of a ring, grid or torus, and stop or keep visiting; how many robots and colours are needed?

First posed in 2018 (Ooshita and Tixeuil 2022). Ring exploration by myopic robots with a two-colour light (SSS 2018).

Problem statement and measures

k anonymous robots, each with a **light** that shows one of c colours, are placed on an anonymous graph $G = (V, E)$ (a ring, a finite grid or a torus) in an initial configuration designated by the algorithm (positions and colours). Each robot repeats Look–Compute–Move cycles. **Look:** it sees every node within distance φ, with the multiset of colours of the robots on each (so it sees **towers**, nodes with several robots), and its own colour. **Compute:** a deterministic function of this snapshot, the same for all robots, gives a new colour and a destination, its own node or a neighbour; the robot sees no identifiers, no ports, and (without chirality) no orientation. **Move:** it moves there. A fair adversarial scheduler (FSYNC, SSYNC or ASYNC) activates the robots. **Terminating exploration:** every node is visited by some robot and eventually no robot is enabled. **Perpetual exploration:** every node is visited infinitely often. With finite φ no algorithm works from every initial configuration, so each result names its initial configurations; some papers also require *exclusive* executions (no two robots on one node or one edge) or opaque robots, recorded under each result. **Measures:** the number of robots k and the number of colours c.

Measures: **Robots**, number of robots k needed; **Colours**, number of colours c of each light; $c = 1$ means no usable light (oblivious robots).

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: three white robots W W W on neighbouring nodes

(b) The pair meets the waiting G: W joins the front robot; no rule applies, all stop

Figure 16.7: Terminating ring exploration by myopic luminous robots in FSYNC, Algorithm 2 of (Ooshita and Tixeuil 2022): three robots with visibility one and two colours (G green, W white) start as W W W on an anonymous ring ($n = 8$). In the first round both end robots switch their lights to G; then the middle robot moves towards one end (the adversary picks which), forming a pair G W that moves round the ring, while the other G waits as a lighthouse. When the pair reaches it from the other side, every node has been visited; the white robot forms a tower with the front robot and no rule applies any more.

A first look

Imagine a circular race track of n squares and a few identical toy robots on it. The track is a **ring**: its squares are its n **nodes**, and two squares side by side are **neighbours**. The robots are **myopic**: each sees only its own node and its two neighbours. They cannot talk, and they are **oblivious**: they forget everything after each cycle. Each robot carries a **light** (think of a small lamp) that can glow in one of two colours, say green (G) or white (W). The light stays on until its owner switches it, and every robot that sees the node, the owner included, sees the colour. Each robot works in **Look–Compute–Move cycles**: **Look** (take a **snapshot** of its own node and its two neighbours, with the robots and light colours on them), **Compute** (decide from that snapshot alone a new colour and whether to move to one neighbour, move to the other, or stay) and **Move**. The nodes have no names and the ring has no left or right, and an **adversary** (an imagined opponent) decides when each robot acts. In the basic version all robots act together in **rounds** (**FSYNC**, fully synchronous): k robots start on neighbouring nodes in a colour pattern chosen by the algorithm (the initial **configuration**), and must visit every node and then all stop. We count the robots k and the colours c.

1. Algorithm: A coloured pair moves in step. (Ooshita and Tixeuil 2022) (Rings, FSYNC, perpetual, $\varphi = 1$, p. 144) Two robots stand on neighbouring nodes, one green and one white (Ooshita and Tixeuil 2022).

1. The green robot sees white on one side and nothing on the other: it moves away from the white robot.
2. The white robot sees green on one side and nothing on the other: it moves towards the green robot.
3. Both act in the same round, so after the move they are again on neighbouring nodes, green in front, white behind, and the snapshot each takes is the same as before. They repeat forever.

Why it works and what it costs. The pair moves one node per round in a fixed direction, so it passes every node once every n rounds: perpetual exploration with two robots and two colours. One robot cannot do it: it sees nobody, its snapshot looks the same both ways, and the adversary picks its direction each time, keeping it on two nodes. Two robots of the same colour cannot either: they take mirror-image snapshots and act as mirror images. So two robots and two colours are optimal (Ooshita and Tixeuil

Claude (Anthropic) was used in cataloguing the problems.

2022).

Why it matters. The colours give the pair a front and a back, which the ring itself does not have: the light carries the direction, both as memory and as message.

2. Algorithm: A moving pair and a lighthouse. (Ooshita and Tixeuil 2022) (Rings, FSYNC, terminating, $\varphi = 1$, p. 148) Three robots stand on three consecutive nodes coloured G, W, G (Ooshita and Tixeuil 2022).

1. The middle robot cannot tell its sides apart and moves towards one end robot (the adversary picks which); that end robot moves away from it, and the other end robot moves away too. Now a green–white pair stands on one side and a lone green robot, the **lighthouse**, on the other, with two empty nodes between them.
2. The lighthouse sees nobody and has no rule, so it waits. The pair moves away from it as in the first idea.
3. After going round the ring the pair arrives at the lighthouse from the other side. The front robot now sees a green robot ahead and stops; the white robot moves onto its node, and no rule applies any more.

Why it works and what it costs. The pair visits each node on its way round, so all n nodes are visited in about n rounds, with three robots and two colours. The lighthouse is a message that stays put: meeting it tells the pair that the trip is complete. Without lights five robots are needed (Datta et al. 2013).

Why it matters. Stopping is the hard part of exploration with little memory; here a robot that never moves plays the role of a marker left at the start.

3. Impossibility: Robots that lose sight of each other are lost. (Ooshita and Tixeuil 2022) (Rings, ASYNC, perpetual, $\varphi = 1$, p. 147) Take two robots, any number of colours, and an adversary that may activate one robot at a time (**SSYNC**, semi-synchronous) (Ooshita and Tixeuil 2022).

1. To make progress, some robot must at some moment move away from the other.
2. The adversary activates that robot alone. Now they are two nodes apart and cannot see each other.
3. A robot that sees nobody takes a symmetric snapshot, so whenever it moves the adversary chooses the direction: it sends the robot back and forth between two nodes, its *territory*. Each robot stays in its territory forever.

Why it holds. So from then on the two robots stay on four nodes, whatever their lights. Lights do not help because a robot that sees nobody reads no message, and its own colour tells it nothing about the direction. Three robots suffice in SSYNC and ASYNC (Ooshita and Tixeuil 2022).

Why it matters. This territory argument is behind most lower bounds for myopic robots: a light is a message only for robots within sight.

4. Lower bound: Two robots cannot measure the ring. (Ooshita and Tixeuil 2022) (Rings, FSYNC, terminating, $\varphi = 1$, p. 148) Suppose two robots explore a small ring of n_1 nodes and stop, or separate, at round t (Ooshita and Tixeuil 2022). Run the same robots on a ring of more than $2(t + 1)$ nodes. While they stay together they take exactly the same snapshots as before, so they do exactly the same things.

Why it holds. On the big ring they stop (or separate, and are then stuck in their territories) after visiting at most $2(t + 1)$ nodes, fewer than all of them. This holds for any number of colours, so terminating exploration needs three robots even under FSYNC, one more than perpetual exploration.

Claude (Anthropic) was used in cataloguing the problems.

Why it matters. A robot that sees only nearby nodes cannot know how big the graph is, so stopping needs a landmark, such as the lighthouse of the second idea.

What the community started looking at

Where it started. Oblivious robots on graphs see every robot and remember nothing, so the configuration itself must record progress; the first question was how many robots explore a ring (Flocchini et al. 2013) (Lamani et al. 2010) or a grid (Devismes et al. 2012). Datta, Lamani, Larmore and Petit made the robots *myopic*: with visibility $\varphi = 1$, five robots explore a ring and stop under FSYNC, and no number below n can under SSYNC or ASYNC (Datta et al. 2013). Ooshita and Tixeuil then gave each myopic robot the smallest possible light, two colours: three robots then suffice under FSYNC and four under ASYNC, and perpetual exploration needs only two and three (Ooshita and Tixeuil 2022). This showed that "there exist some tasks that myopic luminous robots with small visibility can achieve, but that non-luminous robots with larger visibility cannot" (Ooshita and Tixeuil 2022). The community then traded robots against colours, visibility and chirality: rings with $\varphi \geq 2$ (Nagahama et al. 2023), perpetual exploration of grids with chirality (Bramas et al. 2023a) and without (Rauch et al. 2021), terminating exploration of grids (Nagahama et al. 2022), tori (Darwich et al. 2023), asynchronous grids (Bramas et al. 2026a), and crash faults on rings (Ooshita et al. 2025a) (Ooshita et al. 2025b).

A light is a message and a memory. Think of a light as a flag that a robot raises on its own mast. It is a *message*: every robot within sight can read it, without being addressed, and the robot that raised it does not know who has read it, so it is a broadcast to the neighbourhood that nobody acknowledges. It *persists*: it stays up until its owner changes it, so a robot that looks much later still reads it, and in ASYNC it may read a flag that is about to change. And it is a *sticky note to oneself*: an oblivious robot forgets everything at the end of a cycle, but in the next cycle it sees its own flag and so recalls $\log_2 c$ bits. The light models separate the two roles (Flocchini et al. 2016): a flag that only its owner can see is memory without messages (FSTA, an internal light); a flag on the robot's back that only the others can see is a message without memory (FCOM, an external light); a flag that everybody, owner included, can see is both (LUMI, a full light) (Das et al. 2016), which is the model of every exploration result here. Two consequences of the picture are worth keeping. First, a LUMI robot's memory is public: whatever it remembers, its neighbours know too. Second, positions are messages as well, and lights were introduced to supplement them (Peleg 2005); on a graph a myopic robot sees so little that the light is often its only memory, which is why two colours change the number of robots so much (Ooshita and Tixeuil 2022).

Further reading

Papers on this problem that are not yet placed in a setting.

- Nagahama et al. (2023). Rings with $\varphi \geq 2$: two robots for perpetual and, per its technical-report abstract, three for terminating exploration with two colours. Not read; the perpetual bound is placed as reported by (Ooshita et al. 2025a).
- Bramas et al. (2021b). Infinite grid exploration (every node eventually visited) by synchronous, opaque, myopic luminous robots with chirality: two robots never suffice, five robots with five colours are optimal. Abstract read.
- Bramas et al. (2021a). Infinite grid exploration by myopic luminous robots: optimal visibility and optimal number of colours cannot be achieved together; algorithms with six and eight robots. Abstract read.
- Bramas et al. (2022). Perpetual exploration of a 3D grid (cuboid) by synchronous myopic luminous robots; two optimal algorithms. Abstract read.
- Bramas et al. (2026b). Perpetual exploration of a finite 3D grid by three synchronous robots without chirality: 77 colours with $\varphi = 1$, or four colours with $\varphi = 2$; three robots are optimal. Preprint; abstract read.

Claude (Anthropic) was used in cataloguing the problems.

- Das et al. (2025c). Exploration of a rectangle-enclosed triangular grid by myopic luminous robots. Not read.
- Kamei et al. (2025). Gathering on rings by myopic ASYNC robots with lights: feasible with three or four colours whenever one of two parameters of the initial configuration is odd. Abstract read.
- Bramas et al. (2026c). Stand-up indulgent gathering (gather, and at the crash site if a robot crashes) on lines by myopic luminous FSYNC robots. Abstract read.
- Bramas et al. (2025). Stand-up indulgent gathering on lines: oblivious robots need one more hop of visibility than robots with one bit of persistent memory, a visibility–memory trade-off on graphs. Abstract read.
- Hideg and Lukovszki (2020). Filling an arbitrary graph through a door by ASYNC myopic luminous robots ($\varphi = 1$, $\Delta + 4$ colours, $O(\log \Delta)$ bits) in $O(n^2)$ epochs. Abstract read.
- Pramanick et al. (2023). MIS filling of an arbitrary graph from one door (ASYNC, $\varphi = 3$, $\Delta + 8$ colours) or several doors (SSYNC). Abstract read.
- Kamei and Tixeuil (2024). Maximum independent set on grids by ASYNC myopic luminous robots entering from a corner: three colours with $\varphi = 2$ and port numbers, or seven colours with $\varphi = 3$. Abstract read.
- Das et al. (2025a). Maximum independent set formation on a finite grid from an arbitrary placement, SSYNC, $\varphi = 2$, three colours. Abstract read.

Relations to other problems

- **Variant of Exploration by oblivious robots.** The same Look–Compute–Move robots, but each carries a light with a constant number c of colours that acts both as persistent memory (the robot sees its own colour in the next cycle) and as a message (every robot within range reads it); in exchange, robots usually see only within a fixed distance φ. An oblivious algorithm is a luminous one with $c = 1$, so algorithms of the parent problem with myopic vision carry over, and lower bounds proved here for any number of colours hold for oblivious robots too (Ooshita and Tixeuil 2022).
- **Related: Exploration of recolourable graphs.** Flag on the robot versus flag on the node: a luminous robot carries its coloured light with it and shows it to the robots in sight, whereas here the colour stays planted in the node and is read by whoever arrives.

Assumptions specific to this problem

Scheduler

The three schedulers come from robots in the plane (see the plane model); on graphs the asynchronous one was used from the first ring exploration paper (Flocchini et al. 2013), and Datta, Lamani, Larmore and Petit compared all three for myopic robots (Datta et al. 2013). For myopic robots on graphs the scheduler decides solvability: oblivious robots that see one hop can explore a ring with five robots under FSYNC but not at all under SSYNC or ASYNC (Datta et al. 2013), and with two-colour lights the robots needed for perpetual (terminating) exploration of a ring are two (three) under FSYNC against three (four) under SSYNC and ASYNC (Ooshita and Tixeuil 2022). Ooshita and Tixeuil ask whether SSYNC and ASYNC still differ for myopic luminous robots, since with full visibility lights make them equivalent (Ooshita and Tixeuil 2022) (Das et al. 2016). Most grid and torus results are for FSYNC only; asynchronous perpetual grid exploration is the subject of (Bramas et al. 2026a).

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**ASYNC**	**ASYNC**: each robot's Look, Compute and Move phases take finite but unbounded, adversarially chosen time; a robot may Look while others are between phases, so it may move on an outdated snapshot. A robot is always seen on a node (an edge traversal is instantaneous), and its new colour is visible from the end of its Compute phase. The scheduler is fair: every robot is activated infinitely often. Called *CORDA* in (Datta et al. 2013).
1	**SSYNC**	**SSYNC**: time is divided into rounds; in each round a fair adversary chooses a non-empty subset of robots, which execute one atomic Look–Compute–Move cycle together. Called *ATOM* in (Datta et al. 2013).
2	**FSYNC**	**FSYNC**: in every round every robot executes one atomic Look–Compute–Move cycle.

Lights (memory and messages)

Peleg suggested giving robots \"colors, flags or visible indicator lights\" and $O(1)$ bits of stable memory (Peleg 2005); Das, Flocchini, Prencipe, Santoro and Yamashita made this the LUMI model, a persistent register visible to all (Das et al. 2016), and Flocchini, Santoro, Viglietta and Yamashita split the light into its two roles: FSTA, a light only its owner sees (memory), and FCOM, a light only the others see (communication) (Flocchini et al. 2016). On graphs, Ooshita and Tixeuil gave myopic robots a light (the first study of lights for myopic robots, by their account), noting that it \"can be used as a constant-space memory\" and that robots communicate \"by changing their color and moving\" (Ooshita and Tixeuil 2022). Every exploration result found for graphs uses full lights; the number of colours c is a measure, and an algorithm with $c = 1$ is an oblivious one. The scale orders the models by what they assume: OBLOT assumes no light; FSTA and FCOM each assume one role of the light and do not imply each other; LUMI assumes both.

Level	Value	Formal definition
0	**oblivious**	**OBLOT**: a robot's Compute phase depends only on the current snapshot; nothing persists across cycles and robots communicate only by moving. Equivalent to a light with $c = 1$ colour.
1	**FSTA**	**FSTA** (*finite-state*): each robot has a light with c colours that only it can see. The colour is set at the end of Compute and persists until the robot changes it: $\log_2 c$ bits of persistent memory, no communication (Flocchini et al. 2016).
1	**FCOM**	**FCOM** (*finite-communication*): each robot has a light with c colours that the other robots within distance φ can see but the robot itself cannot; the robot stays oblivious, but every robot that looks reads its last colour: $\log_2 c$ bits broadcast to the neighbourhood, no memory (Flocchini et al. 2016).
2	**luminous**	**LUMI** (*luminous* robots): each robot has a light with c colours, set at the end of Compute, persistent across cycles, and visible to the robot itself and to every robot within distance φ (Das et al. 2016). It is both memory and message. A Look returns, for every node within distance φ, the multiset of colours of the robots on it.

Visibility range

Myopia, a fixed visibility range φ, replaces the global snapshot of the oblivious-robot literature, which

Claude (Anthropic) was used in cataloguing the problems.

\"somewhat contradicts the principle of very weak mobile entities\" (Ooshita and Tixeuil 2022). Datta, Lamani, Larmore and Petit studied ring exploration by oblivious robots with $\varphi = 1, 2, 3$ (Datta et al. 2013); Ooshita and Tixeuil chose \"the weakest assumption for myopia\", $\varphi = 1$ (Ooshita and Tixeuil 2022), and later work compares $\varphi = 1$ with $\varphi \geq 2$ (Ooshita et al. 2025a) (Nagahama et al. 2022). With a finite range no algorithm explores from every initial configuration (robots that see nobody cannot coordinate), so algorithms start from designated or locally defined configurations (Datta et al. 2013) (Bramas et al. 2023a). Unlimited visibility is the setting of the parent problem.

Level	Value	Formal definition
0	$\varphi = 1$	In a Look phase a robot at node v sees every node at distance at most $\varphi = 1$ from v (in the graph metric), with the robots on it, their colours, and how many robots each node holds. It sees no node identifiers and, without chirality, no orientation.
1	$\varphi = 2$	As for $\varphi = 1$, but the robot sees every node at distance at most $\varphi = 2$. An algorithm for $\varphi = 2$ works for every larger φ, since a robot can ignore what lies further away.
2	$\varphi = 3$	As for $\varphi = 1$, but the robot sees every node at distance at most $\varphi = 3$.

Chirality

Chirality was mostly assumed in the plane; an early use on graphs is by Bramas, Devismes and Lafourcade, for infinite grid exploration (Bramas et al. 2021b) and for perpetual exploration of finite grids (Bramas et al. 2023a), noting that only \"a few works dedicated to discrete environments\" assume it and that it changes the number of robots needed (Bramas et al. 2023a). Nagahama, Ooshita and Inoue give terminating grid algorithms with and without it (Nagahama et al. 2022); without it, perpetual grid exploration needs three robots (Rauch et al. 2021) (as reported in Darwich et al. 2023). Ring results assume an unoriented ring.

Level	Value	Formal definition
0	**no chirality**	Robots share no orientation: on a ring a robot cannot tell its two directions apart; on a grid or torus its snapshot is known only up to rotation and reflection, and the adversary may choose a different transformation in every cycle (*self-inconsistent* robots, as in Darwich et al. 2023 and Bramas et al. 2026a).
1	**chirality**	All robots agree on the clockwise direction: a snapshot is known up to rotation only (no reflection), so a robot on an axis of symmetry of its view can still tell its two sides apart. Robots still share no North.

Exploration goal

Both goals come from the oblivious-robot literature: terminating exploration of rings (Flocchini et al. 2013) and grids (Devismes et al. 2012), and perpetual (often exclusive) exploration, which models patrolling (Bramas et al. 2023a). Ooshita and Tixeuil studied both for myopic luminous robots (Ooshita and Tixeuil 2022). The two goals are incomparable: a terminating algorithm stops, and a perpetual one need never detect that it is done, so the scale puts them side by side above \"both\". In this literature the impossibility proofs for perpetual exploration show that some node is never visited, which then also rules out terminating exploration (Nagahama et al. 2022), and perpetual exploration never needed more robots than terminating exploration.

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**both goals**	Solve both terminating and perpetual exploration with the stated resources (possibly with two algorithms). A lower bound for either goal holds here, and a result here gives both. No variant is catalogued here; the value exists because the two goals do not imply each other.
1	**terminating**	Every node is visited by at least one robot, and from some time on no robot is enabled (no robot changes its position or colour again).
1	**perpetual**	Every node is visited by at least one robot infinitely often; robots never stop.

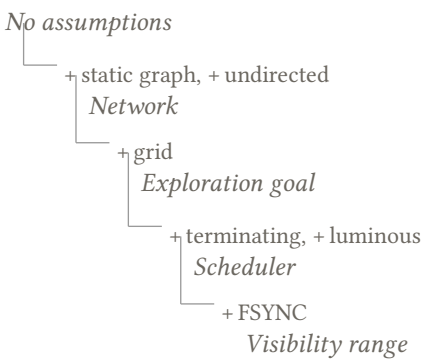
Network

Rings were the first graphs explored by oblivious robots (Flocchini et al. 2013) and by myopic ones (Datta et al. 2013) (Ooshita and Tixeuil 2022). Finite grids followed for myopic luminous robots in perpetual (Bramas et al. 2023a) and terminating (Nagahama et al. 2022) form, and tori, whose missing border removes the landmark that grid algorithms use to turn, in (Darwich et al. 2023). Infinite grids and 3D grids are studied in papers listed under further reading. This axis replaces the model's topology axis because the torus is not one of the model's values.

Level	Value	Formal definition
0	**any graph**	A finite, connected, anonymous graph $G = (V, E)$ with n nodes and no port numbers visible to the robots.
1	**ring**	An anonymous, unoriented ring of n nodes; n is unknown to the robots.
1	**grid**	An $a \times b$ grid ($n = ab$ nodes; nodes (i, j), edges between nodes at Manhattan distance one); anonymous, with a and b unknown. A robot sees the border of the grid when it is within range, since nodes beyond the border are missing from its snapshot.
1	**torus**	An $a \times b$ torus: the grid with wrap-around edges in both dimensions, so every node has degree four and there is no border.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Robots, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Network → Exploration goal → Lights (memory and messages) → Scheduler → Visibility range → Chirality → Faults → Identifiers.



Claude (Anthropic) was used in cataloguing the problems.



Claude (Anthropic) was used in cataloguing the problems.

+ FSYNC
Faults
+ no faults
Rings, FSYNC, perpetual, $\varphi = 1$ ●● p. 144
+ at most crash faults
Rings, FSYNC, perpetual, up to f crashes ●● p. 145

ASYNC, + $\varphi = 2$
Faults
+ no faults
Rings, ASYNC, perpetual, $\varphi \geq 2$ ●● p. 146
+ at most crash faults
Rings, ASYNC, perpetual, up to f crashes, $\varphi \geq 2$ ●● p. 146

ASYNC, $\varphi = 1$, + no faults
Rings, ASYNC, perpetual, $\varphi = 1$ ●● p. 147

ASYNC, $\varphi = 1$, + at most crash faults
Rings, ASYNC, perpetual, up to f crashes, $\varphi = 1$ ●● p. 147

+ terminating
Lights (memory and messages)

+ luminous
Scheduler
+ FSYNC, + no faults
Rings, FSYNC, terminating, $\varphi = 1$ STANDARD SETTING ●● p. 148
ASYNC, + no faults
Rings, ASYNC, terminating, $\varphi = 1$ ●● p. 148

oblivious, + FSYNC, + no faults
Rings, oblivious, FSYNC, terminating, $\varphi = 1$ ●● p. 149

oblivious, ASYNC, + no faults
Rings, oblivious, ASYNC, terminating, $\varphi = 1$ ●● p. 150

+ torus, + perpetual, + luminous, + FSYNC
Visibility range

+ $\varphi = 2$
Chirality
+ chirality, + no faults
Tori, FSYNC, perpetual, chirality, $\varphi = 2$ ●● p. 150
no chirality, + no faults
Tori, FSYNC, perpetual, no chirality, $\varphi = 2$ ●● p. 151

$\varphi = 1$, + chirality, + no faults
Tori, FSYNC, perpetual, chirality, $\varphi = 1$ ●● p. 151

$\varphi = 1$, no chirality, + no faults
Tori, FSYNC, perpetual, no chirality, $\varphi = 1$ ●● p. 152

Claude (Anthropic) was used in cataloguing the problems.

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Robots	Colours	p.
Grids, FSYNC, terminating, chirality, $\varphi = 2$	● tight 2 vs 2	● tight 2 vs 2	136
Grids, FSYNC, terminating, no chirality, $\varphi = 2$	● tight 3 vs 2	● unknown 2 vs —	136
Grids, FSYNC, terminating, chirality, $\varphi = 1$	● tight 2 vs 2	● tight 3 vs 3	137
Grids, FSYNC, terminating, no chirality, $\varphi = 1$	● tight 4 vs 2	● unknown 3 vs —	137
Grids, ASYNC, terminating, chirality, $\varphi = 2$	● tight 2 vs 2	● tight 3 vs 2	138
Grids, ASYNC, terminating, no chirality, $\varphi = 2$	● tight 3 vs 2	● unknown 3 vs —	139
Grids, ASYNC, terminating, chirality, $\varphi = 1$	● tight 3 vs 3	● unknown 3 vs —	139
Grids, ASYNC, terminating, no chirality, $\varphi = 1$	● tight 6 vs 3	● unknown 3 vs —	140
Grids, FSYNC, perpetual, chirality, $\varphi = 1$	● tight 2 vs 2	● tight 3 vs 3	140
Grids, FSYNC, perpetual, chirality, $\varphi = 2$	● tight 2 vs 2	● tight 2 vs 2	141
Grids, FSYNC, perpetual, no chirality, $\varphi = 1$	● tight 3 vs 3	● unknown 3 vs —	141
Grids, ASYNC, perpetual, chirality, $\varphi = 2$	● tight 3 vs 2	● tight 2 vs 2	142
Grids, ASYNC, perpetual, no chirality, $\varphi = 2$	● tight 3 vs 3	● tight 2 vs 2	143
Grids, ASYNC, perpetual, chirality, $\varphi = 1$ (exclusive)	● impossible	● impossible	143
Grids, ASYNC, perpetual, no chirality, $\varphi = 3$	● tight 3 vs 3	● tight 1 vs —	144
Rings, FSYNC, perpetual, $\varphi = 1$	● tight 2 vs 2	● tight 2 vs 2	144
Rings, FSYNC, perpetual, up to f crashes	● tight $f + 2$ vs $f + 2$	● unknown $3f + 2$ vs —	145
Rings, ASYNC, perpetual, $\varphi \geq 2$	● tight 2 vs 2	● unknown 2 vs —	146
Rings, ASYNC, perpetual, up to f crashes, $\varphi \geq 2$	● tight $2f + 2$ vs $2f + 2$	● unknown $2f + 2$ vs —	146
Rings, ASYNC, perpetual, $\varphi = 1$	● tight 3 vs 3	● unknown 2 vs —	147
Rings, ASYNC, perpetual, up to f crashes, $\varphi = 1$	● tight $3f + 3$ vs $3f + 3$	● unknown $2f + 2$ vs —	147
Rings, FSYNC, terminating, $\varphi = 1$ STANDARD SETTING	● tight 3 vs 3	● tight 2 vs 2	148
Rings, ASYNC, terminating, $\varphi = 1$	● tight 4 vs 4	● tight 2 vs —	148

Claude (Anthropic) was used in cataloguing the problems.

Setting	Robots	Colours	p.
Rings, oblivious, FSYNC, terminating, $\varphi = 1$	● tight 5 vs 5	● tight 1 vs —	149
Rings, oblivious, ASYNC, terminating, $\varphi = 1$	● impossible	● impossible	150
Tori, FSYNC, perpetual, chirality, $\varphi = 2$	● tight 3 vs 3	● tight 2 vs 2	150
Tori, FSYNC, perpetual, no chirality, $\varphi = 2$	● tight 3 vs 3	● tight 3 vs —	151
Tori, FSYNC, perpetual, chirality, $\varphi = 1$	● tight 3 vs 3	● tight 2 vs 2	151
Tori, FSYNC, perpetual, no chirality, $\varphi = 1$	● tight 3 vs 3	● tight 3 vs 2	152

16.7.1 Grids, FSYNC, terminating, chirality, $\varphi = 2$

Adds + chirality, + no faults VERIFIED

With $\varphi = 2$ and chirality, two robots with two colours, or three oblivious robots, explore a grid and stop under FSYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: terminating exploration (explore, then stop); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); visibility range: $\varphi = 2$ (results hold for every $\varphi \geq 2$); chirality: common chirality (same clockwise direction); faults: no faults. **Not assumed:** identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	2 (Bramas et al. 2023a) Two.	2 (Nagahama et al. 2022)	● tight
Colours	2 (Bramas et al. 2023a) Two for two robots.	2 (Nagahama et al. 2022)	● tight

Results.

- Bramas et al. (2023a) [lower bound]: robots 2. *Bottleneck:* symmetry breaking. *Holds when:* one robot (perpetual-exploration bound that also holds for terminating exploration Nagahama et al. 2022).
- Bramas et al. (2023a) [lower bound]: colours 2. *Bottleneck:* symmetry breaking. *Holds when:* two robots with one colour, any finite φ (Theorem 2, applied to terminating exploration in Nagahama et al. 2022).
- Nagahama et al. (2022) [current best]: robots 2, colours 2. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Introductory; optimal in robots (Section 4.2.1).
- Nagahama et al. (2022) [also]: robots 3, colours 1. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Oblivious robots (Section 4.2.3); optimal for one colour.

16.7.2 Grids, FSYNC, terminating, no chirality, $\varphi = 2$

Adds no chirality, + no faults VERIFIED

Without chirality and with $\varphi = 2$, three robots with two colours, or four oblivious robots, explore a grid and stop under FSYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: terminating exploration (explore, then stop); lights (memory and messages): full light (LUMI): visible to the robot and to the others;

Claude (Anthropic) was used in cataloguing the problems.

scheduler: fully synchronous (FSYNC); visibility range: $\varphi = 2$ (results hold for every $\varphi \geq 2$); faults: no faults. **Not assumed:** chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	2 (Bramas et al. 2023a)	3 (Nagahama et al. 2022)	• tight
	Constant on both sides, so tight up to a constant: two against three. The exact number is open.		
Colours	none known	2 (Nagahama et al. 2022)	• unknown
	No colour lower bound for three robots.		

Results.

- Bramas et al. (2023a) [lower bound]: robots 2. *Bottleneck:* symmetry breaking. *Holds when:* one robot (perpetual-exploration bound that also holds for terminating exploration Nagahama et al. 2022).
- Nagahama et al. (2022) [current best]: robots 3, colours 2. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Introductory (Section 4.2.2).
- Nagahama et al. (2022) [also]: robots 4, colours 1. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Oblivious robots (Section 4.2.4).

16.7.3 Grids, FSYNC, terminating, chirality, $\varphi = 1$

Adds $\varphi = 1$, + chirality, + no faults VERIFIED

Two robots with three colours, or three with two, explore a grid and stop under FSYNC with chirality.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: terminating exploration (explore, then stop); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); chirality: common chirality (same clockwise direction); faults: no faults. **Not assumed:** visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	2 (Bramas et al. 2023a)	2 (Nagahama et al. 2022)	• tight
	Two.		
Colours	3 (Bramas et al. 2023a)	3 (Nagahama et al. 2022)	• tight
	Three for two robots.		

Results.

- Bramas et al. (2023a) [lower bound]: robots 2. *Bottleneck:* symmetry breaking. *Holds when:* one robot (perpetual-exploration bound that also holds for terminating exploration Nagahama et al. 2022).
- Bramas et al. (2023a) [lower bound]: colours 3. *Bottleneck:* symmetry breaking. *Holds when:* two robots with two colours and $\varphi = 1$ cannot explore perpetually; the proof shows some node is never visited (Theorem 3, applied to terminating exploration in Nagahama et al. 2022).
- Nagahama et al. (2022) [current best]: robots 2, colours 3. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Introductory; optimal in robots (Section 4.2.5).
- Nagahama et al. (2022) [also]: robots 3, colours 2. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Section 4.2.7; optimal in robots for two colours.

16.7.4 Grids, FSYNC, terminating, no chirality, $\varphi = 1$

Adds $\varphi = 1$, no chirality, + no faults VERIFIED

Without chirality, four robots with three colours, or five with two, explore a grid and stop under FSYNC.

Claude (Anthropic) was used in cataloguing the problems.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: terminating exploration (explore, then stop); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); faults: no faults. **Not assumed:** visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	2 (Bramas et al. 2023a)	4 (Nagahama et al. 2022)	• tight
	Constant on both sides, so tight up to a constant: two against four; the two-robot impossibility of (Rauch et al. 2021) is stated for perpetual exploration. The exact number is open.		
Colours	none known	3 (Nagahama et al. 2022)	• unknown
	No colour lower bound for four robots.		

Results.

- Bramas et al. (2023a) [lower bound]: robots 2. *Bottleneck:* symmetry breaking. *Holds when:* one robot (perpetual-exploration bound that also holds for terminating exploration Nagahama et al. 2022).
- Nagahama et al. (2022) [current best]: robots 4, colours 3. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Introductory (Section 4.2.6).
- Nagahama et al. (2022) [also]: robots 5, colours 2. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Section 4.2.8.

Open questions.

Q1. How many robots are needed for terminating grid exploration under FSYNC with $\varphi = 1$ and no chirality: three or four?

Notes. Without chirality two robots cannot explore a grid perpetually (Rauch et al. 2021) (as reported in Darwich et al. 2023); whether that argument also excludes terminating exploration was not checked.

16.7.5 Grids, ASYNC, terminating, chirality, $\varphi = 2$

Adds + chirality, + no faults VERIFIED

With $\varphi = 2$ and chirality, two robots with three colours, or three with two, explore a grid and stop under ASYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: terminating exploration (explore, then stop); lights (memory and messages): full light (LUMI): visible to the robot and to the others; visibility range: $\varphi = 2$ (results hold for every $\varphi \geq 2$); chirality: common chirality (same clockwise direction); faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	2 (Bramas et al. 2023a)	2 (Nagahama et al. 2022)	• tight
	Two.		
Colours	2 (Bramas et al. 2023a)	3 (Nagahama et al. 2022)	• tight
	Constant on both sides, so tight up to a constant: two against three, for two robots. The exact number is open.		

Results.

- Bramas et al. (2023a) [lower bound]: robots 2. *Bottleneck:* symmetry breaking. *Holds when:* one robot (perpetual-exploration bound that also holds for terminating exploration Nagahama et al. 2022).
- Bramas et al. (2023a) [lower bound]: colours 2. *Bottleneck:* symmetry breaking. *Holds when:* two robots with one colour, any finite φ (Theorem 2, applied to terminating exploration in Nagahama et al. 2022).

Claude (Anthropic) was used in cataloguing the problems.

- Nagahama et al. (2022) [current best]: robots 2, colours 3. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Introductory; optimal in robots (Section 4.3.1).
- Nagahama et al. (2022) [also]: robots 3, colours 2. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Section 4.3.3.

16.7.6 Grids, ASYNC, terminating, no chirality, $\varphi = 2$

Adds no chirality, + no faults VERIFIED

Without chirality and with $\varphi = 2$, three robots with three colours, or four with two, explore a grid and stop under ASYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: terminating exploration (explore, then stop); lights (memory and messages): full light (LUMI): visible to the robot and to the others; visibility range: $\varphi = 2$ (results hold for every $\varphi \geq 2$); faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	2 (Bramas et al. 2023a) Constant on both sides, so tight up to a constant: two against three. The exact number is open.	3 (Nagahama et al. 2022)	• tight
Colours	none known No colour lower bound.	3 (Nagahama et al. 2022)	• unknown

Results.

- Bramas et al. (2023a) [lower bound]: robots 2. *Bottleneck:* symmetry breaking. *Holds when:* one robot (perpetual-exploration bound that also holds for terminating exploration Nagahama et al. 2022).
- Nagahama et al. (2022) [current best]: robots 3, colours 3. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Introductory (Section 4.3.2).
- Nagahama et al. (2022) [also]: robots 4, colours 2. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Section 4.3.4.

16.7.7 Grids, ASYNC, terminating, chirality, $\varphi = 1$

Adds ASYNC, $\varphi = 1$, + chirality, + no faults VERIFIED

Three robots with three colours explore a grid and stop under ASYNC with chirality and $\varphi = 1$; two robots cannot.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: terminating exploration (explore, then stop); lights (memory and messages): full light (LUMI): visible to the robot and to the others; chirality: common chirality (same clockwise direction); faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Nagahama et al. 2022) Three.	3 (Nagahama et al. 2022)	• tight
Colours	none known No colour lower bound for three robots.	3 (Nagahama et al. 2022)	• unknown

Results.

- Nagahama et al. (2022) [lower bound]: robots 3. *Bottleneck:* adversarial activation. *Holds when:* two robots, $\varphi = 1$, any number of colours, with or without chirality, under SSYNC (Theorem 1); grids of at least 9×9 nodes; also rules out perpetual exploration.

Claude (Anthropic) was used in cataloguing the problems.

- Nagahama et al. (2022) [current best]: robots 3, colours 3. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Introductory and current best (Section 4.3.5); proceeds along a row with the ring algorithm of (Ooshita and Tixeuil 2022).

16.7.8 Grids, ASYNC, terminating, no chirality, $\varphi = 1$

Adds ASYNC, $\varphi = 1$, no chirality, + no faults VERIFIED

Without chirality, six robots with three colours explore a grid and stop under ASYNC with $\varphi = 1$; at least three are needed.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: terminating exploration (explore, then stop); lights (memory and messages): full light (LUMI): visible to the robot and to the others; faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Nagahama et al. 2022)	6 (Nagahama et al. 2022)	• tight
	Constant on both sides, so tight up to a constant: three against six. The exact number is open.		
Colours	none known	3 (Nagahama et al. 2022)	• unknown
	No colour lower bound.		

Results.

- Nagahama et al. (2022) [lower bound]: robots 3. *Bottleneck:* adversarial activation. *Holds when:* two robots, $\varphi = 1$, any number of colours, with or without chirality, under SSYNC (Theorem 1); grids of at least 9×9 nodes; also rules out perpetual exploration.
- Nagahama et al. (2022) [current best]: robots 6, colours 3. *Assumes:* robots start in a designated formation at a corner of the grid; towers allowed (not exclusive); $a \geq 2$, $b \geq 3$. Introductory and current best (Section 4.3.6); needs $a, b \geq 3$.

Open questions.

Q1. Close the gap between three and six robots for terminating grid exploration under ASYNC with $\varphi = 1$ and no chirality (Nagahama et al. 2022).

16.7.9 Grids, FSYNC, perpetual, chirality, $\varphi = 1$

Adds $\varphi = 1$, + chirality, + no faults VERIFIED

With common chirality and $\varphi = 1$, two robots with three colours explore a grid perpetually, optimal in robots and colours.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); chirality: common chirality (same clockwise direction); faults: no faults. **Not assumed:** visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	2 (Bramas et al. 2023a)	2 (Bramas et al. 2023a)	• tight
	Two.		
Colours	3 (Bramas et al. 2023a)	3 (Bramas et al. 2023a)	• tight
	Three, for two robots.		

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Bramas et al. (2023a) [lower bound]: robots 2. *Bottleneck:* symmetry breaking. *Holds when:* one robot, any finite φ and colours (Theorem 1); holds without exclusiveness.
- Bramas et al. (2023a) [lower bound]: colours 3. *Bottleneck:* symmetry breaking. *Holds when:* two robots with two colours and $\varphi = 1$ (Theorem 3); three robots with one colour and $\varphi = 1$ also fail (Theorem 4).
- Bramas et al. (2023a) [current best]: robots 2, colours 3. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together; opaque robots; locally defined initial configurations; grid with at least k rows and k columns. Introductory and current best (algorithm V_{32}^{one}).

Notes. Lower bounds hold without exclusiveness; Nagahama, Ooshita and Inoue use them for terminating exploration as well (Nagahama et al. 2022).

16.7.10 Grids, FSYNC, perpetual, chirality, $\varphi = 2$

Adds + $\varphi = 2$, + chirality, + no faults VERIFIED

With $\varphi = 2$, two robots with two colours, or three oblivious robots, explore a grid perpetually.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); visibility range: $\varphi = 2$ (results hold for every $\varphi \geq 2$); chirality: common chirality (same clockwise direction); faults: no faults. **Not assumed:** identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	2 (Bramas et al. 2023a) Two.	2 (Bramas et al. 2023a)	• tight
Colours	2 (Bramas et al. 2023a) Two for two robots; one for three.	2 (Bramas et al. 2023a)	• tight

Results.

- Bramas et al. (2023a) [lower bound]: robots 2. *Bottleneck:* symmetry breaking. *Holds when:* one robot, any finite φ (Theorem 1).
- Bramas et al. (2023a) [lower bound]: colours 2. *Bottleneck:* symmetry breaking. *Holds when:* two robots with one colour, any finite φ (Theorem 2).
- Bramas et al. (2023a) [current best]: robots 2, colours 2. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together; opaque robots. Algorithm V_{22}^{two}.
- Bramas et al. (2023a) [also]: robots 3, colours 1. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together; opaque robots. Algorithm V_{13}^{two}: oblivious robots.

16.7.11 Grids, FSYNC, perpetual, no chirality, $\varphi = 1$

Adds $\varphi = 1$, no chirality, + no faults DRAFT

Without chirality, two robots never suffice and three robots with three colours do.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); faults: no faults. **Not assumed:** visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Rauch et al. 2021) Three.	3 (Rauch et al. 2021)	• tight
Colours	none known No colour lower bound is reported for three robots.	3 (Rauch et al. 2021)	• unknown

Results.

- Rauch et al. (2021) [lower bound]: robots 3. *Bottleneck:* symmetry breaking. *Holds when:* two robots, any finite number of colours and any finite φ (as reported in Darwich et al. 2023).
- Rauch et al. (2021) [current best]: robots 3, colours 3. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together. As reported in (Darwich et al. 2023) and (Bramas et al. 2026a); with $\varphi = 2$, five oblivious robots suffice.

Notes. Not read: bounds as reported by (Darwich et al. 2023) (Table 1) and (Bramas et al. 2026a).

16.7.12 Grids, ASYNC, perpetual, chirality, $\varphi = 2$

Adds + chirality, + no faults VERIFIED

Three robots with two colours, or four oblivious robots, explore a grid perpetually under ASYNC with chirality and $\varphi = 2$.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; visibility range: $\varphi = 2$ (results hold for every $\varphi \geq 2$); chirality: common chirality (same clockwise direction); faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	2 (Bramas et al. 2023a) Constant on both sides, so tight up to a constant: two against three: no impossibility for two luminous robots under ASYNC with chirality and $\varphi = 2$ was found (two robots with two colours work under FSYNC Bramas et al. 2023a). The exact number is open.	3 (Bramas et al. 2023a)	• tight
Colours	2 (Bramas et al. 2026a) Two for three robots; one for four.	2 (Bramas et al. 2023a)	• tight

Results.

- Bramas et al. (2023a) [lower bound]: robots 2. *Bottleneck:* symmetry breaking. *Holds when:* one robot, any finite φ (Theorem 1); FSYNC bound.
- Bramas et al. (2026a) [lower bound]: colours 2. *Bottleneck:* symmetry breaking. *Holds when:* three oblivious robots, $\varphi = 2$, even with chirality (Theorem 3).
- Bramas et al. (2023a) [current best]: robots 3, colours 2. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together. Transformation of V_{32}^{one} to ASYNC under $\varphi = 2$, as reported in Table 1 of (Bramas et al. 2026a).
- Bramas et al. (2026a) [also]: robots 4, colours 1. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together; grids with at least five rows and columns (smaller cases by special rules); initial configuration not locally defined. Algorithm A_1: oblivious robots, optimal for $c = 1$.

Open questions.

Q1. Can two luminous robots with common chirality and $\varphi = 2$ explore a finite grid perpetually under ASYNC (or SSYNC)?

Claude (Anthropic) was used in cataloguing the problems.

16.7.13 Grids, ASYNC, perpetual, no chirality, $\varphi = 2$

Adds no chirality, + no faults VERIFIED

Three robots with two colours explore a grid perpetually under ASYNC without chirality, which is optimal.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; visibility range: $\varphi = 2$ (results hold for every $\varphi \geq 2$); faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Rauch et al. 2021) Three.	3 (Bramas et al. 2026a)	• tight
Colours	2 (Bramas et al. 2026a) Two, for three robots.	2 (Bramas et al. 2026a)	• tight

Results.

- Rauch et al. (2021) [lower bound]: robots 3. *Bottleneck:* symmetry breaking. *Holds when:* two robots without chirality, any finite colours and φ, under FSYNC (as reported in Bramas et al. 2026a).
- Bramas et al. (2026a) [lower bound]: colours 2. *Bottleneck:* symmetry breaking. *Holds when:* three oblivious robots, $\varphi = 2$ (Theorem 3).
- Bramas et al. (2026a) [current best]: robots 3, colours 2. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together; transparent robots (no opacity); initial L-shape coloured A, A, B. Introductory and current best (algorithm A_2).

Open questions.

Q1. Is there an optimal ASYNC algorithm (three robots, $\varphi = 2$, no chirality) for opaque robots? Posed in (Bramas et al. 2026a).

16.7.14 Grids, ASYNC, perpetual, chirality, $\varphi = 1$ (exclusive)

Adds ASYNC, $\varphi = 1$, + chirality, + no faults VERIFIED

With $\varphi = 1$ no number of exclusive asynchronous robots explores a grid perpetually, whatever the number of colours.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; chirality: common chirality (same clockwise direction); faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	*impossible* (Bramas et al. 2026a) For exclusive robots.	—	• impossible
Colours	none known For exclusive robots.	—	• impossible

Results.

- Bramas et al. (2026a) [lower bound]: robots *impossible*. *Bottleneck:* locality. *Holds when:* any number of robots and colours, even with common chirality (Theorem 2); robots may never share a node (the model of the paper); the adversary prevents any group of robots from crossing a fence of width two (test of the fence).

Claude (Anthropic) was used in cataloguing the problems.

Open questions.

Q1. Is perpetual grid exploration with $\varphi = 1$ solvable under ASYNC when robots may share nodes (form towers)? Terminating exploration is, with three robots and three colours (Nagahama et al. 2022).

Notes. The impossibility assumes, as the model of (Bramas et al. 2026a) does, that two robots never occupy one node. Terminating exploration in the same scheduler and range is solvable when towers are allowed (Nagahama et al. 2022), so co-location is what the impossibility needs.

16.7.15 Grids, ASYNC, perpetual, no chirality, $\varphi = 3$

Adds ASYNC, + $\varphi = 3$, + no faults VERIFIED

With $\varphi = 3$, three oblivious robots explore a grid perpetually under ASYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: finite grid; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; visibility range: $\varphi = 3$; faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Rauch et al. 2021) Three.	3 (Bramas et al. 2026a)	• tight
Colours	none known No light needed.	1 (Bramas et al. 2026a)	• tight

Results.

- Rauch et al. (2021) [lower bound]: robots 3. *Bottleneck:* symmetry breaking. *Holds when:* two robots without chirality (as reported in Bramas et al. 2026a).
- Bramas et al. (2026a) [current best]: robots 3, colours 1. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together. Algorithm A_3: oblivious robots.

Notes. A larger range replaces the light: compare three robots with two colours at $\varphi = 2$.

16.7.16 Rings, FSYNC, perpetual, $\varphi = 1$

Adds + no faults VERIFIED

Two robots with two colours explore a ring perpetually under FSYNC, which is optimal.

Assumed: graph dynamics: static graph; edge direction: undirected; network: ring; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); faults: no faults. **Not assumed:** visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	2 (Ooshita and Tixeuil 2022) Two.	2 (Ooshita and Tixeuil 2022)	• tight
Colours	2 (Ooshita and Tixeuil 2022) Two, for two robots. With one colour, three robots suffice (the case $f = 0$ of the one-colour algorithm of Ooshita et al. 2025b).	2 (Ooshita and Tixeuil 2022)	• tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Ooshita and Tixeuil (2022) [lower bound]: robots 2. *Bottleneck:* symmetry breaking. *Holds when:* one robot sees nobody, so its snapshot is symmetric and the adversary chooses its direction (any φ, any number of colours); stated in Section 3.1.
- Ooshita and Tixeuil (2022) [lower bound]: colours 2. *Bottleneck:* symmetry breaking. *Holds when:* two robots, $n \geq 6$ (Lemma 6); two robots with the same colour take the same snapshot and move symmetrically.
- Ooshita and Tixeuil (2022) [current best]: robots 2, colours 2. *Assumes:* initial configuration: the two robots on adjacent nodes, coloured G and W; $n \geq 2$. Introductory and current best. Universal for $n \geq 6$: it works from every initial configuration from which two robots with colours $\{G, W\}$ can explore (Theorem 7).

Notes. The robot coloured G steps away from the other robot and the robot coloured W steps towards it; in FSYNC both move at once, so the pair keeps its shape and walks around the ring forever (Algorithm 1 of Ooshita and Tixeuil 2022). The colours give the pair a direction that the unoriented ring does not.

16.7.17 Rings, FSYNC, perpetual, up to f crashes

Adds + at most crash faults VERIFIED

$f + 2$ robots with $3f + 2$ colours explore a ring perpetually despite f crashes, which is optimal in robots.

Assumed: graph dynamics: static graph; edge direction: undirected; network: ring; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); faults: up to f agents may crash (stop forever). **Not assumed:** visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	$f + 2$ (Ooshita et al. 2025a) $f + 2$.	$f + 2$ (Ooshita et al. 2025a)	• tight
Colours	none known No lower bound on colours; with $f + 2$ robots the known algorithm uses $3f + 2$ colours, and fewer colours cost more robots (Ooshita et al. 2025b).	$3f + 2$ (Ooshita et al. 2025a)	• unknown

Results.

- Ooshita et al. (2025a) [lower bound]: robots $f + 2$. *Bottleneck:* trivial. *Holds when:* at least two robots must survive (fault-free bound of two robots); any φ and any number of colours.
- Ooshita et al. (2025a) [current best]: robots $f + 2$, colours $3f + 2$. *Assumes:* specific initial positions and colours; holds for every $\varphi \geq 1$. Introductory and current best in robots. A crashed robot keeps its colour and cannot be told apart from a correct one.
- Ooshita et al. (2025b) [also]: robots $6f + 3$, colours 1. *Assumes:* $\varphi = 1$. Oblivious robots; one of three robots–colours trade-offs.
- Ooshita et al. (2025b) [also]: robots $4f + 2$, colours 2. *Assumes:* $\varphi = 1$. Two colours.
- Ooshita et al. (2025b) [also]: robots $2f + 2$, colours $O(\log f)$. *Assumes:* $\varphi = 1$. Logarithmically many colours.

Open questions.

Q1. With $f + 2$ robots under FSYNC, how many colours are necessary for crash-tolerant perpetual ring exploration? The best algorithm uses $3f + 2$ (Ooshita et al. 2025a).

Notes. The trivial algorithm runs $f + 1$ independent pairs with distinct colours ($2f + 2$ robots, $2f + 2$ colours); (Ooshita et al. 2025a) lowers the robots to $f + 2$.

Claude (Anthropic) was used in cataloguing the problems.

16.7.18 Rings, ASYNC, perpetual, $\varphi \geq 2$

Adds + no faults

DRAFT

With visibility two, two robots with two colours explore a ring perpetually even under ASYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: ring; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; visibility range: $\varphi = 2$ (results hold for every $\varphi \geq 2$); faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	2 (Ooshita et al. 2025a) Two.	2 (Nagahama et al. 2023)	• tight
Colours	none known No lower bound on colours is reported.	2 (Nagahama et al. 2023)	• unknown

Results.

- Ooshita et al. (2025a) [lower bound]: robots 2. *Bottleneck:* symmetry breaking. *Holds when:* one robot sees nobody, so its snapshot is symmetric and the adversary chooses its direction (any φ, any number of colours); as stated for every φ and every number of colours in (Ooshita et al. 2025a).
- Nagahama et al. (2023) [current best]: robots 2, colours 2. As reported in Table 1 of (Ooshita et al. 2025a) (SSYNC and ASYNC, $\varphi \geq 2$); the paper itself was not read.

Notes. Larger visibility lets the distance between robots carry information that otherwise needs a robot, as Ooshita and Tixeuil anticipated (Ooshita and Tixeuil 2022).

16.7.19 Rings, ASYNC, perpetual, up to f crashes, $\varphi \geq 2$

Adds + at most crash faults

VERIFIED

$2f + 2$ robots are necessary and sufficient under SSYNC and ASYNC with $\varphi \geq 2$.

Assumed: graph dynamics: static graph; edge direction: undirected; network: ring; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; visibility range: $\varphi = 2$ (results hold for every $\varphi \geq 2$); faults: up to f agents may crash (stop forever). **Not assumed:** scheduler: asynchronous (ASYNC); chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	$2f + 2$ (Ooshita et al. 2025a) $2f + 2$.	$2f + 2$ (Ooshita et al. 2025a)	• tight
Colours	none known No lower bound on colours.	$2f + 2$ (Ooshita et al. 2025a)	• unknown

Results.

- Ooshita et al. (2025a) [lower bound]: robots $2f + 2$. *Bottleneck:* indistinguishable crashes. *Holds when:* any number of colours, SSYNC (hence ASYNC); $f + 1$ groups of the fault-free minimum of two.
- Ooshita et al. (2025a) [current best]: robots $2f + 2$, colours $2f + 2$. Introductory and current best: $f + 1$ pairs running the two-robot algorithm of (Nagahama et al. 2023).

Claude (Anthropic) was used in cataloguing the problems.

16.7.20 Rings, ASYNC, perpetual, $\varphi = 1$

Adds ASYNC, $\varphi = 1$, + no faults VERIFIED

Three robots with two colours explore a ring perpetually under ASYNC; two robots cannot even under SSYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: ring; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Ooshita and Tixeuil 2022) Three.	3 (Ooshita and Tixeuil 2022)	• tight
Colours	none known No lower bound on colours for three robots is stated.	2 (Ooshita and Tixeuil 2022)	• unknown

Results.

- Ooshita and Tixeuil (2022) [lower bound]: robots 3. *Bottleneck:* adversarial activation. *Holds when:* two robots, any number of colours, $n \geq 6$, proved under SSYNC (Theorem 12); when a robot steps away from the other, the adversary activates it alone; the two robots are then too far apart to see each other and stay confined to two nodes each (Lemma 4).
- Ooshita and Tixeuil (2022) [current best]: robots 3, colours 2. *Assumes:* designated initial configurations of three connected robots (e.g. WWG, WGG); $n \geq 3$. Introductory and current best (Theorem 13); universal for $n \geq 9$ under SSYNC and ASYNC (Theorem 14).

Open questions.

Q1. Can three oblivious robots ($c = 1$) with $\varphi = 1$ explore a ring perpetually under SSYNC or ASYNC, or are two colours necessary?

Notes. With $\varphi \geq 2$ two robots suffice (Nagahama et al. 2023) (variant *Rings, ASYNC, perpetual, $\varphi = 2$*).

16.7.21 Rings, ASYNC, perpetual, up to f crashes, $\varphi = 1$

Adds ASYNC, $\varphi = 1$, + at most crash faults VERIFIED

$3f + 3$ robots are necessary and sufficient under SSYNC and ASYNC with $\varphi = 1$.

Assumed: graph dynamics: static graph; edge direction: undirected; network: ring; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; faults: up to f agents may crash (stop forever). **Not assumed:** scheduler: asynchronous (ASYNC); visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	$3f + 3$ (Ooshita et al. 2025a) $3f + 3$.	$3f + 3$ (Ooshita et al. 2025a)	• tight
Colours	none known No lower bound on colours.	$2f + 2$ (Ooshita et al. 2025a)	• unknown

Results.

- Ooshita et al. (2025a) [lower bound]: robots $3f + 3$. *Bottleneck:* indistinguishable crashes. *Holds when:* any number of colours, SSYNC (hence ASYNC); a slow robot looks crashed; robots must leave a group of three behind each apparent crash, so f dispersed crashes force $f + 1$ groups of the fault-free minimum of three.

Claude (Anthropic) was used in cataloguing the problems.

- Ooshita et al. (2025a) [current best]: robots $3f + 3$, colours $2f + 2$. Introductory and current best: $f + 1$ groups of three, each running the fault-free algorithm of (Ooshita and Tixeuil 2022) with its own pair of colours.

16.7.22 Rings, FSYNC, terminating, $\varphi = 1$

Adds + FSYNC, + no faults STANDARD SETTING VERIFIED

Three robots with two colours explore a ring and stop under FSYNC; two robots cannot, whatever the number of colours.

Assumed: graph dynamics: static graph; edge direction: undirected; network: ring; exploration goal: terminating exploration (explore, then stop); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); faults: no faults. **Not assumed:** visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Ooshita and Tixeuil 2022) Three.	3 (Ooshita and Tixeuil 2022)	• tight
Colours	2 (Datta et al. 2013) Two, for three robots: oblivious robots need five (Datta et al. 2013).	2 (Ooshita and Tixeuil 2022)	• tight

Results.

- Ooshita and Tixeuil (2022) [lower bound]: robots 3. *Bottleneck:* locality. *Holds when:* two robots, any number of colours (Theorem 8); while two robots stay within distance one they cannot tell a ring of n_1 nodes from one of $n_2 > 2(t + 1)$ nodes, so they stop too early in the larger ring; once they separate, each is confined to two nodes (Lemma 4).
- Datta et al. (2013) [lower bound]: colours 2. *Bottleneck:* symmetry breaking. *Holds when:* three or four oblivious robots on rings with $n > 7$ (Theorem 3.2).
- Ooshita and Tixeuil (2022) [current best]: robots 3, colours 2. *Assumes:* designated initial configurations: three robots on consecutive nodes coloured WWW, GWW, WWG or GWG; $n \geq 3$. Introductory and current best (Theorem 9). No algorithm with three robots and two colours is universal (Theorem 11).

Open questions.

Q1. Characterise the initial configurations of three robots with $c \geq 3$ colours from which terminating exploration of a ring is solvable under FSYNC with $\varphi = 1$ (with two colours no universal algorithm exists Ooshita and Tixeuil 2022).

Notes. From GWG the middle robot joins one end robot, forming a pair GW that walks away from the third robot, which waits; when the pair arrives at the waiting robot from the other side every node has been visited, and one more move forms a tower that no rule enables (Algorithm 2 of Ooshita and Tixeuil 2022). Without lights five robots are necessary and sufficient (Datta et al. 2013) (variant *Rings, oblivious, FSYNC, terminating*).

16.7.23 Rings, ASYNC, terminating, $\varphi = 1$

Adds ASYNC, + no faults VERIFIED

Four robots with two colours explore a ring and stop under ASYNC; three cannot even under SSYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: ring; exploration goal: terminating exploration (explore, then stop); lights (memory and messages): full light (LUMI): visible to the robot and to the others; faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Robots	4 (Ooshita and Tixeuil 2022) Four.	4 (Ooshita and Tixeuil 2022)	• tight
Colours	none known Two: oblivious robots cannot explore at all under SSYNC or ASYNC with $\varphi = 1$ (Datta et al. 2013).	2 (Ooshita and Tixeuil 2022)	• tight

Results.

- Ooshita and Tixeuil (2022) [lower bound]: robots 4. *Bottleneck:* locality. *Holds when:* three robots, any number of colours, proved under SSYNC (Theorem 15); robots that stay close cannot tell a ring of $n_1 \geq 9$ nodes from one of $4(t + 1)$ nodes.
- Ooshita and Tixeuil (2022) [current best]: robots 4, colours 2. *Assumes:* designated initial configurations of four robots on consecutive nodes (WWGG, WWWG, WWGW and colour swaps); $n \geq 5$. Introductory and current best (Theorem 16). No universal algorithm with four robots and two colours exists (Theorem 18).

Open questions.

Q1. Does any task separate SSYNC from ASYNC for myopic luminous robots on rings? The known exploration bounds coincide under both (Ooshita and Tixeuil 2022).

Notes. Without lights, terminating exploration with $\varphi = 1$ is impossible under SSYNC and ASYNC for $1 \leq k < n$ (Datta et al. 2013). With $\varphi \geq 2$, Nagahama, Ooshita and Inoue report that three robots with two colours are necessary and sufficient for terminating exploration (abstract of their IPSJ technical report, the precursor of Nagahama et al. 2023); not checked against the paper, and the scheduler was not confirmed.

16.7.24 Rings, oblivious, FSYNC, terminating, $\varphi = 1$

Adds oblivious, + FSYNC, + no faults VERIFIED

Without lights, five robots are necessary and sufficient to explore a ring of more than seven nodes and stop under FSYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: ring; exploration goal: terminating exploration (explore, then stop); scheduler: fully synchronous (FSYNC); faults: no faults. **Not assumed:** lights (memory and messages): oblivious, no light (OBLOT); visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	5 (Datta et al. 2013) Five.	5 (Datta et al. 2013)	• tight
Colours	none known No light.	1 (Datta et al. 2013)	• tight

Results.

- Datta et al. (2013) [lower bound]: robots 5. *Bottleneck:* symmetry breaking. *Holds when:* $n > 7$, $2 \leq k < 5$, deterministic (Theorem 3.2); from one block of adjacent robots, every rule either repeats an indistinguishable configuration or disperses the robots.
- Datta et al. (2013) [current best]: robots 5, colours 1. *Assumes:* initial configuration: one block of five robots on consecutive nodes; $n \geq 7$; for $3 \leq n \leq 6$ the paper uses $n - 1$ robots (4 for $n = 6$). Introductory and current best.

Notes. Baseline for the power of lights: two colours bring the number of robots from five to three (Ooshita and Tixeuil 2022).

Claude (Anthropic) was used in cataloguing the problems.

16.7.25 Rings, oblivious, ASYNC, terminating, $\varphi = 1$

Adds oblivious, ASYNC, + no faults VERIFIED

Without lights and with $\varphi = 1$, no number of robots below n explores a ring under SSYNC or ASYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: ring; exploration goal: terminating exploration (explore, then stop); faults: no faults. **Not assumed:** lights (memory and messages): oblivious, no light (OBLOT); scheduler: asynchronous (ASYNC); visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	*impossible* (Datta et al. 2013) For every $k < n$.	—	• impossible
Colours	none known No light.	—	• impossible

Results.

- Datta et al. (2013) [lower bound]: robots *impossible*. *Bottleneck:* symmetry breaking. *Holds when:* $1 \leq k < n, n > 1$, deterministic, proved under SSYNC (Theorem 3.1, Corollary 3.2).

Notes. With $\varphi = 2$ the same authors explore with seven or nine robots under ASYNC, and with $\varphi = 3$ they prove that fewer than five robots cannot explore (for $n > 13$) and give five- and seven-robot algorithms (Datta et al. 2013). Two colours make $\varphi = 1$ enough with four robots (Ooshita and Tixeuil 2022).

16.7.26 Tori, FSYNC, perpetual, chirality, $\varphi = 2$

Adds + chirality, + no faults VERIFIED

With $\varphi = 2$ and chirality, three robots with two colours or four oblivious robots explore a torus perpetually under FSYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: torus; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); visibility range: $\varphi = 2$ (results hold for every $\varphi \geq 2$); chirality: common chirality (same clockwise direction); faults: no faults. **Not assumed:** identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Darwich et al. 2023) Three.	3 (Darwich et al. 2023)	• tight
Colours	2 (Darwich et al. 2023) Two for three robots; one for four.	2 (Darwich et al. 2023)	• tight

Results.

- Darwich et al. (2023) [lower bound]: robots 3. *Bottleneck:* symmetry breaking. *Holds when:* two robots (Theorem 1).
- Darwich et al. (2023) [lower bound]: colours 2. *Bottleneck:* symmetry breaking. *Holds when:* three oblivious robots, any finite φ (Theorem 2).
- Darwich et al. (2023) [current best]: robots 3, colours 2. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together; opaque robots; torus with both sides at least $k\varphi + 1$. Algorithm A_1, designed for $\varphi = 1$.

Claude (Anthropic) was used in cataloguing the problems.

- Darwich et al. (2023) [also]: robots 4, colours 1. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together; opaque robots; torus with both sides at least $k\varphi + 1$. Algorithm A_3: oblivious robots, optimal for $c = 1$.

16.7.27 Tori, FSYNC, perpetual, no chirality, $\varphi = 2$

Adds no chirality, + no faults VERIFIED

Without chirality and with $\varphi = 2$, three robots with three colours or five oblivious robots explore a torus perpetually under FSYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: torus; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); visibility range: $\varphi = 2$ (results hold for every $\varphi \geq 2$); faults: no faults. **Not assumed:** chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Darwich et al. 2023) Three.	3 (Darwich et al. 2023)	• tight
Colours	none known Constant on both sides, so tight up to a constant: two against three, for three robots. The exact number is open.	3 (Darwich et al. 2023)	• tight

Results.

- Darwich et al. (2023) [lower bound]: robots 3. *Bottleneck:* symmetry breaking. *Holds when:* two robots (Theorem 1).
- Darwich et al. (2023) [current best]: robots 3, colours 3. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together; opaque robots; torus with both sides at least $k\varphi + 1$; self-inconsistent robots. Algorithm A_2, designed for $\varphi = 1$.
- Darwich et al. (2023) [also]: robots 5, colours 1. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together; opaque robots; torus with both sides at least $k\varphi + 1$; self-inconsistent robots. Algorithm A_4: oblivious robots; three oblivious robots cannot (Theorem 2), so four or five is open for $c = 1$.

Open questions.

Q1. Can four oblivious robots with $\varphi = 2$ explore a torus perpetually without chirality?

16.7.28 Tori, FSYNC, perpetual, chirality, $\varphi = 1$

Adds $\varphi = 1$, + chirality, + no faults VERIFIED

Three robots with two colours explore a torus perpetually under FSYNC with chirality, optimal in robots and colours.

Assumed: graph dynamics: static graph; edge direction: undirected; network: torus; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); chirality: common chirality (same clockwise direction); faults: no faults. **Not assumed:** visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Darwich et al. 2023) Three.	3 (Darwich et al. 2023)	• tight
Colours	2 (Darwich et al. 2023) Two.	2 (Darwich et al. 2023)	• tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Darwich et al. (2023) [lower bound]: robots 3. *Bottleneck:* symmetry breaking. *Holds when:* two robots, any finite colours and φ, even with chirality (Theorem 1); two robots that travel together follow an integer vector, which fails on tori whose sides are multiples of its components.
- Darwich et al. (2023) [lower bound]: colours 2. *Bottleneck:* symmetry breaking. *Holds when:* three oblivious robots, any finite φ (Theorem 2).
- Darwich et al. (2023) [current best]: robots 3, colours 2. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together; opaque robots; torus with both sides at least $k\varphi + 1$. Introductory and current best (algorithm A_1). One robot more than on grids, which have walls to turn at.

16.7.29 Tori, FSYNC, perpetual, no chirality, $\varphi = 1$

Adds $\varphi = 1$, no chirality, + no faults VERIFIED

Without chirality, three robots with three colours explore a torus perpetually under FSYNC.

Assumed: graph dynamics: static graph; edge direction: undirected; network: torus; exploration goal: perpetual exploration (visit every node infinitely often); lights (memory and messages): full light (LUMI): visible to the robot and to the others; scheduler: fully synchronous (FSYNC); faults: no faults. **Not assumed:** visibility range: myopic, $\varphi = 1$: a robot sees its node and its neighbours; chirality: no common chirality; identifiers: anonymous: all agents are identical.

Measure	Lower bound	Best upper bound	Status
Robots	3 (Darwich et al. 2023) Three.	3 (Darwich et al. 2023)	• tight
Colours	2 (Darwich et al. 2023) Constant on both sides, so tight up to a constant: two against three. The exact number is open.	3 (Darwich et al. 2023)	• tight

Results.

- Darwich et al. (2023) [lower bound]: robots 3. *Bottleneck:* symmetry breaking. *Holds when:* two robots, even with chirality (Theorem 1).
- Darwich et al. (2023) [lower bound]: colours 2. *Bottleneck:* symmetry breaking. *Holds when:* three oblivious robots (Theorem 2).
- Darwich et al. (2023) [current best]: robots 3, colours 3. *Assumes:* exclusive: no two robots ever share a node or traverse an edge together; opaque robots; torus with both sides at least $k\varphi + 1$; self-inconsistent robots. Algorithm A_2 (journal version only).

Open questions.

Q1. Can three robots with two colours and $\varphi = 1$ explore a torus perpetually without common chirality?

16.8 Exploration of recolourable graphs

A single agent with little or no memory must visit every node of an anonymous graph whose nodes carry colours it can read and rewrite; how many colours are needed, and how much does it help to recolour, to see neighbours or to have port numbers?

First posed in 2023 (Böckenhauer et al. 2023). Böckenhauer, Frei, Unger and Wehner posed exploration by an oblivious agent without port numbers that colours nodes and sees its neighbours' colours (SIROCCO 2023; arXiv January 2023), first with each node coloured once and, in a last section, with recolouring.

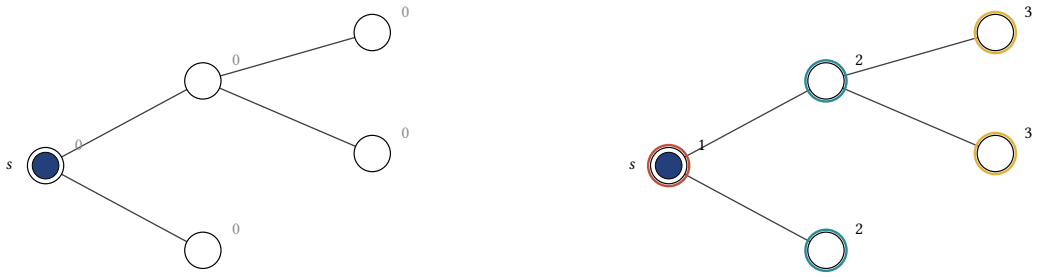
Claude (Anthropic) was used in cataloguing the problems.

Fixed colours chosen by a designer (2005) and the rotor-router pointer (1996, 2003) are earlier node-side precursors.

Problem statement and measures

A single deterministic agent starts at a node s of a connected, simple, undirected, anonymous graph $G = (V, E)$ with n nodes, m edges, maximum degree Δ and diameter D. Every node v holds a colour $\psi(v)$ from a palette of c colours that persists between visits. In each step the agent at v observes $\psi(v)$ (and, depending on the setting, $\deg(v)$, port numbers, the entry port, and the multiset of its neighbours' colours), sets $\psi(v)$ to a new colour (**recolours** v) if the setting allows it, and then moves to a neighbour or stops. In the main model (Böckenhauer et al. 2023) there are no port numbers: the agent names a colour and the adversary moves it to any neighbour of that colour; the agent is **oblivious** (it keeps no memory between steps); all nodes start with the **initial colour** c_0; and the agent must **visit every node, return to** s **and stop there**, from every start and against every adversary. Other settings fix the colours in advance (label-guided exploration), give port numbers, or ask only for perpetual exploration. **Measures:** the number of colours c, counting c_0 (the main measure); time, the number of moves (traversals of one edge) until termination (or until every node is visited, for perpetual exploration); and the agent's memory. All known results are for one agent.

Measures: **Colours**, number c of node colours, counting the initial colour c_0 (for the rotor-router: per node); **Time**, moves until the agent stops (perpetual: until every node is visited); **Memory**, bits of agent memory; 0 for an oblivious agent.



(a) Start: every node has colour 0

(b) Back at s: colour 1, no neighbour of colour 3, so it stops

Figure 16.8: Exploration with colourable nodes (Böckenhauer et al. 2023): an oblivious agent on a tree without port numbers sees the colour of its node and of its neighbours. On an uncoloured node it recolours the node with one more than the largest neighbouring colour (cyclically 1, 2, 3), which is its depth mod 3 plus one; it moves to an uncoloured neighbour if there is one (the adversary picks which) and otherwise to its parent, the unique neighbour of the previous colour; it stops at the start, the only node of colour 1 without a neighbour of colour 3. The coloured rings show the colours of the nodes; the agent itself remembers nothing.

A first look

Imagine exploring a town of identical squares joined by streets, with no street signs and no memory at all: after each step you forget everything. What you do have is a box of coloured flags, one flagpole on every square, and good eyes: standing on a square you see its flag and the flags on the squares next to it. You may change the flag where you stand, then name a colour and walk to a neighbouring square showing that colour; if several do, an **adversary** (an imagined opponent) picks which one. We model the town as a graph: squares are **nodes**, streets are **edges**, the flag on a square is the **colour** of that

Claude (Anthropic) was used in cataloguing the problems.

node, and you are the **agent**, which is **oblivious** (it remembers nothing between steps); n is the number of nodes. In the basic version every node starts with the same colour 0, the **initial colour** (a node that still has it is **uncoloured**), you may **recolour** the node where you stand (change its colour) as often as you like, and you must visit every node, come back to where you started and stop there. The question is how many colours c are needed for every possible graph.

1. Algorithm: Colour by depth mod 3 (trees). (Böckenhauer et al. 2023) (Once-colourable nodes on trees, p. 165) For a graph without cycles (a **tree**):

1. On an uncoloured node, recolour it with the colour one more than the largest colour among its neighbours, counting cyclically 1, 2, 3, 1, ... (at the start all neighbours are 0, so the start gets 1).
2. If a neighbour is uncoloured, walk to one.
3. Otherwise, if your node has colour 1 and no neighbour of colour 3, stop.
4. Otherwise walk to the neighbour whose colour is one less than yours (cyclically): it is your **parent**, the neighbour one edge closer to the start.

Why it works and what it costs. A node at distance j from the start gets colour $(j \bmod 3) + 1$, so its parent has the previous colour and its children the next one; in a tree the parent is the only neighbour with the previous colour, so the way back is never ambiguous. The start is the only node of colour 1 without a neighbour of colour 3. Each edge is walked once down and once up, so the walk has $2(n-1)$ **moves** (one move is one traversal of one edge). Three colours plus the initial one are needed: on a path of seven nodes two colours (plus 0) always leave the agent unable to tell the way back (Böckenhauer et al. 2023). With recolouring allowed, three colours in all suffice on trees (Hiraoka et al. 2026).

Why it matters. Depth mod 3 is the basic trick of the whole area: a designer uses it in label-guided exploration (Cohen et al. 2008), and the general eight-colour algorithm is this idea with a phase colour on top (Böckenhauer et al. 2023).

2. Algorithm: The rotor-router: a pointer as the colour. (Menc et al. 2017) (Rotor-router: a pointer in every node, p. 167) Now the edges at each node are numbered $1, \dots, d$, where d is the **degree** of the node (its number of edges); these numbers are **ports**. You are not told through which port you arrived. The colour of a node of degree d is a number π between 1 and d.

1. Leave through port π.
2. Before leaving, recolour the node to $\pi + 1$ (and after d back to 1).
3. Repeat forever.

Why it works and what it costs. Each node sends you out through its ports in turn, so no edge is neglected for long. Whatever the colours are at the start, after $O(mD)$ moves (m edges, D the largest distance between two nodes) the walk follows a fixed tour that uses every edge once in each direction, so every node is visited (Yanovski et al. 2003) (Menc et al. 2017). A node of degree d needs d colours, since otherwise some port can never be chosen, and no oblivious rule without the arrival port is faster in the worst case, even with unlimited colours (Menc et al. 2017).

Why it matters. It shows that memory can live entirely in the nodes, and that the colours can even start in a mess (self-stabilization).

3. Lower bound: Colours that can never be changed cost n colours. (Böckenhauer et al. 2023) (Once-colourable nodes, any graph, p. 164) Suppose a colour, once set, stays forever. To return to the start the agent must always recognise which neighbour it came from. The adversary builds graphs with a long

Claude (Anthropic) was used in cataloguing the problems.

path whose nodes have extra edges between them, so that a node on the way back sees several coloured neighbours, and any repeated colour lets the adversary send the agent the wrong way or around in a loop.

Why it holds. Böckenhauer et al. turn this into a proof that for every n some n-node graph forces n colours, counting the initial one, and a depth-first search that gives each new node a colour larger than its coloured neighbours achieves it (Böckenhauer et al. 2023). With recolouring the agent can erase and reuse colours, and six colours suffice for all graphs (Takahashi et al. 2026); at least four are needed (Hiraoka et al. 2026).

Why it matters. The gap between n and a constant is the measure of what the right to recolour is worth.

What the community started looking at

A flag planted in the node. Picture every node with a flagpole. The flag shows one of c colours, and it stays up when the agent leaves: whoever arrives later, the same agent included, sees it, and in the main model an agent also sees the flags on the neighbouring nodes. Compare the three other ways the catalogue leaves information behind. A luminous robot's light is a flag on the robot's own mast: it moves with its owner and is a message to the robots in sight and a memory for the owner (Das et al. 2016) (problem *Exploration by luminous robots*); here the flag stays in place and is a memory of the *place*, read by whoever comes by. A whiteboard is the same idea with an unbounded or $O(\log n)$-bit board (Dobrev et al. 2006); a recolourable node is a whiteboard of $\log_2 c$ bits with c a constant. A pebble or token is a marker the agent carries and drops (Blum and Kozen 1978) (Disser et al. 2019) (Dobrev et al. 2013): there are few of them and they move, whereas every node has its own flag and none ever moves.

Where it started. Fixed flags came first: Cohen, Fraigniaud, Ilcinkas, Korman and Peleg let a designer colour the nodes in advance, and three colours (depth mod 3 from a root) let a finite automaton explore every graph (Cohen et al. 2008), although no finite automaton explores all graphs without help (Fraigniaud et al. 2005a). The rotor-router, from the Eulerian walkers of physics (Priezzhev et al. 1996) and the patrolling ants of Yanovski, Wagner and Bruckstein (Yanovski et al. 2003), is a node colour that the agent rewrites by a fixed rule: a pointer to the next port to use, which explores any graph from any initial pointers in $O(mD)$ moves, and no oblivious agent that is not told its entry port does better in the worst case (Menc et al. 2017). Böckenhauer, Frei, Unger and Wehner then posed the problem in its present form: no port numbers, no memory, the agent colours nodes and sees its neighbours' colours. If each node can be coloured only once, n colours are necessary and sufficient, four for trees; with recolouring, eight suffice for every graph (Böckenhauer et al. 2023). Takahashi, Kanaya, Hiraoka, Eguchi and Sudo brought this down to six with a depth-first search whose return path never has a chord (Takahashi et al. 2026), and Hiraoka, Imori, Takahashi and Sudo proved the first lower bound, four colours, by a computer-checked argument, which is tight for cacti, and three colours for trees (Hiraoka et al. 2026).

What the colours do. In every algorithm the colours give the agent a sense of direction that the anonymous graph lacks: depth mod 3 tells parent from child (Cohen et al. 2008) (Böckenhauer et al. 2023), a path colour marks the way back (Takahashi et al. 2026), a pointer tells which port is next (Menc et al. 2017). Recolouring is what makes the palette constant: a once-coloured node must keep a colour that distinguishes it forever, and that costs n colours (Böckenhauer et al. 2023). All known results are for a single agent.

Further reading

Papers on this problem that are not yet placed in a setting.

Claude (Anthropic) was used in cataloguing the problems.

- Bojko et al. (2022). Tree exploration with unknown entry port, constant agent memory and $O(\log \deg)$ bits per node: linear time with clean node memory or one token; quadratic time on paths with one bit at the agent and one per node if node memory is dirty (arbitrary). Abstract and introduction read.
- Inoue et al. (2023). With $O(\log \Delta)$ bits per node, an oblivious agent simulates a one-bit agent in two-edge-connected graphs with $O(n^2)$ overhead; bridges are what separates them. Abstract read.
- Sudo et al. (2026). Self-stabilizing exploration by one agent from arbitrary agent and node memory, using whiteboards of $O(\log)$ size; cover time $O(m)$ expected (randomized) and $O(m + nD)$ (deterministic). Not read; the arXiv versions state different memory bounds.
- Shimoyama et al. (2022). One bit of agent memory suffices for snap-stabilizing perpetual exploration of cacti with distinguishable cycles (title and description in Hiraoka et al. 2026). Not read.
- Sudo et al. (2015). Trade-offs between agent memory and whiteboard size for single-agent exploration. Not read.
- Bampas et al. (2009). Rotor-router lock-in: $\Theta(mD)$ when the adversary sets pointers and ports, $O(m \min(\log m, D))$ when pointers are chosen first. Abstract and talk slides read.
- Zhang et al. (2012). Label-guided exploration with 1-bit labels whose black/white ratio the designer can adjust, with $O(\rho \log \Delta)$ bits of memory. Abstract read.

Relations to other problems

- **Variant of Graph exploration by a single agent.** The same task for one agent (visit every node; with return, stop or perpetually), but every node holds a colour from a palette of c colours that the agent at the node reads and, unless the colours are fixed, overwrites; the agent itself may have no memory at all, and in the main model it has no port numbers and sees its neighbours' colours instead. With $c = 1$ the colours carry nothing: an algorithm of the parent problem that uses no markers works here whenever this setting grants its ports, entry port and memory, and a lower bound of this problem proved for any number of colours holds for the parent setting with the same view. The parent's pebbles are movable markers carried by the agent (Pattanayak and Pelc 2024a) (Disser et al. 2019); here the marks stay in the nodes and there is one per node.
- **Related: Exploration by luminous robots.** Flag on the robot versus flag on the node: a luminous robot carries its coloured light with it and shows it to the robots in sight, whereas here the colour stays planted in the node and is read by whoever arrives.

Assumptions specific to this problem

Port numbers

Port numbers are the usual way an agent tells the edges at a node apart (Fraigniaud et al. 2005a) (Pelc 2019), and most exploration papers also give the agent the port by which it entered. The rotor-router drops the entry port: the agent only follows and advances a pointer stored at the node, which works from any initial pointers (Yanovski et al. 2003) (Menc et al. 2017). Menc, Pająk and Uznański call this model feature an *unknown inport* and prove that it costs time even with unbounded node memory (Menc et al. 2017). Böckenhauer, Frei, Unger and Wehner removed the port numbers altogether: the agent names a colour and the adversary chooses among the neighbours of that colour, so the colours are the only way to address an edge (Böckenhauer et al. 2023). Takahashi, Kanaya, Hiraoka, Eguchi and Sudo note that without port numbers \"most commonly used techniques for graph exploration and related problems are no longer applicable\" (Takahashi et al. 2026). The scale runs from no ports, through ports without the entry port, to ports with it.

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**no ports**	Edges carry no port numbers. To move, the agent at v names a colour x; the adversary moves it to any neighbour u of v with $\psi(u) = x$ (if the agent sees no neighbour colours, to any neighbour). The agent never learns by which edge it arrived.
1	**ports, no entry port**	The edges at v carry port numbers $1, \dots, \deg(v)$, assigned by an adversary independently at each endpoint (port_v). The agent chooses the port by which it leaves, but on arrival it is not told the port by which it entered (*unknown inport*).
2	**ports + entry port**	As for port numbers, and on arrival at v the agent also reads the port $\text{port}_v(e)$ of the edge e by which it entered. This is the standard convention of single-agent exploration.

View

In label-guided exploration (Cohen et al. 2008) and in the rotor-router (Menc et al. 2017) the agent reads only the node it stands on, as in all of single-agent exploration. Böckenhauer, Frei, Unger and Wehner observe that colouring alone \"is still utterly useless for an oblivious agent that has to return to the start vertex\" when it cannot tell edges apart, and therefore let the agent see the colours of the neighbouring nodes (Böckenhauer et al. 2023); every later paper on recolourable exploration keeps this one-hop view (Takahashi et al. 2026) (Hiraoka et al. 2026). It is the graph analogue of the one-hop visibility of myopic luminous robots, except that what is seen are flags in nodes, not robots.

Level	Value	Formal definition
0	**own node**	At node v the agent observes $\psi(v)$, $\deg(v)$ and nothing about its neighbours (and, where ports exist, what the port axis grants).
1	**neighbour colours**	At node v the agent observes $\psi(v)$ and the multiset $\{\!\{\psi(u) : u \in N(v)\}\!\}$ of its neighbours' colours (hence also $\deg(v)$), without knowing which edge leads to which colour unless ports say so.

Agent memory

How little memory an exploring agent needs is the oldest question of the area: no finite automaton explores all graphs (Budach 1978) (Fraigniaud et al. 2005a), and the remedies are markers (Blum and Kozen 1978) (Disser et al. 2019) or help from a designer (Cohen et al. 2008). Recolourable exploration pushes this to the extreme: the agent is oblivious and all memory lives in the nodes (Böckenhauer et al. 2023), as in the rotor-router (Menc et al. 2017) and the memoryless automaton with pebbles (Pattanayak and Pelc 2024a). Both later papers ask what a constant number of bits of agent memory would buy (Böckenhauer et al. 2023) (Takahashi et al. 2026); Inoue, Kitamura, Izumi and Masuzawa compare an oblivious agent with a one-bit agent when nodes hold $O(\log \Delta)$ bits (Inoue et al. 2023).

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**oblivious**	The agent has no internal state: its action (new colour of v, then a move, *stay* or *stop*) is a function of its current observation only. Called *zero-memory*, *memoryless* or a *1-state* robot.
1	**finite automaton**	The agent is a deterministic automaton with K states, K independent of the graph; its action and next state depend on its state and its observation.
2	$O(\log n)$ **bits**	The agent has $b = O(\log n)$ bits of memory, enough to hold a counter up to a polynomial in n. No variant is catalogued here yet.

Recolouring

Fixed colours are the label-guided exploration of Cohen, Fraigniaud, Ilcinkas, Korman and Peleg: a designer writes 2-bit labels at the nodes in advance and a finite automaton reads them (Cohen et al. 2008). Böckenhauer, Frei, Unger and Wehner let the agent write the colour itself while exploring, once per node, and showed in a last section that allowing recolouring \"radically changes our model\": seven colours (eight with the initial one) then explore every graph, against n without it (Böckenhauer et al. 2023). A recolourable node is a whiteboard of $\log_2 c$ bits; the rotor-router is the special case in which the colour is a pointer that the agent advances on every visit (Yanovski et al. 2003) (Menc et al. 2017). Whether a colour may return to c_0 matters for exact counts: Böckenhauer et al. intended that it may not, Takahashi et al. allow it, and their six-colour algorithm needs seven colours without it (Takahashi et al. 2026).

Level	Value	Formal definition
0	**fixed colours**	The colouring $\psi : V \to \{1, \dots, c\}$ never changes; the agent reads $\psi(v)$ (and, with neighbour view, its neighbours' colours) but cannot write.
1	**colour once**	Every node starts with the initial colour c_0. The agent at v may change $\psi(v)$ only while $\psi(v) = c_0$; once v has a colour other than c_0 it keeps it forever. This is the main model of (Böckenhauer et al. 2023).
2	**recolourable**	At every step the agent at v may set $\psi(v)$ to any colour of the palette, including back to c_0, any number of times (the convention of Takahashi et al. 2026 and Hiraoka et al. 2026).

Initial colouring

The rotor-router is studied from arbitrary pointers, which makes it self-stabilizing (Yanovski et al. 2003) (Bampas et al. 2009) (Menc et al. 2017); Bojko, Gotfryd, Kowalski and Pająk call arbitrary initial node memory *dirty*: with one bit at the agent and one bit per node it can force quadratic time on paths, whereas with *clean* (uniformly initialised) logarithmic node memory and constant agent memory trees are explored in linear time (Bojko et al. 2022). Recolourable exploration starts from a uniform colouring: \"at the start of exploration, all nodes are assigned a common initial color\" (Takahashi et al. 2026). A designer's colouring is the preprocessing of label-guided exploration (Cohen et al. 2008). The scale is arbitrary < uniform < designer: an algorithm that works from every colouring works from the uniform one, and a designer may choose the uniform one.

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**arbitrary colours**	The initial colouring $\psi_0 : V \to \{1, \dots, c\}$ is chosen by the adversary; an algorithm must work from every ψ_0 (as with a rotor-router whose pointers are set arbitrarily).
1	**uniform c_0**	Initially $\psi(v) = c_0$ for every node v (*clean* node memory). The colour c_0 counts as one of the c colours.
2	**designer's colours**	Before the agent starts, a designer who knows G (and the start node) chooses ψ_0. Used with fixed colours this is label-guided exploration (Cohen et al. 2008).

Knowledge

Böckenhauer, Frei, Unger and Wehner note that their general lower bound \"relies heavily on the uniformity of the algorithm\" and study algorithms that need only explore graphs of a given size n, to see \"how useful it is to know the size of the graph in our model\" (Böckenhauer et al. 2023). As in single-agent exploration, knowing the size is the classical way to let a walk end (Aleliunas et al. 1979).

Level	Value	Formal definition
0	**nothing known**	The algorithm is the same for all graphs: it knows neither n nor Δ nor any other parameter (a *uniform* algorithm in Böckenhauer et al. 2023).
1	**n known**	The algorithm may depend on n: it must explore every graph with exactly n nodes (a *non-uniform* algorithm).

Termination requirement

The three requirements are those of single-agent exploration (Fraigniaud et al. 2005a). Recolourable exploration asks for the hardest, return: \"visit every vertex of the graph, return to the start vertex, and terminate\" (Böckenhauer et al. 2023) (Takahashi et al. 2026) (Hiraoka et al. 2026). Label-guided exploration asks the agent to stop (Cohen et al. 2008), and the rotor-router never stops (Yanovski et al. 2003).

Level	Value	Formal definition
0	**return**	The agent visits every node and then halts at its start node s (*exploration with return*).
1	**stop**	The agent visits every node and then halts, at any node.
2	**perpetual**	Every node is visited at least once (for the rotor-router: infinitely often); the agent never needs to halt.

Network

Colour counts depend strongly on the class of graphs: for once-colourable nodes trees need four colours and general graphs n (Böckenhauer et al. 2023), and for recolourable nodes the bounds are organised by block structure, three colours for trees and cycles and four for cacti (Hiraoka et al. 2026). This axis replaces the model's topology axis because it must order trees below cacti; the other classes studied (triangle-free, φ-free, graphs whose blocks are cycles, complete bipartite graphs or cliques, graphs of bounded circumference) are recorded in the notes.

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**any graph**	A finite, simple, connected, undirected, anonymous graph $G = (V, E)$ with n nodes.
1	**cactus**	G is a cactus: every edge lies on at most one cycle (equivalently, every block is a cycle or a single edge).
2	**tree**	G is a tree (a cactus without cycles).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Colours, Time, Memory) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Port numbers → View → Agent memory → Recolouring → Initial colouring → Knowledge → Termination requirement → Network.



What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Claude (Anthropic) was used in cataloguing the problems.

Setting	Colours	Time	Memory	p.
Recolourable nodes, oblivious agent, any graph STANDARD SETTING	● tight 6 vs 4	● unknown — vs $\Omega(n)$	● tight 0 vs —	161
Recolourable nodes on cacti	● tight 4 vs 4	● tight $O(n)$ vs $\Omega(n)$	● tight 0 vs —	162
Recolourable nodes on trees	● tight 3 vs 3	● tight $O(n)$ vs $\Omega(n)$	● tight 0 vs —	163
Once-colourable nodes, any graph	● tight n vs n	● unknown — vs —	● tight 0 vs —	164
Once-colourable nodes, n known	● tight $n-1$ vs $n-2$	● unknown — vs —	● tight 0 vs —	165
Once-colourable nodes on trees	● tight 4 vs 4	● unknown — vs —	● tight 0 vs —	165
Label-guided: colours fixed by a designer	● tight 3 vs 2	● unknown $O(m)$ vs $\Omega(n)$	● tight $O(1)$ vs —	166
Label-guided, oblivious agent	● unknown — vs $\lfloor \log \Delta \rfloor - 1$	● unknown — vs —	● tight — vs —	167
Rotor-router: a pointer in every node	● tight $\deg(v)$ vs $\deg(v)$ at each node v	● tight $O(mD)$ vs $\Omega(n^3)$	● tight 0 vs —	167

16.8.1 Recolourable nodes, oblivious agent, any graph

Adds + recolourable, + uniform c_0 STANDARD SETTING VERIFIED

An oblivious agent without port numbers explores every graph and returns with six colours; at least four are needed.

Assumed: graph dynamics: static graph; edge direction: undirected; view: the agent also sees the colours of its neighbours; recolouring: recolourable: any colour, any number of times; initial colouring: uniform: every node has the initial colour c_0. **Not assumed:** port numbers: no port numbers: the agent names a colour, the adversary picks the neighbour; agent memory: oblivious (no memory, one state); knowledge: nothing known about the graph; termination requirement: explore, then stop at the start node; network: arbitrary connected simple graph.

Statement for this setting. The BFUW model: no port numbers, the agent sees the colour of its node and the multiset of its neighbours' colours, recolours its node freely and names a colour to move to; the adversary chooses among neighbours of that colour. All nodes start with c_0. The agent must visit every node, return to s and stop there, for every start and every adversary. Colours are counted including c_0.

Measure	Lower bound	Best upper bound	Status
Colours	4 (Hiraoka et al. 2026)	6 (Takahashi et al. 2026)	● tight
	Constant: between 4 and 6 colours (counting c_0); the exact value is open.		
Time	$\Omega(n)$ (Bojko et al. 2022)	—	● unknown
	No time bound is proved for any general-graph algorithm; only the trivial $\Omega(n)$.		
Memory	none known	0 (Takahashi et al. 2026)	● tight
	Zero bits: the agent is oblivious.		

Results.

- Hiraoka et al. (2026) [lower bound]: colours 4. *Bottleneck:* indistinguishable starts. *Holds when:* no single rule with at most three colours explores every subcubic pseudotree (graphs with maximum degree three and at most one cycle; Theorem 13); realised on a family of nine graphs with at most five nodes, among them C_3, C_4, C_5 and $K_{1,3}$; computer-assisted: the cases left after a hand reduction are refuted by a SAT solver with a

Claude (Anthropic) was used in cataloguing the problems.

DRAT-checked certificate; uses that two executions from different starts which reach the same configuration cannot both stop at their own start (Lemma 2).

- Bojko et al. (2022) [lower bound]: time $\Omega(n)$. *Bottleneck:* every node visited. *Holds when:* trivial: any agent must visit all n nodes (stated for trees in Bojko et al. 2022).
- Böckenhauer et al. (2023) [introductory]: colours 8, memory 0. Theorem 11 (stated as 7 colours: the paper does not count c_0). Breadth-first: a node at depth j gets label (j mod 3) + 1, in green or red according to the phase, plus a colour for deleted (finished) nodes. Never recolours a node back to c_0. Time not analysed.
- Takahashi et al. (2026) [current best]: colours 6, memory 0. *Assumes:* recolouring back to c_0 allowed; seven colours without it (Note 1). Theorem 1: *semi-DFS*, a depth-first search whose return path is kept an induced path, so that the predecessor of the head is its only neighbour on the path; colours init, path, fin, head1, head2, neigh. Time not analysed.

Open questions.

Q1. Is the colour complexity of recolourable exploration of arbitrary graphs 4, 5 or 6 (Takahashi et al. 2026) (Hiraoka et al. 2026)?

Q2. Give a proof of the four-colour lower bound without computer assistance, or for a single graph (Hiraoka et al. 2026).

Q3. How many moves does a constant-colour algorithm need on general graphs? No bound is known; for graphs whose blocks are cliques or triangle-free, five colours and $O(n\Delta)$ moves suffice (Hiraoka et al. 2026).

Q4. How many colours suffice if the agent has a constant number of bits, or even one bit, of memory, or may move at random (Böckenhauer et al. 2023) (Takahashi et al. 2026)?

Q5. Does forbidding recolouring back to c_0 change the colour complexity, or only the algorithms (Takahashi et al. 2026)?

Notes. Colour counts include the initial colour c_0, as in (Takahashi et al. 2026) and (Hiraoka et al. 2026); Böckenhauer, Frei, Unger and Wehner count without it, so their 7 is 8 here. Restricted classes: five colours suffice for triangle-free graphs (Theorem 2 of the arXiv v2 of Takahashi et al. 2026) and for *φ-free* graphs, those in which, for every node v of degree at least four, each component of $G - v$ contains at most two neighbours of v; they include all graphs of maximum degree three and all cacti (Theorem 3 of Takahashi et al. 2026; the name is the paper's). Hiraoka, Imori, Takahashi and Sudo extend the five-colour algorithm to graphs whose blocks are cliques or triangle-free, with $O(n\Delta)$ moves (Theorem 11), and show that this class is exactly where its cleanup conditions hold (Hiraoka et al. 2026). All known results are for a single agent. Checked against the full texts; (Hiraoka et al. 2026) is a preprint and its lower bound is computer-assisted.

16.8.2 Recolourable nodes on cacti

Adds + cactus **to** Recolourable nodes, oblivious agent, any graph (p. 161) VERIFIED

Four colours explore every cactus in $O(n)$ moves, and four are necessary.

Assumed: graph dynamics: static graph; edge direction: undirected; view: the agent also sees the colours of its neighbours; recolouring: recolourable: any colour, any number of times; initial colouring: uniform: every node has the initial colour c_0; network: cactus (every edge on at most one cycle). **Not assumed:** port numbers: no port numbers: the agent names a colour, the adversary picks the neighbour; agent memory: oblivious (no memory, one state); knowledge: nothing known about the graph; termination requirement: explore, then stop at the start node.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Colours	4 (Hiraoka et al. 2026) Exactly 4 (counting c_0); the lower bound is computer-assisted.	4 (Hiraoka et al. 2026)	• tight
Time	$\Omega(n)$ (Bojko et al. 2022) $\Theta(n)$.	$O(n)$ (Hiraoka et al. 2026)	• tight
Memory	none known Zero bits.	0 (Hiraoka et al. 2026)	• tight

Results.

- Hiraoka et al. (2026) [lower bound]: colours 4. *Bottleneck:* indistinguishable starts. *Holds when:* subcubic pseudotrees, which are cacti (Theorem 13); computer-assisted (SAT certificate checked with DRAT).
- Bojko et al. (2022) [lower bound]: time $\Omega(n)$. *Bottleneck:* every node visited. *Holds when:* trivial: any agent must visit all n nodes (stated for trees in Bojko et al. 2022).
- Takahashi et al. (2026) [also]: colours 5, memory 0. Theorem 3: cacti are φ-free. Uses the three colours d_0, d_1, d_2 to keep a directed acyclic graph that guides forward moves and backtracking. Time not analysed.
- Hiraoka et al. (2026) [current best]: colours 4, time $O(n)$, memory 0. *Assumes:* holds for every graph whose blocks are cycles or complete bipartite graphs. Theorem 6. Introductory for four colours and current best.

Open questions.

Q1. Is four colours enough for φ-free graphs, or for graphs of maximum degree three? Both lie between 4 and 5 (Hiraoka et al. 2026).

Notes. The four-colour algorithm treats each block on its own, either as a cycle or as a complete bipartite graph $K_{p,q}$, and handles child blocks recursively (Hiraoka et al. 2026). Checked against the full text of the preprint.

16.8.3 Recolourable nodes on trees

Adds + tree **to** Recolourable nodes, oblivious agent, any graph (p. 161) VERIFIED

Three colours explore every tree in $O(n)$ moves; two colours cannot explore the path on three nodes.

Assumed: graph dynamics: static graph; edge direction: undirected; view: the agent also sees the colours of its neighbours; recolouring: recolourable: any colour, any number of times; initial colouring: uniform: every node has the initial colour c_0; network: tree. **Not assumed:** port numbers: no port numbers: the agent names a colour, the adversary picks the neighbour; agent memory: oblivious (no memory, one state); knowledge: nothing known about the graph; termination requirement: explore, then stop at the start node.

Measure	Lower bound	Best upper bound	Status
Colours	3 (Hiraoka et al. 2026) Exactly 3 (counting c_0).	3 (Hiraoka et al. 2026)	• tight
Time	$\Omega(n)$ (Bojko et al. 2022) $\Theta(n)$.	$O(n)$ (Hiraoka et al. 2026)	• tight
Memory	none known Zero bits.	0 (Hiraoka et al. 2026)	• tight

Results.

- Hiraoka et al. (2026) [lower bound]: colours 3. *Bottleneck:* indistinguishable starts. *Holds when:* the path P_3 on three nodes, every start node (Lemma 4); case analysis of all two-colour rules: each loops, stops early or lets two starts reach the same configuration.

Claude (Anthropic) was used in cataloguing the problems.

- Bojko et al. (2022) [lower bound]: time $\Omega(n)$. *Bottleneck:* every node visited. *Holds when:* trivial: any agent must visit all n nodes (stated for trees in Bojko et al. 2022).
- Hiraoka et al. (2026) [current best]: colours 3, time $O(n)$, memory 0. *Assumes:* the same algorithm explores every simple cycle. Theorem 3. Introductory and current best.

Notes. Without recolouring, trees need four colours (setting *Once-colourable nodes on trees*). Böckenhauer et al. remark that, with recolouring, a path explored from an end node needs only one colour besides c_0: colour forward, then erase on the way back (Böckenhauer et al. 2023); Lemma 4 of (Hiraoka et al. 2026) shows that two colours fail once the start may be the middle node. On simple cycles the same three-colour algorithm works, but no lower bound for cycles alone is stated. Checked against the full text of the preprint.

16.8.4 Once-colourable nodes, any graph

Adds + colour once, + uniform c_0 VERIFIED

If a coloured node can never be recoloured, n colours are necessary and sufficient.

Assumed: graph dynamics: static graph; edge direction: undirected; view: the agent also sees the colours of its neighbours; recolouring: each node may be coloured once; initial colouring: uniform: every node has the initial colour c_0. **Not assumed:** port numbers: no port numbers: the agent names a colour, the adversary picks the neighbour; agent memory: oblivious (no memory, one state); knowledge: nothing known about the graph; termination requirement: explore, then stop at the start node; network: arbitrary connected simple graph.

Measure	Lower bound	Best upper bound	Status
Colours	n (Böckenhauer et al. 2023) Exactly n (counting c_0).	n (Böckenhauer et al. 2023)	• tight
Time	none known Time not studied.	—	• unknown
Memory	none known Zero bits.	0 (Böckenhauer et al. 2023)	• tight

Results.

- Böckenhauer et al. (2023) [lower bound]: colours n. *Bottleneck:* predecessor identification. *Holds when:* uniform algorithm: it must explore all graphs, n unknown (Theorem 4; stated as $n-1$ without c_0); for every n there is an n-node graph on which the algorithm uses n colours; the proof forces the agent along a path of a larger graph that it cannot tell from the small one; also holds for bipartite graphs up to one colour: $n-1$ (as summarised in Takahashi et al. 2026).
- Böckenhauer et al. (2023) [current best]: colours n, memory 0. Introductory and current best (Theorem 3, DepthFirstSearch): every newly visited node gets a colour larger than all its coloured neighbours, so the predecessor is always identifiable. Time not analysed.

Open questions.

Q1. For graphs of circumference $\mathrm{circ}(G) \leq g$, close the gap between $\min(2g-2, n)$ and $\min(2g, n)$ colours (Böckenhauer et al. 2023).

Notes. The driving parameter is the circumference $\mathrm{circ}(G)$, the length of a longest cycle (their k): for graphs of circumference at most g, $\min(2g-2, n)$ colours are necessary and $\min(2g, n)$ suffice, counting c_0 (Theorems 8 and 9 of Böckenhauer et al. 2023). A special class can be much cheaper: squares of paths need only five colours although their circumference is maximal (Theorem 10, four without c_0). Checked against the full text.

Claude (Anthropic) was used in cataloguing the problems.

16.8.5 Once-colourable nodes, n known

Adds + n known **to** Once-colourable nodes, any graph (p. 164) VERIFIED

Knowing n saves one or two colours: $n-1$ suffice and $n-2$ are necessary.

Assumed: graph dynamics: static graph; edge direction: undirected; view: the agent also sees the colours of its neighbours; recolouring: each node may be coloured once; initial colouring: uniform: every node has the initial colour c_0; knowledge: the exact number of nodes n is known. **Not assumed:** port numbers: no port numbers: the agent names a colour, the adversary picks the neighbour; agent memory: oblivious (no memory, one state); termination requirement: explore, then stop at the start node; network: arbitrary connected simple graph.

Measure	Lower bound	Best upper bound	Status
Colours	$n-2$ (Böckenhauer et al. 2023) $n-2$ versus $n-1$: an additive gap of one colour.	$n-1$ (Böckenhauer et al. 2023)	• tight
Time	none known Time not studied.	—	• unknown
Memory	none known Zero bits.	0 (Böckenhauer et al. 2023)	• tight

Results.

- Böckenhauer et al. (2023) [lower bound]: colours $n-2$. *Bottleneck:* predecessor identification. *Holds when:* Theorem 6 (stated as $n-3$ without c_0); realised by cliques of $n-3$ nodes with two pendant leaves and a special node s.
- Böckenhauer et al. (2023) [current best]: colours $n-1$, memory 0. *Assumes:* $n \geq 5$. Introductory and current best (Theorem 5, DFS_n; stated as $n-2$ without c_0): the last two nodes share a colour.

Open questions.

Q1. Is the colour complexity with known n equal to $n-2$ or $n-1$ (Böckenhauer et al. 2023)?

Notes. Checked against the full text.

16.8.6 Once-colourable nodes on trees

Adds nothing known, + tree **to** Once-colourable nodes, any graph (p. 164) VERIFIED

Colouring by depth modulo three explores every tree with four colours, and four are necessary.

Assumed: graph dynamics: static graph; edge direction: undirected; view: the agent also sees the colours of its neighbours; recolouring: each node may be coloured once; initial colouring: uniform: every node has the initial colour c_0; network: tree. **Not assumed:** port numbers: no port numbers: the agent names a colour, the adversary picks the neighbour; agent memory: oblivious (no memory, one state); knowledge: nothing known about the graph; termination requirement: explore, then stop at the start node.

Measure	Lower bound	Best upper bound	Status
Colours	4 (Böckenhauer et al. 2023) Exactly 4 (counting c_0).	4 (Böckenhauer et al. 2023)	• tight
Time	none known Not analysed in the paper.	—	• unknown
Memory	none known Zero bits.	0 (Böckenhauer et al. 2023)	• tight

Results.

Claude (Anthropic) was used in cataloguing the problems.

- Böckenhauer et al. (2023) [lower bound]: colours 4. *Bottleneck:* predecessor identification. *Holds when:* the path on seven nodes started at an end (Theorem 2; stated as 3 without c_0); an uncoloured node cannot be told from the node two steps back, and on the way back two nodes at distance two must differ.
- Böckenhauer et al. (2023) [current best]: colours 4, memory 0. Introductory and current best (Theorem 1, TreeExploration): a node at depth j gets colour (j mod 3) + 1; the agent prefers uncoloured neighbours, otherwise returns to the parent (the unique neighbour of the previous colour), and stops at the only node of colour 1 with no neighbour of colour 3, the start. Time not analysed.

Notes. The same three-by-depth idea guides label-guided exploration (Cohen et al. 2008) and the breadth-first algorithm with recolouring (Böckenhauer et al. 2023). Checked against the full text.

16.8.7 Label-guided: colours fixed by a designer

Adds + finite automaton, + designer's colours, + stop VERIFIED

If a designer colours the nodes with three colours, a finite automaton explores every graph and stops within $20m$ moves.

Assumed: graph dynamics: static graph; edge direction: undirected; port numbers: port numbers and the entry port; agent memory: finite automaton (a constant number K of states); initial colouring: colours chosen by a designer who knows the graph; termination requirement: explore, then stop anywhere. **Not assumed:** view: the agent sees its own node only; recolouring: colours fixed: the agent reads but cannot change them; knowledge: nothing known about the graph; network: arbitrary connected simple graph.

Measure	Lower bound	Best upper bound	Status
Colours	2 (Cohen et al. 2008)	3 (Cohen et al. 2008)	• tight
	Constant: 2 versus 3; whether two colours suffice for arbitrary graphs is open (Cohen et al. 2008).		
Time	$\Omega(n)$ (Bojko et al. 2022)	$O(m)$ (Cohen et al. 2008)	• unknown
	Only the trivial $\Omega(n)$ against $O(m)$.		
Memory	none known	$O(1)$ (Cohen et al. 2008)	• tight
	Constant.		

Results.

- Cohen et al. (2008) [lower bound]: colours 2. *Bottleneck:* trap graphs. *Holds when:* with one colour the labels carry nothing, and no finite automaton explores all graphs (Fraigniaud et al. 2005a) (Section 5 of Cohen et al. 2008); holds even for bounded-degree graphs.
- Bojko et al. (2022) [lower bound]: time $\Omega(n)$. *Bottleneck:* every node visited. *Holds when:* trivial: any agent must visit all n nodes (stated for trees in Bojko et al. 2022).
- Cohen et al. (2008) [current best]: colours 3, time $O(m)$, memory $O(1)$. *Assumes:* fewer than 25 states; labels computed in linear time: colour = depth mod 3 from a root. Introductory and current best (Theorem 2.1): at most $20m$ moves. With 1-bit (two-colour) labels, $O(\log \Delta)$ bits of memory and $O(\Delta^{O(1)}m)$ moves (Theorem 3.1).

Open questions.

Q1. Is there a 1-bit (two-colour) labeling with which a finite automaton explores every graph (Cohen et al. 2008)?

Notes. This is the setting *Node labels chosen by the designer* of single-agent exploration, seen from the colour side: the flag is planted by a designer and the agent cannot change it. The designer's colouring by depth mod 3 is the same device that Böckenhauer et al. let the agent build itself. Zhang, Zhang and Tang trade the share of black nodes in 1-bit labelings against memory (Zhang et al. 2012). Checked against the full text.

Claude (Anthropic) was used in cataloguing the problems.

16.8.8 Label-guided, oblivious agent

Adds oblivious, + designer's colours, + perpetual VERIFIED

Fixed colours cannot replace memory cheaply: an oblivious agent needs more than $\lfloor \log \Delta \rfloor - 2$ *designer colours.*

Assumed: graph dynamics: static graph; edge direction: undirected; port numbers: port numbers and the entry port; initial colouring: colours chosen by a designer who knows the graph; termination requirement: visit every node; never required to stop. **Not assumed:** view: the agent sees its own node only; agent memory: oblivious (no memory, one state); recolouring: colours fixed: the agent reads but cannot change them; knowledge: nothing known about the graph; network: arbitrary connected simple graph.

Measure	Lower bound	Best upper bound	Status
Colours	$\lfloor \log \Delta \rfloor - 1$ (Cohen et al. 2008) No algorithm is catalogued for an oblivious agent with designer colours.	—	● unknown
Time	none known No algorithm catalogued.	—	● unknown
Memory	none known Zero bits.	—	● tight

Results.

- Cohen et al. (2008) [lower bound]: colours $\lfloor \log \Delta \rfloor - 1$. *Bottleneck:* port addressing. *Holds when:* for every $\Delta > 4$ and every one-state agent with at most $\lfloor \log \Delta \rfloor - 2$ colours there is a simple graph of maximum degree Δ it does not explore (Theorem 4.5); with self-loops allowed the bound rises to $\Delta/2$ colours (Theorem 4.2); star-like graphs whose central node is entered and left by ports the agent's colour functions cannot cover; applies to visiting all nodes, so to every termination requirement.

Open questions.

Q1. How many designer colours does an oblivious agent need to explore all graphs of maximum degree Δ (Cohen et al. 2008)?

Notes. With recolourable colours the same oblivious agent needs only a constant number of colours, even without port numbers (setting *Recolourable nodes, oblivious agent, any graph*): the power lies in writing, not in reading. Checked against the full text.

16.8.9 Rotor-router: a pointer in every node

Adds + ports, no entry port, + recolourable, + perpetual DRAFT

A node's colour is a pointer to one of its ports; an oblivious agent that follows and advances it explores any graph in $O(mD)$ *moves from arbitrary pointers, and no oblivious agent without the entry port is faster in the worst case.*

Assumed: graph dynamics: static graph; edge direction: undirected; port numbers: port numbers, entry port unknown; recolouring: recolourable: any colour, any number of times; termination requirement: visit every node; never required to stop. **Not assumed:** view: the agent sees its own node only; agent memory: oblivious (no memory, one state); initial colouring: arbitrary initial colours (chosen by the adversary); knowledge: nothing known about the graph; network: arbitrary connected simple graph.

Statement for this setting. Ports are numbered, the entry port is not revealed, the agent sees only its own node. The colour of v is a pointer $\pi_v \in \{1, \dots, \deg(v)\}$, initially arbitrary. The agent leaves v by port π_v and advances π_v cyclically. It must visit every node (and keeps going). So a node of degree d uses d colours, $\lceil \log_2 d \rceil$ bits.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Colours	$\deg(v)$ at each node v (Menc et al. 2017) $\deg(v)$ colours per node, the minimum.	$\deg(v)$ (Yanovski et al. 2003)	• tight
Time	$\Omega(n^3)$ (Menc et al. 2017) Worst case over n-node graphs: $O(mD) = O(n^3)$ against $\Omega(n^3)$. For a given graph the rotor-router may be slower than a cleverer rule.	$O(mD)$ (Yanovski et al. 2003)	• tight
Memory	none known Zero bits.	0 (Yanovski et al. 2003)	• tight

Results.

- Menc et al. (2017) [lower bound]: colours $\deg(v)$ at each node v. *Bottleneck:* port addressing. *Holds when:* an oblivious agent without the entry port needs $\deg(v)$ distinct inputs at v to leave by every port (Observation 1.1).
- Menc et al. (2017) [lower bound]: time $\Omega(n^3)$. *Bottleneck:* unknown inport. *Holds when:* any oblivious agent without the entry port, even with unbounded node memory (Theorem 2.3); uniform initial node memory, so it holds a fortiori from arbitrary colours; on a path, $(n-1)^2$ moves are needed for a worst-case port numbering (Theorem 2.2), which the rotor-router attains.
- Yanovski et al. (2003) [current best]: colours $\deg(v)$, time $O(mD)$, memory 0. Introductory and current best, as reported by (Menc et al. 2017): from arbitrary pointers the walk settles into an Euler tour within $O(mD)$ moves (the abstract gives $2mD$ steps), which is $\Theta(mD)$ in the worst case (Bampas et al. 2009). Not checked against the full text.

Open questions.

Q1. For which graphs (other than paths) is the rotor-router exactly time-optimal among oblivious agents without the entry port (Menc et al. 2017)?

Notes. The rotor-router (also *Propp machine*, *Edge Ant Walk*) goes back to the Eulerian walkers of Priezzhev, D. Dhar, A. Dhar and Krishnamurthy (as cited by Takahashi et al. 2026) (Priezzhev et al. 1996); Yanovski, Wagner and Bruckstein used it for patrolling (Yanovski et al. 2003). Bampas et al. show that if the pointers may be chosen and the adversary then numbers the ports, the lock-in time drops to $O(m \min(\log m, D))$ (abstract) (Bampas et al. 2009). It is a recolourable node whose palette grows with the degree: a deterministic recolouring rule fixed in advance. **Unverified:** the $O(mD)$ bound, taken from (Menc et al. 2017), the abstract of (Yanovski et al. 2003) and the slides of (Bampas et al. 2009).

16.9 Collaborative exploration

> *k agents start together at the root of an unknown graph and must visit all its nodes; how much faster than one agent can a team be?*

First posed in 2004 (Fraigniaud et al. 2006a). Collective tree exploration was posed at LATIN 2004 (journal version 2006).

Problem statement and measures

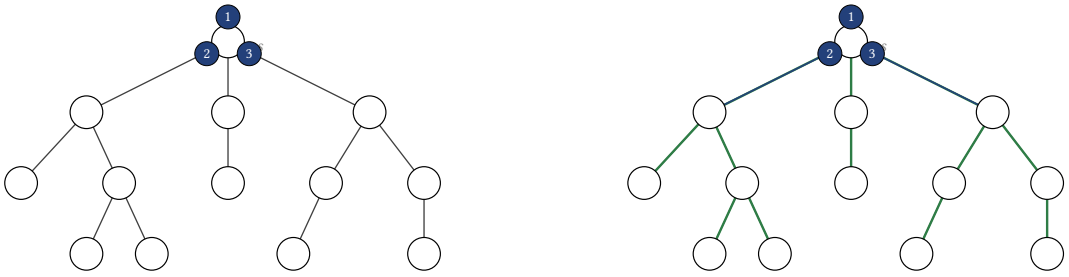
A team of k distinguishable agents starts at a node s (the root) of an unknown connected graph with n nodes; D is the largest distance from s. In synchronous rounds every agent traverses at most one edge. When an agent reaches a node it sees the incident edges. The team must **visit every node** (some papers also require all agents to return to s; the bounds coincide asymptotically).

Claude (Anthropic) was used in cataloguing the problems.

Time is the number of rounds (in the asynchronous model: the total number of moves). The offline optimum is $\Theta(n/k + D)$, so an algorithm that explores every tree in $\mathrm{CR}(k) \cdot (n/k + D)$ rounds is called $\mathrm{CR}(k)$-competitive; a single depth-first search is $O(k)$-competitive. **Memory** is bits per agent; most of this literature lets it be unbounded.

Unless a variant says otherwise, nodes are **labelled** (an agent recognises a node it has seen), which is the standard assumption of collective tree exploration; on trees it does not matter for lower bounds (Disser et al. 2020). With anonymous nodes and $k = n$, the problem is solved by rooted dispersion.

Measures: **Time**, rounds until every node is visited (asynchronous: total moves); **Memory**, bits per agent.



(a) Start: k = 3 agents at the root s of an unknown tree

(b) Final: every node visited, all agents back at s after 10 rounds

Figure 16.9: Collective tree exploration (Fraigniaud et al. 2006a), here by Breadth-First Depth-Next (Cosson et al. 2023): $k = 3$ agents start at the root s of an unknown tree and share one map. An agent at the root is sent to the shallowest node with an unexplored edge (its anchor); from there it walks down unexplored edges that no other agent takes in that round, and steps back towards the root when there is none. One frame per round; the time is compared with the offline optimum $\Theta(n/k + D)$.

A first look

Imagine k rescuers entering a cave system through one entrance; they want every chamber checked as soon as possible. We model the cave as a graph: chambers are **nodes**, tunnels between two chambers are **edges**, and each rescuer is an **agent**. The cave is a **tree** (a graph without cycles, so there is exactly one route between any two nodes) with n nodes; the entrance is its **root** s, and the **depth** D is the largest distance from s. All k agents start at s; time passes in **rounds**, and in each round every agent may walk along one edge. Nodes are **labelled** (an agent recognises a node it has seen before), and an agent at a node sees the edges leaving it. In the basic version all agents share everything they learn (**global communication**), so they act on one **map**: the part of the tree seen so far. The team must visit every node; the time is the number of rounds, and an algorithm is judged by how close it gets to the best possible time of about $n/k + D$ rounds.

1. Algorithm: Breadth-First Depth-Next. (Cosson et al. 2023) (Trees, complete communication, p. 174) The algorithm BFDN of (Cosson et al. 2023). Call an edge **unexplored** if it has been seen but no agent has walked it yet (the paper calls it *dangling*).

1. When an agent is at the root, give it an **anchor**: a node of the map with an unexplored edge, preferring nodes closest to the root and, among those, the ones with the fewest agents already anchored there.
2. The agent walks along the map to its anchor.

Claude (Anthropic) was used in cataloguing the problems.

3. From there it does a **depth-first search**: if an unexplored edge leaves its node and no other agent has chosen it this round, it takes it; otherwise it steps one edge back towards the root.
4. Back at the root it gets a new anchor. The team stops when all agents are at the root and no unexplored edge is left.

Why it works and what it costs. Each agent mostly does what a single agent exploring alone does (walk down unexplored edges, back up when stuck), and the anchors spread the team over the shallowest unfinished parts of the tree. (Cosson et al. 2023) prove a time of $\frac{2n}{k} + D^2(\min\{\log \Delta, \log k\} + 2)$ rounds, where Δ is the maximum degree; the $2n/k$ term is what k agents need just to walk every edge twice. The best known algorithm today takes $\frac{2n}{k} + O(D \log^2 k)$ rounds (Cosson and Massoulié 2026).

Why it matters. Its analysis through a simple game against an adversary opened the series of papers that brought the overhead from roughly a factor $k/\log k$ down to polylogarithmic in k.

2. Lower bound: Even knowing the tree: n/k + D. (Cosson et al. 2023) (Trees, complete communication, p. 174) Suppose the agents knew the whole tree in advance. Two things still take time.

1. Some agent must reach the deepest node, which is D edges from the root: at least D rounds.
2. Each of the $n - 1$ nodes other than the root must be entered for the first time by some agent, and in one round k agents enter at most k nodes: at least $(n - 1)/k$ rounds.

Why it holds. So no algorithm beats about $\max\{n/k, D\}$, which is at least half of $n/k + D$. Knowing the tree, this is also achievable up to a factor 2: cut a depth-first tour of the tree (length $2(n - 1)$) into k pieces of length $2(n - 1)/k$ and send one agent to each piece, for $2(n/k + D)$ rounds in total, as recalled in (Cosson et al. 2023). An online algorithm (one that discovers the tree as it goes) that is always within a factor $\mathrm{CR}(k)$ of $n/k + D$ is called $\mathrm{CR}(k)$**-competitive**; a single depth-first search by one agent is $O(k)$-competitive.

Why it matters. This simple yardstick is what every result on collaborative exploration is measured against.

3. Lower bound: An adversary that grows the tree. (Disser et al. 2020) (Trees, complete communication, p. 174) When the tree is not known in advance, it is revealed only where agents stand, so one can imagine an **adversary** (an opponent who builds the worst input) drawing the tree during the run. When the agents reach a node, the adversary decides how many children it has. Whatever way the agents split up, the adversary can make the long branches continue where few agents went and end the branches that received many agents. The full proof that this costs a factor $\Omega(\log k/\log\log k)$ is longer (Disser et al. 2020).

Why it holds. Every deterministic algorithm takes $\Omega\big(\frac{\log k}{\log\log k}(\frac{n}{k} + D)\big)$ rounds on some tree, even with global communication and unlimited memory (Disser et al. 2020); it was first shown by (Dynia et al. 2007) for $k < \sqrt{n}$. The best algorithm is $O(\log^2 k)$-competitive (Cosson and Massoulié 2026), so the remaining gap is between $\log k/\log\log k$ and $\log^2 k$.

Why it matters. It refuted the hope of a constant competitive ratio and shows that not knowing where the work is costs something no amount of communication can repair.

What the community started looking at

Collaborative exploration asked a different question from the single-agent line: not how little memory, but **how much faster** k agents can be than one. Fraigniaud, Gąsieniec, Kowalski and Pelc measured an algorithm by its competitive ratio against the offline optimum $\Theta(n/k + D)$ and showed that communication matters: without it, k agents gain nothing (Fraigniaud et al. 2006a). The following twenty years

Claude (Anthropic) was used in cataloguing the problems.

were spent narrowing the ratio from $O(k/\log k)$ towards the $\Omega(\log k/\log\log k)$ lower bound (Dynia et al. 2007) (Disser et al. 2020) (Cosson and Massoulié 2024), and on what very large teams can do (Dereniowski et al. 2015).

Relations to other problems

- **Derived problem: Dispersion** (derived from). With $k = n$ agents, a dispersed configuration has an agent on every node, so any dispersion algorithm solves n-agent collaborative exploration under the same assumptions (Augustine and Moses Jr. 2018). Consequently every lower bound for n-agent exploration is a lower bound for dispersion (this is how the $\Omega(D^2)$ tree bound of Disser et al. 2020 enters), and every dispersion algorithm is an exploration algorithm for anonymous graphs with local communication. For $k < n$ dispersion asks for less: only k nodes must be occupied.
- **Derived problem: Black hole search** (variant of). Locating the black hole means exploring every other node safely, so black hole search is exploration of a dangerous graph by a team, where agents can be lost.
- **Derived problem: Exploration of dynamic graphs by teams** (variant of). When an adversary removes edges, a single agent can be blocked forever; the question shifts from speed to **how many agents** are needed to visit every node.

Assumptions specific to this problem

Team size

Fraigniaud, Gąsieniec, Kowalski and Pelc posed collective tree exploration for any number k of agents and measured algorithms by their competitive ratio against the offline optimum (Fraigniaud et al. 2006a). Two team sizes then turned out to have sharper answers than arbitrary k. For large teams, Dereniowski, Disser, Kosowski, Pająk and Uznański explore in $O(D)$ rounds with $k = Dn^{1+\varepsilon}$ agents, a constant competitive ratio, and Disser, Mousset, Noever, Steger and Škorić showed that $k = Dn^{1+o(1)}$ agents do not suffice for that, so the threshold is essentially tight (Dereniowski et al. 2015) (Disser et al. 2020). For $k = n$, Disser et al. gave a simple algorithm taking D^2 rounds on every tree and showed that trees of arbitrarily large height need $\Omega(D^2)$ rounds (Disser et al. 2020). This team size matters beyond exploration because, as Augustine and Moses Jr. observed when they introduced dispersion, "any solution to a problem of dispersion acts as a solution to n robot exploration under the same conditions" (Augustine and Moses Jr. 2018).

Level	Value	Formal definition
0	**any** k	Any number $k \geq 2$ of agents.
1	$k = n$	Exactly $k = n$ agents.
1	$k \geq Dn^{1+\varepsilon}$	$k \geq D\,n^{1+\varepsilon}$ agents for a constant $\varepsilon > 0$ (written $k = Dn^{\gamma}$, $\gamma > 1$).

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Memory) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Node identifiers → Randomness → Communication → Node memory → Timing → Team size → Topology.



What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Claude (Anthropic) was used in cataloguing the problems.

Setting	Time	Memory	p.
Arbitrary graphs, complete communication	● unknown $\frac{2n}{k} + D^2(\min\{\log\Delta, \log k\} + 2)$ vs — vs —	● unknown — vs —	173
Trees, complete communication STANDARD SETTING	● near-optimal $\frac{2n}{k} + O(D\log^2 k)$ vs $\Omega\big(\frac{\log k}{\log\log k}(\frac{n}{k} + D)\big)$	● unknown — vs —	174
Trees with $k = n$ agents	● polynomial gap D^2 vs $D^2/3$	● unknown — vs —	175
Large teams, complete communication	● tight $D(1 + \frac{1}{\gamma-1} + o(1))$ vs $D(1 + \frac{1}{\gamma} - o(1))$	● unknown — vs —	175
Trees, asynchronous, complete communication	● tight $2n + O(kD\log^2 k)$ vs $\Omega(\log^2 k)\cdot(n + kD)$	● unknown — vs —	176
Trees, communication by whiteboards only	● polynomial gap $O(D + n/\log k)$ vs —	● unknown — vs —	177
Trees, asynchronous, whiteboards only	● polynomial gap $2n + O(k^2 2^k D)$ vs —	● unknown — vs —	177
Trees, no communication	● tight $O(k)\cdot$ optimal vs $\Omega(k)\cdot$ optimal	● unknown — vs —	178
Large teams, local communication	● tight $D(1 + \frac{2}{\gamma-1} + o(1))$ vs $D(1 + \frac{2}{c} - o(1))$	● unknown — vs —	178
Anonymous graphs, a team of constant-memory agents	● unknown $n^{O(1)}$ vs —	● tight $O(1)$ bits each, $O(\log\log n)$ agents vs $\Omega(\log\log n)$ agents	179
Anonymous graphs with $k = n$ agents	● tight $O(n)$ vs $\Omega(n)$	● unknown $O(\log n)$ vs —	179

16.9.1 Arbitrary graphs, complete communication

Adds + synchronous VERIFIED

On graphs with cycles it is not even known whether any algorithm beats depth-first search by more than a constant.

Assumed: graph dynamics: static graph; edge direction: undirected; node identifiers: labelled nodes (an agent can recognise a node it has visited); communication: global (any agent can read any other agent's state); timing: synchronous rounds. **Not assumed:** randomness: deterministic; node memory: no whiteboards; team size: any number k of agents; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known	$\frac{2n}{k} + D^2(\min\{\log\Delta, \log k\} + 2)$ (Cosson et al. 2023)	● unknown
	No $o(k)$-competitive algorithm is known without extra knowledge; lower bounds are inherited from trees.		
Memory	none known Memory is not bounded in these papers.	—	● unknown

Results.

Claude (Anthropic) was used in cataloguing the problems.

- Cosson et al. (2023) [current best]: time $\frac{2n}{k} + D^2(\min\{\log\Delta, \log k\} + 2)$. *Assumes:* agents know their distance to the root. Proposition 7; n counts edges here.

Open questions.

Q1. Is there an $o(k)$-competitive collaborative exploration algorithm for arbitrary graphs? (Stated as open in Cosson and Massoulié 2026.)

16.9.2 Trees, complete communication

Adds any k, + tree **to** Arbitrary graphs, complete communication (p. 173) STANDARD SETTING VERIFIED

Collective tree exploration as posed by Fraigniaud, Gąsieniec, Kowalski and Pelc: the competitive ratio is now between $\Omega(\log k/\log\log k)$ and $O(\log^2 k)$.

Assumed: graph dynamics: static graph; edge direction: undirected; node identifiers: labelled nodes (an agent can recognise a node it has visited); communication: global (any agent can read any other agent's state); timing: synchronous rounds; topology: tree. **Not assumed:** randomness: deterministic; node memory: no whiteboards; team size: any number k of agents.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega\big(\frac{\log k}{\log\log k}\,(\frac{n}{k} + D)\big)$ (Disser et al. 2020)	$\frac{2n}{k} + O(D\log^2 k)$ (Cosson and Massoulié 2026)	• near-optimal
	$O(\log^2 k)$ versus $\Omega(\log k/\log\log k)$ competitive: about a $\log k \log\log k$ factor (upper bound from a preprint; the peer-reviewed gap is $O(\sqrt{k})$ versus the same lower bound).		
Memory	none known Memory is not bounded in these papers.	—	• unknown

Results.

- Disser et al. (2020) [lower bound]: time $\Omega\big(\frac{\log k}{\log\log k}\,(\frac{n}{k} + D)\big)$. *Bottleneck:* online tree revelation. *Holds when:* deterministic; holds with global communication and unbounded memory; for $k \le n\log^\kappa n$; first shown by (Dynia et al. 2007) for $k < \sqrt{n}$; the hard instances are trees. Before that, a ratio of at least $2-1/k$ for deterministic and randomized algorithms when $k < \sqrt{n}$ (Fraigniaud et al. 2006a), as cited by (Dereniowski et al. 2015).
- Fraigniaud et al. (2006a) [introductory]: time $O(D + n/\log k)$. Works already with write-read communication at nodes; $O(k/\log k)$-competitive. The analysis is tight on a tree of (Higashikawa et al. 2014).
- Cosson et al. (2023) [also]: time $\frac{2n}{k} + D^2(\min\{\log\Delta, \log k\} + 2)$. Breadth-first depth-next; optimal when $D = o(\sqrt{n})$.
- Cosson (2024) [also]: time $\frac{2n}{k} + O(k^{\log_2 k - 1}D)$. First ratio below $k/\log k$: $O(k/\exp(\sqrt{\ln 2 \ln k}))$.
- Cosson and Massoulié (2024) [also]: time $\frac{2n}{k} + O(kD)$. First polynomial improvement: $O(\sqrt{k})$-competitive by using $\sqrt{k}$ of the agents. Current best among peer-reviewed results.
- Cosson and Massoulié (2026) [current best]: time $\frac{2n}{k} + O(D\log^2 k)$. $O(\log^2 k)$-competitive. A September 2026 preprint, not yet peer-reviewed.

Open questions.

Q1. Determine the competitive ratio of synchronous collective tree exploration: between $\Omega(\log k/\log\log k)$ and $O(\log^2 k)$.

Q2. Do randomized algorithms help? A randomized lower bound of $\Omega(\sqrt{\log k}/\log\log k)$ is attributed to Ortolf and Schindelhauer (SPAA 2012), not checked here. Cosson and Massoulié conjecture they do not (Cosson and Massoulié 2025).

Claude (Anthropic) was used in cataloguing the problems.

Notes. Fraigniaud et al. conjectured that a constant ratio is possible; (Dynia et al. 2007) refuted it. Greedy algorithms cannot beat $\Omega(k/\log k)$ (Higashikawa et al. 2014), which is why progress needed new ideas: tree-mining games (Cosson 2024) and potential functions (Cosson and Massoulié 2024).

16.9.3 Trees with $k = n$ agents

Adds $+k = n$, + tree **to** Arbitrary graphs, complete communication (p. 173) VERIFIED

With as many agents as nodes, D^2 rounds suffice, and some trees need $D^2/3$; this is the bound dispersion inherits.

Assumed: graph dynamics: static graph; edge direction: undirected; node identifiers: labelled nodes (an agent can recognise a node it has visited); communication: global (any agent can read any other agent's state); timing: synchronous rounds; team size: $k = n$ agents; topology: tree. **Not assumed:** randomness: deterministic; node memory: no whiteboards.

Measure	Lower bound	Best upper bound	Status
Time	$D^2/3$ (Disser et al. 2020); $\Omega(D)$ (Disser et al. 2020) D^2 is matched only for trees of depth about $\sqrt{\log n}$; for other depths the gap is a factor D.	D^2 (Disser et al. 2020)	• polynomial gap
Memory	none known Memory is not bounded in these papers.	—	• unknown

Results.

- Disser et al. (2020) [lower bound]: time $D^2/3$. *Bottleneck:* online tree revelation. *Holds when:* deterministic, global communication; **existential in** D: the hard tree has depth $D = \lceil\sqrt{\log n}\rceil$ (Theorem 2.5); not shown for larger heights.
- Disser et al. (2020) [lower bound]: time $\Omega(D)$. *Bottleneck:* distance. *Holds when:* the root must reach the deepest node.
- Disser et al. (2020) [current best]: time D^2. Introductory and current best: phase-wise breadth-first search, D phases of D rounds.

Open questions.

Q1. For $k = n$ and trees of depth D well above $\sqrt{\log n}$ (e.g. $D = \sqrt{n}$), is $\Omega(D^2)$ still necessary?

Notes. Dispersion papers cite this bound as "$\Omega(D^2)$ for n agents on trees" (Augustine and Moses Jr. 2018) (Kshemkalyani et al. 2022); the theorem is existential in D, see the overview *From exploration to dispersion.*

16.9.4 Large teams, complete communication

Adds $+k \geq Dn^{1+\varepsilon}$ **to** Arbitrary graphs, complete communication (p. 173) VERIFIED

With $k = Dn^{\gamma}$ agents ($\gamma > 1$), exploration takes $D(1 + \frac{1}{\gamma-1} + o(1))$ rounds, almost the trivial D.

Assumed: graph dynamics: static graph; edge direction: undirected; node identifiers: labelled nodes (an agent can recognise a node it has visited); communication: global (any agent can read any other agent's state); timing: synchronous rounds; team size: large teams, $k \geq Dn^{1+\varepsilon}$. **Not assumed:** randomness: deterministic; node memory: no whiteboards; topology: arbitrary connected graph.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	$D(1+\frac{1}{\gamma}-o(1))$ (Dereniowski et al. 2015) $\Theta(D)$; the constants differ.	$D(1+\frac{1}{\gamma-1}+o(1))$ (Dereniowski et al. 2015)	• tight
Memory	none known Memory is not bounded in these papers.	—	• unknown

Results.

- Dereniowski et al. (2015) [lower bound]: time $D(1+\frac{1}{\gamma}-o(1))$. *Bottleneck:* distance. *Holds when:* $k = Dn^{\gamma}$ agents; $D = n^{o(1)}$; trees.
- Dereniowski et al. (2015) [current best]: time $D(1+\frac{1}{\gamma-1}+o(1))$. *Assumes:* n known. Introductory and current best; trees and general graphs.

Open questions.

Q1. Which team size is necessary and sufficient for $O(D)$-round exploration of graphs of larger diameter? (Stated in Dereniowski et al. 2015.)

Notes. For $k \le Dn^{1+\varepsilon}$, some trees need $D/(5\varepsilon)$ rounds (Disser et al. 2020), so the team size of (Dereniowski et al. 2015) is essentially needed for a constant ratio.

16.9.5 Trees, asynchronous, complete communication

Adds asynchronous, + tree

VERIFIED

An adversary chooses which agent moves; the cost is the total number of moves, and the optimal regret is $\Theta(\log^2 k)$ *per unit of depth.*

Assumed: graph dynamics: static graph; edge direction: undirected; node identifiers: labelled nodes (an agent can recognise a node it has visited); communication: global (any agent can read any other agent's state); topology: tree. **Not assumed:** randomness: deterministic; node memory: no whiteboards; timing: asynchronous; team size: any number k of agents.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(\log^2 k)\cdot(n+kD)$ (Cosson and Massoulié 2025) Regret $O(\log^2 k)$ matches $\Omega(\log^2 k)$; both bounds are from preprints.	$2n + O(kD\log^2 k)$ (Cosson and Massoulié 2026)	• tight
Memory	none known Memory is not bounded in these papers.	—	• unknown

Results.

- Cosson and Massoulié (2025) [lower bound]: time $\Omega(\log^2 k)\cdot(n+kD)$. *Bottleneck:* adversarial activation. *Holds when:* asynchronous: the adversary picks the agent that moves; centralised or distributed; from a layered-graph-traversal lower bound. Equivalently, any CR$(k)(n+kD)$-move algorithm has CR$(k) = \Omega(\log^2 k)$.
- Cosson et al. (2023) [introductory]: time $2n + kD^2(\log k + 2)$. Proposition 6: adversarial break-downs, the setting later formalised as asynchronous exploration; stated as an average of $2n/k + D^2(\log k + 2)$ moves per agent.
- Cosson and Massoulié (2024) [also]: time $2n + O(k^2 D)$.
- Cosson and Massoulié (2026) [current best]: time $2n + O(kD\log^2 k)$. Order-optimal regret. Preprint (2026).

Notes. The asynchronous model was introduced by (Cosson 2024). Both matching bounds are from preprints.

Claude (Anthropic) was used in cataloguing the problems.

16.9.6 Trees, communication by whiteboards only

Adds + synchronous, + tree VERIFIED

Agents talk only by writing at the node they occupy; the original algorithm still gives $O(k/\log k)$.

Assumed: graph dynamics: static graph; edge direction: undirected; node identifiers: labelled nodes (an agent can recognise a node it has visited); node memory: whiteboards at nodes; timing: synchronous rounds; topology: tree. **Not assumed:** randomness: deterministic; communication: none (silent agents; only see how many agents are co-located); team size: any number k of agents.

Measure	Lower bound	Best upper bound	Status
Time	none known $O(k/\log k)$ versus the inherited $\Omega(\log k/\log\log k)$ competitive ratio.	$O(D+n/\log k)$ (Fraigniaud et al. 2006a)	• polynomial gap
Memory	none known Memory is not bounded in these papers.	—	• unknown

Results.

- Fraigniaud et al. (2006a) [current best]: time $O(D+n/\log k)$. Introductory and current best in this model.

Open questions.

Q1. Is there a separation between write-read and complete communication? (Cosson and Massoulié 2025) conjecture that centralised communication helps.

16.9.7 Trees, asynchronous, whiteboards only

Adds asynchronous, + tree VERIFIED

The fully distributed asynchronous setting: the first competitive algorithm appeared in 2025.

Assumed: graph dynamics: static graph; edge direction: undirected; node identifiers: labelled nodes (an agent can recognise a node it has visited); node memory: whiteboards at nodes; topology: tree. **Not assumed:** randomness: deterministic; communication: none (silent agents; only see how many agents are co-located); timing: asynchronous; team size: any number k of agents.

Measure	Lower bound	Best upper bound	Status
Time	none known $O(k/\log k)$ versus the inherited $\Omega(\log^2 k)$ competitive ratio.	$2n+O(k^2 2^k D)$ (Cosson and Massoulié 2025)	• polynomial gap
Memory	none known Memory is not bounded in these papers.	—	• unknown

Results.

- Cosson and Massoulié (2025) [current best]: time $2n+O(k^2 2^k D)$. Introductory and current best; a variant is $O(k/\log k)$-competitive. Previous distributed algorithms, including that of (Fraigniaud et al. 2006a), fail here.

Open questions.

Q1. Is there a distributed asynchronous algorithm with polylogarithmic competitive ratio?

Claude (Anthropic) was used in cataloguing the problems.

16.9.8 Trees, no communication

Adds no communication, no whiteboards, + synchronous, + tree VERIFIED

Without any communication, k agents are no faster than one.

Assumed: graph dynamics: static graph; edge direction: undirected; node identifiers: labelled nodes (an agent can recognise a node it has visited); timing: synchronous rounds; topology: tree. **Not assumed:** randomness: deterministic; communication: none (silent agents; only see how many agents are co-located); node memory: no whiteboards; team size: any number k of agents.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)\cdot$ optimal (Fraigniaud et al. 2006a) Tight: a single depth-first search is $O(k)$-competitive, and without communication no algorithm is better by more than a constant (Fraigniaud et al. 2006a).	$O(k)\cdot$ optimal (Fraigniaud et al. 2006a)	• tight
Memory	none known Memory is not bounded in these papers.	—	• unknown

Results.

- Fraigniaud et al. (2006a) [lower bound]: time $\Omega(k)\cdot$ optimal. *Bottleneck:* no information sharing. *Holds when:* each agent knows only what it explored itself; some trees.
- Fraigniaud et al. (2006a) [current best]: time $O(k)\cdot$ optimal. The baseline that the lower bound matches: one agent explores the whole tree by depth-first search, in $2(n-1)$ moves, while the others wait; this is within a factor $O(k)$ of the optimum $\Theta(n/k+D)$ (Fraigniaud et al. 2006a).

16.9.9 Large teams, local communication

Adds + local communication, + synchronous, + $k \geq Dn^{1+\varepsilon}$ VERIFIED

If agents only talk when they meet, large teams still explore in $O(D)$ rounds, and the price of locality is visible in the constant.

Assumed: graph dynamics: static graph; edge direction: undirected; node identifiers: labelled nodes (an agent can recognise a node it has visited); communication: local (co-located agents communicate); timing: synchronous rounds; team size: large teams, $k \geq Dn^{1+\varepsilon}$. **Not assumed:** randomness: deterministic; node memory: no whiteboards; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	$D(1+\frac{2}{c}-o(1))$ (Dereniowski et al. 2015) $\Theta(D)$; the constants differ.	$D(1+\frac{2}{\gamma-1}+o(1))$ (Dereniowski et al. 2015)	• tight
Memory	none known Memory is not bounded in these papers.	—	• unknown

Results.

- Dereniowski et al. (2015) [lower bound]: time $D(1+\frac{2}{c}-o(1))$. *Bottleneck:* locality of communication. *Holds when:* $k = Dn^{\gamma}$ agents; $D = n^{o(1)}$; separates local from global communication.
- Dereniowski et al. (2015) [current best]: time $D(1+\frac{2}{\gamma-1}+o(1))$. Introductory and current best; $O(D)$ also when n and D are unknown.

Claude (Anthropic) was used in cataloguing the problems.

16.9.10 Anonymous graphs, a team of constant-memory agents

Adds anonymous nodes, + local communication, + synchronous VERIFIED

$\Theta(\log\log n)$ agents with constant memory are necessary and sufficient to explore any anonymous graph.

Assumed: graph dynamics: static graph; edge direction: undirected; communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** node identifiers: anonymous nodes; randomness: deterministic; node memory: no whiteboards; team size: any number k of agents; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known Polynomial; exponent not tracked.	$n^{O(1)}$ (Disser et al. 2019)	● unknown
Memory	$\Omega(\log\log n)$ agents (Disser et al. 2019) $\Theta(\log\log n)$ agents with $O(1)$ bits.	$O(1)$ bits each, $O(\log\log n)$ agents (Disser et al. 2019)	● tight

Results.

- Disser et al. (2019) [lower bound]: memory $\Omega(\log\log n)$ agents. *Bottleneck:* trap graphs. *Holds when:* each agent has $O((\log n)^{1-\varepsilon})$ bits; deterministic.
- Disser et al. (2019) [current best]: time $n^{O(1)}$, memory $O(1)$ bits each, $O(\log\log n)$ agents. *Assumes:* n unknown. Introductory and current best. One agent walks, the others act as pebbles.

Notes. Here the question is not speed but the size of a team of finite automata; it mirrors the single-agent pebble variant.

16.9.11 Anonymous graphs with $k = n$ agents

Adds $+\,k = n$ **to** Anonymous graphs, a team of constant-memory agents (p. 179) VERIFIED

Nodes cannot be recognised and agents only talk when co-located: n agents explore by dispersing, in $O(n)$ rounds with $O(\log n)$ bits each.

Assumed: graph dynamics: static graph; edge direction: undirected; communication: local (co-located agents communicate); timing: synchronous rounds; team size: $k = n$ agents. **Not assumed:** node identifiers: anonymous nodes; randomness: deterministic; node memory: no whiteboards; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(n)$ (Kshemkalyani and Ali 2019) $\Theta(n)$ in the worst case over graphs.	$O(n)$ (Kshemkalyani et al. 2025a)	● tight
Memory	none known Dispersion needs $\Omega(\log n)$ bits (Augustine and Moses Jr. 2018), but exploration need not disperse; no memory lower bound for exploration alone is recorded.	$O(\log n)$ (Kshemkalyani et al. 2025a)	● unknown

Results.

- Kshemkalyani and Ali (2019) [lower bound]: time $\Omega(n)$. *Bottleneck:* distance. *Holds when:* a path with all agents on one endpoint.
- Augustine and Moses Jr. (2018) [introductory]: time $O(m)$, memory $O(\log n)$. *Assumes:* knows n and m. Rooted dispersion; Augustine and Moses Jr. observe that any dispersion algorithm with $k = n$ solves n-agent exploration under the same conditions.
- Kshemkalyani et al. (2025a) [current best]: time $O(n)$, memory $O(\log n)$. Rooted dispersion in $O(k)$ rounds with $O(\log(k + \Delta))$ bits; exploration follows by the observation above (our inference, not stated in the paper).

Claude (Anthropic) was used in cataloguing the problems.

Open questions.

Q1. Can n agents explore an anonymous tree in $o(n)$ rounds, e.g. $O(D^2)$, with local communication? With labelled nodes and global communication D^2 suffices (Disser et al. 2020).

Notes. See the overview *From exploration to dispersion.*

16.10 Dispersion

Spread $k \leq n$ agents over the nodes of an n-node anonymous graph so that no node hosts more than one agent, as fast as possible and with as little memory per agent as possible.

First posed in 2018 (Augustine and Moses Jr. 2018). Posed at ICDCN 2018, fourteen years after collaborative exploration; the introductory paper already relates dispersion to n-robot collaborative exploration and takes its tree lower bound from it.

Problem statement and measures

Given a connected graph G with n nodes and m edges, and $k \leq n$ agents placed on the nodes of G in any initial **configuration** (the node of each agent; several agents may share a node), the agents must reach a configuration in which **every node hosts at most one agent**, and stay there.

Time is the number of rounds (synchronous) or epochs (asynchronous; an **epoch** is an interval in which every agent completes at least one full Communicate–Compute–Move step, not necessarily the shortest one) until the dispersed configuration is reached; where a result also makes every agent terminate explicitly, the notes say so. **Memory** is the number of bits of persistent memory each agent needs, counting its $O(\log k)$-bit identifier. Unless a variant or a result says otherwise, agents know nothing about n, m, k, Δ or D.

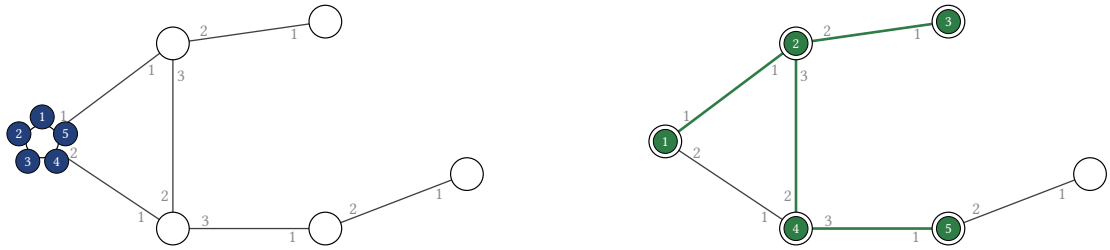
Dispersion is the natural "one agent per node" problem: it is equivalent to leaving one agent at every node of a depth-first search (DFS) tree of k nodes, which is why depth-first search bounds such as $\min(m, k\Delta)$ appear throughout, and it is a building block for exploration, leader election and self-deployment of agents.

Measures: **Time**, rounds in the synchronous model, epochs in the asynchronous model, until dispersion is reached; **Memory**, bits of persistent memory per agent, including its identifier.

A first look

Think of k electric cars arriving in a town where each charging point can serve only one car, and no car has a map. We model the town as a **graph**: charging points are **nodes**, roads between them are **edges**, and each car is an **agent**. The graph is connected and has n nodes and m edges; the nodes are **anonymous** (they have no names and nothing can be written on them). The $k \leq n$ agents have distinct **identifiers** (ID numbers) and start on nodes, several possibly on the same node. Time passes in synchronous **rounds**: in each round, agents at the same node can read each other's memory (**local communication**), and then each agent may move along one edge, choosing it by its **port** (the local number of that edge at the node). The agents know nothing about the graph, not even n. They must reach a **configuration** (the node of each agent) in which **every node holds at most one agent**; we measure the number of rounds and the number of bits of **memory** each agent needs.

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: k = 5 agents on one node; ports numbered at each node

(b) Final: agent 5 settles; at most one agent per node

Figure 16.10: Dispersion from a rooted start, by the depth-first search of (Augustine and Moses Jr. 2018) and (Kshemkalyani and Ali 2019): $k = 5$ agents start on one node of an anonymous port-labelled graph and move as one group. At each new node the agent with the smallest ID settles (green) and records its parent port; the group tries the ports in order, goes back when a node is occupied, and returns through the parent port when no port is left. One frame per round; the green edges form the depth-first tree.

1. Algorithm: Depth-first search with a settled agent per node. (Augustine and Moses Jr. 2018) (Rooted initial configuration, p. 190) When all k agents start on one node (the **rooted** case), the agents move as one group and run a **depth-first search** (DFS: go forward through an untried port while there is one, go back otherwise), using agents that stay behind on nodes (**settled agents**) as markers, as in (Augustine and Moses Jr. 2018) and (Kshemkalyani and Ali 2019).

1. At the start node, the agent with the smallest ID **settles**: it stays there for good and remembers by which port the group arrived (its **parent port**; none at the start).
2. The rest of the group leaves by the next port not yet tried at this node.
3. If the node the group reaches already has a settled agent, it has been visited: the group goes back by the port it came in and tries the next port there.
4. If the node is empty, the smallest ID in the group settles there and records the parent port; go to step 2.
5. When every port of a node has been tried, the group goes back through that node's parent port, which the settled agent tells it.
6. The group shrinks by one at each new node, and the procedure ends when the last agent settles.

Why it works and what it costs. This is a depth-first search in which the settled agents replace the "visited" marks that anonymous nodes cannot hold. Each edge is tried at most once from each of its two endpoints, and each try costs two moves (there and back) or turns into a tree edge, so the time is $O(m)$ rounds; tries happen only at the at most k nodes that receive an agent, each with at most Δ ports, which gives $O(\min(m, k\Delta))$, where Δ is the maximum degree. A settled agent remembers its ID and a few port numbers, $O(\log(k + \Delta))$ bits. The current best is $O(k)$ rounds with $O(\log(k + \Delta))$ bits, from any start (Kshemkalyani et al. 2025a): it gets rid of the cost of checking every port, which held the field at $O(\min(m, k\Delta))$ until (Sudo et al. 2024).

Why it matters. Using the agents themselves as markers is the idea every dispersion algorithm builds on, and it ties dispersion to collaborative exploration with anonymous nodes.

2. Lower bound: Distance: Omega(k) rounds. (Kshemkalyani and Ali 2019) (Arbitrary graph, arbitrary initial configuration, p. 186) Put all k agents on one end of a path (nodes in a row, each joined by an edge to

Claude (Anthropic) was used in cataloguing the problems.

the next). In the end the k agents must stand on k different nodes of the path, so one of them stands at least $k-1$ edges from the end where they all started. An agent crosses at most one edge per round.

Why it holds. So some agent needs at least $k-1$ rounds, whatever the algorithm: $\Omega(k)$ (Kshemkalyani and Ali 2019). The argument uses only distance, so it holds even for randomised algorithms, unlimited memory and global communication. The $O(k)$ algorithm of (Kshemkalyani et al. 2025a) matches it, so the time question is settled up to a constant factor.

Why it matters. It fixed the target: after it, every dispersion algorithm was measured by how far it is from k rounds.

3. Lower bound: Symmetry: Omega(log k) bits. (Augustine and Moses Jr. 2018) (Arbitrary graph, arbitrary initial configuration, p. 186) Suppose all agents run the same deterministic algorithm with the same amount of memory, and put them all on one node. If two of them start with exactly the same memory contents, they see the same things, make the same decisions and take the same ports in every round, so they stay together forever and dispersion fails. Hence the k agents on that node need k different memory contents at the start.

Why it holds. With b bits there are 2^b possible memory contents, so $2^b \geq k$, that is $b \geq \log k$ (Augustine and Moses Jr. 2018). This is why the bound counts the identifier: the ID is exactly what makes the memories differ. The best algorithms use $O(\log(k+\Delta))$ bits, so memory is optimal whenever Δ is at most polynomial in k; whether the $\log \Delta$ term is needed has no published proof for agents with identifiers.

Why it matters. It is the simplest instance of symmetry breaking, the obstacle behind the anonymous-agent results in dispersion, rendezvous and gathering.

What the community started looking at

Augustine and Moses Jr. posed dispersion as **a study of memory–time trade-offs** (Augustine and Moses Jr. 2018): the primary aim was to **minimise the number of rounds**, and then the **memory per agent**, with $\Omega(k)$ rounds and $\Omega(\log k)$ bits as the targets. Most of the later work closes these two gaps under weaker assumptions (arbitrary start, asynchrony, faults, dynamic graphs); the time-optimal $O(k)$ algorithm with $O(\log(k+\Delta))$ bits came in 2025 (Kshemkalyani et al. 2025a). Its roots are in n-agent collaborative exploration, which a dispersion with $k=n$ solves.

Relations to other problems

- **Derived from Collaborative exploration.** With $k=n$ agents, a dispersed configuration has an agent on every node, so any dispersion algorithm solves n-agent collaborative exploration under the same assumptions (Augustine and Moses Jr. 2018). Consequently every lower bound for n-agent exploration is a lower bound for dispersion (this is how the $\Omega(D^2)$ tree bound of Disser et al. 2020 enters), and every dispersion algorithm is an exploration algorithm for anonymous graphs with local communication. For $k<n$ dispersion asks for less: only k nodes must be occupied.
- **Related family: Computing with agents.** Most algorithms first bring the n agents into a dispersed configuration, one agent per node, so that every node has an agent to compute for it (Kshemkalyani et al. 2026) (Chand and Molla 2026).

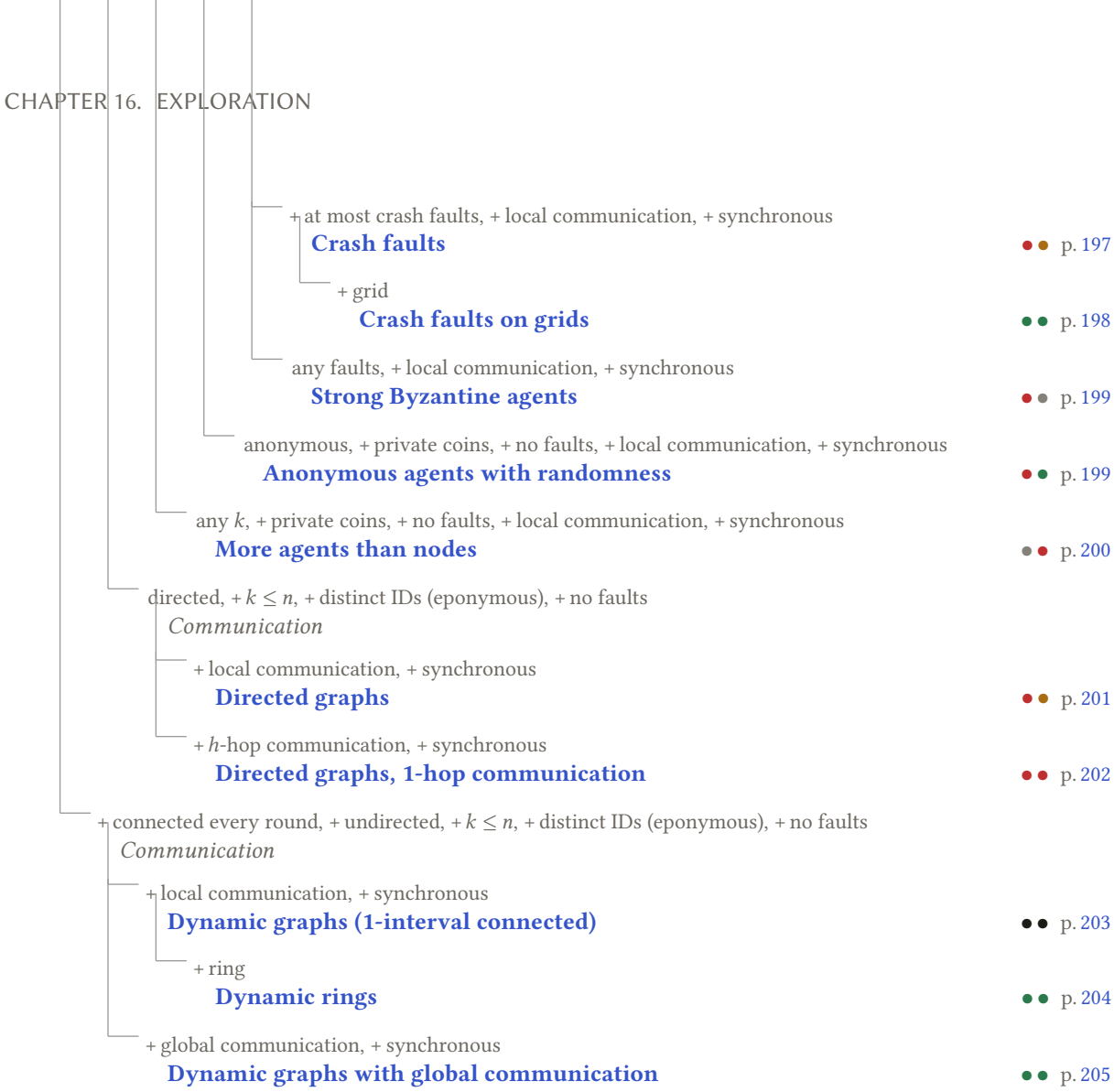
Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Memory) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Number of agents → Identifiers → Randomness → Faults → Communication → Timing → Node memory → Node identifiers → Topology → Initial configuration.



Claude (Anthropic) was used in cataloguing the problems.



What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Memory	p.
Asynchronous agents	● tight $O(k)$ vs $\Omega(k)$	● near-optimal $O(\log(k+\Delta))$ vs $\Omega(\log k)$	185
Arbitrary graph, arbitrary initial configuration STANDARD SETTING	● tight $O(k)$ vs $\Omega(k)$	● near-optimal $O(\log(k+\Delta))$ vs $\Omega(\log k)$	186
Trees	● polynomial gap $O(k)$ vs $\Omega(D^2)$	● near-optimal $O(\log(k+\Delta))$ vs $\Omega(\log k)$	188
Rooted trees	● polynomial gap $O(D^2)$ vs $\Omega(D^2)$	● tight $O(\log n)$ vs $\Omega(\log k)$	189
Rooted initial configuration	● tight $O(k)$ vs $\Omega(k)$	● near-optimal $O(\log(k+\Delta))$ vs $\Omega(\log k)$	190
Rings and paths	● tight $O(k)$ vs $\Omega(k)$	● tight $O(\log k)$ vs $\Omega(\log k)$	190
Grids	● polynomial gap $O(\min(k,\sqrt{n}))$ vs $\Omega(\sqrt{k})$	● tight $O(\log k)$ vs $\Omega(\log k)$	191

Claude (Anthropic) was used in cataloguing the problems.

Setting	Time	Memory	p.
Silent agents (no communication)	● polynomial gap $O(k \log L + k^2 \log \Delta)$ vs $\Omega(k + \log L)$	● tight $O(\log L + \log \Delta)$ vs $\Omega(\log L + \log \Delta)$	192
Silent agents on rings	● tight $O(k + \log L)$ vs $\Omega(k + \log L)$	● tight $O(\log L)$ vs $\Omega(\log L)$	193
Global communication	● tight $O(k)$ vs $\Omega(k)$	● near-optimal $O(\log(k + \Delta))$ vs $\Omega(\log k)$	194
Weak Byzantine agents	● polynomial gap $\tilde{O}(n^9)$ vs $\Omega(k)$	● unknown — vs $\Omega(\log k)$	194
Weak Byzantine agents on rings	● tight $O(n)$ vs $\Omega(n)$	● tight $O(\log n)$ vs $\Omega(\log n)$	196
Weak Byzantine agents on grids	● tight $O(\sqrt{n})$ vs $\Omega(\sqrt{n})$	● polynomial gap $O(n \log n)$ vs $\Omega(\log n)$	196
Crash faults	● polynomial gap $O(\min(m, k\Delta))$ vs $\Omega(k)$	● near-optimal $O(k \log(k + \Delta))$ vs $\Omega(\log k)$	197
Crash faults on grids	● tight $O(\sqrt{n})$ vs $\Omega(\sqrt{k})$	● tight $O(\log n)$ vs $\Omega(\log k)$	198
Strong Byzantine agents	● polynomial gap $O(X(n) + n^3)$ vs $\Omega(k)$	● unknown — vs —	199
Anonymous agents with randomness	● polynomial gap $O(k^2)$ vs $\Omega(k)$	● tight $O(\log \Delta)$ vs $\Omega(\log \Delta)$	199
More agents than nodes	● unknown $O(m + k)$ vs —	● polynomial gap $O(\Delta)$ vs $\Omega(\log \Delta)$	200
Directed graphs	● polynomial gap $O(k^2 \Delta_{out})$ vs $\Omega(k)$	● near-optimal $\Theta(\log(k + \Delta_{out}))$ vs $\Omega(\log k)$	201
Directed graphs, 1-hop communication	● polynomial gap $O(kD)$ vs $\Omega(k)$	● polynomial gap $O(k \log(k + \Delta_{out}))$ vs $\Omega(\log k)$	202
Dynamic graphs (1-interval connected)	● impossible	● impossible	203
Dynamic rings	● tight $O(n)$ vs $\Omega(n)$	● tight $O(1)$ vs —	204
Dynamic graphs with global communication	● tight $\Theta(k)$ vs $\Omega(k)$	● tight $\Theta(\log k)$ vs $\Omega(\log k)$	205

16.10.1 Asynchronous agents

Adds + local communication VERIFIED

Each agent's communicate–compute–move step takes finite but unbounded time; time is counted in epochs, intervals in which every agent completes at least one step.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: local (co-located agents communicate). **Not assumed:** randomness: deterministic; timing: asynchronous; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Statement for this setting. Same task; an adversarial scheduler activates agents at arbitrary speeds. An **epoch** is an interval of time in which every agent performs at least one full step; it need not be the shortest one, and bounds hold for every division of the execution into consecutive epochs. Settled agents must typically stay active, because an agent that stops cannot be told that it is needed.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Kshemkalyani and Ali 2019) $O(k)$ epochs versus $\Omega(k)$.	$O(k)$ (Pattanayak et al. 2025)	• tight
Memory	$\Omega(\log k)$ (Augustine and Moses Jr. 2018) $O(\log(k+\Delta))$ versus $\Omega(\log k)$.	$O(\log(k+\Delta))$ (Pattanayak et al. 2025)	• near-optimal

Results.

- Kshemkalyani and Ali (2019) [lower bound]: time $\Omega(k)$. *Bottleneck:* distance. *Holds when:* worst case over inputs: a path with all k agents on one endpoint; one edge per round; holds for randomized algorithms, unbounded memory and global communication; counted in epochs. Epochs; the synchronous argument carries over.
- Augustine and Moses Jr. (2018) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier.
- Kshemkalyani and Ali (2019) [introductory]: time $O(m)$, memory $O(k \log \Delta)$. *Assumes:* knows k. Three asynchronous algorithms; the only terminating one uses $O(D \log \Delta)$ bits and $O(\Delta^D)$ time. (Kshemkalyani 2025) is the journal treatment: $O(\min(m, k\Delta))$ time with $O(k \log \Delta)$ bits and no parameter knowledge.
- Kshemkalyani and Sharma (2021) [also]: time $O(\min(m, k\Delta))$, memory $O(\log(k+\Delta))$. Same bounds as its synchronous algorithm, in epochs.
- Kshemkalyani et al. (2025a) [also]: time $O(k \log k)$, memory $O(\log(k+\Delta))$. First near-linear asynchronous bound, for rooted and arbitrary configurations.
- Pattanayak et al. (2025) [current best]: time $O(k)$, memory $O(\log(k+\Delta))$. Time-optimal, rooted and arbitrary configurations. Uses the **port-one tree**: following port 1 from every node defines a structure that lets an agent check a neighbour's status in $O(1)$ epochs without timing assumptions.

Open questions.

Q1. Can an asynchronous algorithm let every agent **terminate** in polynomial time without knowledge of the graph? The only terminating asynchronous algorithm known uses $O(\Delta^{\min(D,k)+1})$ time (Kshemkalyani 2025); the linear-time algorithms keep settled agents active.

Q2. Semi-synchronous schedulers sit between the two models; no separation or dedicated bound is known.

Notes. The asynchronous time gap closed in two steps in 2025: $O(k \log k)$ (Kshemkalyani et al. 2025a), then $O(k)$ (Pattanayak et al. 2025). Sudo et al. (2024)'s rooted algorithms also work under any fair scheduler with atomic moves, but its general-configuration algorithm needs full synchrony.

16.10.2 Arbitrary graph, arbitrary initial configuration

Adds + synchronous **to** Asynchronous agents (p. 185) STANDARD SETTING VERIFIED

The standard setting: $k \leq n$ agents with unique identifiers, anywhere on an arbitrary anonymous graph, synchronous rounds, local communication, no faults, no knowledge of the graph.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Kshemkalyani and Ali 2019)	$O(k)$ (Kshemkalyani et al. 2025a)	• tight
	$O(k)$ matches $\Omega(k)$ as of 2025; the question raised in 2019–2021 of whether $\Omega(\min(m, k\Delta))$ could be a lower bound is settled in the negative.		
Memory	$\Omega(\log k)$ (Augustine and Moses Jr. 2018)	$O(\log(k+\Delta))$ (Kshemkalyani et al. 2025a)	• near-optimal
	$O(\log(k+\Delta))$ versus $\Omega(\log k)$: the two differ only when Δ is super-polynomial in k, and no $\Omega(\log \Delta)$ lower bound is proven.		

Results.

- Kshemkalyani and Ali (2019) [lower bound]: time $\Omega(k)$. *Bottleneck:* distance. *Holds when:* worst case over inputs: a path with all k agents on one endpoint; one edge per round; holds for randomized algorithms, unbounded memory and global communication. A path with all k agents at one end. Also $\Omega(D)$, and $\Omega(n)$ in the worst case, when $k = n$ (Augustine and Moses Jr. 2018).
- Augustine and Moses Jr. (2018) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier. Two agents with identical state move in lockstep forever; stated as $\Omega(\log n)$ for $k = n$. Several papers quote $\Omega(\log(k+\Delta))$, but the $\log \Delta$ term has no published proof for deterministic agents with identifiers.
- Augustine and Moses Jr. (2018) [introductory]: time $O(mn)$, memory $O(\log n)$. *Assumes:* $k = n$; knows n and m. Also $O(m)$ time with $O(n \log n)$ bits. Introduces the problem and the memory–time trade-off view.
- Kshemkalyani and Ali (2019) [also]: time $O(m)$, memory $O(k \log \Delta)$. *Assumes:* knows k. First treatment of $k \le n$. Also $O((m-n)k)$ time with $O(\log k + \log \Delta)$ bits.
- Kshemkalyani et al. (2019) [also]: time $O(\min(m, k\Delta) \log k)$, memory $O(\log n)$. *Assumes:* knows m, Δ, k. Parallel DFS traversals merged over $O(\log \ell)$ passes (ℓ = number of initially occupied nodes, so $\log \ell \le \log k$). Without knowing m: $O(k\Delta \log k)$ time with $O(\log(k+\Delta))$ bits.
- Shintaku et al. (2020) [also]: time $O(\min(m, k\Delta, k^2) \log \ell)$, memory $O(\log(k+\Delta))$. *Assumes:* knows a polynomial upper bound on identifiers. Same time as (Kshemkalyani et al. 2019) with no knowledge of m, k or Δ.
- Kshemkalyani and Sharma (2021) [also]: time $O(\min(m, k\Delta))$, memory $O(\log(k+\Delta))$. Removes the $\log \ell$ factor; matches single-source DFS. Optimal in time and memory on constant-degree graphs. Holds in the asynchronous model too.
- Sudo et al. (2024) [also]: time $O(k \log k \log \min(k, \Delta))$, memory $O(\log(k+\Delta))$. *Assumes:* knows a polynomial upper bound on identifiers. First bound independent of m and Δ: $O(k \log^2 k)$. Terminating all agents simultaneously needs upper bounds on k and on the number of edges among any k nodes.
- Kshemkalyani et al. (2025a) [current best]: time $O(k)$, memory $O(\log(k+\Delta))$. Time-optimal. Settled agents oscillate between their node and a neighbour so that the DFS head can test whether a neighbour is occupied in $O(1)$ rounds; multiple DFS traversals are merged without the log factors.

Open questions.

Q1. Can the memory be reduced to $O(\log k)$ bits, independent of Δ, while keeping $O(k)$ time? Equivalently, is there an $\Omega(\log \Delta)$ memory lower bound for deterministic agents with identifiers? The bound $\Omega(\log(k+\Delta))$ is quoted in several papers but only the $\log k$ term is argued.

Q2. Can all agents **terminate explicitly** in $O(k)$ rounds without any knowledge of k, m or Δ? (Sudo et al. 2024) terminates the rooted case in $O(k)$ extra rounds but needs global upper bounds in the general case; (Kshemkalyani et al. 2025a) does not address termination detection for arbitrary initial configurations.

Q3. **Ongoing dispersion** (Kshemkalyani and Ali 2019): agents that have settled must periodically leave (for example to recharge) and the configuration must be re-dispersed; no bounds beyond the original proposal are known.

Claude (Anthropic) was used in cataloguing the problems.

Notes. Progression. (Augustine and Moses Jr. 2018) posed the problem for $k = n$ with $O(mn)$ time and $O(\log n)$ bits, or $O(m)$ time with $O(n \log n)$ bits, and asked whether $o(m)$ time was possible. (Kshemkalyani and Ali 2019) allowed $k \leq n$, gave $O(m)$ time with $O(k \log \Delta)$ bits and proved the $\Omega(k)$ time bound that every later paper is measured against. (Kshemkalyani et al. 2019) reached $O(\min(m, k\Delta) \log k)$ time with logarithmic memory but needed m, Δ and k; (Shintaku et al. 2020) removed that knowledge; (Kshemkalyani and Sharma 2021) removed the log factor and reached the DFS bound $O(\min(m, k\Delta))$, optimal for constant degree. The DFS barrier was broken by (Sudo et al. 2024) with $O(k \log^2 k)$, and (Kshemkalyani et al. 2025a) closed the time question with $O(k)$.

What the DFS barrier was. A DFS head arriving at a node v cannot tell which neighbours already hold an agent without visiting them, which costs $\Theta(\deg(v))$ rounds per node and $\Theta(\min(m, k\Delta))$ in total. The near-linear and linear algorithms avoid this by letting settled agents advertise themselves at neighbours (Sudo et al. 2024: time slots; Kshemkalyani et al. 2025a: oscillation), or in the asynchronous case by exploiting the port-one tree structure (Pattanayak et al. 2025).

Knowledge. Where a result needs knowledge of a parameter it is listed on the result. The 2025 linear-time algorithms do not state any parameter knowledge explicitly; their descriptions use k (for example, to select $\lceil k/3 \rceil$ seekers), so treat "no knowledge" for them as unconfirmed.

16.10.3 Trees

Adds + tree **to** Arbitrary graph, arbitrary initial configuration (p. 186) VERIFIED

No cycles, so a DFS never needs to test for revisits; the difficulty is spreading agents that start far apart across long branches.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: local (co-located agents communicate); timing: synchronous rounds; topology: tree. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(D^2)$ (Disser et al. 2020)	$O(k)$ (Kshemkalyani et al. 2025a)	• polynomial gap
	$O(n)$ for $k = n$ versus a lower bound of $\Omega(D^2)$ that is proved only for $D \approx \sqrt{\log n}$; in general the gap is polynomial.		
Memory	$\Omega(\log k)$ (Augustine and Moses Jr. 2018)	$O(\log(k + \Delta))$ (Kshemkalyani et al. 2025a)	• near-optimal
	$O(\log(k + \Delta))$ versus $\Omega(\log k)$.		

Results.

- Disser et al. (2020) [lower bound]: time $\Omega(D^2)$. *Bottleneck:* collaborative tree exploration. *Holds when:* $k = n$: every node must be reached; deterministic; **existential in** D: the hard tree has height $D = \lceil \sqrt{\log n} \rceil$, so the bound is about $\log n$ rounds; it is not shown for larger heights; proved for collaborative exploration with labelled nodes and global communication, so it holds for dispersion (Augustine and Moses Jr. 2018) (Kshemkalyani et al. 2022). Transferred from collaborative tree exploration (Theorem 2.5 of Disser et al. 2020) by (Augustine and Moses Jr. 2018); re-proven for the global model in (Kshemkalyani et al. 2022). In addition $\Omega(k)$ (Kshemkalyani and Ali 2019).
- Augustine and Moses Jr. (2018) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier.
- Augustine and Moses Jr. (2018) [introductory]: time $O(n)$, memory $O(\log n)$. *Assumes:* $k = n$; knows n. Memory-optimal; the authors ask whether $o(n)$ rounds are possible.
- Kshemkalyani et al. (2025a) [current best]: time $O(k)$, memory $O(\log(k + \Delta))$. Inherited from arbitrary graphs. For $k = n$ this is $O(n)$, no better than 2018.

Claude (Anthropic) was used in cataloguing the problems.

Open questions.

Q1. (Augustine and Moses Jr. 2018) asked for $o(n)$-round dispersion on arbitrary trees. Precisely: for $k = n$ and an arbitrary initial configuration, is there an algorithm whose time is bounded by a function of D alone, ideally $O(D^2)$ as in the rooted case? No tree-specific algorithm has appeared since 2018; the linear-time general algorithms give only $O(n)$.

Q2. With global communication, trees admit $O(D \cdot \max(D, k))$ rounds with $O(\max(D, \Delta \log k))$ bits (arXiv version of Kshemkalyani et al. 2020b); whether the $\max(D, k)$ factor can be reduced to D, and whether local communication suffices, is open.

Notes. Anonymous randomized agents from a rooted tree configuration: $O(n)$ rounds with $O(\log \Delta)$ bits (Molla and Moses Jr. 2019).

16.10.4 Rooted trees

Adds + rooted start **to** Trees (p. 188) VERIFIED

All agents start at one node of a tree; the time is governed by the diameter, as in collaborative tree exploration.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: local (co-located agents communicate); timing: synchronous rounds; topology: tree; initial configuration: rooted (all agents on one node). **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(D^2)$ (Disser et al. 2020)	$O(D^2)$ (Augustine and Moses Jr. 2018)	• polynomial gap
	$O(D^2)$ is matched only for trees of height about $\sqrt{\log n}$; for larger heights only $\Omega(D)$ is known, a factor D.		
Memory	$\Omega(\log k)$ (Augustine and Moses Jr. 2018)	$O(\log n)$ (Augustine and Moses Jr. 2018)	• tight
	$\Theta(\log n)$ for $k = n$.		

Results.

- Disser et al. (2020) [lower bound]: time $\Omega(D^2)$. *Bottleneck:* collaborative tree exploration. *Holds when:* $k = n$: every node must be reached; deterministic; **existential in** D: the hard tree has height $D = \lceil\sqrt{\log n}\rceil$, so the bound is about $\log n$ rounds; it is not shown for larger heights; proved for collaborative exploration with labelled nodes and global communication, so it holds for dispersion (Augustine and Moses Jr. 2018) (Kshemkalyani et al. 2022). Transferred from collaborative tree exploration (Theorem 2.5 of Disser et al. 2020) by (Augustine and Moses Jr. 2018); re-proven for the global model in (Kshemkalyani et al. 2022).
- Augustine and Moses Jr. (2018) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier.
- Augustine and Moses Jr. (2018) [current best]: time $O(D^2)$, memory $O(\log n)$. *Assumes:* $k = n$; knows n. Introductory and current best; the full (arXiv) version improves the memory from $O(\log n + \Delta)$ in the conference version to $O(\log n)$.
- Kshemkalyani et al. (2025a) [also]: time $O(k)$, memory $O(\log(k + \Delta))$. Inherited; better than $O(D^2)$ only when $k = o(D^2)$.

Open questions.

Q1. For $k < n$ the lower bound $\Omega(D^2)$ no longer applies (only k nodes must be reached). Whether $O(\min(k, D^2))$ time is achievable, and whether it is necessary, has not been addressed.

Q2. For $k = n$ agents rooted on a tree of height D much larger than $\sqrt{\log n}$, is $\Omega(D^2)$ time necessary? The known bound is existential in D (Disser et al. 2020).

Claude (Anthropic) was used in cataloguing the problems.

16.10.5 Rooted initial configuration

Adds any topology, + rooted start **to** Arbitrary graph, arbitrary initial configuration (p. 186) VERIFIED

All k agents start on the same node, so a single DFS traversal that drops one agent per new node solves the problem; the question is how fast and with how little memory.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: local (co-located agents communicate); timing: synchronous rounds; initial configuration: rooted (all agents on one node). **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Kshemkalyani and Ali 2019)	$O(k)$ (Kshemkalyani et al. 2025a)	• tight
	$O(k)$ versus $\Omega(k)$.		
Memory	$\Omega(\log k)$ (Augustine and Moses Jr. 2018)	$O(\log(k+\Delta))$ (Kshemkalyani et al. 2025a)	• near-optimal
	$O(\log(k+\Delta))$ versus $\Omega(\log k)$; see the base case.		

Results.

- Kshemkalyani and Ali (2019) [lower bound]: time $\Omega(k)$. *Bottleneck:* distance. *Holds when:* worst case over inputs: a path with all k agents on one endpoint; one edge per round; holds for randomized algorithms, unbounded memory and global communication. The $\Omega(k)$ argument is already rooted: all agents at one end of a path.
- Augustine and Moses Jr. (2018) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier.
- Augustine and Moses Jr. (2018) [introductory]: time $O(m)$, memory $O(\log n)$. *Assumes:* $k = n$; knows n and m. DFS that leaves the smallest-identifier agent at every newly discovered node; time $O(\min(m, k\Delta))$ for $k \leq n$ in later restatements.
- Sudo et al. (2024) [also]: time $O(k \log \min(k, \Delta))$, memory $O(\log(k+\Delta))$. Also $O(k)$ time with $O(\Delta + \log k)$ bits, the first linear-time algorithm. Both terminate: the last settled agent detects completion and relays it through the DFS tree in $O(k)$ extra rounds. Work under any fair scheduler with atomic moves.
- Kshemkalyani et al. (2025a) [current best]: time $O(k)$, memory $O(\log(k+\Delta))$. Linear time with logarithmic memory.

Open questions.

Q1. Is $O(k)$ time achievable with $O(\log k)$ bits? Memory $O(\log(k+\Delta))$ is needed by every known algorithm to remember a port number.

Notes. The rooted case is the one solved first and the one where every measure is settled up to the $\log \Delta$ term. In the asynchronous model the rooted case also has $O(k)$ epochs (Pattanayak et al. 2025) (and $O(k \log k)$ in Kshemkalyani et al. 2025a).

16.10.6 Rings and paths

Adds + ring **to** Arbitrary graph, arbitrary initial configuration (p. 186) VERIFIED

Degree two everywhere; both measures are settled.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: local (co-located agents communicate); timing: synchronous rounds; topology: ring or path. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Kshemkalyani and Ali 2019) $O(k)$ versus $\Omega(k)$.	$O(k)$ (Kshemkalyani et al. 2025a)	• tight
Memory	$\Omega(\log k)$ (Augustine and Moses Jr. 2018) $O(\log k)$ versus $\Omega(\log k)$.	$O(\log k)$ (Kshemkalyani et al. 2025a)	• tight

Results.

- Kshemkalyani and Ali (2019) [lower bound]: time $\Omega(k)$. *Bottleneck:* distance. *Holds when:* all k agents on one node: the farthest target is $k/2$ away; any algorithm. $\Omega(n)$ when $k = n$, since $D = \Theta(n)$ (Augustine and Moses Jr. 2018).
- Augustine and Moses Jr. (2018) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier.
- Augustine and Moses Jr. (2018) [introductory]: time $O(n)$, memory $O(\log n)$. *Assumes:* $k = n$; knows n. One algorithm for paths, rings and trees; optimal in both measures on paths and rings for $k = n$.
- Kshemkalyani et al. (2025a) [current best]: time $O(k)$, memory $O(\log k)$. Inherited from arbitrary graphs; $O(\log(k+\Delta)) = O(\log k)$ because $\Delta = 2$.

Open questions.

Q1. Anonymous randomized agents disperse from a **rooted** ring configuration with $O(1)$ bits in $O(n)$ rounds (Molla and Moses Jr. 2019). From an **arbitrary** configuration the known algorithm never terminates; is $O(1)$-bit dispersion with termination possible on rings?

Notes. The ring is the testbed for every harder variant: silent agents, Byzantine agents and dynamic edges all have ring-specific results, catalogued under those branches.

16.10.7 Grids

Adds + grid **to** Arbitrary graph, arbitrary initial configuration (p. 186) VERIFIED

A $\sqrt{n} \times \sqrt{n}$ anonymous grid; the diameter gives an $\Omega(\sqrt{k})$ bound and the question is whether local communication can reach it for small k.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \le n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: local (co-located agents communicate); timing: synchronous rounds; topology: grid. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; initial configuration: arbitrary (agents on any $\ell \le k$ nodes).

Statement for this setting. The graph is a $\sqrt{n} \times \sqrt{n}$ grid with $\Delta = 4$. Whether an agent can tell the four directions apart consistently (an **oriented** grid) or only sees locally labelled ports (an **unoriented** grid) is stated on each result.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(\sqrt{k})$ (Kshemkalyani et al. 2020c) $O(\min(k, \sqrt{n}))$ versus $\Omega(\sqrt{k})$: tight for $k = \Omega(n)$, a factor $\sqrt{k}$ apart for $k \le \sqrt{n}$ under local communication.	$O(\min(k, \sqrt{n}))$ (Kshemkalyani et al. 2020c)	• polynomial gap
Memory	$\Omega(\log k)$ (Kshemkalyani et al. 2020c) $\Theta(\log k)$.	$O(\log k)$ (Kshemkalyani et al. 2020c)	• tight

Results.

Claude (Anthropic) was used in cataloguing the problems.

- Kshemkalyani et al. (2020c) [lower bound]: time $\Omega(\sqrt{k})$. *Bottleneck:* distance. *Holds when:* k agents on one node: a grid ball of radius r holds $O(r^2)$ nodes; any algorithm. Any k agents on one node must spread over a region of diameter $\Omega(\sqrt{k})$; $\Omega(\sqrt{n})$ when $k = \Omega(n)$.
- Kshemkalyani et al. (2020c) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier.
- Kshemkalyani et al. (2020c) [current best]: time $O(\min(k, \sqrt{n}))$, memory $O(\log k)$. *Assumes:* knows n and k (arXiv version). Introductory and current best for square grids with local communication; optimal in both measures when $k = \Omega(n)$. With global communication: $O(\sqrt{k})$ rounds and $O(\log k)$ bits for every k, optimal.
- Banerjee et al. (2024) [also]: time $O(\sqrt{n})$, memory $O(\log n)$. *Assumes:* unoriented grid; knows n. First result without a sense of direction; the authors note the algorithm can be adapted to unknown n. Crash-tolerant versions are under the crash-fault branch.
- Himani and Pandit (2025) [also]: time $O(\sqrt{n})$, memory $O(\log n)$. *Assumes:* unoriented triangular grid. No knowledge of n; the agents compute $\sqrt{n}$ themselves.

Open questions.

Q1. Can $k \leq n$ agents disperse on a grid in $O(\sqrt{k})$ rounds with **local** communication? Global communication achieves it (Kshemkalyani et al. 2020c); asked again in (Banerjee et al. 2025).

Q2. A lower bound in terms of k for $k = o(\sqrt{n})$ with $O(\log n)$ memory on unoriented grids (Banerjee et al. 2024).

Q3. Unoriented grids with no knowledge of n and with faults, matching the fault-free bounds (Banerjee et al. 2024).

Notes. Grids are the one class where global communication is known to help: $O(\sqrt{k})$ versus $O(\min(k, \sqrt{n}))$. Rectangular $x \times y$ grids with x, y known: $O(\min(k, \max(x, y)))$ rounds with $\Theta(\log k)$ bits (arXiv version of Kshemkalyani et al. 2020c).

16.10.8 Silent agents (no communication)

Adds no communication, + synchronous VERIFIED

Agents cannot exchange any information; at its node an agent only learns whether it is alone and whether the number of agents there went up or down since the last round.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; timing: synchronous rounds. **Not assumed:** randomness: deterministic; communication: none (silent agents; only see how many agents are co-located); node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Statement for this setting. No communication at all. At the start of a round an agent at v learns (i) whether it is alone at v and (ii) whether the number of agents at v increased or decreased since the previous round. Agents have identifiers in $[0, L]$ and know only their own; memory is counted as **additional** bits beyond the identifier.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k + \log L)$ (Das et al. 2025b) $O(k \log L + k^2 \log \Delta)$ versus $\Omega(k + \log L)$.	$O(k \log L + k^2 \log \Delta)$ (Gorain et al. 2022b)	• polynomial gap
Memory	$\Omega(\log L + \log \Delta)$ (Gorain et al. 2022b) $\Theta(\log L + \log \Delta)$ additional bits.	$O(\log L + \log \Delta)$ (Gorain et al. 2022b)	• tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Das et al. (2025b) [lower bound]: time $\Omega(k + \log L)$. *Bottleneck:* timing is the only channel. *Holds when:* no communication: only alone / count-change sensing; identifiers in $[0, L]$, unknown to the other agents; $\Omega(k)$ from distance; $\Omega(\log L)$ from distinguishing identifiers by waiting; proven on rings, hence for arbitrary graphs. Proven on rings, hence for arbitrary graphs.
- Gorain et al. (2022b) [lower bound]: memory $\Omega(\log L + \log \Delta)$. *Bottleneck:* encoding identifier and ports. *Holds when:* additional bits beyond the identifier; rooted configuration; deterministic. Additional bits; no algorithm with o(log L + log Δ) additional bits exists in this model.
- Gorain et al. (2022b) [current best]: time $O(k \log L + k^2 \log \Delta)$, memory $O(\log L + \log \Delta)$. *Assumes:* rooted. Introductory and current best for general graphs; the agents terminate. Information is conveyed only by the timing of moves and by counts of co-located agents.

Open questions.

Q1. Close the time gap on general graphs: is $O(k + \log L)$ achievable, as on rings?

Q2. Arbitrary (multi-source) initial configurations on general graphs; only rings are solved (Das et al. 2025b).

Q3. Weaker sensing: agents that only learn whether they are alone and whether the count decreased, and agents that wake up at different times (a 2026 ICDCIT paper by Gorain, Mandal and Sharma treats this; its bounds are not recorded here).

16.10.9 Silent agents on rings

Adds + ring **to** Silent agents (no communication) (p. 192) VERIFIED

On an oriented ring, silent agents from arbitrary configurations disperse in optimal $\Theta(k + \log L)$ rounds.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; timing: synchronous rounds; topology: ring or path. **Not assumed:** randomness: deterministic; communication: none (silent agents; only see how many agents are co-located); node memory: no whiteboards; node identifiers: anonymous nodes; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k + \log L)$ (Das et al. 2025b) $\Theta(k + \log L)$.	$O(k + \log L)$ (Das et al. 2025b)	• tight
Memory	$\Omega(\log L)$ (Gorain et al. 2022b) $\Theta(\log L)$.	$O(\log L)$ (Das et al. 2025b)	• tight

Results.

- Das et al. (2025b) [lower bound]: time $\Omega(k + \log L)$. *Bottleneck:* timing is the only channel. *Holds when:* no communication: only alone / count-change sensing; identifiers in $[0, L]$, unknown to the other agents.
- Gorain et al. (2022b) [lower bound]: memory $\Omega(\log L)$. *Bottleneck:* encoding the identifier. *Holds when:* additional bits beyond the identifier; $\Delta = 2$, so only the $\log L$ term remains. $\Delta = 2$.
- Das et al. (2025b) [current best]: time $O(k + \log L)$, memory $O(\log L)$. *Assumes:* oriented ring. Introductory and current best; arbitrary initial configurations.

Open questions.

Q1. Unoriented rings, and rings where agents cannot detect count changes.

Claude (Anthropic) was used in cataloguing the problems.

16.10.10 Global communication

Adds + global communication, + synchronous VERIFIED

Any agent can read the state of any other agent at any time, but still does not know the graph or where the others are.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: global (any agent can read any other agent's state); timing: synchronous rounds. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Kshemkalyani et al. 2022) Inherited $O(k)$ versus $\Omega(k)$.	$O(k)$ (Kshemkalyani et al. 2025a)	• tight
Memory	$\Omega(\log k)$ (Kshemkalyani et al. 2022) $O(\log(k+\Delta))$ versus $\Omega(\log k)$.	$O(\log(k+\Delta))$ (Kshemkalyani et al. 2025a)	• near-optimal

Results.

- Kshemkalyani et al. (2022) [lower bound]: time $\Omega(k)$. *Bottleneck:* distance. *Holds when:* worst case over inputs: a path with all k agents on one endpoint; one edge per round; re-proven for global communication. Re-proven for the global model; $\Omega(D^2)$ on trees when $k = n$.
- Kshemkalyani et al. (2022) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier; identifiers in $[1, k]$ counted as memory.
- Kshemkalyani et al. (2022) [introductory]: time $O(k\Delta)$, memory $O(\log(k+\Delta))$. Or $O(\min(m, k\Delta))$ time with $O(\Delta + \log k)$ bits. A BFS-based algorithm gives $O((D+k)\Delta(D+\Delta))$ time with $O(\log D + \Delta \log k)$ bits. The conference version (Kshemkalyani et al. 2020b) claimed $O(\min(m, k\Delta))$ time with $O(\log(k+\Delta))$ bits; the journal version withdraws that combination.
- Kshemkalyani et al. (2025a) [current best]: time $O(k)$, memory $O(\log(k+\Delta))$. Inherited from the local-communication model, which is weaker; no algorithm specific to global communication does better.

Open questions.

Q1. Does global communication buy anything for dispersion? Since $O(k)$ time with $O(\log(k+\Delta))$ bits holds with local communication, a separation could only come from memory ($O(\log k)$ bits), from termination detection, or from the BFS-style question of achieving time close to $O(D)$-dependent bounds for small D.

Q2. (Kshemkalyani et al. 2022) asks for $O(\min(m, k\Delta))$ time with only $O(\log(k+\Delta))$ bits in the global model; this is implied by the later local-model result, so the live version of the question is the memory question above.

Notes. The global model was introduced to see how much easier dispersion becomes when agents can coordinate freely. The answer so far is: asymptotically not at all in time, once the local-model algorithms of 2025 are taken into account.

16.10.11 Weak Byzantine agents

Adds + at most weak Byzantine faults, + local communication, + synchronous VERIFIED

Up to f agents behave arbitrarily and may lie, but cannot fake their identifiers; the non-faulty agents must

Claude (Anthropic) was used in cataloguing the problems.

still settle on distinct nodes and terminate.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: up to f weak Byzantine agents (arbitrary behaviour, cannot fake identifiers); communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Statement for this setting. Up to f agents are controlled by an adversary that knows the algorithm and the honest agents' states; they may move and communicate arbitrarily but cannot change their identifiers. The goal is a configuration in which every node holds at most one **non-Byzantine** agent, with all non-Byzantine agents terminated. Results below take $k = n$ and assume the agents know n. $X(n)$ denotes the time to explore any n-node graph, $\tilde{O}(n^5)$.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Kshemkalyani and Ali 2019); *impossible* (Molla et al. 2021) Polynomial, but $\tilde{O}(n^9)$ versus $\Omega(n)$; the bottleneck is Byzantine gathering.	$\tilde{O}(n^9)$ (Molla et al. 2021)	• polynomial gap
Memory	$\Omega(\log k)$ (Augustine and Moses Jr. 2018) Memory is not bounded in the general-graph algorithms.	—	• unknown

Results.

- Kshemkalyani and Ali (2019) [lower bound]: time $\Omega(k)$. *Bottleneck:* distance. *Holds when:* fault-free bound ($f = 0$); a path with all k agents on one endpoint. Fault-free bound; no Byzantine-specific time lower bound is known.

- Molla et al. (2021) [lower bound]: time *impossible*. *Bottleneck:* indistinguishable Byzantine agents. *Holds when:* $\lceil k/n \rceil > \lceil (k-f)/n \rceil$; weak Byzantine agents; deterministic; even with known n, k and f. The Byzantine agents can behave exactly like honest agents and occupy nodes, so the honest agents cannot reach the balanced target.

- Augustine and Moses Jr. (2018) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier.

- Molla et al. (2021) [current best]: time $\tilde{O}(n^9)$. *Assumes:* $k = n$; knows n; $f \leq \lfloor n/2 \rfloor - 1$. Introductory and current best on general graphs. Precisely $O(n^4 \cdot |\Lambda_{good}| \cdot X(n))$ rounds for arbitrary configurations, $|\Lambda_{good}|$ the length of the largest honest identifier; $O(n^4)$ if the agents start gathered; $O(n^3)$ gathered with $f \leq \lfloor n/3 \rfloor - 1$; $O((f + |\Lambda|) \cdot X(n))$ for $f = O(\sqrt{n})$. Tolerates $f \leq n - 1$ in polynomial time on graphs isomorphic to their quotient graph. Memory per agent is not analysed except for that last algorithm ($O(m \log n)$ bits).

Open questions.

Q1. Faster Byzantine dispersion on general graphs: the time is dominated by Byzantine gathering, and the authors ask whether building a map before settling is necessary (Molla et al. 2021).

Q2. Tolerating more than $n/2$ weak Byzantine agents on general graphs (possible only on the restricted class so far) (Molla et al. 2021).

Q3. Byzantine dispersion with bounded memory per agent, on trees, with $k < n$, or under asynchrony: nothing is known (Molla et al. 2020) (Chand et al. 2023b).

Notes. Byzantine dispersion was introduced on rings (Molla et al. 2020) and then on general graphs (Molla et al. 2021); no general-graph improvement has appeared since 2021.

Claude (Anthropic) was used in cataloguing the problems.

16.10.12 Weak Byzantine agents on rings

Adds + ring **to** Weak Byzantine agents (p. 194) VERIFIED

Optimal $O(n)$ time and $O(\log n)$ bits are possible on rings when fewer than $n/17$ agents are Byzantine and f is known.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: up to f weak Byzantine agents (arbitrary behaviour, cannot fake identifiers); communication: local (co-located agents communicate); timing: synchronous rounds; topology: ring or path. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(n)$ (Molla et al. 2020) $\Theta(n)$ under the four assumptions; $O(n^2)$ without them.	$O(n)$ (Molla et al. 2020)	• tight
Memory	$\Omega(\log n)$ (Augustine and Moses Jr. 2018) $\Theta(\log n)$.	$O(\log n)$ (Molla et al. 2020)	• tight

Results.

- Molla et al. (2020) [lower bound]: time $\Omega(n)$. *Bottleneck:* distance. *Holds when:* $k = n$: the ring has diameter $n/2$; any algorithm. Diameter $n/2$, $k = n$.
- Augustine and Moses Jr. (2018) [lower bound]: memory $\Omega(\log n)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; $k = n$.
- Molla et al. (2020) [current best]: time $O(n)$, memory $O(\log n)$. *Assumes:* $k = n$; knows n and f; $f \leq \lfloor (n-4)/17 \rfloor$; identifiers in $[1, n]$. Introductory and current best. Without those assumptions: $O(n^2)$ rounds with $O(\log n)$ bits for any $f \leq n-1$ ($O(fn)$ if f is known), or n rounds with $O(n \log n)$ bits. From a rooted configuration, $n-1$ rounds with $O(\log n)$ bits for any $f \leq k-1$ without knowing n or f.

Open questions.

Q1. Keep $O(n)$ time and $O(\log n)$ bits while dropping any of: identifiers in $[1, n]$, knowledge of f, the bound $f \leq (n-4)/17$, the follow primitive (Molla et al. 2020).

Q2. Asynchronous Byzantine dispersion on rings (Molla et al. 2020).

16.10.13 Weak Byzantine agents on grids

Adds + grid **to** Weak Byzantine agents (p. 194) VERIFIED

Time-optimal $O(\sqrt{n})$ rounds on oriented grids for any number of weak Byzantine agents, at the price of $O(n \log n)$ bits.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: up to f weak Byzantine agents (arbitrary behaviour, cannot fake identifiers); communication: local (co-located agents communicate); timing: synchronous rounds; topology: grid. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(\sqrt{n})$ (Kshemkalyani et al. 2020c) $\Theta(\sqrt{n})$.	$O(\sqrt{n})$ (Banerjee et al. 2025)	• tight
Memory	$\Omega(\log n)$ (Augustine and Moses Jr. 2018) $O(n \log n)$ versus $\Omega(\log n)$.	$O(n \log n)$ (Banerjee et al. 2025)	• polynomial gap

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Kshemkalyani et al. (2020c) [lower bound]: time $\Omega(\sqrt{n})$. *Bottleneck:* distance. *Holds when:* $k = n$: the grid has diameter $2\sqrt{n}$; any algorithm. $k = n$.
- Augustine and Moses Jr. (2018) [lower bound]: memory $\Omega(\log n)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; $k = n$.
- Banerjee et al. (2025) [current best]: time $O(\sqrt{n})$, memory $O(n \log n)$. *Assumes:* oriented grid; $k = n$; $f \leq n$. Introductory and current best.

Open questions.

Q1. Reduce the memory below $O(n \log n)$ bits on grids with weak Byzantine agents.

Q2. Strong Byzantine agents (able to fake identifiers) on grids (Banerjee et al. 2025).

16.10.14 Crash faults

Adds + at most crash faults, + local communication, + synchronous VERIFIED

Up to $f \leq k$ agents may crash at any time; a crashed agent disappears with everything it knew, and the surviving $k - f$ agents must still end on distinct nodes.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: up to f agents may crash (stop forever); communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Statement for this setting. An adversary may crash up to $f \leq k$ agents at arbitrary times. A crashed agent loses its memory and can no longer move or be seen by others. Dispersion is achieved when the $k - f$ non-faulty agents occupy $k - f$ distinct nodes. ℓ is the number of initially occupied nodes.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Kshemkalyani and Ali 2019)	$O(\min(m, k\Delta))$ (Pattanayak et al. 2023)	• polynomial gap
	$O(\min(m, k\Delta))$ versus $\Omega(k)$: a factor Δ, as in the fault-free DFS era; the fault-free $O(k)$ techniques (oscillating settled agents) have not been made crash-tolerant.		
Memory	$\Omega(\log k)$ (Augustine and Moses Jr. 2018)	$O(k \log(k + \Delta))$ (Pattanayak et al. 2023)	• near-optimal
	$O(\log(k + \Delta))$ bits are enough but only with an extra factor $(\ell + f)$ in time; the fast algorithm uses $O(k \log(k + \Delta))$.		

Results.

- Kshemkalyani and Ali (2019) [lower bound]: time $\Omega(k)$. *Bottleneck:* distance. *Holds when:* fault-free bound ($f = 0$); worst case over inputs: a path with all k agents on one endpoint; one edge per round; holds for randomized algorithms, unbounded memory and global communication. Fault-free lower bound; no crash-specific lower bound is known.
- Augustine and Moses Jr. (2018) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier.
- Pattanayak et al. (2021) [introductory]: time $O(f \cdot \min(m, k\Delta))$, memory $O(\log(k + \Delta))$. *Assumes:* rooted. First crash-tolerant dispersion on arbitrary graphs: a DFS that restarts whenever the agent carrying the traversal state crashes.
- Chand et al. (2023b) [also]: time $O(k^2)$, memory $O(\log(k + \Delta))$. *Assumes:* rooted. Independent of f. For arbitrary configurations: $O((f + \ell) \cdot \min(m, k\Delta, k^2))$ rounds with $O(\log(k + \Delta))$ bits when k, f, Δ, ℓ and m are known, and $O(k^3)$ when only k is known.

Claude (Anthropic) was used in cataloguing the problems.

- Pattanayak et al. (2023) [current best]: time $O(\min(m, k\Delta))$, memory $O(k \log(k + \Delta))$. Arbitrary configurations, no knowledge of m, Δ, ℓ, k or f, time independent of f. A second algorithm trades memory for time: $O(\min(m, k\Delta) \cdot (\ell + f))$ rounds with $O(\log(k + \Delta))$ bits.

Open questions.

Q1. Is $O(k)$ time achievable with crash faults, or does fault tolerance cost a factor of Δ? No lower bound beyond the fault-free $\Omega(k)$ is known, even with $O(\log(k + \Delta))$ memory (Chand et al. 2023b).

Q2. Simultaneously $O(\min(m, k\Delta))$ (or better) time and $O(\log(k+\Delta))$ memory for arbitrary configurations, without knowledge of f, ℓ, Δ or m (Chand et al. 2023b) (Pattanayak et al. 2023).

Q3. Crash faults under asynchrony: no result exists.

Notes. On 1-interval connected dynamic graphs with global communication, crashes cost nothing: $O(k-f)$ rounds with $\Theta(\log k)$ bits (Kshemkalyani et al. 2020a). The directed-graph algorithms of (Italiano et al. 2022) also extend to crash faults (bounds not transcribed here).

16.10.15 Crash faults on grids

Adds + grid **to** Crash faults (p. 197) VERIFIED

On oriented grids crashes are free: the fault-free $\Theta(\sqrt{n})$ bound survives any number of crashes.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: up to f agents may crash (stop forever); communication: local (co-located agents communicate); timing: synchronous rounds; topology: grid. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(\sqrt{k})$ (Kshemkalyani et al. 2020c) $\Theta(\sqrt{n})$ on oriented grids for $k = \Omega(n)$; the small-k question is the same as in the fault-free case.	$O(\sqrt{n})$ (Banerjee et al. 2025)	• tight
Memory	$\Omega(\log k)$ (Kshemkalyani et al. 2020c) $\Theta(\log n)$.	$O(\log n)$ (Banerjee et al. 2025)	• tight

Results.

- Kshemkalyani et al. (2020c) [lower bound]: time $\Omega(\sqrt{k})$. *Bottleneck:* distance. *Holds when:* k agents on one node: a grid ball of radius r holds $O(r^2)$ nodes; any algorithm. $\Omega(\sqrt{n})$ when $k = \Omega(n)$.
- Kshemkalyani et al. (2020c) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier.
- Banerjee et al. (2024) [introductory]: time $O(\sqrt{n})$, memory $O(\log n)$. *Assumes:* oriented grid; knows n. Unoriented grid with crashes: $O(\sqrt{n} \log n)$ rounds with $O(\sqrt{n} \log n)$ bits.
- Banerjee et al. (2025) [current best]: time $O(\sqrt{n})$, memory $O(\log n)$. *Assumes:* oriented grid. Any $f \leq n$, no knowledge of n; optimal in both measures for $k = \Omega(n)$.

Open questions.

Q1. Unoriented grids with crashes: close the gap between $O(\sqrt{n} \log n)$ rounds with $O(\sqrt{n} \log n)$ bits and the fault-free $O(\sqrt{n})$, $O(\log n)$ (Banerjee et al. 2024).

Claude (Anthropic) was used in cataloguing the problems.

16.10.16 Strong Byzantine agents

Adds any faults, + local communication, + synchronous VERIFIED

Byzantine agents can also fake identifiers; polynomial time is known only from a gathered start, or with a trusted shepherd agent.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** randomness: deterministic; faults: up to f strong Byzantine agents (arbitrary behaviour, can fake identifiers); node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Kshemkalyani and Ali 2019)	$O(X(n) + n^3)$ (Moses Jr. and Redlich 2024)	• polynomial gap
	Polynomial only with a gathered start or a shepherd; exponential otherwise.		
Memory	none known	—	• unknown

Results.

- Kshemkalyani and Ali (2019) [lower bound]: time $\Omega(k)$. *Bottleneck:* distance. *Holds when:* fault-free bound ($f = 0$). Fault-free bound.
- Molla et al. (2021) [introductory]: time $O(n^3)$. *Assumes:* $k = n$; knows n; $f \leq \lfloor n/4 \rfloor - 1$; gathered start. From an arbitrary configuration the time is exponential in n and f must be known.
- Moses Jr. and Redlich (2024) [current best]: time $O(X(n) + n^3)$. *Assumes:* one trusted shepherd agent; $f < \lfloor (k-1)/2 \rfloor$; shepherd knows n and k. A single never-Byzantine agent recognisable by others makes arbitrary-configuration strong-Byzantine dispersion polynomial, $O(n^3)$ if gathered; also handles nodes with capacities. Memory is unbounded in this analysis. Without a shepherd, $X(n)$ rounds for $f < k$ when the total node capacity exceeds $zf + k - f$, z the number of nodes of non-zero capacity.

Open questions.

Q1. Polynomial-time strong-Byzantine dispersion from arbitrary configurations without a trusted agent, and the largest tolerable f.

Q2. Asynchronous versions: a slow agent cannot be distinguished from a silent Byzantine one (Moses Jr. and Redlich 2024).

16.10.17 Anonymous agents with randomness

Adds anonymous, + private coins, + no faults, + local communication, + synchronous VERIFIED

Agents have no identifiers but private coins. Both axes change together: identical deterministic agents on one node can never behave differently, so randomness is needed to break symmetry.

Assumed: graph dynamics: static graph; edge direction: undirected; number of agents: $k \leq n$; randomness: randomized (agents have private coins); faults: no faults; communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** identifiers: anonymous: all agents are identical; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Statement for this setting. Agents are identical and run a randomized algorithm; deterministic algorithms are impossible for anonymous agents (Das et al. 2021b). Memory is measured without an identifier, so bounds below $\log k$ become possible. Results hold with high probability unless stated otherwise.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Kshemkalyani and Ali 2019)	$O(k^2)$ (Das et al. 2021b)	• polynomial gap
	$\Theta(k^2)$ versus $\Omega(k)$ for rooted configurations; for arbitrary configurations only the cover time of G, up to $O(nm)$, without termination.		
Memory	$\Omega(\log \Delta)$ (Molla and Moses Jr. 2019)	$O(\log \Delta)$ (Das et al. 2021b)	• tight
	$\Theta(\log \Delta)$ for rooted configurations.		

Results.

- Kshemkalyani and Ali (2019) [lower bound]: time $\Omega(k)$. *Bottleneck:* distance. *Holds when:* a path with all k agents on one endpoint; no identifiers needed; holds for randomized algorithms. The path argument does not use identifiers.
- Molla and Moses Jr. (2019) [lower bound]: memory $\Omega(\log \Delta)$. *Bottleneck:* port addressing. *Holds when:* all agents have the same memory bound; an agent with b bits can choose among at most 2^b ports; worst case over graphs: a node of degree Δ whose ports lead to the free nodes; holds for randomized algorithms. For any randomized algorithm when all agents have the same amount of memory.
- Molla and Moses Jr. (2019) [introductory]: time $O(m)$, memory $O(\max(\log \Delta, \log D))$. *Assumes:* rooted. Local leader election by coin flips replaces identifiers. Rooted graphs: $O(m)$ rounds with $O(\max(\log \Delta, \log D))$ or $O(\Delta)$ bits; rooted rings with $O(1)$ bits and rooted trees with $O(\log \Delta)$ bits in $O(n)$ rounds. Arbitrary configurations ($k = n$): a random-walk algorithm with $O(\log \Delta)$ bits disperses within the cover time of G but the agents never terminate.
- Das et al. (2021b) [current best]: time $O(k^2)$, memory $O(\log \Delta)$. *Assumes:* rooted. Memory matches the $\Omega(\log \Delta)$ lower bound; the agents terminate. The time is $\Theta(k^2)$ in the worst case for this algorithm.

Open questions.

Q1. Close the gap between $\Omega(k)$ and $\Theta(k^2)$ rounds for anonymous agents from a rooted configuration with $O(\log \Delta)$ bits (Das et al. 2021b).

Q2. Can anonymous randomized agents disperse from an **arbitrary** configuration and then stop, rather than staying active forever? (Molla and Moses Jr. 2019) (Das et al. 2021b)

Q3. On rings the memory bound is $O(1)$; is $O(1)$-bit dispersion with termination possible on rings from arbitrary configurations?

Notes. The deterministic memory bound $\Omega(\log k)$ comes entirely from storing an identifier. Removing identifiers and adding coins lowers the bound to $\Omega(\log \Delta)$, achieved for rooted configurations.

16.10.18 More agents than nodes

Adds any k, + private coins, + no faults, + local communication, + synchronous VERIFIED

With $k > n$ the target is a balanced configuration: at most $\lceil k/n \rceil$ agents per node. Only randomized anonymous results exist.

Assumed: graph dynamics: static graph; edge direction: undirected; randomness: randomized (agents have private coins); faults: no faults; communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** number of agents: any number of agents, possibly $k > n$; identifiers: anonymous: all agents are identical; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Statement for this setting. $k > n$ agents must reach a configuration in which every node hosts at most $\lceil k/n \rceil$ agents (sometimes called k-dispersion). Agents do not know n.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known	$O(m+k)$ (Molla and Moses Jr. 2019)	● unknown
	The $\Omega(k)$ path argument of (Kshemkalyani and Ali 2019) is stated for $k \le n$; no time lower bound for $k > n$ is published.		
Memory	$\Omega(\log \Delta)$ (Molla and Moses Jr. 2019)	$O(\Delta)$ (Molla and Moses Jr. 2019)	● polynomial gap
	$O(\Delta)$ or $O(\max\{\log \Delta, \log D\})$ bits on general rooted graphs against $\Omega(\log \Delta)$; tight on rooted rings and trees. Open Problem 2 of (Molla and Moses Jr. 2019).		

Results.

- Molla and Moses Jr. (2019) [lower bound]: memory $\Omega(\log \Delta)$. *Bottleneck:* port addressing. *Holds when:* all agents have the same memory bound; model assumption of (Molla and Moses Jr. 2019): an agent with b bits sees at most 2^b ports of its node, and the adversary chooses which; holds for randomized algorithms. Theorem 7 of (Molla and Moses Jr. 2019); it rests on that paper's port-visibility assumption, not on a general information argument.
- Molla and Moses Jr. (2019) [current best]: time $O(m+k)$, memory $O(\Delta)$. Introductory and current best. Rooted configurations on arbitrary graphs; the bound is a sketched modification of the rooted algorithm in Section 8, not a numbered theorem. The same paper gives $O(mk/n)$ rounds with $O(\max\{\log \Delta, \log D\})$ bits, and $O(k)$ rounds with $O(\log \Delta)$ bits on rooted rings and rooted trees.
- Molla and Moses Jr. (2019) [also]: time *cover time*, memory $O(\log \Delta)$. *Assumes:* arbitrary initial configuration; $n < k \le \Delta^\kappa n$. With high probability; this algorithm does not terminate: settled agents stay active forever.

Open questions.

Q1. **Deterministic** dispersion with $k > n$ agents with identifiers on static graphs has not been studied: no algorithm and no lower bound beyond $\Omega(k)$ is recorded in the literature surveyed here. (Related work exists only on dynamic rings and dynamic graphs under the name balanced dispersion.)

Q2. Is $O(k)$ time possible for rooted configurations when $k > n$, matching the $k \le n$ case?

Q3. Terminating dispersion with $k > n$ on general rooted graphs with $O(\log \Delta)$ bits per agent (Open Problem 2 of Molla and Moses Jr. 2019).

Notes. The balanced target, at most $\lceil k/n \rceil$ agents per node, is the formulation of (Molla and Moses Jr. 2019) itself. Many agents may cross one edge in the same round, so the $\Omega(k)$ argument for $k \le n$ does not carry over. Checked against the full text (arXiv 1902.10489).

16.10.19 Directed graphs

Adds + local communication, + synchronous VERIFIED

Edges can be traversed in one direction only, so an agent cannot backtrack; the problem is solvable only on strongly connected digraphs and the known time bounds are quadratic in k.

Assumed: graph dynamics: static graph; number of agents: $k \le n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** edge direction: directed (strongly connected); randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \le k$ nodes).

Statement for this setting. G is a directed graph with maximum out-degree Δ_{out}; at a node an agent sees only the out-ports $1, \dots, \deg_{out}(v)$. Dispersion is not always solvable when G is not strongly connected, so results assume strong connectivity. ϕ denotes the deficiency of G, the number of arcs whose addition makes G Eulerian; $f \le k$ agents may crash.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Italiano et al. 2022); $\Omega(D^2)$ (Italiano et al. 2022); $\Omega(k \cdot \min\{k, \phi\})$ (Italiano et al. 2022); *impossible* (Italiano et al. 2022)	$O(k^2\Delta_{out})$ (Italiano et al. 2022)	• polynomial gap
	$O(k^2\Delta_{out})$ versus $\Omega(k)$ in general; DFS-based algorithms need $\Omega(k \min\{k, \phi\})$, which $O(\phi k^2)$ nearly matches.		
Memory	$\Omega(\log k)$ (Italiano et al. 2022)	$\Theta(\log(k + \Delta_{out}))$ (Italiano et al. 2022)	• near-optimal
	$\Theta(\log(k + \Delta_{out}))$ is achieved; the proved lower bound is $\Omega(\log k)$.		

Results.

- Italiano et al. (2022) [lower bound]: time $\Omega(k)$. *Bottleneck:* distance. *Holds when:* Theorem 3: deterministic; a directed cycle.
- Italiano et al. (2022) [lower bound]: time $\Omega(D^2)$. *Bottleneck:* collaborative tree exploration. *Holds when:* Theorem 4: deterministic, $k = n$, $D = \omega(1)$; a strongly connected digraph built from a tree, after the exploration bound of (Disser et al. 2020).
- Italiano et al. (2022) [lower bound]: time $\Omega(k \cdot \min\{k, \phi\})$. *Bottleneck:* no backtracking. *Holds when:* Theorem 5: only for algorithms based on DFS traversal; deficiency $\phi > 0$.
- Italiano et al. (2022) [lower bound]: time *impossible*. *Bottleneck:* no backtracking. *Holds when:* Theorem 2: on some digraphs that are not strongly connected; a rooted start at a node from which some node is unreachable.
- Italiano et al. (2022) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* Theorem 6: deterministic; a rooted start; memory counts the identifier.
- Italiano et al. (2022) [current best]: time $O(k^2\Delta_{out})$, memory $\Theta(\log(k + \Delta_{out}))$. Introductory and current best (Theorem 1(b)): DFS with ranked backtracking paths of at most $k - 1$ arcs; $O(fk^2\Delta_{out})$ with f crashes. Optimal memory.
- Italiano et al. (2022) [also]: time $O(\phi k^2)$, memory $O(k \log(k+\Delta_{out}))$. Theorem 1(a), simulating Kwek's exploration; $O(f\phi k^2)$ with f crashes. Better when $\phi < \Delta_{out}$.

Open questions.

Q1. Close the time gaps; in particular $O((k + f)k\Delta_{out})$ with f crashes (Italiano et al. 2022).

Q2. BFS-based dispersion in the local model, without 1-hop communication (Italiano et al. 2022).

Q3. Dispersion on dynamic directed graphs, and in semi-synchronous and asynchronous settings (Italiano et al. 2022).

Notes. The algorithms do not terminate on their own: agents cannot tell that dispersion is complete. With k, Δ_{out}, f (and ϕ or D) known in advance, a round counter makes them terminate (Corollary 1). Checked against the full text of the JPDC version.

16.10.20 Directed graphs, 1-hop communication

Adds + h-hop communication, + synchronous VERIFIED

On strongly connected digraphs, agents that also talk to agents on neighbouring nodes disperse in $O(kD)$ rounds by breadth-first search, at the price of $O(k \log(k + \Delta_{out}))$ bits.

Assumed: graph dynamics: static graph; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: h-hop (agents within h hops communicate); timing: synchronous rounds. **Not assumed:** edge direction: directed (strongly connected); randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Claude (Anthropic) was used in cataloguing the problems.

Statement for this setting. G is a directed graph with maximum out-degree Δ_{out}; at a node an agent sees only the out-ports $1, \dots, \deg_{out}(v)$. Dispersion is not always solvable when G is not strongly connected, so results assume strong connectivity. ϕ denotes the deficiency of G, the number of arcs whose addition makes G Eulerian; $f \le k$ agents may crash. Agents communicate with co-located agents and with agents on nodes at distance 1 ($h = 1$); communication does not follow arc directions.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Italiano et al. 2022); $\Omega(D^2)$ (Italiano et al. 2022); *impossible* (Italiano et al. 2022) $O(kD)$ against $\Omega(k)$ (and $\Omega(D^2)$ for $k = n$): a factor up to D.	$O(kD)$ (Italiano et al. 2022)	• polynomial gap
Memory	$\Omega(\log k)$ (Italiano et al. 2022) $O(k \log(k + \Delta_{out}))$ against $\Omega(\log k)$: a factor k.	$O(k \log(k + \Delta_{out}))$ (Italiano et al. 2022)	• polynomial gap

Results.

- Italiano et al. (2022) [lower bound]: time $\Omega(k)$. *Bottleneck:* distance. *Holds when:* Theorem 3: deterministic; a directed cycle; stated for any deterministic algorithm, so it holds with 1-hop communication.
- Italiano et al. (2022) [lower bound]: time $\Omega(D^2)$. *Bottleneck:* collaborative tree exploration. *Holds when:* Theorem 4: deterministic, $k = n$, $D = \omega(1)$; a strongly connected digraph built from a tree, after the exploration bound of (Disser et al. 2020); stated for any deterministic algorithm, so it holds with 1-hop communication.
- Italiano et al. (2022) [lower bound]: time *impossible*. *Bottleneck:* no backtracking. *Holds when:* Theorem 2: on some digraphs that are not strongly connected; a rooted start at a node from which some node is unreachable; stated for any deterministic algorithm, so it holds with 1-hop communication.
- Italiano et al. (2022) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* Theorem 6: deterministic; a rooted start; memory counts the identifier; stated for any deterministic algorithm, so it holds with 1-hop communication.
- Italiano et al. (2022) [current best]: time $O(kD)$, memory $O(k \log(k + \Delta_{out}))$. *Assumes:* $h = 1$. Introductory and current best (Theorem 1(c)): BFS-based; in every $O(D)$ rounds at least one agent settles on an empty node. The bound also holds with $f \le k$ crashes, with no extra time.

Open questions.

Q1. A BFS-based algorithm with $O(\log(k + \Delta_{out}))$ bits per agent, as with local communication (Italiano et al. 2022).

Q2. Does $O(kD)$ improve to $O(k + D^2)$ or $O(k)$ with 1-hop communication?

Notes. The lower bounds of Theorems 2, 3, 4 and 6 of (Italiano et al. 2022) are stated for every deterministic algorithm and hold here; Theorem 5 bounds only DFS-based algorithms and does not apply to the BFS algorithm. Italiano, Pattanayak and Sharma remark that 1-hop information "is of no use in the DFS based algorithm". Checked against the full text of the JPDC version.

16.10.21 Dynamic graphs (1-interval connected)

Adds + local communication, + synchronous VERIFIED

An adversary changes the edge set every round, keeping the graph connected in every round. With only local communication the problem is impossible.

Assumed: graph dynamics: dynamic graph, connected in every round (1-interval connected); edge direction: undirected; number of agents: $k \le n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** randomness;

Claude (Anthropic) was used in cataloguing the problems.

deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Statement for this setting. The node set is fixed; in every round the adversary chooses the edge set, subject to the graph being connected in that round (1-interval connectivity). Agents see the ports of the current round. **1-neighbourhood knowledge** means an agent knows which neighbouring nodes are occupied and by whom.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Kshemkalyani et al. 2020a)	—	• impossible
	Local communication is not enough; see the global-communication branch.		
Memory	*impossible* (Kshemkalyani et al. 2020a)	—	• impossible

Results.

- Kshemkalyani et al. (2020a) [lower bound]: time *impossible. Bottleneck:* adversarial edge changes. *Holds when:* $k \geq 5$; deterministic; local communication; holds even with 1-neighbourhood knowledge and unbounded memory. For $k \geq 5$ with local communication, even with 1-neighbourhood knowledge and unbounded memory. Also impossible for $k \geq 3$ without 1-neighbourhood knowledge, even with global communication. $\Omega(k)$ rounds where solvable.
- Kshemkalyani et al. (2020a) [lower bound]: memory *impossible. Bottleneck:* adversarial edge changes. *Holds when:* $k \geq 5$; deterministic; local communication.

Open questions.

Q1. Which combination of communication range and visibility is **necessary** for dispersion on T-interval connected graphs with $T > 1$, where the impossibility proof for $T = 1$ does not apply? (Saxena and Mondal 2025)

Q2. Dynamic graphs with a bounded number of missing edges per round: (Saxena et al. 2025) gives impossibilities on 1-bounded 1-interval connected graphs for several weaker communication and visibility combinations and an $O(k)$ algorithm with global communication and 1-hop visibility; the local-communication case with few missing edges is not settled.

Notes. The impossibility is the reason the dynamic-graph literature works with global communication. Rings are the exception: with full visibility of the ring, dispersion on a dynamic ring is possible with no communication model at all (see the dynamic ring branch).

16.10.22 Dynamic rings

Adds + ring **to** Dynamic graphs (1-interval connected) (p. 203) VERIFIED

A ring from which the adversary removes at most one edge per round (or permutes the nodes); solvable without communication if agents can see the whole ring.

Assumed: graph dynamics: dynamic graph, connected in every round (1-interval connected); edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: local (co-located agents communicate); timing: synchronous rounds; topology: ring or path. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Statement for this setting. Two kinds of dynamism are studied: **1-interval connectivity** (at most one edge missing per round) and **vertex permutation** (the adversary reorders the nodes every round,

Claude (Anthropic) was used in cataloguing the problems.

keeping a ring). Visibility is a parameter: with full visibility an agent sees which nodes of the ring are empty, single or multiply occupied.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(n)$ (Mondal et al. 2025a) $O(n)$ versus $\Omega(n)$ with full visibility and chirality.	$O(n)$ (Agarwalla et al. 2018)	• tight
Memory	none known Memoryless apart from identifiers.	$O(1)$ (Agarwalla et al. 2018)	• tight

Results.

- Mondal et al. (2025a) [lower bound]: time $\Omega(n)$. *Bottleneck:* distance. *Holds when:* rooted configuration on an n-node ring, static or 1-interval connected; $k = n$. For rooted configurations on an n-node ring, static or 1-interval connected; a distance argument already in (Agarwalla et al. 2018).
- Agarwalla et al. (2018) [current best]: time $O(n)$, memory $O(1)$. *Assumes:* $k = n$; full visibility; chirality. Introductory and current best. $O(n)$ rounds under vertex permutation, under 1-interval connectivity, and under both. Without chirality: $O(n)$ for rooted configurations and for odd n, $O(1)$ for $n = 4$. The algorithms are essentially memoryless. Without any visibility the problem is impossible (vertex permutation, $n \geq 3$; 1-interval connectivity, $n \geq 2$).
- Mondal et al. (2025a) [also]: time $\Theta(n)$. *Assumes:* rooted; chirality; limited visibility. Distance-k-dispersion on a 1-interval connected ring with local communication; $k = 1$ is dispersion. Impossible under a semi-synchronous scheduler if the initial configuration has a multiplicity node. Memory not recorded here.
- Mondal et al. (2025b) [also]: time $O(\ell n)$. *Assumes:* no chirality; ℓ agents, $3 \leq \ell \leq n$. Arbitrary initial configurations without chirality; partially answers the even-ring question of (Agarwalla et al. 2018).

Open questions.

Q1. Visibility strictly between none and full: how much must an agent see of a dynamic ring for dispersion to be possible? (Agarwalla et al. 2018)

Q2. Achiral agents on even rings of size at least 6 under vertex permutation and 1-interval connectivity: is $O(n)$ time possible from arbitrary configurations? (Agarwalla et al. 2018); $O(\ell n)$ is known without vertex permutation (Mondal et al. 2025b).

16.10.23 Dynamic graphs with global communication

Adds + global communication, + synchronous VERIFIED

Global communication plus 1-neighbourhood knowledge make dispersion solvable on 1-interval connected graphs, and in optimal $\Theta(k)$ time.

Assumed: graph dynamics: dynamic graph, connected in every round (1-interval connected); edge direction: undirected; number of agents: $k \leq n$; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; faults: no faults; communication: global (any agent can read any other agent's state); timing: synchronous rounds. **Not assumed:** randomness: deterministic; node memory: no whiteboards; node identifiers: anonymous nodes; topology: arbitrary connected graph; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes).

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(k)$ (Kshemkalyani et al. 2020a) $\Theta(k)$; $\Theta(kT)$ in the T-path connected model.	$\Theta(k)$ (Kshemkalyani et al. 2020a)	• tight
Memory	$\Omega(\log k)$ (Kshemkalyani et al. 2020a) $\Theta(\log k)$.	$\Theta(\log k)$ (Kshemkalyani et al. 2020a)	• tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Kshemkalyani et al. (2020a) [lower bound]: time $\Omega(k)$. *Bottleneck:* adversarial edge changes. *Holds when:* a 1-interval connected dynamic tree with dynamic diameter $O(1)$; even with unbounded memory and global communication; $\Omega(kT)$: T-path connected graphs, even with full visibility and known k, n, T. On a 1-interval connected tree with dynamic diameter $O(1)$, even with unbounded memory. $\Omega(kT)$ on T-path connected graphs (Saxena and Mondal 2025).
- Kshemkalyani et al. (2020a) [lower bound]: memory $\Omega(\log k)$. *Bottleneck:* symmetry breaking. *Holds when:* deterministic; all agents have the same memory bound; an initial configuration with co-located agents (rooted suffices); memory counts the identifier.
- Kshemkalyani et al. (2020a) [current best]: time $\Theta(k)$, memory $\Theta(\log k)$. *Assumes:* 1-neighbourhood knowledge. Introductory and current best. With $f \leq k$ crash faults the same algorithm runs in $O(k - f)$ rounds.
- Saxena and Mondal (2025) [also]: time $\Theta(kT)$, memory $O(\log \max(k, T))$. *Assumes:* 1-hop visibility; knows T (for termination). T-path connected graphs: every pair of nodes shares a connected component at least once in every window of T rounds, so the graph may be disconnected in some rounds. Impossible on connectivity-time graphs (union of T consecutive rounds connected) for $k \geq 3$.
- Saxena et al. (2025) [also]: time $O(k)$, memory $O(\log n)$. *Assumes:* 1-hop visibility; ℓ-bounded 1-interval connected; any k, including $k > n$. **Balanced** dispersion: every node ends with $\lfloor k/n \rfloor$ or $\lceil k/n \rceil$ agents. Impossible on temporally connected graphs even with unbounded memory and full visibility.

Open questions.

Q1. Byzantine agents on dynamic graphs (Kshemkalyani et al. 2020a).

Q2. Memory $O(\log k)$ instead of $O(\log \max(k, T))$ on T-path connected graphs when T need not be known (Saxena and Mondal 2025).

16.11 Black hole search

A black hole destroys every agent that enters it, without trace; a team must locate it, losing as few agents as possible.

First posed in 2001 (Dobrev et al. 2007). Dobrev, Flocchini, Prencipe and Santoro introduced black hole search in rings (DISC 2001) and in arbitrary networks (PODC 2002) (Dobrev et al. 2006).

Problem statement and measures

A connected graph contains one **black hole**: a node that destroys any agent arriving there, leaving no trace. All other nodes are **safe**. Agents start at a safe node s (the **home base**) and must determine the location of the black hole; at least one agent must survive and know it. The primary measure is the **team size** (number of agents needed); the secondary measure is the **cost** (total number of moves, a move being one traversal of one edge) or, for synchronous agents, the time. Some papers also start the agents **scattered**, at distinct safe nodes; those results are recorded in the notes, because every catalogued variant starts at a home base.

Measures: **Agents**, number of agents needed (team size); **Cost**, total moves (asynchronous) or rounds (synchronous).

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: agents A, B at s; ring of 6

(b) B reads 'trying' on edge v1-v2: the black hole is v2

Figure 16.11: Black hole search on a ring by two agents with cautious walk (Dobrev et al. 2007) (the problem: Dobrev et al. 2006). Agents A and B start at the safe home base *s* of a ring of six nodes; one node (black) destroys every agent that enters it. Each explores half of the unexplored nodes: before crossing a new edge it marks it *trying* on the whiteboard (dashed), and after arriving it returns to mark it *safe* (green) before going on. A dies on its second new edge; B finishes its half, walks back through the explored part and finds the edge still marked *trying*: the one unexplored node beyond it is the black hole. One agent survives and knows it.

A first look

Imagine a computer network in which one computer has been taken over: every travelling program that arrives there is deleted without a trace, and nobody outside can tell. Find that computer by sending such programs, while losing as few of them as possible. We model the network as a graph: computers are **nodes**, cables between them are **edges**, and each travelling program is an **agent**. In the basic version the graph has n nodes and contains exactly one **black hole**, a node that destroys every agent that enters it; every other node is **safe**. A team of identical agents starts at a safe node s, the **home base**; they have no map, but they know n. At each node the edges leaving it are numbered $1, 2, \ldots$ (these numbers are called **ports**), each node has a **whiteboard** (a small memory on which agents can read and write notes), and the agents are **asynchronous**: every **move** (one traversal of one edge) takes a finite but unpredictable time, chosen by an **adversary** (an imagined opponent who picks the worst timing). The task: at least one agent must survive and know exactly which node is the black hole. We count the agents needed and the total number of moves.

1. Algorithm: Cautious walk and halving the unexplored ring. (Dobrev et al. 2007) (Asynchronous agents with whiteboards in a ring, p. 212) Two agents start together at s on a ring of n nodes (a cycle in which every node has two neighbours); they know n.

1. **Cautious walk** (used for every step into the unknown): an agent that crosses an edge nobody has crossed before first writes *\"I am trying this edge\"* on the whiteboard where it starts. If it arrives safely, it immediately goes back and changes the note to *\"this edge is safe\"*, and only then continues. Nobody else ever enters an edge marked *\"trying\"*.
2. The unexplored nodes form one stretch of the ring. A **phase** starts: cut it into two halves of almost equal length; one agent explores the left half, the other the right half, both by cautious walk.
3. The black hole lies in at most one half, so one agent finishes its half. It walks through the explored part to the last safe edge of the other agent.
4. What is still unexplored is again one stretch, at most about half as long. A new phase starts: the finishing agent cuts it in two again, writes the new plan on the whiteboard there for its partner (in case the partner is only slow and comes back), and takes the half on the far side, away from

Claude (Anthropic) was used in cataloguing the problems.

the edge its partner is trying.

5. When only one unexplored node remains, that node is the black hole.

Why it works and what it costs. Under cautious walk an agent that does not come back has died on the very first node beyond its last safe edge, so the black hole, if it has killed the partner, lies in the near half; the finishing agent always takes the far half and never enters an edge marked *\"trying\"*, so exactly one agent dies and the other survives. Each phase at least halves the unexplored stretch, so there are about $\log n$ phases, and each phase costs $O(n)$ moves (walking around the explored part and exploring), giving $O(n \log n)$ moves. (Dobrev et al. 2007) show the exact cost $2n \log n + O(n)$, and they prove that any algorithm, with any number of agents, needs at least $(n-1)\log(n-1) + O(n)$ moves, so this simple plan is optimal up to a factor of two.

Why it matters. Cautious walk is the basic safety tool of almost every later black hole search algorithm, in rings and in arbitrary graphs.

2. Lower bound: Every unexplored port may lead to the black hole. (Dobrev et al. 2006) (Asynchronous agents with whiteboards, p. 211) Without a map, the agents do not know where a port leads. The adversary may therefore decide, as the algorithm runs, that the **first** port tried out of a node is the one that leads to the black hole. Take a graph in which the black hole is a neighbour of many nodes, for example the black hole 0 joined to every other node, with the home base at node 1. The first agent leaving node 1 through a new port is sent into the black hole. Some other agent must then reach node 2; the first agent that leaves node 2 through a new port is again sent into the black hole, and so on, node after node. Each such node costs one agent, and the search is not finished while two or more nodes are still unexplored, because a node that nobody has returned from may be the black hole or may simply be slow to reach.

Why it holds. In this example the algorithm loses an agent at almost every node adjacent to the black hole, and the black hole can have up to Δ neighbours, where Δ is the largest number of edges at any node (the **maximum degree**). With a cleverer graph (a ring of nodes all joined to a central black hole, which the adversary later replaces by a graph of maximum degree Δ that looks the same to the agents) (Dobrev et al. 2006) prove that $\Delta + 1$ agents are needed; the full proof is longer. The same paper gives an algorithm with exactly $\Delta + 1$ agents and $O(n^2)$ moves, and shows that $\Omega(n^2)$ moves are necessary for a team of this size, so both bounds are tight.

Why it matters. It shows that what limits a team without a map is the number of ports that can lead to the black hole, which is why a map or a sense of direction lowers the team to two agents.

3. Impossibility: Without knowing n, slow and dead look the same. (Dobrev et al. 2006) (Asynchronous agents with whiteboards, p. 211) Because moves take unpredictable time, an agent that has not come back may be dead or just slow. Suppose the agents do not know n. Take a graph G with the black hole at node v, and a second graph G' in which v is replaced by two nodes v_1 and v_2 joined to each other, sharing the edges that went to v. In G' the adversary makes every edge into v_1 or v_2 extremely slow. Up to the moment the algorithm stops, the agents see exactly the same things in G and in G', so they stop in G' too, with both v_1 and v_2 unexplored: they cannot know which one is the black hole.

Why it holds. The same reasoning shows that the agents can never stop while two nodes are unexplored, and that the problem is unsolvable when the graph has a node (other than s) whose removal disconnects it: if that node is the black hole, everything behind it stays unexplored (Dobrev et al. 2006). This is why the basic version assumes that n is known and that the graph stays connected after removing any one node (it is *2-connected*).

Why it matters. The indistinguishability of slow and dead agents is the reason asynchronous black hole search needs knowledge of n, and it reappears in every asynchronous lower bound for the problem.

Claude (Anthropic) was used in cataloguing the problems.

What the community started looking at

From the start the objective was the **minimum number of agents** that guarantees success, since every agent that enters the black hole is lost; the number of moves came second (Dobrev et al. 2006). Later work replaced whiteboards by tokens (Dobrev et al. 2013) (Flocchini et al. 2012b) (Balamohan et al. 2016), turned to synchronous agents where time can be optimised (Czyzowicz et al. 2006) (Klasing et al. 2007), to dynamic graphs (Di Luna et al. 2025b), and to Byzantine companions (Di Luna et al. 2025a).

Relations to other problems

- **Variant of Collaborative exploration.** Locating the black hole means exploring every other node safely, so black hole search is exploration of a dangerous graph by a team, where agents can be lost.
- **Related: Network decontamination.** Both are security problems for agents in a hostile network: in black hole search the network destroys agents, in decontamination agents must clear a network that an arbitrarily fast intruder (or contamination) can re-enter.

Assumptions specific to this problem

Knowledge of the graph

Dobrev, Flocchini, Prencipe and Santoro organised black hole search in arbitrary networks around the agents' a priori knowledge: with topological ignorance, $\Delta + 1$ agents are necessary and sufficient and the cost is $\Theta(n^2)$; with sense of direction two agents suffice, still at cost $\Theta(n^2)$; with complete topological knowledge two agents suffice at cost $\Theta(n \log n)$ (Dobrev et al. 2006). Knowing n is the floor of the axis, because they prove that asynchronous black hole search is unsolvable when n is unknown (Dobrev et al. 2006). Sense of direction is the notion of Flocchini, Mans and Santoro: a globally consistent labelling of the ports from which one can decide, from the label sequences alone, whether two walks from a node end at the same node (Flocchini et al. 1998) (as defined in Dobrev et al. 2006); Dobrev et al. trace its effect on exploration back to Blum and Kozen's study of the power of a compass (Blum and Kozen 1978). The three levels thus separate team size (from $\Delta + 1$ down to 2) and, independently, cost (from n^2 down to $n \log n$).

Level	Value	Formal definition
0	**topology unknown**	Agents know n but not the topology of G.
1	**sense of direction**	Agents have a consistent sense of direction: a globally consistent labelling of edges from which they can tell whether two walks lead to the same node.
2	**map**	Agents have a map of G and know their starting position on it.

Marking nodes

The first black hole search papers gave the agents a whiteboard at every node (Dobrev et al. 2007) (Dobrev et al. 2006). Flocchini, Ilcinkas and Santoro observe that whiteboards let a protocol assume FIFO links and break symmetry, whereas "the theoretical quest, on the contrary, has been for the weakest interaction mechanism allowing the problem to be solved"; that weaker mechanism is identical tokens, taken from the early work on exploring safe graphs (Flocchini et al. 2012b). Dobrev, Flocchini, Královič and Santoro showed that $\Delta + 1$ agents, each with a token that may be placed on a node or a port, locate the black hole without a map at polynomial cost (Dobrev et al. 2013) (as summarised in Flocchini et al. 2012b); with a map, Flocchini, Ilcinkas and Santoro gave the first proof that one pebble per agent,

Claude (Anthropic) was used in cataloguing the problems.

on nodes only, matches the whiteboard bounds of two agents and $\Theta(n \log n)$ moves, even on non-FIFO links (Flocchini et al. 2012b). Without a map the equivalence fails for a constant number of tokens: Balamohan, Dobrev, Flocchini and Santoro show that $\Delta + 2$ agents with three tokens are necessary and sufficient, one agent more than with whiteboards (Balamohan et al. 2016).

Level	Value	Formal definition
0	**no marking**	Agents cannot leave information at nodes.
1	**tokens**	Agents carry tokens (pebbles) they can place at nodes or on ports; tokens are identical and carry no other information.
2	**whiteboards**	Every node has a whiteboard that agents read and write under mutual exclusion.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Agents, Cost) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Faults → Timing → Knowledge of the graph → Marking nodes → Topology.

No assumptions

+ static graph, + undirected

Faults

+ no faults

Timing

asynchronous, topology unknown, + whiteboards

Asynchronous agents with whiteboards STANDARD SETTING — ●● p. 211

+ ring

Asynchronous agents with whiteboards in a ring — ●● p. 212

asynchronous, + map

Marking nodes

+ whiteboards

Whiteboards and a map — ●● p. 212

+ tokens

Tokens and a map — ●● p. 213

asynchronous, + sense of direction, + whiteboards

Whiteboards and sense of direction — ●● p. 213

asynchronous, topology unknown, + tokens

Tokens, no map — ●● p. 214

+ synchronous, + map

Synchronous agents with a map — ●● p. 214

+ at most weak Byzantine faults, + synchronous

Knowledge of the graph

+ map

Byzantine companions, with a map — ●● p. 215

topology unknown, + whiteboards

Byzantine companions, whiteboards, no map — ●● p. 215

Claude (Anthropic) was used in cataloguing the problems.

+ connected every round, + undirected, + no faults, + synchronous, + tokens, + ring
Dynamic rings ●● p. 215

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Agents	Cost	p.
Asynchronous agents with whiteboards STANDARD SETTING	● tight $\Delta+1$ vs $\Delta+1$	● tight $O(n^2)$ vs $\Omega(n^2)$	211
Asynchronous agents with whiteboards in a ring	● tight 2 vs 2	● tight $2n\log n + O(n)$ vs $\Omega(n\log n)$	212
Whiteboards and a map	● tight 2 vs 2	● tight $\Theta(n\log n)$ vs —	212
Tokens and a map	● tight 2 vs 2	● tight $\Theta(n\log n)$ vs —	213
Whiteboards and sense of direction	● tight 2 vs 2	● tight $O(n^2)$ vs —	213
Tokens, no map	● tight $\Delta+2$ vs $\Delta+2$	● polynomial gap $O(mn)$ vs $\Omega(n^2/\Delta)$	214
Synchronous agents with a map	● tight 2 vs —	● polynomial gap $3\frac{3}{8}$-approx. vs *NP-hard*	214
Byzantine companions, with a map	● near-optimal $2f+2$ vs —	● unknown $O(n+f)$ vs —	215
Byzantine companions, whiteboards, no map	● near-optimal $(f+1)(\Delta+1)$ vs —	● unknown $O(m+f)$ vs —	215
Dynamic rings	● tight 3 vs 3	● tight $O(n^{1.5})$ vs $\Omega(n^{1.5})$	215

16.11.1 Asynchronous agents with whiteboards

Adds asynchronous, topology unknown, + whiteboards STANDARD SETTING VERIFIED

Without a map, $\Delta+1$ asynchronous agents are necessary and sufficient, and they need $\Theta(n^2)$ moves.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: no faults; marking nodes: whiteboards at the nodes. **Not assumed:** timing: asynchronous; knowledge of the graph: no knowledge of the topology (n is known); topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Agents	$\Delta+1$ (Dobrev et al. 2006); *impossible* (Dobrev et al. 2006) $\Delta+1$.	$\Delta+1$ (Dobrev et al. 2006)	● tight
Cost	$\Omega(n^2)$ (Dobrev et al. 2006) $\Theta(n^2)$ moves.	$O(n^2)$ (Dobrev et al. 2006)	● tight

Results.

- Dobrev et al. (2006) [lower bound]: agents $\Delta+1$. *Bottleneck:* agents lost in the black hole. *Holds when:* generic algorithms without topological knowledge; even if Δ and n are known.

Claude (Anthropic) was used in cataloguing the problems.

- Dobrev et al. (2006) [lower bound]: cost $\Omega(n^2)$. *Bottleneck:* asynchrony. *Holds when:* size-optimal generic solutions.
- Dobrev et al. (2006) [lower bound]: agents *impossible*. *Bottleneck:* asynchrony. *Holds when:* if n is unknown, or the graph is not 2-connected.
- Dobrev et al. (2006) [current best]: agents $\Delta + 1$, cost $O(n^2)$. Introductory and current best.

Notes. The anonymous ring was treated first (Dobrev et al. 2007); see the variant for rings.

16.11.2 Asynchronous agents with whiteboards in a ring

Adds + ring **to** Asynchronous agents with whiteboards (p. 211) VERIFIED

In an anonymous ring of known size, two co-located asynchronous agents are necessary and sufficient, at optimal cost $\Theta(n \log n)$.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: no faults; marking nodes: whiteboards at the nodes; topology: ring or path. **Not assumed:** timing: asynchronous; knowledge of the graph: no knowledge of the topology (n is known).

Measure	Lower bound	Best upper bound	Status
Agents	2 (Dobrev et al. 2007); *impossible* (Dobrev et al. 2007) Two.	2 (Dobrev et al. 2007)	• tight
Cost	$\Omega(n \log n)$ (Dobrev et al. 2007) $\Theta(n \log n)$ moves; the constants differ by a factor of two.	$2n \log n + O(n)$ (Dobrev et al. 2007)	• tight

Results.

- Dobrev et al. (2007) [lower bound]: agents 2. *Bottleneck:* agents lost in the black hole. *Holds when:* at least one agent enters the black hole.
- Dobrev et al. (2007) [lower bound]: cost $\Omega(n \log n)$. *Bottleneck:* asynchrony. *Holds when:* Theorem 1: $(n-1)\log(n-1) + O(n)$ moves, whatever the number of agents; an adversarial scheduler delays moves along edges; holds also for dispersed agents in oriented rings (Theorem 8).
- Dobrev et al. (2007) [lower bound]: agents *impossible*. *Bottleneck:* asynchrony. *Holds when:* if n is unknown: no deterministic protocol always terminates correctly.
- Dobrev et al. (2007) [current best]: agents 2, cost $2n \log n + O(n)$. *Assumes:* co-located agents; n known; $O(\log n)$-bit whiteboards. Introductory and current best (Algorithm DIVIDE, Theorems 2 and 3), size-optimal and cost-optimal within a factor of two; works in unoriented rings, since co-located agents agree on a direction through the whiteboard.

Notes. Checked against the full text of (Dobrev et al. 2007). Agents are anonymous and asynchronous, and the problem is solved when at least one agent survives and all survivors know the location of the black hole. In ideal time (every move one time unit) $2n - 4$ units are necessary whatever the number of agents (Theorem 4) and $n - 1$ co-located agents achieve it (Theorems 5 and 6); $n^{1/q} + 1$ agents need $2(q+1)n - q - o(n)$ time for $1 \le q \le \log n$ (Theorem 7). With **dispersed** agents the cost is $\Omega(n \log n)$, or $\Omega(n \log(n-k))$ when the number k of agents is known (Theorems 8 and 9); in an oriented ring two dispersed agents are necessary and sufficient at cost $O(n \log n)$ (Theorem 10), and in an unoriented ring three are necessary and sufficient at the same cost (Theorem 13).

16.11.3 Whiteboards and a map

Adds + whiteboards VERIFIED

With a map, two agents locate the black hole in $\Theta(n \log n)$ moves.

Claude (Anthropic) was used in cataloguing the problems.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: no faults; knowledge of the graph: a map of the graph; marking nodes: whiteboards at the nodes. **Not assumed:** timing: asynchronous; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Agents	2 (Dobrev et al. 2006) Two agents.	2 (Dobrev et al. 2006)	• tight
Cost	none known $\Theta(n \log n)$.	$\Theta(n \log n)$ (Dobrev et al. 2006)	• tight

Results.

- Dobrev et al. (2006) [lower bound]: agents 2. *Bottleneck:* agents lost in the black hole. *Holds when:* a single agent never suffices.
- Dobrev et al. (2006) [current best]: agents 2, cost $\Theta(n \log n)$. Introductory and current best.

16.11.4 Tokens and a map

Adds + tokens VERIFIED

Whiteboards can be replaced by one pebble per agent: two agents, $\Theta(n \log n)$ moves, even on non-FIFO edges.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: no faults; knowledge of the graph: a map of the graph; marking nodes: tokens (pebbles) that agents carry and drop. **Not assumed:** timing: asynchronous; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Agents	2 (Dobrev et al. 2006) Two agents.	2 (Flocchini et al. 2012b)	• tight
Cost	none known Same as with whiteboards.	$\Theta(n \log n)$ (Flocchini et al. 2012b)	• tight

Results.

- Dobrev et al. (2006) [lower bound]: agents 2. *Bottleneck:* agents lost in the black hole. *Holds when:* a single agent never suffices.
- Flocchini et al. (2012b) [current best]: agents 2, cost $\Theta(n \log n)$. *Assumes:* one identical pebble per agent. Introductory and current best.

16.11.5 Whiteboards and sense of direction

Adds asynchronous, + sense of direction, + whiteboards VERIFIED

With sense of direction two agents suffice, still with $\Theta(n^2)$ moves.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: no faults; knowledge of the graph: sense of direction; marking nodes: whiteboards at the nodes. **Not assumed:** timing: asynchronous; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Agents	2 (Dobrev et al. 2006) Two agents.	2 (Dobrev et al. 2006)	• tight
Cost	none known $\Theta(n^2)$.	$O(n^2)$ (Dobrev et al. 2006)	• tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Dobrev et al. (2006) [lower bound]: agents 2. *Bottleneck:* agents lost in the black hole. *Holds when:* a single agent never suffices.
- Dobrev et al. (2006) [current best]: agents 2, cost $O(n^2)$. Introductory and current best.

16.11.6 Tokens, no map

Adds asynchronous, topology unknown, + tokens VERIFIED

With a constant number of tokens and no map, $\Delta + 2$ agents and 3 tokens are necessary and sufficient.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: no faults; marking nodes: tokens (pebbles) that agents carry and drop. **Not assumed:** timing: asynchronous; knowledge of the graph: no knowledge of the topology (n is known); topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Agents	$\Delta + 2$ (Balamohan et al. 2016) $\Delta + 2$ with $O(1)$ tokens.	$\Delta + 2$ (Balamohan et al. 2016)	• tight
Cost	$\Omega(n^2/\Delta)$ (Balamohan et al. 2016) $O(mn)$ versus $\Omega(n^2/\Delta)$; move-optimal in sparse graphs.	$O(mn)$ (Balamohan et al. 2016)	• polynomial gap

Results.

- Balamohan et al. (2016) [lower bound]: agents $\Delta + 2$. *Bottleneck:* agents lost in the black hole. *Holds when:* $O(1)$ tokens; $\Delta + 1$ agents are not enough; 2 tokens never suffice.
- Balamohan et al. (2016) [lower bound]: cost $\Omega(n^2/\Delta)$. *Bottleneck:* asynchrony. *Holds when:* teams of $(2 - \varepsilon)\Delta$ agents.
- Dobrev et al. (2013) [introductory]: agents $\Delta + 1$. *Assumes:* enhanced tokens. First token-based exploration of an unknown dangerous graph; polynomial cost.
- Balamohan et al. (2016) [current best]: agents $\Delta + 2$, cost $O(mn)$. *Assumes:* 3 tokens; the number of agents is known (5 tokens otherwise).

16.11.7 Synchronous agents with a map

Adds + synchronous, + map VERIFIED

With synchrony, two agents and a map, the question becomes the fastest exploration schedule, which is NP-hard to find.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: no faults; timing: synchronous rounds; knowledge of the graph: a map of the graph. **Not assumed:** marking nodes: no marking; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Agents	none known Two agents.	2 (Klasing et al. 2007)	• tight
Cost	*NP-hard* (Klasing et al. 2007) Approximation ratio $3\frac{3}{8}$ versus inapproximability 389/388.	$3\frac{3}{8}$-approx. (Klasing et al. 2007)	• polynomial gap

Results.

- Klasing et al. (2007) [lower bound]: cost *NP-hard*. *Bottleneck:* combinatorial hardness. *Holds when:* computing an optimal schedule for two agents; no 389/388-approximation unless P = NP (APX-hard).
- Czyzowicz et al. (2006) [introductory]: agents 2, cost 9.3-approx..

Claude (Anthropic) was used in cataloguing the problems.

- Klasing et al. (2007) [current best]: agents 2, cost $3\frac{3}{8}$-approx..

Notes. On trees the problem has a 5/3-approximation (CPC 2007, not yet in the bibliography).

16.11.8 Byzantine companions, with a map

Adds + map VERIFIED

If up to f agents are Byzantine (they collude with the black hole and survive it), $2f + 2$ agents suffice with a map.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: up to f weak Byzantine agents (arbitrary behaviour, cannot fake identifiers); timing: synchronous rounds; knowledge of the graph: a map of the graph. **Not assumed:** marking nodes: no marking; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Agents	none known Asymptotically optimal.	$2f + 2$ (Di Luna et al. 2025a)	• near-optimal
Cost	none known No time lower bound recorded.	$O(n + f)$ (Di Luna et al. 2025a)	• unknown

Results.

- Di Luna et al. (2025a) [current best]: agents $2f + 2$, cost $O(n + f)$. Introductory and current best; also with face-to-face communication. The agent counts are asymptotically optimal.

Notes. The Byzantine agents are weakly Byzantine: they cannot forge identities (confirmed by the catalogue owner, a co-author).

16.11.9 Byzantine companions, whiteboards, no map

Adds topology unknown, + whiteboards VERIFIED

Without a map but with whiteboards, $(f + 1)(\Delta + 1)$ agents locate the black hole in $O(m + f)$ rounds.

Assumed: graph dynamics: static graph; edge direction: undirected; faults: up to f weak Byzantine agents (arbitrary behaviour, cannot fake identifiers); timing: synchronous rounds; marking nodes: whiteboards at the nodes. **Not assumed:** knowledge of the graph: no knowledge of the topology (n is known); topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Agents	none known Asymptotically optimal.	$(f + 1)(\Delta + 1)$ (Di Luna et al. 2025a)	• near-optimal
Cost	none known No time lower bound recorded.	$O(m + f)$ (Di Luna et al. 2025a)	• unknown

Results.

- Di Luna et al. (2025a) [current best]: agents $(f + 1)(\Delta + 1)$, cost $O(m + f)$. With face-to-face communication instead: $(f + 1)(\Delta + 1) + 3f + 1$ agents, $O(mn + f)$ rounds.

16.11.10 Dynamic rings

Adds + connected every round, + undirected, + no faults, + synchronous, + tokens, + ring VERIFIED

In a 1-interval-connected ring three agents are necessary and sufficient; with pebbles they need $\Theta(n^{1.5})$

Claude (Anthropic) was used in cataloguing the problems.

moves.

Assumed: graph dynamics: dynamic graph, connected in every round (1-interval connected); edge direction: undirected; faults: no faults; timing: synchronous rounds; marking nodes: tokens (pebbles) that agents carry and drop; topology: ring or path. **Not assumed:** knowledge of the graph: no knowledge of the topology (n is known).

Measure	Lower bound	Best upper bound	Status
Agents	3 (Di Luna et al. 2025b) Three.	3 (Di Luna et al. 2025b)	• tight
Cost	$\Omega(n^{1.5})$ (Di Luna et al. 2025b) $\Theta(n^{1.5})$.	$O(n^{1.5})$ (Di Luna et al. 2025b)	• tight

Results.

- Di Luna et al. (2025b) [lower bound]: agents 3. *Bottleneck:* adversarial edge changes. *Holds when:* fewer agents fail even when co-located with whiteboards and identifiers; n must be known.
- Di Luna et al. (2025b) [lower bound]: cost $\Omega(n^{1.5})$. *Bottleneck:* adversarial edge changes. *Holds when:* co-located agents, whiteboards.
- Di Luna et al. (2025b) [current best]: agents 3, cost $O(n^{1.5})$. *Assumes:* co-located start, pebbles. Introductory and current best; with face-to-face communication or vision $\Theta(n^2)$.

Notes. The ring is oriented and each agent carries one pebble; n must be known (Di Luna et al. 2025b). With **scattered** agents (distinct arbitrary starting nodes) the cost rises to $\Theta(n^2)$ with pebbles, and with only face-to-face communication or vision scattered agents cannot solve the problem (Di Luna et al. 2025b). For arbitrary dynamic graphs in which at most one edge is missing per round, six co-located agents with $O(\log n)$-bit whiteboards suffice in $O(m^2)$ rounds, while 2 $\deg(v_{BH})$ agents from an arbitrary configuration cannot (Kaur et al. 2026a); $\deg(v_{BH})$ is the degree of the black hole.

16.12 Exploration of dynamic graphs by teams

An adversary changes the graph over time; how many agents are needed to explore it?

First posed in 2016 (Di Luna et al. 2020a). Live exploration of dynamic rings was posed at ICDCS 2016.

Problem statement and measures

Agents must visit every node of a **dynamic graph**: a sequence of graphs $G_t = (V, E_t)$ on one node set, whose edges are removed round by round by an adversary subject to a connectivity constraint (**1-interval connectivity**: every G_t is connected; **temporal connectivity**: every node can reach every other through edges present at non-decreasing times). The primary measure is the **number of agents**; the secondary is time in rounds.

Measures: **Agents**, number of agents needed; **Time**, rounds.

A first look

Imagine guards who must check every station on a circular railway, while the operator closes some piece of track at any moment without warning. We model the railway as a graph: stations are **nodes**, the pieces of track between neighbouring stations are **edges**, and each guard is an **agent**. In the basic version the graph is a **ring** of n nodes (the nodes form one cycle, each joined by an edge to its two

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: a and b on one node; a goes clockwise, b the other way

(b) After N - 1 = 7 rounds every node is visited; both stop

Figure 16.12: Live exploration of a dynamic ring (Di Luna et al. 2020a): in every round an adversary removes at most one edge of the ring (1-interval connectivity; the missing edge is dashed red). Two agents start on one node, know $N = n = 8$ and walk in opposite directions, waiting while their next edge is missing; the adversary can block only one of them, so every round visits a new node, and after $N - 1$ rounds both stop. One agent alone can be blocked forever.

neighbours). Time runs in **rounds**: in each round every agent may cross one edge or wait. An **adversary** (an imagined opponent, the operator of the picture) chooses, in every round, which edges are **missing** (removed for that round), with one rule: the graph must stay connected in every round (**1-interval connectivity**); in a ring this means that at most one edge is missing per round. A graph that changes like this is a **dynamic graph**. The agents must together visit every node; in the version recorded here they also know an upper bound $N \geq n$ and must stop. We count the agents needed and the rounds.

1. Algorithm: Two agents walking in opposite directions. (Di Luna et al. 2020a) (1-interval-connected rings, p. 218) The simplest case: two agents start on the same node and know $N \geq n$.

1. The two agents set off in opposite directions around the ring (the paper shows how two identical agents can agree on this).
2. In every round each agent tries to cross the next edge in its direction; if the edge is missing, it waits (it is **blocked**) and tries again in the next round.
3. After $N - 1$ rounds both agents stop.

Why it works and what it costs. At most one edge is missing per round, and until the two agents have between them covered the whole ring they are trying to cross two different edges, so at least one of them moves forward, onto a node that nobody has visited. Each round therefore visits at least one new node, and the $n - 1$ other nodes are all visited within $n - 1 \leq N - 1$ rounds. (Di Luna et al. 2020a) handle the harder cases (different starting nodes, no common sense of left and right) with one extra rule, turning back after being blocked for long, and both agents stop after $3N - 6$ rounds.

Why it matters. It shows that a second agent turns an impossible task into an easy one, because the adversary can only block one of them at a time.

2. Impossibility: One agent can always be blocked. (Di Luna et al. 2020a) (1-interval-connected rings, p. 218) A single agent stands on a node of the ring and, in some round, decides to cross one of the two edges at its node. The adversary removes exactly that edge in that round. Removing one edge from a ring leaves a path, which is still connected, so this is allowed. The adversary can do this in every round, so the agent never leaves its starting node.

Why it holds. So at least two agents are needed (Di Luna et al. 2020a). The same paper also shows that the

Claude (Anthropic) was used in cataloguing the problems.

adversary can prevent two agents that start on different nodes from ever meeting, by removing an edge only at the moment the two would otherwise meet: the agents cannot rely on meeting to coordinate.

Why it matters. The number of agents becomes the main measure on dynamic graphs, because a few agents may be unable to explore at all, whatever time they are given.

What the community started looking at

In dynamic graphs the community first asked **how many agents** make exploration possible at all, under each connectivity assumption (Di Luna et al. 2020a) (Gotoh et al. 2021) (Saxena and Mondal 2026); time came second.

Relations to other problems

- **Variant of Collaborative exploration.** When an adversary removes edges, a single agent can be blocked forever; the question shifts from speed to **how many agents** are needed to visit every node.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Agents, Time) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Timing → Topology.

No assumptions

+ connected every round, + undirected, + synchronous, + ring
1-interval-connected rings STANDARD SETTING ●● p. 218

any dynamics, + undirected, + synchronous
Temporally connected graphs, arbitrary topology ●● p. 219

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Agents	Time	p.
1-interval-connected rings STANDARD SETTING	● tight 3 vs 2	● unknown $3N - 6$ vs —	218
Temporally connected graphs, arbitrary topology	● tight $2\eta + 1$ vs $2\eta + 1$	● unknown — vs —	219

16.12.1 1-interval-connected rings

Adds + connected every round, + undirected, + synchronous, + ring STANDARD SETTING VERIFIED

One agent cannot explore a dynamic ring; two agents with a known bound on n explore in $3N - 6$ rounds.

Assumed: graph dynamics: dynamic graph, connected in every round (1-interval connected); edge direction: undirected; timing: synchronous rounds; topology: ring or path. **Not assumed:** —.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Agents	2 (Di Luna et al. 2020a) Two (with a bound on n).	3 (Das et al. 2022)	• tight
Time	none known $O(n \log n)$ with a landmark but no chirality; whether that is necessary is open.	$3N - 6$ (Di Luna et al. 2020a)	• unknown

Results.

- Di Luna et al. (2020a) [lower bound]: agents 2. *Bottleneck:* adversarial edge changes. *Holds when:* one agent cannot explore; anonymous agents without a landmark and with n unknown cannot explore with termination at all.
- Di Luna et al. (2020a) [introductory]: agents 2, time $3N - 6$. *Assumes:* a known bound $N \geq n$.
- Mandal et al. (2023) [also]: agents 3. *Assumes:* identifiers, no landmark. Explicit termination with three agents.
- Das et al. (2022) [also]: agents 3. *Assumes:* identifiers, no landmark, no edge-crossing detection.

Open questions.

Q1. With a landmark but no chirality, is $O(n \log n)$ time necessary? (Di Luna et al. 2020a)

16.12.2 Temporally connected graphs, arbitrary topology

Adds any dynamics, + undirected, + synchronous VERIFIED

Perpetual exploration of a temporally connected graph with η transient edges needs exactly $2\eta + 1$ agents.

Assumed: edge direction: undirected; timing: synchronous rounds. **Not assumed:** graph dynamics: time-varying graph, only temporally connected; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Agents	$2\eta + 1$ (Gotoh et al. 2021) $2\eta + 1$.	$2\eta + 1$ (Gotoh et al. 2021)	• tight
Time	none known —	—	• unknown

Results.

- Gotoh et al. (2021) [lower bound]: agents $2\eta + 1$. *Bottleneck:* adversarial edge changes. *Holds when:* 2η agents fail even with identifiers, knowledge of n and k, and unbounded whiteboards.
- Gotoh et al. (2021) [current best]: agents $2\eta + 1$. Introductory and current best; whiteboards of $O(\log \deg(v))$ bits at node v.

16.13 Network decontamination

A team of agents must clean a contaminated network, or equivalently capture an invisible, arbitrarily fast intruder, using as few agents as possible.

First posed in 2002 (Barrière et al. 2002). Capture of an intruder by mobile agents (SPAA 2002).

Claude (Anthropic) was used in cataloguing the problems.

Problem statement and measures

All nodes are initially **contaminated**. A node is **cleaned** when an agent visits it and is **recontaminated** when it has a contaminated neighbour and no agent **guards** it (stands on it). Agents move along edges from a **home base** s, the node where they all start; the clean part must stay connected. **Agents** is the number needed (the primary measure); **moves** (edge traversals) and time are secondary.

Measures: **Agents**, number of agents needed; **Moves**, total moves (or time, as stated).



(a) Start: all nodes contaminated; agents 1, 2 at s

(b) Last branch: the guard joins; every node is clean with 2 agents

Figure 16.13: Decontamination of a tree (Barrière et al. 2002) with the rule "guard the node, clean its branches one by one": every node is contaminated (red) and agents start at the home base s; a node is clean (green) once visited and would be recontaminated if left unguarded next to a red node. One agent guards s while the other cleans the smaller branch and comes back; then the guard is free, both enter the other branch, and the same rule is applied at a. Nothing is ever recontaminated, the clean part stays connected, and two agents suffice.

A first look

Imagine a network of pipes filled with a contaminating gas: cleaners must clean every junction, but gas flows back into any clean junction that is left unguarded next to one still full of gas. We model the network as a graph: junctions are **nodes**, pipes are **edges**, and each cleaner is an **agent**. At the start every node is **contaminated**. An agent makes a node **clean** by standing on it; a clean node with no agent on it becomes contaminated again as soon as one of its neighbours is contaminated (**recontamination**). An agent that stays on a clean node to prevent this is a **guard**. All agents start at one node s, the **home base**, and **move** along edges (one move is one traversal of one edge), and the clean part must always be connected. In the basic version the graph is a **tree** (a graph without cycles) whose map the agents know. The task: clean every node; we count the agents needed first, and moves and time second.

1. Algorithm: Guard the node, clean its branches one by one. (Trees, p. 223) A standard approach on a tree, rooted at the home base s. Write $a(v)$ for the number of agents this method uses to clean the **branch** of node v: v together with all nodes below it.

1. A node with no children (a **leaf**) needs one agent: $a(v) = 1$.
2. At a node v, sort the branches of its children so that the first one needs the most agents.
3. Keep one agent on v as a guard. Clean the other branches one after another, each with its own team; when a branch is clean, its team comes back to v to clean the next one.
4. Finally the guard is no longer needed at v (all its other neighbours are clean), so it joins the team for the first, largest branch.

Claude (Anthropic) was used in cataloguing the problems.

5. This gives $a(v) = \max\big(a(c_1),\ a(c_2)+1\big)$, where c_1 and c_2 are the children with the largest and second-largest needs.

Why it works and what it costs. No clean node is ever left unguarded next to a contaminated one, so nothing gets recontaminated, and the clean part always contains s and stays connected. A path needs one agent; a star centred at s needs two; a complete binary tree of height h needs $h+1$ with this rule, so about $\log n$ agents for n nodes. (Barrière et al. 2002) compute the smallest possible team for any tree, and a strategy for it, in linear time; this primer does not claim that the simple rule above always matches it.

Why it matters. The idea of a guard that waits at a node until its last branch is clean underlies decontamination strategies on trees and on many structured networks.

2. Lower bound: One agent cannot clean a cycle. (Unknown graphs, monotone strategies, p. 223) A standard argument. Let the graph be a ring and let one agent start at s. As soon as the agent steps to a neighbour, s is unguarded, and its other neighbour is still contaminated, so s becomes contaminated again. The agent can never have more than its current node clean.

Why it holds. So any graph containing a cycle needs at least two agents: one guards while the other advances. More generally, the team must be large enough to guard every clean node that touches a contaminated node at every moment, which is why the answer depends on the shape of the graph (for the hypercube, $\Theta(n/\sqrt{\log n})$ agents for n nodes Flocchini et al. 2008c).

Why it matters. It shows the basic counting idea of the problem: count the boundary between clean and contaminated parts that must be guarded.

3. Lower bound: The adversary grows the tree as you clean it. (Ilcinkas et al. 2009) (Unknown graphs, monotone strategies, p. 223) Now the agents do not know the graph, and they must never let a clean part become contaminated again (the strategy is **monotone**). An **adversary** (an imagined opponent) builds the graph while the agents clean it (Ilcinkas et al. 2009). Whenever an agent crosses an edge into the unknown, the adversary attaches at the far end a new node with two more unexplored edges. Every clean node that still has an unexplored edge must be guarded, because the edge may lead to contamination. When the graph has reached n nodes, the adversary stops adding edges: the result is a tree in which each node has at most three edges.

Why it holds. At the end, every parent of a leaf in this tree has had to be guarded, and there are at least about $n/4$ such nodes, so the protocol used at least about $n/4$ agents. A team that knew the tree in advance could clean it with only $O(\log n)$ agents. So not knowing the graph costs a factor $\Omega(n/\log n)$ in team size, and (Ilcinkas et al. 2009) give an algorithm that achieves this factor, even for asynchronous agents.

Why it matters. It is a clean example of a *competitive* lower bound: comparing a distributed team without a map to the best team with a map.

What the community started looking at

From the start the objective was the **minimum number of agents** (Barrière et al. 2002), first on special topologies (Flocchini et al. 2008c) and then on unknown graphs, where the question becomes the competitive ratio against the optimal team (Ilcinkas et al. 2009).

Relations to other problems

- **Related: Black hole search.** Both are security problems for agents in a hostile network: in black hole search the network destroys agents, in decontamination agents must clear a network that an

Claude (Anthropic) was used in cataloguing the problems.

arbitrarily fast intruder (or contamination) can re-enter.

Assumptions specific to this problem

Knowledge

Barrière, Flocchini, Fraigniaud and Santoro introduced contiguous search by mobile agents, in which agents cannot jump and the cleared part must stay connected; they solved it on trees with a linear-time algorithm that computes the minimum team and its strategy, and stressed that the algorithm is naturally distributed, so a tree network can compute its own strategy (Barrière et al. 2002) (abstract read). Later work on specific topologies also builds the strategy from the known structure; for hypercubes, Flocchini, Huang and Luccio use a coordinator that leads the other agents (Flocchini et al. 2008c) (abstract read). The unknown level asks what is lost when searchers must clear a graph they discover as they go. Blin, Fraigniaud, Nisse and Vial cleared any unknown graph with one searcher more than the optimum, but not monotonically and possibly in exponentially many steps (Blin et al. 2008) (as reported by Ilcinkas et al. 2009); Ilcinkas, Nisse and Soguet showed that if clearing must also be monotone, a protocol with no a priori information about the graph has competitive ratio $\Theta(n/\log n)$ in the number of searchers, the lower bound holding even for synchronous searchers on trees of maximum degree 3 (Ilcinkas et al. 2009).

Level	Value	Formal definition
0	**unknown graph**	Agents know nothing about G.
1	**known graph**	The topology is known to the agents (or to a coordinator).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Agents, Moves) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Timing → Knowledge → Topology.

No assumptions

+ static graph, + undirected
Unknown graphs, monotone strategies ●● p. 223

+ known graph
Topology

+ tree
Trees STANDARD SETTING ●● p. 223

+ hypercube
Hypercubes ●● p. 224

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Claude (Anthropic) was used in cataloguing the problems.

Setting	Agents	Moves	p.
Unknown graphs, monotone strategies	• tight $O(n/\log n)\cdot$ opt vs $\Omega(n/\log n)\cdot$ opt	• unknown — vs —	223
Trees STANDARD SETTING	• tight *optimal* vs —	• unknown — vs —	223
Hypercubes	• tight $O(n/\sqrt{\log n})$ vs $\Omega(n/\sqrt{\log n})$	• unknown $O(n\log n)$ vs —	224

16.13.1 Unknown graphs, monotone strategies

Adds + static graph, + undirected VERIFIED

Clearing an unknown graph monotonically costs a factor $\Theta(n/\log n)$ in agents over the optimum.

Assumed: graph dynamics: static graph; edge direction: undirected. **Not assumed:** timing: asynchronous; knowledge: the topology is unknown; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Agents	$\Omega(n/\log n)\cdot$ opt (Ilcinkas et al. 2009) Competitive ratio $\Theta(n/\log n)$.	$O(n/\log n)\cdot$ opt (Ilcinkas et al. 2009)	• tight
Moves	none known —	—	• unknown

Results.

- Ilcinkas et al. (2009) [lower bound]: agents $\Omega(n/\log n)\cdot$opt. *Bottleneck:* unknown topology. *Holds when:* monotone connected strategies; holds even for synchronous agents.
- Ilcinkas et al. (2009) [current best]: agents $O(n/\log n)\cdot$ opt. Achievable even asynchronously.

16.13.2 Trees

Adds + tree **to** Unknown graphs, monotone strategies (p. 223) STANDARD SETTING VERIFIED

On trees the minimum team and a strategy are computed in linear time.

Assumed: graph dynamics: static graph; edge direction: undirected; knowledge: the topology is known; topology: tree. **Not assumed:** timing: asynchronous.

Measure	Lower bound	Best upper bound	Status
Agents	none known Optimal team size.	*optimal* (Barrière et al. 2002)	• tight
Moves	none known —	—	• unknown

Results.

- Barrière et al. (2002) [current best]: agents *optimal.* Introductory and current best; also a decentralised version with a linear number of messages.

Claude (Anthropic) was used in cataloguing the problems.

16.13.3 Hypercubes

Adds + hypercube **to** Unknown graphs, monotone strategies (p. 223) VERIFIED

A hypercube needs $\Theta(n/\sqrt{\log n})$ agents, matching its pathwidth.

Assumed: graph dynamics: static graph; edge direction: undirected; knowledge: the topology is known; topology: hypercube.
Not assumed: timing: asynchronous.

Measure	Lower bound	Best upper bound	Status
Agents	$\Omega(n/\sqrt{\log n})$ (Flocchini et al. 2008c) $\Theta(n/\sqrt{\log n})$.	$O(n/\sqrt{\log n})$ (Flocchini et al. 2008c)	• tight
Moves	none known —	$O(n \log n)$ (Flocchini et al. 2008c)	• unknown

Results.

- Flocchini et al. (2008c) [lower bound]: agents $\Omega(n/\sqrt{\log n})$. *Bottleneck:* pathwidth. *Holds when:* connected monotone strategies.
- Flocchini et al. (2008c) [current best]: agents $O(n/\sqrt{\log n})$, moves $O(n \log n)$. Introductory and current best (with a coordinator); with visibility, $n/2$ agents in $O(\log n)$ time.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 17

Computing with agents

Agents solve graph problems on the anonymous graph they move in: leader election, independent sets, dominating sets, spanning trees, subgraph counts. Most algorithms first disperse the agents, so that every node has an agent to compute for it.
This family is related to Dispersion: Most algorithms first bring the n agents into a dispersed configuration, one agent per node, so that every node has an agent to compute for it (Kshemkalyani et al. 2026) (Chand and Molla 2026).

Problems in this family:

- Leader election by agents
- Maximal independent set by agents
- Minimal dominating set by agents
- Triangle and butterfly counting by agents
- Minimum spanning tree by agents
- BFS tree by agents

17.1 Leader election by agents

n agents in an anonymous graph must elect one leader; how fast, and with how little memory?

First posed in 2024 (Kshemkalyani et al. 2024). Leader election in the agent-based model, with MST as an application (DISC 2024).

Problem statement and measures

n agents with distinct identifiers (IDs) are placed on the nodes of an anonymous port-labelled graph with n nodes, m edges and maximum degree Δ. The start is rooted (all agents on one node), dispersed (one agent on each node) or arbitrary. Agents compute only by moving and by talking with agents on the same node. The agents must elect exactly one leader: every agent (or node) learns its part of the output. **Time** is in synchronous rounds; **memory** in bits per agent.

Measures: **Time**, rounds; **Memory**, bits per agent.

A first look

Picture n **agents** (small mobile computers) dropped into a building with n rooms, where the rooms carry no names and an agent can only talk to agents standing in the same room. The building is a **graph**: rooms are its **nodes** and corridors are its **edges**; there are n nodes and m edges, and from here on we



(a) Start: dispersed; IDs written in 2 bits

(b) Final: after n−1 = 2 simulated rounds, 1 leads

Figure 17.1: Leader election by simulating message passing (Kshemkalyani et al. 2024): the agents, already dispersed one per node of a path, write their IDs in 2 bits. In block i (of $2\Delta = 4$ rounds) an agent whose i-th bit is 1 visits each neighbour and comes back, one edge per round, while agents with bit 0 wait at home (the ID-bit schedule); every visit to a waiting neighbour (blue edge) carries the messages of one simulated round, here the smallest ID seen ("min"). Both simulated rounds are shown round by round; after $n - 1 = 2$ of them every agent knows that agent 1 is the leader.

say only node and edge. Two nodes joined by an edge are *neighbours*. The **degree** $\deg(v)$ of a node v is the number of edges at it, and Δ is the maximum degree. Nodes have no names, but the edges at a node v are numbered $1, 2, \dots, \deg(v)$; these numbers are **ports**, so an agent can say "leave by port 3". Every agent carries a distinct **identifier** (an ID number). Time runs in **rounds**: in one round every agent may talk to the agents on its node and then move along at most one edge. The task is to pick exactly one agent as **leader**: at the end the leader knows it is the leader and every other agent knows it is not, and we count the rounds and the bits of **memory** each agent needs. If all agents start on one node the task is easy (the smallest ID wins), so the interesting case is when they start with one agent on each node (a **dispersed** start), or in a few clusters.

1. Algorithm: Disperse, then simulate message passing by visiting neighbours. (Kshemkalyani et al. 2024) (Knowing n and Δ, p. 229) This is the idea behind every problem in this family. In the classic **message-passing model**, a computer sits still on each node and in each round sends a message to every neighbour. Agents can pretend to be such computers, provided there is one agent per node and neighbouring agents can meet once per simulated round. Here the agents know n and Δ.

1. **Disperse.** Run a dispersion algorithm so that every node holds exactly one agent; that node is the agent's **home**. Knowing n tells the agents how long to wait before they can be sure dispersion is over.

2. **Pad the IDs.** Each agent writes its ID in binary with the same number of bits, $O(\log n)$, adding leading zeros (knowing n fixes this number).

3. **Meet every neighbour.** Split time into blocks of 2Δ rounds, one block per bit. In block i, an agent whose i-th bit is 1 visits its neighbours one by one (leave by port 1, come back, leave by port 2, come back, ...); an agent whose i-th bit is 0 stays at home. This meeting schedule is the **ID-bit schedule**.

4. Two neighbouring agents have different IDs, so in some block one of them has bit 1 and the other bit 0: the first arrives while the second is at home, and they meet. A visitor recognises the agent at home as the one whose current bit is 0.

5. **Simulate.** When two neighbouring agents meet they hand over the messages the message-passing algorithm would send along their edge. So one pass through all the blocks simulates one round of message passing.

Claude (Anthropic) was used in cataloguing the problems.

6. Run any message-passing leader election this way, for example one that needs $O(D)$ rounds, where D is the *diameter* (the largest distance between two nodes).

Why it works and what it costs. A block lasts 2Δ rounds because a visit to a neighbour and back takes 2 rounds and there are at most Δ neighbours. So one message-passing round costs $O(\log n)$ blocks of 2Δ rounds, that is $O(\Delta \log n)$ rounds, and a message-passing algorithm of R rounds costs $O(\Delta R \log n)$ rounds, plus the time to disperse ($O(n \log^2 n)$ rounds with the dispersion algorithm the paper uses Sudo et al. 2024). With an $O(D)$-round election this gives $O(\Delta D \log n + n \log^2 n)$ rounds. The catch is memory: each agent must store whatever a message-passing node stores, which can be large; (Kshemkalyani et al. 2026) reach a bound of the same form, $O(n \log^2 n + D\Delta \log n)$, with only $O(\log n)$ bits per agent.

Why it matters. It turns decades of message-passing algorithms into agent algorithms at a cost of a factor $\Delta \log n$, and the ID-bit schedule is reused by almost every later paper on computing with agents.

2. Algorithm: Walk every edge and come home. (Kshemkalyani et al. 2024) (No knowledge, p. 229) Without knowing n or Δ, the agents cannot use blocks of fixed length, and an agent that starts alone on its node does not even know whether the others are also alone. The first algorithm for this setting works as follows (simplified here for a dispersed start).

1. Each agent visits its neighbours once and returns home. It stays a *candidate* only if its node has the smallest degree among its neighbours, with ties broken by ID; otherwise it gives up. At least one candidate survives.
2. Each surviving candidate tells a neighbouring agent where its home is, so that others do not mistake its empty home for an unvisited node.
3. Each candidate then performs a *depth-first walk* (it leaves by a port it has not tried yet when there is one, and otherwise goes back the way it came) that crosses every edge of the graph, carrying a tag (start round, ID).
4. When two walks meet, the one with the smaller tag stops and its owner gives up.
5. A candidate whose walk crosses every edge and returns home declares itself leader.

Why it works and what it costs. A depth-first walk crosses each edge at most twice (once out, once back), so it ends within $O(m)$ rounds. The delicate part, which the paper proves, is that once one candidate declares itself leader no other can; this needs the home markers of step 2 and care when walks pass each other. When the agents start in clusters the same paper first disperses them with depth-first walks, and then needs $O(n \log n)$ bits per agent. Later work brought the time down to $O(n)$ rounds with $O(\log n)$ bits (Chand and Molla 2026), building on optimal dispersion (Kshemkalyani et al. 2025a).

Why it matters. It showed that leader election, and with it MST, MIS and dominating sets, can be solved with no knowledge of the graph at all, which is the setting all later papers aim for.

3. Impossibility: Why the agents need distinct identifiers. (No knowledge, p. 229) Every setting in this catalogue gives the agents distinct IDs. Here is why. Take a ring of 6 nodes, one agent on each, and number the ports so that at every node port 1 leads clockwise and port 2 counter-clockwise. Suppose the 6 agents are identical (no IDs) and follow the same deterministic program. In round 1 they all see exactly the same thing, so they all do the same thing: all stay, or all move clockwise, or all move counter-clockwise. After the move each agent again sees exactly what every other agent sees. By the same reasoning round after round, all 6 agents are always in the same state. If one of them ever decides "I am the leader", all 6 decide it at the same moment.

Why it holds. So no deterministic algorithm elects a leader among identical agents on this ring, no matter how many rounds it uses or how much memory the agents have. The argument is a standard symmetry

Claude (Anthropic) was used in cataloguing the problems.

argument, not a result of a particular paper in this catalogue. Distinct IDs break the symmetry; the algorithms above use them both to decide who wins and, through the bits of the IDs (the ID-bit schedule), to schedule meetings.

Why it matters. Symmetry is the basic obstacle to leader election, and it explains why identifiers (or randomness) are part of every model in which leaders are elected.

What the community started looking at

The agent-based model asks what graph problems **agents** can solve on an anonymous network, with **rounds first and memory second** as the measures (Kshemkalyani et al. 2024). Leader election came first because it unlocks the rest (MST, BFS trees, gathering).

Assumptions specific to this problem

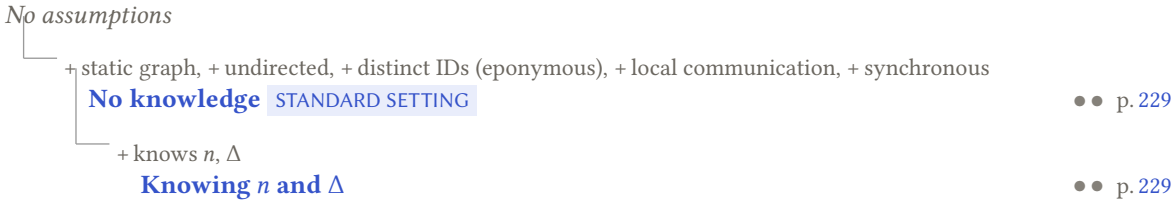
Knowledge

Whether agents know global parameters is inherited from message passing, where nodes typically know n or an upper bound on it, and possibly Δ and D; Kshemkalyani, Kumar, Molla and Sharma set the agent-based model against this and asked for algorithms that rely on no such knowledge, "neither exact nor an upper bound" (Kshemkalyani et al. 2024). Knowing n and Δ tells an agent how long to wait: in the arbitrary-start minimal dominating set algorithm of Chand, Molla and Sivasubramaniam every robot waits $\kappa n\Delta$ rounds before leader election begins (Chand et al. 2023a), and the introductory maximal independent set algorithm assumes both n and Δ (Pattanayak et al. 2024) (as reported by Kaur et al. 2026b and Kshemkalyani et al. 2024). Kshemkalyani et al. state the trade-off: solutions without knowledge "are more resilient to network changes and device faults", but "algorithm design becomes challenging since devices may not know how long to run a procedure to guarantee a solution" (Kshemkalyani et al. 2024). Later papers remove the assumption by first dispersing with termination detection and electing a leader, after which a spanning tree lets the agents compute Δ and the largest identifier themselves (Chand et al. 2026); knowing only that largest identifier L is a weaker assumption, recorded on results rather than as a level of this axis (Chand et al. 2025b). For leader election both levels are catalogued: without knowledge (Kshemkalyani et al. 2024) (Chand and Molla 2026), and faster with n and Δ known (Kshemkalyani et al. 2026).

Level	Value	Formal definition
0	**no knowledge**	Agents know no graph parameter (n, m, Δ, D).
1	**knows n, Δ**	Every agent knows n and Δ (upper bounds suffice where stated).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Memory) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Identifiers → Communication → Timing → Knowledge → Initial configuration → Topology.



Claude (Anthropic) was used in cataloguing the problems.

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Memory	p.
No knowledge STANDARD SETTING	● unknown $O(n)$ vs —	● unknown $O(\log n)$ vs —	229
Knowing n and Δ	● unknown $O(n \log^2 n + D\Delta \log n)$ vs —	● unknown $O(\log n)$ vs —	229

17.1.1 No knowledge

Adds + static graph, + undirected, + distinct IDs (eponymous), + local communication, + synchronous
STANDARD SETTING VERIFIED

Without any knowledge of the graph, leader election takes $O(n)$ rounds with $O(\log n)$ bits.

Assumed: graph dynamics: static graph; edge direction: undirected; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** knowledge: no knowledge of graph parameters; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes); topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known No time lower bound is stated in the papers.	$O(n)$ (Chand and Molla 2026)	● unknown
Memory	none known $\Omega(\log n)$ for distinct identifiers is folklore, not recorded here.	$O(\log n)$ (Chand and Molla 2026)	● unknown

Results.

- Kshemkalyani et al. (2024) [introductory]: time $O(m)$, memory $O(n \log n)$.
- Kshemkalyani et al. (2025b) [also]: time $O(n \log^2 n)$, memory $O(\log n)$.
- Chand and Molla (2026) [current best]: time $O(n)$, memory $O(\log n)$. A by-product of the minimal dominating set algorithm, built on optimal dispersion (Kshemkalyani et al. 2025a).

Notes. Spanning trees: BFS and MST with $O(\log n)$ bits from a dispersed start (Chand et al. 2025b).

17.1.2 Knowing n and Δ

Adds + knows n, Δ **to** No knowledge (p. 229) VERIFIED

With n and Δ known, leader election takes $O(n \log^2 n + D\Delta \log n)$ rounds with $O(\log n)$ bits.

Assumed: graph dynamics: static graph; edge direction: undirected; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; communication: local (co-located agents communicate); timing: synchronous rounds; knowledge: n and Δ are known. **Not assumed:** initial configuration: arbitrary (agents on any $\ell \leq k$ nodes); topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known —	$O(n \log^2 n + D\Delta \log n)$ (Kshemkalyani et al. 2026)	● unknown
Memory	none known —	$O(\log n)$ (Kshemkalyani et al. 2026)	● unknown

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Kshemkalyani et al. (2026) [current best]: time $O(n \log^2 n + D\Delta \log n)$, memory $O(\log n)$. The bound with the $\log n$ factor is the correct one (confirmed by the catalogue owner, a co-author); the abstract omits it.

17.2 Maximal independent set by agents

n agents in an anonymous graph must mark a maximal independent set of nodes.

First posed in 2024 (Pattanayak et al. 2024). Introduced at ICDCN 2024.

Problem statement and measures

n agents with distinct identifiers (IDs) are placed on the nodes of an anonymous port-labelled graph with n nodes, m edges and maximum degree Δ. The start is rooted (all agents on one node), dispersed (one agent on each node) or arbitrary. Agents compute only by moving and by talking with agents on the same node. The agents must settle on the nodes of a maximal independent set, every other agent knowing it is not in the set: every agent (or node) learns its part of the output. **Time** is in synchronous rounds; **memory** in bits per agent.

Measures: **Time**, rounds; **Memory**, bits per agent.



(a) Start: all 5 agents at node a

(b) Final: black nodes a, d, e form an MIS

Figure 17.2: MIS by a depth-first walk that colours as it settles (Chand et al. 2023a) (other algorithms, e.g. from an arbitrary start, in Pattanayak et al. 2024): all agents start at node a; the group moves one edge per round in depth-first order (the walk's tree is drawn in grey), and at each empty node the agent with the smallest ID settles. A settled agent whose parent is black turns grey at once; otherwise it visits each neighbour and comes back, and turns black only if none is black. The black nodes form a maximal independent set.

A first look

Imagine a building with n rooms that carry no names, and n **agents** (small mobile computers) who must choose some rooms to stand in so that no two chosen rooms are next to each other, yet every room is either chosen or next to a chosen room. The building is a **graph**: rooms are its **nodes** and corridors are its **edges**; there are n nodes and m edges, and from here on we say only node and edge. Two nodes joined by an edge are *neighbours*. The **degree** $\deg(v)$ of a node v is the number of edges at it, and Δ is the maximum degree. Nodes have no names, but the edges at a node are numbered $1, 2, \ldots$; these numbers are **ports**. Each agent has a distinct **identifier** (ID). A set of nodes in which no two nodes are neighbours is *independent*; it is a **maximal independent set** (MIS) if no further node can be added to it without losing this. The agents move in synchronous **rounds** (in each round every agent may talk

Claude (Anthropic) was used in cataloguing the problems.

to the agents on its node, then move along at most one edge), and at the end every agent must know whether it stands on a chosen node. We count rounds and bits of memory per agent. As for all problems in this family, a general method is to make the agents **dispersed** (one agent on each node) and then let them imitate a classical message-passing algorithm by visiting their neighbours (explained under leader election by agents).

1. Algorithm: Depth-first walk that colours as it settles. (Chand et al. 2023a) (No knowledge, p. 233) All n agents start together on one node (a *rooted* start) and know nothing about the graph.

1. The group moves through the graph in a **depth-first walk**: it leaves its current node by a port not tried yet when there is one, and otherwise goes back to the node from which it reached the current node. Whenever it reaches an empty node, the agent with the smallest ID among those still walking stays there for good: it becomes a **settled agent**, and it remembers the port by which the group came in, so the group can later walk back.
2. Before the group moves on, the new settled agent checks its neighbours. If its **parent** (the node from which the walk reached it) holds a *black* agent, it colours itself *grey*. Otherwise it visits every neighbour and comes back: if some neighbour is black it becomes grey, else black.
3. The rest of the group continues the depth-first walk; finding a settled agent on a node tells it that the node was visited before, so it turns back.
4. When every agent has settled, the black agents form the answer.

Why it works and what it costs. A black agent never has a black neighbour, so the black nodes are independent; every grey agent has a black neighbour, so no node can be added: the set is maximal. The depth-first walk crosses each edge a constant number of times, and the neighbour checks cost $2\deg(v)$ rounds at each node v, which sums to $4m$; so the total is $O(m)$ rounds with $O(\log n)$ bits per agent (Chand et al. 2023a). The current best without knowledge, from any start, is $O(\max\{n \log^2 n, m\})$ rounds with $O(\log n)$ bits (Kaur et al. 2026b).

Why it matters. It shows that settling agents one at a time along a depth-first walk gives a greedy algorithm for free, the same pattern as dispersion.

2. Lower bound: Some agent must walk almost the whole diameter. (No knowledge, p. 233) Let all agents start on the end s of a path with $D + 1$ nodes, so the *diameter* D (largest distance between two nodes) is the length of the path. Maximality says that the far end u or its only neighbour must be chosen, and a chosen node needs an agent standing on it. Both nodes are at distance at least $D - 1$ from s.

Why it holds. An agent moves along at most one edge per round, so no agent reaches distance $D - 1$ before round $D - 1$: from a rooted start every algorithm needs at least $D - 1$ rounds. This simple argument is our own, not from the papers; no better lower bound is recorded in the catalogue, so the gap to the known $O(\max\{n \log^2 n, m\})$ rounds is wide open.

Why it matters. Distance is the most basic obstacle for moving agents, and the gap between this bound and the known algorithms shows how little is known about the optimal time.

3. Algorithm: Greedy by identifiers, simulated after dispersion. (Kshemkalyani et al. 2024) (Knowing n and Δ, p. 233) Here the agents know n and Δ, which lets them use the disperse-and-simulate method.

1. Disperse, so that every node holds one agent.
2. Repeat simulated message-passing rounds, each made of $O(\log n)$ blocks of 2Δ rounds in which neighbouring agents meet by the **ID-bit schedule**: in block i an agent whose i-th ID bit is 1 visits

Claude (Anthropic) was used in cataloguing the problems.

its neighbours one by one and an agent whose bit is 0 waits on its node (see leader election by agents).

3. An agent joins the set as soon as every neighbour with a larger ID has decided and none of them joined; it stays out as soon as some neighbour with a larger ID joined.
4. Each agent tells its neighbours its decision at the next meetings.

Why it works and what it costs. The agent with the largest ID decides at once, and an agent waits only for neighbours with larger IDs, so a chain of waiting agents has decreasing IDs and at most n members: at most n simulated rounds. Each costs $O(\Delta \log n)$ real rounds, so the total is $O(n\Delta \log n)$ plus the time to disperse, the same order as the introductory $O(n\Delta \log n)$ bound of (Pattanayak et al. 2024) (whose algorithm may differ). With n and Δ known the current best is $O(n \log^2 n + D\Delta \log n)$ rounds with $O(\log n)$ bits (Kshemkalyani et al. 2026).

Why it matters. It shows the cost of the generic method: a slow message-passing algorithm becomes a slow agent algorithm, which is why later work designs algorithms for agents directly.

What the community started looking at

MIS by agents was posed with **time first, memory second**: rooted and arbitrary starts, trees and grids (Pattanayak et al. 2024); later work removed the knowledge of n and Δ (Kaur et al. 2026b).

Assumptions specific to this problem

Knowledge

Whether agents know global parameters is inherited from message passing, where nodes typically know n or an upper bound on it, and possibly Δ and D; Kshemkalyani, Kumar, Molla and Sharma set the agent-based model against this and asked for algorithms that rely on no such knowledge, "neither exact nor an upper bound" (Kshemkalyani et al. 2024). Knowing n and Δ tells an agent how long to wait: in the arbitrary-start minimal dominating set algorithm of Chand, Molla and Sivasubramaniam every robot waits $\kappa n\Delta$ rounds before leader election begins (Chand et al. 2023a), and the introductory maximal independent set algorithm assumes both n and Δ (Pattanayak et al. 2024) (as reported by Kaur et al. 2026b and Kshemkalyani et al. 2024). Kshemkalyani et al. state the trade-off: solutions without knowledge "are more resilient to network changes and device faults", but "algorithm design becomes challenging since devices may not know how long to run a procedure to guarantee a solution" (Kshemkalyani et al. 2024). Later papers remove the assumption by first dispersing with termination detection and electing a leader, after which a spanning tree lets the agents compute Δ and the largest identifier themselves (Chand et al. 2026); knowing only that largest identifier L is a weaker assumption, recorded on results rather than as a level of this axis (Chand et al. 2025b). For maximal independent sets the introductory algorithm assumes n and Δ (Pattanayak et al. 2024); (Kshemkalyani et al. 2024) and (Kaur et al. 2026b) remove the assumption.

Level	Value	Formal definition
0	**no knowledge**	Agents know no graph parameter (n, m, Δ, D).
1	**knows n, Δ**	Every agent knows n and Δ (upper bounds suffice where stated).

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Memory) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Identifiers → Communication → Timing → Knowledge → Initial configuration → Topology.

No assumptions

+ static graph, + undirected, + distinct IDs (eponymous), + local communication, + synchronous

No knowledge ●● p. 233

+ knows n, Δ

Knowing n and Δ STANDARD SETTING ●● p. 233

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Memory	p.
No knowledge	● unknown $O(\max\{n \log^2 n, m\})$ vs —	● unknown $O(\log n)$ vs —	233
Knowing n and Δ STANDARD SETTING	● unknown $O(n \log^2 n + D\Delta \log n)$ vs —	● unknown $O(\log n)$ vs —	233

17.2.1 No knowledge

Adds + static graph, + undirected, + distinct IDs (eponymous), + local communication, + synchronous VERIFIED

Without knowledge: $O(\max\{n \log^2 n, m\})$ rounds with $O(\log n)$ bits.

Assumed: graph dynamics: static graph; edge direction: undirected; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** knowledge: no knowledge of graph parameters; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes); topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known —	$O(\max\{n \log^2 n, m\})$ (Kaur et al. 2026b)	● unknown
Memory	none known —	$O(\log n)$ (Kaur et al. 2026b)	● unknown

Results.

- Kshemkalyani et al. (2024) [introductory]: time $O(n\Delta)$, memory $O(n \log n)$.
- Kaur et al. (2026b) [current best]: time $O(\max\{n \log^2 n, m\})$, memory $O(\log n)$.

17.2.2 Knowing n and Δ

Adds + knows n, Δ **to** No knowledge (p. 233) STANDARD SETTING VERIFIED

With n and Δ known: $O(n\Delta)$ rounds from a rooted start, and $O(n \log^2 n + D\Delta \log n)$ from any start, with $O(\log n)$ bits.

Claude (Anthropic) was used in cataloguing the problems.

Assumed: graph dynamics: static graph; edge direction: undirected; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; communication: local (co-located agents communicate); timing: synchronous rounds; knowledge: n and Δ are known. **Not assumed:** initial configuration: arbitrary (agents on any $\ell \leq k$ nodes); topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound recorded.	$O(n \log^2 n + D\Delta \log n)$ (Kshemkalyani et al. 2026)	• unknown
Memory	none known —	$O(\log n)$ (Kshemkalyani et al. 2026)	• unknown

Results.

- Pattanayak et al. (2024) [introductory]: time $O(n\Delta \log n)$, memory $O(\log n)$. $O(n\Delta)$ from a rooted start; trees: $O(D^2)$ rooted, $O(D)$ dispersed.
- Kshemkalyani et al. (2026) [current best]: time $O(n \log^2 n + D\Delta \log n)$, memory $O(\log n)$.

17.3 Minimal dominating set by agents

n agents in an anonymous graph must mark a minimal dominating set.

First posed in 2023 (Chand et al. 2023a). Introduced at ALGOSENSORS 2023.

Problem statement and measures

n agents with distinct identifiers (IDs) are placed on the nodes of an anonymous port-labelled graph with n nodes, m edges and maximum degree Δ. The start is rooted (all agents on one node), dispersed (one agent on each node) or arbitrary. Agents compute only by moving and by talking with agents on the same node. The agents must mark a minimal dominating set of nodes: every agent (or node) learns its part of the output. **Time** is in synchronous rounds; **memory** in bits per agent.

Measures: **Time**, rounds; **Memory**, bits per agent.



(a) Start: all 5 agents at node a

(b) Final: black a, c, d dominate, minimally

Figure 17.3: Minimal dominating set by a depth-first walk that colours as it settles (Chand et al. 2023a): all agents start at node a; the group moves one edge per round in depth-first order (the walk's tree is drawn in grey), and at each empty node the agent with the smallest ID settles. A settled agent whose parent is black turns grey at once; otherwise it visits each neighbour and comes back, and turns black only if none is black. Every node is black or next to a black node, and no black node can be dropped.

Claude (Anthropic) was used in cataloguing the problems.

A first look

Imagine placing first-aid kits in some rooms of a building so that every room either has a kit or is next to a room with one, and no kit can be removed without leaving some room uncovered. The building is a **graph**: rooms are its **nodes** and corridors are its **edges**; there are n nodes and m edges, and from here on we say only node and edge. Two nodes joined by an edge are *neighbours*. The **degree** $\deg(v)$ of a node v is the number of edges at it, and Δ is the maximum degree. Nodes have no names, but the edges at a node are numbered $1, 2, \dots$; these numbers are **ports**. A set of nodes is a *dominating set* if every node is in it or is a neighbour of a node in it; it is a **minimal dominating set** if removing any one node breaks this property (it need not be the smallest possible). n **agents** (small mobile computers) with distinct **identifiers** (IDs) move in synchronous **rounds** (in each round an agent may talk to the agents on its node, then move along at most one edge) and must mark such a set: at the end every agent knows whether its node is in the set. We count rounds and bits of memory per agent. As for all problems in this family, one general method is to place one agent on each node and then imitate a message-passing algorithm by visiting neighbours (explained under leader election by agents).

1. Algorithm: Depth-first walk that colours as it settles. (Chand et al. 2023a) (No knowledge, p. 237)
All n agents start on one node (a *rooted* start) and know nothing about the graph.

1. The group moves through the graph in a **depth-first walk**: it leaves its current node by a port not tried yet when there is one, and otherwise goes back to the node from which it reached the current node. At each empty node it reaches, the walking agent with the smallest ID stays for good, becoming a **settled agent**, and remembers the port leading back.
2. The new settled agent decides its colour: if its **parent** (the node from which the walk reached it) holds a *black* agent it becomes *grey*; otherwise it visits all its neighbours and becomes grey if one of them is black, and black if none is.
3. The others continue; a settled agent on a node tells them the node was visited before.
4. When all agents have settled, the black nodes are the dominating set.

Why it works and what it costs. Every agent ends black or grey, and grey means "next to a black one", so the black nodes dominate the graph. They are also minimal: a black node has no black neighbour (all its neighbours are grey or settled after it and saw it), so removing it would leave it uncovered. In fact the black nodes form a maximal independent set, and every maximal independent set is a minimal dominating set. The walk crosses each edge a constant number of times and the checks cost $2\deg(v)$ rounds per node v, $4m$ in total, so the algorithm takes $O(m)$ rounds with $O(\log n)$ bits (Chand et al. 2023a). The current best takes $O(n)$ rounds, from a rooted or arbitrary start, with $O(\log n)$ bits (Chand and Molla 2026), building on a faster dispersion algorithm (Kshemkalyani et al. 2025a).

Why it matters. It is the template that later dominating-set and MIS algorithms for agents refine: settle agents greedily during exploration and let each settled agent decide from its neighbourhood.

2. Lower bound: Some agent must walk almost the whole diameter. (No knowledge, p. 237) Let all agents start on the end s of a path with $D + 1$ nodes, so the *diameter* D (largest distance between two nodes) is the path's length. The far end u must be dominated: u or its only neighbour must be marked, and a marked node holds an agent. Both are at distance at least $D - 1$ from s.

Why it holds. Agents move along at most one edge per round, so the algorithm needs at least $D - 1$ rounds on this path. This is our own short argument; the catalogue records no lower bound, so whether the $O(n)$ rounds of (Chand and Molla 2026) are optimal is open. On a path $D = n - 1$, so on paths the $O(n)$ algorithm is optimal up to a constant factor.

Claude (Anthropic) was used in cataloguing the problems.

Why it matters. It shows that linear time is the right target on long, thin graphs, and that a better lower bound would need graphs with small diameter.

What the community started looking at

Dominating sets by agents were posed with time and memory as measures (Chand et al. 2023a); building on optimal dispersion, linear time without any knowledge followed (Chand and Molla 2026).

Assumptions specific to this problem

Knowledge

Whether agents know global parameters is inherited from message passing, where nodes typically know n or an upper bound on it, and possibly Δ and D; Kshemkalyani, Kumar, Molla and Sharma set the agent-based model against this and asked for algorithms that rely on no such knowledge, "neither exact nor an upper bound" (Kshemkalyani et al. 2024). Knowing n and Δ tells an agent how long to wait: in the arbitrary-start minimal dominating set algorithm of Chand, Molla and Sivasubramaniam every robot waits $\kappa n\Delta$ rounds before leader election begins (Chand et al. 2023a), and the introductory maximal independent set algorithm assumes both n and Δ (Pattanayak et al. 2024) (as reported by Kaur et al. 2026b and Kshemkalyani et al. 2024). Kshemkalyani et al. state the trade-off: solutions without knowledge "are more resilient to network changes and device faults", but "algorithm design becomes challenging since devices may not know how long to run a procedure to guarantee a solution" (Kshemkalyani et al. 2024). Later papers remove the assumption by first dispersing with termination detection and electing a leader, after which a spanning tree lets the agents compute Δ and the largest identifier themselves (Chand et al. 2026); knowing only that largest identifier L is a weaker assumption, recorded on results rather than as a level of this axis (Chand et al. 2025b). For minimal dominating sets the rooted algorithm of (Chand et al. 2023a) already needs no knowledge, and (Chand and Molla 2026) removes it from the arbitrary start as well.

Level	Value	Formal definition
0	**no knowledge**	Agents know no graph parameter (n, m, Δ, D).
1	**knows n, Δ**	Every agent knows n and Δ (upper bounds suffice where stated).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Memory) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Identifiers → Communication → Timing → Knowledge → Initial configuration → Topology.

No assumptions

+ static graph, + undirected, + distinct IDs (eponymous), + local communication, + synchronous

No knowledge STANDARD SETTING ●● p. 237

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Claude (Anthropic) was used in cataloguing the problems.

Setting	Time	Memory	p.
No knowledge STANDARD SETTING	● unknown $O(n)$ vs —	● unknown $O(\log n)$ vs —	237

17.3.1 No knowledge

Adds + static graph, + undirected, + distinct IDs (eponymous), + local communication, + synchronous
STANDARD SETTING VERIFIED

A minimal dominating set in $O(n)$ rounds with $O(\log n)$ bits, from a rooted or arbitrary start.

Assumed: graph dynamics: static graph; edge direction: undirected; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** knowledge: no knowledge of graph parameters; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes); topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound stated.	$O(n)$ (Chand and Molla 2026)	● unknown
Memory	none known —	$O(\log n)$ (Chand and Molla 2026)	● unknown

Results.

- Chand et al. (2023a) [introductory]: time $O(m)$, memory $O(\log n)$. *Assumes:* rooted start.
- Chand and Molla (2026) [current best]: time $O(n)$, memory $O(\log n)$.

17.4 Triangle and butterfly counting by agents

n agents in an anonymous graph must count small subgraphs (triangles; butterflies in bipartite graphs) at every node.

First posed in 2024 (Chand et al. 2026). Agent-based triangle counting (arXiv 2024; IEEE Transactions on Networking 2026).

Problem statement and measures

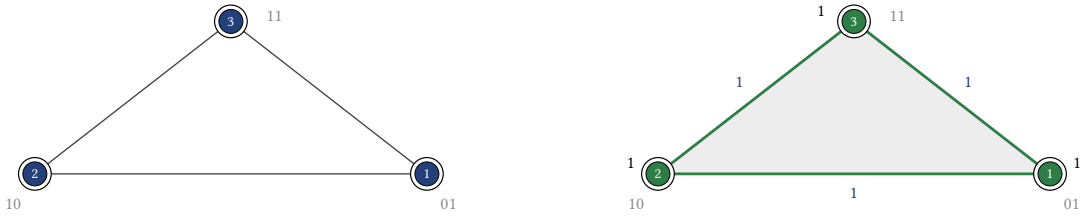
n agents with distinct identifiers (IDs) are placed on the nodes of an anonymous port-labelled graph with n nodes, m edges and maximum degree Δ. The start is rooted (all agents on one node), dispersed (one agent on each node) or arbitrary. Agents compute only by moving and by talking with agents on the same node. The agents must compute, at every node, the number of triangles (or butterflies) it belongs to: every agent (or node) learns its part of the output. **Time** is in synchronous rounds; **memory** in bits per agent.

Measures: **Time**, rounds; **Memory**, bits per agent.

A first look

Imagine n **agents** (small mobile computers), one in each room of a building, who want to know how tightly the rooms are connected: for each room, how many pairs of the rooms next to it are also next to

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: dispersed; IDs written in 2 bits

(b) Final: (1 + 1) / 2 = 1 triangle at every node

Figure 17.4: Triangle counting (Chand et al. 2026) on a triangle: after dispersing (one agent per node), electing a leader and learning Δ (not shown), the agents write their IDs in 2 bits and follow the ID-bit schedule: in block i (of $2\Delta = 4$ rounds) agents whose i-th bit is 1 visit each neighbour and come back, one edge per round, while the others wait at home. In the first pass a visit to a waiting neighbour records both IDs (green edges); in the second pass, round by round with the same walks, the two ends of each edge count the IDs on both lists (blue numbers). Each agent adds its edges' counts and halves the sum: the number of triangles at its node.

each other. The building is a **graph**: rooms are its **nodes** and corridors are its **edges**; there are n nodes and m edges, and from here on we say only node and edge. Two nodes joined by an edge are *neighbours*. The **degree** of a node is the number of edges at it, and Δ is the maximum degree. Nodes have no names, but the edges at a node are numbered $1, 2, \ldots$; these numbers are **ports**. A **triangle** is three nodes all joined to each other. Agents have distinct **identifiers** (IDs) between 1 and L, can talk only to agents on the same node, and move in synchronous **rounds** (in each round: talk, then move along at most one edge). For the basic version the agents start anywhere and know nothing about the graph; at the end the agent on each node v must know the number of triangles containing v. We count rounds and bits of memory per agent. Like all problems in this family, the work starts by making the agents **dispersed** (one agent on each node, which becomes the agent's **home**) and letting neighbouring agents meet, the method explained under leader election by agents.

1. Algorithm: Learn your neighbours, then compare lists. (Chand et al. 2026) (Triangle counting, p. 241)

1. **Get organised.** The agents disperse (one per node), elect a leader and build a spanning tree, learning Δ on the way.
2. **Meet your neighbours.** Write IDs with $\log L$ bits and follow the **ID-bit schedule**: in block i (of 2Δ rounds), agents whose i-th bit is 1 visit their neighbours one by one, and agents whose bit is 0 stay at home. Every pair of neighbouring agents meets in some block, and each agent writes down the IDs of all its neighbours.
3. **Compare lists.** Repeat the meetings. When the agents on neighbouring nodes v and u meet, they count the IDs that appear on both lists: that is the number of triangles that contain the edge $\{v, u\}$.
4. **Add up.** The agent on v adds these counts over all its neighbours and divides by 2: each triangle $\{v, u, w\}$ was counted once through u and once through w.

Why it works and what it costs. Step 2 and step 3 each take $\log L$ blocks of 2Δ rounds, i.e. $O(\Delta \log L)$ rounds, and each agent stores up to Δ IDs, i.e. $O(\Delta \log L)$ bits. Step 1 costs $O(n \log^2 n)$ rounds, and it dominates the total $O(n \log^2 n + \Delta \log L)$ (Chand et al. 2026). The same counts per edge give truss decomposition and local clustering coefficients. No lower bound is proved.

Why it matters. It shows that once agents are organised, local graph statistics cost only $O(\Delta \log L)$ rounds, so the bottleneck is getting organised, not counting.

Claude (Anthropic) was used in cataloguing the problems.

2. Lower bound: The total count needs information from both ends of the graph. (Triangle counting, p. 241) Besides the per-node counts, the algorithm also lets the agents learn the total number of triangles. Information moves at most one edge per round, because it travels only inside agents and an agent moves along at most one edge per round. So after t rounds, what an agent standing on node y knows depends only on the part of the graph within distance t of y. Take a long path and, at each end, attach two extra nodes a and b to the end node; adding the edge $\{a, b\}$ closes a triangle. If an agent announces the total after fewer than about $D/2$ rounds, where D is the *diameter* (largest distance between two nodes), one end lies outside what it can know about. Adding or removing the edge $\{a, b\}$ there changes the true total but not what this agent knows, so in one of the two graphs its answer is wrong.

Why it holds. So computing the total needs $\Omega(D)$ rounds. This is our own short argument, not a result recorded in the catalogue; the known algorithm needs $O(n + \Delta \log L)$ rounds for the total after the set-up, and on a path D is about n, so it is optimal there up to a constant factor. For the per-node counts this argument gives nothing, because they depend only on nearby nodes.

Why it matters. This "information moves one edge per round" argument is the simplest lower-bound tool for agents, and it separates local tasks (per-node counts) from global ones (totals).

3. Algorithm: Bipartite graphs: one side walks while the other waits. (Chand et al. 2025a) (Butterfly counting in bipartite graphs, p. 241) In a *bipartite* graph the nodes split into two sides V_1 and V_2 and every edge joins the two sides. A *butterfly* is a 4-cycle: two nodes a, a' on one side with two common neighbours on the other.

1. The agents, starting dispersed (one per node), elect a leader, build a spanning tree and learn which side they are on.
2. In odd rounds the agents on one side visit their neighbours while the other side waits at home; in even rounds the roles swap. Within 2Δ rounds every agent knows its neighbours' IDs.
3. The agent on a visits its neighbours again and reads their lists, so it learns, for every a' on its side, the number c of common neighbours of a and a'.
4. Each such pair lies on $\binom{c}{2}$ butterflies (choose two of the common neighbours); summing over a' gives the number of butterflies through a.

Why it works and what it costs. Because the two sides take turns, no ID-bit schedule is needed: neighbours meet within 2Δ rounds rather than $O(\Delta \log L)$, and the per-node counts take $O(\Delta)$ rounds. The set-up takes $O(n \log L)$ rounds, for a total of $O(n \log L + \Delta)$ rounds with $O(\Delta \log L)$ bits per agent (Chand et al. 2025a).

Why it matters. Knowing more structure (here, a 2-colouring of the graph) can remove the $\log L$ cost of making neighbours meet.

What the community started looking at

Counting problems extend agent-based computing from symmetry breaking to **graph analytics**: triangle counting and its applications (Chand et al. 2026), and butterfly counting in bipartite graphs (Chand et al. 2025a).

Assumptions specific to this problem

Knowledge

Claude (Anthropic) was used in cataloguing the problems.

Whether agents know global parameters is inherited from message passing, where nodes typically know n or an upper bound on it, and possibly Δ and D; Kshemkalyani, Kumar, Molla and Sharma set the agent-based model against this and asked for algorithms that rely on no such knowledge, "neither exact nor an upper bound" (Kshemkalyani et al. 2024). Knowing n and Δ tells an agent how long to wait: in the arbitrary-start minimal dominating set algorithm of Chand, Molla and Sivasubramaniam every robot waits $\kappa n\Delta$ rounds before leader election begins (Chand et al. 2023a), and the introductory maximal independent set algorithm assumes both n and Δ (Pattanayak et al. 2024) (as reported by Kaur et al. 2026b and Kshemkalyani et al. 2024). Kshemkalyani et al. state the trade-off: solutions without knowledge "are more resilient to network changes and device faults", but "algorithm design becomes challenging since devices may not know how long to run a procedure to guarantee a solution" (Kshemkalyani et al. 2024). Later papers remove the assumption by first dispersing with termination detection and electing a leader, after which a spanning tree lets the agents compute Δ and the largest identifier themselves (Chand et al. 2026); knowing only that largest identifier L is a weaker assumption, recorded on results rather than as a level of this axis (Chand et al. 2025b). For triangle counting, Chand, Das and Molla remove the need to know Δ in exactly this way (Chand et al. 2026); their butterfly counting still assumes L (Chand et al. 2025a).

Level	Value	Formal definition
0	**no knowledge**	Agents know no graph parameter (n, m, Δ, D).
1	**knows n, Δ**	Every agent knows n and Δ (upper bounds suffice where stated).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Memory) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Identifiers → Communication → Timing → Knowledge → Initial configuration → Topology.

No assumptions

+ static graph, + undirected, + distinct IDs (eponymous), + local communication, + synchronous

Triangle counting STANDARD SETTING ●● p. 241

+ dispersed start, + bipartite

Butterfly counting in bipartite graphs ●● p. 241

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Memory	p.
Triangle counting STANDARD SETTING	● unknown $O(n \log^2 n + \Delta \log L)$ vs —	● unknown $O(\Delta \log L)$ vs —	241
Butterfly counting in bipartite graphs	● unknown $O(n \log L + \Delta)$ vs —	● unknown $O(\Delta \log L)$ vs —	241

Claude (Anthropic) was used in cataloguing the problems.

17.4.1 Triangle counting

Adds + static graph, + undirected, + distinct IDs (eponymous), + local communication, + synchronous
STANDARD SETTING VERIFIED

From an arbitrary placement and without prior knowledge, agents count triangles per node and per edge, which yields truss decomposition, triangle centrality and local clustering coefficients.

Assumed: graph dynamics: static graph; edge direction: undirected; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; communication: local (co-located agents communicate); timing: synchronous rounds. **Not assumed:** knowledge: no knowledge of graph parameters; initial configuration: arbitrary (agents on any $\ell \leq k$ nodes); topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound is proved.	$O(n \log^2 n + \Delta \log L)$ (Chand et al. 2026)	• unknown
Memory	none known No lower bound is proved.	$O(\Delta \log L)$ (Chand et al. 2026)	• unknown

Results.

- Chand et al. (2026) [current best]: time $O(n \log^2 n + \Delta \log L)$, memory $O(\Delta \log L)$. Introductory and current best (Theorem 2): dispersion, leader election and a spanning tree in $O(n \log^2 n)$ rounds, then per-node and per-edge counts in $O(\Delta \log L)$ rounds and the total in $O(n + \Delta \log L)$. Applications: truss decomposition in $O(m\Delta \log L + mD)$ rounds with $O(\Delta^2 \log L)$ bits; triangle centrality and local clustering coefficients in $O(\Delta \log L)$ rounds.

Notes. L is the largest agent identifier, at most n^K, so $\log L = O(\log n)$. Exactly n agents. The problem is posed from a dispersed start, but Theorem 2 needs none. Checked against the full text (arXiv 2402.03653, version 3).

17.4.2 Butterfly counting in bipartite graphs

Adds + dispersed start, + bipartite **to** Triangle counting (p. 241) VERIFIED

In a bipartite graph, agents starting dispersed count butterflies ($K_{2,2}$) per node in $O(n \log L + \Delta)$ rounds.

Assumed: graph dynamics: static graph; edge direction: undirected; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; communication: local (co-located agents communicate); timing: synchronous rounds; initial configuration: dispersed or scattered (one agent per occupied node); topology: bipartite. **Not assumed:** knowledge: no knowledge of graph parameters.

Measure	Lower bound	Best upper bound	Status
Time	none known —	$O(n \log L + \Delta)$ (Chand et al. 2025a)	• unknown
Memory	none known —	$O(\Delta \log L)$ (Chand et al. 2025a)	• unknown

Results.

- Chand et al. (2025a) [current best]: time $O(n \log L + \Delta)$, memory $O(\Delta \log L)$. *Assumes:* knows L, an upper bound on the identifiers. Introductory and current best. Leader election, a spanning tree and the two sides of the bipartition take $O(n \log L)$ rounds with $O(\log L)$ bits (Theorem 3.5), on any graph; per-node butterfly counts then take $O(\Delta)$ rounds and the total $O(\Delta + \min\{|V_1|, |V_2|\})$ (Theorem 4.4), with $O(\Delta \log L)$ bits (Lemma 4.3).

Notes. V_1 and V_2 are the two sides of the bipartite graph. Checked against the full text (arXiv 2506.17721).

Claude (Anthropic) was used in cataloguing the problems.

17.5 Minimum spanning tree by agents

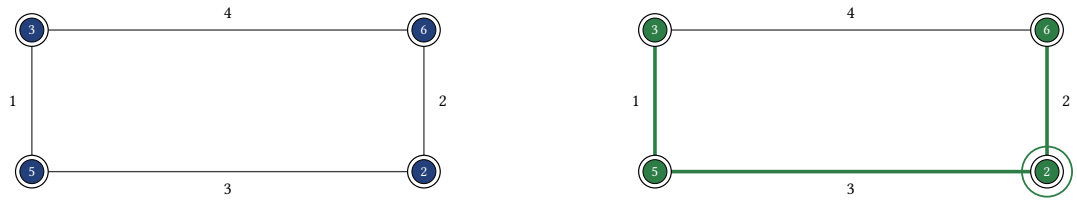
n agents in an anonymous weighted graph must build a minimum spanning tree.

First posed in 2024 (Kshemkalyani et al. 2024). MST in the agent-based model appeared together with leader election (DISC 2024).

Problem statement and measures

n agents with distinct identifiers (IDs) are placed on the nodes of an anonymous port-labelled graph with n nodes, m edges and maximum degree Δ. The start is rooted (all agents on one node), dispersed (one agent on each node) or arbitrary. Agents compute only by moving and by talking with agents on the same node. The agents must build a minimum spanning tree of the edge-weighted graph, each agent knowing which incident edges belong to it: every agent (or node) learns its part of the output. **Time** is in synchronous rounds; **memory** in bits per agent.

Measures: **Time**, rounds; **Memory**, bits per agent.



(a) Start: dispersed; every node its own fragment

(b) Final: both pick weight 3; one fragment, the MST

Figure 17.5: MST by growing fragments along their lightest outgoing edges (Borůvka, Gallager-Humblet-Spira) (Chand et al. 2025b): agents dispersed on a weighted cycle meet their neighbours with the ID-bit schedule (in each block of $2\Delta = 4$ rounds, agents whose current ID bit is 1 visit each neighbour and come back, one edge per round). Each one-node fragment adds its lightest edge (weights 1 and 2), giving two fragments (colours) with leaders 3 and 2 (circled). In phase 2 the agents meet again with the same walks (shown round by round) and find the edges that leave their fragment (red); 5 and 6 walk to their leaders to report the lightest outgoing edge, both fragments pick weight 3 and merge into the minimum spanning tree (thick).

A first look

Imagine n **agents** (small mobile computers), one in each room of a building, who must choose which corridors to keep open so that every room can still reach every other, using the cheapest possible total length. The building is a **graph**: rooms are its **nodes** and corridors are its **edges**; there are n nodes and m edges, and from here on we say only node and edge. Two nodes joined by an edge are *neighbours*. Every edge has a **weight** (its cost), which the agents on its two ends can see; Δ is the maximum degree (the largest number of edges at one node). Nodes have no names, but the edges at a node are numbered $1, 2, \ldots$; these numbers are **ports**. A *spanning tree* is a set of $n - 1$ edges that connects all nodes without forming a cycle, and a **minimum spanning tree** (MST) is one with the smallest total weight. In the basic version the agents start one per node (a **dispersed** start), have distinct **identifiers** (IDs), move in synchronous **rounds** (talk to the agents on the same node, then move along at most one edge), and at the end every agent must know which of the edges at its node belong to the MST. We count rounds and

Claude (Anthropic) was used in cataloguing the problems.

bits of memory per agent. As for all problems in this family, the agents can imitate a message-passing algorithm by visiting their neighbours, the method explained under leader election by agents.

1. Algorithm: Grow fragments along their lightest outgoing edges. (Chand et al. 2025b) (Dispersed start, p. 245) This is the classic Borůvka / Gallager-Humblet-Spira idea, carried out by agents. Assume all edge weights differ (agents can break ties using their IDs and ports).

1. At the start every node is its own **fragment**: a set of nodes joined by edges already chosen for the tree, so a small tree.
2. In each phase, every agent meets its neighbours by the **ID-bit schedule** (in block i of 2Δ rounds, agents whose i-th ID bit is 1 visit their neighbours one by one while the others wait; see leader election by agents) and learns which of the edges at its node leave its fragment and how heavy they are.
3. The agents of each fragment pass the lightest such edge up their tree to the fragment's **leader** (one agent of the fragment that decides for it), which picks the **lightest outgoing edge** of the whole fragment: the edge of smallest weight with one end in the fragment and the other outside.
4. Every fragment adds its lightest outgoing edge; fragments joined by these edges merge into one, with one leader.
5. Repeat until one fragment remains: it is the MST, and its leader is also an elected leader for all agents.

Why it works and what it costs. Step 4 is safe because of the *cut property*: the lightest edge leaving any group of nodes always belongs to the MST. Every fragment merges with at least one other in each phase, so the number of fragments at least halves, and $O(\log n)$ phases suffice. In total this gives $O(n \log n + \Delta \log^2 n)$ rounds with $O(\log n)$ bits per agent, assuming the agents know L, the largest ID (Chand et al. 2025b). The first agent algorithm handled the fragments one at a time and needed $O(m + n \log n)$ rounds and $O(n \log n)$ bits (Kshemkalyani et al. 2024).

Why it matters. Halving the number of fragments in every phase is the standard way to build spanning trees in parallel, and the resulting tree gives agents a root and a structure for broadcasting, used afterwards for BFS and counting problems.

2. Lower bound: On a ring, the heaviest edge is far away. (Dispersed start, p. 245) Information moves at most one edge per round, because it travels only inside agents and each agent moves along at most one edge per round. So after t rounds, what the agent on node y knows depends only on the part of the graph within distance t of y.

Take a ring of n nodes with distinct weights. The MST of a ring is the ring minus its heaviest edge. Look at an edge $e = \{u, v\}$ and an edge f on the opposite side of the ring, at distance about $n/2$ from u and v. Give every edge other than e and f a weight below the weight of e. If f is lighter than e, then e is the heaviest edge and is not in the MST; if f is heavier, e is in the MST.

Why it holds. Within fewer than about $n/2 - 1$ rounds the agents on u and v cannot tell these two rings apart, so they answer the same way and one answer is wrong. Hence any algorithm needs $\Omega(n)$ rounds on a ring, and $\Omega(D)$ rounds in general, where D is the diameter (largest distance between two nodes). This is our own short argument; the catalogue records no lower bound. On a ring $\Delta = 2$, so the best algorithm, $O(n \log n + \Delta \log^2 n)$, is within a $\log n$ factor of it there.

Why it matters. It shows that an MST is a global object: an edge's membership can depend on an edge at the other end of the graph.

Claude (Anthropic) was used in cataloguing the problems.

What the community started looking at

Spanning trees were the first application of agent-based leader election (Kshemkalyani et al. 2024); the question since has been **rounds, then memory**, without knowledge of graph parameters (Chand et al. 2025b).

Assumptions specific to this problem

Knowledge

Whether agents know global parameters is inherited from message passing, where nodes typically know n or an upper bound on it, and possibly Δ and D; Kshemkalyani, Kumar, Molla and Sharma set the agent-based model against this and asked for algorithms that rely on no such knowledge, "neither exact nor an upper bound" (Kshemkalyani et al. 2024). Knowing n and Δ tells an agent how long to wait: in the arbitrary-start minimal dominating set algorithm of Chand, Molla and Sivasubramaniam every robot waits $\kappa n\Delta$ rounds before leader election begins (Chand et al. 2023a), and the introductory maximal independent set algorithm assumes both n and Δ (Pattanayak et al. 2024) (as reported by Kaur et al. 2026b and Kshemkalyani et al. 2024). Kshemkalyani et al. state the trade-off: solutions without knowledge "are more resilient to network changes and device faults", but "algorithm design becomes challenging since devices may not know how long to run a procedure to guarantee a solution" (Kshemkalyani et al. 2024). Later papers remove the assumption by first dispersing with termination detection and electing a leader, after which a spanning tree lets the agents compute Δ and the largest identifier themselves (Chand et al. 2026); knowing only that largest identifier L is a weaker assumption, recorded on results rather than as a level of this axis (Chand et al. 2025b). For minimum spanning trees, Kshemkalyani et al. need no parameter (Kshemkalyani et al. 2024), and Chand, Kumar and Molla reduce memory to $O(\log n)$ bits knowing only L (Chand et al. 2025b).

Level	Value	Formal definition
0	**no knowledge**	Agents know no graph parameter (n, m, Δ, D).
1	**knows n, Δ**	Every agent knows n and Δ (upper bounds suffice where stated).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Memory) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Identifiers → Communication → Timing → Knowledge → Initial configuration → Topology.

No assumptions

+ static graph, + undirected, + distinct IDs (eponymous), + local communication, + synchronous, + dispersed start

Dispersed start STANDARD SETTING ●● p. 245

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Claude (Anthropic) was used in cataloguing the problems.

Setting	Time	Memory	p.
Dispersed start STANDARD SETTING	● unknown $O(n \log n + \Delta \log^2 n)$ vs —	● unknown $O(\log n)$ vs —	245

17.5.1 Dispersed start

Adds + static graph, + undirected, + distinct IDs (eponymous), + local communication, + synchronous, + dispersed start
STANDARD SETTING VERIFIED

From a dispersed start, an MST takes $O(n \log n + \Delta \log^2 n)$ rounds with $O(\log n)$ bits.

Assumed: graph dynamics: static graph; edge direction: undirected; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; communication: local (co-located agents communicate); timing: synchronous rounds; initial configuration: dispersed or scattered (one agent per occupied node). **Not assumed:** knowledge: no knowledge of graph parameters; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound stated.	$O(n \log n + \Delta \log^2 n)$ (Chand et al. 2025b)	● unknown
Memory	none known —	$O(\log n)$ (Chand et al. 2025b)	● unknown

Results.

- Kshemkalyani et al. (2024) [introductory]: time $O(m + n \log n)$, memory $O(n \log n)$.
- Chand et al. (2025b) [current best]: time $O(n \log n + \Delta \log^2 n)$, memory $O(\log n)$. *Assumes:* agents know L, the largest identifier.

17.6 BFS tree by agents

n agents in an anonymous graph must build a breadth-first spanning tree, without being told the root.

First posed in 2025 (Chand et al. 2025b). BFS without root knowledge in the agent-based model (SSS 2025).

Problem statement and measures

n agents with distinct identifiers (IDs) are placed on the nodes of an anonymous port-labelled graph with n nodes, m edges and maximum degree Δ. The start is rooted (all agents on one node), dispersed (one agent on each node) or arbitrary. Agents compute only by moving and by talking with agents on the same node. The agents must build a BFS tree from a root they choose, each agent knowing its parent and children: every agent (or node) learns its part of the output. **Time** is in synchronous rounds; **memory** in bits per agent.

Measures: **Time**, rounds; **Memory**, bits per agent.

A first look

Imagine n **agents** (small mobile computers), one in each room of a building, who want a plan for emergencies: one room is the exit, and every other agent should know which room next to its own is

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: dispersed; root 1 has depth 0

(b) Final: no new depth; BFS tree from root 1

Figure 17.6: BFS tree, grown layer by layer (Chand et al. 2025b): the agents are dispersed and the root (agent 1, chosen by leader election, not shown) has depth 0. In phase d (of $2\Delta = 6$ rounds) every agent of depth d visits each neighbour and comes back, one edge per round, while all others wait at home; a waiting agent without a depth that is visited takes depth $d + 1$ and the visitor's home as its parent (thick edges). The tree is complete when a phase gives no new depth.

one step closer to the exit along a shortest route. The building is a **graph**: rooms are its **nodes** and corridors are its **edges**; there are n nodes and m edges, and from here on we say only node and edge. Two nodes joined by an edge are *neighbours*. The **degree** of a node is the number of edges at it, Δ is the maximum degree, and the *diameter* D is the largest distance between two nodes. Nodes have no names, but the edges at a node are numbered $1, 2, \ldots$; these numbers are **ports**. A **BFS tree** (breadth-first search tree) with **root** s (the exit of the picture) gives every other node v a **parent**: a neighbour of v that is one step closer to s, so following parents gives a shortest path to s. In the basic version the agents start one per node (a **dispersed** start; an agent's starting node is its **home**), have distinct **identifiers** (IDs), know nothing about the graph, and must choose the root themselves; they move in synchronous **rounds** (talk to the agents on the same node, then move along at most one edge), and at the end every agent knows its parent and its children. We count rounds and bits of memory per agent. As for all problems in this family, agents can imitate message-passing algorithms by visiting neighbours, the method explained under leader election by agents.

1. Algorithm: Grow the tree layer by layer from a known root. (Chand et al. 2025b) (Dispersed start, root unknown, p. 248) Suppose the agent on the root s is already known and the agents know Δ. Time is cut into phases of 2Δ rounds. Each agent will record a **depth**, a number that is unknown at the start.

1. In phase 0 the root agent takes depth 0.
2. In phase d, every agent of depth d visits its neighbours one by one (out along one edge, back along it); all other agents stay at home.
3. An agent that has no depth yet and receives such a visit takes depth $d + 1$ and records the visitor's home as its parent (if several visitors arrive, it picks the smallest ID). The visitor records its new child.
4. Repeat with phase $d + 1$ until no agent gets a new depth.

Why it works and what it costs. An agent gets depth d exactly when its distance to s is d, so the parents form a BFS tree. During phase d only depth-d agents move, so the agents they visit are at home. Each phase lasts 2Δ rounds and there are at most D phases with new nodes, so the tree is built in $O(D\Delta)$ rounds; (Chand et al. 2025b) cite this bound for a known root. Without a known root they first build an MST to get a root and the values m and Δ, which gives $O(\min(D\Delta, m \log n) + n \log n + \Delta \log^2 n)$ rounds with $O(\log n)$ bits.

Why it matters. It is the agent version of the textbook BFS, and it shows why knowing Δ matters: it fixes how long a phase must last.

Claude (Anthropic) was used in cataloguing the problems.

2. Lower bound: A BFS tree contains an elected leader. (Dispersed start, root unknown, p. 248) When a BFS tree has been built, the agent on the root knows it has no parent, and every other agent knows it has one. So "I am the root" is exactly a leader election: one agent knows it is the leader and all others know they are not.

Why it holds. Hence building a BFS tree without a known root is at least as hard as leader election, and any lower bound for leader election also holds for BFS. This is why the best known algorithm spends $O(n \log n + \Delta \log^2 n)$ rounds electing a leader (via an MST) before the BFS itself (Chand et al. 2025b). The catalogue records no lower bound for either problem, so the time this preliminary step really needs is open.

Why it matters. It explains why leader election is the first building block in this family: trees, MIS and counting all need an agent that starts things off.

What the community started looking at

BFS trees are the next building block after leader election; the question is **rounds then memory** without knowledge of the root or of graph parameters (Chand et al. 2025b).

Assumptions specific to this problem

Knowledge

Whether agents know global parameters is inherited from message passing, where nodes typically know n or an upper bound on it, and possibly Δ and D; Kshemkalyani, Kumar, Molla and Sharma set the agent-based model against this and asked for algorithms that rely on no such knowledge, "neither exact nor an upper bound" (Kshemkalyani et al. 2024). Knowing n and Δ tells an agent how long to wait: in the arbitrary-start minimal dominating set algorithm of Chand, Molla and Sivasubramaniam every robot waits $\kappa n\Delta$ rounds before leader election begins (Chand et al. 2023a), and the introductory maximal independent set algorithm assumes both n and Δ (Pattanayak et al. 2024) (as reported by Kaur et al. 2026b and Kshemkalyani et al. 2024). Kshemkalyani et al. state the trade-off: solutions without knowledge "are more resilient to network changes and device faults", but "algorithm design becomes challenging since devices may not know how long to run a procedure to guarantee a solution" (Kshemkalyani et al. 2024). Later papers remove the assumption by first dispersing with termination detection and electing a leader, after which a spanning tree lets the agents compute Δ and the largest identifier themselves (Chand et al. 2026); knowing only that largest identifier L is a weaker assumption, recorded on results rather than as a level of this axis (Chand et al. 2025b). For BFS trees the catalogued algorithm uses no graph parameter, only L (Chand et al. 2025b).

Level	Value	Formal definition
0	**no knowledge**	Agents know no graph parameter (n, m, Δ, D).
1	**knows n, Δ**	Every agent knows n and Δ (upper bounds suffice where stated).

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Memory) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Graph dynamics → Edge direction → Identifiers → Communication → Timing → Knowledge → Initial configuration → Topology.

No assumptions

+ static graph, + undirected, + distinct IDs (eponymous), + local communication, + synchronous, + dispersed start
Dispersed start, root unknown STANDARD SETTING ●● p. 248

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Memory	p.
Dispersed start, root unknown STANDARD SETTING	● unknown $O(\min(D\Delta, m \log n) + n \log n + \Delta \log^2 n)$ vs —	● unknown $O(\log n)$ vs —	248

17.6.1 Dispersed start, root unknown

Adds + static graph, + undirected, + distinct IDs (eponymous), + local communication, + synchronous, + dispersed start STANDARD SETTING VERIFIED

A BFS tree in $O(\min(D\Delta, m \log n) + n \log n + \Delta \log^2 n)$ rounds with $O(\log n)$ bits.

Assumed: graph dynamics: static graph; edge direction: undirected; identifiers: distinct identifiers (eponymous): agents have pairwise distinct identifiers; communication: local (co-located agents communicate); timing: synchronous rounds; initial configuration: dispersed or scattered (one agent per occupied node). **Not assumed:** knowledge: no knowledge of graph parameters; topology: arbitrary connected graph.

Measure	Lower bound	Best upper bound	Status
Time	none known —	$O(\min(D\Delta, m \log n) + n \log n + \Delta \log^2 n)$ (Chand et al. 2025b)	● unknown
Memory	none known —	$O(\log n)$ (Chand et al. 2025b)	● unknown

Results.

- Chand et al. (2025b) [current best]: time $O(\min(D\Delta, m \log n) + n \log n + \Delta \log^2 n)$, memory $O(\log n)$. Introductory and current best without knowledge of the root; $O(D\Delta)$ was known with a known root.

Claude (Anthropic) was used in cataloguing the problems.

Part IV

Mobile robots in the plane

The model and its assumption axes are in Chapter 3.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 18

Formation problems

Form a geometric pattern from a starting configuration. Gathering (one point, through k-grouping), circle formation and uniform circle formation are special cases; convergence is gathering's relaxation.

Problems in this family:

- Pattern formation
- Circle formation — special case of Pattern formation
- Uniform circle formation — special case of Pattern formation
- Universal pattern formation — strengthening of Pattern formation
- Pattern sequences — generalisation of Pattern formation
- Mutual visibility — special case of Pattern formation
- k-grouping — special case of Pattern formation
- Gathering — special case of k-grouping
- Near-gathering — relaxation of Gathering
- Partitioning — special case of Pattern formation
- Convergence

18.1 Pattern formation

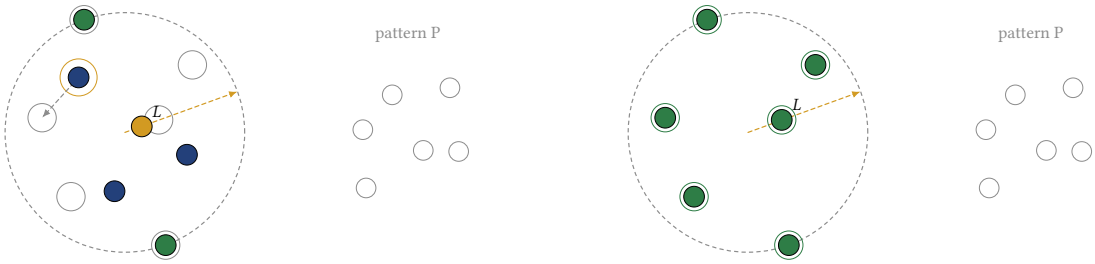
Robots start at distinct points and must form a given geometric pattern, up to similarity; which patterns are formable, and from which initial configurations?

First posed in 1999 (Suzuki and Yamashita 1999). Suzuki and Yamashita characterised the patterns that anonymous robots can form in terms of their initial configuration.

Problem statement and measures

n robots start at distinct points of the plane. Each is given the same **pattern** P, a set (or multiset) of n points in its own local coordinate system. The robots must reach a configuration similar to P (equal up to translation, rotation, uniform scaling and, without **chirality** (a common clockwise direction), reflection) and stay there. The question is for which pairs (initial configuration, pattern) this is possible under the given assumptions, and at what cost in time and colours.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).



(a) Start: SEC = unit circle; leader L fixes east

(b) Final: the robots form a copy of P

Figure 18.1: Pattern formation: n oblivious robots at distinct points are given the same pattern P in their local coordinate systems and must reach a configuration similar to P, up to translation, rotation and scaling (Suzuki and Yamashita 1999). Shown, with chirality, the plan of (Dieudonné et al. 2010) in SSYNC rounds: the smallest enclosing circle (dashed) gives the origin and unit, the ray to the leader L (the robot nearest the centre) gives east; P is placed accordingly (hollow points: the target points), and ringed (activated) robots walk to their target points one at a time without changing the circle or the leader; L moves last.

A first look

Picture drones at a light show that must arrange themselves into a picture, say a star, with no director and no radio. There are n **robots**, tiny points at different places in the plane; the positions of all robots at one instant form a **configuration**. Each robot repeats a **Look–Compute–Move cycle**: it takes a **snapshot** of where the other robots are (Look), decides on a destination point (Compute) and moves towards it (Move). The robots are identical, silent and **oblivious** (they forget everything at the end of each cycle), and each one sees the world in its own **local coordinate system**, with its own origin, unit of length and directions. An adversary called the **scheduler** decides the **activation** of each robot, that is, when it performs a cycle; in the standard version (ASYNC) every phase may take any finite time, so a robot may look while others are still moving. All robots receive the same **pattern** P, a list of n points, and must end in a configuration equal to P up to shifting, turning, resizing and mirroring, and then stop; the question is from which initial configurations this is possible.

1. Algorithm: Elect a leader, then build a shared coordinate system from the smallest enclosing circle. (Dieudonné et al. 2010) (ASYNC with common chirality, p. 257)

1. Every robot computes the **smallest enclosing circle** (SEC): the smallest circle that contains all robots. It is the same circle for everyone, whatever their local coordinate systems.
2. The centre of the SEC becomes the common origin and its radius the common unit of length.
3. The robots elect a **leader**, a robot whose position in the configuration looks different from every other robot's (for example the only robot closest to the centre). The ray from the centre through the leader becomes the common \"east\"; because all robots agree on what clockwise means (**chirality**), this also fixes a common \"north\".
4. Now all robots use the shared coordinate system, so they all place the pattern P in the same spot (scaled to the circle) and compute the same assignment of robots to points of P; the point assigned to a robot is its **target point**.
5. The robots move to their target points in a careful order that never changes the SEC and never lets another robot take over the leader's special position, so the shared coordinate system survives every move.

Why it works and what it costs. Once everyone agrees on coordinates, the task is just \"walk to your

Claude (Anthropic) was used in cataloguing the problems.

target point\"; the difficulty is step 5, because an oblivious robot recomputes everything in every cycle, and one careless move (a robot on the circle stepping inwards, say) can change the circle or the leader and with them every robot's target point. Dieudonné, Petit and Villain used this plan to show that, with chirality, pattern formation is solvable exactly when leader election is, for $n \geq 4$ robots in ASYNC (Dieudonné et al. 2010). The catalogue's current best drops chirality: arbitrary pattern formation is solvable exactly when a leader can be elected (Cicerone et al. 2019). Running time is not analysed in these papers.

Why it matters. It turned a geometric problem into a symmetry-breaking problem: forming any pattern is as hard as choosing one leader.

2. Impossibility: A symmetric initial configuration that the scheduler keeps symmetric forever. (Suzuki and Yamashita 1999) (ASYNC, no agreement on coordinates, p. 256) Place $n = 4$ robots on the corners of a square and ask for a pattern that does not look the same after a quarter turn, for example four points on a line. The adversary gives each robot a local coordinate system with its origin at the robot, the x-axis pointing to the centre of the square, and the same unit; the four systems are then copies of each other turned by 90°. Every robot now gets exactly the same snapshot. The scheduler activates all four together. Identical robots with identical snapshots choose the same move in their own local coordinate systems, which in the real plane is the same move turned by 90°. So after these cycles the robots again form a square (or all stand at its centre), and the scheduler repeats this forever. The line is never formed.

Why it holds. The same argument works for any initial configuration that looks the same after some rotation: whatever the algorithm, the robots can only form patterns that keep at least that much symmetry. Suzuki and Yamashita measured this with the **symmetricity** of a configuration (Suzuki and Yamashita 1999). The scheduler only needs to activate every robot at once, so the obstacle holds in the fully synchronous model (FSYNC) and therefore under every less synchronous scheduler. Hence, without any agreement between local coordinate systems, an arbitrary pattern cannot be formed from every initial configuration (Flocchini et al. 2008a); from a regular n-gon only a point and the regular n-gon itself can be formed (Flocchini et al. 2017).

Why it matters. It explains why every pattern-formation result is about breaking symmetry, and why leader election keeps appearing.

3. Algorithm: With a common compass nobody is symmetric. (Flocchini et al. 2008a) (ASYNC with a common compass, p. 258) Suppose all robots agree on north and east (a **common compass**), though not on origin or unit.

1. Each robot sorts all robots from west to east, and robots on the same north-south line from south to north. Since all robots agree on north and east, they all obtain the same order, even though their origins and units differ.
2. Each robot sorts the points of the pattern P in the same way and matches the i-th robot with the i-th point of P, its target point, (after placing and scaling P by a rule everyone computes alike).
3. The robots move in an order chosen so that the order of step 1, and hence the assignment of target points, does not change while they move.

Why it works and what it costs. A common compass removes the adversary's trick: no rotation or mirror image can map a set of points to itself while keeping north and east fixed, so every configuration has a first robot that all robots recognise. Flocchini, Prencipe, Santoro and Widmayer showed that with a common compass any number of robots form any pattern in ASYNC, and that agreement on one axis suffices for an odd number of robots (Flocchini et al. 2008a). The steps above are a simplified sketch that was not checked against that paper's full text; the fact that a compass gives all robots the same order is Lemma 2.1 of (Efrima and Peleg 2009).

Claude (Anthropic) was used in cataloguing the problems.

Why it matters. It shows that symmetry, not obliviousness or asynchrony, is the real obstacle to forming patterns.

What the community started looking at

The founding question was **which patterns robots can form at all** (Suzuki and Yamashita 1999): Suzuki and Yamashita characterised formable patterns by the symmetry of the starting configuration, for SSYNC robots with unbounded memory and common handedness: only a point and a regular n-gon can be formed from every starting configuration. Flocchini, Prencipe, Santoro and Widmayer then asked the question that shaped the field, **how much agreement is needed**: none is not enough, one axis is enough for an odd number of robots, two axes for all (Flocchini et al. 1999) (Flocchini et al. 2008a). The aim has been to remove assumptions one at a time (chirality, synchrony, rigidity), ending in the equivalence with leader election without chirality (Cicerone et al. 2019).

Relations to other problems

- **Derived problem: Circle formation** (special case of). A circle is a family of patterns; forming some circle is easier than forming a given pattern, and it is the precursor of uniform circle formation (Feletti et al. 2024).
- **Derived problem: Uniform circle formation** (special case of). Uniform circle formation is "an important special case of the geometric pattern formation problem" (Feletti et al. 2024): the target is the regular n-gon. It is also the special case of circle formation with equal spacing; circle formation is "its precursor" (Feletti et al. 2024).
- **Derived problem: Universal pattern formation** (strengthening of). Pattern formation asks which patterns are formable from a given start; universal pattern formation asks for all patterns from all starts, which symmetry forbids under SSYNC and FSYNC but sequential schedulers allow (Flocchini et al. 2026).
- **Derived problem: Pattern sequences** (generalisation of). Forming one pattern is the case $m = 1$; a sequence needs the robots to remember (or encode in the configuration) which pattern comes next.
- **Derived problem: Mutual visibility** (special case of). Mutual visibility asks for any pattern in general position, a family of patterns; it is the usual first step of pattern formation for opaque robots.
- **Derived problem: k-grouping** (special case of). When the groups sit at points, k-grouping forms a pattern of k points; which points, and how many robots each holds, is up to the robots. Gathering is its special case $k = 1$. Splitting needs symmetry breaking (Efrima and Peleg 2009).
- **Derived problem: Partitioning** (special case of). A partition into k groups at points is a pattern of k points, each of multiplicity n/k.
- **Related: Leader election in the plane.** Arbitrary pattern formation is solvable from a configuration exactly when leader election is, in ASYNC without chirality (Cicerone et al. 2019); with chirality and $n \geq 4$ this was shown by (Dieudonné et al. 2010).
- **Related: Shape formation in programmable matter.** "Shape formation is a basic distributed problem for systems of computational mobile entities. Intensively studied for systems of autonomous mobile robots, it has recently been investigated in the realm of programmable matter" (Di Luna et al. 2020c).

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point
Visibility

+ unlimited visibility
Opacity

+ transparent, + no faults
ASYNC, no agreement on coordinates STANDARD SETTING ●● p. 256

+ chirality
ASYNC with common chirality ●● p. 257

+ one axis
ASYNC with one common axis ●● p. 257

+ compass
ASYNC with a common compass ●● p. 258

disoriented, + randomized
Randomized, ASYNC, no agreement ●● p. 258

opaque, + no faults
Lights (persistent memory)

+ lights (LUMI)
Agreement on coordinates

+ one axis, + rigid
Opaque robots with lights, ASYNC, one axis, rigid ●● p. 259

+ compass, + rigid
Opaque robots with lights, ASYNC, compass, rigid ●● p. 259

oblivious, + compass
Opaque oblivious robots, ASYNC, compass ●● p. 260

limited visibility, + transparent, + no faults
Limited visibility ●● p. 260

fat, + unlimited visibility, + no faults
Scheduler

ASYNC, + lights (LUMI), + one axis
Opaque fat robots with lights, ASYNC, one axis ●● p. 261

+ FSYNC, + lights (LUMI), + rigid, + randomized
Fat robots with lights, FSYNC, randomized, no agreement ●● p. 261

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Claude (Anthropic) was used in cataloguing the problems.

Setting	Time	Colours	p.
ASYNC, no agreement on coordinates STANDARD SETTING	● unknown *finite* vs *impossible*	● tight 0 vs —	256
ASYNC with common chirality	● unknown *finite* vs —	● tight 0 vs —	257
ASYNC with one common axis	● unknown *finite* vs *impossible*	● tight 0 vs —	257
ASYNC with a common compass	● unknown *finite* vs —	● tight 0 vs —	258
Randomized, ASYNC, no agreement	● unknown — vs —	● tight — vs —	258
Opaque robots with lights, ASYNC, one axis, rigid	● unknown *finite* vs —	● unknown 6 vs —	259
Opaque robots with lights, ASYNC, compass, rigid	● unknown *finite* vs —	● unknown 3 vs —	259
Opaque oblivious robots, ASYNC, compass	● impossible	● impossible	260
Limited visibility	● unknown *finite* vs —	● tight 0 vs —	260
Opaque fat robots with lights, ASYNC, one axis	● unknown *finite* vs —	● unknown 10 vs —	261
Fat robots with lights, FSYNC, randomized, no agreement	● unknown $O(n) + O(\kappa \log n)$ vs —	● unknown 7 vs —	261

18.1.1 ASYNC, no agreement on coordinates

Adds + transparent, + no faults

STANDARD SETTING VERIFIED

Without any common orientation, an arbitrary pattern can be formed exactly from the configurations in which a leader can be elected.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Flocchini et al. 2008a) Time is not analysed in these papers.	*finite* (Cicerone et al. 2019)	● unknown
Colours	none known Oblivious robots: no colours.	0 (Cicerone et al. 2019)	● tight

Results.

- Flocchini et al. (2008a) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* for some initial configurations: symmetric configurations cannot be broken by oblivious deterministic robots; holds in every scheduler.
- Flocchini et al. (2008a) [introductory]: time *finite*, colours 0. Introduces arbitrary pattern formation for asynchronous oblivious robots and shows that with no common agreement an arbitrary pattern cannot be formed in general.

Claude (Anthropic) was used in cataloguing the problems.

- Cicerone et al. (2019) [current best]: time *finite*, colours 0. Characterisation: arbitrary pattern formation is solvable exactly when leader election is, in ASYNC without chirality. The paper also states that earlier proofs (Fujinaga et al. 2015, Bramas and Tixeuil 2016, the arXiv version of Dieudonné et al. 2010) are not correct.

Open questions.

Q1. Bound the number of epochs needed by an arbitrary pattern formation algorithm in ASYNC; no time analysis is recorded.

Notes. Patterns are formed up to translation, rotation, scaling and reflection, starting from distinct positions. The four-robot case needed a separate geometric invariant (Bramas and Tixeuil 2018). The point (gathering) and the regular polygon (uniform circle) are the only patterns formable from every initial configuration (Flocchini et al. 2017).

18.1.2 ASYNC with common chirality

Adds + chirality **to** ASYNC, no agreement on coordinates (p. 256) VERIFIED

With common chirality, pattern formation and leader election are equivalent for $n \geq 4$.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; agreement on coordinates: common chirality (same clockwise orientation). **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not analysed.	*finite* (Dieudonné et al. 2010)	● unknown
Colours	none known No colours.	0 (Dieudonné et al. 2010)	● tight

Results.

- Dieudonné et al. (2010) [current best]: time *finite*, colours 0. *Assumes:* $n \geq 4$. Introductory and current best for the equivalence with chirality.
- Fujinaga et al. (2015) [disputed proof]: time *finite*, colours 0. Patterns formable by oblivious ASYNC robots are exactly those formable by non-oblivious FSYNC robots, except a point of multiplicity 2. Its proof is reported as not correct by (Cicerone et al. 2019).

Notes. Disputed: the correctness of the proof of (Fujinaga et al. 2015) is questioned by (Cicerone et al. 2019); it is shown as disputed and never as the current best.

18.1.3 ASYNC with one common axis

Adds + one axis **to** ASYNC, no agreement on coordinates (p. 256) VERIFIED

With a common North and chirality, any odd number of robots forms any pattern; an even number cannot, in the worst case.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; agreement on coordinates: agreement on one axis (no common chirality). **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Flocchini et al. 2008a) Time not analysed.	*finite* (Flocchini et al. 2008a)	• unknown
Colours	none known No colours.	0 (Flocchini et al. 2008a)	• tight

Results.

- Flocchini et al. (2008a) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* even n, for some initial configurations.
- Flocchini et al. (2008a) [current best]: time *finite*, colours 0. *Assumes:* odd n. Introductory and current best.

18.1.4 ASYNC with a common compass

Adds + compass **to** ASYNC, no agreement on coordinates (p. 256) VERIFIED

If robots agree on both axes, any number of robots forms any pattern.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; agreement on coordinates: agreement on both axes (common compass). **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not analysed.	*finite* (Flocchini et al. 2008a)	• unknown
Colours	none known No colours.	0 (Flocchini et al. 2008a)	• tight

Results.

- Flocchini et al. (2008a) [current best]: time *finite*, colours 0. Introductory and current best.

18.1.5 Randomized, ASYNC, no agreement

Adds disoriented, + randomized **to** ASYNC, no agreement on coordinates (p. 256) VERIFIED

Randomness breaks symmetry: with one random bit per cycle, $n \geq 5$ robots form any pattern except gathering.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; randomness: randomized. **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not analysed.	—	• unknown
Colours	none known No colours.	—	• tight

Results.

- Bramas and Tixeuil (2016) [disputed proof]: time *finite*, colours 0. *Assumes:* $n \geq 5$; one random bit per cycle. Patterns with multiplicity points are allowed, except gathering. Its proof is reported as not correct by (Cicerone et al. 2019).

Notes. Yamauchi and Yamashita gave a randomized algorithm with chirality (DISC 2014, not yet in the bibliography). **Disputed:** the proof of (Bramas and Tixeuil 2016) is questioned by (Cicerone et al. 2019); no undisputed result is recorded for this setting.

18.1.6 Opaque robots with lights, ASYNC, one axis, rigid

Adds + one axis, + rigid VERIFIED

Opaque point robots with lights that agree on one axis form any pattern from the starts the paper characterises, with 6 colours.

Assumed: body: point robots; visibility: unlimited; faults: no faults; lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: agreement on one axis (no common chirality); movement: rigid: every move reaches its destination. **Not assumed:** opacity: opaque: collinear robots hide each other; scheduler: asynchronous (ASYNC); multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Bose et al. 2021)	• unknown
Colours	none known No lower bound beyond trivial; the minimum is open.	6 (Bose et al. 2021)	• unknown

Results.

- Bose et al. (2021) [current best]: time *finite*, colours 6. Introductory and current best. Formation fails from starts symmetric about a robot-free line parallel to the agreed axis.

Open questions.

Q1. The minimum number of colours, and formation with chirality only or with no agreement (Bose et al. 2021).

Notes. Checked against the full text.

18.1.7 Opaque robots with lights, ASYNC, compass, rigid

Adds + compass, + rigid VERIFIED

With a common compass, opaque robots with lights form any pattern with 3 colours.

Assumed: body: point robots; visibility: unlimited; faults: no faults; lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: agreement on both axes (common compass); movement: rigid: every move reaches its destination. **Not assumed:** opacity: opaque: collinear robots hide each other; scheduler: asynchronous (ASYNC); multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Bose et al. 2021)	• unknown
Colours	none known The minimum is open.	3 (Bose et al. 2021)	• unknown

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Bose et al. (2021) [current best]: time *finite*, colours 3. Introductory and current best.

Notes. Checked against the full text.

18.1.8 Opaque oblivious robots, ASYNC, compass

Adds oblivious, + compass VERIFIED

Without lights, opaque robots cannot form arbitrary patterns in ASYNC, even with a common compass.

Assumed: body: point robots; visibility: unlimited; faults: no faults; agreement on coordinates: agreement on both axes (common compass). **Not assumed:** opacity: opaque: collinear robots hide each other; scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Bose et al. 2021) Unsolvable without lights.	—	• impossible
Colours	none known Needs lights.	—	• impossible

Results.

- Bose et al. (2021) [lower bound]: time *impossible*. *Bottleneck:* obstruction. *Holds when:* Theorem 1: oblivious opaque point robots, ASYNC; holds even with a common compass.

Notes. Checked against the full text.

18.1.9 Limited visibility

Adds limited visibility, + transparent, + no faults VERIFIED

With limited visibility, oblivious robots form a pattern under the usual symmetry condition if they start within diameter 1; the general case is open.

Assumed: body: point robots; opacity: transparent: every robot sees every robot within range; faults: no faults. **Not assumed:** visibility: limited: robots see only within a fixed radius; scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not analysed.	*finite* (Hahn et al. 2024)	• unknown
Colours	none known No colours.	0 (Hahn et al. 2024)	• tight

Results.

- Hahn et al. (2024) [current best]: time *finite*, colours 0. *Assumes:* initial diameter at most 1 (the visibility range). Introductory and current best for oblivious robots.

Open questions.

Q1. Arbitrary pattern formation by oblivious robots with limited visibility from a general (connected) initial configuration (Hahn et al. 2024).

18.1.10 Opaque fat robots with lights, ASYNC, one axis

Adds ASYNC, + lights (LUMI), + one axis VERIFIED

Opaque fat robots with lights that agree on one axis form any pattern from every start not symmetric about a robot-free line parallel to that axis, with 10 colours.

Assumed: visibility: unlimited; faults: no faults; lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: agreement on one axis (no common chirality). **Not assumed:** body: fat robots (unit discs); opacity: opaque: collinear robots hide each other; scheduler: asynchronous (ASYNC); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Bose et al. 2020)	• unknown
Colours	none known The minimum is open.	10 (Bose et al. 2020)	• unknown

Results.

- Bose et al. (2020) [current best]: time *finite*, colours 10. Introductory and current best. Theorem 1: solvable if and only if the start is not symmetric about a line parallel to the agreed axis through no robot centre. $\Theta(n)$ moves in total (Theorem 8), analysed under SSYNC with rigid moves.

Notes. Checked against the full text (arXiv 1910.02706).

18.1.11 Fat robots with lights, FSYNC, randomized, no agreement

Adds + FSYNC, + lights (LUMI), + rigid, + randomized VERIFIED

Randomized fat robots with lights and no common orientation form any pattern in FSYNC in $O(n)+O(\kappa \log n)$ rounds with probability at least $1-n^{-\kappa}$, with 7 colours.

Assumed: visibility: unlimited; faults: no faults; scheduler: fully synchronous (FSYNC); lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; movement: rigid: every move reaches its destination; randomness: randomized. **Not assumed:** body: fat robots (unit discs); opacity: opaque: collinear robots hide each other; agreement on coordinates: no agreement on local coordinate systems; multiplicity detection: none: a point with several robots looks like one robot; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound recorded.	$O(n)+O(\kappa \log n)$ (Alsaedi et al. 2025)	• unknown
Colours	none known The minimum is open.	7 (Alsaedi et al. 2025)	• unknown

Results.

- Alsaedi et al. (2025) [current best]: time $O(n)+O(\kappa \log n)$, colours 7. *Assumes: n* unknown; the pattern is scaled by a constant (8 colours without scaling). Introductory and current best (Theorems 11, 16, 20, 21); with probability at least $1-n^{-\kappa}$. Earlier versions used 10 and 11 colours.

Claude (Anthropic) was used in cataloguing the problems.

Notes. With $O(1)$ words of $O(\log n)$ bits of persistent memory instead of lights, the same bound holds (Alsaedi, Gudmundsson and van Renssen, Computational Geometry 2025). Checked against the full text of (Alsaedi et al. 2025).

18.2 Circle formation

Robots must reach positions on a common circle.

First posed in 2002 (Défago and Konagaya 2002).

Problem statement and measures

n robots start at distinct points and must reach a configuration in which all of them lie on one circle, at distinct points, and stay there. Uniform circle formation additionally asks for a regular n-gon.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).



(a) Start: the smallest enclosing circle is fixed

(b) Final: all robots on the circle (arrows: moves; dashed circles: starting positions)

Figure 18.2: Circle formation: robots at distinct points must place themselves at distinct points of one common circle and stay there; the spacing need not be uniform (Défago and Konagaya 2002). Shown: the first phase of (Flocchini et al. 2017) in SSYNC rounds: every robot computes the smallest enclosing circle (dashed), which never changes; a ringed (activated) robot inside moves out along the ray from the centre, possibly stopping early; the robot at the centre first steps off it.

A first look

Think of dancers who must spread out on one common circle, each at a different spot on it, without anyone drawing the circle first. There are n **robots**, points at distinct places in the plane; their positions form the **configuration**. Each robot repeats a **Look–Compute–Move cycle**: it takes a **snapshot** of where the other robots are (Look), decides on a destination point (Compute) and moves towards it (Move). They are identical, silent, **oblivious** (no memory between cycles), and each uses its own **local coordinate system** (origin, axes, unit of length and handedness). A **scheduler** (an adversary) decides the **activation** of robots, that is, when each performs a cycle; in the basic version it is ASYNC, so each phase may take any finite time, and every move is a **non-rigid move**: the adversary may stop a robot after it has travelled some minimum distance δ. The robots must reach a configuration in which all of them lie on one circle, at distinct points (not necessarily evenly spaced), and stay there.

Claude (Anthropic) was used in cataloguing the problems.

1. Algorithm: Use the smallest enclosing circle and walk out to it. (Flocchini et al. 2017) (Circle formation, p. 264)

1. Every robot computes the **smallest enclosing circle** (SEC) of all robots. It is unique and does not depend on any local coordinate system, so all robots compute the same circle.
2. A robot already on the circle stays where it is.
3. A robot inside the circle moves straight outwards, along the ray from the centre through itself, to the circle.
4. A robot exactly at the centre has no ray; it first steps a little away from the centre.
5. If several robots lie on the same ray, only the outermost one moves out; the others first step sideways onto free rays, so that no two robots ever land on the same point.

Why it works and what it costs. Robots that move from inside onto the circle never change the smallest enclosing circle, so the circle stays fixed while robots move at different speeds and act on outdated snapshots. Each robot inside needs only finitely many moves, since each move covers at least δ or reaches the circle. This is the first phase of the uniform circle algorithm of Flocchini, Prencipe, Santoro and Viglietta, which forms even a regular n-gon (hence a circle) in ASYNC without **chirality** (agreement on the clockwise direction) (Flocchini et al. 2017), the catalogue's current best. Steps 4 and 5 are simplified here; the paper handles robots at the centre and robots on a common ray as separate cases.

Why it matters. The smallest enclosing circle is the standard common reference for disoriented robots, reused in pattern formation, gathering and leader election.

2. Algorithm: Move only inside your own zone, then slide along the circle. (Défago and Souissi 2008) (Circle formation, SSYNC, p. 265)

1. Compute the smallest enclosing circle and use its centre as the origin of polar coordinates.
2. A robot at the centre steps off it, by less than the distance of any other robot from the centre.
3. Otherwise, find your two neighbours in angular order around the centre and draw the rays halfway (by angle) to each of them. The region between these two rays is your **exclusive zone** for this cycle: no other robot will move into it.
4. A robot inside the circle moves outwards within its zone towards the circle; a robot on the circle slides along it towards the angle halfway between its neighbours, without ever leaving an empty arc larger than a half circle (which would shrink the circle).

Why it works and what it costs. Exclusive zones guarantee that two robots never end up at the same point. That matters: two robots on one point get the same snapshot, so a scheduler that always activates them together, with suitable local coordinate systems, can keep them together forever. Défago and Souissi show that in SSYNC the robots reach the circle and then converge towards even spacing without necessarily reaching it (Défago and Souissi 2008); reaching exactly even spacing needed later work (see uniform circle formation).

Why it matters. It introduced the exclusive-zone trick for avoiding inseparable robots and the "slide towards the middle of your neighbours" rule for even spacing.

What the community started looking at

Circle formation was first proposed as a way to reach a uniform circle by oblivious robots without a common orientation (Défago and Konagaya 2002); the first algorithms formed a circle and only

Claude (Anthropic) was used in cataloguing the problems.

converged towards uniformity (Défago and Souissi 2008). The question was **whether disoriented, oblivious robots can form a circle at all**, under which scheduler.

Relations to other problems

- **Special case of Pattern formation.** A circle is a family of patterns; forming some circle is easier than forming a given pattern, and it is the precursor of uniform circle formation (Feletti et al. 2024).
- **Related: Uniform circle formation.** A uniform circle is a circle, so every uniform-circle algorithm forms a circle.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point, + unlimited visibility, + transparent, + no faults

Circle formation STANDARD SETTING ● ● p. 264

+ SSYNC

Circle formation, SSYNC ● ● p. 265

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
Circle formation STANDARD SETTING	● unknown *finite* vs —	● tight 0 vs —	264
Circle formation, SSYNC	● unknown *finite* vs —	● tight 0 vs —	265

18.2.1 Circle formation

Adds + point, + unlimited visibility, + transparent, + no faults STANDARD SETTING VERIFIED

Robots must place themselves on a common circle, not necessarily evenly spaced; solved in ASYNC without chirality through uniform circle formation.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not analysed.	*finite* (Flocchini et al. 2017)	● unknown
Colours	none known No colours.	0 (Flocchini et al. 2017)	● tight

Results.

- Flocchini et al. (2017) [current best]: time *finite*, colours 0. Introductory for ASYNC and current best: Theorem 6.1 forms a uniform circle, hence a circle, in ASYNC without chirality and with non-rigid moves.

Notes. The first circle formation papers are semi-synchronous; see the SSYNC setting. Checked against the full text of (Flocchini et al. 2017).

18.2.2 Circle formation, SSYNC

Adds + SSYNC **to** Circle formation (p. 264) VERIFIED

In SSYNC the first algorithms formed a circle and then converged towards a uniform one.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC). **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not analysed.	*finite* (Défago and Souissi 2008)	● unknown
Colours	none known No colours.	0 (Défago and Souissi 2008)	● tight

Results.

- Défago and Konagaya (2002) [introductory]: time *finite*, colours 0. No chirality needed; proofs only sketched (as stated by Défago and Souissi 2008).
- Défago and Souissi (2008) [current best]: time *finite*, colours 0. *Assumes:* chirality; a maximum step length per robot instead of adversarial non-rigid moves. Forms a circle and converges towards a uniform one without reaching it.

Notes. Défago and Souissi call their robots disoriented although they assume chirality; Flocchini et al. use disoriented to exclude chirality. Checked against the full text of (Défago and Souissi 2008); (Défago and Konagaya 2002) was not reachable.

18.3 Uniform circle formation

Robots must form a regular n-gon: a circle with equal spacing.

First posed in 2006 (Dieudonné et al. 2008). The first deterministic solution (SSS 2006); circle formation with convergence towards uniformity predates it (Défago and Konagaya 2002).

Claude (Anthropic) was used in cataloguing the problems.

Problem statement and measures

n robots start at distinct points and must reach the vertices of a regular n-gon (in any position and size) and stay there. **Time** is in epochs (ASYNC: intervals in which every robot completes at least one full Look–Compute–Move cycle) or rounds; **colours** count light colours in the luminous model.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).



(a) Start: inner robots move out to the fixed circle

(b) Final: a regular hexagon (arrows: moves; dashed circles: starting positions)

Figure 18.3: Uniform circle formation: n oblivious robots at distinct points must occupy the vertices of a regular n-gon, of any position and size, and stay there (Flocchini et al. 2017). Shown in SSYNC rounds (ringed robots are activated), simplified from (Flocchini et al. 2017): the smallest enclosing circle (dashed) never changes; robots inside move out to it, then robots on it move to the vertices of a regular hexagon inscribed in it (hollow).

A first look

Picture guards who must stand evenly spaced around a circular fence that nobody has built: the circle and the spacing must come from the guards themselves. There are n **robots**, points at distinct places in the plane; their positions form the **configuration**. Each robot repeats a **Look–Compute–Move cycle**: it takes a **snapshot** of where the other robots are (Look), decides on a destination point (Compute) and moves towards it (Move). They are identical, silent, **oblivious** (no memory between cycles), and each uses its own **local coordinate system**, so they agree on no direction and no handedness. The **scheduler** (an adversary) decides the **activation** of robots, that is, when each performs a cycle, and is ASYNC: each phase may take any finite time. Every move is a **non-rigid move**: a robot may be stopped after a minimum distance δ. The robots must reach the **vertices** of a **regular n-gon** (a polygon with n equal sides and equal angles), one robot on each vertex, of any size and position, and stay there.

1. Algorithm: Fix the smallest enclosing circle, reach it, then let symmetric robots move as one. (Flocchini et al. 2017) (ASYNC, oblivious, disoriented, p. 269)

1. Compute the **smallest enclosing circle** (SEC) of all robots; every robot computes the same circle. All moves are chosen so that this circle does not change.
2. Robots inside the circle move out to it, as in circle formation.
3. With all robots on the circle, group them into **analogy classes**: two robots are analogous if some symmetry of the whole configuration maps one to the other.
4. The scheduler could make analogous robots act identically anyway, so the algorithm makes each class always move together, deliberately and symmetrically.

Claude (Anthropic) was used in cataloguing the problems.

5. The robots compute the vertices of a regular n-gon on the circle and move to them class by class. Special configurations that are already close to regular (pre-regular, biangular) get their own procedure.

Why it works and what it costs. The overall plan is simple; the difficulty is that asynchronous robots act on outdated snapshots, so the analogy classes and destinations that one robot computes may already be stale. Flocchini, Prencipe, Santoro and Viglietta show that oblivious, disoriented robots with non-rigid moves form a regular n-gon in ASYNC from every initial configuration at distinct points, for every n (Flocchini et al. 2017); the case $n = 4$ is solved in (Mamino and Viglietta 2016). Time is not analysed; for opaque robots with lights, $O(1)$ epochs and $O(1)$ colours suffice (Feletti et al. 2024).

Why it matters. It closed a long-standing question and showed that, for pattern formation, asynchrony, chirality and rigid moves do not add computational power.

2. Impossibility: Two robots on the same point can never be separated. (Flocchini et al. 2017) (ASYNC, oblivious, disoriented, p. 269) Suppose two robots start at the same point. They get exactly the same snapshot. The adversary gives them identical local coordinate systems and the scheduler always activates them together. Identical robots with identical snapshots and identical local coordinate systems compute the same destination, so they move together and are still on one point after every cycle. A regular n-gon has n distinct vertices, so it is never formed.

Why it holds. This is why the problem asks for distinct starting points; Flocchini et al. note that with two robots on one point the problem is \"easily seen to be unsolvable\" (Flocchini et al. 2017). The same trap is the reason careful algorithms never let two robots meet while they move.

Why it matters. It is the simplest example of a symmetry that oblivious deterministic robots can never break.

3. Impossibility: Why only the point and the regular polygon can be formed from every initial configuration. (Suzuki and Yamashita 1999) (ASYNC, oblivious, disoriented, p. 269) Start the robots on a regular n-gon. Give every robot a local coordinate system whose x-axis points to the centre, all with the same unit. All robots get the same snapshot; if the scheduler activates all of them together, they all make the same move up to rotation, so the configuration stays a regular n-gon (or collapses to its centre). So from this initial configuration the only formable patterns are the regular n-gon and the single point.

Why it holds. Suzuki and Yamashita showed that the patterns formable from an initial configuration depend on its **symmetricity**, its amount of rotational symmetry (Suzuki and Yamashita 1999); the regular n-gon has the most. Hence the point (gathering) and the regular n-gon (uniform circle) are the only patterns that might be formable from every initial configuration, and both are (Flocchini et al. 2017).

Why it matters. It explains why the uniform circle is singled out among all patterns: it is one of only two universally formable ones.

What the community started looking at

Uniform circle formation was first asked as **solvability**: can disoriented oblivious robots form a regular polygon? It was solved for SSYNC (Dieudonné et al. 2008) and then for ASYNC without chirality or rigidity (Flocchini et al. 2017) (Mamino and Viglietta 2016). Only after feasibility was settled did the community turn to **time and colours** for luminous opaque robots (Feletti et al. 2023b) (Pattanayak and Sharma 2024), reaching $O(1)$ epochs with $O(1)$ colours (Feletti et al. 2024).

Claude (Anthropic) was used in cataloguing the problems.

Relations to other problems

- **Special case of Pattern formation.** Uniform circle formation is "an important special case of the geometric pattern formation problem" (Feletti et al. 2024): the target is the regular n-gon. It is also the special case of circle formation with equal spacing; circle formation is "its precursor" (Feletti et al. 2024).
- **Related: Circle formation.** A uniform circle is a circle, so every uniform-circle algorithm forms a circle.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point, + unlimited visibility
Opacity

+ transparent, + no faults
ASYNC, oblivious, disoriented STANDARD SETTING ●● p. 269

+ SSYNC
SSYNC, oblivious ●● p. 269

opaque, + no faults
Scheduler

ASYNC, + lights (LUMI), + rigid
Opaque robots with lights, ASYNC ●● p. 270

+ FSYNC, + lights (LUMI), + rigid
Opaque robots with lights, FSYNC ●● p. 271

fat, + unlimited visibility, + transparent, + no faults, + SSYNC, + one axis, + rigid
Transparent fat robots, SSYNC ●● p. 271

fat, limited visibility, opaque, + no faults, + compass, + rigid
Opaque fat robots, ASYNC, limited visibility, compass ●● p. 272

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
ASYNC, oblivious, disoriented STANDARD SETTING	● unknown *finite* vs *impossible*	● tight 0 vs —	269
SSYNC, oblivious	● unknown *finite* vs —	● tight 0 vs —	269
Opaque robots with lights, ASYNC	● tight $O(1)$ vs $\Omega(1)$	● tight $O(1)$ vs $\Omega(1)$	270

Claude (Anthropic) was used in cataloguing the problems.

Setting	Time	Colours	p.
Opaque robots with lights, FSYNC	● tight 3 vs —	● tight $O(1)$ vs —	271
Transparent fat robots, SSYNC	● unknown *finite* vs —	● tight 0 vs —	271
Opaque fat robots, ASYNC, limited visibility, compass	● unknown *finite* vs —	● tight 0 vs —	272

18.3.1 ASYNC, oblivious, disoriented

Adds + transparent, + no faults STANDARD SETTING VERIFIED

Oblivious, disoriented, non-rigid robots form a regular n-gon from any configuration of distinct points, for every n.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Flocchini et al. 2017)	*finite* (Flocchini et al. 2017)	● unknown
	No time analysis of oblivious ASYNC uniform circle formation is recorded.		
Colours	none known	0 (Flocchini et al. 2017)	● tight
	No colours.		

Results.

- Flocchini et al. (2017) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* if two robots start at the same point.
- Flocchini et al. (2017) [current best]: time *finite*, colours 0. *Assumes:* $n \neq 4$. Introductory for ASYNC and current best. The case $n = 4$ (square formation) is solved by (Mamino and Viglietta 2016).
- Mamino and Viglietta (2016) [also]: time *finite*, colours 0. *Assumes:* $n = 4$. Completes the picture: synchrony, chirality and rigidity are computationally irrelevant for pattern formation.

Open questions.

Q1. Can oblivious robots without lights form a uniform circle in $O(1)$ epochs, with rigid or non-rigid moves? (Feletti et al. 2024)

Notes. Uniform circle and point are the only patterns formable from every initial configuration (Flocchini et al. 2017).

18.3.2 SSYNC, oblivious

Adds + SSYNC **to** ASYNC, oblivious, disoriented (p. 269) VERIFIED

The first deterministic uniform circle formation, for $n \geq 5$ semi-synchronous robots.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC). **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move

Claude (Anthropic) was used in cataloguing the problems.

after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not analysed.	*finite* (Dieudonné et al. 2008)	● unknown
Colours	none known No colours.	0 (Dieudonné et al. 2008)	● tight

Results.

- Dieudonné et al. (2008) [introductory]: time *finite*, colours 0. *Assumes:* $n \geq 5$. Concentric circles; first deterministic solution.

Notes. Dieudonné and Petit later solved $n \neq 4$ and $n = 4$ in SSYNC (SIROCCO 2007, ISAAC 2008; not yet in the bibliography).

18.3.3 Opaque robots with lights, ASYNC

Adds ASYNC, + lights (LUMI), + rigid VERIFIED

With lights, opaque robots form a uniform circle in $O(1)$ epochs with $O(1)$ colours, which is optimal in both.

Assumed: body: point robots; visibility: unlimited; faults: no faults; lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; movement: rigid: every move reaches its destination. **Not assumed:** opacity: opaque: collinear robots hide each other; scheduler: asynchronous (ASYNC); agreement on coordinates: no agreement on local coordinate systems; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(1)$ (Feletti et al. 2024) $\Theta(1)$ epochs.	$O(1)$ (Feletti et al. 2024)	● tight
Colours	$\Omega(1)$ (Feletti et al. 2024) $\Theta(1)$ colours; the exact minimum is open.	$O(1)$ (Feletti et al. 2024)	● tight

Results.

- Feletti et al. (2024) [lower bound]: time $\Omega(1)$. *Bottleneck:* trivial. *Holds when:* trivial: at least one epoch.
- Feletti et al. (2024) [lower bound]: colours $\Omega(1)$. *Bottleneck:* trivial. *Holds when:* trivial: at least one colour.
- Feletti et al. (2023b) [introductory]: time $O(n)$, colours $O(1)$. First ASYNC result for opaque luminous robots (ICTCS 2022; journal 2023); $O(1)$ time in FSYNC and SSYNC.
- Feletti et al. (2023a) [also]: time $O(\log n)$, colours $O(1)$.
- Pattanayak and Sharma (2024) [also]: time $O(x)$, colours $O(n^{1/2^x})$. A time–colour trade-off for $x \in [1, O(\log \log n)]$: $O(1)$ time with $O(\sqrt{n})$ colours, or $O(\log \log n)$ time with $O(1)$ colours. Abstract only.
- Feletti et al. (2024) [current best]: time $O(1)$, colours $O(1)$. Asymptotically optimal in time and colours.

Open questions.

Q1. What is the exact minimum number of colours for $O(1)$-epoch uniform circle formation in ASYNC? (Feletti et al. 2024)

Q2. Uniform circle formation with non-rigid moves in the luminous opaque model (Feletti et al. 2024).

Claude (Anthropic) was used in cataloguing the problems.

Notes. Robots are punctiform, disoriented and do not know n; collisions are avoided. The only lower bounds are trivial. Checked against the full texts of (Feletti et al. 2023a) and (Feletti et al. 2024).

18.3.4 Opaque robots with lights, FSYNC

Adds + FSYNC, + lights (LUMI), + rigid VERIFIED

In FSYNC, opaque luminous robots form a uniform circle in 3 cycles with $O(1)$ colours.

Assumed: body: point robots; visibility: unlimited; faults: no faults; scheduler: fully synchronous (FSYNC); lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; movement: rigid: every move reaches its destination. **Not assumed:** opacity: opaque: collinear robots hide each other; agreement on coordinates: no agreement on local coordinate systems; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Constant.	3 (Feletti et al. 2023b)	• tight
Colours	none known $\Theta(1)$.	$O(1)$ (Feletti et al. 2023b)	• tight

Results.

- Feletti et al. (2018) [introductory]: time $O(1)$, colours $O(1)$. First paper on the opaque luminous model; as described by (Feletti et al. 2024).
- Feletti et al. (2023b) [current best]: time 3, colours $O(1)$. Three cycles.

Notes. In SSYNC the same paper takes $O(1)$ rounds. The ICTCS 2022 version was read in full.

18.3.5 Transparent fat robots, SSYNC

Adds fat, + unlimited visibility, + transparent, + no faults, + SSYNC, + one axis, + rigid VERIFIED

Transparent fat robots that agree on one axis form a regular n-gon of given side in SSYNC.

Assumed: visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); agreement on coordinates: agreement on one axis (no common chirality); movement: rigid: every move reaches its destination. **Not assumed:** body: fat robots (unit discs); lights (persistent memory): oblivious, no light (OBLOT); multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Mondal and Chaudhuri 2017)	• unknown
Colours	none known No colours.	0 (Mondal and Chaudhuri 2017)	• tight

Results.

- Mondal and Chaudhuri (2017) [current best]: time *finite*, colours 0. *Assumes:* n and the side length are known; only the direction of the y axis is shared, without chirality. Introductory and current best (Theorem 1). A preprint.

Notes. The paper assumes agreement on one axis without chirality, which is the catalogue's one-axis value. Checked against the full text.

Claude (Anthropic) was used in cataloguing the problems.

18.3.6 Opaque fat robots, ASYNC, limited visibility, compass

Adds fat, limited visibility, opaque, + no faults, + compass, + rigid VERIFIED

Opaque fat robots with limited visibility and a common coordinate system form a uniform circle of given centre and radius in ASYNC.

Assumed: faults: no faults; agreement on coordinates: agreement on both axes (common compass); movement: rigid: every move reaches its destination. **Not assumed:** body: fat robots (unit discs); visibility: limited: robots see only within a fixed radius; opacity: opaque: collinear robots hide each other; scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Mondal et al. 2020)	● unknown
Colours	none known No colours.	0 (Mondal et al. 2020)	● tight

Results.

- Mondal et al. (2020) [current best]: time *finite*, colours 0. *Assumes:* common origin and axes; the circle's centre and radius are given. Introductory and current best (Theorem 1). A preprint.

Notes. Checked against the full text.

18.4 Universal pattern formation

> *Robots must be able to form any pattern from any initial configuration, not only the ones symmetry allows.*

First posed in 2025 (Flocchini et al. 2026). Introduced at SIROCCO 2025.

Problem statement and measures

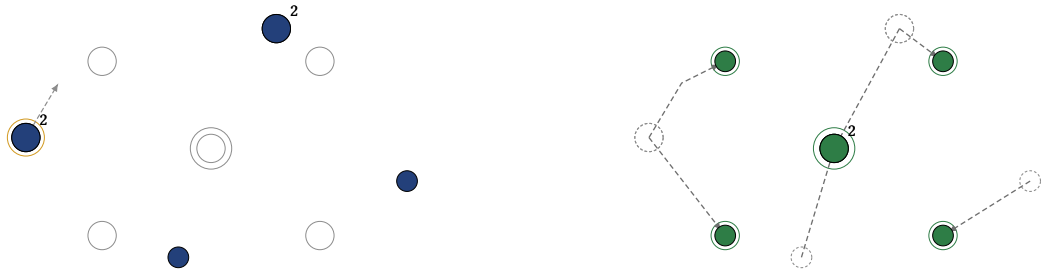
From every initial configuration of n robots (several robots may start on the same point, forming a multiplicity Flocchini et al. 2026), the robots must form every given **pattern** of n points (a multiset of points given to every robot; multiplicities allowed). With lights, universal dancing asks for any sequence of patterns.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).

A first look

Imagine a troupe that must be able to form every shape the director might ask for, from wherever its members happen to stand, even if some of them are standing on top of each other. There are n **robots**, points in the plane; their positions form the **configuration**, and a point that holds two or more robots is a **multiplicity**. Each robot repeats a **Look–Compute–Move cycle**: it takes a **snapshot** of where the other robots are (Look), decides on a destination point (Compute) and moves towards it (Move). They are identical, silent and **oblivious** (no memory between cycles), and each uses its own **local coordinate system**, so they agree on no direction. The **scheduler**, an adversary, decides the **activation** of robots,

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: 6 robots on 4 points; pattern has 5

(b) Final: the pattern, with its multiplicity (arrows: moves; dashed circles: starting positions)

Figure 18.4: Universal pattern formation: from every configuration of n robots, possibly with several robots on one point (a multiplicity), the robots must be able to form every given pattern of n points, including patterns with multiplicities (Flocchini et al. 2026). Shown, simplified, the plan of (Flocchini et al. 2026) under the sequential scheduler (one ringed robot is activated per round): a robot leaves a multiplicity until the robots occupy as many points as the pattern (hollow), then the points of the pattern are filled one by one; the last robot creates the pattern's multiplicity.

that is, when each performs a cycle; here it is **sequential**: time runs in **rounds**, in each round it activates exactly one robot, and every robot is activated again and again. Universal pattern formation asks for one algorithm that, from every initial configuration, forms every given **pattern** of n points (a list of points that every robot is given, to be matched up to position, rotation and size; it may contain multiplicities) and then stops.

1. Algorithm: One robot at a time: create enough occupied points, agree on a map, fill it in. (Flocchini et al. 2026) (Sequential schedulers, p. 275)

1. **Initialization.** If the robots occupy fewer distinct points than the pattern has, robots leave multiplicities one at a time until there are enough occupied points.
2. Scale the pattern so that its **smallest enclosing circle** (the smallest circle containing all its points) matches the smallest enclosing circle of the configuration.
3. **Leader configuration.** Move robots so that all of them agree on how the pattern is laid over the robots' positions (a common map from robots to pattern points).
4. **Partial formation.** Fill the empty points of the pattern one by one in an agreed order, moving along detours that never pass over another robot.
5. **Finalization.** When only one robot is off the pattern, the last robots move to their places.

Why it works and what it costs. Under a sequential scheduler only one robot moves at a time, so the moving robot is automatically different from all others; in particular, a multiplicity can be split, since only one of its robots moves. Flocchini, Navarra, Pattanayak, Piselli and Santoro show that this gives oblivious robots universal pattern formation with no extra assumption, except for the single point (gathering), which needs **weak multiplicity detection** (a robot sees whether a point holds more than one robot), necessary and sufficient (Flocchini et al. 2026). The steps above are their high-level plan; the actual algorithm handles many special cases, and patterns with fewer than 5 points separately. Time is not analysed.

Why it matters. It shows that sequential activation is more than a way to elect a leader: it gives power that even a leader with full synchrony does not have.

Claude (Anthropic) was used in cataloguing the problems.

2. Impossibility: Fully synchronous robots cannot split a multiplicity. (Flocchini et al. 2026) (Sequential schedulers, p. 275) Take three robots: two on one point, the third alone elsewhere. Under FSYNC every robot is activated in every round. The two robots on the multiplicity get the same snapshot; even if they have identical local coordinate systems, move rigidly, can count robots on each point, and the third robot is a recognisable leader, the two compute the same destination and move together. They stay on one point forever, so no pattern with three distinct points is ever formed.

Why it holds. This is Theorem 1 of (Flocchini et al. 2026): universal pattern formation is unsolvable under FSYNC for $n \geq 3$, even with all these extra capabilities, and hence also under SSYNC and ASYNC, where the scheduler may choose to activate every robot. Gathering alone, in contrast, is solvable under FSYNC with no extra capability.

Why it matters. It shows that the fully synchronous and the sequential schedulers are incomparable: each solves something the other cannot.

What the community started looking at

Universal pattern formation asked whether a **scheduler alone** can break the symmetry that blocks pattern formation: under sequential schedulers it can, except for a single point (Flocchini et al. 2026); with lights, robots can then perform any sequence of patterns (Feletti et al. 2026).

Relations to other problems

- **Strengthening of Pattern formation.** Pattern formation asks which patterns are formable from a given start; universal pattern formation asks for all patterns from all starts, which symmetry forbids under SSYNC and FSYNC but sequential schedulers allow (Flocchini et al. 2026).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point, + unlimited visibility, + transparent, + no faults, + sequential

Sequential schedulers STANDARD SETTING ●● p. 275

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
Sequential schedulers STANDARD SETTING	● unknown *finite* vs *impossible*	● tight 0 vs —	275

Claude (Anthropic) was used in cataloguing the problems.

18.4.1 Sequential schedulers

Adds + point, + unlimited visibility, + transparent, + no faults, + sequential STANDARD SETTING VERIFIED

Under any sequential scheduler, oblivious robots can form every pattern from every starting configuration, except a single point, which needs weak multiplicity detection.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: sequential: exactly one robot is activated per round. **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Flocchini et al. 2026) Time not analysed.	*finite* (Flocchini et al. 2026)	● unknown
Colours	none known No colours.	0 (Flocchini et al. 2026)	● tight

Results.

- Flocchini et al. (2026) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* under FSYNC, even with strong extra capabilities.
- Flocchini et al. (2026) [current best]: time *finite*, colours 0. Introductory and current best; no additional assumptions except for point formation.

18.5 Pattern sequences

Robots must form a given sequence of patterns, one after the other, forever.

First posed in 2010 (Das et al. 2015). Introduced at PODC 2010.

Problem statement and measures

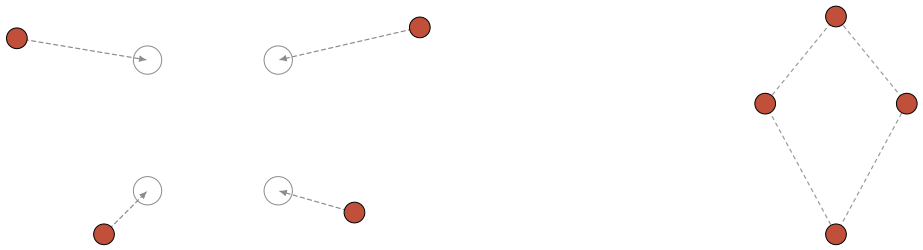
Robots are given a sequence of **patterns** $P_1, P_2, \ldots, P_m$ (sets of points in the plane) and must form them in order, repeatedly, with all robots taking part in each.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).

A first look

Picture a marching band that must form a sequence of shapes, for instance a square, then a line, then a triangle, then the square again, over and over. There are n **robots**, points in the plane; their positions form the **configuration**. Each robot repeats a **Look–Compute–Move cycle**: it takes a **snapshot** of where the other robots are (Look), decides on a destination point (Compute) and moves towards it (Move). In the basic version they are **oblivious** (no memory between cycles), agree on what clockwise means (**chirality**) but on nothing else, every move is a **rigid move** (it always reaches the computed destination), and the **scheduler**, an adversary, is SSYNC: time runs in **rounds**, and in each round it decides the **activation** of some of the robots, each of which performs one cycle. Given **patterns** $P_1, P_2, \ldots, P_m$ (m is the number of patterns; a pattern is a set of points, formed when the configuration

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: lights show step 1 (square)

(b) Lights change to step 1: the cycle repeats

Figure 18.5: Pattern sequences: robots must form the patterns $P_1, P_2, \ldots, P_m$ one after another, all robots taking part in each, and repeat the sequence forever (Das et al. 2015). Shown in FSYNC rounds: the sequence square, line, square, kite, which oblivious robots cannot form because the two squares look alike; with lights, the colour records the step, so the second square differs from the first by its colour.

has its shape), the robots must form P_1, then P_2, and so on up to P_m, then P_1 again, forever, with every robot taking part in each pattern.

1. Impossibility: Oblivious robots cannot repeat a pattern. (Das et al. 2015) (Oblivious, SSYNC, chirality, p. 278) Take $n = 4$ and the sequence square, line, square, kite. When the robots stand in the square they must decide where to go next: to the line (if this is the first square) or to the kite (if it is the second). But oblivious robots use only the current snapshot, and the two squares look exactly alike to robots that see only shapes, not sizes or directions. With the same snapshot a robot computes the same destination, so it cannot follow both continuations. A second obstacle is symmetry: robots can form a pattern only if it has at least the symmetry of the configuration they start from. In a cycle $P_1 \to P_2 \to \ldots \to P_m \to P_1$ each pattern must have at least the symmetry of the one before, and going round the cycle this forces all of them to have the same symmetry.

Why it holds. Das, Flocchini, Santoro and Yamashita characterised the sequences formable by oblivious robots under these assumptions (Das et al. 2015): as summarised in (Das et al. 2020), all patterns must have the same number of points and no pattern may appear twice, and the catalogue records that the patterns must have equal **symmetricity** (the largest q such that the points split into regular q-gons centred at the centre of their smallest enclosing circle, a point at that centre counting as a 1-gon Das et al. 2020), a multiple of that of the initial configuration. With distinct identifiers, any sequence of patterns of size at most n can be formed.

Why it matters. It isolates what memory buys: forming one pattern needs none, but following a script needs a way to know where you are in it.

2. Algorithm: Keep your place in the sequence with a coloured light. (Das et al. 2020) (Luminous robots, ASYNC, non-rigid, chirality, p. 278) Give each robot a **light**: a lamp that shows one of c colours, set by its robot, visible to all robots and kept between cycles.

1. Use the colours shown by the robots to record which step of the sequence is being formed.
2. Form the current pattern with a single-pattern algorithm.
3. When the pattern is complete, the robots change their colours to record the next step, and start forming the next pattern.
4. A repeated pattern is no longer a problem: the positions are the same, but the colours differ.

Why it works and what it costs. Lights act as a small shared memory, so repetitions, and even patterns with

Claude (Anthropic) was used in cataloguing the problems.

fewer points than robots, become possible. Das, Flocchini, Prencipe and Santoro give one protocol that forms every formable sequence, in ASYNC and with non-rigid moves (Das et al. 2020). The number of colours is bounded but not by a constant: it must grow with the number of repetitions and contractions in the sequence, by a lower bound that holds even for fully synchronous robots with rigid moves (Theorem 18 of Das et al. 2020). The steps above are the basic idea, not the paper's protocol, which must also cope with robots acting on outdated snapshots and with symmetric initial configurations.

Why it matters. It measures, in colours, exactly how much memory a choreography needs.

What the community started looking at

Forming a sequence of patterns asked how much **memory the configuration itself can hold** for oblivious robots: the answer is a symmetricity condition (Das et al. 2015); lights remove the restriction and work asynchronously (Das et al. 2020).

Relations to other problems

- **Generalisation of Pattern formation.** Forming one pattern is the case $m = 1$; a sequence needs the robots to remember (or encode in the configuration) which pattern comes next.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point, + unlimited visibility, + transparent, + no faults

Scheduler

+ SSYNC, + chirality, + rigid

Oblivious, SSYNC, chirality STANDARD SETTING — ● ● p. 278

ASYNC, + lights (LUMI), + chirality

Luminous robots, ASYNC, non-rigid, chirality — ● ● p. 278

+ sequential, + lights (LUMI), + strong multiplicity

Luminous robots, sequential scheduler (universal dancing) — ● ● p. 279

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
Oblivious, SSYNC, chirality STANDARD SETTING	● unknown *finite* vs —	● tight 0 vs —	278

Claude (Anthropic) was used in cataloguing the problems.

Setting	Time	Colours	p.
Luminous robots, ASYNC, non-rigid, chirality	● unknown *finite* vs *impossible*	● polynomial gap $\alpha(S)+\mu(S)+O(1)$ vs $\max\{\alpha(S)/\min_i \alpha(S_i), \mu(S)^{1/\alpha(S)}\}$	278
Luminous robots, sequential scheduler (universal dancing)	● unknown *finite* vs —	● near-optimal $c \geq 4$ vs with c colours,length $\leq \binom{n+c-1}{c-1}$	279

18.5.1 Oblivious, SSYNC, chirality

Adds + SSYNC, + chirality, + rigid STANDARD SETTING VERIFIED

A cyclic sequence of patterns is formable exactly when all patterns have the same symmetricity, a multiple of the start's.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); agreement on coordinates: common chirality (same clockwise orientation); movement: rigid: every move reaches its destination. **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Das et al. 2015)	● unknown
Colours	none known No colours.	0 (Das et al. 2015)	● tight

Results.

- Das et al. (2015) [current best]: time *finite*, colours 0. Introductory and current best; with distinct identifiers, any sequence of patterns of size at most n.

Open questions.

Q1. Form a sequence of patterns in ASYNC with oblivious robots (Das et al. 2015).

18.5.2 Luminous robots, ASYNC, non-rigid, chirality

Adds ASYNC, + lights (LUMI), + chirality VERIFIED

With lights and common chirality, the formable pattern sequences are characterised under ASYNC and non-rigid moves; the number of colours grows with the sequence and is not known to be optimal.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: common chirality (same clockwise orientation). **Not assumed:** scheduler: asynchronous (ASYNC); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Das et al. 2020) Only solvability.	*finite* (Das et al. 2020)	● unknown
Colours	$\max\{\alpha(S)/\min_i \alpha(S_i),\ \mu(S)^{1/\alpha(S)}\}$ (Das et al. 2020) $\alpha(S)+\mu(S)+O(1)$ against the Theorem 18 bound; closing it is called an important open question.	$\alpha(S)+\mu(S)+O(1)$ (Das et al. 2020)	● polynomial gap

Results.

- Das et al. (2020) [lower bound]: colours $\max\{\alpha(S)/\min_i \alpha(S_i),\ \mu(S)^{1/\alpha(S)}\}$. *Bottleneck:* memory. *Holds when:* Theorem 18; holds even in FSYNC with rigid moves.
- Das et al. (2020) [lower bound]: time *impossible*. *Bottleneck:* symmetry breaking. *Holds when:* Theorem 2: sequences that violate the paper's symmetry conditions.
- Das et al. (2020) [current best]: time *finite*, colours $\alpha(S)+\mu(S)+O(1)$. *Assumes:* weak chromatic multiplicity detection. Introductory and current best: a universal protocol for every formable sequence (Theorem 17). The number of colours is "bounded but not by a constant".

Open questions.

Q1. Close the gap between the colours used and the lower bound of Theorem 18 (Das et al. 2020).

Q2. Pattern sequences with lights without chirality, with faults, or self-stabilising (Das et al. 2020).

Notes. $\alpha(S)$ and $\mu(S)$ are parameters of the sequence $S = \langle S_1, \dots \rangle$ defined in (Das et al. 2020) (equation 1 and Theorem 18). Checked against the full text (HAL copy of the IEEE Access version).

18.5.3 Luminous robots, sequential scheduler (universal dancing)

Adds + sequential, + lights (LUMI), + strong multiplicity VERIFIED

Under a sequential scheduler, disoriented luminous robots form every sequence of patterns up to a length fixed by the number of colours.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: sequential: exactly one robot is activated per round; lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; multiplicity detection: strong: a robot sees how many robots are at each point. **Not assumed:** agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Feletti et al. 2026)	● unknown
Colours	with c colours,length $\leq \binom{n+c-1}{c-1}$ (Feletti et al. 2026) Upper and lower bounds on the sequence length differ by a shift in n and c.	$c \geq 4$ (Feletti et al. 2026)	● near-optimal

Results.

- Feletti et al. (2026) [lower bound]: colours with c colours,length $\leq \binom{n+c-1}{c-1}$. *Bottleneck:* memory. *Holds when:* longer sequences cannot be formed with c colours.
- Feletti et al. (2026) [current best]: time *finite*, colours $c \geq 4$. Introductory and current best: with $c \geq 4$ colours every sequence of length up to $\binom{n+c-7}{c-4}$ is formed; non-rigid moves, disoriented robots.

Notes. Checked against the full text.

Claude (Anthropic) was used in cataloguing the problems.

18.6 Mutual visibility

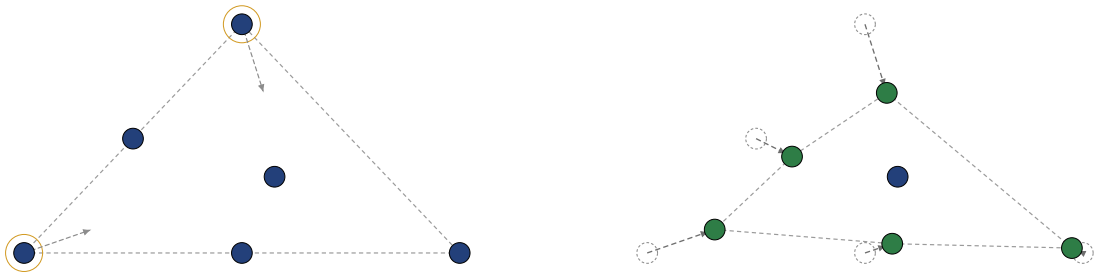
Opaque robots must reach positions where no three are collinear, so that every robot sees every other.

First posed in 2014 (Di Luna et al. 2017). Introduced at CCCG 2014 for oblivious robots and at SSS 2014 for robots with lights (Di Luna et al. 2014); journal version (Di Luna et al. 2017).

Problem statement and measures

n **opaque** point robots (or unit discs) start at distinct points: a robot hides the robots behind it on the same line of sight. They must reach a configuration in which every robot sees every other (no three collinear), without collisions.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).



(a) Start: hidden robots on hull edges and inside

(b) Final: all see each other; the robots stop (arrows: moves; dashed circles: starting positions)

Figure 18.6: Mutual visibility: opaque robots at distinct points, where a robot hides those behind it, must reach a configuration with no three robots collinear, so that every robot sees every other, without collisions. Shown: the Shrink idea of (Di Luna et al. 2017) in SSYNC rounds with rigid moves: a ringed (activated) robot that is at a corner of the convex hull (dashed) of the robots it sees moves a little into the triangle with its two hull neighbours, staying at a corner, and sets its light to Vertex (green); hidden robots become corner robots in turn.

A first look

Picture people at a party who want every pair of them to be able to see each other, but bodies block the view: you cannot see someone directly behind another person. There are n **robots**, points at distinct places in the plane; their positions form the **configuration**. The robots are **opaque**: robot r sees robot s only if no third robot lies on the straight segment between them. Each robot repeats a **Look–Compute–Move cycle**: it takes a **snapshot** of the robots it sees (Look), decides on a destination point (Compute) and moves towards it (Move). Each robot carries a **light** showing one of c colours, which all robots can see and which persists between cycles; otherwise robots remember nothing. In the basic version the **scheduler**, an adversary, is SSYNC: time runs in **rounds**, and in each round it decides the **activation** of some robots, each of which performs one cycle; every move is a **rigid move** (a robot always reaches its destination). The robots must reach, without two of them ever colliding, a configuration in which no three are on a line, so all see each other, and then stop.

1. Algorithm: Shrink: only corner robots move, cutting corners of the convex hull. (Di Luna et al. 2017) (Lights, SSYNC, rigid, p. 283) The **convex hull** is the shape of a rubber band stretched around all

Claude (Anthropic) was used in cataloguing the problems.

robots; its corner points are its **corners**, and a robot at one of them is a corner robot. Lights have two colours, Off and Vertex (meaning: I am at a corner), all starting Off.

1. A robot that is not at a corner of the convex hull of the robots it sees does nothing.
2. A corner robot sets its light to Vertex, finds its two neighbours a and b along the hull, and moves a little into the triangle formed by itself, a and b, cutting off its corner.
3. The move is kept inside a smaller safe triangle, so that robots moving at the same time cannot collide, and the robot stops on the nearest line parallel to ab through another robot, so that it stays at a corner.
4. A robot that sees only robots with light Vertex knows that it sees everybody, and stops.

Why it works and what it costs. Each cut shrinks the hull and can turn a robot that was hidden on an edge or inside into a new corner robot; eventually all robots are at the corners of a convex polygon, and then nobody blocks anybody. Some configurations need an inside robot to move: in a centrally symmetric configuration with a robot at the centre, symmetric robots can be made to move symmetrically forever, and the central robot keeps hiding pairs; the paper handles such cases separately. Di Luna, Flocchini, Gan Chaudhuri, Poloni, Santoro and Viglietta prove that this solves the problem in rigid SSYNC with 2 colours, which is optimal, and with no lights if n is known (Di Luna et al. 2017). In ASYNC, $O(1)$ epochs (intervals in which every robot completes a cycle) are achievable with more colours (Sharma et al. 2021).

Why it matters. It introduced the luminous-robot approach to obstructed visibility, and its output (all robots on a convex polygon) is a starting point for other tasks on opaque robots.

2. Lower bound: Without lights, a robot cannot know when to stop. (Di Luna et al. 2017) (Lights, SSYNC, rigid, p. 283) Suppose robots have no lights (one colour) and do not know n. A robot sees, say, four other robots at the vertices of a square. Maybe that is everybody, and it should stop. Or maybe a fifth robot hides exactly behind one of the four, on the same line of sight, and the robots must still move. The two situations give the robot the same snapshot, and stopping cannot be undone, so whatever it decides is wrong in one of them.

Why it holds. Hence at least 2 colours are needed when n is unknown and robots must stop (Section 7 of Di Luna et al. 2017), and Shrink shows that 2 suffice in rigid SSYNC. If n is known, a robot can count the robots it sees and stop when it sees $n - 1$ others, so no light is needed. The authors note that if robots only need to stand still forever, without explicitly stopping, this argument does not apply.

Why it matters. It shows that the cost of detecting completion, not of reaching it, is what forces memory.

What the community started looking at

Mutual visibility arose from **obstructed visibility**: before opaque robots can coordinate they must see each other. The first questions were feasibility and the **number of colours** (Di Luna et al. 2017) (Sharma et al. 2015); then **time** with $O(1)$ colours (Vaidyanathan et al. 2015) (Sharma et al. 2021), and fat robots (Sharma et al. 2018) (Bhagat et al. 2024).

Relations to other problems

- **Special case of Pattern formation.** Mutual visibility asks for any pattern in general position, a family of patterns; it is the usual first step of pattern formation for opaque robots.

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point, + unlimited visibility
Faults

+ no faults
Scheduler

+ SSYNC, + lights (LUMI)
Lights, SSYNC, non-rigid ●● p. 283

+ rigid
Lights, SSYNC, rigid ●● p. 283

ASYNC, + lights (LUMI)
Agreement on coordinates

disoriented, + rigid
Lights, ASYNC, rigid ●● p. 284

+ one axis
Lights, ASYNC, non-rigid, one axis ●● p. 285

+ FSYNC, + lights (LUMI), + chirality
Lights, FSYNC, chirality ●● p. 285

+ crash faults, + SSYNC, + lights (LUMI), + compass, + rigid
Lights, one crash, SSYNC, rigid, compass ●● p. 286

Byzantine, + SSYNC, + lights (LUMI), + compass, + rigid
Lights, one Byzantine robot ●● p. 286

fat, + unlimited visibility, + no faults
Scheduler

ASYNC, + lights (LUMI), + one axis
Fat robots, ASYNC, one axis ●● p. 287

+ FSYNC, + lights (LUMI), + rigid
Fat robots, FSYNC, rigid ●● p. 287

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
Lights, SSYNC, non-rigid	● unknown *finite* vs —	● tight 3 vs 2	283
Lights, SSYNC, rigid	● tight *finite* vs —	● tight 2 vs 2	283
Lights, ASYNC, rigid	● tight $O(1)$ vs $\Omega(1)$	● tight 47 vs 2	284

Claude (Anthropic) was used in cataloguing the problems.

Setting	Time	Colours	p.
Lights, ASYNC, non-rigid, one axis	● unknown *finite* vs —	● tight 3 vs 2	285
Lights, FSYNC, chirality	● unknown $O(\log n)$ vs —	● tight 12 vs —	285
Lights, one crash, SSYNC, rigid, compass	● unknown *finite* vs —	● tight 3 vs —	286
Lights, one Byzantine robot	● impossible	● impossible	286
Fat robots, ASYNC, one axis	● unknown $O(n)$ vs —	● tight $O(1)$ vs —	287
Fat robots, FSYNC, rigid	● unknown $O(n)$ vs —	● tight 2 vs 2	287

18.6.1 Lights, SSYNC, non-rigid

Adds + SSYNC, + lights (LUMI) VERIFIED

With non-rigid moves, 3 colours suffice in SSYNC, and 2 if δ or n is known.

Assumed: body: point robots; visibility: unlimited; faults: no faults; scheduler: semi-synchronous (SSYNC); lights (persistent memory): full light (LUMI): every robot sees all lights, including its own. **Not assumed:** opacity: opaque: collinear robots hide each other; agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Not analysed.	*finite* (Di Luna et al. 2017)	● unknown
Colours	2 (Di Luna et al. 2017) 2 or 3 colours.	3 (Di Luna et al. 2017)	● tight

Results.

- Di Luna et al. (2017) [lower bound]: colours 2. *Bottleneck:* memory. *Holds when:* Section 7: one colour does not suffice when n is unknown and robots must terminate.
- Di Luna et al. (2017) [current best]: time *finite*, colours 3. Introductory and current best (Theorem 1.2); 2 colours if δ or n is known.

Open questions.

Q1. Does the 2-colour algorithm work in non-rigid SSYNC without knowing δ (Di Luna et al. 2017)?

Notes. Checked against the full text.

18.6.2 Lights, SSYNC, rigid

Adds + rigid **to** Lights, SSYNC, non-rigid (p. 283) VERIFIED

Opaque luminous robots reach mutual visibility in rigid SSYNC with 2 colours, which is optimal; oblivious robots suffice if n is known.

Assumed: body: point robots; visibility: unlimited; faults: no faults; scheduler: semi-synchronous (SSYNC); lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; movement: rigid: every move reaches its destination. **Not assumed:** opacity: opaque: collinear robots hide each other; agreement on coordinates: no agreement on local coordinate

Claude (Anthropic) was used in cataloguing the problems.

systems; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known $O(1)$ rounds with 12 colours (Sharma et al. 2016).	*finite* (Di Luna et al. 2017)	• tight
Colours	2 (Di Luna et al. 2017) Exactly 2 colours.	2 (Di Luna et al. 2017)	• tight

Results.

- Di Luna et al. (2017) [lower bound]: colours 2. *Bottleneck:* memory. *Holds when:* Section 7: one colour does not suffice when n is unknown and robots must terminate.
- Di Luna et al. (2017) [current best]: time *finite*, colours 2. Introductory and current best (Theorem 1.1(b)); first announced at SSS 2014 (Di Luna et al. 2014). With n known, oblivious robots (no colours) suffice.
- Sharma et al. (2016) [also]: time $O(1)$, colours 12. $O(1)$ rounds; as described by later papers, not read.

Notes. Checked against the full text of (Di Luna et al. 2017).

18.6.3 Lights, ASYNC, rigid

Adds disoriented, + rigid

VERIFIED

In rigid ASYNC, 3 colours suffice, and $O(1)$ epochs are achievable with 47 colours.

Assumed: body: point robots; visibility: unlimited; faults: no faults; lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; movement: rigid: every move reaches its destination. **Not assumed:** opacity: opaque: collinear robots hide each other; scheduler: asynchronous (ASYNC); agreement on coordinates: no agreement on local coordinate systems; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(1)$ (Sharma et al. 2021) $\Theta(1)$ epochs.	$O(1)$ (Sharma et al. 2021)	• tight
Colours	2 (Di Luna et al. 2017) $\Theta(1)$; 2 or 3 colours for finite time, 47 for $O(1)$ epochs.	47 (Sharma et al. 2021)	• tight

Results.

- Di Luna et al. (2017) [lower bound]: colours 2. *Bottleneck:* memory. *Holds when:* Section 7: one colour does not suffice when n is unknown and robots must terminate.
- Sharma et al. (2021) [lower bound]: time $\Omega(1)$. *Bottleneck:* trivial. *Holds when:* at least one epoch.
- Di Luna et al. (2017) [introductory]: time *finite*, colours 3. Collision-free.
- Sharma et al. (2015) [also]: time *finite*, colours 2. Claimed optimal colours; later papers disagree on its setting, and the paper was not read.
- Sharma et al. (2017) [also]: time $O(\log n)$, colours $O(1)$. Abstract only.
- Sharma et al. (2021) [current best]: time $O(1)$, colours 47. Theorem 1: $O(1)$ epochs, collision-free; first appeared in 2017. Its conclusion says 60 colours.

Open questions.

Q1. Fewer colours with $O(1)$ epochs (Sharma et al. 2021).

Q2. Time bounds with non-rigid moves.

Claude (Anthropic) was used in cataloguing the problems.

Notes. **Unverified:** (Sharma et al. 2015) and (Sharma et al. 2017) were not read in full.

18.6.4 Lights, ASYNC, non-rigid, one axis

Adds + one axis VERIFIED

With non-rigid moves, 3 colours suffice in ASYNC if robots agree on one axis; without axis agreement the problem is open.

Assumed: body: point robots; visibility: unlimited; faults: no faults; lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: agreement on one axis (no common chirality). **Not assumed:** opacity: opaque: collinear robots hide each other; scheduler: asynchronous (ASYNC); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Not analysed.	*finite* (Di Luna et al. 2017)	● unknown
Colours	2 (Di Luna et al. 2017) 2 or 3 colours.	3 (Di Luna et al. 2017)	● tight

Results.

- Di Luna et al. (2017) [lower bound]: colours 2. *Bottleneck:* memory. *Holds when:* Section 7: one colour does not suffice when n is unknown and robots must terminate.
- Di Luna et al. (2017) [current best]: time *finite*, colours 3. Introductory and current best (Theorem 1.2(d)).

Open questions.

Q1. Mutual visibility in non-rigid ASYNC without axis agreement, ideally with 2 colours (Di Luna et al. 2017).

Notes. Checked against the full text.

18.6.5 Lights, FSYNC, chirality

Adds + FSYNC, + lights (LUMI), + chirality DRAFT

In FSYNC with chirality, complete visibility takes $O(\log n)$ rounds with 12 colours.

Assumed: body: point robots; visibility: unlimited; faults: no faults; scheduler: fully synchronous (FSYNC); lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: common chirality (same clockwise orientation). **Not assumed:** opacity: opaque: collinear robots hide each other; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known $O(1)$ is known in SSYNC with rigid moves (Sharma et al. 2016), which needs no chirality but is not a weaker setting on every axis.	$O(\log n)$ (Vaidyanathan et al. 2015)	● unknown
Colours	none known $\Theta(1)$.	12 (Vaidyanathan et al. 2015)	● tight

Results.

Claude (Anthropic) was used in cataloguing the problems.

- Vaidyanathan et al. (2015) [current best]: time $O(\log n)$, colours 12. Introductory and current best; robot paths may cross. Abstract only.

Notes. **Unverified:** read from the abstract only.

18.6.6 Lights, one crash, SSYNC, rigid, compass

Adds + crash faults, + SSYNC, + lights (LUMI), + compass, + rigid VERIFIED

With one crash fault, robots that share a compass reach mutual visibility in rigid SSYNC with 3 colours.

Assumed: body: point robots; visibility: unlimited; faults: up to f robots may crash; scheduler: semi-synchronous (SSYNC); lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: agreement on both axes (common compass); movement: rigid: every move reaches its destination. **Not assumed:** opacity: opaque: collinear robots hide each other; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Aljohani and Sharma 2018)	● unknown
Colours	none known $\Theta(1)$.	3 (Aljohani and Sharma 2018)	● tight

Results.

- Aljohani and Sharma (2018) [current best]: time *finite*, colours 3. Introductory and current best; one crash fault.

Open questions.

Q1. Tolerating three or more crashes, and using 2 colours (Aljohani and Sharma 2018).

Notes. With f crashes, robots that agree on one axis reach complete visibility in ASYNC in $O(n)$ epochs with 3 colours (Poudel et al. 2021) (abstract only). Checked against the full text of (Aljohani and Sharma 2018).

18.6.7 Lights, one Byzantine robot

Adds Byzantine, + SSYNC, + lights (LUMI), + compass, + rigid VERIFIED

With one Byzantine robot among three, mutual visibility is impossible.

Assumed: body: point robots; visibility: unlimited; scheduler: semi-synchronous (SSYNC); lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: agreement on both axes (common compass); movement: rigid: every move reaches its destination. **Not assumed:** opacity: opaque: collinear robots hide each other; faults: up to f Byzantine robots; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Aljohani and Sharma 2018) For $n = 3$.	—	● impossible
Colours	none known For $n = 3$.	—	● impossible

Results.

Claude (Anthropic) was used in cataloguing the problems.

- Aljohani and Sharma (2018) [lower bound]: time *impossible. Bottleneck:* indistinguishable Byzantine agents. *Holds when:* $n = 3$ with one Byzantine robot.

Notes. Checked against the full text.

18.6.8 Fat robots, ASYNC, one axis

Adds ASYNC, + lights (LUMI), + one axis DRAFT

Fat robots that agree on one axis reach mutual visibility in ASYNC in $O(n)$ epochs with $O(1)$ colours.

Assumed: visibility: unlimited; faults: no faults; lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: agreement on one axis (no common chirality). **Not assumed:** body: fat robots (unit discs); opacity: opaque: collinear robots hide each other; scheduler: asynchronous (ASYNC); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	$O(n)$ (Bhagat et al. 2024)	● unknown
Colours	none known $\Theta(1)$.	$O(1)$ (Bhagat et al. 2024)	● tight

Results.

- Bhagat et al. (2024) [current best]: time $O(n)$, colours $O(1)$. Introductory and current best for ASYNC. Abstract only.

Open questions.

Q1. Non-rigid moves, and SSYNC and ASYNC extensions of the 2-colour FSYNC algorithm (Alsaedi et al. 2023).

Notes. **Unverified:** read from the abstract only.

18.6.9 Fat robots, FSYNC, rigid

Adds + FSYNC, + lights (LUMI), + rigid VERIFIED

In FSYNC, fat robots without chirality reach mutual visibility in $O(n)$ rounds with 2 colours, which is optimal.

Assumed: visibility: unlimited; faults: no faults; scheduler: fully synchronous (FSYNC); lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; movement: rigid: every move reaches its destination. **Not assumed:** body: fat robots (unit discs); opacity: opaque: collinear robots hide each other; agreement on coordinates: no agreement on local coordinate systems; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	$O(n)$ (Alsaedi et al. 2023)	● unknown
Colours	2 (Alsaedi et al. 2023) Exactly 2 colours.	2 (Alsaedi et al. 2023)	● tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Alsaedi et al. (2023) [lower bound]: colours 2. *Bottleneck:* memory. *Holds when:* inherited from point robots (a lower bound for point robots holds for fat ones); not reproved for discs in (Alsaedi et al. 2023).
- Sharma et al. (2018) [introductory]: time $O(n)$, colours 9. Abstract only.
- Alsaedi et al. (2023) [current best]: time $O(n)$, colours 2. Current best; no chirality. A preprint, not peer reviewed.

Notes. Checked against the full text of (Alsaedi et al. 2023).

18.7 k-grouping

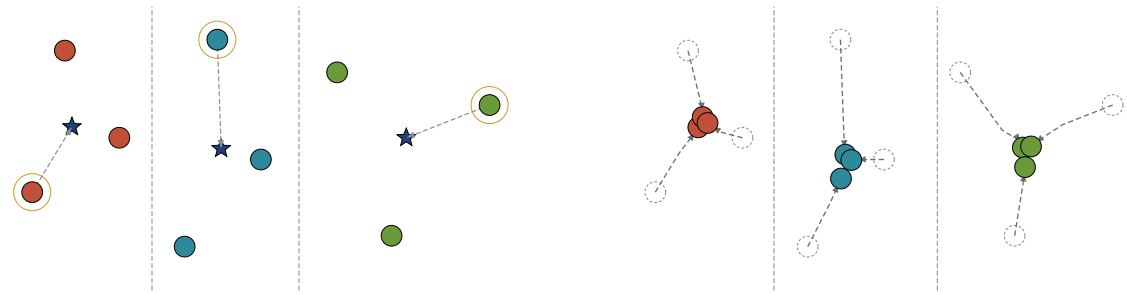
Robots must form k distinct groups, each group at its own point.

First posed in 2007 (Efrima and Peleg 2009). Splitting a swarm into a given number of groups was introduced at SIROCCO 2007, under the name partitioning. No paper titled "k-grouping" was found.

Problem statement and measures

n robots must form exactly k distinct **groups**, each group gathered at its own point or converging towards one, and stay apart. The number of groups k is given; the group sizes are free unless the variant fixes them. With $k = 1$ this is gathering (or convergence).

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).



(a) Start: $n = 9$, $k = 3$; blocks of 3 by x-order

(b) Final: 3 groups, each converging on its own point (arrows: moves; dashed circles: starting positions)

Figure 18.7: k-grouping: n robots must split into exactly k groups, each gathered at (or converging to) its own point, the points distinct. Shown: the algorithm of (Efrima and Peleg 2009) with a common compass, $n = 9$, $k = 3$, in SSYNC rounds: robots are cut into blocks of n/k by x-order (colours), and each ringed (activated) robot moves towards the centre of gravity (star) of its block, possibly stopping early; blocks never mix.

A first look

Imagine a swarm that must split into k teams of equal size, each team clustering in its own place, far from the others. In the problem the members of the swarm are n **robots**, points at distinct places in the plane, and the teams are **groups**; k divides n. The positions of all robots form the **configuration**. Each robot reads positions in its own **local coordinate system** (its private axes, unit of length and handedness) and repeats a **Look–Compute–Move cycle**: it takes a **snapshot** of where the other robots

Claude (Anthropic) was used in cataloguing the problems.

are (Look), decides on a destination point (Compute) and moves towards it (Move). Robots are identical, silent and **oblivious** (no memory between cycles). In the basic version robots share a **compass** (all local coordinate systems agree on north and east, though not on origin or unit of length); the **scheduler**, which decides the **activation** of robots (when each starts a cycle) and how long each phase takes, is ASYNC (each phase may take any finite time); and some robots may **crash** (stop forever). The robots must split into k groups of n/k robots each, every group converging towards its own point, with the groups staying a fixed distance apart; with $k = 1$ this is just convergence.

1. Algorithm: Sort by the compass, cut into blocks, converge within each block. (Efrima and Peleg 2009) (k groups of equal size, p. 291)

1. Sort the robots from west to east, and robots on one north-south line from south to north. All robots get the same order, because they share north and east.
2. Cut the order into k consecutive blocks of n/k robots: the first n/k robots, the next n/k, and so on. Each robot finds its own block.
3. Move towards the **centre of gravity** of your block, the average of the positions of its robots.

Why it works and what it costs. The region spanned by a block's robots and their destinations never grows when they move to their centre of gravity (Lemma 2.5 of Efrima and Peleg 2009, from Cohen and Peleg). The blocks start in disjoint regions ordered from west to east, so two robots of different blocks can never swap order, and the blocks cut in step 2 stay the same forever, even though robots forget them. Each block therefore converges on its own, and a crashed robot simply stays inside its block's region, so any number of crashes is tolerated (Proposition 2.7 of Efrima and Peleg 2009). This is the catalogue's introductory and current best result.

Why it matters. It shows a clean way to make oblivious robots keep an agreement forever: choose it so that the motion itself preserves it.

2. Impossibility: No compass: the regular polygon cannot be split. (Efrima and Peleg 2009) (k equal groups, no common orientation, deterministic, p. 292) Start the n robots on the corners of a regular n-gon, with local coordinate systems that are copies of each other turned around the centre. Every robot sees the same snapshot, so in every **round** of the fully synchronous scheduler FSYNC (a step of time in which every robot performs one full cycle) all robots make the same move up to rotation, and the configuration stays a regular polygon (or a single point). But $k \geq 2$ groups of several robots each, far apart, never look like a regular polygon or a point. So the groups never form.

Why it holds. This is Proposition 3.3 of (Efrima and Peleg 2009): deterministic robots without a compass cannot partition themselves into $k > 1$ groups, already in FSYNC and therefore in SSYNC and ASYNC. Agreeing only on the axis lines but not their directions is still not enough for some n and k (a symmetric square with $n = 4$, $k = 2$; Proposition 3.4).

Why it matters. It shows that partitioning, unlike gathering, needs a way to break symmetry.

3. Algorithm: A coin flip breaks the tie. (Efrima and Peleg 2009) (k equal groups, randomized, SSYNC, p. 291) Without a compass, the trouble is sets of robots that look exactly alike. Randomness helps: a robot in such a tied set chooses its direction of motion at random. Two robots that would otherwise mirror each other forever now choose differently with probability at least $1/2$ in each attempt, so the tie is broken after a few attempts on average, and the robots then proceed with the deterministic method.

Why it works and what it costs. Efrima and Peleg give a randomized algorithm, of the kind that always ends correctly and is only random in its running time, that partitions robots without any common orientation in FSYNC and SSYNC and tolerates crashes (Efrima and Peleg 2009). The description above is simplified: in the paper the random choice is used only to separate robots that lie on the same line.

Claude (Anthropic) was used in cataloguing the problems.

Why it matters. It is the standard illustration that randomness breaks the symmetry that defeats deterministic robots.

What the community started looking at

Splitting a swarm into a given number of groups was asked as **solvability under each timing model and amount of orientation** (Efrima and Peleg 2009), who called it partitioning and required equal group sizes. Team assembling of heterogeneous robots, where the composition of each team is prescribed, is a related task (Liu et al. 2018).

Relations to other problems

- **Special case of Pattern formation.** When the groups sit at points, k-grouping forms a pattern of k points; which points, and how many robots each holds, is up to the robots. Gathering is its special case $k = 1$. Splitting needs symmetry breaking (Efrima and Peleg 2009).
- **Derived problem: Gathering** (special case of). Gathering is k-grouping with $k = 1$, a single group of all n robots, that is, point formation: the pattern consisting of one point of multiplicity n (Flocchini et al. 2017) (Flocchini et al. 2026). General pattern formation algorithms usually exclude it, because multiplicity points need detection (Bramas and Tixeuil 2016).
- **Related: Partitioning.** k-grouping fixes the number of groups; partitioning fixes the size of each group. With equal sizes the two coincide: k groups of n/k robots.
- **Related: Separation.** Both split the swarm into classes; in separation the classes are fixed by hidden colours, in k-grouping and partitioning they are chosen by the robots.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point, + unlimited visibility, + transparent

Faults

+ crash faults

Scheduler

ASYNC, + compass

k groups of equal size STANDARD SETTING ● ● p. 291

+ SSYNC, + randomized

k equal groups, randomized, SSYNC ● ● p. 291

+ no faults, + FSYNC

k equal groups, no common orientation, deterministic ● ● p. 292

Claude (Anthropic) was used in cataloguing the problems.

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
k groups of equal size STANDARD SETTING	● unknown *finite* vs —	● tight 0 vs —	291
k equal groups, randomized, SSYNC	● unknown *finite* vs —	● tight 0 vs —	291
k equal groups, no common orientation, deterministic	● impossible	● tight — vs —	292

18.7.1 k groups of equal size

Adds ASYNC, + compass STANDARD SETTING VERIFIED

Robots split into k groups of n/k that converge apart; with a full compass a deterministic crash-tolerant algorithm works in ASYNC.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: up to f robots may crash; agreement on coordinates: agreement on both axes (common compass). **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Statement for this setting. Partition(n, k) of Efrima and Peleg: k is given to the robots, every group has exactly n/k robots ($\lfloor n/k \rfloor$ or $\lfloor n/k \rfloor + 1$ when k does not divide n), robots in one group **converge** (for every ε they are eventually within ε of each other) and groups stay more than 2η apart, for some fixed $\eta > 0$. Robots start at distinct points and can count the robots at a point.

Measure	Lower bound	Best upper bound	Status
Time	none known Only solvability.	*finite* (Efrima and Peleg 2009)	● unknown
Colours	none known No colours.	0 (Efrima and Peleg 2009)	● tight

Results.

- Efrima and Peleg (2009) [current best]: time *finite*, colours 0. *Assumes:* strong multiplicity detection. Introductory and current best: deterministic, crash-fault tolerant, in every timing model with a full compass. Also crash-tolerant in FSYNC and SSYNC with a half compass (both axis directions, one positive orientation, no chirality), and not crash-tolerant in ASYNC with a half compass.

Notes. Efrima and Peleg call this partitioning; the catalogue's partitioning also fixes, in advance, the k points at which the groups of n/k robots gather. Convergence is Partition$(n, 1)$. Checked against the full text.

18.7.2 k equal groups, randomized, SSYNC

Adds + SSYNC, + randomized VERIFIED

With randomness, robots without any common orientation partition in SSYNC, tolerating crashes.

Claude (Anthropic) was used in cataloguing the problems.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: up to f robots may crash; scheduler: semi-synchronous (SSYNC); randomness: randomized. **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; identifiers: anonymous: all robots are identical.

Statement for this setting. Partition(n, k) of Efrima and Peleg: k is given to the robots, every group has exactly n/k robots ($\lfloor n/k \rfloor$ or $\lfloor n/k \rfloor + 1$ when k does not divide n), robots in one group **converge** (for every ε they are eventually within ε of each other) and groups stay more than 2η apart, for some fixed $\eta > 0$. Robots start at distinct points and can count the robots at a point.

Measure	Lower bound	Best upper bound	Status
Time	none known Only solvability.	*finite* (Efrima and Peleg 2009)	● unknown
Colours	none known No colours.	0 (Efrima and Peleg 2009)	● tight

Results.

- Efrima and Peleg (2009) [current best]: time *finite*, colours 0. *Assumes:* strong multiplicity detection; adaptive adversary. Introductory and current best: randomized, crash-fault tolerant, FSYNC and SSYNC.

Open questions.

Q1. Partitioning in ASYNC without a common orientation, by a randomized algorithm with no extra memory.

Notes. Checked against the full text.

18.7.3 k equal groups, no common orientation, deterministic

Adds + no faults, + FSYNC VERIFIED

Without any common orientation no deterministic algorithm partitions robots into $k > 1$ groups, already in FSYNC.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: fully synchronous (FSYNC). **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Statement for this setting. Partition(n, k) of Efrima and Peleg: k is given to the robots, every group has exactly n/k robots ($\lfloor n/k \rfloor$ or $\lfloor n/k \rfloor + 1$ when k does not divide n), robots in one group **converge** (for every ε they are eventually within ε of each other) and groups stay more than 2η apart, for some fixed $\eta > 0$. Robots start at distinct points and can count the robots at a point.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Efrima and Peleg 2009) Deterministically unsolvable for $k > 1$.	—	● impossible
Colours	none known No colours.	—	● tight

Results.

Claude (Anthropic) was used in cataloguing the problems.

- Efrima and Peleg (2009) [lower bound]: time *impossible*. *Bottleneck:* symmetry breaking. *Holds when:* Proposition 3.3: deterministic, $k > 1$; robots start at the vertices of a regular n-gon and always make the same move; holds even in FSYNC, hence in SSYNC and ASYNC; with common axes but no orientation, still unsolvable for some n, k (Proposition 3.4).

Notes. In a companion paper the same authors show that the problem becomes solvable in ASYNC without a compass if the start is asymmetric, if one robot is identifiable, or, randomized against a non-adaptive adversary, with one bit of memory and communication (as stated in Efrima and Peleg 2009). Checked against the full text.

18.8 Gathering

Robots must meet at a single point that is not fixed in advance, and stay there.

First posed in 1999 (Suzuki and Yamashita 1999). Suzuki and Yamashita gave an algorithm "moving the robots to a single point in finite steps" and showed that two oblivious robots cannot meet in SSYNC.

Problem statement and measures

n robots start at distinct points and must all reach the same point in finite time and remain there. For $n = 2$ the problem is called rendezvous. **Time** is counted in rounds (FSYNC, SSYNC) or in epochs (ASYNC: intervals in which every robot completes at least one full Look–Compute–Move cycle); **colours** count light colours.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).



(a) Start: symmetric, no multiplicity: all aim at the centre

(b) Final: all robots on one point (arrows: moves; dashed circles: starting positions)

Figure 18.8: Gathering: anonymous oblivious robots at distinct points of the plane must meet at one point, not fixed in advance, and stay there. Shown: the strategy of (Cieliebak et al. 2012) with weak multiplicity detection, in SSYNC rounds (ringed robots are activated; a move may stop early). In a symmetric configuration every robot computes the same centre; once a single multiplicity exists, every robot walks to it.

A first look

Imagine robots scattered over a field that must all meet at one spot, although nobody has said where. There are n **robots**, points in the plane at distinct places; their positions form the **configuration**. Each robot repeats a **Look–Compute–Move cycle**: it takes a **snapshot** of where the other robots are (Look),

Claude (Anthropic) was used in cataloguing the problems.

decides on a destination point (Compute) and moves towards it (Move). They are identical, silent and **oblivious** (they remember nothing from one cycle to the next), and each uses its own **local coordinate system** (origin, axes, unit of length and handedness), so they agree on no direction. An adversary, the **scheduler**, decides the **activation** of robots, that is, when each performs a cycle: in SSYNC time runs in **rounds** and in each round it activates some of the robots; in ASYNC every phase may take any finite time. The robots must all reach the same point in finite time and stay there (for $n = 2$ this is called rendezvous). In the standard version $n \geq 3$, the scheduler is ASYNC, and robots have **weak multiplicity detection**: they can tell whether a point holds one robot or several (a **multiplicity**), but not how many.

1. Algorithm: Create one multiplicity, then every robot walks to it. (Cieliebak et al. 2012) (ASYNC with weak multiplicity detection, p. 307)

1. If exactly one point is a multiplicity, walk straight to it.
2. Otherwise, look at the shape of the configuration and choose a point that every robot identifies in the same way despite their different local coordinate systems (for example the centre of a symmetric configuration), and move so that exactly one multiplicity appears.
3. From then on every robot follows rule 1, along paths that never create a second multiplicity.

Why it works and what it costs. A multiplicity is a landmark that no local coordinate system can hide: it looks special to every robot. Step 2 is the hard part, because the robots agree on no direction and the scheduler may stop a robot partway; Cieliebak, Flocchini, Prencipe and Santoro solve it with a case analysis of configuration types and gather $n \geq 3$ robots in ASYNC from every initial configuration (Cieliebak et al. 2012), the catalogue's introductory and current best result for this setting. Time is not analysed. The outline above is the idea of the approach and was not checked against the paper's full text.

Why it matters. It shows how a tiny sensing ability, telling one robot from several, makes an unsolvable problem solvable.

2. Impossibility: Two oblivious robots cannot meet in SSYNC. (Suzuki and Yamashita 1999) (Two robots, SSYNC, p. 303) Take two robots, A and B, and look at the round in which they would finally meet.

(a) If both moved in that round, the scheduler could have activated only one: it lands where the other was heading, not where the other is, so they miss.

(b) If only A moved and landed on B, the scheduler could have activated both; unless B's rule tells it to stay put, B moves away and they miss again.

So meeting needs a situation in which, when both are activated, one robot jumps onto the other while the other stays. (c) But the robots are identical and agree on no direction, so the adversary can give them local coordinate systems in which each sees the other at the same spot, say one unit to its east. Identical robots with identical snapshots make the same choice: both stay or both move. The lopsided situation never occurs, and they never meet.

Why it holds. Suzuki and Yamashita proved this as Theorem 3.1 of (Suzuki and Yamashita 1999), even for robots with common handedness whose moves always reach their destination, so it also holds without these advantages. Natural rules fail in instructive ways: \"move to the midpoint\" brings the robots closer forever, but the scheduler activates one robot at a time so they never coincide; \"move to the other robot\" makes them swap places forever when both are activated. Extra power fixes this: with lights that both robots see, two colours suffice even in ASYNC (Heriban et al. 2018).

Why it matters. It is the founding impossibility of the field: the scheduler keeps identical robots symmetric, and most later assumptions (multiplicity detection, lights, compasses) are ways to break that symmetry.

Claude (Anthropic) was used in cataloguing the problems.

3. Impossibility: Without multiplicity detection, a multiplicity looks like one robot. (Prencipe 2007) (No multiplicity detection, SSYNC, p. 302) Suppose the robots stand on only two points, say $n-1$ robots on p and one robot on q. A robot that cannot detect multiplicities sees just two occupied points, exactly what it would see if there were only two robots. The robots are thrown back into the two-robot situation above, and the scheduler, by activating only some of the robots on a point, can move groups around so that they never all stand together. Prencipe shows that against any algorithm the scheduler can first drive the robots into such special configurations and then keep them from ever gathering; the full proof is a longer case analysis.

Why it holds. The result holds for deterministic oblivious robots without multiplicity detection, for every $n \geq 2$, in SSYNC and ASYNC (Prencipe 2007). In the fully synchronous model, by contrast, gathering needs no multiplicity detection (Flocchini et al. 2026), and in ASYNC weak multiplicity detection is enough (Cieliebak et al. 2012).

Why it matters. It explains why weak multiplicity detection became the standard assumption for gathering.

4. Algorithm: Short sight but a common compass: drift right, then down. (Flocchini et al. 2005) (Limited visibility with a compass, ASYNC, p. 311) Now each robot sees only the robots within distance V (its **visibility radius**), but all robots agree on north and east (a **common compass**).

1. If you see a robot to your left, or above you on your own vertical line, do nothing.
2. If you see robots only below you on your vertical line, move down towards the nearest one.
3. If you see robots only to your right, move horizontally to the vertical line of the nearest one.
4. If you see robots both below you and to your right, make a diagonal move right and down, chosen so that you lose sight of nobody.

Why it works and what it costs. Robots only move right or down and never beyond the rightmost vertical line that holds a robot in the initial configuration, so they collect on that line and then at its lowest occupied point; the delicate part of the proof is that nobody is left behind out of sight. Flocchini, Prencipe, Santoro and Widmayer prove that this gathers ASYNC robots with non-rigid moves (Theorem 4.2 of Flocchini et al. 2005), provided the initial configuration is connected (robots joined when at most V apart); a disconnected initial configuration cannot be gathered at all. With agreement on one axis only, time linear in the initial diameter is achievable and optimal (Poudel and Sharma 2021).

Why it matters. It shows how a shared direction can replace global vision: robots that see little can still agree where to go.

What the community started looking at

Gathering started with an impossibility: two oblivious robots cannot meet in SSYNC (Suzuki and Yamashita 1999). The community's main question became **the weakest set of capabilities that makes gathering solvable**: multiplicity detection (Cieliebak et al. 2012), a compass under limited visibility (Flocchini et al. 2005), lights for two robots (Das et al. 2016) (Flocchini et al. 2016) (Heriban et al. 2018), and how many faulty robots can be tolerated (Agmon and Peleg 2006) (Pattanayak et al. 2019b). Running time (rounds, epochs) became a measure mainly for limited visibility (Degener et al. 2011) (Castenow et al. 2022).

Relations to other problems

- **Special case of k-grouping.** Gathering is k-grouping with $k = 1$, a single group of all n robots, that is, point formation: the pattern consisting of one point of multiplicity n (Flocchini et al.

Claude (Anthropic) was used in cataloguing the problems.

2017) (Flocchini et al. 2026). General pattern formation algorithms usually exclude it, because multiplicity points need detection (Bramas and Tixeuil 2016).

- **Derived problem: Near-gathering** (relaxation of). Gathering requires a common point; near-gathering only requires being close, but forbids collisions. It is used as a first phase before forming patterns.
- **Related: Convergence.** Every gathering algorithm solves convergence, but not conversely: two oblivious robots in SSYNC cannot gather, while "two oblivious robots can converge to a point with a naive algorithm" (Izumi et al. 2012). In ASYNC the centre-of-gravity algorithm converges for any n (Cohen and Peleg 2005), whereas gathering needs extra assumptions such as multiplicity detection.
- **Related: Scattering.** Scattering is the opposite task: gathering creates a multiplicity point, scattering removes all of them. Pattern formation from configurations with multiplicities needs scattering first (Bramas and Tixeuil 2016).

Assumptions specific to this problem

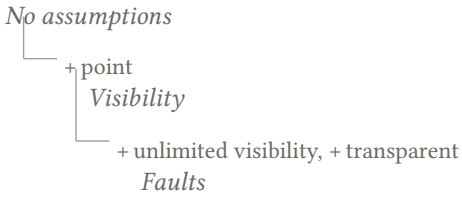
Number of robots

Suzuki and Yamashita (Suzuki and Yamashita 1999) showed that two oblivious robots cannot gather deterministically in SSYNC, although the task is trivial in FSYNC; rendezvous thus became the first separator between the two schedulers (Flocchini et al. 2023) (Heriban et al. 2018). For $n \geq 3$ robots the situation differs: with multiplicity detection, oblivious disoriented robots gather from any configuration even in ASYNC (Cieliebak et al. 2012), because a multiplicity point, once formed, can be kept unique and serve as the target (Bandettini et al. 2011). As Flocchini, Santoro, Viglietta and Yamashita put it, "Rendezvous is very different from the Gathering problem of having $k \geqslant 3$ robots meet" (Flocchini et al. 2016) (their k is n here). Two robots cannot create such an asymmetry, so the two-robot case became the test bed for each new capability: unreliable compasses (Izumi et al. 2012), persistent memory (Suzuki and Yamashita 1999), lights (Viglietta 2013) (Heriban et al. 2018) (Okumura et al. 2023), and finite-state or finite-communication robots (Flocchini et al. 2016). The catalogue therefore splits gathering by team size: any $n \geq 2$, exactly two (rendezvous), and $n \geq 3$.

Level	Value	Formal definition
0	**any** n	Any number $n \geq 2$ of robots.
1	$n = 2$	Exactly two robots (rendezvous).
1	$n \geq 3$	$n \geq 3$ robots.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers → Number of robots.



Claude (Anthropic) was used in cataloguing the problems.

+ no faults
Scheduler
+ SSYNC
Lights (persistent memory)
+ internal light (FSTA)
Agreement on coordinates
disoriented, non-rigid, + $n = 2$
Rendezvous, internal light (FSTA), SSYNC, non-rigid ●● p. 299
disoriented, + rigid, + $n = 2$
Rendezvous, internal light (FSTA), SSYNC, rigid ●● p. 300
+ chirality, + $n \geq 3$
Gathering, internal light (FSTA), SSYNC, non-rigid ●● p. 301
+ external light (FCOM)
Agreement on coordinates
disoriented, non-rigid, + $n = 2$
Rendezvous, external light (FCOM), SSYNC ●● p. 301
+ chirality, + rigid, + $n \geq 3$
Gathering, external light (FCOM), SSYNC, rigid ●● p. 302
oblivious, disoriented, anonymous, + $n \geq 3$
No multiplicity detection, SSYNC ●● p. 302
oblivious, disoriented, anonymous, + $n = 2$
Two robots, SSYNC ●● p. 303
oblivious, disoriented, + homonymous
Homonymous robots, SSYNC ●● p. 303
+ lights (LUMI), + chirality, + $n \geq 3$
Gathering, full lights, SSYNC, no multiplicity detection ●● p. 304
ASYNC, + lights (LUMI)
Agreement on coordinates
disoriented, + $n = 2$
Rendezvous, full lights (LUMI), ASYNC, non-rigid ●● p. 305
+ chirality, + $n \geq 3$
Gathering, full lights, ASYNC, no multiplicity detection ●● p. 305
ASYNC, + external light (FCOM)
Movement
non-rigid, + $n = 2$
Rendezvous, external light (FCOM), ASYNC, non-rigid ●● p. 306
+ rigid, + $n = 2$
Rendezvous, external light (FCOM), ASYNC, rigid ●● p. 307
ASYNC, oblivious, + weak multiplicity, + $n \geq 3$
ASYNC with weak multiplicity detection STANDARD SETTING ●● p. 307
ASYNC, + internal light (FSTA), + $n = 2$
Rendezvous, internal light (FSTA), ASYNC ●● p. 308
+ sequential, + external light (FCOM), + $n \geq 3$
Gathering, external light, sequential scheduler ●● p. 308

Claude (Anthropic) was used in cataloguing the problems.

+ crash faults, + SSYNC
Lights (persistent memory)
oblivious, disoriented, non-rigid, + weak multiplicity, + $n \geq 3$
Crash faults, SSYNC ●● p. 309
+ lights (LUMI), + compass, + rigid, + $n = 2$
Rendezvous, full lights, a crash, SSYNC ●● p. 310
Byzantine, + SSYNC, + $n \geq 3$
Byzantine faults, SSYNC ●● p. 310
Byzantine, + FSYNC, + $n \geq 3$
Byzantine faults, FSYNC ●● p. 311
limited visibility, + transparent, + no faults
Scheduler
ASYNC, + compass
Limited visibility with a compass, ASYNC ●● p. 311
ASYNC, + one axis
Limited visibility, one common axis, ASYNC ●● p. 312
+ FSYNC
Limited visibility, FSYNC, disoriented ●● p. 313
fat, + unlimited visibility
Opacity
opaque, + no faults
Agreement on coordinates
disoriented, + $n \geq 3$
Fat robots, few robots, no common orientation ●● p. 313
+ chirality, + $n \geq 3$
Fat robots, chirality, any number ●● p. 314
+ transparent, + no faults, + $n \geq 3$
Transparent fat robots, no chirality ●● p. 314

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
Rendezvous, internal light (FSTA), SSYNC, non-rigid	● unknown *finite* vs —	● tight 3 vs 3	299
Rendezvous, internal light (FSTA), SSYNC, rigid	● unknown *finite* vs —	● tight 3 vs 3	300
Gathering, internal light (FSTA), SSYNC, non-rigid	● unknown *finite* vs —	● unknown 2 vs 3	301
Rendezvous, external light (FCOM), SSYNC	● unknown *finite* vs —	● tight 3 vs 3	301
Gathering, external light (FCOM), SSYNC, rigid	● unknown *finite* vs —	● tight 3 vs 3	302
No multiplicity detection, SSYNC	● impossible	● impossible	302
Two robots, SSYNC	● impossible	● impossible	303

Claude (Anthropic) was used in cataloguing the problems.

Setting	Time	Colours	p.
Homonymous robots, SSYNC	● unknown *finite* vs *impossible*	● tight 0 vs —	303
Gathering, full lights, SSYNC, no multiplicity detection	● unknown *finite* vs —	● tight 2 vs 2	304
Rendezvous, full lights (LUMI), ASYNC, non-rigid	● unknown *finite* vs —	● tight 2 vs 2	305
Gathering, full lights, ASYNC, no multiplicity detection	● unknown *finite* vs —	● tight 3 vs 2	305
Rendezvous, external light (FCOM), ASYNC, non-rigid	● unknown *finite* vs —	● tight 3 vs 3	306
Rendezvous, external light (FCOM), ASYNC, rigid	● unknown *finite* vs —	● tight 6 vs 3	307
ASYNC with weak multiplicity detection STANDARD SETTING	● unknown *finite* vs —	● tight 0 vs —	307
Rendezvous, internal light (FSTA), ASYNC	● unknown — vs —	● unknown — vs 3	308
Gathering, external light, sequential scheduler	● unknown *finite* vs —	● tight 2 vs 2	308
Crash faults, SSYNC	● unknown *finite* vs *impossible*	● tight 0 vs —	309
Rendezvous, full lights, a crash, SSYNC	● impossible	● impossible	310
Byzantine faults, SSYNC	● impossible	● impossible	310
Byzantine faults, FSYNC	● unknown *finite* vs —	● tight 0 vs —	311
Limited visibility with a compass, ASYNC	● unknown *finite* vs *impossible*	● tight 0 vs —	311
Limited visibility, one common axis, ASYNC	● tight $O(\Delta_0)$ vs $\Omega(\Delta_0)$	● tight 0 vs —	312
Limited visibility, FSYNC, disoriented	● tight $O(\Delta_0^2)$ vs $\Omega(\Delta_0^2)$	● tight 0 vs —	313
Fat robots, few robots, no common orientation	● unknown *finite* vs —	● tight 0 vs —	313
Fat robots, chirality, any number	● unknown *finite* vs —	● tight 0 vs —	314
Transparent fat robots, no chirality	● unknown *finite* vs —	● tight 0 vs —	314

18.8.1 Rendezvous, internal light (FSTA), SSYNC, non-rigid

Adds disoriented, non-rigid, $+n = 2$ VERIFIED

A light only its owner sees, that is constant memory, lets two robots meet in SSYNC with 3 colours if they know δ; 2 colours are impossible.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); lights (persistent memory): internal light only (FSTA): a robot sees its own light, not the others'; number of robots: $n = 2$ (rendezvous). **Not assumed:** agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed; only solvability.	*finite* (Flocchini et al. 2016)	● unknown
Colours	3 (Otaka et al. 2026) Exactly 3 colours when δ is known.	3 (Flocchini et al. 2016)	● tight

Results.

- Otaka et al. (2026) [lower bound]: colours 3. *Bottleneck:* memory. *Holds when:* Theorem 1: 2 colours are impossible, for every $n \geq 2$; holds even with rigid moves, chirality and a common unit.
- Flocchini et al. (2016) [current best]: time *finite*, colours 3. *Assumes:* the robots know the minimum move length δ. Introductory and current best (Theorem 18).

Open questions.

Q1. Rendezvous with internal lights in non-rigid SSYNC when δ is unknown.

Notes. No self-stabilizing algorithm with internal lights exists, for any number of colours (Otaka et al. 2026). No colour-only (class L) algorithm works with internal lights, whatever the memory (Theorem 1 of Flocchini et al. 2016). If the adversary may rescale a robot's frame at every Look, rendezvous with internal lights is impossible for any number of colours (Theorem 1(i) of Masuzawa et al. 2026). Checked against the full texts.

18.8.2 Rendezvous, internal light (FSTA), SSYNC, rigid

Adds disoriented, + rigid, + $n = 2$ VERIFIED

With rigid moves, robots with internal lights meet in SSYNC with exactly 3 colours.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); lights (persistent memory): internal light only (FSTA): a robot sees its own light, not the others'; movement: rigid: every move reaches its destination; number of robots: $n = 2$ (rendezvous). **Not assumed:** agreement on coordinates: no agreement on local coordinate systems; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed; only solvability.	*finite* (Masuzawa et al. 2026)	● unknown
Colours	3 (Otaka et al. 2026) Exactly 3 colours.	3 (Masuzawa et al. 2026)	● tight

Results.

- Otaka et al. (2026) [lower bound]: colours 3. *Bottleneck:* memory. *Holds when:* Theorem 1: 2 colours are impossible.
- Flocchini et al. (2016) [introductory]: time *finite*, colours 6. Theorem 13, from a preset state; each robot uses its own unit of distance.
- Masuzawa et al. (2026) [current best]: time *finite*, colours 3. *Assumes:* each robot keeps a fixed unit of distance, while its rotation and reflection may change at every Look. Theorem 8, from a designated colour. A preprint, not peer reviewed.

Notes. Checked against the full texts.

Claude (Anthropic) was used in cataloguing the problems.

18.8.3 Gathering, internal light (FSTA), SSYNC, non-rigid

Adds + chirality, + $n \geq 3$ VERIFIED

With internal lights, 2 colours gather $n \geq 3$ robots in SSYNC only from restricted starts; 2 colours do not suffice in general, and whether more do is open.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); lights (persistent memory): internal light only (FSTA): a robot sees its own light, not the others'; agreement on coordinates: common chirality (same clockwise orientation); number of robots: $n \geq 3$. **Not assumed:** movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed; only solvability.	*finite* (Otaka et al. 2026)	• unknown
Colours	3 (Otaka et al. 2026) Unconditional gathering with more colours is open.	2 (Otaka et al. 2026)	• unknown

Results.

- Otaka et al. (2026) [lower bound]: colours 3. *Bottleneck:* memory. *Holds when:* Theorem 1: 2 colours are impossible without restrictions on the start.
- Terai et al. (2023) [introductory]: time *finite*, colours 2. *Assumes:* δ known and common; restricted initial configurations. Theorem 8, from starts in which robots are at most 2δ apart.
- Otaka et al. (2026) [current best]: time *finite*, colours 2. *Assumes:* δ known and common; some two-point starts excluded. Theorem 3: far fewer excluded starts; the remaining exclusions cannot all be removed (Theorem 2(3)).

Open questions.

Q1. Gathering with internal lights in SSYNC from every start, with more than 2 colours, and in ASYNC (Otaka et al. 2026).

Notes. Checked against the full texts.

18.8.4 Rendezvous, external light (FCOM), SSYNC

Adds disoriented, non-rigid, + $n = 2$ VERIFIED

With a light only the other robot sees, two robots meet in SSYNC with 3 colours, and 2 colours are impossible.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); lights (persistent memory): external light only (FCOM): a robot sees the others' lights, not its own; number of robots: $n = 2$ (rendezvous). **Not assumed:** agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed; only solvability.	*finite* (Flocchini et al. 2016)	• unknown
Colours	3 (Otaka et al. 2026); 3 (Okumura et al. 2023) Exactly 3 colours.	3 (Flocchini et al. 2016)	• tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Otaka et al. (2026) [lower bound]: colours 3. *Bottleneck:* memory. *Holds when:* Theorem 1: 2 colours are impossible for external lights, for every $n \geq 2$; holds even with rigid moves, chirality and a common unit.
- Okumura et al. (2023) [lower bound]: colours 3. *Bottleneck:* memory. *Holds when:* Theorem 8: for colour-only (class L) algorithms, rigid SSYNC.
- Flocchini et al. (2016) [current best]: time *finite*, colours 3. *Assumes:* class L: the destination is a point on the segment to the other robot and depends only on the colours. Introductory and current best (Theorem 16): self-stabilizing, non-rigid moves, no common unit.

Notes. Checked against the full texts.

18.8.5 Gathering, external light (FCOM), SSYNC, rigid

Adds + chirality, + rigid, + $n \geq 3$ VERIFIED

With external lights, $n \geq 3$ robots gather in rigid SSYNC with 3 colours, which is optimal; 2 suffice if a robot sees who shares its point.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); lights (persistent memory): external light only (FCOM): a robot sees the others' lights, not its own; agreement on coordinates: common chirality (same clockwise orientation); movement: rigid: every move reaches its destination; number of robots: $n \geq 3$. **Not assumed:** multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed; only solvability.	*finite* (Terai et al. 2023)	● unknown
Colours	3 (Otaka et al. 2026) Exactly 3 colours.	3 (Terai et al. 2023)	● tight

Results.

- Otaka et al. (2026) [lower bound]: colours 3. *Bottleneck:* memory. *Holds when:* Theorem 2(1); without seeing the robots at its own point.
- Terai et al. (2023) [current best]: time *finite*, colours 3. Introductory and current best (Theorem 6). With local awareness 2 colours suffice (Theorem 7), and local awareness cannot be dropped (Theorem 2(2) of Otaka et al. 2026).

Open questions.

Q1. Gathering with external lights in ASYNC (Otaka et al. 2026).

Notes. Checked against the full texts.

18.8.6 No multiplicity detection, SSYNC

Adds oblivious, disoriented, anonymous, + $n \geq 3$ VERIFIED

Without multiplicity detection, deterministic oblivious robots cannot gather, for any $n \geq 2$, in SSYNC or ASYNC.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); number of robots: $n \geq 3$. **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary

Claude (Anthropic) was used in cataloguing the problems.

may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Prencipe 2007) Impossible in general.	—	• impossible
Colours	none known Impossible for oblivious robots.	—	• impossible

Results.

- Prencipe (2007) [lower bound]: time *impossible*. *Bottleneck:* indistinguishable multiplicities. *Holds when:* Theorem 2: deterministic oblivious robots, no multiplicity detection; $n \geq 2$: Prencipe proves $n \geq 3$ (Lemmas 5 and 6) and takes $n = 2$ from Suzuki and Yamashita; SSYNC (Atom) and ASYNC (Corda); non-rigid moves, unlimited visibility, no common coordinate system.

Notes. Prencipe uses chirality to mean agreement on North and East (a compass), not handedness. (Agmon and Peleg 2006) attribute the necessity of multiplicity detection in SSYNC and ASYNC to Prencipe's thesis (2002). Checked against the SIROCCO 2005 version; the TCS 2007 version was not read.

18.8.7 Two robots, SSYNC

Adds oblivious, disoriented, anonymous, + $n = 2$ VERIFIED

Two oblivious robots without a common orientation cannot meet in SSYNC: rendezvous needs extra power.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); number of robots: $n = 2$ (rendezvous). **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Flocchini et al. 2016) Impossible.	—	• impossible
Colours	none known Oblivious robots cannot do it; see the luminous variant.	—	• impossible

Results.

- Flocchini et al. (2016) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* $n = 2$, oblivious; without additional assumptions, already in SSYNC; first shown by (Suzuki and Yamashita 1999) (Theorem 3.1), in a model with common handedness and moves that reach their computed destination, so it holds without these.

Notes. Two robots with unreliable compasses can gather if the deviation is below $\pi/2$ (static compasses, SSYNC and ASYNC), $\pi/4$ (dynamic, SSYNC) or $\pi/6$ (dynamic, ASYNC, sufficient only) (Izumi et al. 2012).

18.8.8 Homonymous robots, SSYNC

Adds oblivious, disoriented, + homonymous VERIFIED

Two distinct identifiers (two different algorithms) are necessary and sufficient for gathering in SSYNC.

Claude (Anthropic) was used in cataloguing the problems.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); identifiers: homonymous: robots carry labels, but several may share a label. **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; number of robots: any n.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Asahiro and Yamashita 2025) Time not analysed.	*finite* (Asahiro and Yamashita 2025)	● unknown
Colours	none known No colours.	0 (Asahiro and Yamashita 2025)	● tight

Results.

- Asahiro and Yamashita (2025) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* with one identifier (anonymous robots); with one crash fault, for any number of identifiers.
- Asahiro and Yamashita (2025) [current best]: time *finite*, colours 0. *Assumes:* 2 identifiers. The minimum algorithm size of gathering is 2; gathering the non-faulty robots under crashes needs 3.

Open questions.

Q1. What is the minimum number of identifiers for Byzantine-tolerant gathering? (Asahiro and Yamashita 2025)

Notes. Homonymous robots were first studied for pattern formation (Bouzid and Lamani 2011); the term comes from Byzantine agreement with homonyms (Delporte-Gallet et al. 2013).

18.8.9 Gathering, full lights, SSYNC, no multiplicity detection

Adds + lights (LUMI), + chirality, + $n \geq 3$ VERIFIED

With lights and chirality, $n \geq 3$ robots that cannot detect multiplicities gather in SSYNC with 2 colours, which is optimal.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: common chirality (same clockwise orientation); number of robots: $n \geq 3$. **Not assumed:** movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed; only solvability.	*finite* (Terai et al. 2023)	● unknown
Colours	2 (Prencipe 2007) Exactly 2 colours.	2 (Terai et al. 2023)	● tight

Results.

- Prencipe (2007) [lower bound]: colours 2. *Bottleneck:* indistinguishable multiplicities. *Holds when:* oblivious robots without multiplicity detection cannot gather in SSYNC (one colour).
- Terai et al. (2023) [current best]: time *finite*, colours 2. Introductory and current best (Theorem 5); non-rigid moves; a robot sees the set of colours at a point, not the number of robots.

Claude (Anthropic) was used in cataloguing the problems.

Open questions.

Q1. Gathering with lights without chirality (Terai et al. 2023).

Notes. Checked against the full text.

18.8.10 Rendezvous, full lights (LUMI), ASYNC, non-rigid

Adds disoriented, $+n = 2$ VERIFIED

Two robots whose lights both can see meet in ASYNC with non-rigid moves using 2 colours, which is optimal, from any initial colours.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; number of robots: $n = 2$ (rendezvous). **Not assumed:** scheduler: asynchronous (ASYNC); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed; only solvability.	*finite* (Heriban et al. 2018)	● unknown
Colours	2 (Viglietta 2013) Exactly 2 colours.	2 (Heriban et al. 2018)	● tight

Results.

- Viglietta (2013) [lower bound]: colours 2. *Bottleneck:* symmetry breaking. *Holds when:* Proposition 1.1: one colour is an oblivious robot, which cannot meet in SSYNC already with rigid moves (Suzuki and Yamashita).
- Das et al. (2016) [introductory]: time *finite*, colours 4. Theorem 3.3; the authors call it self-stabilizing.
- Viglietta (2013) [also]: time *finite*, colours 3. *Assumes:* class L: the destination is a point on the segment to the other robot and depends only on the colours. Theorem 2.7: self-stabilizing, colours only; 3 colours are optimal among such algorithms (Theorems 4.10 and 4.11).
- Heriban et al. (2018) [current best]: time *finite*, colours 2. *Assumes:* a robot can tell whether the other robot is at its position. Theorem 1: self-stabilizing; a robot only tests whether the other robot is at its own position.

Notes. Easier settings were solved first with 2 colours: SSYNC with colours only, from any initial colours (Theorem 2.2 of Viglietta 2013); rigid ASYNC from a preset colour (Theorem 2.4 of Viglietta 2013, Theorem 3 of Okumura et al. 2017); ASYNC in which no robot looks between another's Look and Compute, or with δ known (Okumura et al. 2017). A model checker confirms the 2- and 3-colour ASYNC algorithms and finds Viglietta's 2-colour SSYNC algorithm incorrect in ASYNC (Défago et al. 2023). Checked against the full texts.

18.8.11 Gathering, full lights, ASYNC, no multiplicity detection

Adds + chirality, $+n \geq 3$ VERIFIED

With lights and chirality, $n \geq 3$ robots gather in ASYNC with non-rigid moves and no multiplicity detection using 3 colours; whether 2 suffice is open.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: common chirality (same clockwise orientation); number of robots: $n \geq 3$. **Not assumed:** scheduler: asynchronous (ASYNC); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed; only solvability.	*finite* (Nakai et al. 2021)	● unknown
Colours	2 (Prencipe 2007) $\Theta(1)$; the exact number is 2 or 3.	3 (Nakai et al. 2021)	● tight

Results.

- Prencipe (2007) [lower bound]: colours 2. *Bottleneck:* indistinguishable multiplicities. *Holds when:* inherited: one colour is an oblivious robot.
- Terai et al. (2023) [introductory]: time *finite*, colours 10. Concluding remarks: the SSYNC algorithm through the $5c$-colour simulation of a c-colour algorithm (Das et al. 2016).
- Nakai et al. (2021) [current best]: time *finite*, colours 3. Theorem 6; 6 colours by a new $3c$-colour simulation (for a c-colour algorithm) of unfair SSYNC (Theorem 4). The authors conjecture that 3 is optimal.

Open questions.

Q1. Is 3 the minimum number of colours for ASYNC gathering with lights, and can chirality be dropped (Nakai et al. 2021)?

Q2. Self-stabilizing gathering of n robots with lights (Heriban et al. 2018).

Notes. Checked against the full texts.

18.8.12 Rendezvous, external light (FCOM), ASYNC, non-rigid

Adds non-rigid, $+n = 2$ VERIFIED

In ASYNC with non-rigid moves, robots with external lights meet with 3 colours if they know the minimum move δ; without δ the question is open, and colour-only algorithms cannot work.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; lights (persistent memory): external light only (FCOM): a robot sees the others' lights, not its own; number of robots: $n = 2$ (rendezvous). **Not assumed:** scheduler: asynchronous (ASYNC); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed; only solvability.	*finite* (Flocchini et al. 2016)	● unknown
Colours	3 (Otaka et al. 2026) 3 colours when δ is known.	3 (Flocchini et al. 2016)	● tight

Results.

- Otaka et al. (2026) [lower bound]: colours 3. *Bottleneck:* memory. *Holds when:* SSYNC bound (Theorem 1), hence also in ASYNC.
- Flocchini et al. (2016) [current best]: time *finite*, colours 3. *Assumes:* the robots know the minimum move length δ. Introductory and current best (Theorem 19), from a preset colour.

Open questions.

Q1. Rendezvous with external lights in non-rigid ASYNC when δ is unknown (Flocchini et al. 2016) (Masuzawa et al. 2026).

Claude (Anthropic) was used in cataloguing the problems.

Notes. No colour-only (class L) algorithm with external lights works in ASYNC, for any number of colours (Theorem 14 of Flocchini et al. 2016). If no robot looks between another robot's Look and Compute, colour-only algorithms with 3, 4 and 5 colours are optimal for rigid moves, non-rigid moves from equal colours, and self-stabilization respectively (Okumura et al. 2023). Checked against the full texts.

18.8.13 Rendezvous, external light (FCOM), ASYNC, rigid

Adds + rigid, + $n = 2$ VERIFIED

With rigid moves, robots with external lights meet in ASYNC; the best count is 6 colours against a lower bound of 3.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; lights (persistent memory): external light only (FCOM): a robot sees the others' lights, not its own; movement: rigid: every move reaches its destination; number of robots: $n = 2$ (rendezvous). **Not assumed:** scheduler: asynchronous (ASYNC); agreement on coordinates: no agreement on local coordinate systems; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed; only solvability.	*finite* (Masuzawa et al. 2026)	● unknown
Colours	3 (Otaka et al. 2026) $\Theta(1)$ colours; the exact number is between 3 and 6.	6 (Masuzawa et al. 2026)	● tight

Results.

- Otaka et al. (2026) [lower bound]: colours 3. *Bottleneck:* memory. *Holds when:* SSYNC bound (Theorem 1), hence also in ASYNC.
- Flocchini et al. (2016) [introductory]: time *finite*, colours 12. Theorem 15, from a preset colour; each robot uses its own unit of distance.
- Masuzawa et al. (2026) [current best]: time *finite*, colours 6. *Assumes:* each robot keeps a fixed unit of distance, while its rotation and reflection may change at every Look. Theorem 10, from a designated colour. A preprint, not peer reviewed.

Open questions.

Q1. Close the gap between 3 and 6 colours (Masuzawa et al. 2026).

Notes. If the adversary may also rescale a robot's frame at every Look, rendezvous with external lights is impossible in ASYNC for any number of colours (Theorem 1(ii) of Masuzawa et al. 2026). Checked against the full texts.

18.8.14 ASYNC with weak multiplicity detection

Adds ASYNC, oblivious, + weak multiplicity, + $n \geq 3$ STANDARD SETTING VERIFIED

With weak multiplicity detection, $n \geq 3$ oblivious, disoriented, asynchronous robots gather from any initial configuration.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; multiplicity detection: weak: a robot sees whether a point holds more than one robot; number of robots: $n \geq 3$. **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; randomness: deterministic; identifiers: anonymous: all robots are identical.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known No epoch bound is recorded.	*finite* (Cieliebak et al. 2012)	• unknown
Colours	none known No colours.	0 (Cieliebak et al. 2012)	• tight

Results.

- Cieliebak et al. (2012) [current best]: time *finite*, colours 0. Introductory and current best: solvable "for any $n > 2$ and any initial configuration", with non-rigid moves.

Open questions.

Q1. How many epochs does oblivious ASYNC gathering with weak multiplicity detection need?

Notes. Gathering is point formation, the pattern of a single point with multiplicity n (Flocchini et al. 2017). Under sequential schedulers, weak multiplicity detection is necessary and sufficient for gathering, and under FSYNC no multiplicity detection is needed (Flocchini et al. 2026).

18.8.15 Rendezvous, internal light (FSTA), ASYNC

Adds ASYNC, + internal light (FSTA), + $n = 2$ VERIFIED

Whether robots with internal lights can meet in ASYNC is open.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; lights (persistent memory): internal light only (FSTA): a robot sees its own light, not the others'; number of robots: $n = 2$ (rendezvous). **Not assumed:** scheduler: asynchronous (ASYNC); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Open.	—	• unknown
Colours	3 (Otaka et al. 2026) Open.	—	• unknown

Results.

- Otaka et al. (2026) [lower bound]: colours 3. *Bottleneck:* memory. *Holds when:* SSYNC bound (Theorem 1), hence also in ASYNC.

Open questions.

Q1. Can robots with internal lights solve rendezvous in ASYNC (Flocchini et al. 2016) (Masuzawa et al. 2026)?

Notes. If no robot looks while another moves (Move-atomic ASYNC), 4 colours suffice with rigid moves (Theorem 9 of Masuzawa et al. 2026, a preprint). Checked against the full texts.

18.8.16 Gathering, external light, sequential scheduler

Adds + sequential, + external light (FCOM), + $n \geq 3$ VERIFIED

If one robot moves per round, 2 external colours gather $n \geq 3$ robots, self-stabilizing, without chirality;

Claude (Anthropic) was used in cataloguing the problems.

oblivious robots cannot.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: sequential: exactly one robot is activated per round; lights (persistent memory): external light only (FCOM): a robot sees the others' lights, not its own; number of robots: $n \geq 3$. **Not assumed:** agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed; only solvability.	*finite* (Terai et al. 2023)	● unknown
Colours	2 (Terai et al. 2023) Exactly 2 colours.	2 (Terai et al. 2023)	● tight

Results.

- Terai et al. (2023) [lower bound]: colours 2. *Bottleneck:* memory. *Holds when:* oblivious robots cannot gather under this scheduler.
- Terai et al. (2023) [current best]: time *finite*, colours 2. Introductory and current best (Theorem 9); non-rigid moves. With internal lights and a round-robin scheduler, 2 colours suffice with rigid moves (Theorem 10).

Notes. Checked against the full text.

18.8.17 Crash faults, SSYNC

Adds oblivious, disoriented, non-rigid, + weak multiplicity, + $n \geq 3$ VERIFIED

With weak multiplicity detection and no common orientation, the correct robots gather despite up to $n - 1$ crashes.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: up to f robots may crash; scheduler: semi-synchronous (SSYNC); multiplicity detection: weak: a robot sees whether a point holds more than one robot; number of robots: $n \geq 3$. **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Pattanayak et al. 2019b) Time not analysed.	*finite* (Pattanayak et al. 2019b)	● unknown
Colours	none known No colours.	0 (Pattanayak et al. 2019b)	● tight

Results.

- Pattanayak et al. (2019b) [lower bound]: time *impossible*. *Bottleneck:* indistinguishable multiplicities. *Holds when:* if the capabilities are relaxed further (the paper shows its model is minimal).
- Agmon and Peleg (2006) [introductory]: time *finite*, colours 0. *Assumes:* one crash; common chirality; $n \geq 3$; a Look returns a multiset (multiplicities are seen); initial configuration with at most one point with a multiplicity. Theorem 3.8, SSYNC, non-rigid moves of at least a fixed distance S; checked against the full text.
- Bouzid et al. (2013) [also]: time *finite*, colours 0. *Assumes:* strong multiplicity detection; chirality; any $f < n$. Wait-free; arbitrary initial configurations.
- Bramas and Tixeuil (2015) [also]: time *finite*, colours 0. *Assumes:* strong multiplicity detection; no chirality.

Claude (Anthropic) was used in cataloguing the problems.

- Pattanayak et al. (2019b) [current best]: time *finite*, colours 0. *Assumes:* $f \leq n-1$. Wait-free, weak multiplicity detection, no agreement on coordinates. In ASYNC with instantaneous computation it tolerates $\lfloor n/2 \rfloor - 2$ crashes for $n \geq 7$.

Open questions.

Q1. "The maximum number of faults under which a solution is still feasible, for gathering and other tasks, has yet to be established" (Agmon and Peleg 2006), in particular in ASYNC.

18.8.18 Rendezvous, full lights, a crash, SSYNC

Adds + lights (LUMI), + compass, + rigid, + $n = 2$ VERIFIED

If the correct robot must reach a crashed one, two robots cannot meet in SSYNC, even with unboundedly many colours, rigid moves and a common coordinate system.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: up to f robots may crash; scheduler: semi-synchronous (SSYNC); lights (persistent memory): full light (LUMI): every robot sees all lights, including its own; agreement on coordinates: agreement on both axes (common compass); movement: rigid: every move reaches its destination; number of robots: $n = 2$ (rendezvous). **Not assumed:** multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Statement for this setting. Stand-up indulgent rendezvous: if one robot crashes, the other must reach the crashed robot's position.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Bramas et al. 2023b) Unsolvable.	—	• impossible
Colours	none known Unsolvable for any number of colours.	—	• impossible

Results.

- Bramas et al. (2023b) [lower bound]: time *impossible*. *Bottleneck:* indistinguishable crashes. *Holds when:* Theorem 1: SSYNC; holds with full lights of infinitely many colours, rigid moves and a common coordinate system.

Notes. With mobility faults, where a faulty robot stops moving but its light keeps working, 3 colours are necessary and sufficient for two robots in ASYNC (Pramanick et al. 2025), a preprint. Checked against the full texts.

18.8.19 Byzantine faults, SSYNC

Adds Byzantine, + SSYNC, + $n \geq 3$ VERIFIED

Three robots with one Byzantine robot can neither gather nor converge in SSYNC.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; scheduler: semi-synchronous (SSYNC); number of robots: $n \geq 3$. **Not assumed:** faults: up to f Byzantine robots; lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Agmon and Peleg 2006) Impossible for $n = 3$, $f = 1$.	—	• impossible
Colours	none known Impossible.	—	• impossible

Results.

- Agmon and Peleg (2006) [lower bound]: time *impossible*. *Bottleneck:* indistinguishable Byzantine agents. *Holds when:* $n = 3$, $f = 1$ (Theorem 4.4); proved for SSYNC, hence also for ASYNC, which is what the abstract states; impossible for convergence too; holds even with common chirality and multiplicity detection (the paper's model); the adversary moves the Byzantine robot to make the configurations seen by two correct robots equivalent, and stops correct robots early (non-rigid moves).

Notes. Checked against the full text of (Agmon and Peleg 2006): Theorem 4.4 is stated for SSYNC; the abstract says "in an asynchronous environment", and the impossibility carries over to ASYNC because, as the paper notes citing Prencipe, every problem solvable in ASYNC is solvable in SSYNC. Theorem 4.1 first shows that any algorithm that sometimes keeps an active robot still fails.

18.8.20 Byzantine faults, FSYNC

Adds Byzantine, + FSYNC, + $n \geq 3$ VERIFIED

In FSYNC, gathering tolerates f Byzantine robots when $n \geq 3f + 1$.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; scheduler: fully synchronous (FSYNC); number of robots: $n \geq 3$. **Not assumed:** faults: up to f Byzantine robots; lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Whether $3f + 1$ is necessary is open.	*finite* (Agmon and Peleg 2006)	• unknown
Colours	none known No colours.	0 (Agmon and Peleg 2006)	• tight

Results.

- Agmon and Peleg (2006) [current best]: time *finite*, colours 0. *Assumes:* $n \geq 3f + 1$; common chirality; a Look returns a multiset (multiplicities are seen). Introductory and current best (Theorem 5.10, with $n \geq 3f + 1$); $n = 3$ tolerates $f = 1$ (Theorem 5.3). Checked against the full text.

Notes. Défago et al. study self-stabilizing gathering under crash or Byzantine faults (Distributed Computing 2020; bounds not checked).

18.8.21 Limited visibility with a compass, ASYNC

Adds ASYNC, + compass VERIFIED

With a common compass, asynchronous robots with limited visibility gather in finite time.

Assumed: body: point robots; opacity: transparent: every robot sees every robot within range; faults: no faults; agreement on coordinates: agreement on both axes (common compass). **Not assumed:** visibility: limited: robots see only within a fixed radius; scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the

Claude (Anthropic) was used in cataloguing the problems.

adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical; number of robots: any n.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Flocchini et al. 2005) Time not analysed.	*finite* (Flocchini et al. 2005)	● unknown
Colours	none known No colours.	0 (Flocchini et al. 2005)	● tight

Results.

- Flocchini et al. (2005) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* if the initial distance graph is disconnected (Lemma 4.7); two robots at distance more than V under a fully synchronous scheduler see the same empty view and keep their distance; holds for any deterministic algorithm, with a compass. Not a bound on the variant itself: it shows that connectivity of the initial distance graph is necessary.
- Flocchini et al. (2005) [current best]: time *finite*, colours 0. *Assumes:* initial distance graph (edges between robots at distance at most V) connected; no multiplicity detection; agreement on the directions and orientations of both axes, not on unit or origin. Introductory and current best (Theorem 4.2): all robots gather at a point on the rightmost initial vertical axis. Non-rigid moves with a minimum distance δ.

Notes. Checked against the full text of (Flocchini et al. 2005) (TCS 2005; preliminary version STACS 2001): robots are oblivious, anonymous, fully asynchronous (ASYNC), with non-rigid moves that cover at least a distance $\delta_r > 0$, no multiplicity detection, and "orientation": agreement on the directions and orientations of both axes, but not on the unit distance or the origin. The authors conclude that "with respect to gathering, orientation is at least as powerful as instantaneous movements".

18.8.22 Limited visibility, one common axis, ASYNC

Adds ASYNC, + one axis

VERIFIED

With one common axis, asynchronous robots with limited visibility gather in time linear in the initial diameter, which is optimal.

Assumed: body: point robots; opacity: transparent: every robot sees every robot within range; faults: no faults; agreement on coordinates: agreement on one axis (no common chirality). **Not assumed:** visibility: limited: robots see only within a fixed radius; scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical; number of robots: any n.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(\Delta_0)$ (Poudel and Sharma 2021) $\Theta(\Delta_0)$.	$O(\Delta_0)$ (Poudel and Sharma 2021)	● tight
Colours	none known No colours.	0 (Poudel and Sharma 2021)	● tight

Results.

- Poudel and Sharma (2021) [lower bound]: time $\Omega(\Delta_0)$. *Bottleneck:* distance. *Holds when:* Δ_0 = Euclidean diameter of the initial configuration.
- Poudel and Sharma (2021) [current best]: time $O(\Delta_0)$, colours 0. Introductory and current best; FSYNC $O(g)$, g the diameter (in hops) of the initial visibility graph, was known before.

Claude (Anthropic) was used in cataloguing the problems.

18.8.23 Limited visibility, FSYNC, disoriented

Adds + FSYNC VERIFIED

Disoriented robots with limited visibility gather in $O(\Delta_0^2)$ synchronous rounds, Δ_0 the initial diameter.

Assumed: body: point robots; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: fully synchronous (FSYNC). **Not assumed:** visibility: limited: robots see only within a fixed radius; lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical; number of robots: any n.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(\Delta_0^2)$ (Castenow et al. 2022) Tight within the class of local convex-hull protocols; for arbitrary protocols only the trivial bound is known.	$O(\Delta_0^2)$ (Castenow et al. 2022)	• tight
Colours	none known No colours.	0 (Castenow et al. 2022)	• tight

Results.

- Castenow et al. (2022) [lower bound]: time $\Omega(\Delta_0^2)$. *Bottleneck:* locality. *Holds when:* only for protocols that move inside the local convex hull of the neighbourhood; Δ_0 = largest initial distance between two robots.
- Degener et al. (2011) [introductory]: time $O(n^2)$, colours 0. Go-To-The-Center; $\Theta(n^2)$ for that algorithm (as restated by Castenow et al. 2022).
- Castenow et al. (2022) [current best]: time $O(\Delta_0^2)$, colours 0. Every λ-contracting protocol, including Go-To-The-Center; works in two and three dimensions.

Open questions.

Q1. Is $\Omega(\Delta_0^2)$ necessary for every protocol, not only local convex-hull ones?

Notes. Δ_0 here is a distance, not the maximum degree.

18.8.24 Fat robots, few robots, no common orientation

Adds disoriented, + $n \geq 3$ VERIFIED

Three and four oblivious fat robots, which block each other's view and motion, gather in ASYNC without any common orientation.

Assumed: visibility: unlimited; faults: no faults; number of robots: $n \geq 3$. **Not assumed:** body: fat robots (unit discs); opacity: opaque: collinear robots hide each other; scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Statement for this setting. Fat robots (unit discs) cannot share a point: gathering asks for a connected configuration of touching discs in which every robot sees all others.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Czyzowicz et al. 2009)	• unknown
Colours	none known No colours.	0 (Czyzowicz et al. 2009)	• tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Czyzowicz et al. (2009) [current best]: time *finite*, colours 0. *Assumes:* $n \leq 4$; robots know n. Introductory and current best for few robots: Theorem 3.1 ($n = 3$) and the four-robot algorithm of Section 4.

Open questions.

Q1. Gathering any number of fat robots without chirality (Agathangelou et al. 2013).

Notes. Checked against the full text of (Czyzowicz et al. 2009) (TCS 2009; OPODIS 2006). Model: closed unit discs, identical and anonymous, oblivious, silent, no common coordinate system (but a common unit, the radius), ASYNC. A robot sees another if some segment between their boundary circles meets no third robot; a robot that touches another stops and ends its cycle; the adversary may stop a move after a distance δ unknown to the robots. Gathering means a connected union of discs in which every robot sees all others. The authors could not generalise to more robots ("We were unable to do it").

18.8.25 Fat robots, chirality, any number

Adds + chirality, + $n \geq 3$ VERIFIED

With chirality and known n, any number of opaque fat robots gather in ASYNC with non-rigid moves.

Assumed: visibility: unlimited; faults: no faults; agreement on coordinates: common chirality (same clockwise orientation); number of robots: $n \geq 3$. **Not assumed:** body: fat robots (unit discs); opacity: opaque: collinear robots hide each other; scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Statement for this setting. Gathering is Definition 1 of (Agathangelou et al. 2013): a connected configuration (every disc tangent to another), fully visible (every robot sees all others) and terminal, with all centres on the convex hull.

Measure	Lower bound	Best upper bound	Status
Time	none known Time is not analysed.	*finite* (Agathangelou et al. 2013)	● unknown
Colours	none known No colours.	0 (Agathangelou et al. 2013)	● tight

Results.

- Agathangelou et al. (2013) [current best]: time *finite*, colours 0. *Assumes:* n known; the radius is the common unit of distance; robots that become tangent stop. Introductory for any n and current best (Theorem 26).

Open questions.

Q1. Remove the chirality assumption (Agathangelou et al. 2013).

Notes. Checked against the full text (arXiv 1209.3904).

18.8.26 Transparent fat robots, no chirality

Adds + transparent, + no faults, + $n \geq 3$ VERIFIED

Transparent fat robots without chirality gather into a layered pattern from every start whose robots can be ordered, which is exactly the asymmetric starts.

Claude (Anthropic) was used in cataloguing the problems.

Assumed: visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; number of robots: $n \geq 3$. **Not assumed:** body: fat robots (unit discs); scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Statement for this setting. Transparent fat robots do not block sight but block movement. Gathering asks for a layered pattern around the centre of the smallest enclosing circle, with a unique robot at the centre (Chaudhuri and Mukhopadhyaya 2015).

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Chaudhuri and Mukhopadhyaya 2015)	• unknown
Colours	none known No colours.	0 (Chaudhuri and Mukhopadhyaya 2015)	• tight

Results.

- Chaudhuri and Mukhopadhyaya (2010) [introductory]: time *finite*, colours 0. As described by (Chaudhuri and Mukhopadhyaya 2015); not read.
- Chaudhuri and Mukhopadhyaya (2015) [current best]: time *finite*, colours 0. *Assumes:* $n \geq 5$; robots start separated (no two discs touching); rigid moves. A configuration can be ordered, and a leader elected, if and only if it is asymmetric (Theorem 6); the algorithm gathers from every orderable start.

Open questions.

Q1. Gathering transparent fat robots from symmetric starts; the paper proves no general impossibility (Chaudhuri and Mukhopadhyaya 2015).

Notes. Gathering at a given point with limited visibility (dense packing around the point) is a different goal; it is studied by Cord-Landwehr et al. (SOFSEM 2011) and Sharma et al. (TAAS 2017), not yet read. Checked against the full text of (Chaudhuri and Mukhopadhyaya 2015) (arXiv 1208.4484).

18.9 Near-gathering

Robots must get within a small distance of each other without ever colliding.

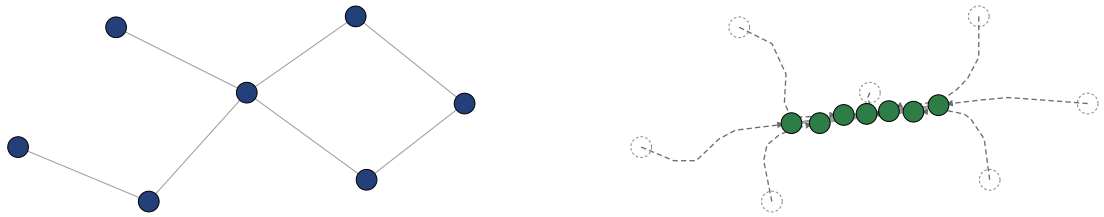
First posed in 2012 (Pagli et al. 2015). Introduced at SIROCCO 2012.

Problem statement and measures

Robots with limited visibility, each seeing only the robots within its **visibility radius** V, start at distinct points whose **visibility graph** (two robots joined by an edge when they are within distance V) is connected. They must reach a configuration in which every robot sees all others, without a **collision** (two robots at the same point) ever happening.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: neighbours within V share an edge

(b) Final: every robot sees all, none collide (arrows: moves; dashed circles: starting positions)

Figure 18.9: Near-gathering: robots with limited visibility, starting at distinct points whose visibility graph is connected, must come close enough that every robot sees all others, without two robots ever occupying the same point (Pagli et al. 2015). Shown in FSYNC rounds: the Go-To-the-Center rule of (Castenow et al. 2022): each robot moves part of the way towards the centre of the smallest circle enclosing the robots it sees, staying within $V/2$ of the midpoint with each neighbour, so edges of the visibility graph are never lost.

A first look

Imagine short-sighted people in fog who must come close enough that everybody can see everybody, without bumping into each other. In the problem the people are n **robots**, points at distinct places in the plane, and each sees only the robots within distance V (its **visibility radius**). Join two robots by an **edge** when they are within distance V, and call them **neighbours**; the robots and these edges form the **visibility graph**, and at the start it is **connected** (every robot can be reached from every other along edges). Each robot reads positions in its own **local coordinate system** (its private axes and unit of length) and repeats a **Look–Compute–Move cycle**: it takes a **snapshot** of where the robots it sees are (Look), decides on a destination point (Compute) and moves towards it (Move). Robots are identical, silent and **oblivious** (no memory between cycles); in the basic version their local coordinate systems agree on the direction of one axis, and the **scheduler**, controlled by an **adversary** (an imagined opponent) that decides when each robot acts and how long each phase takes, is ASYNC (each phase may take any finite time). The robots must reach a **configuration** (the positions of all robots) in which every robot sees all others, and two robots may never be at the same point (a **collision**).

1. Algorithm: Go to the centre of what you see, in small steps, keeping your neighbours. (Castenow et al. 2022) (SSYNC, limited visibility, disoriented, p. 319)

1. Compute the **smallest enclosing circle** of the robots you can see, and aim for its centre (the **Go-To-the-Center** rule).
2. For every robot you see (every neighbour), stay inside the disc of radius $V/2$ around the midpoint between you and it. Your neighbour obeys the same rule, so the two of you stay within distance V, your edge is kept, and the visibility graph stays connected.
3. To avoid collisions, move only a short distance and only part of the way to your target. Here robots see a little farther than V: edges still join robots within distance V (the **connectivity range**), but each robot sees every robot within distance $V + \tau$ (its **viewing range**), for a constant $0 < \tau \leq 2V/3$ (Castenow et al. 2022). So a robot can also compute where its nearby neighbours are heading and keep clear of them.
4. Stop when everyone you see is very close: since the visibility graph is connected, then everyone is close.

Claude (Anthropic) was used in cataloguing the problems.

Why it works and what it costs. Collisions are the new difficulty compared with gathering. Even for two robots, the simple rule "move half-way to the centre of gravity" can collide in ASYNC: robot r at $(0, 0)$ heads for $(781, 0)$; the scheduler pauses it at $(52, 0)$ while robot s, starting at $(3124, 0)$ and always seeing r at $(52, 0)$, makes five cycles and arrives at $(781, 0)$; then r finishes its move onto s (Pagli et al. 2015). Castenow, Harbig, Jung, Kling, Knollmann and Meyer auf der Heide show that any gathering rule of a broad class, including Go-To-the-Center, can be made collision-free and solves near-gathering for disoriented robots in $O(\Delta_0^2)$ **epochs** (an epoch is an interval of time in which every robot completes at least one full cycle) in SSYNC, where Δ_0 is the initial diameter (largest distance between two robots) and V is the unit (Theorem 18 of Castenow et al. 2022). The steps above simplify their construction.

Why it matters. It lets short-sighted robots reach a state where they all see each other, after which any algorithm for robots with unlimited visibility can take over.

2. Lower bound: A big regular polygon shrinks slowly. (Castenow et al. 2022) (SSYNC, limited visibility, disoriented, p. 319) Place n robots on a regular n-gon with sides of length 1, the visibility radius, so each robot sees only its two neighbours. The adversary chooses local coordinate systems so that all robots see the same thing and the polygon stays regular forever. Each robot sees itself and two neighbours nearly on a straight line: the triangle they form has height at most about π/n. If a robot always moves inside the triangle of what it sees, it moves towards the centre by at most about π/n per round (a step of time in which each robot that the scheduler activates performs one full cycle). The polygon's radius is about $n/(2\pi)$, so shrinking it to two thirds of its size takes about n^2 rounds.

Why it holds. The diameter of this start is $\Delta_0 \approx n/\pi$, so $\Omega(\Delta_0^2)$ rounds are needed by every protocol whose robots always move inside the region spanned by what they see (Theorem 4 of Castenow et al. 2022); Go-To-the-Center and its collision-free version are of this kind, so their $O(\Delta_0^2)$ bound is tight for them. Whether moving outside that region can be faster is open.

Why it matters. It shows that local rules pay quadratically in the size of the swarm, and points to where a faster algorithm would have to do something different.

3. Impossibility: Robots that see nobody stay apart. (Pagli et al. 2015) (ASYNC, limited visibility, one common axis, p. 318) Why must the start be connected? Take two robots farther apart than V. Each sees only itself, in every cycle. A deterministic oblivious robot with the same snapshot always computes the same move in its own coordinates. The adversary gives both robots the same orientation and unit and wakes them together: they make exactly the same move in the plane, so their distance never changes. They never see each other.

Why it holds. So near-gathering cannot be solved from a disconnected start, which Pagli, Prencipe and Viglietta note is "easily seen" (Pagli et al. 2015); the same holds for gathering (Flocchini et al. 2005). From a connected start, Pagli et al. solve near-gathering in ASYNC, without collisions and with termination, for robots that agree on one axis (Pagli et al. 2015).

Why it matters. It is the reason every limited-visibility algorithm assumes a connected start and works hard to keep it connected.

What the community started looking at

Near-gathering was introduced to **get robots close without collisions**, a safe first phase for other tasks (Pagli et al. 2015); later work asked how fast disoriented robots can do it (Castenow et al. 2022).

Claude (Anthropic) was used in cataloguing the problems.

Relations to other problems

- **Relaxation of Gathering.** Gathering requires a common point; near-gathering only requires being close, but forbids collisions. It is used as a first phase before forming patterns.
- **Related: Convergence.** Near-gathering is also a relaxation of gathering: robots must get close (within visibility of each other) without ever colliding (Pagli et al. 2015), while convergence lets them approach a point only in the limit.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions
- + point, + transparent, + no faults
  - *Scheduler*
    - ASYNC, + one axis
      - **ASYNC, limited visibility, one common axis** STANDARD SETTING ●● p. 318
    - + SSYNC
      - **SSYNC, limited visibility, disoriented** ●● p. 319

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
ASYNC, limited visibility, one common axis STANDARD SETTING	● unknown *finite* vs —	● tight 0 vs —	318
SSYNC, limited visibility, disoriented	● tight $O(\Delta_0^2)$ vs $\Omega(\Delta_0^2)$	● tight 0 vs —	319

18.9.1 ASYNC, limited visibility, one common axis

Adds ASYNC, + one axis STANDARD SETTING VERIFIED

Robots get close together without ever colliding, from a connected start, if they agree on one axis.

Assumed: body: point robots; opacity: transparent: every robot sees every robot within range; faults: no faults; agreement on coordinates: agreement on one axis (no common chirality). **Not assumed:** visibility: limited: robots see only within a fixed radius; scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Pagli et al. 2015)	● unknown
Colours	none known No colours.	0 (Pagli et al. 2015)	● tight

Results.

- Pagli et al. (2015) [current best]: time *finite*, colours 0. Introductory and current best; collision-free with termination.

Open questions.

Q1. Pagli, Prencipe and Viglietta conjecture that near-gathering is impossible without agreement on one axis.

18.9.2 SSYNC, limited visibility, disoriented

Adds + SSYNC VERIFIED

Disoriented robots near-gather in $O(\Delta_0^2)$ epochs in SSYNC.

Assumed: body: point robots; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC). **Not assumed:** visibility: limited: robots see only within a fixed radius; lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(\Delta_0^2)$ (Castenow et al. 2022) Within contracting protocols.	$O(\Delta_0^2)$ (Castenow et al. 2022)	● tight
Colours	none known No colours.	0 (Castenow et al. 2022)	● tight

Results.

- Castenow et al. (2022) [lower bound]: time $\Omega(\Delta_0^2)$. *Bottleneck:* locality. *Holds when:* only for λ-contracting protocols.
- Castenow et al. (2022) [current best]: time $O(\Delta_0^2)$, colours 0. Theorem 18; first near-gathering protocols for disoriented robots.

Open questions.

Q1. Near-gathering in $O(\Delta_0)$ epochs?

18.10 Partitioning

Robots must partition themselves into k groups of exactly n/k robots each, each group at its own point.

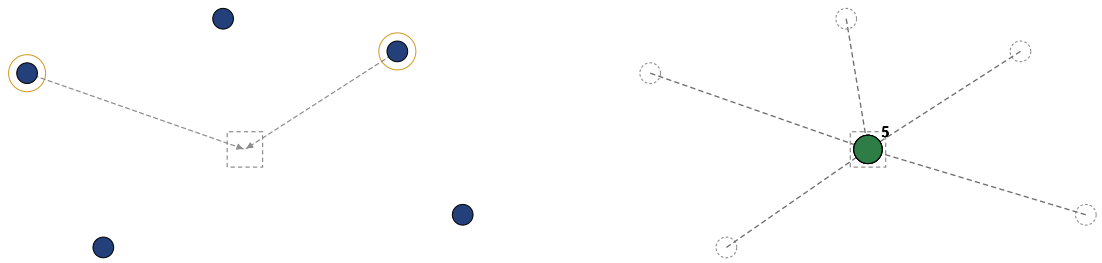
First posed in 2021 (Bhagat et al. 2021). The fixed-point version (k-EPF) was posed with k-circle formation in 2021; the earlier partitioning of Efrima and Peleg fixes the number of groups instead and is recorded under k-grouping.

Claude (Anthropic) was used in cataloguing the problems.

Problem statement and measures

n robots, with k dividing n, must partition themselves into k groups of exactly n/k robots, each group gathered at its own point, and stay there. The k points, and so the group size n/k, are given (the papers' k-EPF and k-circle formation name the group size k, which is n/k here).

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).



(a) Start: $n = 5$, $k = 1$: one fixed point

(b) Final: all $n = 5$ robots on the fixed point (arrows: moves; dashed circles: starting positions)

Figure 18.10: Partitioning (k-EPF): n robots, k dividing n, must gather in groups of exactly n/k robots at the k given fixed points, each point receiving one group (Bhagat et al. 2021). Shown: the simplest case $k = 1$, one fixed point (dashed square), in SSYNC rounds; ringed robots are activated and move straight towards the point, a move possibly stopping early, until all are on it.

A first look

Imagine cars that must park in equal groups at a few marked charging stations. In the problem the cars are n **robots**, points in the plane, and the stations are k **fixed points** that every robot can see; k divides n. Each robot reads positions in its own **local coordinate system** (its private axes, unit of length and handedness) and repeats a **Look–Compute–Move cycle**: it takes a **snapshot** of where the other robots and the fixed points are (Look), decides on a destination point (Compute) and moves towards it (Move). Robots are identical, silent and **oblivious** (no memory between cycles); in the basic version their local coordinate systems agree on the direction of one axis (say north), all robots see everything, and the **scheduler**, which decides when each robot acts and how long each phase takes, is ASYNC (each phase may take any finite time). The robots must end with exactly n/k robots, a **group**, on each fixed point, and stay there. The question is for which starting **configurations** (positions of all robots) and numbers k of fixed points this is possible.

1. Algorithm: The easy extreme: one fixed point for everybody. (Exactly n/k robots at each fixed point, p. 322) A standard approach for the simplest case, $k = 1$, where there is only one fixed point:

1. Look for the fixed point.
2. Move straight towards it.
3. Once on it, stay.

Why it works and what it costs. No coordination is needed, because the fixed point is a landmark all robots recognise: every move either reaches the point or covers at least a minimum distance, so each robot arrives after finitely many cycles. With more than one fixed point the robots must agree who goes

Claude (Anthropic) was used in cataloguing the problems.

where, and robots that look alike may be unable to choose differently; Bhagat, Das, Chakraborty and Mukhopadhyaya characterise the configurations and numbers k of fixed points for which the problem is solvable with one common axis (Bhagat et al. 2021), the catalogue's introductory and current best result (not described in detail here). The opposite extreme, $k = n$, asks for one robot per fixed point.

Why it matters. It shows the role of fixed points: they are landmarks that break much of the symmetry robots cannot break themselves.

2. Impossibility: Mirror twins arrive in pairs. (Exactly n/k robots at each fixed point, p. 322) A standard symmetry argument, for robots that cannot tell left from right. Let $n = 6$ and $k = 2$ (groups of 3), and put both fixed points on a vertical line L. Place three robots left of L and their three mirror images to the right, none on L; a robot and its mirror image are **mirror twins**. The **adversary**, an imagined opponent that chooses the local coordinate systems and when robots act, gives each pair of mirror twins mirror-image local coordinate systems and always activates them together. Twins see mirror-image snapshots, so they make mirror-image moves and stay mirror images forever. A robot can reach a point of L only together with its twin, so every point of L always holds an even number of robots. It can never hold exactly 3.

Why it holds. The same parity argument rules out every odd group size n/k whenever all fixed points lie on a line of symmetry of the start that holds no robot. This is why the catalogue's setting assumes a common axis, and why the answer depends on the start; the exact characterisation is in (Bhagat et al. 2021).

Why it matters. It is a compact example of how symmetry and counting combine to make a placement task impossible.

What the community started looking at

Partitioning into groups of a **given size** (n/k here; the papers write k) was first studied with the group locations fixed in advance (k-EPF), together with k-circle formation; the question was **for which configurations and group sizes it is solvable** when robots share only one axis (Bhagat et al. 2021).

Relations to other problems

- **Special case of Pattern formation.** A partition into k groups at points is a pattern of k points, each of multiplicity n/k.
- **Related: k-grouping.** Partitioning fixes the points, and so the number k of groups and their size n/k; k-grouping fixes only the number of groups. They coincide for equal-sized groups (Efrima and Peleg 2009).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point, + unlimited visibility, + transparent, + no faults, + one axis

Exactly n/k robots at each fixed point STANDARD SETTING ●● p. 322

Claude (Anthropic) was used in cataloguing the problems.

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
Exactly n/k robots at each fixed point STANDARD SETTING	● unknown *finite* vs —	● tight 0 vs —	322

18.10.1 Exactly n/k robots at each fixed point

Adds + point, + unlimited visibility, + transparent, + no faults, + one axis STANDARD SETTING VERIFIED

With one common axis, the configurations and numbers k of fixed points for which exactly n/k robots can gather at each of given fixed points are characterised.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; agreement on coordinates: agreement on one axis (no common chirality). **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Bhagat et al. 2021)	● unknown
Colours	none known No colours.	0 (Bhagat et al. 2021)	● tight

Results.

- Bhagat et al. (2021) [current best]: time *finite*, colours 0. Introductory and current best. Also k-circle formation: disjoint circles of n/k robots each, centred at the fixed points.

Open questions.

Q1. Partitioning into k groups of size n/k at points chosen by the robots (no fixed points given).

Notes. The k-EPF problem generalises embedded pattern formation: the fixed points are visible to all robots and each must end with exactly n/k robots (their k is n/k here) (Bhagat et al. 2021).

18.11 Convergence

> *Robots must get arbitrarily close to a common point; they need not ever meet.*

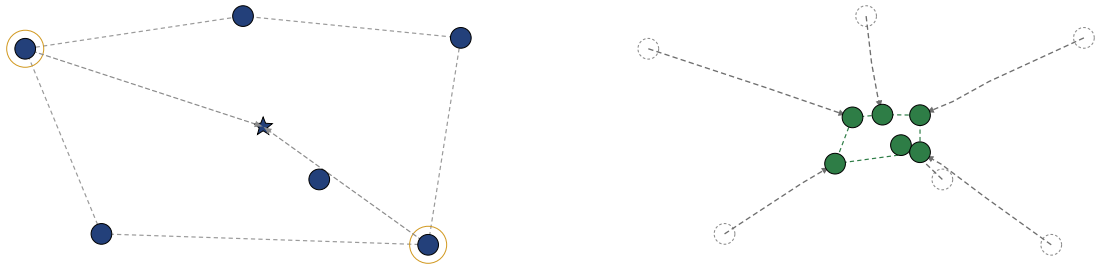
First posed in 1999 (Suzuki and Yamashita 1999). Suzuki and Yamashita gave algorithms for converging to a point and for reaching a point; Ando et al. treated limited visibility the same year (Ando et al. 1999).

Problem statement and measures

n robots start at arbitrary points. For every $\varepsilon > 0$ there must be a time after which all robots stay within distance ε of each other (equivalently, of a common limit point). Robots never need to coincide.

Claude (Anthropic) was used in cataloguing the problems.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).



(a) Start: ringed robots move towards the centre of gravity

(b) After 9 rounds: the convex hull keeps shrinking (arrows: moves; dashed circles: starting positions)

Figure 18.11: Convergence: robots at arbitrary points must, for every $\varepsilon > 0$, eventually stay within distance ε of each other; they approach a common limit point but need never meet (Suzuki and Yamashita 1999). Shown: the centre-of-gravity rule of (Cohen and Peleg 2005) in SSYNC rounds; ringed robots are activated and move towards the centre of gravity (star) of all robots, some stopping early.

A first look

Think of a flock that must squeeze ever closer together, without ever needing to land on one spot. There are n **robots**, points in the plane; their positions form the **configuration**. Each robot repeats a **Look–Compute–Move cycle**: it takes a **snapshot** of where the other robots are (Look), decides on a destination point (Compute) and moves towards it (Move). They are identical, silent, **oblivious** (no memory between cycles), and each has its own **local coordinate system**; the **scheduler**, an adversary, decides the **activation** of robots, that is, when each performs a cycle. In the basic version the scheduler is ASYNC (each phase may take any finite time, so a robot may act on an outdated snapshot) and every robot sees all others. The robots must converge: for every distance $\varepsilon > 0$, there must come a time after which all robots stay within ε of each other. Unlike gathering, they never have to meet exactly.

1. Algorithm: Go to the centre of gravity. (Cohen and Peleg 2005) (ASYNC, unlimited visibility, p. 326)

1. Look at the positions of all robots.
2. Compute their **centre of gravity**: the average of all positions (add up the x-coordinates and divide by n, and the same for y).
3. Move towards it.

Why it works and what it costs. The centre of gravity does not depend on the local coordinate system, so robots with different local coordinate systems aim at the same point. In FSYNC (every robot is activated in every **round**) with **rigid moves** (moves that reach their destination), all robots compute the same point and meet in one round. In ASYNC the snapshots are outdated, but each destination is an average of positions that were already inside the **convex hull** of the robots (the smallest convex region containing them), so the convex hull of the robots and their destinations never grows; Cohen and Peleg prove that it actually shrinks to a point (Cohen and Peleg 2005). A later analysis shows that $O(n)$ rounds halve the diameter, improving $O(n^2)$ (Cord-Landwehr et al. 2011); the catalogue has not checked which scheduler that analysis assumes.

Why it matters. It is the simplest convergence rule, and the never-growing convex hull is reused as a building block, for example to split robots into groups (Efrima and Peleg 2009).

Claude (Anthropic) was used in cataloguing the problems.

2. Algorithm: Short sight: go to the centre of what you see, but keep your neighbours. (Ando et al. 1999) (Limited visibility, SSYNC, rigid moves, p. 326) Now each robot sees only the robots within distance V (its **visibility radius**).

1. Compute the **smallest enclosing circle** of the robots you see, and head for its centre.
2. Do not go farther than a fixed maximum step.
3. For every robot you see, stay inside the disc of radius $V/2$ around the midpoint between you and it.

Why it works and what it costs. If two robots that see each other both obey rule 3, both end in the same disc of radius $V/2$, so they are still within V and still see each other (Lemma 2 of Ando et al. 1999). Groups therefore never split, the convex hull of each group never grows, and Ando, Oasa, Suzuki and Yamashita prove that each connected group converges to a point in SSYNC with rigid moves (Theorem 1). The argument uses that a whole cycle happens at once; in ASYNC a robot may move on outdated information and break a visibility link. The catalogue records limited-visibility convergence as solved for every bounded amount of asynchrony and open for full ASYNC (Kirkpatrick et al. 2024).

Why it matters. Its "stay within $V/2$ of the midpoint" trick for keeping neighbours is reused by many limited-visibility algorithms, including Go-To-the-Center gathering (Castenow et al. 2022).

What the community started looking at

Convergence was introduced alongside gathering as its relaxation (Suzuki and Yamashita 1999). The first questions were **which simple rules converge** under each scheduler (centre of gravity in ASYNC Cohen and Peleg 2005, limited visibility in SSYNC Ando et al. 1999); then the **rate** of convergence (Cord-Landwehr et al. 2011), and how much asynchrony is compatible with limited visibility (Kirkpatrick et al. 2024).

Relations to other problems

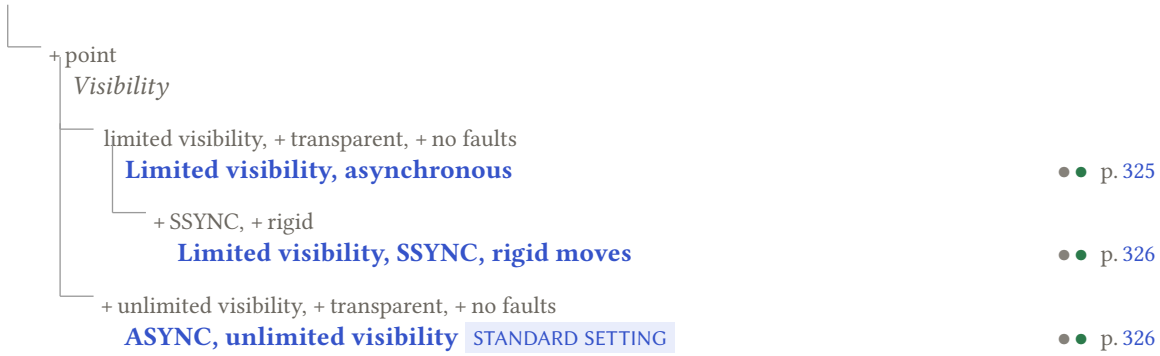
- **Related: Gathering.** Every gathering algorithm solves convergence, but not conversely: two oblivious robots in SSYNC cannot gather, while "two oblivious robots can converge to a point with a naive algorithm" (Izumi et al. 2012). In ASYNC the centre-of-gravity algorithm converges for any n (Cohen and Peleg 2005), whereas gathering needs extra assumptions such as multiplicity detection.
- **Related: Near-gathering.** Near-gathering is also a relaxation of gathering: robots must get close (within visibility of each other) without ever colliding (Pagli et al. 2015), while convergence lets them approach a point only in the limit.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

Claude (Anthropic) was used in cataloguing the problems.



What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
Limited visibility, asynchronous	● unknown *finite* vs —	● tight 0 vs —	325
Limited visibility, SSYNC, rigid moves	● unknown *finite* vs —	● tight 0 vs —	326
ASYNC, unlimited visibility STANDARD SETTING	● unknown *finite* vs —	● tight 0 vs —	326

18.11.1 Limited visibility, asynchronous

Adds limited visibility, + transparent, + no faults VERIFIED

With limited visibility, convergence is solved for every bounded asynchrony (k-ASYNC) but open for full ASYNC.

Assumed: body: point robots; opacity: transparent: every robot sees every robot within range; faults: no faults. **Not assumed:** visibility: limited: robots see only within a fixed radius; scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Full ASYNC is open.	*finite* (Kirkpatrick et al. 2024)	● unknown
Colours	none known No colours.	0 (Kirkpatrick et al. 2024)	● tight

Results.

- Katreniak (2011) [introductory]: time *finite*, colours 0. *Assumes:* 1-ASYNC. Also solves unlimited-visibility ASYNC convergence without multiplicity detection.
- Kirkpatrick et al. (2024) [current best]: time *finite*, colours 0. *Assumes:* k-ASYNC for any fixed bound on the asynchrony. Also shows that cohesive convergence is impossible in ASYNC for hull-diminishing or coherent algorithms when perception has errors.

Open questions.

Claude (Anthropic) was used in cataloguing the problems.

Q1. Is point convergence with limited visibility solvable in full ASYNC, without perception or motion errors? (Kirkpatrick et al. 2024)

Q2. Is there a problem that separates k-ASYNC from SSYNC? (Kirkpatrick et al. 2024)

18.11.2 Limited visibility, SSYNC, rigid moves

Adds + SSYNC, + rigid **to** Limited visibility, asynchronous (p. 325) VERIFIED

With limited visibility, semi-synchronous oblivious robots whose moves reach their computed destination converge within each connected component of the visibility graph.

Assumed: body: point robots; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); movement: rigid: every move reaches its destination. **Not assumed:** visibility: limited: robots see only within a fixed radius; lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Time not analysed here.	*finite* (Ando et al. 1999)	● unknown
Colours	none known No colours.	0 (Ando et al. 1999)	● tight

Results.

- Ando et al. (1999) [current best]: time *finite*, colours 0. *Assumes:* each connected component of the initial visibility graph converges to its own point; each move reaches its computed destination, at most a small constant away; V known (the algorithm keeps visible robots within distance V). Introductory and current best (Theorem 1).

Notes. Checked against the full text of (Ando et al. 1999). The formal model (assumptions A1–A6) has point robots that do not obstruct each other, phases of "negligibly small" duration, so that a robot observes others only when they are stationary (an SSYNC scheduler, although the paper calls the robots asynchronous), every robot active infinitely often, and exact sensing and motion. There is no common coordinate system. A robot moves towards the centre of the smallest enclosing circle of the robots it sees, but no farther than keeps every visible robot within distance V (Lemma 2: mutually visible robots stay mutually visible). Whether the same algorithm works with non-rigid moves is not addressed in the paper.

18.11.3 ASYNC, unlimited visibility

Adds + unlimited visibility, + transparent, + no faults STANDARD SETTING VERIFIED

Moving to the centre of gravity makes oblivious robots converge in ASYNC, for any n.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known See the notes for convergence-time results.	*finite* (Cohen and Peleg 2005)	● unknown
Colours	none known No colours.	0 (Cohen and Peleg 2005)	● tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Cohen and Peleg (2005) [current best]: time *finite*, colours 0. Introductory and current best for ASYNC; also tolerates crash faults (the exact bound on f was not checked).

Notes. Convergence time of the centre-of-gravity algorithm: (Cord-Landwehr et al. 2011) show that $O(n)$ rounds halve the diameter of the convex hull, improving $O(n^2)$, against a known $\Omega(n)$. **Unverified:** the scheduler of that analysis.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 19

Symmetry breaking and spreading

Tasks whose difficulty is breaking the symmetry of anonymous robots: electing a leader, scattering from shared points, and separating by colour.

Problems in this family:

- Scattering
- Leader election in the plane
- Separation
- Flocking
- Uniform deployment on a ring

19.1 Scattering

Robots that may start at shared points must move to pairwise distinct points.

First posed in 2009 (Dieudonné and Petit 2009). Dieudonné and Petit showed scattering is deterministically impossible and gave a randomized solution (preprint 2007).

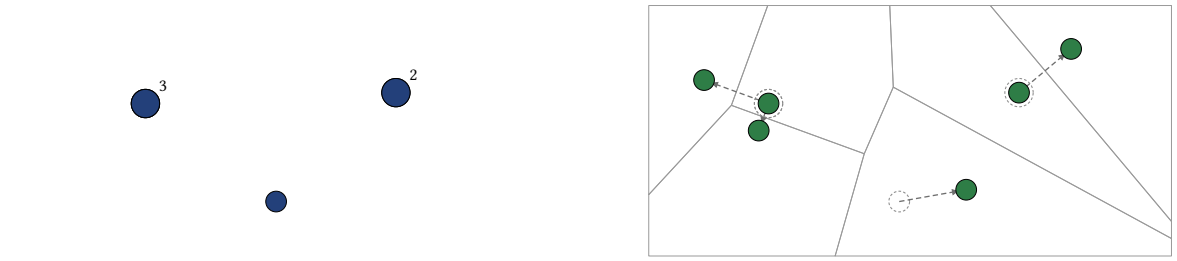
Problem statement and measures

n robots start at points of the plane, several possibly at the same point (a **multiplicity**). They must reach a configuration in which all positions are distinct, and stay there.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).

A first look

Imagine a box of marbles that were all tipped onto the same spot of a table, and you want them to roll apart so that no two touch the same point. In the scattering problem the marbles are n **robots**: they start in the plane, several possibly at the same point (a **multiplicity**), and must reach a **configuration** (the positions of all robots) in which all positions are distinct, and then stay distinct. Robots are **oblivious** (they forget everything after each move) and work in **Look–Compute–Move cycles**: take a **snapshot** of everyone, compute a destination, move. In each **round** an **adversary** (an opponent who controls everything the algorithm does not) chooses which robots act, and each of them performs one full cycle; this is the SSYNC **scheduler**. In the basic catalogued version robots may flip coins (**randomized**), and a robot can see how many robots share each point (**strong multiplicity detection**).



(a) Start: points of multiplicity 3, 2 and 1

(b) Final: all positions distinct (arrows: moves; dashed circles: starting positions)

Figure 19.1: Scattering: robots start at points of the plane, several possibly at the same point (multiplicities shown as numbers), and must reach distinct positions and stay there; deterministically this is impossible (Dieudonné and Petit 2009). The animation runs the randomized SSYNC algorithm of (Dieudonné and Petit 2009): in each round the adversary activates some robots (coin shown under their point), and a robot that gets heads moves to a point of its own Voronoi cell (thin lines), so robots on different points never meet again.

1. Impossibility: Two robots on one spot can never be told apart. (Dieudonné and Petit 2009) (Randomized, SSYNC, strong multiplicity detection, p. 331) Start with all robots on the same point, give them all the same **local coordinate system** (the axes, unit and handedness in which a robot reads its snapshot), and let the adversary activate all of them in every round. Each robot sees exactly the same snapshot, so a **deterministic** rule (one without coin flips) sends every robot to the same destination. After the move they are again all on one point, with identical snapshots, and so on forever.

Why it holds. By induction, there is an execution in which the robots never separate, so no deterministic algorithm solves scattering (Dieudonné and Petit 2009). The argument does not care whether robots remember the past or can count robots at a point: the robots' memories are equal as well, so their histories never differ. Only randomness can make two robots on the same point behave differently.

Why it matters. It is the cleanest example of why some robot problems need randomness at all, and it explains why scattering is studied only with coin flips.

2. Algorithm: Flip a coin, then move inside your own Voronoi cell. (Dieudonné and Petit 2009) (Randomized, SSYNC, strong multiplicity detection, p. 331) Each time a robot is activated:

1. Look at the occupied points and compute the *Voronoi diagram*: the **Voronoi cell** of an occupied point is the set of places strictly closer to it than to any other occupied point.
2. Flip a fair coin.
3. Heads: move to some point of your own cell other than where you stand. Tails: stay.

Why it works and what it costs. Robots on different points never collide again, because each moves only inside its own cell and the cells do not overlap. Two robots on the same point are separated in a round where exactly one of them moves; if both are active this happens with probability $1/2$, so with probability 1 they eventually separate (Dieudonné and Petit 2009). If all movers from a point pick the same destination, a crowd of q robots is at best split in two per round, so about $\log_2 q$ rounds are needed; robots that can count the robots on their point scatter in $O(1)$ expected rounds (Clement et al. 2010), and with weaker counting $O(\log \log n)$ rounds, which is optimal for algorithms that use the fewest random bits (Bramas and Tixeuil 2017).

Why it matters. Moving only inside your own Voronoi cell is a reusable trick for keeping already separated robots apart, and randomized scattering is used to break symmetry before running deterministic algorithms such as

Claude (Anthropic) was used in cataloguing the problems.

pattern formation.

What the community started looking at

Scattering was first a **feasibility** question: it cannot be solved deterministically, so randomness is needed (Dieudonné and Petit 2009). The measures then became **expected rounds and random bits** (Clement et al. 2010) (Bramas and Tixeuil 2017), and keeping the swarm connected under limited visibility (Izumi et al. 2018).

Relations to other problems

- **Related: Gathering.** Scattering is the opposite task: gathering creates a multiplicity point, scattering removes all of them. Pattern formation from configurations with multiplicities needs scattering first (Bramas and Tixeuil 2016).
- **Related: Uniform deployment on a ring.** Both spread robots out; scattering only needs distinct positions, deployment needs equal spacing.
- **Related: Scattering of particles.** Robots in the plane scatter to distinct points; particles must scatter so that no two are neighbours.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point

Visibility

+ unlimited visibility, + transparent, + no faults

Scheduler

+ SSYNC

Multiplicity detection

+ strong multiplicity, + randomized

Randomized, SSYNC, strong multiplicity detection STANDARD SETTING ●● p. 331

+ weak multiplicity, + randomized

Randomized, SSYNC, weak multiplicity detection ●● p. 331

ASYNC, + randomized

Randomized, ASYNC ●● p. 332

limited visibility, + transparent, + no faults, + SSYNC, + randomized

Limited visibility, connectivity preserving ●● p. 332

Claude (Anthropic) was used in cataloguing the problems.

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
Randomized, SSYNC, strong multiplicity detection STANDARD SETTING	● tight $O(1)$ vs *impossible*	● tight 0 vs —	331
Randomized, SSYNC, weak multiplicity detection	● tight $O(\log \log n)$ vs not $O(1)$	● tight 0 vs —	331
Randomized, ASYNC	● unknown — vs —	● unknown — vs —	332
Limited visibility, connectivity preserving	● tight $O(\min\{n, g^2 + \log n\})$ vs $\Omega(n)$	● tight 0 vs —	332

19.1.1 Randomized, SSYNC, strong multiplicity detection

Adds + strong multiplicity, + randomized STANDARD SETTING VERIFIED

Deterministic scattering is impossible; with randomness and strong multiplicity detection robots scatter in $O(1)$ expected rounds.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); multiplicity detection: strong: a robot sees how many robots are at each point; randomness: randomized. **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Dieudonné and Petit 2009) $O(1)$ expected rounds.	$O(1)$ (Clement et al. 2010)	● tight
Colours	none known No colours.	0 (Clement et al. 2010)	● tight

Results.

- Dieudonné and Petit (2009) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* deterministic algorithms; robots at the same point are indistinguishable.
- Dieudonné and Petit (2009) [introductory]: time *finite*, colours 0. Randomized and self-stabilizing.
- Clement et al. (2010) [current best]: time $O(1)$, colours 0. Expected rounds; $O(\log n \log \log n)$-type bounds with weak multiplicity detection.

19.1.2 Randomized, SSYNC, weak multiplicity detection

Adds + weak multiplicity, + randomized VERIFIED

Without strong multiplicity detection no algorithm scatters in $O(1)$ expected rounds; with an optimal number of random bits $O(\log \log n)$ rounds suffice.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); multiplicity detection: weak: a robot sees whether a point holds more than one robot; randomness: randomized. **Not assumed:** lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; identifiers: anonymous: all robots are identical.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	not $O(1)$ (Bramas and Tixeuil 2017); $\Omega(\log\log n)$ (Bramas and Tixeuil 2017) $\Theta(\log\log n)$ among bit-optimal algorithms; in general only "not $O(1)$" is known.	$O(\log\log n)$ (Bramas and Tixeuil 2017)	• tight
Colours	none known No colours.	0 (Bramas and Tixeuil 2017)	• tight

Results.

- Bramas and Tixeuil (2017) [lower bound]: time not $O(1)$. *Bottleneck:* symmetry breaking. *Holds when:* Theorem 4.1: no algorithm runs in $O(1)$ expected rounds without strong multiplicity detection.
- Bramas and Tixeuil (2017) [lower bound]: time $\Omega(\log\log n)$. *Bottleneck:* symmetry breaking. *Holds when:* Theorem 4.5: only for algorithms that use the optimal number of random bits.
- Bramas and Tixeuil (2017) [current best]: time $O(\log\log n)$, colours 0. Corollary 4.1, with an optimal number of random bits. The analysis is in FSYNC; the algorithms are valid in SSYNC. Without any multiplicity detection, algorithm SA_ϕ (their SA_f) scatters in $O(\phi(n))$ expected rounds for any increasing surjective function ϕ (Theorem 4.2), but the robots never stop moving.

Notes. $n \log n$ random bits are necessary in any setting (Bramas and Tixeuil 2017). Bramas and Tixeuil split multiplicity detection into local and global. Checked against arXiv 1309.6603 version 2, not the journal version.

19.1.3 Randomized, ASYNC

Adds ASYNC, + randomized DRAFT

Scattering in ASYNC is open in peer-reviewed work.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; randomness: randomized. **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known Open.	—	• unknown
Colours	none known Open.	—	• unknown

Open questions.

Q1. Solve scattering in ASYNC; Bramas and Tixeuil call it "the open problem of ASYNC scattering" (Bramas and Tixeuil 2016) (Bramas and Tixeuil 2017). A 2019 preprint by Léchine and Tixeuil (arXiv 1905.09177, not peer reviewed) solves it with vision and weak local multiplicity detection (its Theorem 4), and leaves open the plane with vision and no multiplicity detection.

19.1.4 Limited visibility, connectivity preserving

Adds limited visibility, + transparent, + no faults, + SSYNC, + randomized VERIFIED

Robots with limited visibility scatter without ever disconnecting their visibility graph, in $O(\min\{n, g^2+\log n\})$ expected rounds, g the diameter of the initial visibility graph.

Claude (Anthropic) was used in cataloguing the problems.

Assumed: body: point robots; opacity: transparent: every robot sees every robot within range; faults: no faults; scheduler: semi-synchronous (SSYNC); randomness: randomized. **Not assumed:** visibility: limited: robots see only within a fixed radius; lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(n)$ (Izumi et al. 2018); $\Omega(g^2)$ (Izumi et al. 2018) Optimal within conservative algorithms.	$O(\min\{n, g^2 + \log n\})$ (Izumi et al. 2018)	• tight
Colours	none known No colours.	0 (Izumi et al. 2018)	• tight

Results.

- Izumi et al. (2018) [lower bound]: time $\Omega(n)$. *Bottleneck:* distance. *Holds when:* any algorithm, some configurations.
- Izumi et al. (2018) [lower bound]: time $\Omega(g^2)$. *Bottleneck:* locality. *Holds when:* only for conservative algorithms.
- Izumi et al. (2018) [current best]: time $O(\min\{n, g^2 + \log n\})$, colours 0. Introduced at SSS 2010.

Notes. g is the diameter (in hops) of the initial visibility graph.

19.2 Leader election in the plane

Anonymous robots must agree on one of them, using only their positions.

First posed in 2007 (Dieudonné et al. 2013). Posed for disoriented robots at OPODIS 2007 (Dieudonné and Petit); characterised in (Dieudonné et al. 2013).

Problem statement and measures

n **anonymous** robots (identical, with no names) at distinct points must agree on a single robot (the **leader**), each deciding from its own snapshot.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).

A first look

Imagine n identical drones hovering at different spots in a field. They carry no name tags, they cannot talk, and each one has its own private idea of which way is north, what counts as one metre, and possibly even which way is clockwise. In the problem the drones are **robots**, points at distinct places in the plane, and a robot's private north, unit of length and sense of clockwise form its **local coordinate system**. Each robot takes a **snapshot**, a photograph of where all the robots are in its local coordinate system, and, using the same program as every other robot, must decide "I am the leader" or "I am not", so that exactly one robot says yes; that robot is the **leader**. The robots are **anonymous** (they look the same and run the same program) and *disoriented* (no shared compass, unit of length or, without **chirality**, shared sense of clockwise). The question is: for which **configurations** (arrangements of the robots' positions) can this be done at all?

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: anonymous robots at distinct points

(b) Final: the robot on it nearest O is the leader

Figure 19.2: Leader election: anonymous robots at distinct points must agree on one robot, each deciding from its own snapshot, without moving. With chirality: draw the smallest enclosing circle, a ray from its centre O through every robot, read the configuration clockwise from each ray as a word (distances along the ray, angle to the next ray); the ray with the unique smallest word carries the leader, its robot nearest O. A leader can be elected exactly when this word is unique, that is, when the configuration has no rotational symmetry (Dieudonné et al. 2013).

1. Algorithm: Read the configuration as a word around the smallest enclosing circle. (Dieudonné et al. 2013) (Disoriented robots, with or without chirality, p. 336) This is the version where the robots agree on clockwise (chirality).

1. Draw the smallest circle that encloses all robots, the **smallest enclosing circle**; every robot finds the same circle. Call its centre O. If a robot sits exactly at O, it is the leader; stop.
2. Draw a **ray** (a half-line) from O through every robot (several robots may share a ray).
3. For each ray, list the distances between consecutive robots on it, starting from O, measured as fractions of the circle's radius (so the private unit of length does not matter).
4. Starting at a ray and walking clockwise around the circle, write down a **word**: the list for this ray, then the angle to the next ray, then the list for the next ray, and so on, once around.
5. Every robot computes the word of every ray and picks the ray whose word is alphabetically smallest. If only one ray has that word, the leader is the robot on that ray nearest to O.

Why it works and what it costs. All robots compute the same words, because everything is measured relative to the circle and the shared clockwise direction, so they all pick the same robot. The words of different rays are rotations of one another; two rays can have the same word only if turning the whole picture around O moves one ray onto the other, that is, if the configuration has **rotational symmetry**. (Dieudonné et al. 2013) prove that this is the exact borderline: with chirality, a leader can be elected exactly when the smallest word is unique (they phrase it with Lyndon words), and they give a similar but more delicate rule, reading in both directions, when robots do not share clockwise.

Why it matters. Turning a picture into a word and comparing words is the standard way robot papers break symmetry, and electing a leader this way is the first step of arbitrary pattern formation (Dieudonné et al. 2010).

2. Impossibility: Symmetric pictures look the same to everybody. (Dieudonné et al. 2013) (Disoriented robots, with or without chirality, p. 336) Place six robots at the corners of a regular hexagon. The **adversary** (an imagined opponent who chooses everything the algorithm does not control, here the robots' private coordinate systems) gives each robot a local coordinate system whose north points from the centre through that robot, and the same unit and handedness for all. Now every robot's snapshot is literally the same list of coordinates. The same happens for any $n = 2$: two points are always swapped by a half-turn around their midpoint.

Claude (Anthropic) was used in cataloguing the problems.

Why it holds. A **deterministic** program (no coin flips) that receives the same input gives the same output. So either every robot declares itself leader or none does; exactly one is impossible. The same argument works for any configuration that some rotation maps onto itself while moving robots, which is why the characterisation of (Dieudonné et al. 2013) is phrased in terms of symmetry.

Why it matters. Almost every impossibility for robots in the plane, from pattern formation to gathering, is this argument: build a symmetric start and choose coordinate systems so that symmetric robots see identical snapshots.

What the community started looking at

Leader election in the plane was asked as **a characterisation of the configurations from which anonymous robots can agree on one of them** (Dieudonné et al. 2013); its importance comes from its equivalence with arbitrary pattern formation (Dieudonné et al. 2010) (Cicerone et al. 2019).

Relations to other problems

- **Related: Pattern formation.** Arbitrary pattern formation is solvable from a configuration exactly when leader election is, in ASYNC without chirality (Cicerone et al. 2019); with chirality and $n \geq 4$ this was shown by (Dieudonné et al. 2010).
- **Related: Leader election in programmable matter.** Both elect a leader among anonymous entities from geometry alone; particles on the lattice can also move and talk to neighbours, robots in the plane only see (Dufoulon et al. 2021).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point, + unlimited visibility, + transparent, + no faults
Disoriented robots, with or without chirality STANDARD SETTING ● ● p. 336

fat, + unlimited visibility, + transparent
Transparent fat robots, no chirality ● ● p. 336

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
Disoriented robots, with or without chirality STANDARD SETTING	● unknown *finite* vs *impossible*	● tight 0 vs —	336
Transparent fat robots, no chirality	● unknown *finite* vs *impossible*	● tight 0 vs —	336

Claude (Anthropic) was used in cataloguing the problems.

19.2.1 Disoriented robots, with or without chirality

Adds + point, + unlimited visibility, + transparent, + no faults STANDARD SETTING VERIFIED

Whether anonymous robots can agree on a leader from their positions is characterised by Lyndon words around the smallest enclosing circle.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Dieudonné et al. 2013) Leader election here is a static decision.	*finite* (Dieudonné et al. 2013)	● unknown
Colours	none known No colours.	0 (Dieudonné et al. 2013)	● tight

Results.

- Dieudonné et al. (2013) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* $n = 2$; symmetric configurations.
- Dieudonné et al. (2013) [current best]: time *finite*, colours 0. Complete characterisation for every $n > 1$, with and without chirality. Introduced at OPODIS 2007 by Dieudonné and Petit.

19.2.2 Transparent fat robots, no chirality

Adds fat, + unlimited visibility, + transparent VERIFIED

Transparent fat robots without chirality can agree on an ordering, and so on a leader, exactly when their configuration is asymmetric.

Assumed: visibility: unlimited; opacity: transparent: every robot sees every robot within range. **Not assumed:** body: fat robots (unit discs); faults: up to f Byzantine robots; scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Chaudhuri and Mukhopadhyaya 2015) Solvable exactly on asymmetric configurations.	*finite* (Chaudhuri and Mukhopadhyaya 2015)	● unknown
Colours	none known No colours.	0 (Chaudhuri and Mukhopadhyaya 2015)	● tight

Results.

- Chaudhuri and Mukhopadhyaya (2015) [lower bound]: time *impossible*. *Bottleneck:* symmetry breaking. *Holds when:* symmetric configurations (Theorem 6); deterministic.
- Chaudhuri and Mukhopadhyaya (2015) [current best]: time *finite*, colours 0. Introductory and current best (Theorems 1–6, algorithm MakeAsym).

Notes. Checked against the full text (arXiv 1208.4484).

Claude (Anthropic) was used in cataloguing the problems.

19.3 Separation

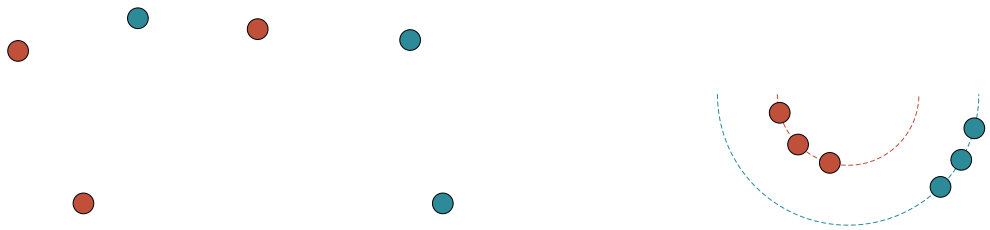
Robots of different (hidden) colours must separate so that each colour class occupies its own region.

First posed in 2023 (Seike and Yamauchi 2023). Introduced at SSS 2023.

Problem statement and measures

Each robot has a fixed **colour** that the others can see but it cannot (an **unconscious** robot). The robots must arrange themselves so that the **colour classes** (the sets of robots of one colour) are separated (into points, lines or circles, depending on the variant).

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).



(a) Start: robots see the others' colours, not their own (b) Final: one semicircle per colour

Figure 19.3: Separation: each robot carries a fixed colour that the others see but it cannot; the robots must arrange themselves so that the colour classes are separated, into parallel lines in the standard ASYNC setting (Seike and Yamauchi 2023). The animation sketches the SSYNC algorithm for opaque robots (Pyati et al. 2026) (read in summary only): the robots move to a semicircle, one sector per colour; a signaller points at a misplaced robot, which then moves to its sector; finally each colour class moves onto its own concentric semicircle.

A first look

Imagine a party where each guest wears a coloured hat they cannot see, but they can see everybody else's hat, and nobody may speak. The guests must sort themselves so that all red hats stand on one line, all blue hats on another, and so on. In the separation problem the guests are n point **robots** in the plane, and a hat is a robot's **colour**: fixed, seen by the others but not by the robot itself (an **unconscious** robot). Each robot repeats a **Look–Compute–Move cycle**: it looks at the others, computes a destination and moves there. In the basic version robots are **oblivious** (they forget everything after each move), see everybody, and their **local coordinate systems** (each robot's private axes and unit of length) agree on the direction of one axis; the **scheduler**, an adversary that decides when each robot acts, is asynchronous (ASYNC): a robot may look while others are still moving. They must end up separated into parallel lines, one **colour class** (all robots of one colour) per line.

1. Algorithm: Others tell you where to go. (Pyati et al. 2026) (Opaque robots, SSYNC, p. 340) The key difficulty is that a robot does not know its own colour, but every other robot does. One way out, used with **opaque** robots (robots that block the view of robots behind them) under SSYNC (time is divided into **rounds**, and in each round some robots each perform one full cycle):

Claude (Anthropic) was used in cataloguing the problems.

1. Bring all robots to the lower half of a circle, where everybody can see everybody.
2. Divide this semicircle into **sectors**, one per colour.
3. A few chosen robots act as **signallers**: by moving to a carefully chosen point they encode "the robot of colour c at this position should go to sector j".
4. The robot addressed reads the signal and moves to its sector, without ever learning its colour directly.
5. Finally each sector is moved onto its own semicircle, so colour classes lie on concentric semicircles.

Why it works and what it costs. Positions are the only way robots can pass information, so the algorithm turns positions into messages. (Pyati et al. 2026) reach concentric semicircles in $O(n)$ epochs (an **epoch** is an interval of time in which every robot completes at least one full cycle) without collisions and without knowing n. We read only a summary of this paper's arXiv version, so the steps above give the idea, not its details. The algorithms for the standard ASYNC variant are different and are not described here.

Why it matters. Using movement as a message channel is how silent robots share information they cannot sense themselves, the same idea behind signalling with lights.

2. Impossibility: A colour with only one robot. (Flocchini et al. 2024) (ASYNC, one common axis, p. 339) Asynchronous separation into parallel lines is solvable when robots share one axis, provided no colour class has a single robot. If some robot is the only one of its colour, (Flocchini et al. 2024) show that no algorithm works. The proof is a symmetry argument in the paper and is not reproduced here.

Why it holds. The catalogue records the condition but not its proof, and we have not read the full text. A student should take away the shape of the result: solvability depends on how many robots share each colour, because a robot can learn about its own colour only from what the other robots do.

Why it matters. It marks the exact boundary between possible and impossible for the asynchronous algorithm of (Flocchini et al. 2024).

What the community started looking at

Separation of unconscious coloured robots asked whether robots that **cannot see their own colour** can still sort themselves by colour, first synchronously (Seike and Yamauchi 2023), then asynchronously (Flocchini et al. 2024) and with opaque robots (Pyati et al. 2026).

Relations to other problems

- **Related: k-grouping.** Both split the swarm into classes; in separation the classes are fixed by hidden colours, in k-grouping and partitioning they are chosen by the robots.
- **Related: Separation of coloured particles.** Robots in the plane separate by hidden colours with deterministic algorithms; here a stochastic local rule separates two visible colours.

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point, + unlimited visibility

Opacity

+ transparent, + no faults, + one axis

ASYNC, one common axis STANDARD SETTING ● ● p. 339

opaque, + no faults, + SSYNC, + one axis

Opaque robots, SSYNC ● ● p. 340

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
ASYNC, one common axis STANDARD SETTING	● unknown *finite* vs *impossible*	● tight 0 vs —	339
Opaque robots, SSYNC	● unknown $O(n)$ vs —	● tight 0 vs —	340

19.3.1 ASYNC, one common axis

Adds + transparent, + no faults, + one axis STANDARD SETTING VERIFIED

Robots that carry a colour visible to others but not to themselves separate by colour into parallel lines, under ASYNC, if they share one axis.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; agreement on coordinates: agreement on one axis (no common chirality). **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Flocchini et al. 2024) —	*finite* (Flocchini et al. 2024)	● unknown
Colours	none known The colours are given, not chosen.	0 (Flocchini et al. 2024)	● tight

Results.

- Flocchini et al. (2024) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* if some robot is uniquely coloured.
- Seike and Yamauchi (2023) [introductory]: time *finite*, colours 0. *Assumes:* synchronous setting.
- Flocchini et al. (2024) [current best]: time *finite*, colours 0. Separation into parallel lines.

Claude (Anthropic) was used in cataloguing the problems.

19.3.2 Opaque robots, SSYNC

Adds opaque, + no faults, + SSYNC, + one axis VERIFIED

Opaque unconscious robots separate into concentric semicircles in $O(n)$ epochs.

Assumed: body: point robots; visibility: unlimited; faults: no faults; scheduler: semi-synchronous (SSYNC); agreement on coordinates: agreement on one axis (no common chirality). **Not assumed:** opacity: opaque: collinear robots hide each other; lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound recorded.	$O(n)$ (Pyati et al. 2026)	● unknown
Colours	none known —	0 (Pyati et al. 2026)	● tight

Results.

- Pyati et al. (2026) [current best]: time $O(n)$, colours 0. Collision-free; robots do not know n.

19.4 Flocking

Robots must move together as a group, following a leader while roughly keeping a formation.

First posed in 2004 (Gervasi and Prencipe 2004). Coordination without communication: the flocking problem.

Problem statement and measures

Robots, the **followers**, follow a designated robot, the **leader**, whose path is not known in advance, while keeping (approximately) a given **formation** (a shape the robots must hold, placed relative to the leader) and avoiding **collisions** (two robots at the same position).

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).

A first look

Think of a flock of birds following a leader whose route nobody knows in advance, keeping roughly the same shape and never bumping into each other. In the flocking problem the birds are **robots** in the plane: the **followers** must follow a designated **leader** robot along an unpredictable path while keeping approximately a given **formation**, a shape placed relative to the leader (for example, a V shape), and avoiding **collisions** (two robots at the same position). The robots are **oblivious** (they forget everything after each move) and silent: they coordinate only by looking at each other's positions. The catalogue records only the first formal treatment (Gervasi and Prencipe 2004) and has not checked its exact assumptions, so this page describes the problem and the simplest reasoning about it.

1. Algorithm: Keep your slot relative to the leader. (Oblivious robots following a leader, p. 342) A standard approach, assuming the followers can recognise the leader and know the target formation:

1. Look at all robots and find the leader.

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: followers scattered behind the leader L

(b) Final: L stops; the followers hold the V

Figure 19.4: Flocking: the robots follow a leader (L) whose path is not known in advance, keeping approximately a given formation and avoiding collisions (Gervasi and Prencipe 2004). The animation shows the basic slot-chasing idea (not the method of Gervasi and Prencipe 2004): in each round the leader moves; each follower places the V formation behind the leader's last observed heading, takes the nearest free slot (dashed circles) and moves towards it at a speed higher than the leader's.

2. Place the target formation with the leader at its tip, oriented, say, along the leader's last observed direction of motion.
3. Choose the free **slot** of the formation (a position in it reserved for one follower) that is closest to you (with a fixed tie-break rule).
4. Move towards that slot, but never closer to another robot than a safety distance.

Why it works and what it costs. Each follower chases a target that moves with the leader, so if followers are fast enough they stay within a bounded distance of their slots. The details that make this hard are exactly those the model hides: an oblivious follower cannot remember the leader's past direction, and slot assignment must not change from look to look. We do not attribute this sketch to (Gervasi and Prencipe 2004), whose method we have not read.

Why it matters. Reducing a moving formation to "chase your slot" is the basic design idea of leader-follower formation control.

2. Lower bound: Followers must be faster than the leader. (Oblivious robots following a leader, p. 342) Suppose followers can move no faster than the leader. Let the leader run at full speed in a straight line. A follower that is a distance $\eta > 0$ behind its slot moves at most as fast as the slot moves, so the gap never shrinks.

Why it holds. So exact formation keeping is impossible in general with equal top speeds; a follower can only stay within the error it started with. This simple observation, our own and not taken from a paper, is why flocking asks for approximate formations or gives followers extra speed.

Why it matters. It explains why "approximately" in the problem statement is not a detail but a necessity.

What the community started looking at

Flocking was posed as a **coordination problem without communication** for oblivious robots (Gervasi and Prencipe 2004); the questions were feasibility and, later, fault tolerance and self-stabilisation.

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point, + unlimited visibility, + transparent, + no faults

Oblivious robots following a leader STANDARD SETTING ●● p. 342

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
Oblivious robots following a leader STANDARD SETTING	● unknown *finite* vs —	● tight 0 vs —	342

19.4.1 Oblivious robots following a leader

Adds + point, + unlimited visibility, + transparent, + no faults STANDARD SETTING DRAFT

The first formal treatment of flocking by oblivious robots that communicate only by observing each other.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults. **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); agreement on coordinates: no agreement on local coordinate systems; movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Gervasi and Prencipe 2004)	● unknown
Colours	none known No colours.	0 (Gervasi and Prencipe 2004)	● tight

Results.

- Gervasi and Prencipe (2004) [current best]: time *finite*, colours 0. Introductory; the exact assumptions and guarantees were not read in the paper.

Notes. Self-stabilising and crash-tolerant flocking followed (SSS 2007, OPODIS 2008; not yet in the bibliography).

Claude (Anthropic) was used in cataloguing the problems.

19.5 Uniform deployment on a ring

Robots on a circle must spread out to equal spacing.

First posed in 2006 (Flocchini et al. 2008b). Self-deployment of mobile sensors on a ring (ALGOSENSORS 2006).

Problem statement and measures

n robots on a **ring** (a circle of length L) must reach positions at equal distance $d = L/n$ from their **neighbours** (the next robots along the ring in either direction) and stop. Spreading on a segment is the one-dimensional analogue.

Measures: **Time**, rounds (FSYNC, SSYNC) or epochs (ASYNC) until the task is done; "finite" when only solvability is known; **Colours**, colours of the light per robot (0 for oblivious robots).



(a) Start: oriented ring, $n = 6$, robots know $d = L/6$ (b) Final: every gap equals d; nobody moves

Figure 19.5: Uniform deployment on a ring: robots on a circle of length L must reach positions at distance exactly $d = L/n$ from their neighbours and stop (Flocchini et al. 2008b). The animation runs their rule for an oriented ring with known d: an activated robot whose clockwise gap (red) exceeds d moves clockwise along the ring until it is exactly d behind the next robot; here the asynchronous adversary activates one robot at a time.

A first look

Imagine n guards spread at random along a circular wall who must spread out so that every two neighbours are the same distance apart. In **uniform deployment** the guards are n **robots** at distinct points of a **ring**, a circle of length L. Consecutive robots along the ring are **neighbours**, and the **gap** of a robot is the distance along the ring to its clockwise neighbour. The robots must reach positions where every gap is exactly $d = L/n$, and then stop. In the basic version robots are **oblivious** (they forget everything after each move), the **scheduler** is asynchronous (ASYNC: an **adversary**, an imagined opponent, decides the **activation** of each robot, that is, when it looks and moves, and a robot may see others in mid-move), and robots agree on which way is clockwise (an **oriented ring**).

1. Algorithm: Close the gap in front of you, only clockwise. (Flocchini et al. 2008b) (Oriented ring, asynchronous, oblivious, p. 345) Robots know d. Each time a robot is activated:

1. Look clockwise and measure the distance to the next robot.
2. If that distance is at most d, do nothing.
3. If it is more than d, move clockwise until you are exactly d behind that robot.

Claude (Anthropic) was used in cataloguing the problems.

Why it works and what it costs. All gaps add up to $L = nd$, so if some gap is smaller than d, another is larger. Robots only move clockwise, so the robot in front never comes closer; a robot whose gap is at least d therefore never again has a gap below d, and the number of such robots never decreases. (Flocchini et al. 2008b) prove that this protocol (Uniform Known) reaches exact deployment in finite time, even with limited visibility $2d$; when d is unknown, a "go to the half" rule reaches every gap within any ε of d. No running-time bound is recorded in the catalogue.

Why it matters. It shows that a shared sense of direction plus one number (d) turns an impossible task into a three-line rule.

2. Impossibility: Two mirror-image teams that always overshoot. (Flocchini et al. 2008b) (Oriented ring, asynchronous, oblivious, p. 345) Without a shared clockwise direction exact deployment is impossible, even with full memory and SSYNC activation (time is divided into **rounds**; in each round the adversary activates some robots, and each looks and moves once). Take $n = 4$: black robots at two opposite points, white robots at two opposite points, the white pair rotated by a small angle from the uniform position; the two black robots form one **team** and the two white robots the other. Black robots think clockwise is one way, white robots think it is the other, so each black robot sees exactly what each white robot sees, in mirror image. The adversary then plays:

1. If moving only the blacks would not finish the job, move only the blacks (the **configuration**, the positions of all robots, keeps the same two-team shape).
2. If moving only the blacks would finish, then by the mirror image moving only the whites would also finish; so move both. Each team closes the same gap, together they overshoot, and again the configuration has the same shape.
3. Alternate between starting with the blacks and the whites, so every robot moves infinitely often.

Why it holds. Every round leaves the robots in two rotated teams, never equally spaced, so the algorithm never finishes (Flocchini et al. 2008b). The argument works for every even n with two interleaved regular $n/2$-gons. It is why the catalogued variant assumes an oriented ring.

Why it matters. It is a model example of an adversary that exploits mirror symmetry and the freedom to choose who moves.

What the community started looking at

Self-deployment asked **under which assumptions robots can spread uniformly** around a ring (Flocchini et al. 2008b) or along a segment (Cohen and Peleg 2008): orientation and knowledge of the target distance turned out to be the deciding factors.

Relations to other problems

- **Related: Scattering.** Both spread robots out; scattering only needs distinct positions, deployment needs equal spacing.

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Colours) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Body → Visibility → Opacity → Faults → Scheduler → Lights (persistent memory) → Agreement on coordinates → Movement → Multiplicity detection → Randomness → Identifiers.

No assumptions

+ point, + unlimited visibility, + transparent, + no faults, + chirality

Oriented ring, asynchronous, oblivious STANDARD SETTING ●● p. 345

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Colours	p.
Oriented ring, asynchronous, oblivious STANDARD SETTING	● unknown *finite* vs *impossible*	● tight 0 vs —	345

19.5.1 Oriented ring, asynchronous, oblivious

Adds + point, + unlimited visibility, + transparent, + no faults, + chirality STANDARD SETTING VERIFIED

On an oriented ring oblivious asynchronous robots deploy exactly if they know d, and within any ε if they do not.

Assumed: body: point robots; visibility: unlimited; opacity: transparent: every robot sees every robot within range; faults: no faults; agreement on coordinates: common chirality (same clockwise orientation). **Not assumed:** scheduler: asynchronous (ASYNC); lights (persistent memory): oblivious, no light (OBLOT); movement: non-rigid: the adversary may stop a move after some minimum distance δ; multiplicity detection: none: a point with several robots looks like one robot; randomness: deterministic; identifiers: anonymous: all robots are identical.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Flocchini et al. 2008b) —	*finite* (Flocchini et al. 2008b)	● unknown
Colours	none known No colours.	0 (Flocchini et al. 2008b)	● tight

Results.

- Flocchini et al. (2008b) [lower bound]: time *impossible*. *Bottleneck:* symmetry. *Holds when:* exact deployment without orientation, even semi-synchronous.
- Flocchini et al. (2008b) [current best]: time *finite*, colours 0. *Assumes:* d known for exact deployment. Introductory and current best; ε-approximate deployment without knowing d.

Open questions.

Q1. Is knowledge of d necessary for exact deployment on an oriented ring? Which capabilities allow asynchronous exact deployment on unoriented rings? (Flocchini et al. 2008b)

Claude (Anthropic) was used in cataloguing the problems.

Notes. The chirality value stands for an oriented ring. Spreading on a segment: (Cohen and Peleg 2008).

Claude (Anthropic) was used in cataloguing the problems.

Part V

Search in geometric domains

The model and its assumption axes are in Chapter 4.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 20

Search, evacuation and patrolling

Find a target, or get every robot out through an exit, at an unknown point of a known continuous domain. Evacuation builds on linear search (the cow-path problem).

Problems in this family:

- Linear search (cow path)
- Evacuation — generalisation of Linear search (cow path)
- Chauffeuring a crashed robot — variant of Evacuation
- Patrolling

20.1 Linear search (cow path)

A robot on a line (or on m rays from a point) must find a target at unknown distance; how small can the competitive ratio be?

First posed in 1964 (Beck 1964). Bellman posed the problem in 1963 and Beck analysed it in 1964; computer scientists rediscovered it in the late 1980s as the cow-path problem (Kupavskii and Welzl 2018).

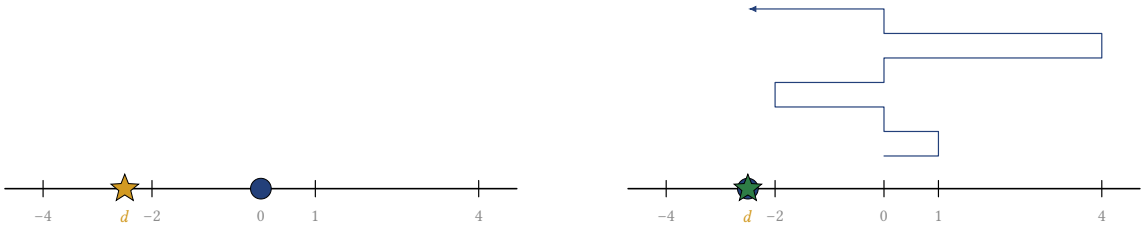
Problem statement and measures

One or more unit-speed robots start at the origin of a line (or of m rays sharing an endpoint). A target lies at an unknown point at distance $d \geq 1$. **Time** is the competitive ratio: the worst case, over target positions, of the time until the target is found (in **search**: by the first robot; in **evacuation**: by all robots) divided by d.

Measures: **Competitive ratio**, worst-case time divided by the distance to the target.

A first look

A cow stands at a gate on an endless straight fence and knows that a hole is somewhere along the fence, but not on which side or how far. She walks at unit speed and sees the hole only when she reaches it. In linear search the cow is a **robot**, a point that moves at unit speed and sees the target only when it reaches it; the gate is the **origin**, the point 0 of an infinite line where the robot starts; and the hole is the **target**, at an unknown distance $d \geq 1$ on an unknown side. We measure the **competitive ratio**: the worst case, over all target positions, of (time to find the target) divided by d. A competitive ratio of 9 means the robot never walks more than about nine times the distance it would walk if it knew where the target was.



(a) Start: robot at 0; the target is somewhere

(b) Final: found at time 16.5 = 6.6 d (worst case < 9d)

Figure 20.1: Linear search (cow path): a robot starts at the origin of a line and must reach a target at unknown distance $d \geq 1$ on an unknown side. The animation runs the doubling strategy at unit speed, with turning points $1, -2, 4, -8, \ldots$ (trace above the line, one level per trip); the target, at $d = 2.5$ just beyond the turning point -2, is found on the next trip to the left at time $16.5 = 6.6\,d$. In the worst case the strategy finds the target within time $9d$, which is optimal (Baeza-Yates et al. 1993).

1. Algorithm: Doubling: go 1 right, 2 left, 4 right, ... (Baeza-Yates et al. 1993) (One robot on a line, p. 353) A **strategy** is the rule that fixes where the robot walks. The doubling strategy:

1. Set $i = 0$.
2. **Trip** i: walk distance 2^i to one side (right when i is even, left when i is odd), checking for the target; the point where the robot turns back is a **turning point**.
3. If the target is not there, walk back to the origin; this ends trip i.
4. Increase i by one and repeat.

Why it works and what it costs. The worst place for the target is just past a turning point, say at $d = 2^i + \varepsilon$: the robot turned back there and finds it only on the next trip to that side, trip $i+2$. By then it has walked $2(1 + 2 + \cdots + 2^{i+1}) = 2(2^{i+2} - 1) < 8 \cdot 2^i < 8d$, plus the final d, in total less than $9d$. So the competitive ratio is at most 9, and (Baeza-Yates et al. 1993) prove that this is optimal for **deterministic** strategies, those that toss no coins.

Why it matters. Doubling is the universal answer to "I do not know how far it is": it reappears in search on rays, in online algorithms, and in algorithms that do not know n.

2. Lower bound: Why no deterministic strategy beats 9. (Baeza-Yates et al. 1993) (One robot on a line, p. 353) Any deterministic strategy can be described by its turning points $x_1, x_2, x_3, \ldots$, alternating sides, and one can show it is enough to consider growing ones. The **adversary**, an opponent who knows the strategy, puts the target just beyond x_i, on that side. The robot finds it only after the next turning point on the other side, so its time is at least $2(x_1 + \cdots + x_{i+1}) + x_i$. Write $S_j = x_1 + \cdots + x_j$. A competitive ratio at most $1 + 2c$ for every i needs $S_{i+1} \leq c\, x_i = c(S_i - S_{i-1})$.

Why it holds. Let $q_i = S_{i+1}/S_i$, which is greater than 1. Dividing the inequality by S_i gives $q_i \leq c - c/q_{i-1}$. Because $q + c/q \geq 2\sqrt{c}$ for every $q > 0$, we get $q_i \leq q_{i-1} - (2\sqrt{c} - c)$. If $c < 4$, the amount $2\sqrt{c} - c$ is a fixed positive number, so q_i drops by at least that much at every step and eventually falls below 1, a contradiction. Hence $c \geq 4$ and the competitive ratio is at least $1 + 2 \cdot 4 = 9$, matching doubling (up to small terms in d) (Baeza-Yates et al. 1993), (Beck 1964).

Why it matters. It is the prototype adversary argument for online search: hide the target just past a turning point of the robot.

Claude (Anthropic) was used in cataloguing the problems.

3. Algorithm: Randomize the first side and the scale. (Kao et al. 1996) (One randomized robot on a line, p. 354) A **randomized** strategy tosses coins:

1. Toss a coin to choose which side to try first.
2. Choose a random distance for the first turning point.
3. From then on, multiply the distance of the turning point by the same fixed factor on every trip, alternating sides.

Why it works and what it costs. The adversary, who must fix the target before the coins are tossed, no longer knows where the turning points are, so it cannot always place the target just past one. The expected competitive ratio drops to about 4.5911, and (Kao et al. 1996) prove that no randomized strategy does better on the line. On m **rays** (half-lines that share the origin as their endpoint) the optimal deterministic competitive ratio is $1 + 2m^m/(m-1)^{m-1}$ (Baeza-Yates et al. 1993), which gives 9 for $m = 2$.

Why it matters. It shows how randomness weakens an adversary that relies on knowing your strategy.

What the community started looking at

The first question was the **optimal search plan** for one searcher on a line (Beck 1964), posed by Bellman. Computer scientists recast it as competitive analysis and found that doubling is optimal with ratio 9, and the exact ratio for m rays, where the turning distances grow by the factor $m/(m-1)$ (Baeza-Yates et al. 1993); randomization lowers the ratio on the line to about 4.5911 (Kao et al. 1996). Recent work asks for **group search with faulty robots** (Czyzowicz et al. 2016b) (Kupavskii and Welzl 2018) (Czyzowicz et al. 2021a) and robots of different speeds (Bampas et al. 2019).

Relations to other problems

- **Derived problem: Evacuation** (generalisation of). Evacuation is a form of search with an extra requirement: "optimality in evacuation is measured by the time it takes for the last robot to reach the exit, whereas in traditional search, optimality is measured by the time it takes the first robot to reach the exit" (Czyzowicz et al. 2017a). The introductory paper notes its similarity with the cow-path problem, and that evacuating uniform agents on a line "is as hard as the cow-path problem" (Czyzowicz et al. 2014).

Assumptions specific to this problem

Domain

Bellman and Beck posed search for a target on the real line (Bellman 1963) (Beck 1964) (as reported by Czyzowicz et al. 2021a). Baeza-Yates, Culberson and Rawlins treated the line as two rays and generalised to m rays from a common origin, with the optimal deterministic competitive ratio $1 + 2m^m/(m-1)^{m-1}$, attained by turning at distances $(m/(m-1))^i$ while visiting the rays cyclically (Theorem 2.3 of Baeza-Yates et al. 1993). Searching on m concurrent rays has, in the words of Hammar et al. quoted by Brandt et al., "paradigmatic status" in competitive analysis (Brandt et al. 2020). The ray star is level 0 here because it contains the line ($m = 2$) as a special case.

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	m **rays**	$m \geq 2$ half-lines sharing their endpoint, the origin, where the robots start.
1	**line**	The real line, robots start at 0; the target is at an unknown x with $\|x\| \geq 1$.

Number of robots

The classical problem has one searcher (Beck 1964) (Baeza-Yates et al. 1993). Chrobak et al. and Czyzowicz et al. proposed parallel versions with k robots on the line, where communication and faults make the team size matter (Chrobak et al. 2015) (Czyzowicz et al. 2016b) (as reported by Kupavskii and Welzl 2018). With k fault-free robots, two of them already finish in time equal to the distance by going in opposite directions, so the multi-robot questions are interesting only with faults, with evacuation, or with unequal speeds (Kupavskii and Welzl 2018) (Bampas et al. 2019). Two robots are the smallest team in which speeds and communication interact, and Bampas et al. solve that case for every speed ratio (Bampas et al. 2019).

Level	Value	Formal definition
0	**any** k	$k \geq 1$ robots.
1	**one robot**	A single robot.
1	**two robots**	Two robots.

Goal

In search the time is the first arrival at the target, while in group evacuation it is the arrival of the last robot (Bampas et al. 2019) (Czyzowicz et al. 2021b). Chrobak et al. showed that evacuating several uniform agents on the line is as hard as the cow-path problem (as reported by Czyzowicz et al. 2014) (Chrobak et al. 2015). Bampas et al. compare search with rendezvous: in both the time is set by the last robot to arrive, but in search the meeting point is chosen by the adversary (Bampas et al. 2019). With faults the two goals are closely linked: when $k = 2f + 1$ all agents may have to reach the target, and crash evacuation is then "a sub-problem of the more difficult byzantine search problem" (Czyzowicz et al. 2021c).

Level	Value	Formal definition
0	**evacuation**	The task ends when every robot has reached the target.
1	**search**	The task ends when the first robot reaches the target (and, with faults, the others can be sure of it).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Competitive ratio) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Domain → Number of robots → Goal → Communication → Faults → Speeds → Randomness.

No assumptions

+ line

Number of robots

+ one robot, + search, + no faults, + equal speeds

One robot on a line STANDARD SETTING • p. 353

Claude (Anthropic) was used in cataloguing the problems.



What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Competitive ratio	p.
One robot on a line STANDARD SETTING	● tight 9 vs 9	353
One randomized robot on a line	● tight ≈ 4.5911 vs ≈ 4.5911	354
k robots on a line, f crash faults	● tight $2\rho^\rho/(\rho-1)^{\rho-1}+1$ vs $2\rho^\rho/(\rho-1)^{\rho-1}+1$	354
k robots on a line, Byzantine faults	● polynomial gap depends on f/k vs ≈ 5.23	355
Two robots of different speeds, both must reach the target	● tight *optimal for every speed ratio* vs *matching*	355
One robot on m rays	● tight $1+\frac{2m^m}{(m-1)^{m-1}}$ vs $1+\frac{2m^m}{(m-1)^{m-1}}$	356

20.1.1 One robot on a line

Adds + one robot, + search, + no faults, + equal speeds STANDARD SETTING VERIFIED

Doubling the turning point each time gives competitive ratio 9, and nothing deterministic does better.

Assumed: domain: the line (two rays); number of robots: a single robot; goal: search: the first robot to reach the target ends the task; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Competitive ratio	9 (Baeza-Yates et al. 1993) Exactly 9.	9 (Baeza-Yates et al. 1993)	● tight

Results.

- Baeza-Yates et al. (1993) [lower bound]: competitive ratio 9. *Bottleneck:* unknown distance. *Holds when:* deterministic; target at an unknown integer distance d on an unknown side; ratio up to lower-order terms in d.

Claude (Anthropic) was used in cataloguing the problems.

Theorem 2.1 of (Baeza-Yates et al. 1993). Also due to (Beck 1964) in the original formulation.

- Baeza-Yates et al. (1993) [current best]: competitive ratio 9. Introductory (in the competitive formulation) and current best: linear spiral search (turning points 2^i) walks at most $9d$ steps; there are algorithms with $9d - \Theta((\log d)^i)$ steps for any i.

Notes. Checked against the full text of (Baeza-Yates et al. 1993) (scan, Section 2.1). The bound is on the constant multiple of the distance; lower-order terms are not optimised.

20.1.2 One randomized robot on a line

Adds + randomized **to** One robot on a line (p. 353) VERIFIED

Randomization lowers the optimal expected ratio on the line to about 4.5911.

Assumed: domain: the line (two rays); number of robots: a single robot; goal: search: the first robot to reach the target ends the task; faults: no faults; speeds: all robots have the same unit speed; randomness: randomized. **Not assumed:** communication: face-to-face: robots exchange information only when they meet.

Measure	Lower bound	Best upper bound	Status
Competitive ratio	≈ 4.5911 (Kao et al. 1996) Optimal among randomized algorithms.	≈ 4.5911 (Kao et al. 1996)	• tight

Results.

- Kao et al. (1996) [lower bound]: competitive ratio ≈ 4.5911. *Bottleneck:* unknown distance. *Holds when:* expected competitive ratio; any randomized algorithm; goal position fixed in advance (oblivious adversary); two paths (the line). Theorem 4.1 of (Kao et al. 1996).
- Kao et al. (1996) [current best]: competitive ratio ≈ 4.5911. Introductory and current best (SmartCow): randomness only in a random permutation of the paths and a random initial distance; ratio ≈ 4.5911 on the line, and about $3.088\,m + o(m)$ on m rays.

Notes. Checked against the full text of (Kao et al. 1996). The ratio is the fixed point of an equation and is computed numerically (Theorem 5.1). Kao, Reif and Tate say Gal had obtained many of these results earlier in a game-theoretic setting, and that optimality for every number $m \geq 2$ of rays was proved afterwards by Kao et al. (SODA 1994; not in the bibliography).

20.1.3 k robots on a line, f crash faults

Adds + crash faults, + equal speeds VERIFIED

With k wireless robots of which f may fail to report the target, the optimal ratio is known exactly.

Assumed: domain: the line (two rays); goal: search: the first robot to reach the target ends the task; communication: wireless: instant communication at any distance; faults: up to f crash-faulty robots (fail to report the target); speeds: all robots have the same unit speed. **Not assumed:** number of robots: any number k of robots; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Competitive ratio	$2\rho^\rho/(\rho-1)^{\rho-1}+1$ (Kupavskii and Welzl 2018) Matching bounds.	$2\rho^\rho/(\rho-1)^{\rho-1}+1$ (Czyzowicz et al. 2016b)	• tight

Results.

Claude (Anthropic) was used in cataloguing the problems.

- Kupavskii and Welzl (2018) [lower bound]: competitive ratio $2\rho^\rho/(\rho-1)^{\rho-1}+1$. *Bottleneck:* faulty reports. *Holds when:* $\rho = 2(f+1)/k$, for $k < 2(f+1)$; ratio 1 if $k \geq 2f+2$.
- Czyzowicz et al. (2016b) [current best]: competitive ratio $2\rho^\rho/(\rho-1)^{\rho-1}+1$. Introductory and current best (proportional schedules); optimality proved by (Kupavskii and Welzl 2018).

Notes. On m rays the same formula holds with $\rho = m(f+1)/k$ (Kupavskii and Welzl 2018).

20.1.4 k robots on a line, Byzantine faults

Adds Byzantine, + equal speeds VERIFIED

Byzantine robots may lie about the target; the optimal ratio is known only for some densities f/k.

Assumed: domain: the line (two rays); goal: search: the first robot to reach the target ends the task; communication: wireless: instant communication at any distance; speeds: all robots have the same unit speed. **Not assumed:** number of robots: any number k of robots; faults: up to f Byzantine robots (may lie or stay silent); randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Competitive ratio	≈ 5.23 (Kupavskii and Welzl 2018)	depends on f/k (Czyzowicz et al. 2021a)	• polynomial gap
	For $k = 3$, $f = 1$ the upper bound is not matched.		

Results.

- Kupavskii and Welzl (2018) [lower bound]: competitive ratio ≈ 5.23. *Bottleneck:* indistinguishable Byzantine agents. *Holds when:* $k = 3$, $f = 1$; improves 3.93 of (Czyzowicz et al. 2021a).
- Czyzowicz et al. (2021a) [current best]: competitive ratio depends on f/k. Introductory and current best; optimal for some densities of faulty robots.

Open questions.

Q1. Determine the optimal competitive ratio $B(k, f)$ for Byzantine search on the line; already $B(3, 1)$ is open.

Notes. For three robots with one Byzantine robot the upper bound is about 7.437 (Czyzowicz et al. 2021c), improving 8.653 (Sun et al. 2020).

20.1.5 Two robots of different speeds, both must reach the target

Adds + two robots, + no faults VERIFIED

For two robots with different maximum speeds that must both reach the target, the optimal strategy is known for every speed ratio when they communicate by meeting.

Assumed: domain: the line (two rays); number of robots: two robots; faults: no faults. **Not assumed:** goal: evacuation: every robot must reach the target; communication: face-to-face: robots exchange information only when they meet; speeds: robots may have different maximum speeds; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Competitive ratio	*matching* (Bampas et al. 2019)	*optimal for every speed ratio* (Bampas et al. 2019)	• tight
	Optimal (face-to-face); the wireless case is optimal within the stated speed range.		

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Bampas et al. (2019) [lower bound]: competitive ratio *matching*. *Bottleneck:* unknown distance. *Holds when:* face-to-face communication; all speed ratios.
- Bampas et al. (2019) [current best]: competitive ratio *optimal for every speed ratio.* Wireless communication helps only while the faster robot is at most $\sqrt{17}+4 \approx 8.12$ times faster.

Notes. Introduced by Chrobak, Gąsieniec, Gorry and Martin (SOFSEM 2015, not yet in the bibliography).

20.1.6 One robot on m rays

Adds m rays, + one robot, + search, + no faults, + equal speeds VERIFIED

On m rays the optimal deterministic ratio is $1 + 2m^m/(m-1)^{m-1}$.

Assumed: number of robots: a single robot; goal: search: the first robot to reach the target ends the task; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** domain: m rays from a common origin; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Competitive ratio	$1+\frac{2m^m}{(m-1)^{m-1}}$ (Baeza-Yates et al. 1993) Exact.	$1+\frac{2m^m}{(m-1)^{m-1}}$ (Baeza-Yates et al. 1993)	• tight

Results.

- Baeza-Yates et al. (1993) [lower bound]: competitive ratio $1+\frac{2m^m}{(m-1)^{m-1}}$. *Bottleneck:* unknown distance. *Holds when:* deterministic; target at an unknown integer distance on one of m rays; ratio up to lower-order terms; also the special case $k=1$, $f=0$ of (Kupavskii and Welzl 2018). Theorem 2.3 of (Baeza-Yates et al. 1993).
- Baeza-Yates et al. (1993) [current best]: competitive ratio $1+\frac{2m^m}{(m-1)^{m-1}}$. Introductory and current best: generalized linear spiral search, turning points $(m/(m-1))^i$ and rays visited cyclically.

Notes. Checked against the full text of (Baeza-Yates et al. 1993) (scan, Section 2.3).

20.2 Evacuation

Robots start at the centre of a disk (or a polygon) and must all leave through an exit at an unknown point of the boundary, as fast as possible.

First posed in 2014 (Czyzowicz et al. 2014). Czyzowicz et al. posed evacuation of k robots from a disk through an unknown exit on its boundary at DISC 2014.

Problem statement and measures

k unit-speed robots start together at the centre of a unit disk (or the centroid of a unit equilateral triangle or square). An exit lies at an unknown point of the boundary; a robot recognises it only on reaching it. **Time** is the worst case, over exit positions, of the time until the last robot reaches the exit (in priority evacuation, where one robot is the **queen**: until the queen does).

Measures: **Evacuation time**, worst-case time until the last robot (or the queen) reaches the exit; unit disk, unit speed.

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: both robots at the centre O

(b) Final: robot 2 arrives by the chord at 1 + 2π/3 + √3

Figure 20.2: Evacuation from a disk: two robots start at the centre of a unit disk and must both reach an exit at an unknown point of the boundary. The animation runs the wireless algorithm of (Czyzowicz et al. 2014) at unit speed: both walk to the boundary together, then search it in opposite directions; the one that finds the exit tells the other by wireless communication, and the other walks straight across the disk along the chord. With the exit placed at arc distance $2\pi/3$ (the worst case) the time is $1 + 2\pi/3 + \sqrt{3} \approx 4.83$.

A first look

Two people stand in the middle of a dark round room of radius 1 and know there is one door somewhere along its edge, but not where; they can talk by walkie-talkie. In evacuation the people are **robots**, points that move at unit speed; the room is a unit **disk**, the points at distance at most 1 from its centre; its edge is the **boundary**, the circle of radius 1; and the door is the **exit**, a point of the boundary that a robot recognises only when it stands on it. k robots start at the centre of the disk. The goal is to minimise, in the worst case over exit positions, the time until the **last** robot reaches the exit. In the basic version there are two robots with **wireless** communication: a robot that finds the exit tells the other instantly, wherever it is.

1. Algorithm: Go to the boundary together, then split. (Czyzowicz et al. 2014) (Two wireless robots, disk, p. 365)

1. Both robots walk together from the centre to the same point A of the boundary (time 1).
2. One walks clockwise along the boundary, the other counter-clockwise, at full speed.
3. The robot that finds the exit tells the other by wireless communication.
4. The other walks straight across the disk to the exit, along a **chord** (the straight segment between two points of the boundary).

Why it works and what it costs. If the exit is found after each robot has walked an **arc** (a piece of the boundary) of length x (so $x \le \pi$), the robots are $2x$ apart along the boundary, and the chord between them has length $2\sin x$. The total time is $1 + x + 2\sin x$. Its derivative $1 + 2\cos x$ is zero at $x = 2\pi/3$, so the worst case is $1 + 2\pi/3 + \sqrt{3} \approx 4.826$ (Czyzowicz et al. 2014). This is exactly optimal (next idea).

Why it matters. Searching the boundary in opposite directions and cutting across on a chord is the template for almost every later disk evacuation algorithm.

2. Lower bound: Two unexplored points far apart. (Czyzowicz et al. 2014) (Two wireless robots, disk, p. 365) Look at time $1 + 2\pi/3$. Each robot needed time 1 just to reach the boundary, so together they have walked at most $4\pi/3$ of the boundary, and at least $2\pi/3$ of the boundary (minus a tiny ε) is unexplored. (Czyzowicz et al. 2014) show that then there are two unexplored points at arc distance about $2\pi/3$ (measured along the boundary), so the chord between them has length $\sqrt{3}$. The **adversary**, an opponent

Claude (Anthropic) was used in cataloguing the problems.

who watches the robots and chooses the exit position, waits until some robot first reaches one of these two points and places the exit at the other one.

Why it holds. That robot is then at least $\sqrt{3}$ away from the exit and must still get there, and this happens no earlier than time $1 + 2\pi/3$. So every algorithm needs at least $1 + 2\pi/3 + \sqrt{3}$ in the worst case, which matches the algorithm above (Czyzowicz et al. 2014).

Why it matters. Counting how much of the boundary can be explored by a given time, then hiding the exit at the far end of an unexplored chord, is the basic lower-bound tool for evacuation.

3. Algorithm: Without wireless communication: catch the other robot. (Czyzowicz et al. 2014) (Two face-to-face robots, disk, p. 367) If robots exchange information only when they meet (**face-to-face** communication):

1. Go to the boundary together and split in opposite directions, as before.
2. The robot that finds the exit cannot call; it walks along a chord to the point where the other robot will be, meets it,
3. and both walk back along a chord to the exit.

Why it works and what it costs. The robot that finds the exit must cover the distance to the other robot and back, so the cost is larger: about 5.74 for this simple version (Czyzowicz et al. 2014). Cleverer paths that leave the boundary and come back improved it to about 5.6234 (Disser and Schmitt 2019), while the best lower bound is $3 + \pi/6 + \sqrt{3} \approx 5.255$ (Czyzowicz et al. 2020). This gap has been open since 2019.

Why it matters. It shows how much communication is worth, and it is one of the simplest-to-state geometric questions still open in the area.

What the community started looking at

Evacuation was introduced to capture the tension between **spreading the robots** to find the exit fast and **keeping them close** so that the others can reach it quickly (Czyzowicz et al. 2014). The first results already separated wireless from face-to-face communication. The community then tightened the two-robot face-to-face constants (Czyzowicz et al. 2020) (Brandt et al. 2017) (Disser and Schmitt 2019) and added faults (Czyzowicz et al. 2017a), priority (one robot must leave) (Czyzowicz et al. 2018b), asymmetric communication (Georgiou et al. 2022), other domains (Chuangpishit et al. 2020b) and several exits (Pattanayak et al. 2018). Beyond disks, triangles and squares, the question was asked on the line and on rays (Chrobak et al. 2015) (Brandt et al. 2020), in regular polygons with priority (Georgiou et al. 2025a), on known trees as a planning problem that is NP-hard to solve exactly (Devillez et al. 2021), and in the plane for an unknown shoreline (Acharjee et al. 2020) or an unknown convex region to escape (WEI et al. 2016).

Further reading

Papers on this problem that are not yet placed in a setting.

- Czyzowicz et al. (2019b). Survey chapter on group search and evacuation. Not read.
- Kranakis (2024). Survey of how knowledge affects the competitive ratio of linear search, including evacuation. Not read.
- Georgiou et al. (2023b). Circle search with crash and Byzantine robots. Not read.
- Georgiou and Lucier (2023). Weighted group search on the line, with consequences for priority evacuation. Not read.

Claude (Anthropic) was used in cataloguing the problems.

- Czyzowicz et al. (2021d). Time–energy trade-offs for evacuating two wireless robots from the line.
- Jawhar and Kranakis (2021). Introduces bike-assisted evacuation on the line. Abstract only.
- Pattanayak et al. (2020). One-page summary of collaborative disk evacuation (two exits, chauffeuring). Abstract only.
- Chuangpishit et al. (2020a). Average versus worst case for two face-to-face robots in the disk; probably the journal version of (Chuangpishit et al. 2019). Not read.
- Liu et al. (2022). Evacuating crashed robots from a square. Abstract only.
- Fekete et al. (2010). Evacuating rectilinear polygons; background from before the disk line of work.
- Borowiecki et al. (2025). Discrete evacuation of many agents through known exits of a graph, limited by congestion: a different sense of evacuation. Not read.
- Czyzowicz et al. (2019c). Group search of the plane with crash and Byzantine robots. Abstract only.
- Dobrev et al. (2020). Lower bounds for searching an unknown line in the plane. Not read.
- Xu and Qin (2013). Groups escaping an unknown convex region of the plane; the group lower bound 3 is cited by (WEI et al. 2016).
- Wei et al. (2014). Online strategies for escaping a convex region. Not read.
- Li and Xu (2017). Online strategies for escaping a convex region. Not read.
- Liu et al. (2015). Groups escaping a convex region. Not read.
- Jiang et al. (2015). Groups escaping a convex region. Title only.
- Wei et al. (2016). Escaping an unknown affected area. Not read.
- Conley and Georgiou (2025). Related task: several agents inspect the boundary of a disk.
- Georgiou (2026). Related task: optimal average-case disk inspection.
- Coleman et al. (2023). Related task: two robots deliver a target to safety. Not read.
- Georgiou et al. (2025b). Plane search with agents of several speeds; a preprint, not read.
- Coleman et al. (2026). Search and rescue in the plane; a preprint, not read.

Relations to other problems

- **Generalisation of Linear search (cow path).** Evacuation is a form of search with an extra requirement: "optimality in evacuation is measured by the time it takes for the last robot to reach the exit, whereas in traditional search, optimality is measured by the time it takes the first robot to reach the exit" (Czyzowicz et al. 2017a). The introductory paper notes its similarity with the cow-path problem, and that evacuating uniform agents on a line "is as hard as the cow-path problem" (Czyzowicz et al. 2014).
- **Derived problem: Chauffeuring a crashed robot** (variant of). Evacuation where one robot has crashed inside the disk: the active robot must first find it, then carry it, more slowly, to an exit.

Assumptions specific to this problem

Domain

Czyzowicz et al. posed evacuation in the unit disk with the exit on its boundary (Czyzowicz et al. 2014), which remains the standard domain. Evacuation on the line had been raised just before by Chrobak et al., who showed that evacuating several uniform agents is as hard as the cow-path problem (as reported by

Claude (Anthropic) was used in cataloguing the problems.

Czyzowicz et al. 2014) (Chrobak et al. 2015). Polygons followed: equilateral triangles and squares, first with wireless robots (Czyzowicz et al. 2015) and then face-to-face (Chuangpishit et al. 2020b). Brandt et al. studied m concurrent rays, citing Hammar et al. that "a problem with paradigmatic status in this framework is searching on m concurrent rays" (Brandt et al. 2020). Evacuation from a general convex region appears in a separate line of online-strategy papers (Xu and Qin 2013) (Wei et al. 2014).

Level	Value	Formal definition
0	**any domain**	A bounded convex domain, robots start at an interior point.
1	**disk**	The unit disk; robots start at its centre; the exit is a point of the boundary circle.
1	**triangle**	The equilateral triangle of unit side; robots start at its centroid; the exit is a point of the boundary.
1	**square**	The unit square; robots start at its centroid; the exit is a point of the boundary.
1	**line**	The robots start at the origin of the real line; the exit is at an unknown point at distance at least 1; the measure is the competitive ratio, the evacuation time divided by that distance.
1	**rays**	$m \geq 2$ half-lines share an endpoint, the junction; the exit lies on one ray at distance at least 1 from the start; the measure is the competitive ratio.

Number of robots

Czyzowicz et al. studied the exact optimal evacuation time for $k = 2$ and $k = 3$ robots in the disk, and the asymptotic behaviour as k grows: $3 + 2\pi/k$ face-to-face and $3 + \pi/k$ wireless, up to lower-order terms (Czyzowicz et al. 2014). Most later disk papers fix $k = 2$, and even there the face-to-face case is not closed after several improvements (Czyzowicz et al. 2020) (Brandt et al. 2017) (Disser and Schmitt 2019). Specific small team sizes also appear in priority evacuation with one or two servants (Czyzowicz et al. 2018a) and with polygons, where triangles and squares were solved for $k = 2, 3, 4$ (Chuangpishit et al. 2020b).

Level	Value	Formal definition
0	**any k**	$k \geq 1$ robots.
1	**two robots**	Two robots.
1	**three robots**	Three robots.

Goal

Search ends at the first arrival and evacuation at the last (Czyzowicz et al. 2014). Czyzowicz et al. introduced *priority evacuation*, in which only a distinguished robot, the queen, must reach the exit while servants help. They motivate it by a robot that alone "has the capability to address an urgent need at the target, for example, performing an emergency procedure, or closing a breach in the perimeter" (Czyzowicz et al. 2018b). Georgiou, Karakostas and Kranakis introduced *search-and-fetch* (treasure evacuation), where a robot must carry a treasure to the exit. It is motivated by search-and-rescue, where drones must "locate a victim (the treasure) and bring her to safety (the exit)" (Georgiou et al. 2019). They note that "this is the first time an explicit ordering on the tasks to be performed is imposed on two moving robots" (Georgiou et al. 2019).

Claude (Anthropic) was used in cataloguing the problems.

Level	Value	Formal definition
0	**all robots**	The task ends when the last robot reaches the exit.
1	**priority (queen)**	One designated robot (the queen) must reach the exit; the others (servants) only help; the task ends when the queen exits.
1	**search-and-fetch**	A treasure lies at an unknown point of the boundary, and the exit at a known arc distance α from it; the task ends when some robot has carried the treasure to the exit.

Exits

Evacuation was posed with a single unknown exit (Czyzowicz et al. 2014). Czyzowicz et al. considered two robots on a circle with k exits whose distribution is known, where the robots do not know their own position (Czyzowicz et al. 2016a). Pattanayak, Ramesh, Mandal and Schmid studied two robots starting at the centre of the disk with two exits at unknown points whose arc distance α is known. They worked in both communication models and introduced labelled exits (Pattanayak et al. 2018). More exits help the robots, but only if the algorithm can use what it knows about the exits' relative position.

Level	Value	Formal definition
0	**one exit**	One exit, at an unknown point of the boundary.
1	**two exits**	Two exits at unknown points of the boundary; the task ends when every robot has reached one of them.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Evacuation time) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Domain → Number of robots → Goal → Exits → Communication → Faults → Speeds → Randomness.

No assumptions

+ disk

Number of robots

+ two robots

Goal

all robots, one exit, + wireless, + no faults

Disk, two wireless robots of different speeds ● p. 364

+ equal speeds

Two wireless robots, disk STANDARD SETTING ● p. 365

all robots, + two exits

Communication

face-to-face, + no faults, + equal speeds

Two robots, two exits, face-to-face ● p. 365

+ wireless, + no faults, + equal speeds

Two robots, two exits, wireless ● p. 366

+ search-and-fetch

Communication

+ wireless, + no faults

Disk, search-and-fetch, two wireless robots ● p. 366

Claude (Anthropic) was used in cataloguing the problems.

face-to-face, + no faults
Disk, search-and-fetch, two face-to-face robots ● p. 367

all robots, one exit, face-to-face, + no faults, + equal speeds
Two face-to-face robots, disk ● p. 367

all robots, one exit, + sender/receiver, + no faults, + equal speeds
Two robots with asymmetric communication, disk ● p. 368

+ priority (queen), + wireless, + no faults
Disk, priority evacuation, queen and one servant ● p. 368

+ three robots
Goal

all robots, + wireless
Faults

+ no faults, + equal speeds
Three wireless robots, disk ● p. 369

+ crash faults, + equal speeds
Three wireless robots, one crash fault ● p. 370

Byzantine, + equal speeds
Three wireless robots, one Byzantine ● p. 370

all robots, face-to-face, + no faults, + equal speeds
Three face-to-face robots, disk ● p. 370

+ priority (queen), + wireless, + no faults
Disk, priority evacuation, queen and two servants ● p. 371

any k, all robots, + wireless, + no faults, + equal speeds
Many wireless robots, disk ● p. 371

any k, all robots, face-to-face, + no faults, + equal speeds
Many face-to-face robots, disk ● p. 372

any k, + priority (queen), + wireless, + no faults, + equal speeds
Priority evacuation (queen and servants), disk ● p. 372

+ line
Number of robots

+ two robots
Communication

face-to-face, + no faults
Line, two face-to-face robots of different speeds ● p. 373

+ wireless, + no faults
Line, two wireless robots of different speeds ● p. 373

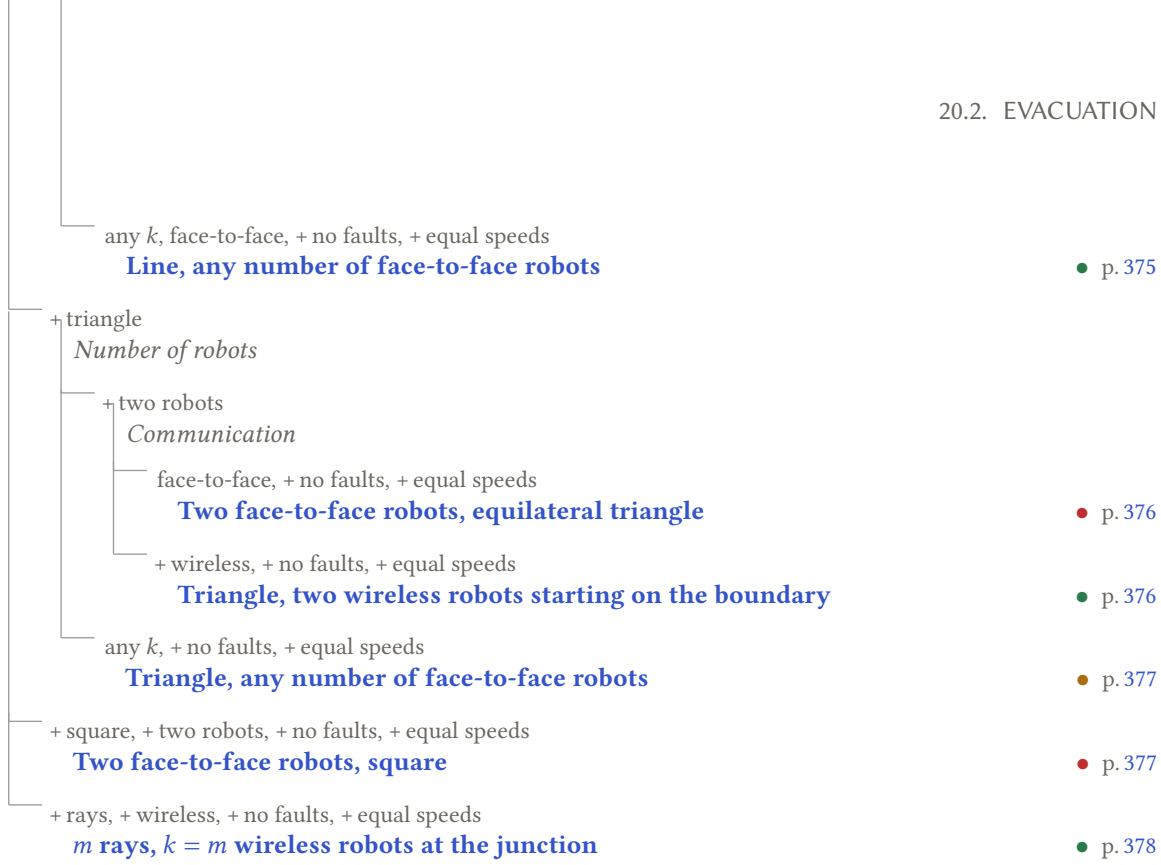
+ sender/receiver, + no faults, + equal speeds
Line, a sender and a receiver ● p. 374

any k, + wireless
Faults

+ no faults, + equal speeds
Line, any number of wireless robots ● p. 374

+ crash faults, + equal speeds
Line, $2f + 1$ wireless robots, f crash faults ● p. 375

Claude (Anthropic) was used in cataloguing the problems.



What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Evacuation time	p.
Disk, two wireless robots of different speeds	● polynomial gap $\frac{1+2\arccos(-2/v)}{v} + \sqrt{1-4/v^2}$ vs matching for $v \geq 2.75$	364
Two wireless robots, disk STANDARD SETTING	● tight $1 + 2\pi/3 + \sqrt{3}$ vs $1 + 2\pi/3 + \sqrt{3}$	365
Two robots, two exits, face-to-face	● tight $1 + \pi/2$ vs $1 + \pi/2$	365
Two robots, two exits, wireless	● polynomial gap $1 + \pi/3$ vs 2	366
Disk, search-and-fetch, two wireless robots	● unknown $1 + \pi - \alpha + 4\sin(\alpha/2)$ vs —	366
Disk, search-and-fetch, two face-to-face robots	● polynomial gap $1 + \pi - \alpha/2 + 3\sin(\alpha/2)$ vs $1 + \pi/3 + 4\sin(\alpha/2)$	367
Two face-to-face robots, disk	● polynomial gap ≈ 5.6234 vs $3 + \pi/6 + \sqrt{3}$	367
Two robots with asymmetric communication, disk	● polynomial gap $\pi + 2$ vs $> 1 + 2\pi/3 + \sqrt{3}$	368
Disk, priority evacuation, queen and one servant	● polynomial gap ≈ 4.81854 vs ≈ 4.56798	368
Three wireless robots, disk	● polynomial gap < 4.22 vs ≈ 4.159	369
Three wireless robots, one crash fault	● polynomial gap ≈ 6.309 vs ≈ 5.082	370
Three wireless robots, one Byzantine	● polynomial gap ≈ 6.921 vs ≈ 5.948	370
Three face-to-face robots, disk	● polynomial gap $3 + 2\pi/3$ vs ≈ 4.519	370

Claude (Anthropic) was used in cataloguing the problems.

Setting	Evacuation time	p.
Disk, priority evacuation, queen and two servants	● polynomial gap ≈ 3.8327 vs ≈ 3.6307	371
Many wireless robots, disk	● near-optimal $3+\pi/k+O(k^{-4/3})$ vs $3+\pi/k$	371
Many face-to-face robots, disk	● near-optimal $3+2\pi/k$ vs $3+2\pi/k-O(k^{-2})$	372
Priority evacuation (queen and servants), disk	● polynomial gap $2+4(\sqrt{2}-1)\pi/\sigma$ vs $1+\frac{2}{\sigma}\arccos\left(-\frac{2}{\sigma}\right)+\sqrt{1-\frac{4}{\sigma^2}}$	372
Line, two face-to-face robots of different speeds	● tight $\frac{1+3v}{1-v}$ $(v\le\frac{1}{3})$, $9v$ $(v\ge\frac{1}{3})$ vs *matching*	373
Line, two wireless robots of different speeds	● tight $\frac{1+3v}{1-v}$ $(v\le\sqrt{17}-4)$, $\frac{2+v+\sqrt{v^2+8v}}{2}$ above vs *matching*	373
Line, a sender and a receiver	● tight $3+2\sqrt{2}$ vs $3+2\sqrt{2}$	374
Line, any number of wireless robots	● tight 3 vs 3	374
Line, $2f+1$ wireless robots, f crash faults	● polynomial gap ≈ 7.437 vs ≈ 5.233	375
Line, any number of face-to-face robots	● tight 9 vs 9	375
Two face-to-face robots, equilateral triangle	● polynomial gap 2.3367 vs $1+2/\sqrt{3}$	376
Triangle, two wireless robots starting on the boundary	● tight $(a+b+c)/2$ vs $(a+b+c)/2$	376
Triangle, any number of face-to-face robots	● near-optimal $\sqrt{3}+3/k$ vs $\sqrt{3}$	377
Two face-to-face robots, square	● polynomial gap 3.4645 vs 3.118	377
m rays, $k=m$ wireless robots at the junction	● tight 3 vs 3	378

20.2.1 Disk, two wireless robots of different speeds

Adds all robots, one exit, + wireless, + no faults VERIFIED

With speeds 1 *and* $v\ge 1$*, two wireless robots evacuate optimally for* $v\ge 2.75$*; for smaller* v *the bounds are within a factor* 1.22*.*

Assumed: domain: unit disk, start at the centre; number of robots: two robots; communication: wireless: instant communication at any distance; faults: no faults. **Not assumed:** goal: every robot must reach the exit; exits: one exit; speeds: robots may have different maximum speeds; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	matching for $v\ge 2.75$ (Lamprou et al. 2020) Tight for $v\ge 2.75$; a constant factor below it.	$\frac{1+2\arccos(-2/v)}{v}+\sqrt{1-4/v^2}$ (Lamprou et al. 2020)	● polynomial gap

Results.

- Lamprou et al. (2020) [lower bound]: evacuation time matching for $v\ge 2.75$. *Bottleneck:* unknown exit. *Holds when:* speeds 1 and $v\ge 1$, both known; Section 5.

Claude (Anthropic) was used in cataloguing the problems.

- Lamprou et al. (2020) [current best]: evacuation time $\frac{1+2\arccos(-2/v)}{v} + \sqrt{1-4/v^2}$. Introductory and current best: optimal (Half-Chord) for $v \geq 2.75$; for $1 \leq v < 2.75$ the best of three algorithms, within a factor 1.22 of the lower bound.

Open questions.

Q1. Optimal evacuation for $1 < v < 2.75$, for more than two robots, and face-to-face (Lamprou et al. 2020).

Notes. Checked against the full text (arXiv version).

20.2.2 Two wireless robots, disk

Adds + equal speeds **to** Disk, two wireless robots of different speeds (p. 364) STANDARD SETTING VERIFIED

Two wireless robots evacuate the unit disk in exactly $1 + 2\pi/3 + \sqrt{3} \approx 4.826$.

Assumed: domain: unit disk, start at the centre; number of robots: two robots; communication: wireless: instant communication at any distance; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; exits: one exit; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$1 + 2\pi/3 + \sqrt{3}$ (Czyzowicz et al. 2014) ≈ 4.826, exact.	$1 + 2\pi/3 + \sqrt{3}$ (Czyzowicz et al. 2014)	• tight

Results.

- Czyzowicz et al. (2014) [lower bound]: evacuation time $1 + 2\pi/3 + \sqrt{3}$. *Bottleneck:* unknown exit. *Holds when:* deterministic; start at the centre.
- Czyzowicz et al. (2014) [current best]: evacuation time $1 + 2\pi/3 + \sqrt{3}$. Introductory and current best.

Notes. With one robot that has crashed and must be carried out, see Chauffeuring. In the unit disk of an ℓ_p norm, two wireless robots evacuate optimally for every $p \geq 1$, in time 5 for $p = 1$ and $p = \infty$ and about 4.7544 at best (Georgiou et al. 2023a). If every robot must visit every one of k treasures, $k = 1$ is this problem (Kaminski et al. 2026), a preprint.

20.2.3 Two robots, two exits, face-to-face

Adds face-to-face, + no faults, + equal speeds VERIFIED

With two exits at unknown positions and communication only on meeting, the optimal evacuation time is $(1 + \pi/2)R \approx 2.57R$.

Assumed: domain: unit disk, start at the centre; number of robots: two robots; exits: two exits at unknown positions; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$1 + \pi/2$ (Pattanayak et al. 2018) Optimal: $(1 + \pi/2)R$.	$1 + \pi/2$ (Pattanayak et al. 2018)	• tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Pattanayak et al. (2018) [lower bound]: evacuation time $1 + \pi/2$. *Bottleneck:* unknown exits. *Holds when:* two robots start at the centre; radius $R = 1$; oblivious robots.
- Pattanayak et al. (2018) [current best]: evacuation time $1 + \pi/2$. Introductory and current best; optimal.

Notes. Bounds from the full version, supplied by the catalogue owner (a co-author); the robots are oblivious.

20.2.4 Two robots, two exits, wireless

Adds + wireless, + no faults, + equal speeds VERIFIED

With two exits and wireless communication, evacuation takes between $2R$ and $(1 + \pi/3)R \approx 2.047R$.

Assumed: domain: unit disk, start at the centre; number of robots: two robots; exits: two exits at unknown positions; communication: wireless: instant communication at any distance; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	2 (Pattanayak et al. 2018) $(1 + \pi/3)R$ versus $2R$: a constant gap.	$1 + \pi/3$ (Pattanayak et al. 2018)	• polynomial gap

Results.

- Pattanayak et al. (2018) [lower bound]: evacuation time 2. *Bottleneck:* unknown exits. *Holds when:* two robots start at the centre; radius $R = 1$.
- Pattanayak et al. (2018) [current best]: evacuation time $1 + \pi/3$. Introductory and current best.

Open questions.

Q1. Close the gap between $2R$ and $(1 + \pi/3)R$ for two wireless robots and two exits (Pattanayak et al. 2018).

Notes. Bounds supplied by the catalogue owner (a co-author). When robots move only along a circle and the exits' arrangement is known, two robots starting at known distance L evacuate optimally in $3/4 + L/2$ (one exit, circle of unit length) (Czyzowicz et al. 2016a).

20.2.5 Disk, search-and-fetch, two wireless robots

Adds + wireless, + no faults VERIFIED

Two wireless robots find a treasure on the boundary and bring it to an exit at known arc distance α in $1 + \pi - \alpha + 4\sin(\alpha/2)$.

Assumed: domain: unit disk, start at the centre; number of robots: two robots; goal: search-and-fetch: a treasure must be brought to the exit; communication: wireless: instant communication at any distance; faults: no faults. **Not assumed:** exits: one exit; speeds: robots may have different maximum speeds; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	none known No lower bound is given.	$1 + \pi - \alpha + 4\sin(\alpha/2)$ (Georgiou et al. 2019)	• unknown

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Georgiou et al. (2019) [current best]: evacuation time $1 + \pi - \alpha + 4\sin(\alpha/2)$. Introductory and current best (Theorem 2.1).

Open questions.

Q1. Lower bounds that separate wireless from face-to-face search-and-fetch (Georgiou et al. 2019).

Notes. With one robot, the treasure-evacuation version is solved optimally when only a lower bound on the treasure–exit distance is known (Georgiou et al. 2021) (abstract only). Checked against the full text of (Georgiou et al. 2019).

20.2.6 Disk, search-and-fetch, two face-to-face robots

Adds face-to-face, + no faults VERIFIED

Face-to-face, two robots fetch the treasure in $1 + \pi - \alpha/2 + 3\sin(\alpha/2)$*, against* $1 + \pi/3 + 4\sin(\alpha/2)$ *for* $\alpha \le 2\pi/3$.

Assumed: domain: unit disk, start at the centre; number of robots: two robots; goal: search-and-fetch: a treasure must be brought to the exit; faults: no faults. **Not assumed:** exits: one exit; communication: face-to-face: robots exchange information only when they meet; speeds: robots may have different maximum speeds; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$1 + \pi/3 + 4\sin(\alpha/2)$ (Georgiou et al. 2019) Constant gap.	$1 + \pi - \alpha/2 + 3\sin(\alpha/2)$ (Georgiou et al. 2019)	• polynomial gap

Results.

- Georgiou et al. (2019) [lower bound]: evacuation time $1 + \pi/3 + 4\sin(\alpha/2)$. *Bottleneck:* unknown exit. *Holds when:* Theorem 4.1; $\alpha \le 2\pi/3$; $1 + \pi/3 + 2\sin\alpha + 2\sin(\alpha/2)$ for larger α.
- Georgiou et al. (2019) [current best]: evacuation time $1 + \pi - \alpha/2 + 3\sin(\alpha/2)$. Introductory and current best (Theorem 3.1).

Notes. Checked against the full text.

20.2.7 Two face-to-face robots, disk

Adds all robots, one exit, face-to-face, + no faults, + equal speeds VERIFIED

Face-to-face, two robots need between ≈ 5.255 *and* ≈ 5.6234*; the gap has been open since 2019.*

Assumed: domain: unit disk, start at the centre; number of robots: two robots; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; exits: one exit; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$3 + \pi/6 + \sqrt{3}$ (Czyzowicz et al. 2020)	≈ 5.6234 (Disser and Schmitt 2019)	• polynomial gap
	≈ 5.6234 versus ≈ 5.255: a constant gap of about 0.37.		

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Czyzowicz et al. (2020) [lower bound]: evacuation time $3 + \pi/6 + \sqrt{3}$. *Bottleneck:* unknown exit. *Holds when:* face-to-face; improves $3 + \pi/4 + \sqrt{2}$ of (Czyzowicz et al. 2014).
- Czyzowicz et al. (2014) [introductory]: evacuation time ≈ 5.74.
- Czyzowicz et al. (2020) [also]: evacuation time ≈ 5.628.
- Brandt et al. (2017) [also]: evacuation time ≈ 5.625. Forced meetings exchange no useful information; simpler algorithm.
- Disser and Schmitt (2019) [current best]: evacuation time ≈ 5.6234. A second detour through the interior.

Open questions.

Q1. Close the gap between $3 + \pi/6 + \sqrt{3} \approx 5.255$ and 5.6234 for two face-to-face robots.

Notes. The "second cut" of (Disser and Schmitt 2019) is a second detour through the interior: an upper-bound technique. Measured in the average case over the exit position, the trade-off between average and worst-case time is mapped, and $1 + \pi$ is conjectured to be the optimal average for any number of robots (Chuangpishit et al. 2019).

20.2.8 Two robots with asymmetric communication, disk

Adds all robots, one exit, + sender/receiver, + no faults, + equal speeds VERIFIED

When one robot can only send and the other only receive, evacuation lies strictly between the wireless and face-to-face optima; $\pi + 2$ suffices.

Assumed: domain: unit disk, start at the centre; number of robots: two robots; communication: asymmetric: senders and receivers; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; exits: one exit; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$> 1 + 2\pi/3 + \sqrt{3}$ (Georgiou et al. 2022)	$\pi + 2$ (Georgiou et al. 2022)	• polynomial gap
	No explicit lower bound.		

Results.

- Georgiou et al. (2022) [lower bound]: evacuation time $> 1 + 2\pi/3 + \sqrt{3}$. *Bottleneck:* one-way communication. *Holds when:* strictly above the wireless optimum; no explicit value.
- Georgiou et al. (2022) [current best]: evacuation time $\pi + 2$. Introductory and current best.

Open questions.

Q1. Give an explicit lower bound above ≈ 4.826 for sender/receiver evacuation.

20.2.9 Disk, priority evacuation, queen and one servant

Adds + priority (queen), + wireless, + no faults VERIFIED

A queen and one servant evacuate the queen in about 4.819*, strictly faster than evacuating both; the lower bound is about* 4.568*.*

Claude (Anthropic) was used in cataloguing the problems.

Assumed: domain: unit disk, start at the centre; number of robots: two robots; goal: priority: only the queen must reach the exit; communication: wireless: instant communication at any distance; faults: no faults. **Not assumed:** exits: one exit; speeds: robots may have different maximum speeds; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	≈ 4.56798 (Georgiou and Wang 2025) About 4.819 against 4.568.	≈ 4.81854 (Georgiou and Wang 2025)	• polynomial gap

Results.

- Georgiou and Wang (2025) [lower bound]: evacuation time ≈ 4.56798. *Bottleneck:* unknown exit. *Holds when:* Theorem 1; a linear-programming relaxation over polygon instances.
- Czyzowicz et al. (2018a) [introductory]: evacuation time ≈ 4.81854. Theorem 8; also the first lower bound, about 4.3896 (Theorem 17).
- Georgiou and Wang (2025) [current best]: evacuation time ≈ 4.81854. Same algorithm, with the improved lower bound.

Open questions.

Q1. Close the gap between 4.568 and 4.819 (Georgiou and Wang 2025).

Notes. Weighted group search, where the two robots' arrival times are weighted 1 and w, is solved within a gap in the same paper (Georgiou and Wang 2025). Checked against the full texts.

20.2.10 Three wireless robots, disk

Adds + no faults, + equal speeds VERIFIED

Three wireless robots: between ≈ 4.159 *and* 4.22.

Assumed: domain: unit disk, start at the centre; number of robots: three robots; communication: wireless: instant communication at any distance; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; exits: one exit; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	≈ 4.159 (Czyzowicz et al. 2014) A small constant gap.	< 4.22 (Czyzowicz et al. 2014)	• polynomial gap

Results.

- Czyzowicz et al. (2014) [lower bound]: evacuation time ≈ 4.159. *Bottleneck:* unknown exit. *Holds when:* three robots, wireless.
- Czyzowicz et al. (2014) [current best]: evacuation time < 4.22. Introductory and current best.

Open questions.

Q1. Find the optimal evacuation time for three wireless robots.

Claude (Anthropic) was used in cataloguing the problems.

20.2.11 Three wireless robots, one crash fault

Adds + crash faults, + equal speeds VERIFIED

If one of three wireless robots may crash, evacuation takes between ≈ 5.082 *and* ≈ 6.309.

Assumed: domain: unit disk, start at the centre; number of robots: three robots; communication: wireless: instant communication at any distance; faults: up to f crash-faulty robots (fail to report the target); speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; exits: one exit; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	≈ 5.082 (Czyzowicz et al. 2017a)	≈ 6.309 (Czyzowicz et al. 2017a)	• polynomial gap
	Constant gap.		

Results.

- Czyzowicz et al. (2017a) [lower bound]: evacuation time ≈ 5.082. *Bottleneck:* faulty reports. *Holds when:* three robots, at most one crash.
- Czyzowicz et al. (2017a) [current best]: evacuation time ≈ 6.309. Introductory and current best; time until the last non-faulty robot exits.

Open questions.

Q1. Tight bounds for disk evacuation with one crash-faulty robot among three.

20.2.12 Three wireless robots, one Byzantine

Adds Byzantine, + equal speeds VERIFIED

With one Byzantine robot among three: between ≈ 5.948 *and* ≈ 6.921.

Assumed: domain: unit disk, start at the centre; number of robots: three robots; communication: wireless: instant communication at any distance; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; exits: one exit; faults: up to f Byzantine robots (may lie or stay silent); randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	≈ 5.948 (Czyzowicz et al. 2017a)	≈ 6.921 (Czyzowicz et al. 2017a)	• polynomial gap
	Constant gap.		

Results.

- Czyzowicz et al. (2017a) [lower bound]: evacuation time ≈ 5.948. *Bottleneck:* indistinguishable Byzantine agents. *Holds when:* three robots, at most one Byzantine.
- Czyzowicz et al. (2017a) [current best]: evacuation time ≈ 6.921. Introductory and current best.

Notes. Byzantine evacuation with k robots and f faults is studied by (Leonardos et al. 2023) and (Behrouz et al. 2025); those papers were not read.

20.2.13 Three face-to-face robots, disk

Adds all robots, face-to-face, + no faults, + equal speeds VERIFIED

Three face-to-face robots: between ≈ 4.519 *and* $3 + 2\pi/3 \approx 5.09$.

Claude (Anthropic) was used in cataloguing the problems.

Assumed: domain: unit disk, start at the centre; number of robots: three robots; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; exits: one exit; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	≈ 4.519 (Czyzowicz et al. 2014) Constant gap.	$3 + 2\pi/3$ (Czyzowicz et al. 2014)	• polynomial gap

Results.

- Czyzowicz et al. (2014) [lower bound]: evacuation time ≈ 4.519. *Bottleneck:* unknown exit. *Holds when:* three robots, face-to-face.
- Czyzowicz et al. (2014) [current best]: evacuation time $3 + 2\pi/3$. Introductory and current best.

20.2.14 Disk, priority evacuation, queen and two servants

Adds + priority (queen), + wireless, + no faults VERIFIED

A queen and two servants evacuate the queen in about 3.833 *against a lower bound of about* 3.631.

Assumed: domain: unit disk, start at the centre; number of robots: three robots; goal: priority: only the queen must reach the exit; communication: wireless: instant communication at any distance; faults: no faults. **Not assumed:** exits: one exit; speeds: robots may have different maximum speeds; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	≈ 3.6307 (Czyzowicz et al. 2018a) Constant gap.	≈ 3.8327 (Czyzowicz et al. 2018a)	• polynomial gap

Results.

- Czyzowicz et al. (2018a) [lower bound]: evacuation time ≈ 3.6307. *Bottleneck:* unknown exit. *Holds when:* Theorem 19.
- Czyzowicz et al. (2018a) [current best]: evacuation time ≈ 3.8327. Introductory and current best (Theorem 10).

Notes. Checked against the full text.

20.2.15 Many wireless robots, disk

Adds any k, all robots, + wireless, + no faults, + equal speeds VERIFIED

With k wireless robots the time approaches $3 + \pi/k$.

Assumed: domain: unit disk, start at the centre; communication: wireless: instant communication at any distance; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** number of robots: any number k of robots; goal: every robot must reach the exit; exits: one exit; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$3 + \pi/k$ (Czyzowicz et al. 2014) Matching up to $O(k^{-4/3})$.	$3 + \pi/k + O(k^{-4/3})$ (Czyzowicz et al. 2014)	• near-optimal

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Czyzowicz et al. (2014) [lower bound]: evacuation time $3 + \pi/k$. *Bottleneck:* unknown exit. *Holds when:* k robots, wireless.
- Czyzowicz et al. (2014) [current best]: evacuation time $3 + \pi/k + O(k^{-4/3})$. Introductory and current best.

20.2.16 Many face-to-face robots, disk

Adds any k, all robots, face-to-face, + no faults, + equal speeds VERIFIED

With k face-to-face robots the time is $3 + 2\pi/k$ up to $O(k^{-2})$; the separation from wireless is a factor 2 in the second term.

Assumed: domain: unit disk, start at the centre; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** number of robots: any number k of robots; goal: every robot must reach the exit; exits: one exit; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$3 + 2\pi/k - O(k^{-2})$ (Czyzowicz et al. 2014) Matching up to $O(k^{-2})$.	$3 + 2\pi/k$ (Czyzowicz et al. 2014)	• near-optimal

Results.

- Czyzowicz et al. (2014) [lower bound]: evacuation time $3 + 2\pi/k - O(k^{-2})$. *Bottleneck:* unknown exit. *Holds when:* k robots, face-to-face.
- Czyzowicz et al. (2014) [current best]: evacuation time $3 + 2\pi/k$. Introductory and current best.

20.2.17 Priority evacuation (queen and servants), disk

Adds any k, + priority (queen), + wireless, + no faults, + equal speeds VERIFIED

Only the queen must leave; σ servants (the paper writes n; $k = \sigma + 1$ robots) help her find the exit.

Assumed: domain: unit disk, start at the centre; goal: priority: only the queen must reach the exit; communication: wireless: instant communication at any distance; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** number of robots: any number k of robots; exits: one exit; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$1 + \frac{2}{\sigma}\arccos\left(-\frac{2}{\sigma}\right) + \sqrt{1 - \frac{4}{\sigma^2}}$ (Czyzowicz et al. 2018b) For large σ the gap between the closed-form bounds is about $(4\sqrt{2} - 5)\pi/\sigma \approx 0.657\pi/\sigma$ (Czyzowicz et al. 2018b).	$2 + 4(\sqrt{2} - 1)\pi/\sigma$ (Czyzowicz et al. 2018b)	• polynomial gap

Results.

- Czyzowicz et al. (2018b) [lower bound]: evacuation time $1 + \frac{2}{\sigma}\arccos\left(-\frac{2}{\sigma}\right) + \sqrt{1 - \frac{4}{\sigma^2}}$. *Bottleneck:* unknown exit. *Holds when:* $\sigma \geq 4$ servants, wireless; all robots start at the centre of a unit disk. Theorem 5 of (Czyzowicz et al. 2018b); it exceeds $2 + \pi/\sigma + 2/\sigma^2$, the leading terms of its expansion in $1/\sigma$, which is the form stated in the abstract (e.g. 2.913 against 2.910 for $\sigma = 4$). The naive bound is $2 + \pi/(\sigma + 1)$.
- Czyzowicz et al. (2018b) [current best]: evacuation time $2 + 4(\sqrt{2} - 1)\pi/\sigma$. *Assumes:* $\sigma \geq 4$ servants. Introductory and current best (Theorem 1). The exact times of the algorithm are computed numerically, e.g. 3.113 for $\sigma = 4$ against the closed-form upper bound 3.301.

Claude (Anthropic) was used in cataloguing the problems.

Open questions.

Q1. Close the gap for priority evacuation; for one servant the bounds differ by about 0.25 (a 2024 preprint, not yet in the bibliography).

Notes. For a queen and three servants the bounds are about 3.374 and 3.202 (Czyzowicz et al. 2018a). The authors conjecture their algorithm for $\sigma \geq 4$ servants is optimal (Czyzowicz et al. 2018b). The lower bound was checked against the full text (arXiv version) of (Czyzowicz et al. 2018b): Section 4 proves the exact expression and the abstract states its asymptotic form $2 + \pi/\sigma + 2/\sigma^2$, so the two agree.

20.2.18 Line, two face-to-face robots of different speeds

Adds face-to-face, + no faults VERIFIED

With speeds 1 *and* $v \leq 1$, *two face-to-face robots evacuate the line optimally: ratio* $(1 + 3v)/(1 - v)$ *for* $v \leq 1/3$ *and* $9v$ *above.*

Assumed: domain: the infinite line; number of robots: two robots; faults: no faults. **Not assumed:** goal: every robot must reach the exit; exits: one exit; communication: face-to-face: robots exchange information only when they meet; speeds: robots may have different maximum speeds; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	*matching* (Bampas et al. 2019) Exact for every v.	$\frac{1+3v}{1-v}$ ($v \leq \frac{1}{3}$), $9v$ ($v \geq \frac{1}{3}$) (Bampas et al. 2019)	• tight

Results.

- Bampas et al. (2019) [lower bound]: evacuation time *matching*. *Bottleneck:* unknown distance. *Holds when:* Theorem 3.11.
- Bampas et al. (2019) [current best]: evacuation time $\frac{1+3v}{1-v}$ ($v \leq \frac{1}{3}$), $9v$ ($v \geq \frac{1}{3}$). Introductory and current best (Theorem 3.11); both robots must reach the exit.

Notes. Checked against the full text.

20.2.19 Line, two wireless robots of different speeds

Adds + wireless, + no faults VERIFIED

With speeds 1 *and* $v \leq 1$, *two wireless robots evacuate the line optimally, with a ratio that changes form at* $v = \sqrt{17} - 4$.

Assumed: domain: the infinite line; number of robots: two robots; communication: wireless: instant communication at any distance; faults: no faults. **Not assumed:** goal: every robot must reach the exit; exits: one exit; speeds: robots may have different maximum speeds; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	*matching* (Bampas et al. 2019) Exact for every v.	$\frac{1+3v}{1-v}$ ($v \leq \sqrt{17} - 4$), $\frac{2+v+\sqrt{v^2+8v}}{2}$ above (Bampas et al. 2019)	• tight

Results.

- Bampas et al. (2019) [lower bound]: evacuation time *matching*. *Bottleneck:* unknown distance. *Holds when:* Theorem 4.8.

Claude (Anthropic) was used in cataloguing the problems.

- Bampas et al. (2019) [current best]: evacuation time $\frac{1+3v}{1-v}$ ($v \leq \sqrt{17}-4$), $\frac{2+v+\sqrt{v^2+8v}}{2}$ above. Introductory and current best (Theorem 4.8).

Notes. Checked against the full text.

20.2.20 Line, a sender and a receiver

Adds + sender/receiver, + no faults, + equal speeds VERIFIED

A robot that can only send and one that can only receive wirelessly evacuate the line with ratio exactly $3+2\sqrt{2}$.

Assumed: domain: the infinite line; number of robots: two robots; communication: asymmetric: senders and receivers; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; exits: one exit; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$3+2\sqrt{2}$ (Czyzowicz et al. 2021b) Exactly $3+2\sqrt{2}$.	$3+2\sqrt{2}$ (Czyzowicz et al. 2021b)	• tight

Results.

- Czyzowicz et al. (2021b) [lower bound]: evacuation time $3+2\sqrt{2}$. *Bottleneck:* one-way communication. *Holds when:* Theorem 10.
- Czyzowicz et al. (2021b) [current best]: evacuation time $3+2\sqrt{2}$. Introductory and current best (Theorem 1).

Open questions.

Q1. Close the gaps for one sender and many receivers ($2+\sqrt{5}$ against 5) and for many senders and one receiver (3 against about 5.681) (Czyzowicz et al. 2021b).

Notes. If an extra vehicle (a bike) can carry a robot faster, sender/receiver evacuation on the line has piecewise bounds in the bike's speed (Jawhar and Kranakis 2025). Checked against the full text of (Czyzowicz et al. 2021b).

20.2.21 Line, any number of wireless robots

Adds + no faults, + equal speeds DRAFT

With wireless communication, two or more robots evacuate the line with competitive ratio exactly 3.

Assumed: domain: the infinite line; communication: wireless: instant communication at any distance; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** number of robots: any number k of robots; goal: every robot must reach the exit; exits: one exit; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	3 (Chrobak et al. 2015) Exactly 3.	3 (Chrobak et al. 2015)	• tight

Results.

- Chrobak et al. (2015) [lower bound]: evacuation time 3. *Bottleneck:* unknown distance. *Holds when:* as cited by (Czyzowicz et al. 2021b).

Claude (Anthropic) was used in cataloguing the problems.

- Chrobak et al. (2015) [current best]: evacuation time 3. Introductory and current best: two robots search in opposite directions and call the others; as cited by (Czyzowicz et al. 2021b).

Notes. **Unverified:** (Chrobak et al. 2015) was not read; the bounds are as cited by later papers.

20.2.22 Line, $2f + 1$ wireless robots, f crash faults

Adds + crash faults, + equal speeds VERIFIED

With a near majority of faulty robots, evacuation on the line takes ratio about 7.437 for $(k, f) = (3, 1)$, tending to $4 + 2\sqrt{2}$; the lower bounds come from search.

Assumed: domain: the infinite line; communication: wireless: instant communication at any distance; faults: up to f crash-faulty robots (fail to report the target); speeds: all robots have the same unit speed. **Not assumed:** number of robots: any number k of robots; goal: every robot must reach the exit; exits: one exit; randomness: deterministic.

Statement for this setting. $k = 2f + 1$ robots, of which up to f may fail to report the exit; every non-faulty robot must reach it.

Measure	Lower bound	Best upper bound	Status
Evacuation time	≈ 5.233 (Kupavskii and Welzl 2018)	≈ 7.437 (Czyzowicz et al. 2021c)	• polynomial gap
	Constant gap against the search lower bounds.		

Results.

- Kupavskii and Welzl (2018) [lower bound]: evacuation time ≈ 5.233. *Bottleneck:* faulty reports. *Holds when:* $(k, f) = (3, 1)$; inherited from search with crash faults.
- Czyzowicz et al. (2021c) [current best]: evacuation time ≈ 7.437. Introductory and current best for $(3, 1)$; 7.254 for $(5, 2)$ and $(7, 3)$, 7.147 for $(9, 4)$, tending to $4 + 2\sqrt{2}$ (Theorems 2.2, 3.1, 3.2).

Open questions.

Q1. Tight bounds for evacuation with $k = 2f + 1$ crash or Byzantine robots (Czyzowicz et al. 2021c).

Notes. When every attempt to turn fails with known probability $\beta < 1/2$ (the paper's p), two wireless robots achieve ratio $3 + 4\sqrt{\beta(1-\beta)} + \varepsilon$ (Georgiou et al. 2024). Checked against the full text of (Czyzowicz et al. 2021c).

20.2.23 Line, any number of face-to-face robots

Adds any k, face-to-face, + no faults, + equal speeds VERIFIED

On the line, face-to-face robots evacuate with competitive ratio exactly 9: extra robots do not help.

Assumed: domain: the infinite line; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** number of robots: any number k of robots; goal: every robot must reach the exit; exits: one exit; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	9 (Bampas et al. 2019)	9 (Chrobak et al. 2015)	• tight
	Exactly 9.		

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Bampas et al. (2019) [lower bound]: evacuation time 9. *Bottleneck:* unknown distance. *Holds when:* any number of robots of maximum speed at most 1; full proof of $9d - o(d)$.
- Chrobak et al. (2015) [current best]: evacuation time 9. Introductory and current best: the robots move together and run the doubling strategy; as described by later papers.

Notes. For two face-to-face robots the energy of group search can be traded against time; with maximum speed v and time at most $9d/v$ the energy is about $8.42588\,v^2 d$ (the paper writes b for v; Theorem 16) (Czyzowicz et al. 2019a). Checked against the full text of (Bampas et al. 2019); (Chrobak et al. 2015) was not read.

20.2.24 Two face-to-face robots, equilateral triangle

Adds face-to-face, + no faults, + equal speeds VERIFIED

From the centroid of a unit equilateral triangle: between $1 + 2/\sqrt{3}$ and 2.3367.

Assumed: domain: unit equilateral triangle, start at the centroid; number of robots: two robots; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; exits: one exit; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$1 + 2/\sqrt{3}$ (Chuangpishit et al. 2020b) Constant gap.	2.3367 (Chuangpishit et al. 2020b)	• polynomial gap

Results.

- Chuangpishit et al. (2020b) [lower bound]: evacuation time $1 + 2/\sqrt{3}$. *Bottleneck:* unknown exit. *Holds when:* two robots, face-to-face, start at the centroid.
- Chuangpishit et al. (2020b) [current best]: evacuation time 2.3367. For k robots: $\sqrt{3} + 3/k$ against $\sqrt{3}$.

Notes. Wireless evacuation of triangles and squares was introduced at ADHOC-NOW 2015 (not yet in the bibliography).

20.2.25 Triangle, two wireless robots starting on the boundary

Adds + wireless, + no faults, + equal speeds VERIFIED

Two wireless robots starting on the boundary of a non-obtuse triangle evacuate in half the perimeter if they may choose their start, which is optimal.

Assumed: domain: unit equilateral triangle, start at the centroid; number of robots: two robots; communication: wireless: instant communication at any distance; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; exits: one exit; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$(a+b+c)/2$ (Georgiou and Jang 2026) When the algorithm chooses the start.	$(a+b+c)/2$ (Georgiou and Jang 2026)	• tight

Results.

Claude (Anthropic) was used in cataloguing the problems.

- Georgiou and Jang (2026) [lower bound]: evacuation time $(a+b+c)/2$. *Bottleneck:* unknown exit. *Holds when:* Theorem 1; non-obtuse triangle with sides $a \geq b \geq c$; the algorithm chooses the starting edge and point.
- Czyzowicz et al. (2015) [introductory]: evacuation time $3/2 + y$. Equilateral unit triangle, start on the boundary at distance y from a side's midpoint; tight; as cited by later papers.
- Georgiou and Jang (2026) [current best]: evacuation time $(a+b+c)/2$. Non-obtuse triangles (Theorem 1); separate tight or near-tight bounds when the adversary chooses the edge or the point.

Open questions.

Q1. Face-to-face evacuation of general triangles, and wireless evacuation of obtuse triangles (Georgiou and Jang 2026).

Notes. With many wireless robots an equilateral triangle is evacuated in $1 + \sqrt{3}/3$, optimal with 6 robots (Czyzowicz et al. 2015); with communication range r, $6 + 2\lceil 1/r - 1 \rceil$ robots achieve it (Bagheri et al. 2019). Checked against the full texts of (Georgiou and Jang 2026) and (Bagheri et al. 2019).

20.2.26 Triangle, any number of face-to-face robots

Adds any k, + no faults, + equal speeds VERIFIED

k face-to-face robots at the centroid of an equilateral unit triangle evacuate in at most $\sqrt{3} + 3/k$, against $\sqrt{3}$.

Assumed: domain: unit equilateral triangle, start at the centroid; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** number of robots: any number k of robots; goal: every robot must reach the exit; exits: one exit; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	$\sqrt{3}$ (Chuangpishit et al. 2020b) The gap $3/k$ vanishes as k grows.	$\sqrt{3} + 3/k$ (Chuangpishit et al. 2020b)	• near-optimal

Results.

- Chuangpishit et al. (2020b) [lower bound]: evacuation time $\sqrt{3}$. *Bottleneck:* unknown exit. *Holds when:* Theorem 3.1; every k.
- Chuangpishit et al. (2020b) [current best]: evacuation time $\sqrt{3} + 3/k$. Introductory and current best (Observation 3.2); for $k = 3, 4, 5, 6$ about 2.089, 1.982, 1.876, 1.826.

Notes. Checked against the full text.

20.2.27 Two face-to-face robots, square

Adds + square, + two robots, + no faults, + equal speeds VERIFIED

From the centroid of a unit square: between 3.118 *and* 3.4645.

Assumed: domain: unit square, start at the centroid; number of robots: two robots; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** goal: every robot must reach the exit; exits: one exit; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	3.118 (Chuangpishit et al. 2020b) Constant gap.	3.4645 (Chuangpishit et al. 2020b)	• polynomial gap

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Chuangpishit et al. (2020b) [lower bound]: evacuation time 3.118. *Bottleneck:* unknown exit. *Holds when:* two robots, face-to-face.
- Chuangpishit et al. (2020b) [current best]: evacuation time 3.4645.

Notes. The lower bound is $1 + 3\sqrt{2}/2 \approx 3.121$ in Theorem 4.1 (the abstract states 3.118). Three and four robots evacuate in about 3.178 and 2.664 (Chuangpishit et al. 2020b).

20.2.28 m rays, $k = m$ wireless robots at the junction

Adds + rays, + wireless, + no faults, + equal speeds VERIFIED

With one wireless robot per ray, starting at the junction, the competitive ratio is exactly 3.

Assumed: domain: m rays from a junction (a star); communication: wireless: instant communication at any distance; faults: no faults; speeds: all robots have the same unit speed. **Not assumed:** number of robots: any number k of robots; goal: every robot must reach the exit; exits: one exit; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Evacuation time	3 (Brandt et al. 2020) Exactly 3.	3 (Brandt et al. 2020)	• tight

Results.

- Brandt et al. (2020) [lower bound]: evacuation time 3. *Bottleneck:* unknown exit. *Holds when:* Observation 2; every $2 \le k \le m$.
- Brandt et al. (2020) [current best]: evacuation time 3. Introductory and current best (Observation 1): each robot explores its own ray.

Open questions.

Q1. Starting off the junction with $k = m$: close $2 + \sqrt{7/3} \approx 3.528$ against 4 (Brandt et al. 2020).

Q2. Fewer robots than rays ($1 < k < m$): the bounds $3 + 2\lfloor (m-1)/k \rfloor$ and $1 + 2\frac{m-1}{k}(1 + \frac{k}{m-1})^{1+(m-1)/k}$ differ (Brandt et al. 2020).

Notes. Checked against the full text (conference version).

20.3 Chauffeuring a crashed robot

An active robot must find a crashed robot inside a disk and carry it, at reduced speed, to one of e exits on the boundary.

First posed in 2019 (Pattanayak et al. 2019a). ALGOSENSORS 2019.

Problem statement and measures

A unit disk has $e \ge 1$ exits at unknown points of its boundary. An active robot starts at the centre and moves at speed v; a crashed robot lies at an unknown point inside the disk. The active robot must find it and carry it, at speed $v' < v$, to an exit; it learns the exits' positions once it reaches the boundary. **Ratio**

Claude (Anthropic) was used in cataloguing the problems.

is the competitive ratio of the **evacuation time** (the time until every robot has reached an exit) against the optimum.

Measures: **Competitive ratio**, evacuation time divided by the optimal time.



(a) Start: two robots at the centre, exit unknown

(b) Final: carried to the exit at speed v' = v/2

Figure 20.3: Chauffeuring: an active robot must find a crashed robot inside a unit disk and carry it, at a lower speed $v' < v$, to an exit on the boundary (Pattanayak et al. 2019a). The animation runs the simplest algorithm in the setting of the arXiv version: two robots walk together from the centre to the boundary and along it; when one crashes (at time w), the other picks it up on the spot and carries it along the same route at speed $v' = v/2$ (red) until it finds the exit, within time $w + (1 + 2\pi - w)/v'$.

A first look

A rescue robot in a round room must carry a broken robot to a door; carrying it slows the rescuer down. In chauffeuring the room is a unit **disk**, the points at distance at most 1 from its centre; the edge of the room is the **boundary**, the circle of radius 1; and the doors are one or more **exits** at unknown points of the boundary. The rescuer is the **active robot**: it starts at the centre and moves at speed v. The broken robot is a **crashed robot**, one that has stopped moving and communicating; it lies somewhere inside the disk. The active robot must **carry** the crashed robot to an exit, that is, move together with it, at the slower speed $v' < v$. Performance is the **competitive ratio**: the **evacuation time** (the time until every robot has reached an exit) divided by the best time possible if everything were known. The catalogue's bounds for this formulation (competitive ratio at most $4 + 2\pi$ for any number of exits) were supplied by the catalogue owner and have not been checked against a published text; the ideas below come from the setting of the arXiv version of (Pattanayak et al. 2019a), where two robots start together at the centre, one exit is hidden on the boundary, and one robot may crash at some time w.

1. Algorithm: Move together, so nobody is far from the crash. (Pattanayak et al. 2019a) (One exit, p. 381) Take speed $v = 1$; carrying a robot one unit of distance takes time $1/v'$.

1. Both robots walk together from the centre to the boundary.
2. They walk together along the boundary until they find the exit, then leave.
3. If one crashes on the way, the other picks it up on the spot and continues the same route, carrying it at speed v'.

Why it works and what it costs. Without a crash the robots reach the boundary in time 1 and walk at most the whole boundary, of length 2π, so they leave by time $1 + 2\pi$. If one crashes at time w, the rest of the route, of length at most $1 + 2\pi - w$, is walked at the slow speed, so the time is at most $w + (1 + 2\pi - w)/v'$ (Pattanayak et al. 2019a). It does not use the second robot to search elsewhere; (Pattanayak et al. 2019a) give better algorithms in which the robots search in opposite directions, as in two-robot evacuation.

Claude (Anthropic) was used in cataloguing the problems.

Why it matters. It is the baseline every chauffeuring algorithm must beat, and it shows the trade-off: staying together makes a crash cheap but makes the search slow.

2. Lower bound: Explore the whole boundary and carry from the centre. (Pattanayak et al. 2019a) (One exit, p. 381) Let one robot crash immediately, at the centre. The active robot alone must search the boundary, and the **adversary**, an opponent who chooses the exit position, keeps the exit at a point the active robot has not yet visited, so it must walk the whole boundary, of length 2π. In addition, the crashed robot has to be carried at least the radius, from the centre to the boundary, which takes time at least $1/v'$.

Why it holds. Carrying from the centre to the boundary and walking around the boundary are separate stretches of time, so every algorithm needs at least $2\pi + 1/v'$ in this case (Pattanayak et al. 2019a) (Theorem 1 there, which writes the slowdown as a factor). Compared with the fault-free optimum $1 + 2\pi/3 + \sqrt{3} \approx 4.826$ (Czyzowicz et al. 2014), a crash can cost a lot.

Why it matters. It isolates the two unavoidable costs, searching alone and carrying slowly, that every chauffeuring bound balances.

What the community started looking at

Chauffeuring asked how the cost of evacuation changes when a robot **must be carried** rather than merely informed, and how the number of exits affects the competitive ratio (Pattanayak et al. 2019a).

Relations to other problems

- **Variant of Evacuation.** Evacuation where one robot has crashed inside the disk: the active robot must first find it, then carry it, more slowly, to an exit.

Assumptions specific to this problem

Exits

Pattanayak, Ramesh and Mandal introduced chauffeuring at ALGOSENSORS 2019: a crashed robot "stops moving and sending messages altogether", and instead of abandoning it the non-faulty robot carries it to the exit at a reduced speed (Pattanayak et al. 2019a). That paper has two robots, wireless communication and a single exit on the boundary of the disk (Pattanayak et al. 2019a). Two exits at unknown boundary points come from the two-exit evacuation problem of Pattanayak et al. (Pattanayak et al. 2018). The catalogue's statement with $e \geq 1$ exits, and its bounds for one, two and any number of exits, were supplied by the catalogue owner; they have not been checked against a published text.

Level	Value	Formal definition
0	e **exits**	$e \geq 1$ exits at unknown boundary points.
1	**one exit**	Exactly one exit.
1	**two exits**	Exactly two exits.

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Competitive ratio) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Exits → Communication → Faults → Speeds → Randomness.

e **exits** STANDARD SETTING — ● p. 381

+ one exit
One exit — ● p. 381

+ two exits
Two exits — ● p. 382

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Competitive ratio	p.
e exits STANDARD SETTING	● unknown $4 + 2\pi$ vs —	381
One exit	● unknown 5 vs —	381
Two exits	● unknown 6.6 vs —	382

20.3.1 *e* exits

the root: no assumptions — STANDARD SETTING VERIFIED

A constant competitive ratio, at most $4 + 2\pi \approx 10.28$*, for any number of exits.*

Assumed: nothing. **Not assumed:** exits: $e \geq 1$ exits; communication: face-to-face: robots exchange information only when they meet; faults: up to f Byzantine robots (may lie or stay silent); speeds: robots may have different maximum speeds; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Competitive ratio	none known No lower bound recorded.	$4 + 2\pi$ (Pattanayak et al. 2019a)	● unknown

Results.

- Pattanayak et al. (2019a) [current best]: competitive ratio $4 + 2\pi$. Introductory and current best.

Notes. Results supplied by the catalogue owner (a co-author).

20.3.2 One exit

Adds + one exit **to** *e* exits (p. 381) — VERIFIED

Competitive ratio at most 5 with one exit.

Claude (Anthropic) was used in cataloguing the problems.

Assumed: exits: one exit. **Not assumed:** communication: face-to-face: robots exchange information only when they meet; faults: up to f Byzantine robots (may lie or stay silent); speeds: robots may have different maximum speeds; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Competitive ratio	none known No lower bound recorded.	5 (Pattanayak et al. 2019a)	• unknown

Results.

- Pattanayak et al. (2019a) [current best]: competitive ratio 5.

20.3.3 Two exits

Adds + two exits **to** *e* exits (p. 381) VERIFIED

Competitive ratio at most 6.6 with two exits.

Assumed: exits: two exits. **Not assumed:** communication: face-to-face: robots exchange information only when they meet; faults: up to f Byzantine robots (may lie or stay silent); speeds: robots may have different maximum speeds; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Competitive ratio	none known No lower bound recorded.	6.6 (Pattanayak et al. 2019a)	• unknown

Results.

- Pattanayak et al. (2019a) [current best]: competitive ratio 6.6.

20.4 Patrolling

Robots move forever so that no point of a fence (a segment, a circle, or the edges of a graph) stays unvisited for long.

First posed in 2011 (Czyzowicz et al. 2011). Boundary patrolling by agents with distinct speeds (ESA 2011).

Problem statement and measures

k robots with maximum speeds $v_1, \dots, v_k$ move forever along a **fence**: a segment, a circle, or the edges of a graph. The **idle time** is the largest time interval during which some point is not visited. The aim is to minimise it, or, equivalently, to find the longest fence that can be patrolled with idle time 1.

Measures: **Idle time**, worst idle time, or the longest fence patrollable with idle time 1 (stated per variant).

A first look

Security guards with different walking speeds must watch a long straight fence so that no point is left unvisited for too long. In fence patrolling the guards are k **robots** with maximum speeds $v_1, \dots, v_k$ that move forever along the **fence**, here a segment; a point is **visited** when some robot is at it. The **idle**

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: piece i has length $v_i/2$

(b) Final: back after time 1; every point visited again

Figure 20.4: Fence patrolling: robots with maximum speeds $v_1, \dots, v_k$ move forever along a segment so as to minimise the idle time, the longest interval during which some point is not visited. The animation runs the partition strategy (Czyzowicz et al. 2011) with speeds $1, 2, 1$: robot i sweeps its own piece, of length $v_i/2$, back and forth at full speed (bars: the part swept on the way out, then on the way back), so every point is visited again within time 1 and a fence of length $(v_1 + \cdots + v_k)/2$ is patrolled; this is not always optimal (Kawamura and Kobayashi 2015).

time is the longest stretch of time during which some point of the fence is not visited by any robot. Equivalently, fixing the idle time to 1, we ask for the longest fence, of length ℓ, that the robots can patrol. Robots may pass each other, and the **schedule**, the motion of every robot over time, is planned in advance.

1. Algorithm: Partition: give each robot its own piece. (Czyzowicz et al. 2011) (Open fence, distinct speeds, p. 385) The **partition** strategy:

1. Cut the fence into k **pieces**, piece i of length $v_i/2$.
2. Robot i walks back and forth over its own piece at full speed, forever.

Why it works and what it costs. A round trip over piece i takes $2 \cdot (v_i/2)/v_i = 1$, so every point of the piece is visited at least once in every unit of time, and the fence has length $\ell = (v_1 + \cdots + v_k)/2$. On the other hand, in any unit of time robot i visits a stretch of at most v_i, so no schedule patrols more than $v_1 + \cdots + v_k$: partition is within a factor 2 of optimal. (Czyzowicz et al. 2011) proved it optimal for $k = 2$ and conjectured it optimal for all k; (Kawamura and Kobayashi 2015) confirmed $k = 3$.

Why it matters. Partition is the natural first answer, and it is used as a building block in most patrolling strategies.

2. Lower bound: Every point needs $f + 1$ visitors: a counting bound. (Czyzowicz et al. 2017b) (Graphs with faulty patrollers, p. 386) Now patrol a **graph**: **nodes** joined by **edges**, where each edge is a segment with a length; the fence is the union of the edges, of total length $|E|$. There are k robots of speed 1, of which up to f are **unreliable**: their visits do not count, and the **adversary**, an opponent who knows the schedule, picks them. Take any **window**, an interval of time of length T equal to the idle time. If some point is visited in that window by only f distinct robots or fewer, the adversary declares those robots unreliable and the point counts as not visited. So every point must be visited by at least $f + 1$ different robots in the window.

Why it holds. Together the robots walk at most kT in the window, and they must cover every point of every edge $f + 1$ times, which needs at least $(f + 1)|E|$. Hence $T \geq (f + 1)|E|/k$. (Czyzowicz et al. 2017b) match it on graphs where a **closed walk** (a route along edges that ends where it starts) uses every edge exactly once: spread the robots evenly along that walk, $|E|/k$ apart, so every point is visited by a new robot every $|E|/k$ time units; in general they use a shortest closed walk through all edges and show that finding the optimum is NP-hard.

Why it matters. Counting total work against what must be covered is the simplest lower-bound technique in patrolling, and it is tight here.

Claude (Anthropic) was used in cataloguing the problems.

3. Algorithm: Six robots beat the partition. (Kawamura and Kobayashi 2015) (Open fence, distinct speeds, p. 385) Take six robots with speeds $1, 1, 1, 1, 7/3, 1/2$. Partition patrols a fence of length $(1 + 1 + 1 + 1 + 7/3 + 1/2)/2 = 41/12$. (Kawamura and Kobayashi 2015) patrol a longer fence, of length $7/2$:

1. The four robots of speed 1 walk end to end, covering most of the fence in a staggered pattern.
2. The fast robot covers most of what they leave uncovered.
3. The slowest robot fills the last small gaps.

Why it works and what it costs. Since $7/2 = 42/12 > 41/12$, the conjecture that partition is always optimal is false; adding further robots shows it fails for every $k \geq 6$. Later schedules patrol fences up to $4/3$ times longer than partition (Kawamura and Soejima 2020), and on a circle simple strategies can be beaten by a factor $21/20$. The optimal schedule for general speeds is unknown.

Why it matters. It shows that sharing pieces of the fence can beat dividing it, which is why even this one-dimensional problem is still open.

What the community started looking at

Patrolling asked first for **optimal strategies** and whether the simple ones, partitioning the fence or cycling around it, are optimal (Czyzowicz et al. 2011). They are not in general (Kawamura and Kobayashi 2015) (Kawamura and Soejima 2020); faults later changed the question to the idle time a graph admits when up to f patrollers are unreliable (Czyzowicz et al. 2017b).

Assumptions specific to this problem

Fence

Patrolling asks agents to visit every point of a terrain often enough; the measure is the idle time, the longest time a point is left unvisited (Kawamura and Kobayashi 2015). Czyzowicz, Gąsieniec, Kosowski and Kranakis proposed the open fence (a segment) and the closed fence (a circle) for agents with distinct speeds at ESA 2011, and conjectured that simple strategies are optimal on both (Czyzowicz et al. 2011) (as reported by Kawamura and Soejima 2020). Kawamura and Kobayashi disproved the segment conjecture (Kawamura and Kobayashi 2015), and Kawamura and Soejima showed that optimal schedules can beat the simple strategies by factors $4/3$ on the segment and $21/20$ on the circle (Kawamura and Soejima 2020). They note that "finding an optimal strategy is not at all straightforward, even when the terrain is as simple as it could be" (Kawamura and Soejima 2020). Graphs, the most general domain, were studied by Czyzowicz et al. with unreliable robots: the domain is "represented by all points belonging to the graph edges", and edges are rectifiable curves whose lengths are their weights (Czyzowicz et al. 2017b).

Level	Value	Formal definition
0	**graph**	Every point of every edge of a graph G must be visited; robots move along edges.
1	**segment**	An open fence: the segment $[0, \ell]$; robots may reverse direction anywhere.
1	**circle**	A closed fence: a circle of length ℓ.

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Idle time) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Fence → Faults → Speeds → Communication → Randomness.

No assumptions

+ segment, + no faults
Open fence, distinct speeds STANDARD SETTING — ● p. 385

+ circle, + no faults
Closed fence, distinct speeds — ● p. 386

graph, + crash faults, + equal speeds
Graphs with faulty patrollers — ● p. 386

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Idle time	p.
Open fence, distinct speeds STANDARD SETTING	● unknown $\frac{4}{3}\times$ partition vs *partition is not optimal*	385
Closed fence, distinct speeds	● unknown $\frac{21}{20}\times$ simple vs —	386
Graphs with faulty patrollers	● tight $(f+1)\,\mathrm{CPT}(G)/k$ vs $(f+1)\lvert E\rvert/k$	386

20.4.1 Open fence, distinct speeds

Adds + segment, + no faults — STANDARD SETTING VERIFIED

Partitioning the segment among the agents is optimal for 2 and 3 agents but not in general; optimal schedules can be up to 4/3 better.

Assumed: fence: a segment (open fence); faults: no faults. **Not assumed:** speeds: robots may have different maximum speeds; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Idle time	*partition is not optimal* (Kawamura and Kobayashi 2015) The best ratio between optimal schedules and partition is not known.	$\frac{4}{3}\times$ partition (Kawamura and Soejima 2020)	● unknown

Results.

- Kawamura and Kobayashi (2015) [lower bound]: idle time *partition is not optimal*. *Bottleneck:* distinct speeds. *Holds when:* counterexample with $k = 6$ agents.
- Czyzowicz et al. (2011) [introductory]: idle time partition optimal for $k = 2$. Introduces patrolling by agents with distinct speeds; shows the partition strategy optimal for $k = 2$ and conjectures it optimal for every k, as stated by (Kawamura and Kobayashi 2015), who disproved the conjecture.
- Kawamura and Kobayashi (2015) [also]: idle time partition optimal for $k = 2, 3$.

Claude (Anthropic) was used in cataloguing the problems.

- Kawamura and Soejima (2020) [current best]: idle time $\frac{4}{3}\times$ partition. Schedules that patrol a segment 4/3 times longer than partition.

Open questions.

Q1. How much better than partition can an optimal schedule be on a segment, and is the problem of computing an optimal schedule hard?

20.4.2 Closed fence, distinct speeds

Adds + circle, + no faults VERIFIED

On a circle, optimal schedules can beat the simple strategies by a factor 21/20.

Assumed: fence: a circle (closed fence); faults: no faults. **Not assumed:** speeds: robots may have different maximum speeds; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Measure	Lower bound	Best upper bound	Status
Idle time	none known —	$\frac{21}{20}\times$ simple (Kawamura and Soejima 2020)	• unknown

Results.

- Kawamura and Soejima (2020) [current best]: idle time $\frac{21}{20}\times$ simple.

20.4.3 Graphs with faulty patrollers

Adds graph, + crash faults, + equal speeds VERIFIED

With at most f unreliable robots among k, the optimal idle time lies between $(f+1)|E|/k$ and $(f+1)\,\mathrm{CPT}(G)/k$, which coincide on Eulerian graphs; computing it is NP-hard.

Assumed: faults: up to f crash-faulty robots (fail to report the target); speeds: all robots have the same unit speed. **Not assumed:** fence: the edges of a graph; communication: face-to-face: robots exchange information only when they meet; randomness: deterministic.

Statement for this setting. k unit-speed robots move perpetually along the edges of a connected graph G whose edges are curves of given lengths; $|E|$ is the total length. Knowing the strategy, an adversary chooses a set of f faulty robots, whose visits do not count, a point p and a time t. The idle time of a strategy is the supremum, over these choices, of the length of an interval starting at t during which no reliable robot visits p; the goal is to minimise it.

Measure	Lower bound	Best upper bound	Status
Idle time	$(f+1)\lvert E\rvert/k$ (Czyzowicz et al. 2017b); *NP-hard* (Czyzowicz et al. 2017b) Exact on Eulerian graphs; within a factor $\mathrm{CPT}(G)/\lvert E\rvert \le 2$ in general.	$(f+1)\,\mathrm{CPT}(G)/k$ (Czyzowicz et al. 2017b)	• tight

Results.

- Czyzowicz et al. (2017b) [lower bound]: idle time $(f+1)|E|/k$. *Bottleneck:* faulty reports. *Holds when:* any connected graph, $k \ge 2$, $f \le k-1$; every point must be passed by at least $f+1$ robots within each idle window, else the adversary makes them all faulty. Theorem 3 (lower bound, via Lemma 1).

Claude (Anthropic) was used in cataloguing the problems.

- Czyzowicz et al. (2017b) [lower bound]: idle time *NP-hard. Bottleneck:* combinatorial hardness. *Holds when:* $k = 3$ robots, at most one faulty; graphs with unit-length edges; reduction from 3-edge-colouring of cubic graphs. Theorem 4: deciding whether the idle time equals $\frac{2}{3}|E|$ is NP-hard.
- Czyzowicz et al. (2017b) [current best]: idle time $(f+1)\,\mathrm{CPT}(G)/k$. Introductory and current best: robots equally spaced along a Chinese Postman Tour (Theorem 3); equals $(f+1)|E|/k$ on Eulerian graphs (Corollary 1). For a segment with $f < k/2 - 1$ the paper gives a three-group strategy that is optimal for odd f and almost optimal for even f (Theorems 1 and 2).

Open questions.

Q1. Is the decision version of fault-tolerant patrolling in NP? With k part of the input and integer edge lengths it is in PSPACE (Czyzowicz et al. 2017b).

Q2. Optimal fault-tolerant patrolling of a segment when $f \geq k/2 - 1$, and for even f (Czyzowicz et al. 2017b).

Q3. Approximation algorithms for the optimal idle time on general graphs (Czyzowicz et al. 2017b).

Notes. Faulty here means a robot whose visits do not count; the adversary chooses the faulty set knowing the strategy. $\mathrm{CPT}(G)$ is the length of a Chinese Postman Tour of G. Kotzig graphs (cubic graphs whose three perfect matchings pairwise form Hamiltonian cycles) attain the minimum idle time $\frac{2}{3}|E|$, the number of their vertices, for $k = 3$, $f = 1$ (Corollary 2). The problem was posed at a MITACS workshop in 2012. Checked against the full text of (Czyzowicz et al. 2017b).

Claude (Anthropic) was used in cataloguing the problems.

Part VI

Programmable matter

The model and its assumption axes are in Chapter 5.

Claude (Anthropic) was used in cataloguing the problems.

Chapter 21

Self-organising particle systems

Leader election, shape formation, coating, scattering, stochastic tasks (compression, separation, alignment, bridging), energy distribution, hybrid and centralised reconfiguration.

Problems in this family:

- Leader election in programmable matter
- Shape formation in programmable matter
- Coating
- Compression
- Separation of coloured particles — generalisation of Compression
- Alignment — generalisation of Compression
- Bridging — generalisation of Compression
- Convex hull formation
- Scattering of particles
- Energy distribution
- Shape formation by hybrid programmable matter
- Centralised transformations of modular robots

21.1 Leader election in programmable matter

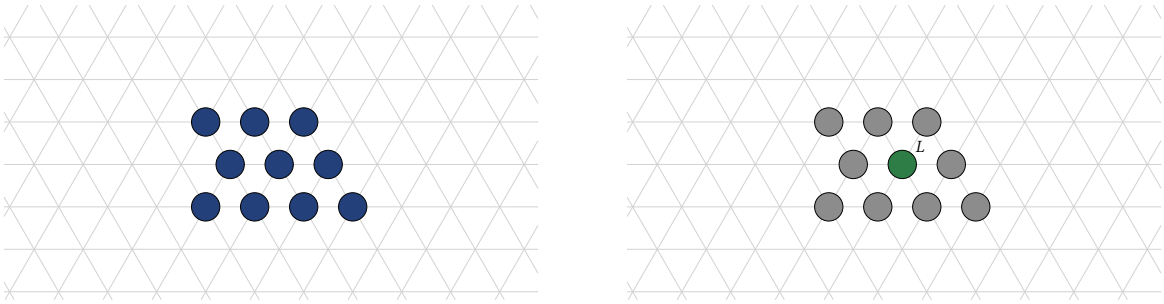
Anonymous constant-memory particles must elect exactly one leader, which most shape-formation algorithms need first.

First posed in 2015 (Derakhshandeh et al. 2015). Introduced with shape formation at DNA 21 (2015).

Problem statement and measures

A connected **configuration** (the set of nodes occupied by particles) of n anonymous particles (initially contracted) must reach a state in which exactly one particle is the leader and all others know they are not. **Time** is in asynchronous rounds; L_{out} is the length of the outer boundary and D the diameter of the configuration.

Measures: **Time**, asynchronous rounds; **Moves**, total expansions and contractions (work).



(a) Start: every particle is eligible

(b) Final: one candidate left, the leader

Figure 21.1: Leader election by erosion on a configuration without holes: no particle moves. An eligible particle (blue) whose eligible neighbours are few and consecutive (none, one, two that are neighbours of each other, three in a row) is a corner and drops out (grey); corners drop out round after round, peeling the configuration from its boundary, until a single candidate remains and becomes the leader (Di Luna et al. 2020c) (D'Angelo et al. 2020). Configurations with holes need other algorithms (Derakhshandeh et al. 2015).

A first look

Imagine a swarm of identical tiny machines, packed together on a floor tiled with triangles, that must pick one boss without any name tags. Each machine is a **particle**. It occupies one **node** of the **triangular lattice**, the infinite tiling of the plane by equilateral triangles whose corners are the nodes; particles on adjacent nodes are **neighbours**. A particle remembers only a few bits, and can read and write the memory of its neighbours. It moves in two steps: an **expansion** into an empty adjacent node, after which it briefly occupies both nodes, then a **contraction** into one of the two; so one expansion and one contraction move it by one node. In the standard version there are n particles whose nodes form one connected **configuration**, possibly with **holes** (empty regions enclosed by particles); all particles agree on what clockwise means (**chirality**), but not on a common north. An adversary, a scheduler that we assume chooses the worst order, decides when each particle acts; each time a particle acts is an **activation**, and a **round** is a stretch of time in which every particle has been activated at least once. The goal: exactly one particle ends in state **leader**, every other particle knows it is not, and we count the rounds this takes.

1. Algorithm: Erosion: peel the configuration from the outside. (D'Angelo et al. 2020) (Silent oblivious particles (SILBOT), p. 400) This works on configurations **without holes**. It is the erosion of (Di Luna et al. 2020c), which (D'Angelo et al. 2020) showed can be run even by particles with no memory and no messages. No particle changes its node (in the memoryless version, expansions serve as signals).

1. Every particle starts as *eligible*.
2. An eligible particle is a *corner* if its eligible neighbours are few and sit next to each other: none, one, two that are neighbours of each other, or three in a row.
3. A corner drops out (becomes ineligible). Corners keep dropping out round after round, so the configuration is eaten from its outer **boundary** (the particles next to the empty outside; a hole has a boundary too) inwards, like a coastline eroded by the sea.
4. A few special cases stop the peeling: a lone eligible particle, two eligible particles that are each other's only eligible neighbour, or a triangle of three, become **candidates** (for leader) instead of dropping out.
5. At the end, 1, 2 or 3 candidates remain, each a neighbour of the others.

Claude (Anthropic) was used in cataloguing the problems.

Why it works and what it costs. A hole-free configuration always has a corner, and removing corners never splits the eligible particles into parts or creates a hole, so erosion never gets stuck. Each removal leaves fewer eligible particles, and (D'Angelo et al. 2020) show the process ends in $O(n)$ rounds. It cannot always leave a single leader: three particles in a triangle look exactly alike (see the next idea). It also fails with holes: in a ring of particles around a hole, each particle has two eligible neighbours that are not neighbours of each other, so nobody is a corner. The best known result elects exactly one leader, holes allowed, deterministically in $O(L_{out} + D)$ rounds, where L_{out} is the length of the outer boundary and D the diameter of the configuration (Dufoulon et al. 2021).

Why it matters. Erosion is the first step of the deterministic shape-formation algorithm and is reused whenever a few candidate leaders are good enough.

2. Impossibility: Three identical particles in a triangle. (Chalopin et al. 2026b) (Stationary, chirality, weak scheduler, p. 398) Take three particles forming a small triangle, and suppose particles never move and the scheduler may activate all three at the same moment. Each particle sees the same picture: two neighbours, one to its left and one to its right (they have chirality, so 'left' means the same to all). They start in the same state, so in their first activation they compute the same thing and write the same thing. After that activation they are again in identical states with identical views, and so on forever. Whatever a deterministic algorithm (one that flips no coins) does, either all three become leader or none does (Chalopin et al. 2026b).

Why it holds. The argument only uses symmetry: an algorithm cannot tell apart particles whose surroundings are rotated copies of each other. The same reasoning shows that a configuration with six-fold symmetry may end with six leaders, which is why stationary particles with chirality elect up to six leaders (Bazzi and Briones 2019). To get exactly one leader, papers add something that breaks the symmetry: coin flips (Derakhshandeh et al. 2015), a scheduler that activates one particle at a time (Briones et al. 2023), or a common direction (Chalopin et al. 2026b).

Why it matters. Symmetry is the basic obstacle in all of programmable matter; it explains why results say 'at most three' or 'at most six' leaders.

3. Algorithm: Coin flips along the boundaries. (Derakhshandeh et al. 2015) (Randomized, with chirality, p. 396) Only particles on a boundary (the outer boundary, or the boundary of a hole) take part. Each boundary closes on itself, and with chirality everyone walks it in the same direction.

1. Every boundary particle starts as a candidate. Between two consecutive candidates lies a *segment* of non-candidates.
2. Compare segments: a candidate whose segment ahead is shorter than the segment behind it gives up (particles compare lengths by passing tokens, one per particle).
3. If all segments are equal, every candidate flips a coin. On heads it passes its candidacy forward to the next candidate's spot; when it does and the next candidate keeps its own, two candidacies land on one spot and merge.
4. A candidate that is alone on its boundary checks whether it is the outer boundary (by adding up the turns along it; the outer boundary turns one full circle one way, the boundary of a hole the other way). Alone on the outer boundary: it is the leader. Alone on the boundary of a hole: it gives up.

Why it works and what it costs. There is always at least one candidate on the outer boundary, and while there are two or more, each phase removes one with probability at least $1/4$, so the number of candidates keeps shrinking. Holes are no problem, because the boundary of a hole eventually loses all its candidates. The expected time is $O(L_{max})$ rounds, L_{max} the length of the longest boundary (Derakhshandeh et al.

Claude (Anthropic) was used in cataloguing the problems.

2015); this was improved to $O(n)$ rounds with high probability (Daymude et al. 2017), and later matched without coins (Dufoulon et al. 2021).

Why it matters. Working on boundaries instead of the whole configuration is the idea behind the fast algorithms that followed.

What the community started looking at

Leader election was the first task studied, because shape formation needs a leader (Derakhshandeh et al. 2015). The first question was **feasibility with constant memory**, then **rounds**: randomized algorithms came first (Derakhshandeh et al. 2015) (Daymude et al. 2017), then deterministic ones without chirality (Emek et al. 2019) and in linear time (Dufoulon et al. 2021). Later work weakened the model in three directions: **stationary** particles (Bazzi and Briones 2019) (Chalopin et al. 2026b), **asynchronous** adversaries (Briones et al. 2023), and **silent, oblivious** particles (D'Angelo et al. 2020); self-stabilising election with constant memory followed (Chalopin et al. 2025) (Chalopin et al. 2026a).

Relations to other problems

- **Related: Leader election in the plane.** Both elect a leader among anonymous entities from geometry alone; particles on the lattice can also move and talk to neighbours, robots in the plane only see (Dufoulon et al. 2021).

Assumptions specific to this problem

Movement

The early leader-election algorithms for amoebots only exchanged messages, and "did not use the movement ability as an aid to the computation" (Dufoulon et al. 2021). Emek et al. showed that movement lets particles elect a unique leader deterministically with no assumption on chirality or holes; the main idea is "the usage of the ability of the amoebots to move, which previous leader election algorithms have not used" (Emek et al. 2019). The stationary setting stayed interesting because particles may be unable to move, "so as to maintain a configuration or due to the presence of obstacles" (Chalopin et al. 2026b). Bazzi and Briones studied stationary deterministic election (Bazzi and Briones 2019), where symmetric configurations can force up to six leaders (as reported by Dufoulon et al. 2021). Chalopin, Das and Kokkou showed that one common direction suffices for a unique stationary leader (Chalopin et al. 2026b).

Level	Value	Formal definition
0	**stationary**	No particle ever expands or contracts; the configuration is fixed and only states change.
1	**may move**	Particles may expand and contract during the election.

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Moves) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Memory → Scheduler → Orientation → Movement → Randomness → Initial shape → Connectivity during execution.



What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Claude (Anthropic) was used in cataloguing the problems.

Setting	Time	Moves	p.
Deterministic, with chirality STANDARD SETTING	● unknown $O(L_{out} + D)$ vs —	● unknown — vs —	395
Randomized, with chirality	● unknown $O(n)$ vs —	● unknown — vs —	396
Stationary, chirality, no holes	● unknown $O(n)$ vs —	● tight — vs —	396
Deterministic, no orientation	● polynomial gap $O(L_{out}n^2)$ vs —	● unknown — vs —	397
Stationary, common direction, no chirality	● unknown $O(n^3)$ vs *explicit termination impossible*	● tight — vs —	397
Stationary, fair scheduler, no orientation, no holes	● unknown *finite* vs —	● tight — vs —	398
Stationary, chirality, weak scheduler	● unknown $O(n^2)$ vs —	● tight — vs —	398
Stationary, asynchronous, no orientation, no holes	● unknown *finite* vs —	● tight — vs —	399
Stationary, sequential, no orientation, no holes	● unknown $O(n)$ vs —	● tight — vs —	399
SILBOT with holes	● impossible	● unknown — vs —	400
Silent oblivious particles (SILBOT)	● unknown $O(n)$ vs —	● unknown — vs —	400
Oblivious, common compass, holes allowed	● unknown *finite* vs —	● unknown — vs —	401

21.1.1 Deterministic, with chirality

Adds deterministic, + may disconnect

STANDARD SETTING VERIFIED

Deterministic leader election in $O(L_{out} + D)$ rounds, as fast as the best randomized algorithm, holes allowed.

Assumed: memory: constant memory and local communication; scheduler: strong: atomic activations under a fair adversary; orientation: common chirality; movement: particles may move; connectivity during execution: the system may disconnect temporarily. **Not assumed:** randomness: deterministic; initial shape: connected, holes allowed.

Measure	Lower bound	Best upper bound	Status
Time	none known Only an informal $\Omega(D)$ remark; no proven lower bound.	$O(L_{out} + D)$ (Dufoulon et al. 2021)	● unknown
Moves	none known —	—	● unknown

Results.

- Dufoulon et al. (2021) [current best]: time $O(L_{out} + D)$. Introductory for linear time and current best; $O(D_A)$, where D_A is the diameter of the area enclosed by the particles (holes included), if particles know whether their boundary is the outer one. The system may temporarily disconnect.

Open questions.

Q1. Fast deterministic leader election under weaker (non-atomic) schedulers, and without letting the system disconnect (Dufoulon et al. 2021).

Claude (Anthropic) was used in cataloguing the problems.

21.1.2 Randomized, with chirality

Adds + randomized VERIFIED

Randomized leader election in a linear number of rounds with high probability.

Assumed: memory: constant memory and local communication; scheduler: strong: atomic activations under a fair adversary; orientation: common chirality; movement: particles may move; randomness: randomized. **Not assumed:** initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known No proven lower bound.	$O(n)$ (Daymude et al. 2017)	● unknown
Moves	none known —	—	● unknown

Results.

- Derakhshandeh et al. (2015) [introductory]: time $O(L_{max})$. Expected rounds; L_{max} is the longest boundary.
- Daymude et al. (2017) [current best]: time $O(n)$. With high probability; restated as $O(L_{out} + D)$ by (Dufoulon et al. 2021).

Notes. In the general amoebot model on arbitrary graphs leader election is impossible, by reduction to anonymous rings (Derakhshandeh et al. 2015).

21.1.3 Stationary, chirality, no holes

Adds stationary, + no holes VERIFIED

On hole-free configurations with chirality, stationary constant-memory particles elect a leader deterministically in $O(n)$ rounds.

Assumed: memory: constant memory and local communication; scheduler: strong: atomic activations under a fair adversary; orientation: common chirality; initial shape: simply connected (no holes). **Not assumed:** movement: stationary: particles never move; randomness: deterministic; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound recorded.	$O(n)$ (Gastineau et al. 2019)	● unknown
Moves	none known No moves.	—	● tight

Results.

- Gastineau et al. (2019) [current best]: time $O(n)$. Introductory and current best: $O(r + m_{tree})$ rounds, fewer than $2n$ (Proposition 1). Also computes k-local identifiers; works on the square, triangular and king grids.

Notes. r and m_{tree} are parameters of the configuration defined in (Gastineau et al. 2019): r is the radius of the configuration with respect to its border particles (the least, over particles u, of the largest distance from u to a border particle), and m_{tree} is the largest height of an induced subgraph of the configuration that is a tree. The scheduler is strong, as listed by (Dufoulon et al. 2021). Checked against the full text.

Claude (Anthropic) was used in cataloguing the problems.

21.1.4 Deterministic, no orientation

Adds no orientation, + may move VERIFIED

Without chirality, a unique leader can still be elected deterministically, in $O(L_{out}n^2)$ rounds.

Assumed: memory: constant memory and local communication; scheduler: strong: atomic activations under a fair adversary; movement: particles may move. **Not assumed:** orientation: no common orientation and no chirality; randomness: deterministic; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known Far from the linear bound known with chirality.	$O(L_{out}n^2)$ (Emek et al. 2019)	• polynomial gap
Moves	none known —	—	• unknown

Results.

- Emek et al. (2019) [current best]: time $O(L_{out}n^2)$. Introductory and current best; also yields agreement on chirality.

Open questions.

Q1. Deterministic leader election without chirality in linear time.

21.1.5 Stationary, common direction, no chirality

Adds + common direction VERIFIED

With one common direction and no chirality, a unique leader is elected without moving, with obstacles allowed, but only with implicit termination.

Assumed: memory: constant memory and local communication; scheduler: strong: atomic activations under a fair adversary; orientation: a common direction, no chirality. **Not assumed:** movement: stationary: particles never move; randomness: deterministic; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	*explicit termination impossible* (Chalopin et al. 2026b) No round bound recorded.	$O(n^3)$ (Chalopin et al. 2026b)	• unknown
Moves	none known No moves.	—	• tight

Results.

- Chalopin et al. (2026b) [lower bound]: time *explicit termination impossible. Bottleneck:* symmetry breaking. *Holds when:* Theorem 1: deterministic algorithms; connected configuration with obstacles (holes); common direction (one axis), no chirality; holds even under a synchronous scheduler; implicit termination remains possible.
- Chalopin et al. (2026b) [current best]: time $O(n^3)$. Introductory and current best (Section 6); implicit termination, unique leader, deterministic, stationary.

Notes. With chirality but no common direction, explicit termination is always possible but the number of leaders depends on the symmetry of the initial configuration (Chalopin et al. 2026b). Here "common direction" means agreement on one axis only. Checked against the full text.

Claude (Anthropic) was used in cataloguing the problems.

21.1.6 Stationary, fair scheduler, no orientation, no holes

Adds no orientation, stationary, + no holes VERIFIED

Stationary constant-memory particles with no orientation and no chirality elect a unique leader from any initial memory: silent self-stabilising election on simply connected systems.

Assumed: memory: constant memory and local communication; scheduler: strong: atomic activations under a fair adversary; initial shape: simply connected (no holes). **Not assumed:** orientation: no common orientation and no chirality; movement: stationary: particles never move; randomness: deterministic; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Chalopin et al. 2025)	● unknown
Moves	none known No moves.	—	● tight

Results.

- Chalopin et al. (2025) [current best]: time *finite*. *Assumes:* Gouda-fair scheduler; self-stabilising: any initial memory. Introductory and current best. The algorithm fails under an unfair scheduler (Section 6). Constant memory suffices although silent self-stabilising election in general networks needs $\Omega(\log n)$ bits, even on rings.

Notes. Self-stabilisation is a stronger requirement than the problem statement and has no axis here. Checked against the full text.

21.1.7 Stationary, chirality, weak scheduler

Adds weak scheduler, + chirality DRAFT

Without moving, particles with chirality elect at most six leaders in $O(n^2)$ rounds, holes allowed.

Assumed: memory: constant memory and local communication; orientation: common chirality. **Not assumed:** scheduler: weak: non-atomic activations, an action may interleave with others; movement: stationary: particles never move; randomness: deterministic; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Statement for this setting. Up to six leaders may be elected.

Measure	Lower bound	Best upper bound	Status
Time	none known —	$O(n^2)$ (Bazzi and Briones 2019)	● unknown
Moves	none known No moves.	—	● tight

Results.

- Bazzi and Briones (2019) [current best]: time $O(n^2)$. Introductory and current best; at most six leaders.

Open questions.

Q1. Is a unique leader ever impossible for stationary particles with chirality? Dufoulon et al. say this was shown in (Bazzi and Briones 2019), at least for a weak scheduler (Dufoulon et al. 2021); the theorem was not read.

Notes. Bounds are from the table of Dufoulon, Kutten and Moses (Dufoulon et al. 2021); the abstract of (Bazzi and Briones 2019) was not read.

Claude (Anthropic) was used in cataloguing the problems.

21.1.8 Stationary, asynchronous, no orientation, no holes

Adds weak scheduler, no orientation, + no holes VERIFIED

Stationary particles with no common orientation elect a leader in connected, contractible 2D or 3D systems under an unfair asynchronous adversary; the running time there is not analysed.

Assumed: memory: constant memory and local communication; initial shape: simply connected (no holes). **Not assumed:** scheduler: weak: non-atomic activations, an action may interleave with others; orientation: no common orientation and no chirality; movement: stationary: particles never move; randomness: deterministic; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known Runtime not analysed under the asynchronous adversary.	*finite* (Briones et al. 2023)	● unknown
Moves	none known No moves.	—	● tight

Results.

- Briones et al. (2023) [current best]: time *finite*. Introductory and current best (Corollary 6): the sequential algorithm transformed by the concurrency control framework (Daymude et al. 2023); the paper says its runtime "has not yet been analyzed".

Open questions.

Q1. Deterministic leader election in $O(L_{out} + D)$ rounds under a weaker scheduler (Dufoulon et al. 2021).

Notes. Contractible means no holes, in 2D or in 3D (face-centred cubic lattice). Particles have "assorted orientations": no common direction and no chirality. Checked against the full text.

21.1.9 Stationary, sequential, no orientation, no holes

Adds + sequential, + no holes VERIFIED

Under an unfair sequential adversary, stationary particles with no common orientation elect a unique leader in $O(n)$ rounds, in 2D or 3D.

Assumed: memory: constant memory and local communication; scheduler: sequential: one particle active at a time; initial shape: simply connected (no holes). **Not assumed:** orientation: no common orientation and no chirality; movement: stationary: particles never move; randomness: deterministic; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound recorded.	$O(n)$ (Briones et al. 2023)	● unknown
Moves	none known No moves.	—	● tight

Results.

- Briones et al. (2023) [current best]: time $O(n)$. *Assumes:* unfair sequential adversary. Introductory and current best (Theorem 3 and Corollary 4).

Notes. Checked against the full text.

Claude (Anthropic) was used in cataloguing the problems.

21.1.10 SILBOT with holes

Adds oblivious, weak scheduler, no orientation, + may move VERIFIED

With holes, SILBOT particles cannot elect up to three leaders while staying connected; if they can tell a hole from the exterior, election with compaction takes $O(n^2)$ rounds.

Assumed: movement: particles may move. **Not assumed:** memory: oblivious, silent particles (SILBOT); scheduler: weak: non-atomic activations, an action may interleave with others; orientation: no common orientation and no chirality; randomness: deterministic; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Statement for this setting. Particle leader election as in the hole-free SILBOT variant (one, two or three mutually adjacent leaders), from any connected configuration of contracted particles, possibly with holes, keeping the system connected.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (D'Angelo et al. 2020)	$O(n^2)$ (D'Angelo et al. 2020)	• impossible
	Impossible in plain SILBOT; with exterior awareness $O(n^2)$ rounds and no lower bound.		
Moves	none known	—	• unknown
	Not analysed.		

Results.

- D'Angelo et al. (2020) [lower bound]: time *impossible*. *Bottleneck:* symmetry breaking. *Holds when:* a connected configuration with a hole in which groups of particles have identical 2-hop views; the adversary activates each such set simultaneously, each particle twice in a row; the system must stay connected; holds even with unlimited memory and chirality. Theorem 2: every move some particle could be instructed to make disconnects the system under some schedule.
- D'Angelo et al. (2020) [current best]: time $O(n^2)$. *Assumes:* exterior awareness: a particle tells an empty node in a hole from an exterior empty node within its view; 2-hop view. Introductory and current best: Algorithm CHLE (Theorem 3) fills the holes (compaction: the configuration becomes simply connected) while electing at most three leaders; on hole-free configurations it behaves as LESC.

Open questions.

Q1. Can SILBOT particles elect leaders from configurations with holes if the system may disconnect temporarily, without exterior awareness? The impossibility of (D'Angelo et al. 2020) assumes that connectivity is preserved.

Notes. *Compaction* (of holes) asks to turn any connected configuration into a simply connected one. Exterior awareness is motivated by local sensing (light or pressure). Checked against the full text of (D'Angelo et al. 2020).

21.1.11 Silent oblivious particles (SILBOT)

Adds + no holes **to** SILBOT with holes (p. 400) VERIFIED

Silent oblivious particles (SILBOT) elect at most three mutually adjacent leaders from hole-free configurations in $O(n)$ rounds.

Assumed: movement: particles may move; initial shape: simply connected (no holes). **Not assumed:** memory: oblivious, silent particles (SILBOT); scheduler: weak: non-atomic activations, an action may interleave with others; orientation: no common orientation and no chirality; randomness: deterministic; connectivity during execution: the system must stay connected.

Claude (Anthropic) was used in cataloguing the problems.

Statement for this setting. Particle leader election (PLE): from a connected configuration of contracted particles, terminate with one, two or three mutually adjacent contracted particles that recognise themselves as LEADER, every other particle expanded (NON-LEADER), and the system connected throughout. A single leader is not required: D'Angelo et al. note that a single particle can be elected only with randomisation or a sequential scheduler, citing (Emek et al. 2019).

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound recorded.	$O(n)$ (D'Angelo et al. 2020)	• unknown
Moves	none known Not analysed.	—	• unknown

Results.

- D'Angelo et al. (2020) [current best]: time $O(n)$. *Assumes:* 2-hop view; initial configuration deeply simply connected (e.g. all particles contracted, no holes); no chirality, deterministic. Introductory and current best: Algorithm LESC (Theorem 1), which only expands particles; an emulation of the erosion of (Di Luna et al. 2020c) with simpler rules.

Notes. SILBOT (D'Angelo et al. 2020) removes from the amoebot model the serialised scheduler (no two neighbouring particles active together), explicit communication, atomicity of actions and randomness. Particles see up to two hops, distinguish empty nodes from contracted and expanded particles, and act asynchronously; a particle that has expanded along an edge is committed and contracts into the target node at its next activation if the node is empty. A round is the shortest period in which every particle has completed an activation. The paper argues that 1-hop visibility is not enough without communication. Holes are handled in the variant without the hole-free assumption. Checked against the full text of (D'Angelo et al. 2020) (IEEE Access version; preliminary version at AAMAS 2020).

21.1.12 Oblivious, common compass, holes allowed

Adds oblivious, + sequential, + common compass, + may move VERIFIED

Oblivious particles that agree on all directions elect a unique leader in connected configurations with holes, self-stabilising, using movement, under an unfair sequential scheduler.

Assumed: scheduler: sequential: one particle active at a time; orientation: full common orientation (compass); movement: particles may move. **Not assumed:** memory: oblivious, silent particles (SILBOT); randomness: deterministic; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known —	*finite* (Chalopin et al. 2026a)	• unknown
Moves	none known —	—	• unknown

Results.

- Chalopin et al. (2026a) [current best]: time *finite*. *Assumes:* unfair sequential scheduler with atomic moves. Introductory and current best; silent self-stabilising.

Notes. The abstract says "common sense of direction"; Section 2 assumes agreement on all directions. Movement together with this orientation overcomes the impossibility of constant-memory self-stabilising election in general networks (Chalopin et al. 2026a). Checked against the full text.

Claude (Anthropic) was used in cataloguing the problems.

21.2 Shape formation in programmable matter

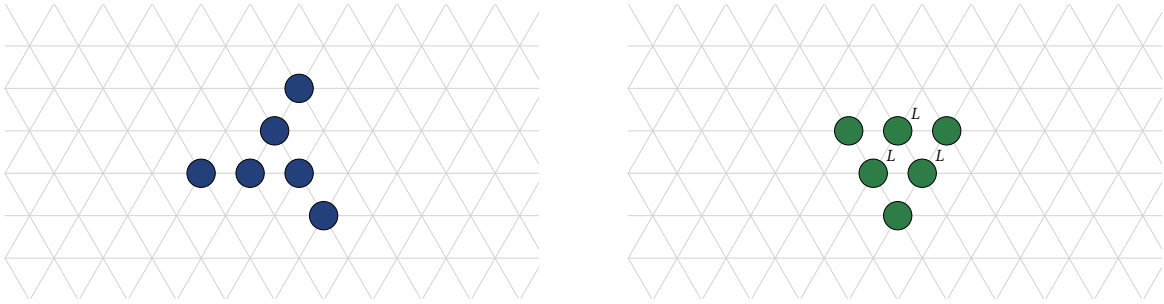
Particles must rearrange into a target shape (a line, a triangle, any shape built from triangles).

First posed in 2015 (Derakhshandeh et al. 2015). Introduced with leader election at DNA 21 (2015).

Problem statement and measures

From a connected initial **configuration** (the set of nodes occupied by particles) of n particles, the particles must reach a configuration forming a given **target shape** (at a scale depending on n). **Time** is in rounds; **moves** counts expansions and contractions.

Measures: **Time**, asynchronous rounds; **Moves**, total expansions and contractions (work).



(a) Start: a connected configuration

(b) Final: the target triangle is formed

Figure 21.2: Shape formation by the universal algorithm of (Di Luna et al. 2020c) (the problem as in Derakhshandeh et al. 2016), here on a configuration with three-fold symmetry and a triangle as target shape. Erosion (no movement) leaves three candidates, neighbours of each other (green), that cannot break the symmetry, so all three lead. Each leader picks a ray (dashed; the three rays are rotations of each other) and pulls its part into a straight segment along it; then each segment folds into its third of the triangle. A particle moves by one node with an expansion followed by a contraction.

A first look

Think of a pile of clay made of tiny identical machines that should mould itself into a triangle, a hexagon or a line. Each machine is a **particle**. It occupies one **node** of the **triangular lattice**, the infinite tiling of the plane by equilateral triangles whose corners are the nodes; particles on adjacent nodes are **neighbours**. There are n particles, and their nodes form a connected **configuration** with no holes (empty regions enclosed by particles); each particle has a few bits of memory and talks only to its neighbours. A particle moves in two steps: an **expansion** into an empty adjacent node, after which it briefly occupies both nodes, then a **contraction** into one of the two; so one expansion and one contraction move it by one node. In the standard version the particles do not agree on clockwise (they have no **chirality**) or on north, and an adversarial scheduler decides when each particle acts; each time a particle acts is an **activation**. They all know a small description of the **target shape**, and must end up forming it, scaled up so that it uses all n particles. We count **rounds** (stretches of time in which every particle is activated at least once) and **moves** (expansions and contractions).

1. Algorithm: Elect, straighten into a line, rebuild. (Di Luna et al. 2020c) (Deterministic, simply connected start, no chirality, p. 405) The universal algorithm of (Di Luna et al. 2020c) has seven phases; in plain words:

Claude (Anthropic) was used in cataloguing the problems.

1. Erosion: particles on corners of the configuration drop out one by one (no movement) until 1, 2 or 3 candidates remain, each a neighbour of the others; a candidate may become a **leader**, a particle that directs the others.
2. Each candidate becomes the root of a tree that spans part of the configuration (every particle remembers a parent).
3. The particles agree on chirality, copying it from the candidates.
4. The candidates try to break ties; if they cannot, the starting configuration was symmetric and several leaders carry on together.
5. Each leader pulls its tree into a straight line; all lines get the same length.
6. Walking along its line, each leader works out the scale of the target shape and tells every particle its future role.
7. Each line, guided by its leader, folds into its share of the target shape.

Why it works and what it costs. A line is easy to control: a leader at one end can count along it and send instructions particle by particle. The whole process takes $O(n^2)$ rounds and $O(n^2)$ moves, and the moves are optimal by the next idea. With coin flips, chirality and one particle activated at a time, shapes made of a constant number of triangles are built in $\Theta(\sqrt{n})$ rounds from a well-prepared configuration (Derakhshandeh et al. 2016).

Why it matters. Using a line as a universal halfway shape is the standard trick for shape formation, reused for tiles and modular robots too.

2. Lower bound: A compact configuration must travel far to become a line. (Di Luna et al. 2020c) (Deterministic, simply connected start, no chirality, p. 405) Start with the n particles packed into a hexagon; it fits inside a circle of radius about $\sqrt{n}$ around its centre c. Ask for a straight line. A line has at most $2r+1$ of its particles within distance r of c. Take $r = n/4$: at most about $n/2$ particles of the final line are within distance $n/4$ of c, so at least about $n/2$ particles end farther than $n/4$ from c. Each of them started within about $\sqrt{n}$ of c, so it must travel at least $n/4 - \sqrt{n}$ nodes, one node per expansion and contraction.

Why it holds. Adding up gives at least about $(n/2)(n/4) = n^2/8$ expansion-and-contraction pairs, so $\Omega(n^2)$ moves for any algorithm (Di Luna et al. 2020c). The algorithm above uses $O(n^2)$ moves, so it is optimal in moves. The argument says nothing about rounds, because many particles can move in the same round.

Why it matters. The same distance-counting argument gives the quadratic lower bounds for line formation, coating and centralised transformations.

3. Impossibility: Symmetry that cannot be broken. (Di Luna et al. 2020c) (Deterministic, simply connected start, no chirality, p. 405) Take three particles in a small triangle and ask them to form a straight line of three. The triangle looks the same after a turn by a third of a circle, and its centre is the middle of a triangular face, not a node. Each particle numbers the six directions around it; these local numbers are its **ports**. Let the scheduler always activate the three rotated copies of a particle together, and give them rotated ports. Then they see rotated copies of the same view and make rotated copies of the same move. Because the centre is not a node, two of them never try to enter the same node, so nothing breaks the tie: the configuration stays three-fold symmetric forever. A line of three is not three-fold symmetric, so it is never formed.

Why it holds. In general, a configuration that is symmetric under a turn of a half or a third of a circle around a point that is not a node can only reach configurations with the same symmetry (Di Luna et al. 2020c) (Theorem 1). The paper also proves the converse for hole-free configurations: every other target

Claude (Anthropic) was used in cataloguing the problems.

shape can be formed, so this is the only obstacle. With coin flips the obstacle disappears and any shape can be formed once n is large (Di Luna et al. 2020c).

Why it matters. It shows exactly which shapes are out of reach and why, and it explains why many algorithms elect up to three leaders instead of one.

What the community started looking at

The first shapes were lines, triangles and hexagons, built after electing a leader (Derakhshandeh et al. 2015); then any shape made of triangles (Derakhshandeh et al. 2016). The deterministic question, **which shapes are formable from which starts**, mirrors pattern formation for robots (Di Luna et al. 2020c). Later work formed arbitrary connected shapes with a constant number of particles acting as a mobile RAM (Di Luna et al. 2020b), reconfigured between arbitrary shapes in a linear number of rounds without a canonical intermediate shape (Kostitsyna et al. 2023), and asked which shapes **silent, oblivious** particles can form, starting with the line (Navarra and Piselli 2023b). Most tasks have a natural $\Omega(D)$ lower bound because particles and information move one node at a time; joint movements and reconfigurable circuits are extensions proposed to break it (Padalkin et al. 2025).

Relations to other problems

- **Related: Pattern formation.** "Shape formation is a basic distributed problem for systems of computational mobile entities. Intensively studied for systems of autonomous mobile robots, it has recently been investigated in the realm of programmable matter" (Di Luna et al. 2020c).
- **Related: Shape formation by hybrid programmable matter.** In the hybrid model the tiles are passive and one active agent moves them; in the amoebot model every particle is active.
- **Related: Centralised transformations of modular robots.** A central controller plans the moves, so the question is which mechanisms are universal and how many moves they need, not how particles coordinate.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Moves) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Memory → Scheduler → Orientation → Randomness → Initial shape → Connectivity during execution.

No assumptions

+ constant memory

Scheduler

+ strong scheduler, + no holes

Deterministic, simply connected start, no chirality STANDARD SETTING ●● p. 405

+ sequential, + chirality, + randomized, + no holes

Universal shapes, sequential, randomized ●● p. 405

oblivious, + chirality

SILBOT, chirality ●● p. 406

oblivious, + common compass, + may disconnect

SILBOT, full orientation ●● p. 406

Claude (Anthropic) was used in cataloguing the problems.

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Moves	p.
Deterministic, simply connected start, no chirality STANDARD SETTING	● unknown $O(n^2)$ vs —	● tight $O(n^2)$ vs $\Omega(n^2)$	405
Universal shapes, sequential, randomized	● tight $O(\sqrt{n})$ vs $\Omega(\sqrt{n})$	● unknown — vs —	405
SILBOT, chirality	● impossible	● impossible	406
SILBOT, full orientation	● unknown — vs —	● tight $O(n^2)$ vs $\Omega(n^2)$	406

21.2.1 Deterministic, simply connected start, no chirality

Adds + strong scheduler, + no holes STANDARD SETTING VERIFIED

Which shapes are deterministically formable from a simply connected start is characterised; $O(n^2)$ rounds and moves, and the moves are optimal.

Assumed: memory: constant memory and local communication; scheduler: strong: atomic activations under a fair adversary; initial shape: simply connected (no holes). **Not assumed:** orientation: no common orientation and no chirality; randomness: deterministic; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known No time lower bound recorded.	$O(n^2)$ (Di Luna et al. 2020c)	● unknown
Moves	$\Omega(n^2)$ (Di Luna et al. 2020c) $\Theta(n^2)$ moves.	$O(n^2)$ (Di Luna et al. 2020c)	● tight

Results.

- Di Luna et al. (2020c) [lower bound]: moves $\Omega(n^2)$. *Bottleneck:* distance. *Holds when:* some initial configurations and shapes.
- Di Luna et al. (2020c) [current best]: time $O(n^2)$, moves $O(n^2)$. Introductory for the deterministic, chirality-free setting and current best; "chirality is computationally irrelevant for shape formation"; with randomization any shape is formable for large n.

21.2.2 Universal shapes, sequential, randomized

Adds + sequential, + chirality, + randomized, + no holes VERIFIED

Any shape made of a constant number of triangles, from a well-initialised start, in $\Theta(\sqrt{n})$ rounds.

Assumed: memory: constant memory and local communication; scheduler: sequential: one particle active at a time; orientation: common chirality; randomness: randomized; initial shape: simply connected (no holes). **Not assumed:** connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(\sqrt{n})$ (Derakhshandeh et al. 2016) $\Theta(\sqrt{n})$.	$O(\sqrt{n})$ (Derakhshandeh et al. 2016)	● tight
Moves	none known —	—	● unknown

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Derakhshandeh et al. (2016) [lower bound]: time $\Omega(\sqrt{n})$. *Bottleneck:* distance. *Holds when:* when the shape differs from the initial configuration.
- Derakhshandeh et al. (2016) [current best]: time $O(\sqrt{n})$. *Assumes:* well-initialised configuration. Introductory and current best.

Notes. The model assumptions (chirality, sequential activation, randomization) are as described by (Di Luna et al. 2020c).

21.2.3 SILBOT, chirality

Adds oblivious, + chirality VERIFIED

Silent oblivious particles cannot form a line if they share only chirality.

Assumed: orientation: common chirality. **Not assumed:** memory: oblivious, silent particles (SILBOT); scheduler: weak: non-atomic activations, an action may interleave with others; randomness: deterministic; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Statement for this setting. The target shape is a line.

Measure	Lower bound	Best upper bound	Status
Time	*impossible* (Navarra and Piselli 2023b) Unsolvable with chirality alone (Navarra and Piselli 2023b).	—	• impossible
Moves	none known —	—	• impossible

Results.

- Navarra and Piselli (2023b) [lower bound]: time *impossible*. *Bottleneck:* symmetry breaking. *Holds when:* a symmetric initial configuration; a synchronous adversary; holds even with common chirality.

Open questions.

Q1. Is line formation solvable in SILBOT when particles share just one direction instead of the full orientation (Navarra and Piselli 2023b)?

21.2.4 SILBOT, full orientation

Adds oblivious, + common compass, + may disconnect VERIFIED

With a common compass, silent oblivious particles form a line with $O(n^2)$ moves, which is optimal.

Assumed: orientation: full common orientation (compass); connectivity during execution: the system may disconnect temporarily. **Not assumed:** memory: oblivious, silent particles (SILBOT); scheduler: weak: non-atomic activations, an action may interleave with others; randomness: deterministic; initial shape: connected, holes allowed.

Statement for this setting. The target shape is a line.

Measure	Lower bound	Best upper bound	Status
Time	none known No round bound recorded.	—	• unknown
Moves	$\Omega(n^2)$ (Derakhshandeh et al. 2015) $\Theta(n^2)$ moves.	$O(n^2)$ (Navarra and Piselli 2023b)	• tight

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Derakhshandeh et al. (2015) [lower bound]: moves $\Omega(n^2)$. *Bottleneck:* distance. *Holds when:* Lemma 7: a compact start of diameter at most $2\sqrt{n}+2$ must become a line; holds for any algorithm.
- Navarra and Piselli (2023b) [current best]: moves $O(n^2)$. *Assumes:* 2-hop view. Introductory and current best.
- Navarra et al. (2025) [also]: moves *finite*. Journal version; adds configurations with immovable obstacles, and scattering.

Open questions.

Q1. Does 1-hop visibility suffice for line formation in SILBOT (Navarra and Piselli 2023b)?

Q2. Which other shapes can SILBOT particles form (Navarra and Piselli 2023b)?

Notes. The model is SILBOT (D'Angelo et al. 2020): no memory, no messages, non-atomic asynchronous actions, view of two hops. The algorithm (WRain) disconnects the system temporarily. Checked against the full text of (Navarra and Piselli 2023b).

21.3 Coating

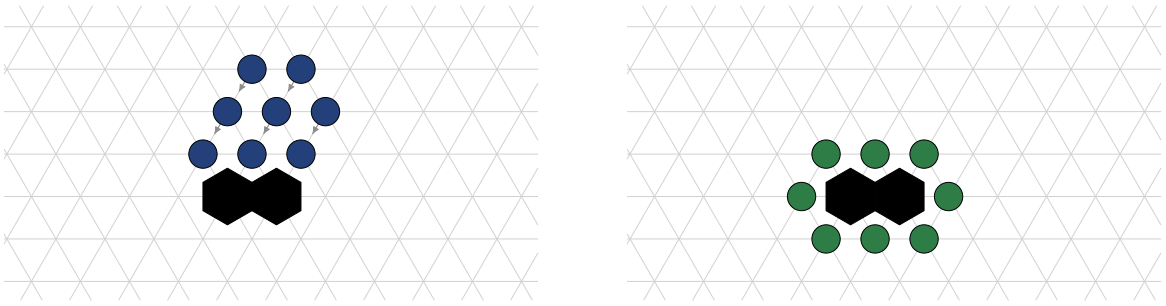
Particles must cover a fixed object as uniformly as possible: in layers, each filled before the next.

First posed in 2014 (Derakhshandeh et al. 2017). Infinite object coating appeared as an arXiv preprint in 2014; universal coating in (Derakhshandeh et al. 2017).

Problem statement and measures

A fixed **object** (a connected set of nodes of the triangular lattice that particles cannot enter) is surrounded by particles. They must arrange themselves into **layers** around the object, each layer filled before the next. **Time** in rounds; **work** is the number of moves (expansions and contractions).

Measures: **Time**, asynchronous rounds; **Moves**, total expansions and contractions (work).



(a) Start: roots touch the object; arrows point to parents

(b) Final: the first layer is filled

Figure 21.3: Coating, following (Derakhshandeh et al. 2017): particles that neighbour the object (black) are roots, the others follow a parent closer to the object (forming trees). Roots walk clockwise along the first layer, one expansion and one contraction per node; when an expanded particle contracts, its child expands into the node it leaves (a handover), so the trees are pulled along without breaking. With as many particles as first-layer nodes, the walk ends when every node that neighbours the object is occupied (green); further particles would start the next layer.

Claude (Anthropic) was used in cataloguing the problems.

A first look

Imagine ants covering a crumb, one even layer at a time. The floor is the **triangular lattice**: the endless graph whose **nodes** are the corners of a tiling of the plane by equal triangles; each node is joined by an edge to six others, its **neighbours**. A **particle** is a tiny machine that occupies one node; it moves in two actions: an **expansion** into an empty neighbouring node, after which it is **expanded** and occupies both, then a **contraction** into one of the two; so one expansion and one contraction move it by one node, and each of them is one **move**. The **object** is a connected set of nodes that particles cannot enter; n particles, connected to each other and to the object, sit around it. They must arrange themselves into **layers** around the object: the first layer (the nodes that neighbour the object) must be full before any particle sits in the second, and so on, with any extra particles in the last layer. In the standard version particles have constant memory, agree on which way is clockwise (**chirality**), and may flip coins; we count **rounds** (intervals of time in which every particle is activated at least once) and work (the total number of moves).

1. Algorithm: Follow the leader around the object, layer by layer. (Derakhshandeh et al. 2017) (Universal coating, p. 410) The particles move in **handovers**: an expanded particle contracts and at the same moment pulls a neighbour into the node it vacates, so the chain of particles never breaks.

1. Particles that neighbour the object become **roots**; every other particle picks a neighbour closer to the object as its *parent*, so the particles form trees hanging off the object.
2. Roots walk clockwise around the object along the first layer; followers trail behind their parents by handovers.
3. A root that bumps into a particle that has already stopped stops too (it *retires*). A follower that still wants a node in the layer sends a 'complaint' to a root with free space ahead, which then moves on and makes room.
4. A leader, elected among the particles of the first layer, marks where the layer begins and ends. When the first layer is full, the next layer starts from that mark, and so on.

Why it works and what it costs. In the worst case the total work is $\Theta(n^2)$, which matches the lower bound below (Derakhshandeh et al. 2017). The running time is $O(n)$ rounds with high probability (Daymude et al. 2018); the coins are needed only for the leader election, and $O(n)$ is the best possible in the worst case. Particles with no memory and no messages can still coat a convex object with one layer, in $O(n^2)$ rounds (Navarra and Piselli 2023a).

Why it matters. Spanning trees plus handovers became the standard way to move all the particles without ever disconnecting them.

2. Lower bound: A long tail of particles. (Daymude et al. 2018) (Universal coating, p. 410) Put all n particles in one straight line that meets the object only at its first particle, and let the first layer have more than n nodes. The particle at distance i from the object must end as a neighbour of it, so it must travel about i nodes, which takes about $2i$ moves. The last particle is at distance n, and an unkind scheduler may let it act only once per round.

Why it holds. The last particle needs about n moves at one move per round, so every algorithm needs $\Omega(n)$ rounds in the worst case (Daymude et al. 2018). Adding the distances, $2(1 + 2 + \cdots + (n-1)) \approx n^2$, so every algorithm needs $\Omega(n^2)$ work (Derakhshandeh et al. 2017). Both bounds are matched by the algorithm above.

Why it matters. It is the simplest example of the distance argument that underlies most lower bounds for programmable matter.

Claude (Anthropic) was used in cataloguing the problems.

3. Lower bound: Local particles cannot see where the gap is. (Daymude et al. 2018) (Universal coating, p. 410) Let the object be a long straight bar. The first layer is completely full except for one empty node exactly in the middle below the bar, and one extra particle sits in the second layer exactly in the middle above it. Someone who sees everything solves it in a constant number of rounds: every particle in the first layer shifts by one node around the bar, closing the gap, and the extra particle drops in. Local particles near the two ends of the bar cannot tell, within about $n/4$ rounds, which way the shift should go: information moves at most one node per round, and the situation looks the same from both ends.

Why it holds. So for about $n/4$ rounds the flows at the two ends are decided independently and by symmetry are equally likely to go either way; in the cases where they do not fit together, the layer is not yet correct. Hence any local algorithm is, on such inputs, about n times slower than the best algorithm with global knowledge: a competitive ratio of $\Omega(n)$ (Daymude et al. 2018) (Theorem 1).

Why it matters. It shows that 'worst case over all inputs' is the right measure here, since comparing to the best possible run on each input is hopeless for local algorithms.

What the community started looking at

Coating was posed as a basic task for self-organising matter with **work** (moves) as the first measure (Derakhshandeh et al. 2017); rounds followed, with matching bounds (Daymude et al. 2018). Coating was later solved by silent, oblivious particles (Navarra and Piselli 2023a) and in 3D hybrid programmable matter (Kostitsyna et al. 2024).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Moves) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Memory → Scheduler → Orientation → Randomness → Initial shape → Connectivity during execution.

No assumptions

+ constant memory, + strong scheduler, + chirality, + randomized
Universal coating STANDARD SETTING ●● p. 410

oblivious, weak scheduler, + chirality
SILBOT, chirality, deterministic ●● p. 410

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Moves	p.
Universal coating STANDARD SETTING	● tight $O(n)$ vs $\Omega(n)$	● tight $O(n^2)$ vs $\Omega(n^2)$	410
SILBOT, chirality, deterministic	● tight $O(n^2)$ vs $\Omega(n^2)$	● unknown — vs —	410

Claude (Anthropic) was used in cataloguing the problems.

21.3.1 Universal coating

Adds + constant memory, + strong scheduler, + chirality, + randomized STANDARD SETTING VERIFIED

Any object admitting a uniform coating is coated with optimal work, and in $\Theta(n)$ rounds with high probability.

Assumed: memory: constant memory and local communication; scheduler: strong: atomic activations under a fair adversary; orientation: common chirality; randomness: randomized. **Not assumed:** initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(n)$ (Daymude et al. 2018) $\Theta(n)$ rounds.	$O(n)$ (Daymude et al. 2018)	• tight
Moves	$\Omega(n^2)$ (Derakhshandeh et al. 2017) $\Theta(n^2)$ moves.	$O(n^2)$ (Derakhshandeh et al. 2017)	• tight

Results.

- Daymude et al. (2018) [lower bound]: time $\Omega(n)$. *Bottleneck:* locality. *Holds when:* Lemma 1: deterministic worst case; a line of n particles attached to the object by one particle, the object's boundary longer than n.
- Derakhshandeh et al. (2017) [lower bound]: moves $\Omega(n^2)$. *Bottleneck:* distance. *Holds when:* Lemma 15.
- Derakhshandeh et al. (2017) [introductory]: moves $O(n^2)$. Worst-case work-optimal: $\Theta(n^2)$ moves (Theorem 2).
- Daymude et al. (2018) [current best]: time $O(n)$. With high probability; randomisation enters only through the leader election.

Notes. Chirality is part of the model of both papers. In 3D hybrid programmable matter (one finite-automaton agent moving passive tiles), universal coating takes $O(n^2)$ steps, worst-case optimal against an agent with global knowledge (Kostitsyna et al. 2024). Checked against the full texts.

21.3.2 SILBOT, chirality, deterministic

Adds oblivious, weak scheduler, + chirality VERIFIED

Silent oblivious particles with chirality coat a convex object with one layer, deterministically, in $O(n^2)$ rounds.

Assumed: orientation: common chirality. **Not assumed:** memory: oblivious, silent particles (SILBOT); scheduler: weak: non-atomic activations, an action may interleave with others; randomness: deterministic; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Statement for this setting. Single-layer coating of a **convex** object, together with compaction of holes.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(n^2)$ (Navarra and Piselli 2023a) $\Theta(n^2)$ rounds; the lower bound is argued informally.	$O(n^2)$ (Navarra and Piselli 2023a)	• tight
Moves	none known —	—	• unknown

Results.

- Navarra and Piselli (2023a) [lower bound]: time $\Omega(n^2)$. *Bottleneck:* oblivious silent particles. *Holds when:* informal argument before Theorem 4, not a theorem.

Claude (Anthropic) was used in cataloguing the problems.

- Navarra and Piselli (2023a) [current best]: time $O(n^2)$. *Assumes:* 2-hop view; exterior awareness: a particle can tell a hole from the outside. Introductory and current best.

Open questions.

Q1. Coating in SILBOT without chirality (Navarra and Piselli 2023a).

Q2. Multi-layer coating, which "seems unfeasible" in SILBOT (Navarra and Piselli 2023a).

Notes. Checked against the full text.

21.4 Compression

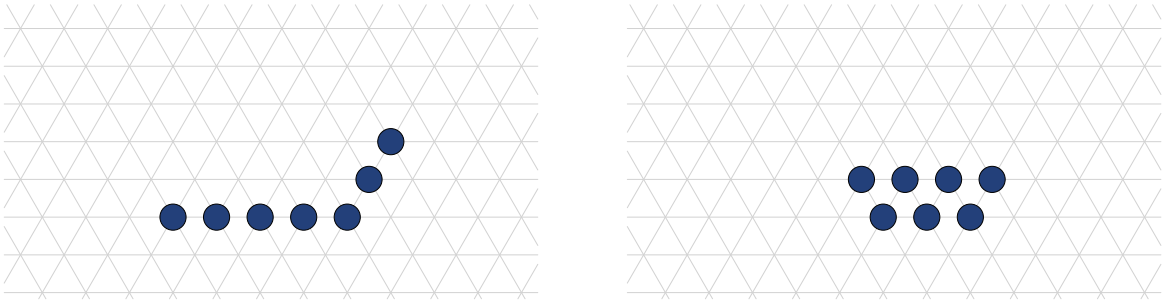
Particles following a local stochastic rule must gather into a configuration with small perimeter.

First posed in 2016 (Cannon et al. 2016). PODC 2016.

Problem statement and measures

Starting connected, the n particles perform a **Markov chain** (at each step one particle, chosen at random, proposes a move and accepts it with a probability that depends only on its neighbourhood) with **bias** λ towards having more neighbours. The system is α-**compressed** if its **perimeter** is at most α times the minimum possible.

Measures: **Time**, asynchronous rounds; **Moves**, total expansions and contractions (work).



(a) Start: a long chain, long perimeter

(b) Final: compressed, short perimeter

Figure 21.4: Compression by the Markov chain of (Cannon et al. 2016): one particle at a time (outlined) expands into a random free neighbouring node and then either contracts forward, with probability $\lambda^{e'-e}$ (e and e' its neighbours before and after; certainly if $e' \geq e$), or contracts back; moves that would disconnect the system or create a hole are refused. For $\lambda > 2 + \sqrt{2}$ the system becomes compressed (perimeter at most α times the minimum) with high probability; for $\lambda < 2.17$ it does not.

A first look

Picture a crowd of penguins huddling together for warmth, each one just shuffling towards places with more neighbours. The floor is the **triangular lattice**: the endless graph whose **nodes** are the corners of a tiling of the plane by equal triangles; each node is joined by an edge to six others, its **neighbours**. A **particle** occupies one node and moves in two actions: an **expansion** into an empty neighbouring node, after which it occupies both, then a **contraction** into one of the two; so one expansion and one

Claude (Anthropic) was used in cataloguing the problems.

contraction move it by one node. The **system** of n particles starts connected. One particle at a time, chosen at random, is activated (an **activation**) and makes a random move; particles have no names, a few bits of memory, and no compass. The goal is to become **compressed**: the **perimeter** (the length of the walk around the outside of the system) should be at most α times the smallest possible perimeter for n particles, for a constant $\alpha > 1$. The question is not how fast this happens, but whether, after running long enough, the system is compressed with very high probability.

1. Algorithm: Move at random, but prefer more neighbours. (Cannon et al. 2016) (Markov chain, sequential, p. 413) Fix a number $\lambda > 0$, the **bias**. A randomly chosen particle does the following.

1. Pick one of its six directions at random. If that node is empty, expand into it.
2. Count e, its neighbours at the old node, and e', the neighbours it would have at the new node.
3. Check, by looking only at nearby nodes, that the move would neither split the system nor enclose a **hole** (a region of empty nodes surrounded by particles). If it would, contract back.
4. Otherwise contract into the new node with probability $\lambda^{e'-e}$ (or with certainty if this number exceeds 1); else contract back.

So moves that gain neighbours are always accepted when $\lambda > 1$, and moves that lose neighbours are accepted only sometimes.

Why it works and what it costs. In the long run, each **configuration** (set of occupied nodes) appears with probability proportional to λ raised to its number of pairs of neighbours, and fewer pairs of neighbours means a longer perimeter. For a large bias, $\lambda > 2 + \sqrt{2} \approx 3.41$, the system is compressed with all but exponentially small probability (Cannon et al. 2016). For a small bias, $\lambda < 2.17$, it is the opposite: the system stays stretched out, with perimeter a constant fraction of the largest possible, even though each particle still prefers neighbours. Between 2.17 and 3.41 the paper's theorems say nothing.

Why it matters. This 'local random rule with a bias' recipe became the template for separation, alignment and bridging.

2. Lower bound: Too many stretched configurations: why a weak bias fails. (Cannon et al. 2016) (Markov chain, sequential, p. 413) Two forces compete. The bias favours each single compact configuration over each single stretched one: a configuration with perimeter p gets weight roughly λ^{-p}. But stretched configurations are far more numerous. The number of configurations with perimeter p grows roughly like $(2 + \sqrt{2})^p$, because the walk around the outside never crosses itself on the honeycomb lattice (the hexagonal lattice of the triangles' centres), and such walks grow at exactly that rate. Multiply: all configurations of perimeter p together weigh about $\left((2 + \sqrt{2})/\lambda\right)^p$.

Why it holds. If $\lambda > 2 + \sqrt{2}$, this total shrinks exponentially as p grows, so long perimeters are very unlikely and the system compresses. If λ is small, the sheer number of long-perimeter configurations outweighs the bias, and (Cannon et al. 2016) prove that for $\lambda < 2.17$ the perimeter stays at least a constant fraction of its maximum. This is the same balance of energy against counting that physicists use for phase transitions.

Why it matters. It explains why 'prefer neighbours' is not enough on its own and how strong the preference must be.

What the community started looking at

Compression introduced the **stochastic** approach to programmable matter, where the question is which bias produces which emergent behaviour (Cannon et al. 2016).

Claude (Anthropic) was used in cataloguing the problems.

Relations to other problems

- **Derived problem: Separation of coloured particles** (generalisation of). Separation adds two colours to compression; the same local chain compresses and, depending on a second bias, separates or integrates the colours (Cannon et al. 2019).
- **Derived problem: Alignment** (generalisation of). Alignment adds an orientation to each particle; the same chains also give compression and expansion (Kedia et al. 2022).
- **Derived problem: Bridging** (generalisation of). Bridging adds terrain: the compression chain is biased to stay on land and to shorten a path across a gap (Arroyo et al. 2018).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Moves) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Memory → Scheduler → Orientation → Randomness → Initial shape → Connectivity during execution.

No assumptions

+ constant memory, + sequential, + randomized

Markov chain, sequential STANDARD SETTING ●● p. 413

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Moves	p.
Markov chain, sequential STANDARD SETTING	● unknown *eventually* vs —	● unknown — vs —	413

21.4.1 Markov chain, sequential

Adds + constant memory, + sequential, + randomized STANDARD SETTING VERIFIED

For $\lambda > 2 + \sqrt{2}$ the system eventually compresses; for $\lambda < 2.17$ it expands.

Assumed: memory: constant memory and local communication; scheduler: sequential: one particle active at a time; randomness: randomized. **Not assumed:** orientation: no common orientation and no chirality; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known Mixing time not bounded.	*eventually* (Cannon et al. 2016)	● unknown
Moves	none known —	—	● unknown

Results.

- Cannon et al. (2016) [current best]: time *eventually*. Introductory and current best; with all but exponentially small probability.

Claude (Anthropic) was used in cataloguing the problems.

Open questions.

Q1. Behaviour for $2.17 \leq \lambda \leq 2 + \sqrt{2}$, and bounds on the convergence time.

21.5 Separation of coloured particles

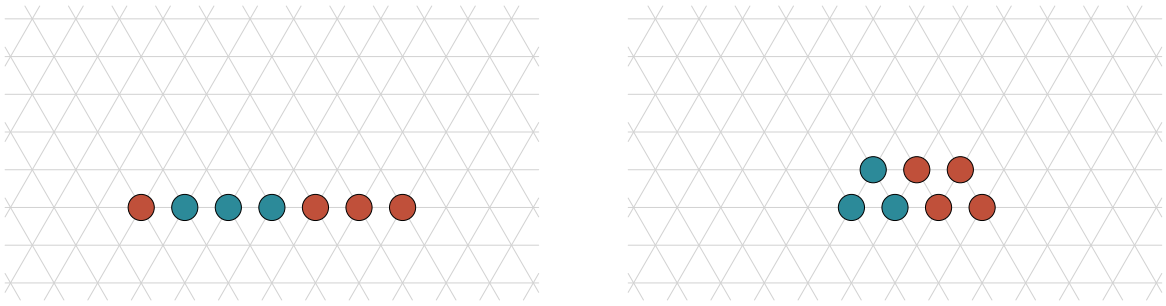
Particles of two colours, following a local stochastic rule, must cluster by colour (or mix).

First posed in 2018 (Cannon et al. 2019). arXiv 2018; APPROX/RANDOM 2019.

Problem statement and measures

Each particle has one of two colours. Particles run a local **Markov chain** with a **bias** λ towards more neighbours and a bias γ towards neighbours of the same colour. The system should be **compressed** and **separated** (few pairs of neighbours of different colours), or **integrated** (colours mixed), with high probability in the stationary distribution.

Measures: **Time**, asynchronous rounds; **Moves**, total expansions and contractions (work).



(a) Start: a chain, colours mixed

(b) Final: compressed and separated

Figure 21.5: Separation by the Markov chain of (Cannon et al. 2019): one particle at a time (outlined) expands into a free neighbouring node and contracts forward with probability $\lambda^{\Delta e}\gamma^{\Delta h}$ (Δe, Δh: the change in its neighbours and in its neighbours of its own colour; certainly if the product is at least 1), and otherwise contracts back; moves that would disconnect the system or create a hole are refused. The chain also lets neighbours of different colours swap places (not shown). For large λ and γ the system becomes compressed and separated with high probability; for γ close to 1 it stays integrated.

A first look

Think of a mixed crowd of red and blue marbles in a tray that, when shaken, sorts itself into a red patch and a blue patch, or stays mixed, depending on how 'sticky' same-coloured marbles are. The floor is the **triangular lattice**: the endless graph whose **nodes** are the corners of a tiling of the plane by equal triangles; each node is joined by an edge to six others, its **neighbours**. A **particle** occupies one node and moves in two actions: an **expansion** into an empty neighbouring node, after which it occupies both, then a **contraction** into one of the two; so one expansion and one contraction move it by one node. The **system** of n particles starts connected; each particle carries a fixed colour, one of two, has constant memory and no compass, and one randomly chosen particle acts at a time. The system should become **compressed** (its **perimeter**, the length of the walk around its outside, is at most a constant times the smallest possible, as in compression) and either **separated** (each colour mostly in one region,

Claude (Anthropic) was used in cataloguing the problems.

with few pairs of neighbours of different colours) or **integrated** (colours mixed), whichever is asked for. As for compression, the question is what the system looks like after running for a long time, with high probability.

1. Algorithm: The compression rule with a second bias for colour. (Cannon et al. 2019) (Markov chain, sequential, p. 416) There are two **biases**: λ rewards neighbours of any colour, and γ rewards neighbours of one's own colour. A randomly chosen particle does one of two things.

1. **Step.** As in compression: pick a random empty neighbouring node; check locally that moving there neither splits the system nor encloses a **hole** (a region of empty nodes surrounded by particles); then move with probability $\lambda^{(\text{gain in neighbours})} \cdot \gamma^{(\text{gain in same-coloured neighbours})}$ (certainly, if this number exceeds 1).
2. **Swap.** Pick a neighbour of the other colour and exchange nodes with it, with probability $\gamma^{(\text{gain in same-coloured neighbours of the two})}$ (certainly, if above 1). Swaps let colours move through the middle of the system.

Why it works and what it costs. With only one colour the rule is exactly the compression rule (Cannon et al. 2016). For a strong colour bias, $\gamma > 4^{5/4} \approx 5.66$ together with $\lambda\gamma > 6.83$ (and $\lambda > 1$), the system is compressed and separated with high probability (Cannon et al. 2019). For γ very close to 1, between 79/81 and 81/79, and λ large enough, it is compressed but integrated. So one rule does both jobs, and a single number set by the environment decides which.

Why it matters. It showed that the biased-rule recipe extends from shape to internal organisation, here the arrangement of types.

2. Lower bound: A mild colour preference does not separate the colours. (Cannon et al. 2019) (Markov chain, sequential, p. 416) One might expect any preference $\gamma > 1$ for same-coloured neighbours to separate the colours eventually. It does not. There are vastly more mixed arrangements of the colours inside a compressed system than separated ones: separation means fixing a short border between the colours, while mixing leaves almost every particle free to be either colour. A slight preference gives each separated arrangement only a slightly larger weight, which cannot make up for the huge number of mixed ones.

Why it holds. (Cannon et al. 2019) prove that for γ between 79/81 and 81/79, which includes values slightly above 1, the system fails to separate with high probability: the number of pairs of neighbours of different colours stays large. Separation needs the colour bias to beat this counting, which is why the positive result asks for $\gamma > 5.66$. What happens for γ between these ranges is not settled by the paper.

Why it matters. It is the colour version of the compression lesson: a preference must be strong enough to beat the number of disordered states.

What the community started looking at

Separation extended the stochastic approach of compression to **heterogeneous** systems; the question is which bias parameters give which emergent behaviour (Cannon et al. 2019).

Relations to other problems

- **Generalisation of Compression.** Separation adds two colours to compression; the same local chain compresses and, depending on a second bias, separates or integrates the colours (Cannon et al. 2019).

Claude (Anthropic) was used in cataloguing the problems.

- **Related: Separation.** Robots in the plane separate by hidden colours with deterministic algorithms; here a stochastic local rule separates two visible colours.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Moves) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Memory → Scheduler → Orientation → Randomness → Initial shape → Connectivity during execution.

No assumptions

+ constant memory, + sequential, + randomized

Markov chain, sequential STANDARD SETTING ●● p. 416

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Moves	p.
Markov chain, sequential STANDARD SETTING	● unknown *eventually* vs —	● unknown — vs —	416

21.5.1 Markov chain, sequential

Adds + constant memory, + sequential, + randomized STANDARD SETTING VERIFIED

One local algorithm separates or integrates the colours depending on a single global parameter.

Assumed: memory: constant memory and local communication; scheduler: sequential: one particle active at a time; randomness: randomized. **Not assumed:** orientation: no common orientation and no chirality; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known Mixing time not bounded.	*eventually* (Cannon et al. 2019)	● unknown
Moves	none known —	—	● unknown

Results.

- Cannon et al. (2019) [current best]: time *eventually*. Introductory and current best; proofs use cluster expansions and the high-temperature expansion of the Ising model.

Open questions.

Q1. Bounds on the convergence (mixing) time.

Claude (Anthropic) was used in cataloguing the problems.

21.6 Alignment

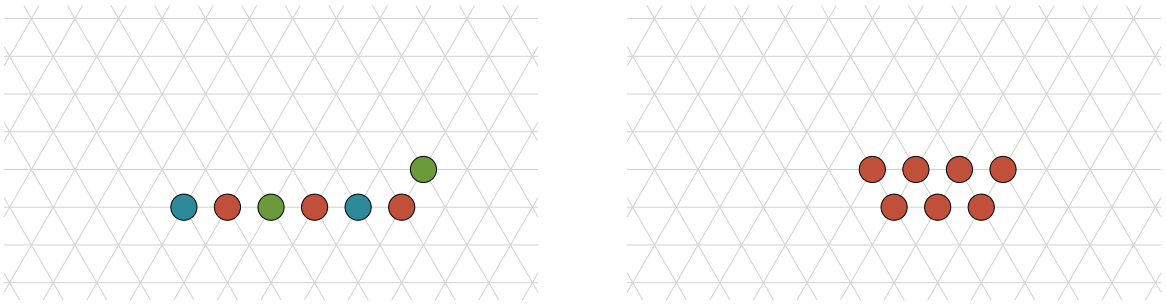
Oriented particles, following a local stochastic rule, must align their orientations.

First posed in 2022 (Kedia et al. 2022). APPROX/RANDOM 2022.

Problem statement and measures

Each particle carries one of $q \geq 2$ **orientations**, a spin state, not a sense of direction. Particles run a local **Markov chain**; the system should be **aligned** (most particles share an orientation) or not, depending on the **bias**, with high probability in the stationary distribution, both when connectivity is kept and when particles may disconnect.

Measures: **Time**, asynchronous rounds; **Moves**, total expansions and contractions (work).



(a) Start: orientations mixed

(b) Final: compressed, one orientation shared

Figure 21.6: Alignment by the Markov chain of (Kedia et al. 2022), connected setting: each particle carries one of q orientations (here $q = 3$, shown as three colours). One particle at a time (outlined) either steps, expanding into a free neighbouring node and contracting forward with a probability that grows with its gain in neighbours (λ) and in agreeing neighbours (γ), or turns to a new orientation, accepted with a probability that grows with its gain in agreeing neighbours; moves that would disconnect the system or create a hole are refused. Shown is a run in which every proposed move is accepted. For large biases most particles end up sharing one orientation, with high probability; for small biases they do not.

A first look

Picture a school of fish or a flock of birds in which every animal turns to face the same way as its neighbours. The floor is the **triangular lattice**: the endless graph whose **nodes** are the corners of a tiling of the plane by equal triangles; each node is joined by an edge to six others, its **neighbours**. A **particle** occupies one node and moves in two actions: an **expansion** into an empty neighbouring node, after which it occupies both, then a **contraction** into one of the two; so one expansion and one contraction move it by one node. The **system** of n particles starts connected, and each particle carries an **orientation**, one of $q \geq 2$ values drawn as an arrow; the orientation is just a state, not a compass, and a particle can only compare its orientation with its neighbours' orientations. One randomly chosen particle acts at a time; it either steps (moves to a neighbouring node) or turns (changes its orientation). The system should become **aligned** (one orientation is held by most particles) or stay *non-aligned* (all orientations about equally common), depending on a bias; this is studied both when the system must stay connected and when it may fall apart.

Claude (Anthropic) was used in cataloguing the problems.

1. Algorithm: Step or turn, with a bias towards agreeing neighbours. (Kedia et al. 2022) (Markov chain, sequential, p. 419) In the connected setting there are two **biases**: λ rewards having neighbours, as in compression, and a second bias, written γ in (Kedia et al. 2022), rewards **agreeing neighbours** (neighbours with the same orientation). A randomly chosen particle does one of two things, with equal chance.

1. **Step.** Pick a random empty neighbouring node; check locally that the system stays connected without **holes** (regions of empty nodes surrounded by particles); move with a probability that grows with the gain in neighbours and in agreeing neighbours.
2. **Turn.** Pick a new orientation; accept it with a probability that grows with the gain in agreeing neighbours (certainly, if it gains).

Why it works and what it costs. With both biases large (for instance $\gamma > 29.3(q-1)$ and $\lambda\gamma$ large enough), the system is **compressed** (its perimeter, the length of the walk around its outside, is at most a constant times the minimum) with high probability, and under further conditions on γ one orientation dominates (Kedia et al. 2022). With λ large but γ small, the system is compressed but its orientations stay evenly split among the q values; with λ small, the system stays stretched out. Compactness matters: a long thin bottleneck lets two regions with different orientations coexist cheaply, so alignment is proved only for well compressed systems.

Why it matters. It brings the Potts and clock models of magnetism into programmable matter, with particles that move as well as turn.

2. Algorithm: When particles may drift apart: one knob. (Kedia et al. 2022) (Markov chain, sequential, p. 419) If the system is allowed to disconnect, the connectivity check is dropped: a particle may step to any empty neighbouring node. A single bias then rewards, at the same time, having neighbours and having agreeing neighbours.

1. Pick step or turn with equal chance.
2. Step: move to a random empty neighbouring node, accepted with a probability that grows with the gain in (agreeing) neighbours.
3. Turn: as before.

Why it works and what it costs. When the single bias is large, the particles gather into compact **clusters** (sets of particles connected through neighbours) and align; when it is small, they disperse and the orientations stay balanced (Kedia et al. 2022). The proof is harder than in the connected case because a spread-out system can contain many small clusters scattered over the lattice.

Why it matters. It shows that the stochastic approach does not need the connectivity rule, which is costly to check and not always realistic.

What the community started looking at

Alignment brought Potts and clock models from statistical physics to programmable matter (Kedia et al. 2022).

Relations to other problems

- **Generalisation of Compression.** Alignment adds an orientation to each particle; the same chains also give compression and expansion (Kedia et al. 2022).

Claude (Anthropic) was used in cataloguing the problems.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Moves) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Memory → Scheduler → Orientation → Randomness → Initial shape → Connectivity during execution.

No assumptions

+ constant memory, + sequential, + randomized

Markov chain, sequential STANDARD SETTING ●● p. 419

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Moves	p.
Markov chain, sequential STANDARD SETTING	● unknown *eventually* vs —	● unknown — vs —	419

21.6.1 Markov chain, sequential

Adds + constant memory, + sequential, + randomized STANDARD SETTING VERIFIED

For every $q \geq 2$, alignment and non-alignment are both achieved by the bias.

Assumed: memory: constant memory and local communication; scheduler: sequential: one particle active at a time; randomness: randomized. **Not assumed:** orientation: no common orientation and no chirality; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known Mixing time not bounded.	*eventually* (Kedia et al. 2022)	● unknown
Moves	none known —	—	● unknown

Results.

- Kedia et al. (2022) [current best]: time *eventually*. Introductory and current best; simply connected and unconstrained settings.

21.7 Bridging

Particles moving between two sites must build a bridge across a gap that balances path length against bridge cost.

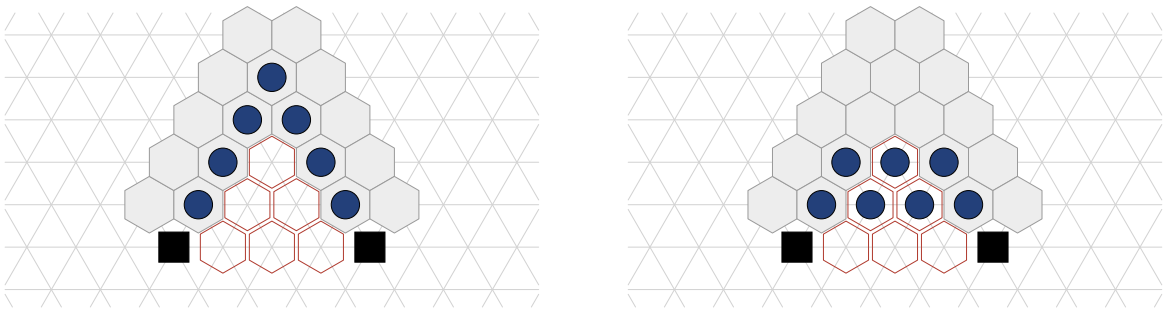
First posed in 2017 (Arroyo et al. 2018). arXiv 2017; Natural Computing 2018.

Claude (Anthropic) was used in cataloguing the problems.

Problem statement and measures

Particles occupy a path between two fixed **sites** across a **gap** in the terrain. They run a local **Markov chain** that rewards neighbours and penalises particles over the gap. The system should form a shortcut **bridge** whose length and cost are balanced, with high probability in the stationary distribution.

Measures: **Time**, asynchronous rounds; **Moves**, total expansions and contractions (work).



(a) Start: particles follow the land

(b) Final: a shortcut bridge over the gap

Figure 21.7: Shortcut bridging by the Markov chain of (Arroyo et al. 2018): particles join two fixed sites (black squares) along the land (shaded); the nodes under the land are over the gap (red outline). One particle at a time (outlined) expands into a free neighbouring node and contracts forward with a probability that rewards a short perimeter (λ) and penalises perimeter over the gap (γ), otherwise it contracts back; moves that would disconnect the particles from the sites, or create a hole, are refused. Shown is a run of accepted moves: the bridge shortcuts the corner but keeps few particles over the gap.

A first look

Army ants build living bridges out of their own bodies to cut a corner across a gap: a longer bridge saves walking but takes more ants out of work. The floor is the **triangular lattice**: the endless graph whose **nodes** are the corners of a tiling of the plane by equal triangles; each node is joined by an edge to six others, its **neighbours**. A **particle** occupies one node and moves in two actions: an **expansion** into an empty neighbouring node, after which it occupies both, then a **contraction** into one of the two; so one expansion and one contraction move it by one node. The particles form a connected **system** joining two fixed **sites** (fixed locations at the two ends of the path); some nodes are marked as **gap** (over empty space) and the others as **land**, and a particle can tell which kind of node it is on. One randomly chosen particle acts at a time, with constant memory and no compass. The system should settle into a **bridge** that balances a short path between the two sites against few particles hanging over the gap, measured by a weighted **perimeter**: the length of the walk around the outside of the system, in which each edge of the walk over the gap costs more than one over land.

1. Algorithm: The compression rule, with the gap made expensive. (Arroyo et al. 2018) (Markov chain, sequential, p. 422) There are two **biases**: $\lambda > 1$ rewards a short perimeter (many neighbours), and a second bias, written γ in (Arroyo et al. 2018), rewards a short perimeter over the gap.

1. A randomly chosen particle picks a random empty neighbouring node.
2. It checks locally that the move neither splits the system nor encloses a **hole** (a region of empty nodes surrounded by particles).
3. It works out, from its neighbourhood, how the move changes the perimeter and the part of the perimeter that lies over the gap.

Claude (Anthropic) was used in cataloguing the problems.

4. It moves with a probability that is larger when both get shorter (certainly, when the move improves both).

Why it works and what it costs. For $\lambda > 2 + \sqrt{2} \approx 3.41$ and $\gamma > 1$, the weighted perimeter is within a constant factor of its minimum with all but exponentially small probability (Arroyo et al. 2018). For a V-shaped gap the bridge settles at a position that depends on the angle of the V, as observed in real ant bridges. The threshold $2 + \sqrt{2}$ is the same as for compression, and for the same counting reason; for small biases the paper gives no guarantee.

Why it matters. It showed that the compression rule is a flexible tool: changing the weights of the edges of the perimeter changes the shape the system settles into.

2. Algorithm: Why only one bridge forms. (Oh et al. 2024) (Markov chain, sequential, p. 422) Fire ants searching for food across water start several bridges but end with only one. (Oh et al. 2024) model this with particles that prefer to move in one direction (towards the food) and prefer having neighbours, with one bias for each preference.

1. Particles move at random, but a step towards the food is more likely than a step away.
2. A step that gains neighbours is more likely than one that loses them.
3. Nothing else: no particle knows how many bridges exist.

Why it works and what it costs. When both biases are large, a single bridge forms almost certainly (Oh et al. 2024). Intuitively, two bridges have a longer perimeter than one bridge made of the same particles, and the neighbour bias makes long perimeters unlikely. The proof follows which nodes are occupied rather than which particle is where, which simplifies the analysis.

Why it matters. It shows that a striking collective decision, choosing one bridge, can arise from two simple preferences without coordination.

What the community started looking at

Bridging was inspired by army ants, whose bridges depend on the gap angle (Arroyo et al. 2018); the later question was whether a **single** bridge forms (Oh et al. 2024).

Relations to other problems

- **Generalisation of Compression.** Bridging adds terrain: the compression chain is biased to stay on land and to shorten a path across a gap (Arroyo et al. 2018).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Moves) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Memory → Scheduler → Orientation → Randomness → Initial shape → Connectivity during execution.

No assumptions

+ constant memory, + sequential, + randomized

Markov chain, sequential STANDARD SETTING ●● p. 422

Claude (Anthropic) was used in cataloguing the problems.

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Moves	p.
Markov chain, sequential STANDARD SETTING	● unknown *eventually* vs —	● unknown — vs —	422

21.7.1 Markov chain, sequential

Adds + constant memory, + sequential, + randomized STANDARD SETTING VERIFIED

A local stochastic rule builds near-optimal shortcut bridges; for large biases a single bridge forms.

Assumed: memory: constant memory and local communication; scheduler: sequential: one particle active at a time; randomness: randomized. **Not assumed:** orientation: no common orientation and no chirality; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known Mixing time not bounded.	*eventually* (Arroyo et al. 2018)	● unknown
Moves	none known —	—	● unknown

Results.

- Arroyo et al. (2018) [current best]: time *eventually*. Introductory and current best for shortcut bridging; the trade-off depends on the gap angle.
- Oh et al. (2024) [also]: time *eventually*. A single bridge almost certainly forms for large biases; occupancy-chain technique.

21.8 Convex hull formation

Particles must enclose an object in its convex hull, sensing only locally.

First posed in 2018 (Daymude et al. 2020). arXiv 2018; ICDCN 2020.

Problem statement and measures

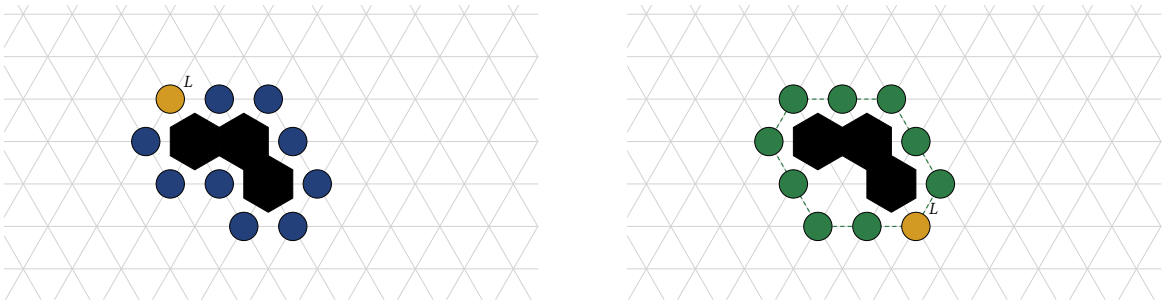
Particles must form the **convex hull** (and the weak $\mathcal{O}_\Delta$-hull) of a fixed **object**, a connected set of nodes of the triangular lattice that particles cannot enter. B is the length of the object's **boundary**.

Measures: **Time**, asynchronous rounds; **Moves**, total expansions and contractions (work).

A first look

Imagine sealing an oddly shaped object inside the tightest convex 'fence', like a rubber band stretched around it. The floor is the **triangular lattice**: the endless graph whose **nodes** are the corners of a tiling of the plane by equal triangles; each node is joined by an edge to six others, its **neighbours**. A **particle** occupies one node and moves in two actions: an **expansion** into an empty neighbouring

Claude (Anthropic) was used in cataloguing the problems.



(a) Start: leader L, all six walls through it

(b) Final: every node next to the hull occupied

Figure 21.8: Convex hull formation, following (Daymude et al. 2020): a leader (L) walks clockwise around the object (black) and keeps its distance to six walls, one per side of a hexagon with lattice-parallel sides; a step beyond a wall pushes that wall outwards (dashed: the current estimate). Here the particles already surround the object, so the leader advances by role swaps, handing its role and counters to the particle ahead, and nobody moves. Once the leader has touched all six walls since its last push, the hull is known, and the particle in the bay steps out (expansion, contraction), so that the particles occupy exactly the nodes that neighbour the hull.

node, after which it occupies both, then a **contraction** into one of the two; so one expansion and one contraction move it by one node. A fixed **object** occupies a connected set of nodes that particles cannot enter, and particles sit next to it; B is the length of the object's **boundary** (the walk around its outside). The particles must occupy exactly the nodes of the object's **convex hull**, taken one layer outside the object: on the triangular lattice, the smallest region bounded by six straight sides in the lattice's three directions that contains the object. Particles have constant memory, no **compass** (no agreement on the six directions of the lattice), and an adversarial scheduler; one particle starts as the **leader** on the object's boundary. Time is counted in **rounds** (intervals of time in which every particle is activated at least once).

1. Algorithm: Walk around the object and push six walls outwards. (Daymude et al. 2020) (No orientation, p. 424)

1. Using its own local names for the six directions, the leader imagines six **walls** (half-planes), one per side of the future hull. It keeps six **counters**: its distance to each wall, all starting at 0.
2. The leader walks clockwise around the object's boundary. Each step brings it closer to two walls and farther from two others, and it updates the counters.
3. If a step would take it beyond a wall (a counter already at 0 must go down), that wall is *pushed* one step outwards: the estimate of the hull grows.
4. The leader also remembers, for each wall, whether it has touched that wall since its last push. Once it has touched all six without pushing any, the walls are in their final places.
5. The counters can be as large as B, too big for one particle, so the followers trailing the leader each store one bit of a binary counter.
6. Finally the particles fill the nodes along the six walls.

Why it works and what it costs. The walls only move outwards and stop at the true hull, so the walk ends with the correct hull. The paper proves that the whole process, including the distributed counters, takes $O(B)$ rounds (Daymude et al. 2020). The same algorithm can be extended to a tighter hull (the weak $\mathcal{O}_\Delta$-hull) in the same time; if there are too few particles, it forms as much of the hull as it can.

Why it matters. Distributed binary counters let constant-memory particles count up to large numbers, a tool other algorithms reuse.

Claude (Anthropic) was used in cataloguing the problems.

2. Lower bound: Someone must reach the far end. (No orientation, p. 424) This simple argument is ours; the catalogue records no lower bound for this problem. Let the object be a long straight bar, so B is about twice its length, and let all the particles start bunched together near one end, enough of them to fill the hull. The hull has nodes next to the far end of the bar, at distance about $B/2$ from where the particles start, so some particle must travel about $B/2$ nodes.

Why it holds. Each node of travel takes one expansion and one contraction, and an unkind scheduler may let each particle act only once per round, so this takes at least about $B/2$ rounds. So $\Omega(B)$ rounds are needed in the worst case, and the $O(B)$ algorithm above is best possible on such inputs.

Why it matters. It shows that the boundary length, not the number of particles, is the right measure for this problem.

What the community started looking at

Convex hull formation asked whether particles **without shared orientation** can compute a global geometric property, and how fast (Daymude et al. 2020).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Moves) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Memory → Scheduler → Orientation → Randomness → Initial shape → Connectivity during execution.

No assumptions

+ constant memory, + strong scheduler

No orientation STANDARD SETTING ●● p. 424

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Moves	p.
No orientation STANDARD SETTING	● unknown $O(B)$ vs —	● unknown — vs —	424

21.8.1 No orientation

Adds + constant memory, + strong scheduler STANDARD SETTING VERIFIED

$O(B)$ rounds without shared orientation.

Assumed: memory: constant memory and local communication; scheduler: strong: atomic activations under a fair adversary. **Not assumed:** orientation: no common orientation and no chirality; randomness: deterministic; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Claude (Anthropic) was used in cataloguing the problems.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound found.	$O(B)$ (Daymude et al. 2020)	• unknown
Moves	none known —	—	• unknown

Results.

- Daymude et al. (2020) [current best]: time $O(B)$. Introductory and current best; uses distributed binary counters.

21.9 Scattering of particles

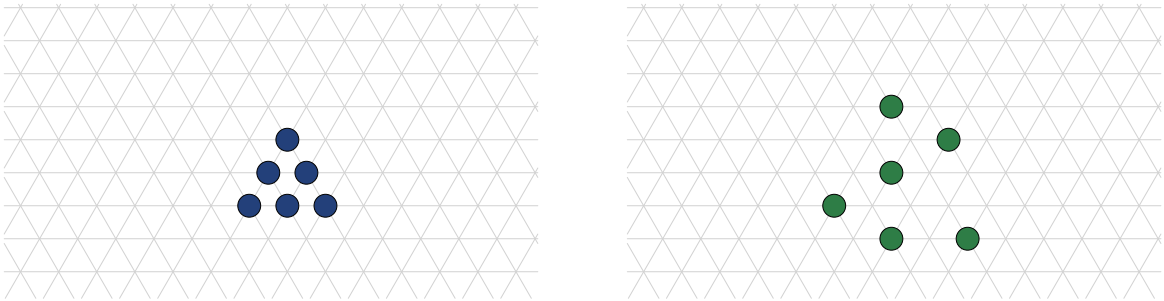
Particles must reach a configuration in which no two particles occupy neighbouring nodes.

First posed in 2023 (Navarra et al. 2025). Scattering in SILBOT first appeared at AINA 2023 (Navarra, Prencipe, Bonini, Tracolli); that paper's DOI was not verified, so the journal version is cited.

Problem statement and measures

From any initial **configuration** of n **contracted** particles (each occupying one node of the triangular lattice), the particles must reach a configuration in which no two occupy neighbouring nodes (a **scattered** configuration), and stop.

Measures: **Time**, asynchronous rounds; **Moves**, total expansions and contractions (work).



(a) Start: crowded particles

(b) Final: no two particles neighbours; all stop

Figure 21.9: Scattering: from a configuration of contracted particles, here a compact triangle, the particles must reach a configuration in which no two occupy neighbouring nodes, and stop. Shown is an illustrative run of event-driven activation: a particle can be activated only while it has a neighbour (blue); an activated particle (outlined) steps, by an expansion and a contraction, to a free node with fewer neighbours; the exact local rule of (Navarra et al. 2025) is not reproduced here. Isolated particles (green) are never activated again.

A first look

Picture people standing in a crowded square who must spread out until nobody is standing right next to anyone else. The floor is the **triangular lattice**: the endless graph whose **nodes** are the corners of a tiling of the plane by equal triangles; each node is joined by an edge to six others, its **neighbours**. A **particle** occupies one node and moves in two actions: an **expansion** into an empty neighbouring node, after which it occupies both, then a **contraction** into one of the two; so one expansion and one

Claude (Anthropic) was used in cataloguing the problems.

contraction move it by one node. There are n particles, each occupying one node at the start; the set of occupied nodes is the **configuration**. The particles are **oblivious** (they remember nothing from one action to the next) and **silent** (they send no messages); each decides only from what it sees around it. The goal is to reach a **scattered** configuration, where no two particles occupy neighbouring nodes, and then stop moving. The basic question is whether this is possible at all, from every starting configuration.

1. Algorithm: Activated only when crowded. (Navarra et al. 2025) (SILBOT, p. 427) The catalogue has not read the full paper, so only its setting is described here. As reported by (Navarra and Piselli 2023b), Navarra, Prencipe, Bonini and Tracolli (Navarra et al. 2025) let each particle see only its six neighbouring nodes, and use **event-driven activation**: a particle is activated (it gets a turn to act, an **activation**) as soon as it has a neighbour, and only then.

1. A particle with no neighbour is never activated, so it stays put.
2. A particle with a neighbour is activated, looks at which of its six neighbouring nodes are occupied, and decides from this view alone whether and where to move.
3. When no particle has a neighbour, no particle is activated again: the task is done and the particles have stopped.

Why it works and what it costs. Event-driven activation gives stopping for free: a scattered configuration is exactly one in which nobody is ever activated again. The difficulty is choosing the local rule so that crowded particles really do spread out instead of moving back and forth forever; the paper proves that its rule always succeeds in finite time. No time bound is recorded in the catalogue.

Why it matters. It shows how much the choice of scheduler can replace memory: with event-driven activation, particles without memory can tell when to stop.

2. Lower bound: A crowded hexagon must spread over three times its area. (SILBOT, p. 427) This simple counting argument is ours; the catalogue records no lower bound. In the triangular lattice every small triangle of three mutually neighbouring nodes can hold at most one particle of a scattered configuration. Counting triangles, a region with N nodes can hold at most about $N/3$ scattered particles. Now start with the n particles packed into a full hexagon of radius r (it has about $3r^2 = n$ nodes). The hexagon of radius $1.5r$ around the same centre has about $6.75r^2$ nodes, so it can hold only about $2.25r^2 = 0.75n$ scattered particles.

Why it holds. So about $n/4$ particles must end more than $1.5r$ from the centre, having started within r of it: each travels at least $r/2$, which is about $\sqrt{n}$ nodes. That is at least about $n\sqrt{n}$ moves in total, and, if the scheduler lets each particle act once per **round** (an interval of time in which every particle is activated at least once), about $\sqrt{n}$ rounds. How close the known algorithm comes to these numbers is not recorded.

Why it matters. It shows that scattering, unlike gathering, forces the occupied region to grow in area, which costs real movement.

What the community started looking at

Scattering was posed for **silent, oblivious** particles, where the question is solvability without memory or messages (Navarra et al. 2025).

Claude (Anthropic) was used in cataloguing the problems.

Relations to other problems

- **Related: Scattering.** Robots in the plane scatter to distinct points; particles must scatter so that no two are neighbours.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time, Moves) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Memory → Scheduler → Orientation → Randomness → Initial shape → Connectivity during execution.

SILBOT STANDARD SETTING ●● p. 427

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	Moves	p.
SILBOT STANDARD SETTING	● unknown *finite* vs —	● unknown — vs —	427

21.9.1 SILBOT

the root: no assumptions STANDARD SETTING DRAFT

Silent oblivious particles scatter from any initial configuration.

Assumed: nothing. **Not assumed:** memory: oblivious, silent particles (SILBOT); scheduler: weak: non-atomic activations, an action may interleave with others; orientation: no common orientation and no chirality; randomness: deterministic; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	none known Bounds not in the abstract.	*finite* (Navarra et al. 2025)	● unknown
Moves	none known —	—	● unknown

Results.

- Navarra et al. (2025) [current best]: time *finite*. *Assumes:* event-driven activation: a particle activates as soon as it has a neighbour; 1-hop view. Introductory journal version and current best.

Notes. Event-driven activation is stronger than the weak scheduler of this setting, so the placement overstates the result. The assumptions are as reported by (Navarra and Piselli 2023b); the full text of (Navarra et al. 2025) was not reachable.

Claude (Anthropic) was used in cataloguing the problems.

21.10 Energy distribution

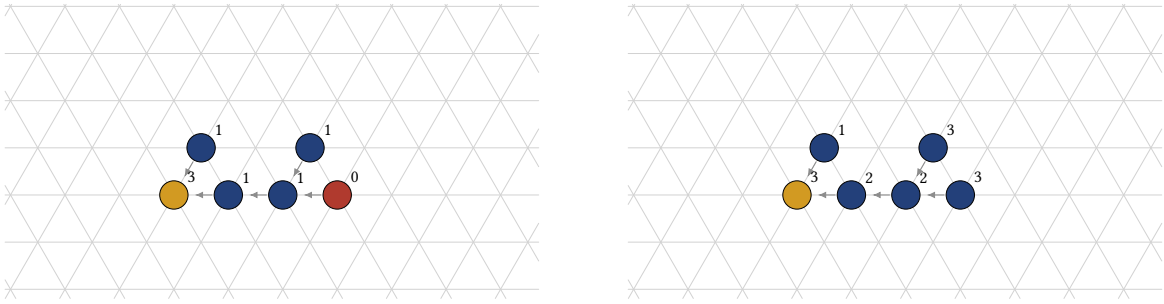
Only some particles reach the energy source; particles must share energy so that every action is paid for.

First posed in 2020 (Daymude et al. 2021). arXiv 2020; ICDCN 2021.

Problem statement and measures

Each action costs energy, and only s particles, the **sources**, can harvest energy from outside. Particles must distribute energy locally so that every particle can act, or so that a given energy-agnostic algorithm runs correctly. **Time** is in rounds.

Measures: **Time**, asynchronous rounds.



(a) Start: one particle has an empty battery

(b) Final: no particle stressed, all can act

Figure 21.10: Energy distribution, following (Daymude et al. 2021): numbers are battery levels (capacity 3) and every action costs 1; only the source (amber) harvests energy, 1 per round. The particles form a tree hanging from the source (arrows point to parents); a particle with energy to spare passes 1 to a child whose battery is not full. A particle with an empty battery is stressed (red): it raises a flag that climbs to the root (outlined), the root then inhibits its tree (grey: inhibited particles stop spending energy on their own actions), and energy flows down until the stress is gone; then the flags clear and everyone acts again. One frame per round.

A first look

Think of a colony of bacteria where only cells at the edge of the colony get food and must pass nutrients inward so that every cell can keep working. The floor is the **triangular lattice**: the endless graph whose **nodes** are the corners of a tiling of the plane by equal triangles; each node is joined by an edge to six others, its **neighbours**. A **particle** occupies one node and moves in two actions: an **expansion** into an empty neighbouring node, after which it occupies both, then a **contraction** into one of the two; so one expansion and one contraction move it by one node. The **system** of n particles is connected; only s of them, the **sources**, can take energy from outside. Every action (a move, or any step of some other algorithm) costs energy; each particle has a **battery** of fixed capacity and can pass a limited amount of energy to a neighbour each time it is activated (an **activation**). The goal is that no particle stays **stressed** (short of energy for its next action) for long, and that the system as a whole keeps acting; we count **rounds** (intervals of time in which every particle is activated at least once). Particles have constant memory and no compass.

1. Algorithm: Share energy down a forest, and hold back when a particle is stressed. (Daymude et al. 2021) (Unfair adversary, p. 430)

Claude (Anthropic) was used in cataloguing the problems.

1. **Set-up.** The sources are **roots**. Every other particle picks a neighbour that is already in a tree as its *parent*, so the system becomes a forest of trees hanging from the sources.
2. **Signal.** A stressed particle raises a 'stress' flag. A particle with a stressed child raises its flag too, so the signal climbs to the root.
3. **Inhibit.** A root that hears of stress raises an 'inhibit' flag, which spreads down its whole tree: **inhibited** particles stop spending energy on their own actions.
4. **Share.** Roots harvest energy; any particle with enough energy passes some to a child whose battery is not full.
5. **Use.** A particle that is not inhibited and has enough energy performs its next action. Flags are cleared when the stress is gone.

Why it works and what it costs. Energy flows down the trees like water through pipes, and the inhibit signal stops particles near the source from using everything before it reaches the far ends. (Daymude et al. 2021) prove that no particle stays stressed for more than $O(n)$ rounds; without the inhibit step, their simulations show particles staying stressed forever. With a constant number of sources this is optimal, by the next idea. (Weber et al. 2023) show how to run any algorithm designed without energy limits on top of this: one that needs T rounds then needs $O(n^2\,T)$ rounds.

Why it matters. It was the first model of energy as a limited resource in programmable matter, and the stress/inhibit signals are modelled on how bacterial biofilms share nutrients.

2. Lower bound: Energy enters only through the sources. (Daymude et al. 2021) (Unfair adversary, p. 430) Start with every battery empty and every particle needing a full battery. Then the system needs n full batteries' worth of energy in total. Energy enters only at the s sources, and each source harvests at most a fixed amount per activation. An unkind scheduler activates each source only once per round.

Why it holds. So at most a constant times s units of energy enter per round, and filling n batteries takes at least a constant times n/s rounds: $\Omega(n/s)$ (Daymude et al. 2021) (Theorem 9). With a constant number of sources this matches the $O(n)$ algorithm; with many sources there is a gap of a factor s between n/s and n that has not been closed.

Why it matters. It is a supply argument rather than a distance argument: the bottleneck is how fast energy can enter, not how far it must travel.

What the community started looking at

Energy distribution was inspired by biofilms (Daymude et al. 2021); the later question was a **general transformation** of energy-agnostic algorithms and its overhead under unfair adversaries (Weber et al. 2023).

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Memory → Scheduler → Orientation → Randomness → Initial shape → Connectivity during execution.

No assumptions

+ constant memory, + sequential

Unfair adversary STANDARD SETTING ● p. 430

Claude (Anthropic) was used in cataloguing the problems.

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	p.
Unfair adversary STANDARD SETTING	● polynomial gap $O(n)$ vs $\Omega(n/s)$	430

21.10.1 Unfair adversary

Adds + constant memory, + sequential STANDARD SETTING VERIFIED

Energy can be distributed in $O(n)$ rounds, against an $\Omega(n/s)$ lower bound; any conforming energy-agnostic algorithm becomes energy-constrained with $O(n^2)$ rounds of overhead per action, even under an unfair adversary.

Assumed: memory: constant memory and local communication; scheduler: sequential: one particle active at a time. **Not assumed:** orientation: no common orientation and no chirality; randomness: deterministic; initial shape: connected, holes allowed; connectivity during execution: the system must stay connected.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(n/s)$ (Daymude et al. 2021) $O(n)$ against $\Omega(n/s)$: a factor s.	$O(n)$ (Daymude et al. 2021)	● polynomial gap

Results.

- Daymude et al. (2021) [lower bound]: time $\Omega(n/s)$. *Bottleneck:* distance. *Holds when:* Theorem 9; s energy sources.
- Daymude et al. (2021) [current best]: time $O(n)$. Introductory and current best (Theorem 8); a self-stabilising spanning forest tolerates crash failures.
- Weber et al. (2023) [also]: time $O(n^2\, T_A(n))$. *Assumes:* unfair adversary. Theorem 4: $T_A(n)$ is the running time of the energy-agnostic algorithm. Composes with the concurrency control framework.

Notes. No orientation is assumed. Checked against the full texts.

21.11 Shape formation by hybrid programmable matter

One finite-automaton agent rearranges passive tiles into a target shape.

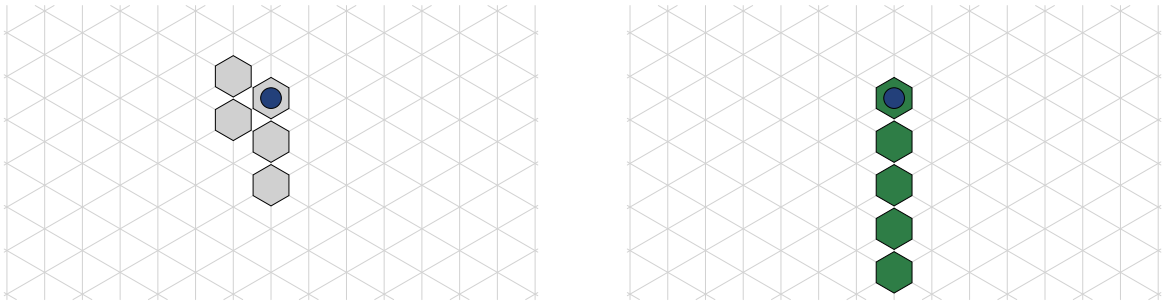
First posed in 2017 (Gmyr et al. 2020). First presented at EuroCG 2017 (non-archival); DNA 24 (2018); Natural Computing 2020. The work began at a Dagstuhl seminar in 2016.

Problem statement and measures

n passive hexagonal tiles (rhombic dodecahedra in 3D) occupy nodes of a lattice and form a connected structure. A single active agent, a deterministic finite automaton, starts on a tile, must always stand on or next to a tile, sees its node and the adjacent nodes, and in each **round** lifts a tile, places the tile it carries, or moves to an adjacent node; it carries at most one tile, and the tiles (with the carried tile counted at the agent's node) must stay connected. It must rearrange the tiles into a target shape (a line, a triangle, or an intermediate structure such as a block or a tree). **Time** counts the agent's rounds.

Claude (Anthropic) was used in cataloguing the problems.

Measures: **Time**, rounds; in each round the agent performs one Look-Compute-Move step.



(a) Start: connected tiles, agent on a tile

(b) Final: the tiles form a line

Figure 21.11: Shape formation by hybrid programmable matter, the line algorithm of (Gmyr et al. 2020): a single agent (blue ring), a finite automaton, walks on passive hexagonal tiles and carries at most one tile (amber). It first walks south as far as there are tiles; then it repeatedly searches, walking N, NW or SW (in that order) while there is a tile there, until it stands on a top-left tile (no tile to its N, NW or SW); it lifts that tile, moves once SE, walks S to the first empty node and places the tile there. One frame per round (move, lift or place); the tiles, with the carried tile counted at the agent, stay connected. It stops when the tiles form a single north-south line.

A first look

Imagine a single small machine on a beach of hexagonal tiles, which can pick up one tile at a time, carry it, and put it down elsewhere, and must rebuild the whole pile into a given shape. Here the machine is an **agent** and the beach is the **triangular lattice**: the infinite tiling of the plane by equilateral triangles, whose corners are its **nodes**. n passive hexagonal **tiles** sit on nodes, one tile per node; together they form a **structure**, which must be **connected**: from every tile one can reach every other tile by moving between tiles on adjacent nodes. The agent has only a fixed finite memory (it is a **finite automaton**) and starts on a tile. Unlike in the amoebot model, the tiles cannot expand or contract; only the agent moves. In each **round** it does one thing: it moves to an adjacent node, always staying on or next to a tile, or it lifts the tile under it, or it places the tile it carries. The tiles (counting the carried one) must stay connected. The goal is a **target shape** such as a straight **line** of tiles (tiles on consecutive nodes in one direction) or a triangle; time is the number of rounds, and D is the **diameter** of the initial structure, the largest distance between two of its tiles, counted in moves between adjacent nodes.

1. Algorithm: Build a line by moving top-left tiles down. (Gmyr et al. 2020) (Planar, p. 434) Here the line runs north-south.

1. The agent walks south (down) as long as there is a tile below it.
2. **Search.** It walks north, north-west or south-west (in that order of preference) as long as one of these has a tile. It stops on a **top-left tile**: a tile with no tile to its north, north-west or south-west.
3. **Move.** It lifts that tile, moves once to the south-east, walks south until it reaches an empty node, and places the tile there.
4. It repeats the search. When a search finds no tile to the east or west, the structure is a line and the agent stops.

Why it works and what it costs. The tile it moves is never needed to hold the structure together, because it is placed back where it reconnects any tiles it might have separated. Tiles only move south-east and south, and each tile is carried across at most about $2n$ nodes, so there are $O(n^2)$ rounds in which the

Claude (Anthropic) was used in cataloguing the problems.

agent carries a tile; a bookkeeping argument bounds the searching rounds by the same amount. Total: $O(n^2)$ rounds (Gmyr et al. 2020). The same paper builds a more compact intermediate shape (a **block**) in $O(nD)$ rounds, and from it a triangle in $O(nD)$ rounds, which is $O(n^{3/2})$ when D is about $\sqrt{n}$.

Why it matters. It shows an agent with constant memory can reshape a large structure, using simple intermediate shapes it can navigate.

2. Lower bound: Carrying one tile at a time is slow. (Gmyr et al. 2020) (Planar, p. 434) Start from a compact structure of n tiles, of diameter about $\sqrt{n}$, and ask for a line. As in shape formation, a constant fraction of the tiles must end at distance proportional to n from where they started. The agent carries one tile at a time and moves to one adjacent node per round, so each such tile costs it about n rounds of carrying.

Why it holds. So a constant fraction of n tiles times about n rounds each gives $\Omega(n^2)$ rounds (Gmyr et al. 2020), and the line algorithm above is optimal. A similar argument gives $\Omega(nD)$ for the block, so the block algorithm is optimal as well.

Why it matters. With a single agent, the total distance tiles must travel directly bounds the time.

3. Impossibility: An agent with finite memory cannot find a safely removable tile. (Gmyr et al. 2020) (Planar, p. 434) The naive approach would be: find a tile whose removal keeps the structure connected (a **safely removable** tile), carry it to its place in the target shape, repeat. Consider a large hollow hexagon of tiles, a closed ring. An agent with s memory states walking along a long side of the ring sees the same thing in every round, so after at most s rounds it repeats its behaviour; it cannot count the length of a side. So it cannot tell this ring apart from a structure that looks the same nearby but is not closed far away, where the same tile is no longer safely removable. (Gmyr et al. 2020) (Theorem 1) make this precise with hollow hexagons and spirals; the full proof is longer.

Why it holds. No finite automaton can, without lifting tiles, always stop on a safely removable tile. With a single **pebble**, a marker it can drop and pick up, it can find one in $O(n^2)$ rounds (Gmyr et al. 2020) (Theorem 2). This is why the algorithms above move tiles only in ways that are always locally safe.

Why it matters. It shows how far a finite memory falls short of seeing the whole picture, and why a single marker helps so much.

What the community started looking at

The hybrid model asked what **one weak active agent** can build from passive material; the first answer was that it cannot always find a tile that is safe to remove, so it builds intermediate structures by local moves instead (Gmyr et al. 2020), and then whether the same holds in 3D (Hinnenthal et al. 2024).

Relations to other problems

- **Related: Shape formation in programmable matter.** In the hybrid model the tiles are passive and one active agent moves them; in the amoebot model every particle is active.

Assumptions specific to this problem

Space

The hybrid model has a few active agents with the power of finite automata and many passive tiles that the agents pick up, carry and place (Hinnenthal et al. 2024). It is motivated by the observation that "while fabricating computing DNA robots remains challenging, producing simple passive tiles from folded

Claude (Anthropic) was used in cataloguing the problems.

DNA strands is efficient and scalable", and an agent can be reused after a task (Hinnenthal et al. 2024). The earliest paper of the model in this atlas, by Gmyr et al. (DNA 24, 2018), works in the plane (Gmyr et al. 2020). Hinnenthal, Liedtke and Scheideler moved to three dimensions, with rhombic-dodecahedral tiles on the face-centred cubic lattice. There "complexity increases significantly, with instances where any tile movement can locally disconnect the structure" (Hinnenthal et al. 2024). Coating in 3D followed (Kostitsyna et al. 2024).

Level	Value	Formal definition
0	**3D**	Tiles occupy nodes of the face-centred cubic lattice; planar structures are a special case.
1	**2D**	Tiles occupy nodes of the triangular lattice.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Time) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Space.

Three-dimensional ● p. 433

+ 2D

Planar STANDARD SETTING ● p. 434

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Time	p.
Three-dimensional	● unknown $O(n^3)$ vs —	433
Planar STANDARD SETTING	● tight $O(n^2)$ vs $\Omega(n^2)$	434

21.11.1 Three-dimensional

the root: no assumptions VERIFIED

In 3D, one agent turns any connected tile structure into a low-diameter icicle in $O(n^3)$ rounds.

Assumed: nothing. **Not assumed:** space: three-dimensional lattice.

Measure	Lower bound	Best upper bound	Status
Time	none known No lower bound recorded.	$O(n^3)$ (Hinnenthal et al. 2024)	● unknown

Results.

- Hinnenthal et al. (2024) [current best]: time $O(n^3)$. *Assumes:* the agent starts carrying a tile; a consistent compass; the agent may pass through tiles. Introductory and current best (Theorem 14); matches the prior runtime of 3D line formation.

Claude (Anthropic) was used in cataloguing the problems.

Notes. Checked against the full text.

21.11.2 Planar

Adds + 2D **to** Three-dimensional (p. 433) STANDARD SETTING VERIFIED

One agent builds a line or a tree in $O(n^2)$ rounds and a block in $O(nD)$ rounds, which is asymptotically optimal, and then a triangle in $O(nD)$ rounds.

Assumed: space: the triangular lattice (planar). **Not assumed:** —.

Measure	Lower bound	Best upper bound	Status
Time	$\Omega(n^2)$ (Gmyr et al. 2020); *impossible* (Gmyr et al. 2020)	$O(n^2)$ (Gmyr et al. 2020)	• tight
	$\Theta(n^2)$ for the line and $\Theta(nD)$ for the block in the worst case; the lower bounds are sketched.		

Results.

- Gmyr et al. (2020) [lower bound]: time $\Omega(n^2)$. *Bottleneck:* one active agent. *Holds when:* line formation from an initial configuration of diameter $O(\sqrt{n})$: a constant fraction of the tiles must move a distance linear in n; a single agent carrying one tile at a time. Stated after Theorem 3 with a one-line argument ("not hard to see"); a similar argument gives $\Omega(nD)$ for a block.
- Gmyr et al. (2020) [lower bound]: time *impossible*. *Bottleneck:* memory. *Holds when:* the agent must stop on a safely removable tile (one whose removal keeps the structure connected) without lifting any tile; no pebble; instances: hollow hexagons and a spiral of long segments. Theorem 1: no finite automaton finds a safely removable tile in general; with one pebble it takes $O(n^2)$ rounds (Theorem 2). This rules out the naive approach, not shape formation.
- Gmyr et al. (2020) [current best]: time $O(n^2)$. *Assumes:* no compass needed: the agent keeps its orientation relative to its initial one. Introductory and current best: line in $O(n^2)$ (Theorem 3), block of diameter $O(D)$ in $O(nD)$ (Theorem 4), tree inside the initial convex hull in $O(n^2)$ (Theorem 5), triangle in $O(nD)$ (Theorem 6), hence $O(n^{3/2})$ when $D = O(\sqrt{n})$. With several agents, line formation was sped up almost linearly for a small number of agents in simulations only.

Notes. D is the diameter of the initial tile set. Checked against the full text of (Gmyr et al. 2020) (Natural Computing version).

21.12 Centralised transformations of modular robots

A centralised algorithm must transform one connected shape of n modules on the square grid into another using local moves.

First posed in 2017 (Michail et al. 2019). arXiv 2017; JCSS 2019.

Problem statement and measures

n **modules** (unit squares, the particles of this model) occupy **cells** of the square grid and form a connected **shape**; modules in cells that share a side are **neighbours**, and the shape is connected when every module can be reached from every other through neighbours. A **centralised algorithm** must transform it into a given target shape with the same number of modules, using only the allowed local moves, keeping the shape connected where required. **Moves** counts sequential moves; **time** is parallel time when several moves may happen at once.

Claude (Anthropic) was used in cataloguing the problems.

Measures: **Moves**, sequential moves; **Time**, parallel time.



(a) Start: a connected shape of 5 modules

(b) Final: the modules form a line

Figure 21.12: Centralised transformation with rotations and slidings, the universal strategy of (Michail et al. 2019): repeatedly pick a module on the outside whose removal keeps the rest connected and walk it around the outside, rotating at corners (a quarter turn around a neighbour, the swept cells being empty) and sliding along straight stretches (one cell along two neighbours), to the end of a line growing to the right along the bottom row (dashed); a target shape is reached by also playing its own plan to the line backwards (not shown). One frame per move; a rotation is drawn along its arc.

A first look

Picture a set of square magnets on a chessboard, all touching one another, that you are allowed to rearrange only by small local moves, and ask: can one arrangement be turned into another, and how many moves does it take? Here the magnets are n square **modules** and the squares of the board are **cells** of the infinite **square grid**. Modules in cells that share a side are **neighbours**, and the modules form a **connected shape**: every module can be reached from every other through neighbours. A **centralised algorithm** sees everything and decides the moves. Unlike amoebot particles, which move by expanding and contracting, a module moves by **rotation** (turning a quarter circle around a neighbour, ending on a diagonal cell) or by **sliding** (moving one cell along two neighbouring modules); a **move** is allowed only if the cells it sweeps through are empty. The goal: transform the shape into a given **target shape** with the same number of modules, and count moves (one at a time) or **parallel time** (the number of steps when several moves may happen at once).

1. Algorithm: Everything turns into a line, and moves can be undone. (Michail et al. 2019) (Rotation and sliding, p. 438) With rotations and slidings both allowed:

1. Find a module on the outside of the shape whose removal keeps the rest connected; every connected shape with at least two modules has at least two.
2. That module can walk around the outside of the rest, rotating at corners and sliding along straight stretches, always staying a neighbour of some module.
3. Walk it to the end of a horizontal **line** (modules in consecutive cells of one row) growing from the shape's lowest right-most module; repeat until all n modules form the line.
4. To reach a target shape B, also plan $B \to$ line, then play that plan backwards: every rotation and sliding can be undone by the opposite move.

Why it works and what it costs. So any two connected shapes with the same number of modules can be transformed into each other without ever disconnecting (Michail et al. 2019). Each module walks at most about the perimeter of the shape, so the plan uses $O(n^2)$ moves, and that is the best possible in the

Claude (Anthropic) was used in cataloguing the problems.

worst case (the third idea). Allowing many moves at once, the same paper reaches $O(n)$ parallel time, also best possible.

Why it matters. Going through a canonical shape and using reversibility is the standard way to prove that a set of moves is universal.

2. Impossibility: Rotations never change a module's chessboard colour. (Michail et al. 2019) (Rotation only, p. 437) Colour the grid like a chessboard. A rotation moves a module to a cell diagonal to its old one, which has the same colour. So with rotations alone, the number of modules on black cells never changes, and neither does the number on white cells. Example: a horizontal line of 3 modules on black, white, black cells has two on black; a shape of 3 modules with two on white cells can never be reached from it.

Why it holds. Two shapes with different colour counts can never be transformed into each other by rotations (Michail et al. 2019). Equal colour counts are not enough either: some shapes have no rotation available at all and are stuck. If disconnecting is allowed, the paper shows that deciding whether a transformation exists takes polynomial time; if connectivity must be kept, the decision is in PSPACE.

Why it matters. An **invariant**, a quantity that no move changes, is the simplest tool for proving that a transformation is impossible.

3. Lower bound: A staircase is far from a line. (Michail et al. 2019) (Rotation and sliding, p. 438) Take a *ladder*: a diagonal staircase two modules wide, climbing one row every two modules. Ask for a horizontal line. Wherever the line is placed, at least half of the ladder's modules lie on one side of it, at heights $1, 1, 2, 2, \ldots$ up to about $n/4$ from it. Each must cover at least that height, so the total distance is at least $2(1 + 2 + \cdots + n/4)$, which is about $n^2/16$.

Why it holds. One rotation or sliding moves one module by a constant distance, so it reduces this total by at most a constant. Hence $\Omega(n^2)$ moves are needed, even if the shape may disconnect (Michail et al. 2019) (Theorem 18), and the line-based plan is optimal. The same measure, total distance under the best matching of modules to target cells, can be computed in polynomial time, so such lower bounds are easy to evaluate.

Why it matters. The total-distance argument gives quadratic lower bounds for every model where one entity moves one step at a time.

What the community started looking at

The first question was **which mechanisms are universal** and how hard it is to decide transformability (Michail et al. 2019); then how far stronger mechanisms reduce the number of moves (Almethen et al. 2020) (Akitaya et al. 2021).

Relations to other problems

- **Related: Shape formation in programmable matter.** A central controller plans the moves, so the question is which mechanisms are universal and how many moves they need, not how particles coordinate.

Assumptions specific to this problem

Mechanism

Claude (Anthropic) was used in cataloguing the problems.

Michail, Skretas and Spirakis chose two movements that are "easily implementable by existing technology": a module may *rotate* 90° around a neighbour, or *slide* one cell over two neighbours (Michail et al. 2019). Rotation and sliding together had been studied before for metamorphic robotic systems (Dumitrescu et al. 2004) (as cited by Michail et al. 2019). Rotation alone is the weaker mechanism. Michail et al. showed that deciding transformability with rotations alone is in P, and in PSPACE when connectivity must be kept; adding sliding makes any two connected shapes of the same size transformable into each other with $\Theta(n^2)$ moves (Michail et al. 2019). Almethen, Michail and Potapov later added a line-pushing mechanism that contains both as special cases and is faster (Almethen et al. 2020).

Level	Value	Formal definition
0	**rotation**	A module may rotate 90° around an adjacent module, if the cells it sweeps are empty.
1	**rotation + sliding**	In addition, a module may slide one cell along a line of modules.

Case tree

The root assumes nothing; going down, each edge adds the assumptions written in grey (+) or leaves an axis unassumed. Bold entries are catalogued settings with their status per measure (Moves, Time) and page; italic entries are branch points, named after the axis on which their branches differ. Axis order: Mechanism.

Rotation only STANDARD SETTING ●● p. 437

+ rotation + sliding

Rotation and sliding ●● p. 438

What is known

The best upper bound against the best lower bound in each catalogued setting, in tree order. The settings follow in detail.

Setting	Moves	Time	p.
Rotation only STANDARD SETTING	● unknown *decidable in P* vs —	● unknown — vs —	437
Rotation and sliding	● tight $O(n^2)$ vs $\Omega(n^2)$	● tight $O(n)$ vs $\Omega(n)$	438

21.12.1 Rotation only

the root: no assumptions STANDARD SETTING VERIFIED

With rotations alone, deciding whether one shape can be transformed into another is in P; with connectivity preserved it is in PSPACE.

Assumed: nothing. **Not assumed:** mechanism: rotation only.

Measure	Lower bound	Best upper bound	Status
Moves	none known —	*decidable in P* (Michail et al. 2019)	● unknown
Time	none known —	—	● unknown

Claude (Anthropic) was used in cataloguing the problems.

Results.

- Michail et al. (2019) [current best]: moves *decidable in P*. Introductory and current best: transformability with rotations, allowing disconnection, is decidable in polynomial time (Theorem 9); preserving connectivity it is in PSPACE (Theorem 10).

Open questions.

Q1. What is the complexity of connectivity-preserving transformability with rotations alone? The paper leaves it open (Section 7) (Michail et al. 2019).

Notes. Checked against the full text.

21.12.2 Rotation and sliding

Adds + rotation + sliding **to** Rotation only (p. 437) VERIFIED

Rotation and sliding are universal for connected shapes: $\Theta(n^2)$ moves sequentially and $O(n)$ parallel time, both optimal.

Assumed: mechanism: rotation and sliding. **Not assumed:** —.

Measure	Lower bound	Best upper bound	Status
Moves	$\Omega(n^2)$ (Michail et al. 2019) $\Theta(n^2)$ moves.	$O(n^2)$ (Michail et al. 2019)	• tight
Time	$\Omega(n)$ (Michail et al. 2019) $\Theta(n)$ parallel time.	$O(n)$ (Michail et al. 2019)	• tight

Results.

- Michail et al. (2019) [lower bound]: moves $\Omega(n^2)$. *Bottleneck:* distance. *Holds when:* Theorem 18: transforming a ladder into a line; holds even without preserving connectivity.
- Michail et al. (2019) [lower bound]: time $\Omega(n)$. *Bottleneck:* distance. *Holds when:* Proposition 15 (arXiv version); connectivity is preserved.
- Michail et al. (2019) [current best]: moves $O(n^2)$, time $O(n)$. Introductory and current best; connectivity preserved.
- Almethen et al. (2020) [also]: moves $O(n \log n)$. *Assumes:* line-pushing mechanism: one move shifts a whole line of modules; the shape may disconnect. A stronger mechanism; a suspected $\Omega(n \log n)$ lower bound is left open.

Open questions.

Q1. Prove the suspected $\Omega(n \log n)$ lower bound for universal transformations with line pushing (Almethen et al. 2020).

Q2. Distributed universal transformations with line pushing (Almethen et al. 2020).

Notes. With pivoting square modules (facet-connected), universal reconfiguration between any two configurations needs only five extra helper modules and $O(n^2)$ pivot moves, worst-case optimal (Akitaya et al. 2021). Checked against the full texts.

Claude (Anthropic) was used in cataloguing the problems.

Appendices

Appendix A

All open questions

The most consequential

A ranking by downstream impact: how many problems and settings a solution would feed, through derived problems, related problems and the carry-over of algorithms and lower bounds; weighed with how long a question has been open and how far apart its bounds are. The ranking is the maintainer's judgement; the counts in the table below are computed from the catalogue.

How the ranking works

A solution propagates along three kinds of links: a derived problem inherits lower bounds from its parent and gives algorithms back to it; problems that use another as a first step inherit its running time; and inside one problem, an algorithm for weaker assumptions carries over to stronger settings while a lower bound carries over to weaker ones. The ranking weighs this reach with how long a question has been open and how far apart its bounds are. It is a judgement; the open questions page shows the underlying counts.

1. **Collective exploration of trees.** Is the competitive ratio of collective tree exploration $O(\log k)$? The bounds are $\Omega(\log k / \log\log k)$ (Disser et al. 2020) against $O(\log^2 k)$ (Cosson and Massoulié 2026), a preprint. In its sharpest form: can n agents explore an anonymous tree of height D in $O(D^2)$ rounds with local communication, and is $\Omega(D^2)$ necessary for every D (Disser et al. 2020)?

   Why it matters. Dispersion is derived from it: with $k = n$ every dispersion algorithm explores, and dispersion's $\Omega(D^2)$ tree bound is borrowed from it (Augustine and Moses Jr. 2018). Leader election, independent and dominating sets, spanning and BFS trees by agents all start by dispersing, so their running times inherit any change. Black hole search and dynamic exploration are team exploration under faults.

   Settings: Collaborative exploration, Collaborative exploration, Collaborative exploration, Dispersion.

2. **Optimal-time rendezvous of labelled agents.** Find an optimal-time rendezvous algorithm for two labelled agents in arbitrary graphs: the best bound is $\tilde{O}(n^5 \log \lambda)$ (Ta-Shma and Zwick 2014) against $\Omega(n^2 + \log \lambda)$ (Dessmark et al. 2006), where λ is the smaller label; the question has been open since 2006 (Pelc 2019).

   Why it matters. Rendezvous is the two-agent core of gathering in graphs, and its algorithms rest on the same universal exploration sequences as treasure hunt; a tight bound would sharpen gathering, treasure hunt and leader election by agents.

   Settings: Rendezvous.

3. **How much memory exploration needs.** Can an agent explore a tree and stop with asymptotically less memory than exploration with return? The bounds are $\Omega(\log\log\log n)$ (Diks et al. 2002) against $O(\log n)$ (Ambühl et al. 2011). More broadly: what is the time of memory-optimal exploration?

   Why it matters. Memory lower bounds for agent problems, dispersion among them, come from exploration arguments; exploration is also the oldest question in the area.

Settings: Graph exploration by a single agent, Graph exploration by a single agent.

4. **Dispersion with optimal memory and termination.** Can k agents disperse in $O(k)$ rounds with $O(\log k)$ bits, independent of Δ, and terminate without knowing k, m or Δ? The best algorithm uses $O(\log(k+\Delta))$ bits (Kshemkalyani et al. 2025a); the lower bound is $\Omega(\log k)$ (Augustine and Moses Jr. 2018).

   Why it matters. Dispersion is the first step of most agent algorithms and has the most open questions in the atlas; a $\log \Delta$ memory lower bound, or its removal, would settle a question shared by all of them.

   Settings: Dispersion, Dispersion.

5. **The asynchronous landscape of robots in the plane.** Can robots with internal lights meet in ASYNC (Flocchini et al. 2016) (Masuzawa et al. 2026)? Can oblivious robots scatter in ASYNC (Bramas and Tixeuil 2016)? Is point convergence with limited visibility solvable in ASYNC without measurement errors (Kirkpatrick et al. 2024)?

   Why it matters. These questions decide where models separate under asynchrony (see the model grid on the plane model page); their answers carry over to every plane problem, and the first would settle five separations that rest on an unproved claim (Flocchini et al. 2023).

   Settings: Gathering, Scattering, Convergence.

6. **The time of arbitrary pattern formation.** Bound the number of epochs an arbitrary pattern formation algorithm needs in ASYNC; only solvability is known (Cicerone et al. 2019).

   Why it matters. Pattern formation has the largest family of special cases in the atlas (circle and uniform circle formation, mutual visibility, k-grouping, partitioning and more), and a time bound would carry over to them.

   Settings: Pattern formation.

7. **How many faults can be tolerated.** "The maximum number of faults under which a solution is still feasible, for gathering and other tasks, has yet to be established" (Agmon and Peleg 2006), in particular in ASYNC; and polynomial-time Byzantine gathering in graphs with f Byzantine agents and teams of close to $f+1$ good agents (Bouchard et al. 2016).

   Why it matters. Fault thresholds recur across gathering, black hole search, dispersion, evacuation and search; the arguments that bound one usually transfer.

   Settings: Gathering, Gathering in graphs.

8. **Leader election in programmable matter.** Deterministic leader election without chirality in linear time (the best is $O(L_{out}n^2)$ Emek et al. 2019), and in $O(L_{out}+D)$ rounds under weaker schedulers and without disconnection (Dufoulon et al. 2021).

   Why it matters. Shape formation, coating and convex hull formation start by electing a leader, so their deterministic bounds depend on it.

   Settings: Leader election in programmable matter, Leader election in programmable matter.

9. **Exact constants for evacuating a disk.** Close the gap for two face-to-face robots: $3+\pi/6+\sqrt{3} \approx 5.255$ (Czyzowicz et al. 2020) against ≈ 5.6234 (Disser and Schmitt 2019).

   Why it matters. The question is old and well studied; its techniques shape the whole search family, though its reach stays inside it.

   Settings: Evacuation.

Claude (Anthropic) was used in cataloguing the problems.

10. Compression and its mixing time. What happens for $2.17 \leq \lambda \leq 2 + \sqrt{2}$, and how fast does the chain compress (Cannon et al. 2016)?

Why it matters. Separation, alignment and bridging generalise compression, so an answer, and especially a mixing-time technique, would transfer to all three.

Settings: Compression.

Who works on what

Counted by distinct authors of the papers the atlas cites, gathering in the plane (64 people), exploration by one agent (57) and dispersion (42) lead, followed by convergence, collaborative exploration and evacuation. The counts measure the atlas's coverage rather than the whole literature: gathering and exploration have far larger literatures than the atlas cites, and the youngest problems (flocking, alignment, subgraph counting) have a single paper each. Counts as of version 2026-10-10.

Every open question

Every open question recorded in the atlas, grouped by problem and ordered along each problem's assumption tree.

Graph exploration by a single agent

Arbitrary graphs, perpetual exploration (p. 82)

Q1. What is the time of memory-optimal perpetual exploration, and is there a trade-off between memory and time?

Trees, exploration with stop (p. 84)

Q1. Can exploration with stop on trees use asymptotically less memory than exploration with return? (Open problem of Diks et al. 2002.) Equivalently: close $\Omega(\log \log \log n)$ versus $O(\log n)$.

Exploration with return and pebbles (p. 85)

Q1. How much memory is needed to encode the agent's transition function? The naive table needs $(\log n)^{O(1)}$ bits (Remark 1 of Disser et al. 2019).

Exploration with stop when n is known (p. 86)

Q1. Is $\Omega(\log n)$ bits necessary when n is known? The trap arguments (Fraigniaud et al. 2005a) are stated for agents that know nothing.

Randomized exploration when n is known (p. 87)

Q1. What is the memory needed by a randomized agent that must stop correctly with high probability, as a function of n?

Exploration with stop and one pebble (p. 88)

Q1. Is $\Omega(D \log \Delta)$ memory needed for exploration with stop by an agent with one pebble (Fraigniaud et al. 2005b)?

Claude (Anthropic) was used in cataloguing the problems.

Port numbers chosen by the designer (periodic exploration) (p. 89)

Q1. Close the constant gaps: $2n-2$ versus $3.5n-2$ for constant memory, $2.8n-2$ versus $4n-2$ for oblivious agents (Czyzowicz et al. 2012c).

Q2. Trade-offs between period and memory: the only lower bound for agents with memory does not depend on memory.

Directed graphs, one pebble (p. 90)

Q1. Close $\Omega(n \log \Delta_{out})$ versus $O(n\Delta_{out} \log n)$ bits for exploring digraphs with one pebble.

Directed graphs, one pebble, bound on n known (p. 91)

Q1. Improve the $O(N^2 n^6 \Delta_{out}^2)$ running time (Section 5 of Bender et al. 2002).

Directed graphs with labelled nodes (p. 91)

Q1. Close $\Omega(\phi^2 m)$ versus $O(\phi^8 m)$ for online exploration of digraphs with deficiency ϕ.

Temporal graphs, schedule known (p. 93)

Q1. Tight bounds when the underlying graph is a grid, has bounded degree or is planar (Erlebach et al. 2021).

Collaborative exploration

Arbitrary graphs, complete communication (p. 173)

Q1. Is there an $o(k)$-competitive collaborative exploration algorithm for arbitrary graphs? (Stated as open in Cosson and Massoulié 2026.)

Trees, complete communication (p. 174)

Q1. Determine the competitive ratio of synchronous collective tree exploration: between $\Omega(\log k / \log\log k)$ and $O(\log^2 k)$.

Q2. Do randomized algorithms help? A randomized lower bound of $\Omega(\sqrt{\log k}/\log\log k)$ is attributed to Ortolf and Schindelhauer (SPAA 2012), not checked here. Cosson and Massoulié conjecture they do not (Cosson and Massoulié 2025).

Trees with $k = n$ agents (p. 175)

Q1. For $k = n$ and trees of depth D well above $\sqrt{\log n}$ (e.g. $D = \sqrt{n}$), is $\Omega(D^2)$ still necessary?

Large teams, complete communication (p. 175)

Q1. Which team size is necessary and sufficient for $O(D)$-round exploration of graphs of larger diameter? (Stated in Dereniowski et al. 2015.)

Trees, communication by whiteboards only (p. 177)

Q1. Is there a separation between write-read and complete communication? (Cosson and Massoulié 2025) conjecture that centralised communication helps.

Trees, asynchronous, whiteboards only (p. 177)

Claude (Anthropic) was used in cataloguing the problems.

Q1. Is there a distributed asynchronous algorithm with polylogarithmic competitive ratio?

Anonymous graphs with $k = n$ agents (p. 179)

Q1. Can n agents explore an anonymous tree in $o(n)$ rounds, e.g. $O(D^2)$, with local communication? With labelled nodes and global communication D^2 suffices (Disser et al. 2020).

Dispersion

Asynchronous agents (p. 185)

Q1. Can an asynchronous algorithm let every agent **terminate** in polynomial time without knowledge of the graph? The only terminating asynchronous algorithm known uses $O(\Delta^{\min(D,k)+1})$ time (Kshemkalyani 2025); the linear-time algorithms keep settled agents active.

Q2. Semi-synchronous schedulers sit between the two models; no separation or dedicated bound is known.

Arbitrary graph, arbitrary initial configuration (p. 186)

Q1. Can the memory be reduced to $O(\log k)$ bits, independent of Δ, while keeping $O(k)$ time? Equivalently, is there an $\Omega(\log \Delta)$ memory lower bound for deterministic agents with identifiers? The bound $\Omega(\log(k + \Delta))$ is quoted in several papers but only the $\log k$ term is argued.

Q2. Can all agents **terminate explicitly** in $O(k)$ rounds without any knowledge of k, m or Δ? (Sudo et al. 2024) terminates the rooted case in $O(k)$ extra rounds but needs global upper bounds in the general case; (Kshemkalyani et al. 2025a) does not address termination detection for arbitrary initial configurations.

Q3. **Ongoing dispersion** (Kshemkalyani and Ali 2019): agents that have settled must periodically leave (for example to recharge) and the configuration must be re-dispersed; no bounds beyond the original proposal are known.

Trees (p. 188)

Q1. (Augustine and Moses Jr. 2018) asked for $o(n)$-round dispersion on arbitrary trees. Precisely: for $k = n$ and an arbitrary initial configuration, is there an algorithm whose time is bounded by a function of D alone, ideally $O(D^2)$ as in the rooted case? No tree-specific algorithm has appeared since 2018; the linear-time general algorithms give only $O(n)$.

Q2. With global communication, trees admit $O(D \cdot \max(D, k))$ rounds with $O(\max(D, \Delta \log k))$ bits (arXiv version of Kshemkalyani et al. 2020b); whether the $\max(D, k)$ factor can be reduced to D, and whether local communication suffices, is open.

Rooted trees (p. 189)

Q1. For $k < n$ the lower bound $\Omega(D^2)$ no longer applies (only k nodes must be reached). Whether $O(\min(k, D^2))$ time is achievable, and whether it is necessary, has not been addressed.

Q2. For $k = n$ agents rooted on a tree of height D much larger than $\sqrt{\log n}$, is $\Omega(D^2)$ time necessary? The known bound is existential in D (Disser et al. 2020).

Rooted initial configuration (p. 190)

Q1. Is $O(k)$ time achievable with $O(\log k)$ bits? Memory $O(\log(k + \Delta))$ is needed by every known algorithm to remember a port number.

Claude (Anthropic) was used in cataloguing the problems.

Rings and paths (p. 190)

Q1. Anonymous randomized agents disperse from a **rooted** ring configuration with $O(1)$ bits in $O(n)$ rounds (Molla and Moses Jr. 2019). From an **arbitrary** configuration the known algorithm never terminates; is $O(1)$-bit dispersion with termination possible on rings?

Grids (p. 191)

Q1. Can $k \leq n$ agents disperse on a grid in $O(\sqrt{k})$ rounds with **local** communication? Global communication achieves it (Kshemkalyani et al. 2020c); asked again in (Banerjee et al. 2025).

Q2. A lower bound in terms of k for $k = o(\sqrt{n})$ with $O(\log n)$ memory on unoriented grids (Banerjee et al. 2024).

Q3. Unoriented grids with no knowledge of n and with faults, matching the fault-free bounds (Banerjee et al. 2024).

Silent agents (no communication) (p. 192)

Q1. Close the time gap on general graphs: is $O(k + \log L)$ achievable, as on rings?

Q2. Arbitrary (multi-source) initial configurations on general graphs; only rings are solved (Das et al. 2025b).

Q3. Weaker sensing: agents that only learn whether they are alone and whether the count decreased, and agents that wake up at different times (a 2026 ICDCIT paper by Gorain, Mandal and Sharma treats this; its bounds are not recorded here).

Silent agents on rings (p. 193)

Q1. Unoriented rings, and rings where agents cannot detect count changes.

Global communication (p. 194)

Q1. Does global communication buy anything for dispersion? Since $O(k)$ time with $O(\log(k + \Delta))$ bits holds with local communication, a separation could only come from memory ($O(\log k)$ bits), from termination detection, or from the BFS-style question of achieving time close to $O(D)$-dependent bounds for small D.

Q2. (Kshemkalyani et al. 2022) asks for $O(\min(m, k\Delta))$ time with only $O(\log(k + \Delta))$ bits in the global model; this is implied by the later local-model result, so the live version of the question is the memory question above.

Weak Byzantine agents (p. 194)

Q1. Faster Byzantine dispersion on general graphs: the time is dominated by Byzantine gathering, and the authors ask whether building a map before settling is necessary (Molla et al. 2021).

Q2. Tolerating more than $n/2$ weak Byzantine agents on general graphs (possible only on the restricted class so far) (Molla et al. 2021).

Q3. Byzantine dispersion with bounded memory per agent, on trees, with $k < n$, or under asynchrony: nothing is known (Molla et al. 2020) (Chand et al. 2023b).

Weak Byzantine agents on rings (p. 196)

Q1. Keep $O(n)$ time and $O(\log n)$ bits while dropping any of: identifiers in $[1, n]$, knowledge of f, the bound $f \leq (n - 4)/17$, the follow primitive (Molla et al. 2020).

Claude (Anthropic) was used in cataloguing the problems.

Q2. Asynchronous Byzantine dispersion on rings (Molla et al. 2020).

Weak Byzantine agents on grids (p. 196)

Q1. Reduce the memory below $O(n \log n)$ bits on grids with weak Byzantine agents.

Q2. Strong Byzantine agents (able to fake identifiers) on grids (Banerjee et al. 2025).

Crash faults (p. 197)

Q1. Is $O(k)$ time achievable with crash faults, or does fault tolerance cost a factor of Δ? No lower bound beyond the fault-free $\Omega(k)$ is known, even with $O(\log(k + \Delta))$ memory (Chand et al. 2023b).

Q2. Simultaneously $O(\min(m, k\Delta))$ (or better) time and $O(\log(k+\Delta))$ memory for arbitrary configurations, without knowledge of f, ℓ, Δ or m (Chand et al. 2023b) (Pattanayak et al. 2023).

Q3. Crash faults under asynchrony: no result exists.

Crash faults on grids (p. 198)

Q1. Unoriented grids with crashes: close the gap between $O(\sqrt{n} \log n)$ rounds with $O(\sqrt{n} \log n)$ bits and the fault-free $O(\sqrt{n})$, $O(\log n)$ (Banerjee et al. 2024).

Strong Byzantine agents (p. 199)

Q1. Polynomial-time strong-Byzantine dispersion from arbitrary configurations without a trusted agent, and the largest tolerable f.

Q2. Asynchronous versions: a slow agent cannot be distinguished from a silent Byzantine one (Moses Jr. and Redlich 2024).

Anonymous agents with randomness (p. 199)

Q1. Close the gap between $\Omega(k)$ and $\Theta(k^2)$ rounds for anonymous agents from a rooted configuration with $O(\log \Delta)$ bits (Das et al. 2021b).

Q2. Can anonymous randomized agents disperse from an **arbitrary** configuration and then stop, rather than staying active forever? (Molla and Moses Jr. 2019) (Das et al. 2021b)

Q3. On rings the memory bound is $O(1)$; is $O(1)$-bit dispersion with termination possible on rings from arbitrary configurations?

More agents than nodes (p. 200)

Q1. **Deterministic** dispersion with $k > n$ agents with identifiers on static graphs has not been studied: no algorithm and no lower bound beyond $\Omega(k)$ is recorded in the literature surveyed here. (Related work exists only on dynamic rings and dynamic graphs under the name balanced dispersion.)

Q2. Is $O(k)$ time possible for rooted configurations when $k > n$, matching the $k \leq n$ case?

Q3. Terminating dispersion with $k > n$ on general rooted graphs with $O(\log \Delta)$ bits per agent (Open Problem 2 of Molla and Moses Jr. 2019).

Directed graphs (p. 201)

Q1. Close the time gaps; in particular $O((k + f)k\Delta_{out})$ with f crashes (Italiano et al. 2022).

Q2. BFS-based dispersion in the local model, without 1-hop communication (Italiano et al. 2022).

Q3. Dispersion on dynamic directed graphs, and in semi-synchronous and asynchronous settings (Italiano et al. 2022).

Claude (Anthropic) was used in cataloguing the problems.

Directed graphs, 1-hop communication (p. 202)

Q1. A BFS-based algorithm with $O(\log(k + \Delta_{out}))$ bits per agent, as with local communication (Italiano et al. 2022).

Q2. Does $O(kD)$ improve to $O(k + D^2)$ or $O(k)$ with 1-hop communication?

Dynamic graphs (1-interval connected) (p. 203)

Q1. Which combination of communication range and visibility is **necessary** for dispersion on T-interval connected graphs with $T > 1$, where the impossibility proof for $T = 1$ does not apply? (Saxena and Mondal 2025)

Q2. Dynamic graphs with a bounded number of missing edges per round: (Saxena et al. 2025) gives impossibilities on 1-bounded 1-interval connected graphs for several weaker communication and visibility combinations and an $O(k)$ algorithm with global communication and 1-hop visibility; the local-communication case with few missing edges is not settled.

Dynamic rings (p. 204)

Q1. Visibility strictly between none and full: how much must an agent see of a dynamic ring for dispersion to be possible? (Agarwalla et al. 2018)

Q2. Achiral agents on even rings of size at least 6 under vertex permutation and 1-interval connectivity: is $O(n)$ time possible from arbitrary configurations? (Agarwalla et al. 2018); $O(\ell n)$ is known without vertex permutation (Mondal et al. 2025b).

Dynamic graphs with global communication (p. 205)

Q1. Byzantine agents on dynamic graphs (Kshemkalyani et al. 2020a).

Q2. Memory $O(\log k)$ instead of $O(\log \max(k, T))$ on T-path connected graphs when T need not be known (Saxena and Mondal 2025).

Treasure hunt

Anonymous finite graphs (p. 98)

Q1. Treasure hunt when the treasure may appear after the agent starts and the agent does **not** see its entry port: "We do not know if an efficient solution is possible when backtracking is not allowed" (Ta-Shma and Zwick 2014).

Q2. Do strongly universal traversal sequences exist? They exist if and only if treasure hunt has a solution independent of the start time (Ta-Shma and Zwick 2014).

Labelled nodes, locally finite graphs (p. 99)

Q1. Remove the $O(\log d)$ factor between $O(e(d) \log d)$ and $\Omega(e(d))$ (Bouchard et al. 2023a).

Rendezvous

Asynchronous labelled agents (p. 105)

Q1. Find an optimal-cost asynchronous rendezvous algorithm in arbitrary graphs (open problem of Pelc 2019).

Claude (Anthropic) was used in cataloguing the problems.

Labelled agents, arbitrary finite graphs (p. 105)

Q1. Find an optimal-time synchronous rendezvous algorithm for labelled agents in arbitrary graphs (open problem of Pelc 2019).

Labelled agents, graphs with infinite degrees (p. 108)

Q1. Find an optimal rendezvous algorithm, "even for finite graphs" (Pattanayak and Pelc 2024b).

Pattern formation

ASYNC, no agreement on coordinates (p. 256)

Q1. Bound the number of epochs needed by an arbitrary pattern formation algorithm in ASYNC; no time analysis is recorded.

Opaque robots with lights, ASYNC, one axis, rigid (p. 259)

Q1. The minimum number of colours, and formation with chirality only or with no agreement (Bose et al. 2021).

Limited visibility (p. 260)

Q1. Arbitrary pattern formation by oblivious robots with limited visibility from a general (connected) initial configuration (Hahn et al. 2024).

Gathering

Rendezvous, internal light (FSTA), SSYNC, non-rigid (p. 299)

Q1. Rendezvous with internal lights in non-rigid SSYNC when δ is unknown.

Gathering, internal light (FSTA), SSYNC, non-rigid (p. 301)

Q1. Gathering with internal lights in SSYNC from every start, with more than 2 colours, and in ASYNC (Otaka et al. 2026).

Gathering, external light (FCOM), SSYNC, rigid (p. 302)

Q1. Gathering with external lights in ASYNC (Otaka et al. 2026).

Homonymous robots, SSYNC (p. 303)

Q1. What is the minimum number of identifiers for Byzantine-tolerant gathering? (Asahiro and Yamashita 2025)

Gathering, full lights, SSYNC, no multiplicity detection (p. 304)

Q1. Gathering with lights without chirality (Terai et al. 2023).

Gathering, full lights, ASYNC, no multiplicity detection (p. 305)

Q1. Is 3 the minimum number of colours for ASYNC gathering with lights, and can chirality be dropped (Nakai et al. 2021)?

Claude (Anthropic) was used in cataloguing the problems.

Q2. Self-stabilizing gathering of n robots with lights (Heriban et al. 2018).

Rendezvous, external light (FCOM), ASYNC, non-rigid (p. 306)

Q1. Rendezvous with external lights in non-rigid ASYNC when δ is unknown (Flocchini et al. 2016) (Masuzawa et al. 2026).

Rendezvous, external light (FCOM), ASYNC, rigid (p. 307)

Q1. Close the gap between 3 and 6 colours (Masuzawa et al. 2026).

ASYNC with weak multiplicity detection (p. 307)

Q1. How many epochs does oblivious ASYNC gathering with weak multiplicity detection need?

Rendezvous, internal light (FSTA), ASYNC (p. 308)

Q1. Can robots with internal lights solve rendezvous in ASYNC (Flocchini et al. 2016) (Masuzawa et al. 2026)?

Crash faults, SSYNC (p. 309)

Q1. "The maximum number of faults under which a solution is still feasible, for gathering and other tasks, has yet to be established" (Agmon and Peleg 2006), in particular in ASYNC.

Limited visibility, FSYNC, disoriented (p. 313)

Q1. Is $\Omega(\Delta_0^2)$ necessary for every protocol, not only local convex-hull ones?

Fat robots, few robots, no common orientation (p. 313)

Q1. Gathering any number of fat robots without chirality (Agathangelou et al. 2013).

Fat robots, chirality, any number (p. 314)

Q1. Remove the chirality assumption (Agathangelou et al. 2013).

Transparent fat robots, no chirality (p. 314)

Q1. Gathering transparent fat robots from symmetric starts; the paper proves no general impossibility (Chaudhuri and Mukhopadhyaya 2015).

Uniform circle formation

ASYNC, oblivious, disoriented (p. 269)

Q1. Can oblivious robots without lights form a uniform circle in $O(1)$ epochs, with rigid or non-rigid moves? (Feletti et al. 2024)

Opaque robots with lights, ASYNC (p. 270)

Q1. What is the exact minimum number of colours for $O(1)$-epoch uniform circle formation in ASYNC? (Feletti et al. 2024)

Q2. Uniform circle formation with non-rigid moves in the luminous opaque model (Feletti et al. 2024).

Claude (Anthropic) was used in cataloguing the problems.

Convergence

Limited visibility, asynchronous (p. 325)

Q1. Is point convergence with limited visibility solvable in full ASYNC, without perception or motion errors? (Kirkpatrick et al. 2024)

Q2. Is there a problem that separates k-ASYNC from SSYNC? (Kirkpatrick et al. 2024)

Linear search (cow path)

k robots on a line, Byzantine faults (p. 355)

Q1. Determine the optimal competitive ratio $B(k, f)$ for Byzantine search on the line; already $B(3, 1)$ is open.

Evacuation

Disk, two wireless robots of different speeds (p. 364)

Q1. Optimal evacuation for $1 < v < 2.75$, for more than two robots, and face-to-face (Lamprou et al. 2020).

Two robots, two exits, wireless (p. 366)

Q1. Close the gap between $2R$ and $(1 + \pi/3)R$ for two wireless robots and two exits (Pattanayak et al. 2018).

Disk, search-and-fetch, two wireless robots (p. 366)

Q1. Lower bounds that separate wireless from face-to-face search-and-fetch (Georgiou et al. 2019).

Two face-to-face robots, disk (p. 367)

Q1. Close the gap between $3 + \pi/6 + \sqrt{3} \approx 5.255$ and 5.6234 for two face-to-face robots.

Two robots with asymmetric communication, disk (p. 368)

Q1. Give an explicit lower bound above ≈ 4.826 for sender/receiver evacuation.

Disk, priority evacuation, queen and one servant (p. 368)

Q1. Close the gap between 4.568 and 4.819 (Georgiou and Wang 2025).

Three wireless robots, disk (p. 369)

Q1. Find the optimal evacuation time for three wireless robots.

Three wireless robots, one crash fault (p. 370)

Q1. Tight bounds for disk evacuation with one crash-faulty robot among three.

Priority evacuation (queen and servants), disk (p. 372)

Claude (Anthropic) was used in cataloguing the problems.

Q1. Close the gap for priority evacuation; for one servant the bounds differ by about 0.25 (a 2024 preprint, not yet in the bibliography).

Line, a sender and a receiver (p. 374)

Q1. Close the gaps for one sender and many receivers ($2+\sqrt{5}$ against 5) and for many senders and one receiver (3 against about 5.681) (Czyzowicz et al. 2021b).

Line, $2f+1$ wireless robots, f crash faults (p. 375)

Q1. Tight bounds for evacuation with $k = 2f+1$ crash or Byzantine robots (Czyzowicz et al. 2021c).

Triangle, two wireless robots starting on the boundary (p. 376)

Q1. Face-to-face evacuation of general triangles, and wireless evacuation of obtuse triangles (Georgiou and Jang 2026).

m rays, $k = m$ wireless robots at the junction (p. 378)

Q1. Starting off the junction with $k = m$: close $2+\sqrt{7/3} \approx 3.528$ against 4 (Brandt et al. 2020).

Q2. Fewer robots than rays ($1 < k < m$): the bounds $3 + 2\lfloor (m-1)/k \rfloor$ and $1 + 2\frac{m-1}{k}(1+\frac{k}{m-1})^{1+(m-1)/k}$ differ (Brandt et al. 2020).

Gathering in graphs

Labelled agents (p. 112)

Q1. Gathering without communication when no bound on n is known.

Byzantine agents (p. 114)

Q1. Polynomial-time Byzantine gathering with teams of size close to $f+1$.

Exploration of dynamic graphs by teams

1-interval-connected rings (p. 218)

Q1. With a landmark but no chirality, is $O(n \log n)$ time necessary? (Di Luna et al. 2020a)

Scattering

Randomized, ASYNC (p. 332)

Q1. Solve scattering in ASYNC; Bramas and Tixeuil call it "the open problem of ASYNC scattering" (Bramas and Tixeuil 2016) (Bramas and Tixeuil 2017). A 2019 preprint by Léchine and Tixeuil (arXiv 1905.09177, not peer reviewed) solves it with vision and weak local multiplicity detection (its Theorem 4), and leaves open the plane with vision and no multiplicity detection.

Claude (Anthropic) was used in cataloguing the problems.

Pattern sequences

Oblivious, SSYNC, chirality (p. 278)

Q1. Form a sequence of patterns in ASYNC with oblivious robots (Das et al. 2015).

Luminous robots, ASYNC, non-rigid, chirality (p. 278)

Q1. Close the gap between the colours used and the lower bound of Theorem 18 (Das et al. 2020).

Q2. Pattern sequences with lights without chirality, with faults, or self-stabilising (Das et al. 2020).

Near-gathering

ASYNC, limited visibility, one common axis (p. 318)

Q1. Pagli, Prencipe and Viglietta conjecture that near-gathering is impossible without agreement on one axis.

SSYNC, limited visibility, disoriented (p. 319)

Q1. Near-gathering in $O(\Delta_0)$ epochs?

Mutual visibility

Lights, SSYNC, non-rigid (p. 283)

Q1. Does the 2-colour algorithm work in non-rigid SSYNC without knowing δ (Di Luna et al. 2017)?

Lights, ASYNC, rigid (p. 284)

Q1. Fewer colours with $O(1)$ epochs (Sharma et al. 2021).

Q2. Time bounds with non-rigid moves.

Lights, ASYNC, non-rigid, one axis (p. 285)

Q1. Mutual visibility in non-rigid ASYNC without axis agreement, ideally with 2 colours (Di Luna et al. 2017).

Lights, one crash, SSYNC, rigid, compass (p. 286)

Q1. Tolerating three or more crashes, and using 2 colours (Aljohani and Sharma 2018).

Fat robots, ASYNC, one axis (p. 287)

Q1. Non-rigid moves, and SSYNC and ASYNC extensions of the 2-colour FSYNC algorithm (Alsaedi et al. 2023).

k-grouping

k equal groups, randomized, SSYNC (p. 291)

Claude (Anthropic) was used in cataloguing the problems.

Q1. Partitioning in ASYNC without a common orientation, by a randomized algorithm with no extra memory.

Leader election in programmable matter

Deterministic, with chirality (p. 395)

Q1. Fast deterministic leader election under weaker (non-atomic) schedulers, and without letting the system disconnect (Dufoulon et al. 2021).

Deterministic, no orientation (p. 397)

Q1. Deterministic leader election without chirality in linear time.

Stationary, chirality, weak scheduler (p. 398)

Q1. Is a unique leader ever impossible for stationary particles with chirality? Dufoulon et al. say this was shown in (Bazzi and Briones 2019), at least for a weak scheduler (Dufoulon et al. 2021); the theorem was not read.

Stationary, asynchronous, no orientation, no holes (p. 399)

Q1. Deterministic leader election in $O(L_{out} + D)$ rounds under a weaker scheduler (Dufoulon et al. 2021).

SILBOT with holes (p. 400)

Q1. Can SILBOT particles elect leaders from configurations with holes if the system may disconnect temporarily, without exterior awareness? The impossibility of (D'Angelo et al. 2020) assumes that connectivity is preserved.

Shape formation in programmable matter

SILBOT, chirality (p. 406)

Q1. Is line formation solvable in SILBOT when particles share just one direction instead of the full orientation (Navarra and Piselli 2023b)?

SILBOT, full orientation (p. 406)

Q1. Does 1-hop visibility suffice for line formation in SILBOT (Navarra and Piselli 2023b)?

Q2. Which other shapes can SILBOT particles form (Navarra and Piselli 2023b)?

Coating

SILBOT, chirality, deterministic (p. 410)

Q1. Coating in SILBOT without chirality (Navarra and Piselli 2023a).

Q2. Multi-layer coating, which "seems unfeasible" in SILBOT (Navarra and Piselli 2023a).

Claude (Anthropic) was used in cataloguing the problems.

Compression

Markov chain, sequential (p. 413)

Q1. Behaviour for $2.17 \leq \lambda \leq 2 + \sqrt{2}$, and bounds on the convergence time.

Patrolling

Open fence, distinct speeds (p. 385)

Q1. How much better than partition can an optimal schedule be on a segment, and is the problem of computing an optimal schedule hard?

Graphs with faulty patrollers (p. 386)

Q1. Is the decision version of fault-tolerant patrolling in NP? With k part of the input and integer edge lengths it is in PSPACE (Czyzowicz et al. 2017b).

Q2. Optimal fault-tolerant patrolling of a segment when $f \geq k/2 - 1$, and for even f (Czyzowicz et al. 2017b).

Q3. Approximation algorithms for the optimal idle time on general graphs (Czyzowicz et al. 2017b).

Exploration by luminous robots

Grids, FSYNC, terminating, no chirality, $\varphi = 1$ (p. 137)

Q1. How many robots are needed for terminating grid exploration under FSYNC with $\varphi = 1$ and no chirality: three or four?

Grids, ASYNC, terminating, no chirality, $\varphi = 1$ (p. 140)

Q1. Close the gap between three and six robots for terminating grid exploration under ASYNC with $\varphi = 1$ and no chirality (Nagahama et al. 2022).

Grids, ASYNC, perpetual, chirality, $\varphi = 2$ (p. 142)

Q1. Can two luminous robots with common chirality and $\varphi = 2$ explore a finite grid perpetually under ASYNC (or SSYNC)?

Grids, ASYNC, perpetual, no chirality, $\varphi = 2$ (p. 143)

Q1. Is there an optimal ASYNC algorithm (three robots, $\varphi = 2$, no chirality) for opaque robots? Posed in (Bramas et al. 2026a).

Grids, ASYNC, perpetual, chirality, $\varphi = 1$ (exclusive) (p. 143)

Q1. Is perpetual grid exploration with $\varphi = 1$ solvable under ASYNC when robots may share nodes (form towers)? Terminating exploration is, with three robots and three colours (Nagahama et al. 2022).

Rings, FSYNC, perpetual, up to f crashes (p. 145)

Q1. With $f + 2$ robots under FSYNC, how many colours are necessary for crash-tolerant perpetual ring exploration? The best algorithm uses $3f + 2$ (Ooshita et al. 2025a).

Claude (Anthropic) was used in cataloguing the problems.

Rings, ASYNC, perpetual, $\varphi = 1$ (p. 147)

Q1. Can three oblivious robots ($c = 1$) with $\varphi = 1$ explore a ring perpetually under SSYNC or ASYNC, or are two colours necessary?

Rings, FSYNC, terminating, $\varphi = 1$ (p. 148)

Q1. Characterise the initial configurations of three robots with $c \geq 3$ colours from which terminating exploration of a ring is solvable under FSYNC with $\varphi = 1$ (with two colours no universal algorithm exists Ooshita and Tixeuil 2022).

Rings, ASYNC, terminating, $\varphi = 1$ (p. 148)

Q1. Does any task separate SSYNC from ASYNC for myopic luminous robots on rings? The known exploration bounds coincide under both (Ooshita and Tixeuil 2022).

Tori, FSYNC, perpetual, no chirality, $\varphi = 2$ (p. 151)

Q1. Can four oblivious robots with $\varphi = 2$ explore a torus perpetually without chirality?

Tori, FSYNC, perpetual, no chirality, $\varphi = 1$ (p. 152)

Q1. Can three robots with two colours and $\varphi = 1$ explore a torus perpetually without common chirality?

Exploration of recolourable graphs

Recolourable nodes, oblivious agent, any graph (p. 161)

Q1. Is the colour complexity of recolourable exploration of arbitrary graphs 4, 5 or 6 (Takahashi et al. 2026) (Hiraoka et al. 2026)?

Q2. Give a proof of the four-colour lower bound without computer assistance, or for a single graph (Hiraoka et al. 2026).

Q3. How many moves does a constant-colour algorithm need on general graphs? No bound is known; for graphs whose blocks are cliques or triangle-free, five colours and $O(n\Delta)$ moves suffice (Hiraoka et al. 2026).

Q4. How many colours suffice if the agent has a constant number of bits, or even one bit, of memory, or may move at random (Böckenhauer et al. 2023) (Takahashi et al. 2026)?

Q5. Does forbidding recolouring back to c_0 change the colour complexity, or only the algorithms (Takahashi et al. 2026)?

Recolourable nodes on cacti (p. 162)

Q1. Is four colours enough for φ-free graphs, or for graphs of maximum degree three? Both lie between 4 and 5 (Hiraoka et al. 2026).

Once-colourable nodes, any graph (p. 164)

Q1. For graphs of circumference $\mathrm{circ}(G) \leq g$, close the gap between $\min(2g-2, n)$ and $\min(2g, n)$ colours (Böckenhauer et al. 2023).

Once-colourable nodes, n known (p. 165)

Q1. Is the colour complexity with known n equal to $n-2$ or $n-1$ (Böckenhauer et al. 2023)?

Claude (Anthropic) was used in cataloguing the problems.

Label-guided: colours fixed by a designer (p. 166)

Q1. Is there a 1-bit (two-colour) labeling with which a finite automaton explores every graph (Cohen et al. 2008)?

Label-guided, oblivious agent (p. 167)

Q1. How many designer colours does an oblivious agent need to explore all graphs of maximum degree Δ (Cohen et al. 2008)?

Rotor-router: a pointer in every node (p. 167)

Q1. For which graphs (other than paths) is the rotor-router exactly time-optimal among oblivious agents without the entry port (Menc et al. 2017)?

Uniform deployment on a ring

Oriented ring, asynchronous, oblivious (p. 345)

Q1. Is knowledge of d necessary for exact deployment on an oriented ring? Which capabilities allow asynchronous exact deployment on unoriented rings? (Flocchini et al. 2008b)

Partitioning

Exactly n/k robots at each fixed point (p. 322)

Q1. Partitioning into k groups of size n/k at points chosen by the robots (no fixed points given).

Separation of coloured particles

Markov chain, sequential (p. 416)

Q1. Bounds on the convergence (mixing) time.

Centralised transformations of modular robots

Rotation only (p. 437)

Q1. What is the complexity of connectivity-preserving transformability with rotations alone? The paper leaves it open (Section 7) (Michail et al. 2019).

Rotation and sliding (p. 438)

Q1. Prove the suspected $\Omega(n \log n)$ lower bound for universal transformations with line pushing (Almethen et al. 2020).

Q2. Distributed universal transformations with line pushing (Almethen et al. 2020).

Claude (Anthropic) was used in cataloguing the problems.

Appendix B

Index of assumptions

Every assumption the atlas uses, alphabetically, with its formal definition. Each entry names its axis and the model or problem that defines it, and the page where the axis is introduced with its origin. Level 0 assumes nothing; higher levels are stronger assumptions.

2D
Space, level 1; Shape formation by hybrid programmable matter, p. 432
Tiles occupy nodes of the triangular lattice.

3D
Space, level 0; Shape formation by hybrid programmable matter, p. 432
Tiles occupy nodes of the face-centred cubic lattice; planar structures are a special case.

adversarial ports
Preprocessing, level 0; Graph exploration by a single agent, p. 79
Port numbers (and any node labels) are chosen by an adversary.

advice
External help, level 1; Treasure hunt, p. 96
An oracle that knows the whole instance gives the agent a binary string before it starts; its length (in bits) is the measure.

all robots
Goal, level 0; Evacuation, p. 360
The task ends when the last robot reaches the exit.

anonymous
Identifiers, level 0; Mobile agents on graphs, p. 10
All agents run the same deterministic algorithm on the same input; no agent has a label. Formally the label function is constant: $\mathrm{id}(a) = \mathrm{id}(b)$ for all agents a, b.

anonymous
Identifiers, level 0; Mobile robots in the plane, p. 25
Robots are indistinguishable and run the same algorithm.

anonymous nodes
Node identifiers, level 0; Mobile agents on graphs, p. 14
Nodes have no identifiers; an agent at v sees only $\deg(v)$ and the port numbers $1, \dots, \deg(v)$, which are assigned locally and independently at each endpoint of an edge.

any delay
Start, level 0; Rendezvous, p. 103
An adversary wakes the agents at times t_1 and t_2; the delay $\tau = |t_1 - t_2|$ is unknown.

any domain
Domain, level 0; Evacuation, p. 359
A bounded convex domain, robots start at an interior point.

any dynamics
Graph dynamics, level 0; Mobile agents on graphs, p. 9
A **temporal graph** is a sequence $\mathcal{G} = (G_t)_{t \geq 0}$ with $G_t = (V, E_t)$ on a fixed node set. It is *temporally connected* if for all $u, v \in V$ and every time t there is a journey (a path whose edges appear at non-decreasing times $\geq t$) from u to v. No G_t need be connected.

any faults
Faults, level 0; Mobile agents on graphs, p. 11
Up to f agents behave arbitrarily (adversarially) and may **forge** identifiers, i.e. report any label in $\mathscr{L}$.

any graph
Network, level 0; Exploration by luminous robots, p. 132
A finite, connected, anonymous graph $G = (V, E)$ with n nodes and no port numbers visible to the robots.

any graph
Network, level 0; Exploration of recolourable graphs, p. 159
A finite, simple, connected, undirected, anonymous graph $G = (V, E)$ with n nodes.

any k
Number of agents, level 0; Mobile agents on graphs, p. 10
The number of agents $k \geq 1$ is arbitrary; $k > n$ is allowed.

any k
Number of robots, level 0; Linear search (cow path), p. 352
$k \geq 1$ robots.

any k
Number of robots, level 0; Evacuation, p. 360
$k \geq 1$ robots.

any k
Team size, level 0; Collaborative exploration, p. 171
Any number $k \geq 2$ of agents.

any n
Number of robots, level 0; Gathering, p. 296
Any number $n \geq 2$ of robots.

any start
Initial configuration, level 0; Mobile agents on graphs, p. 15
Agents start at arbitrary nodes; $\ell \leq k$ nodes are initially occupied, possibly with several agents each.

any topology
Topology, level 0; Mobile agents on graphs, p. 14
G is any connected graph in the class allowed by the other axes.

arbitrary colours
Initial colouring, level 0; Exploration of recolourable graphs, p. 158
The initial colouring $\psi_0 : V \to \{1, \dots, c\}$ is chosen by the adversary; an algorithm must work from every ψ_0 (as with a rotor-router whose pointers are set arbitrarily).

ASYNC
Scheduler, level 0; Exploration by luminous robots, p. 129
ASYNC: each robot's Look, Compute and Move phases take finite but unbounded, adversarially chosen time; a robot may Look while others are between phases, so it may move on an outdated snapshot. A robot is always seen on a node (an edge traversal is instantaneous), and its new colour is visible from the end of its Compute phase. The scheduler is fair: every robot is activated infinitely often. Called *CORDA* in (Datta et al. 2013).

ASYNC
Scheduler, level 0; Mobile robots in the plane, p. 19
ASYNC: each robot's Look, Compute and Move phases take finite but unbounded, adversarially chosen time; a robot may look while others move, and compute on an outdated snapshot. Time is counted in epochs: an **epoch** is an interval of time in which every robot completes at least one full Look–Compute–Move cycle; it need not be the shortest such interval. The execution is divided into consecutive epochs, and a bound of X epochs holds for every such division. In FSYNC and SSYNC time is counted in rounds. Called *Corda* by Prencipe (Prencipe 2007). Intermediate variants restrict interleavings: *LC-atomic* (no Look by another robot between a robot's Look and its Compute), *Move-atomic* (no Look during another robot's Move) and *CM-atomic* (Flocchini et al. 2023).

Claude (Anthropic) was used in cataloguing the problems.

asynchronous
Timing, level 0; Mobile agents on graphs, p. 13
Each agent's Communicate–Compute–Move steps take finite but unbounded, adversarially chosen time. Time is measured in epochs. An **epoch** is an interval of time in which every agent completes at least one full Communicate–Compute–Move step; it need not be the shortest such interval. Time is counted by dividing the execution into consecutive epochs, and a bound of X epochs holds for every such division.

at most crash faults
Faults, level 2; Mobile agents on graphs, p. 11
Up to f agents may stop forever at an adversarially chosen time; a crashed agent stays at its node (or vanishes, as stated per result) and never acts again.

at most weak Byzantine faults
Faults, level 1; Mobile agents on graphs, p. 11
Up to f agents behave arbitrarily but always report their own true identifier.

bipartite
Topology, level 1; Mobile agents on graphs, p. 14
$V = V_1 \cup V_2$ with $V_1 \cap V_2 = \emptyset$ and every edge joining V_1 to V_2; trees are a special case.

both goals
Exploration goal, level 0; Exploration by luminous robots, p. 131
Solve both terminating and perpetual exploration with the stated resources (possibly with two algorithms). A lower bound for either goal holds here, and a result here gives both. No variant is catalogued here; the value exists because the two goals do not imply each other.

bound on n
Knowledge, level 1; Map construction, p. 117
An upper bound $N \geq n$ is known.

Byzantine
Faults, level 0; Mobile robots in the plane, p. 24
Up to f robots move arbitrarily (adversarially); with lights, they may also set arbitrary colours.

Byzantine
Faults, level 0; Search in geometric domains, p. 34
Up to f robots may lie: stay silent when they find the target, or claim to have found it where it is not.

cactus
Network, level 1; Exploration of recolourable graphs, p. 159
G is a cactus: every edge lies on at most one cycle (equivalently, every block is a cycle or a single edge).

chirality
Agreement on coordinates, level 1; Mobile robots in the plane, p. 21
All local coordinate systems have the same handedness (clockwise orientation): no reflection. Also called *handedness*. Prencipe uses *chirality* for agreement on North and East, i.e. a compass (Prencipe 2007).

chirality
Chirality, level 1; Exploration by luminous robots, p. 131
All robots agree on the clockwise direction: a snapshot is known up to rotation only (no reflection), so a robot on an axis of symmetry of its view can still tell its two sides apart. Robots still share no North.

chirality
Orientation, level 1; Programmable matter, p. 38
All particles agree on clockwise order, not on a common direction.

circle
Fence, level 1; Patrolling, p. 384
A closed fence: a circle of length ℓ.

colour once
Recolouring, level 1; Exploration of recolourable graphs, p. 158
Every node starts with the initial colour c_0. The agent at v may change $\psi(v)$ only while $\psi(v) = c_0$; once v has a colour other than c_0 it keeps it forever. This is the main model of (Böckenhauer et al. 2023).

Claude (Anthropic) was used in cataloguing the problems.

common compass
Orientation, level 2; Programmable matter, p. 38
All particles agree on all six directions.

common direction
Orientation, level 1; Programmable matter, p. 38
All particles agree on one lattice direction, not on handedness. Chalopin, Das and Kokkou call this a *common direction* (Chalopin et al. 2026b), but the same words mean agreement on all six directions in other papers (Chalopin et al. 2026a) (Navarra and Piselli 2023b).

compass
Agreement on coordinates, level 2; Mobile robots in the plane, p. 21
All robots agree on the directions and orientations of both axes (a common compass); they may still disagree on origin and unit. Called *full compass* by Efrima and Peleg, who also use *half compass* (both axis directions, the positive sense of one axis, no chirality), *direction-only* and *axes-only* (Efrima and Peleg 2009). Agreement on one axis together with common chirality is equivalent: the orientation of the second axis then follows from handedness.

complete graph
Topology, level 1; Mobile agents on graphs, p. 14
$G = K_n$: every pair of nodes is adjacent.

connected every round
Graph dynamics, level 1; Mobile agents on graphs, p. 9
$\mathcal{G} = (G_t)_{t \geq 0}$ is **1-interval connected** if every snapshot $G_t = (V, E_t)$ is connected. More generally it is T-interval connected if for every t the graph $\left(V, \bigcap_{i=t}^{t+T-1} E_i\right)$ is connected.

constant memory
Memory, level 1; Programmable matter, p. 39
Each particle has $O(1)$ bits of persistent memory and reads or writes neighbours' memory.

crash faults
Faults, level 1; Mobile robots in the plane, p. 24
Up to f robots stop forever at adversarial times and remain visible at their last position.

crash faults
Faults, level 1; Search in geometric domains, p. 34
Up to f robots may fail to detect or to report the target when they pass over it.

designer labels
Preprocessing, level 1; Graph exploration by a single agent, p. 79
Before the agent starts, the designer may write a label of b bits (a constant) at every node, which the agent reads on arrival.

designer ports
Preprocessing, level 1; Graph exploration by a single agent, p. 79
Before the agent starts, the designer may choose the port numbering $\mathrm{port}_v : \{\text{edges at } v\} \to \{1, \dots, \deg(v)\}$ at every node. Ilcinkas calls a designer's port numbering a *local orientation* (Ilcinkas 2008).

designer's colours
Initial colouring, level 2; Exploration of recolourable graphs, p. 158
Before the agent starts, a designer who knows G (and the start node) chooses ψ_0. Used with fixed colours this is label-guided exploration (Cohen et al. 2008).

deterministic
Randomness, level 0; Mobile agents on graphs, p. 11
The next state and move of an agent are a function of its current state and what it observes.

deterministic
Randomness, level 0; Mobile robots in the plane, p. 24
The destination computed in a cycle is a function of the snapshot (and the light, if any).

Claude (Anthropic) was used in cataloguing the problems.

deterministic
Randomness, level 0; Search in geometric domains, p. 35
Trajectories are fixed functions of the information received.

deterministic
Randomness, level 0; Programmable matter, p. 39
No random choices.

directed
Edge direction, level 0; Mobile agents on graphs, p. 10
$G = (V, A)$ with $A \subseteq V \times V$; an agent at u may move along $(u, v) \in A$ only, never back along it unless $(v, u) \in A$. G is strongly connected.

disk
Domain, level 1; Evacuation, p. 359
The unit disk; robots start at its centre; the exit is a point of the boundary circle.

disoriented
Agreement on coordinates, level 0; Mobile robots in the plane, p. 21
Each robot r observes in its own coordinate system, obtained from the global one by an unknown similarity (translation, rotation, uniform scaling, and possibly reflection); systems may differ between robots and, in some models, between cycles. Papers that call robots *disoriented* do not always mean this: Défago and Souissi use the word while assuming chirality (Défago and Souissi 2008). Some papers also let the adversary change a robot's rotation, reflection and unit at every Look (*variable disorientation*) (Flocchini et al. 2023) (Masuzawa et al. 2026).

dispersed start
Initial configuration, level 1; Mobile agents on graphs, p. 15
Every initially occupied node holds exactly one agent ($\ell = k$). Also called a *scattered* start, notably in black hole search.

distinct IDs (eponymous)
Identifiers, level 2; Mobile agents on graphs, p. 10
Distinct identifiers (eponymous agents): the label function $\mathrm{id} : \{1, \dots, k\} \to \{1, \dots, L\}$ is injective; usually $L = k^{O(1)}$, so a label takes $O(\log k)$ bits.

distinct IDs (eponymous)
Identifiers, level 2; Mobile robots in the plane, p. 25
Distinct identifiers (eponymous robots): labels are pairwise distinct ($\ell = n$); whether others can see a robot's label is stated per result.

distinct speeds
Speeds, level 0; Search in geometric domains, p. 35
Each robot i has its own maximum speed v_i, known to all.

***e* exits**
Exits, level 0; Chauffeuring a crashed robot, p. 380
$e \geq 1$ exits at unknown boundary points.

equal speeds
Speeds, level 1; Search in geometric domains, p. 35
All robots move at most at unit speed.

evacuation
Goal, level 0; Linear search (cow path), p. 352
The task ends when every robot has reached the target.

external light (FCOM)
Lights (persistent memory), level 1; Mobile robots in the plane, p. 20
FCOM: each robot has a light with c colours that only the others can see (constant communication, no persistent memory of its own). Also called an *external* light.

Claude (Anthropic) was used in cataloguing the problems.

face-to-face

Communication, level 0; Search in geometric domains, p. 34

Face-to-face: two robots exchange information only when they are at the same point at the same time.

fat

Body, level 0; Mobile robots in the plane, p. 18

Each robot is a closed disc of radius 1 (unit disc). Discs may not overlap, so robots cannot share a point and must avoid collisions; a disc hides what lies behind it from other robots. Gathering then asks for a configuration whose union of discs is connected and in which every robot sees all others (Czyzowicz et al. 2009).

FCOM

Lights (memory and messages), level 1; Exploration by luminous robots, p. 130

FCOM (*finite-communication*): each robot has a light with c colours that the other robots within distance φ can see but the robot itself cannot; the robot stays oblivious, but every robot that looks reads its last colour: $\log_2 c$ bits broadcast to the neighbourhood, no memory (Flocchini et al. 2016).

finite automaton

Agent memory, level 1; Exploration of recolourable graphs, p. 157

The agent is a deterministic automaton with K states, K independent of the graph; its action and next state depend on its state and its observation.

finite graph

Graph size, level 2; Treasure hunt, p. 96

G has finitely many nodes.

finite graph

Graph size, level 2; Rendezvous, p. 102

G has finitely many nodes.

fixed colours

Recolouring, level 0; Exploration of recolourable graphs, p. 158

The colouring $\psi : V \to \{1, \dots, c\}$ never changes; the agent reads $\psi(v)$ (and, with neighbour view, its neighbours' colours) but cannot write.

FSTA

Lights (memory and messages), level 1; Exploration by luminous robots, p. 130

FSTA (*finite-state*): each robot has a light with c colours that only it can see. The colour is set at the end of Compute and persists until the robot changes it: $\log_2 c$ bits of persistent memory, no communication (Flocchini et al. 2016).

FSYNC

Scheduler, level 2; Exploration by luminous robots, p. 129

FSYNC: in every round every robot executes one atomic Look–Compute–Move cycle.

FSYNC

Scheduler, level 2; Mobile robots in the plane, p. 19

FSYNC: every robot executes one atomic cycle in every round.

global communication

Communication, level 3; Mobile agents on graphs, p. 12

At every round every agent can read the memory of every other agent, wherever it is.

graph

Fence, level 0; Patrolling, p. 384

Every point of every edge of a graph G must be visited; robots move along edges.

grid

Network, level 1; Exploration by luminous robots, p. 132

An $a \times b$ grid ($n = ab$ nodes; nodes (i, j), edges between nodes at Manhattan distance one); anonymous, with a and b unknown. A robot sees the border of the grid when it is within range, since nodes beyond the border are missing from its snapshot.

Claude (Anthropic) was used in cataloguing the problems.

grid
Topology, level 1; Mobile agents on graphs, p. 14
G is a $\sqrt{n} \times \sqrt{n}$ grid graph; whether ports are globally consistent (oriented) is stated per result.

h-hop communication
Communication, level 2; Mobile agents on graphs, p. 12
For a fixed $h \geq 1$, stated with each result: at every round an agent at v can read the memory of every agent at a node u with $\mathrm{dist}(v, u) \leq h$ (in the current snapshot G_t for a dynamic graph), in addition to the co-located agents. $h = 1$ is **1-hop communication** (agents at neighbouring nodes communicate); local communication is the case $h = 0$, and global communication the case $h \geq D$. Reading another agent's memory does not tell an agent which port leads to it.

holes allowed
Initial shape, level 0; Programmable matter, p. 40
The initial set of occupied nodes induces a connected subgraph; it may enclose empty regions.

homonymous
Identifiers, level 1; Mobile agents on graphs, p. 10
Each agent a has a label $\mathrm{id}(a) \in \mathscr{L}$ known to itself (and revealed on meeting), but the labelling need not be injective: $1 \leq |\{\mathrm{id}(a)\}| \leq k$. With $|\mathscr{L}|$ distinct labels, $|\mathscr{L}| = 1$ is the anonymous case and $|\mathscr{L}| = k$ the eponymous one (Delporte-Gallet et al. 2013).

homonymous
Identifiers, level 1; Mobile robots in the plane, p. 25
Robots carry labels from a set of size ℓ, $1 \leq \ell \leq n$, which may repeat; equivalently, robots may run one of ℓ algorithms. The minimum ℓ that solves a problem is its *minimum algorithm size* (Asahiro and Yamashita 2025).

hypercube
Topology, level 1; Mobile agents on graphs, p. 14
$G = Q_j$: nodes are the $n = 2^j$ binary strings of length j, adjacent when they differ in one bit.

infinite degrees allowed
Graph size, level 0; Treasure hunt, p. 96
G is finite or countably infinite; a node may have countably infinite degree (ports $1, 2, 3, \ldots$).

infinite degrees allowed
Graph size, level 0; Rendezvous, p. 102
G is finite or countably infinite; degrees may be infinite.

internal light (FSTA)
Lights (persistent memory), level 1; Mobile robots in the plane, p. 20
FSTA: each robot has a light with c colours that only it can see; it persists between cycles (constant persistent memory, no communication). Also called an *internal* light.

$k = n$
Team size, level 1; Collaborative exploration, p. 171
Exactly $k = n$ agents.

$k \geq Dn^{1+\varepsilon}$
Team size, level 1; Collaborative exploration, p. 171
$k \geq D\,n^{1+\varepsilon}$ agents for a constant $\varepsilon > 0$ (written $k = Dn^{\gamma}$, $\gamma > 1$).

$k \leq n$
Number of agents, level 1; Mobile agents on graphs, p. 10
The number of agents satisfies $1 \leq k \leq n$.

known graph
Knowledge, level 1; Network decontamination, p. 222
The topology is known to the agents (or to a coordinator).

knows a bound on n
Knowledge, level 1; Graph exploration by a single agent, p. 78
The agent is given an integer $N \geq n$ (an upper bound on the number of nodes) as input.

Claude (Anthropic) was used in cataloguing the problems.

knows n, Δ
Knowledge, level 1; Leader election by agents, p. 228
Every agent knows n and Δ (upper bounds suffice where stated).

knows n, Δ
Knowledge, level 1; Maximal independent set by agents, p. 232
Every agent knows n and Δ (upper bounds suffice where stated).

knows n, Δ
Knowledge, level 1; Minimal dominating set by agents, p. 236
Every agent knows n and Δ (upper bounds suffice where stated).

knows n, Δ
Knowledge, level 1; Triangle and butterfly counting by agents, p. 239
Every agent knows n and Δ (upper bounds suffice where stated).

knows n, Δ
Knowledge, level 1; Minimum spanning tree by agents, p. 244
Every agent knows n and Δ (upper bounds suffice where stated).

knows n, Δ
Knowledge, level 1; BFS tree by agents, p. 247
Every agent knows n and Δ (upper bounds suffice where stated).

knows the map
Knowledge, level 2; Graph exploration by a single agent, p. 78
The agent is given a graph isomorphic to G, with port numbers, and (where stated) its own position on it; for a dynamic graph, the whole sequence (G_t).

labelled nodes
Node identifiers, level 1; Mobile agents on graphs, p. 14
Each node v carries a distinct identifier that agents at v can read; hence an agent can recognise a node it has visited.

lights (LUMI)
Lights (persistent memory), level 2; Mobile robots in the plane, p. 20
LUMI: each robot has a light with c colours, set at the end of Compute, visible to all robots including itself; it is both memory and communication. Also called *luminous* robots, or a *full* light.

limited visibility
Visibility, level 0; Mobile robots in the plane, p. 23
A robot sees exactly the robots within Euclidean distance V of its position, for a fixed radius $V > 0$.

line
Domain, level 1; Linear search (cow path), p. 351
The real line, robots start at 0; the target is at an unknown x with $|x| \geq 1$.

line
Domain, level 1; Evacuation, p. 359
The robots start at the origin of the real line; the exit is at an unknown point at distance at least 1; the measure is the competitive ratio, the evacuation time divided by that distance.

local communication
Communication, level 1; Mobile agents on graphs, p. 12
Agents at the same node at the same time can read each other's entire memory (face-to-face communication). Agents crossing on an edge do not meet.

locally finite
Graph size, level 1; Treasure hunt, p. 96
G is finite or countably infinite, and every node has finite degree.

locally finite
Graph size, level 1; Rendezvous, p. 102
G is finite or countably infinite with finite degrees.

Claude (Anthropic) was used in cataloguing the problems.

luminous
Lights (memory and messages), level 2; Exploration by luminous robots, p. 130
LUMI (*luminous* robots): each robot has a light with c colours, set at the end of Compute, persistent across cycles, and visible to the robot itself and to every robot within distance φ (Das et al. 2016). It is both memory and message. A Look returns, for every node within distance φ, the multiset of colours of the robots on it.

m rays
Domain, level 0; Linear search (cow path), p. 351
$m \geq 2$ half-lines sharing their endpoint, the origin, where the robots start.

map
Knowledge of the graph, level 2; Black hole search, p. 209
Agents have a map of G and know their starting position on it.

markers
Markers, level 1; Map construction, p. 118
The agent carries markers it can drop and pick up.

may disconnect
Connectivity during execution, level 1; Programmable matter, p. 40
Connectivity is required only initially and at the end.

may move
Movement, level 1; Leader election in programmable matter, p. 393
Particles may expand and contract during the election.

$n = 2$
Number of robots, level 1; Gathering, p. 296
Exactly two robots (rendezvous).

$n \geq 3$
Number of robots, level 1; Gathering, p. 296
$n \geq 3$ robots.

n known
Knowledge, level 1; Exploration of recolourable graphs, p. 159
The algorithm may depend on n: it must explore every graph with exactly n nodes (a *non-uniform* algorithm).

neighbour colours
View, level 1; Exploration of recolourable graphs, p. 157
At node v the agent observes $\psi(v)$ and the multiset $\{\!\{\psi(u) : u \in N(v)\}\!\}$ of its neighbours' colours (hence also $\deg(v)$), without knowing which edge leads to which colour unless ports say so.

no chirality
Chirality, level 0; Exploration by luminous robots, p. 131
Robots share no orientation: on a ring a robot cannot tell its two directions apart; on a grid or torus its snapshot is known only up to rotation and reflection, and the adversary may choose a different transformation in every cycle (*self-inconsistent* robots, as in Darwich et al. 2023 and Bramas et al. 2026a).

no communication
Communication, level 0; Mobile agents on graphs, p. 12
Agents exchange no messages; an agent at v observes only the number of agents at v (and, where stated, whether that number changed).

no faults
Faults, level 3; Mobile agents on graphs, p. 11
All agents follow the algorithm forever.

no faults
Faults, level 2; Mobile robots in the plane, p. 24
All robots follow the algorithm forever.

no faults
Faults, level 2; Search in geometric domains, p. 34
Every robot detects and reports the target.

Claude (Anthropic) was used in cataloguing the problems.

no help
External help, level 0; Treasure hunt, p. 96
No information beyond the agent's own observations.

no holes
Initial shape, level 1; Programmable matter, p. 40
The initial configuration is connected and encloses no empty node. Also called *contractible* (Briones et al. 2023) or *hole-free* (Gastineau et al. 2019).

no knowledge
Knowledge, level 0; Leader election by agents, p. 228
Agents know no graph parameter (n, m, Δ, D).

no knowledge
Knowledge, level 0; Maximal independent set by agents, p. 232
Agents know no graph parameter (n, m, Δ, D).

no knowledge
Knowledge, level 0; Minimal dominating set by agents, p. 236
Agents know no graph parameter (n, m, Δ, D).

no knowledge
Knowledge, level 0; Triangle and butterfly counting by agents, p. 239
Agents know no graph parameter (n, m, Δ, D).

no knowledge
Knowledge, level 0; Minimum spanning tree by agents, p. 244
Agents know no graph parameter (n, m, Δ, D).

no knowledge
Knowledge, level 0; BFS tree by agents, p. 247
Agents know no graph parameter (n, m, Δ, D).

no markers
Markers, level 0; Graph exploration by a single agent, p. 79
The agent cannot mark nodes.

no markers
Markers, level 0; Map construction, p. 118
No markers.

no marking
Marking nodes, level 0; Black hole search, p. 209
Agents cannot leave information at nodes.

no multiplicity detection
Multiplicity detection, level 0; Mobile robots in the plane, p. 23
A snapshot is a set of points: a point occupied by several robots looks like a single robot.

no orientation
Orientation, level 0; Programmable matter, p. 38
Each particle labels its six incident directions in its own clockwise or counter-clockwise order, starting anywhere.

no ports
Port numbers, level 0; Exploration of recolourable graphs, p. 156
Edges carry no port numbers. To move, the agent at v names a colour x; the adversary moves it to any neighbour u of v with $\psi(u) = x$ (if the agent sees no neighbour colours, to any neighbour). The agent never learns by which edge it arrived.

no whiteboards
Node memory, level 0; Mobile agents on graphs, p. 13
Nodes have no storage: no information can be left at a node except by an agent staying there.

Claude (Anthropic) was used in cataloguing the problems.

no whiteboards
Whiteboards, level 0; Map construction, p. 118
No whiteboards.

non-rigid
Movement, level 0; Mobile robots in the plane, p. 22
In a Move phase the adversary may stop a robot before its destination, but only after it has travelled at least $\delta > 0$ (a constant unknown to the robots) or reached the destination.

nothing known
Knowledge, level 0; Graph exploration by a single agent, p. 78
The agent knows nothing about G: not n, m, Δ or D.

nothing known
Knowledge, level 0; Map construction, p. 117
Nothing is known about G.

nothing known
Knowledge, level 0; Exploration of recolourable graphs, p. 159
The algorithm is the same for all graphs: it knows neither n nor Δ nor any other parameter (a *uniform* algorithm in Böckenhauer et al. 2023).

oblivious
Agent memory, level 0; Exploration of recolourable graphs, p. 157
The agent has no internal state: its action (new colour of v, then a move, *stay* or *stop*) is a function of its current observation only. Called *zero-memory*, *memoryless* or a *1-state* robot.

oblivious
Lights (memory and messages), level 0; Exploration by luminous robots, p. 130
OBLOT: a robot's Compute phase depends only on the current snapshot; nothing persists across cycles and robots communicate only by moving. Equivalent to a light with $c = 1$ colour.

oblivious
Lights (persistent memory), level 0; Mobile robots in the plane, p. 20
OBLOT: a robot's computation depends only on the current snapshot; nothing persists across cycles.

oblivious
Memory, level 0; Programmable matter, p. 39
No persistent state and no messages; a particle decides from its view only.

$O(\log n)$ bits
Agent memory, level 2; Exploration of recolourable graphs, p. 157
The agent has $b = O(\log n)$ bits of memory, enough to hold a counter up to a polynomial in n. No variant is catalogued here yet.

one agent
Agents, level 0; Map construction, p. 117
A single agent.

one axis
Agreement on coordinates, level 1; Mobile robots in the plane, p. 21
All robots agree on the direction and orientation of one axis (e.g. North), but not necessarily on handedness: two robots may disagree on which side is East. This is how the papers use *one axis agreement* (e.g. robots "agree on only 'left' and 'right'" Bose et al. 2021; robots "do not necessarily have the same handedness" Pagli et al. 2015). One axis and chirality are alternatives on this axis; together they amount to a common compass.

one exit
Exits, level 0; Evacuation, p. 361
One exit, at an unknown point of the boundary.

one exit
Exits, level 1; Chauffeuring a crashed robot, p. 380
Exactly one exit.

Claude (Anthropic) was used in cataloguing the problems.

one pebble
Markers, level 1; Graph exploration by a single agent, p. 79
The agent carries one pebble that it can drop at its current node and later pick up; it sees whether a pebble is at its node.

one robot
Number of robots, level 1; Linear search (cow path), p. 352
A single robot.

opaque
Opacity, level 0; Mobile robots in the plane, p. 23
Robots are points that obstruct sight: r sees s iff no third robot lies on the open segment $\overline{rs}$.

oracle pebbles
External help, level 1; Treasure hunt, p. 96
An oracle that knows the instance places pebbles on nodes (at most one per node) before the agent starts; their number is the measure.

own node
View, level 0; Exploration of recolourable graphs, p. 157
At node v the agent observes $\psi(v)$, $\deg(v)$ and nothing about its neighbours (and, where ports exist, what the port axis grants).

pebbles
Markers, level 2; Graph exploration by a single agent, p. 79
The agent carries p pebbles (identical or distinguishable, as stated); p is a resource reported with the memory.

perpetual
Exploration goal, level 1; Exploration by luminous robots, p. 131
Every node is visited by at least one robot infinitely often; robots never stop.

perpetual
Termination requirement, level 2; Graph exploration by a single agent, p. 80
Every node is visited at least once (and, for periodic exploration, infinitely often); the agent never needs to halt.

perpetual
Termination requirement, level 2; Exploration of recolourable graphs, p. 159
Every node is visited at least once (for the rotor-router: infinitely often); the agent never needs to halt.

point
Body, level 1; Mobile robots in the plane, p. 18
Each robot is a point of the plane: several robots may share a point (multiplicity), and a robot hides only what lies exactly behind it on a line of sight.

ports + entry port
Port numbers, level 2; Exploration of recolourable graphs, p. 156
As for port numbers, and on arrival at v the agent also reads the port $\mathrm{port}_v(e)$ of the edge e by which it entered. This is the standard convention of single-agent exploration.

ports, no entry port
Port numbers, level 1; Exploration of recolourable graphs, p. 156
The edges at v carry port numbers $1, \dots, \deg(v)$, assigned by an adversary independently at each endpoint (port_v). The agent chooses the port by which it leaves, but on arrival it is not told the port by which it entered (*unknown inport*).

priority (queen)
Goal, level 1; Evacuation, p. 360
One designated robot (the queen) must reach the exit; the others (servants) only help; the task ends when the queen exits.

private coins
Randomness, level 1; Mobile agents on graphs, p. 11
Each agent additionally has access to independent fair coin flips; guarantees hold with probability 1, in expectation, or with high probability, as stated per result.

Claude (Anthropic) was used in cataloguing the problems.

randomized
Randomness, level 1; Mobile robots in the plane, p. 24
Robots may use random bits in Compute; guarantees hold with probability 1 or in expectation, as stated per result.

randomized
Randomness, level 1; Search in geometric domains, p. 35
Trajectories may depend on random choices; the measure is the expected competitive ratio against an oblivious adversary.

randomized
Randomness, level 1; Programmable matter, p. 39
Particles flip fair coins; guarantees hold with high probability or in expectation.

rays
Domain, level 1; Evacuation, p. 359
$m \geq 2$ half-lines share an endpoint, the junction; the exit lies on one ray at distance at least 1 from the start; the measure is the competitive ratio.

recolourable
Recolouring, level 2; Exploration of recolourable graphs, p. 158
At every step the agent at v may set $\psi(v)$ to any colour of the palette, including back to c_0, any number of times (the convention of Takahashi et al. 2026 and Hiraoka et al. 2026).

return
Termination requirement, level 0; Exploration of recolourable graphs, p. 159
The agent visits every node and then halts at its start node s (*exploration with return*).

rigid
Movement, level 1; Mobile robots in the plane, p. 22
Every Move phase ends at the computed destination.

ring
Network, level 1; Exploration by luminous robots, p. 132
An anonymous, unoriented ring of n nodes; n is unknown to the robots.

ring
Topology, level 1; Mobile agents on graphs, p. 14
G is a cycle C_n (every node of degree 2), or a path when stated.

rooted start
Initial configuration, level 1; Mobile agents on graphs, p. 15
All k agents start at the same node ($\ell = 1$). Also called *co-located* (black hole search: all agents at the *home base*) or a *rooted graph* (Molla and Moses Jr. 2019).

rotation
Mechanism, level 0; Centralised transformations of modular robots, p. 436
A module may rotate 90° around an adjacent module, if the cells it sweeps are empty.

rotation + sliding
Mechanism, level 1; Centralised transformations of modular robots, p. 436
In addition, a module may slide one cell along a line of modules.

search
Goal, level 1; Linear search (cow path), p. 352
The task ends when the first robot reaches the target (and, with faults, the others can be sure of it).

search-and-fetch
Goal, level 1; Evacuation, p. 360
A treasure lies at an unknown point of the boundary, and the exit at a known arc distance α from it; the task ends when some robot has carried the treasure to the exit.

segment
Fence, level 1; Patrolling, p. 384
An open fence: the segment $[0, \ell]$; robots may reverse direction anywhere.

Claude (Anthropic) was used in cataloguing the problems.

sender/receiver
Communication, level 1; Search in geometric domains, p. 34
Sender/receiver: one robot can only transmit and the other only receive, instantly at any distance.

sense of direction
Knowledge of the graph, level 1; Black hole search, p. 209
Agents have a consistent sense of direction: a globally consistent labelling of edges from which they can tell whether two walks lead to the same node.

sequential
Scheduler, level 2; Mobile robots in the plane, p. 19
Sequential: an SSYNC scheduler that activates exactly one robot per round (fairly). Also called a *centralized* scheduler (Terai et al. 2023); a *round-robin* scheduler is a fair special case.

sequential
Scheduler, level 2; Programmable matter, p. 38
At most one particle is active at any time.

several agents
Agents, level 1; Map construction, p. 117
$k \geq 2$ identical agents.

simultaneous start
Start, level 1; Rendezvous, p. 103
Both agents start in the same round ($\tau = 0$).

square
Domain, level 1; Evacuation, p. 359
The unit square; robots start at its centroid; the exit is a point of the boundary.

SSYNC
Scheduler, level 1; Exploration by luminous robots, p. 129
SSYNC: time is divided into rounds; in each round a fair adversary chooses a non-empty subset of robots, which execute one atomic Look–Compute–Move cycle together. Called *ATOM* in (Datta et al. 2013).

SSYNC
Scheduler, level 1; Mobile robots in the plane, p. 19
SSYNC: time is divided into rounds; in each round a non-empty subset of robots, chosen by a fair adversary, executes one atomic Look–Compute–Move cycle. Called *Atom* by Prencipe (Prencipe 2007).

static graph
Graph dynamics, level 2; Mobile agents on graphs, p. 9
$G_t = G$ for all t: the edge set never changes.

stationary
Movement, level 0; Leader election in programmable matter, p. 393
No particle ever expands or contracts; the configuration is fixed and only states change.

stays connected
Connectivity during execution, level 0; Programmable matter, p. 40
At every time the occupied nodes induce a connected subgraph.

stop
Termination requirement, level 1; Exploration of recolourable graphs, p. 159
The agent visits every node and then halts, at any node.

strong multiplicity
Multiplicity detection, level 2; Mobile robots in the plane, p. 23
A snapshot is a multiset: each occupied point comes with the exact number of robots on it.

strong scheduler
Scheduler, level 1; Programmable matter, p. 38
Each activation is executed atomically; the adversary chooses the order, fairly.

Claude (Anthropic) was used in cataloguing the problems.

synchronous
Timing, level 1; Mobile agents on graphs, p. 13
Time proceeds in rounds $t = 1, 2, \ldots$; in every round every agent communicates, computes and moves (at most one edge) simultaneously.

terminating
Exploration goal, level 1; Exploration by luminous robots, p. 131
Every node is visited by at least one robot, and from some time on no robot is enabled (no robot changes its position or colour again).

three robots
Number of robots, level 1; Evacuation, p. 360
Three robots.

tokens
Marking nodes, level 1; Black hole search, p. 209
Agents carry tokens (pebbles) they can place at nodes or on ports; tokens are identical and carry no other information.

topology unknown
Knowledge of the graph, level 0; Black hole search, p. 209
Agents know n but not the topology of G.

torus
Network, level 1; Exploration by luminous robots, p. 132
An $a \times b$ torus: the grid with wrap-around edges in both dimensions, so every node has degree four and there is no border.

transparent
Opacity, level 1; Mobile robots in the plane, p. 23
Robots do not obstruct each other's view.

tree
Network, level 2; Exploration of recolourable graphs, p. 159
G is a tree (a cactus without cycles).

tree
Topology, level 1; Mobile agents on graphs, p. 14
G is connected and acyclic: $|E| = n - 1$.

triangle
Domain, level 1; Evacuation, p. 359
The equilateral triangle of unit side; robots start at its centroid; the exit is a point of the boundary.

two exits
Exits, level 1; Evacuation, p. 361
Two exits at unknown points of the boundary; the task ends when every robot has reached one of them.

two exits
Exits, level 1; Chauffeuring a crashed robot, p. 380
Exactly two exits.

two robots
Number of robots, level 1; Linear search (cow path), p. 352
Two robots.

two robots
Number of robots, level 1; Evacuation, p. 360
Two robots.

undirected
Edge direction, level 1; Mobile agents on graphs, p. 10
$G = (V, E)$ with $E \subseteq \binom{V}{2}$; every edge can be traversed in both directions.

Claude (Anthropic) was used in cataloguing the problems.

uniform c_0
Initial colouring, level 1; Exploration of recolourable graphs, p. 158
Initially $\psi(v) = c_0$ for every node v (*clean* node memory). The colour c_0 counts as one of the c colours.

unknown graph
Knowledge, level 0; Network decontamination, p. 222
Agents know nothing about G.

unlimited visibility
Visibility, level 1; Mobile robots in the plane, p. 23
A robot sees all robots (subject to opacity).

$\varphi = 1$
Visibility range, level 0; Exploration by luminous robots, p. 130
In a Look phase a robot at node v sees every node at distance at most $\varphi = 1$ from v (in the graph metric), with the robots on it, their colours, and how many robots each node holds. It sees no node identifiers and, without chirality, no orientation.

$\varphi = 2$
Visibility range, level 1; Exploration by luminous robots, p. 130
As for $\varphi = 1$, but the robot sees every node at distance at most $\varphi = 2$. An algorithm for $\varphi = 2$ works for every larger φ, since a robot can ignore what lies further away.

$\varphi = 3$
Visibility range, level 2; Exploration by luminous robots, p. 130
As for $\varphi = 1$, but the robot sees every node at distance at most $\varphi = 3$.

weak multiplicity
Multiplicity detection, level 1; Mobile robots in the plane, p. 23
A snapshot marks each occupied point as holding one robot or more than one.

weak scheduler
Scheduler, level 0; Programmable matter, p. 38
An activation's look, compute and move need not be atomic; concurrent activations may interfere. The scheduler of Bazzi and Briones is described as weak sequential, as non-atomic, and as fair asynchronous by different later papers (Briones et al. 2023) (Dufoulon et al. 2021) (Chalopin et al. 2026b).

whiteboards
Marking nodes, level 2; Black hole search, p. 209
Every node has a whiteboard that agents read and write under mutual exclusion.

whiteboards
Node memory, level 1; Mobile agents on graphs, p. 13
Each node v has a whiteboard of bounded size that agents at v can read and write, with mutual exclusion.

whiteboards
Whiteboards, level 1; Map construction, p. 118
Each node has a whiteboard.

wireless
Communication, level 2; Search in geometric domains, p. 34
Wireless: every robot can send messages to every other robot instantly at any distance.

with return
Termination requirement, level 0; Graph exploration by a single agent, p. 80
The agent visits every node and then halts at its start node s.

with stop
Termination requirement, level 1; Graph exploration by a single agent, p. 80
The agent visits every node and then halts, at any node. Also called *terminating exploration*.

Claude (Anthropic) was used in cataloguing the problems.

Appendix C

Timeline

Each year lists the problems first posed then and the papers that introduced a setting, gave the current best result, or proved a lower bound, dated by their first appearance (usually the conference version).

1951

Graph exploration by a single agent is posed in Shannon (1951). Shannon's maze-solving mouse Theseus (first prototype 1950) explored a 5×5 grid with constant memory per cell (Disser et al. 2019); the first impossibility theorem for finite automata is Budach's (Budach 1978).

1964

Linear search (cow path) is posed in Beck (1964). Bellman posed the problem in 1963 and Beck analysed it in 1964; computer scientists rediscovered it in the late 1980s as the cow-path problem (Kupavskii and Welzl 2018).

1979

- Aleliunas et al. (1979) — Graph exploration by a single agent: Randomized exploration when n is known [current best]

1990

- Deng and Papadimitriou (1999) — Graph exploration by a single agent: Directed graphs with labelled nodes [introductory], Directed graphs with labelled nodes [lower bound]

1991

Map construction is posed in Dudek et al. (1991). Dudek, Jenkin, Milios and Wilkes framed robotic exploration as graph construction. In the logical view: strengthening of Graph exploration by a single agent, which was posed earlier (1951).

- Dudek et al. (1991) — Map construction: One agent with a marker [current best]

1993

- Baeza-Yates et al. (1993) — Linear search (cow path): One robot on a line [current best], One robot on a line [lower bound], One robot on m rays [current best], One robot on m rays [lower bound]
- Kao et al. (1996) — Linear search (cow path): One randomized robot on a line [current best], One randomized robot on a line [lower bound]

1998

- Bender et al. (2002) — Graph exploration by a single agent: Directed graphs, one pebble, bound on n known [current best]

1999

Pattern formation is posed in Suzuki and Yamashita (1999). Suzuki and Yamashita characterised the patterns that anonymous robots can form in terms of their initial configuration.

Gathering is posed in Suzuki and Yamashita (1999). Suzuki and Yamashita gave an algorithm "moving the robots to a single point in finite steps" and showed that two oblivious robots cannot meet in SSYNC. In the logical view: special case of k-grouping, but was posed before it (2007).

Convergence is posed in Suzuki and Yamashita (1999). Suzuki and Yamashita gave algorithms for converging to a point and for reaching a point; Ando et al. treated limited visibility the same year (Ando et al. 1999).

- Ando et al. (1999) — Convergence: Limited visibility, SSYNC, rigid moves [current best]

2001

Black hole search is posed in Dobrev et al. (2007). Dobrev, Flocchini, Prencipe and Santoro introduced black hole search in rings (DISC 2001) and in arbitrary networks (PODC 2002) (Dobrev et al. 2006). In the logical view: variant of Collaborative exploration, but was posed before it (2004).

- Flocchini et al. (2005) — Gathering: Limited visibility with a compass, ASYNC [current best], Limited visibility with a compass, ASYNC [lower bound]
- Dobrev et al. (2007) — Black hole search: Asynchronous agents with whiteboards in a ring [current best], Asynchronous agents with whiteboards in a ring [lower bound]

2002

Circle formation is posed in Défago and Konagaya (2002). In the logical view: special case of Pattern formation, which was posed earlier (1999).

Network decontamination is posed in Barrière et al. (2002). Capture of an intruder by mobile agents (SPAA 2002).

- Diks et al. (2002) — Graph exploration by a single agent: Trees, perpetual exploration [current best], Trees, perpetual exploration [lower bound], Trees, exploration with stop [lower bound], Trees, exploration with return [introductory], Trees, exploration with return [lower bound]
- Défago and Konagaya (2002) — Circle formation: Circle formation, SSYNC [introductory]
- Dobrev et al. (2006) — Black hole search: Asynchronous agents with whiteboards [current best], Asynchronous agents with whiteboards [lower bound], Whiteboards and sense of direction [current best], Whiteboards and sense of direction [lower bound], Whiteboards and a map [current best], Whiteboards and a map [lower bound], Tokens and a map [lower bound]
- Barrière et al. (2002) — Network decontamination: Trees [current best]

2003

Claude (Anthropic) was used in cataloguing the problems.

Rendezvous is posed in Dessmark et al. (2006). Deterministic rendezvous of labelled agents in arbitrary graphs was posed by Dessmark, Fraigniaud and Pelc at ESA 2003. Rendezvous itself is older (randomized search games); the catalogue covers the deterministic line (Pelc 2019). In the logical view: subproblem of Graph exploration by a single agent, which was posed earlier (1951).

- Dessmark et al. (2006) — Rendezvous: Labelled agents, arbitrary finite graphs [introductory], Labelled agents, arbitrary finite graphs [lower bound], Labelled agents in trees [current best], Labelled agents in trees [lower bound], Labelled agents in rings, simultaneous start [current best], Labelled agents in rings, simultaneous start [lower bound]
- Cieliebak et al. (2012) — Gathering: ASYNC with weak multiplicity detection [current best]
- Yanovski et al. (2003) — Exploration of recolourable graphs: Rotor-router: a pointer in every node [current best]

2004

Collaborative exploration is posed in Fraigniaud et al. (2006a). Collective tree exploration was posed at LATIN 2004 (journal version 2006).

Flocking is posed in Gervasi and Prencipe (2004). Coordination without communication: the flocking problem.

- Fraigniaud et al. (2005a) — Graph exploration by a single agent: Arbitrary graphs, perpetual exploration [current best], Arbitrary graphs, perpetual exploration [lower bound]
- Fraigniaud and Ilcinkas (2004) — Graph exploration by a single agent: Directed graphs, one pebble [current best], Directed graphs, one pebble [lower bound]
- Fraigniaud et al. (2006a) — Collaborative exploration: Trees, complete communication [introductory], Trees, communication by whiteboards only [current best], Trees, no communication [current best], Trees, no communication [lower bound]
- Cohen and Peleg (2005) — Convergence: ASYNC, unlimited visibility [current best]
- Gervasi and Prencipe (2004) — Flocking: Oblivious robots following a leader [current best]

2005

- Reingold (2008) — Graph exploration by a single agent: Exploration with stop when n is known [current best]
- Fraigniaud et al. (2005b) — Graph exploration by a single agent: Exploration with stop and one pebble [current best]
- Fraigniaud et al. (2006c) — Graph exploration by a single agent: Exploration with stop and one pebble [lower bound]
- Cohen et al. (2008) — Graph exploration by a single agent: Node labels chosen by the designer [current best]; Exploration of recolourable graphs: Label-guided: colours fixed by a designer [current best], Label-guided: colours fixed by a designer [lower bound], Label-guided, oblivious agent [lower bound]
- Fleischer and Trippen (2005) — Graph exploration by a single agent: Directed graphs with labelled nodes [current best]
- De Marco et al. (2006) — Rendezvous: Asynchronous labelled agents [introductory]
- Prencipe (2007) — Gathering: No multiplicity detection, SSYNC [lower bound], Gathering, full lights, SSYNC, no multiplicity detection [lower bound], Gathering, full lights, ASYNC, no multiplicity detection [lower bound]
- Klasing et al. (2007) — Black hole search: Synchronous agents with a map [current best], Synchronous agents with a map [lower bound]

Claude (Anthropic) was used in cataloguing the problems.

- Flocchini et al. (2008c) — Network decontamination: Hypercubes [current best], Hypercubes [lower bound]

2006

Uniform circle formation is posed in Dieudonné et al. (2008). The first deterministic solution (SSS 2006); circle formation with convergence towards uniformity predates it (Défago and Konagaya 2002). In the logical view: special case of Pattern formation, which was posed earlier (1999).

Uniform deployment on a ring is posed in Flocchini et al. (2008b). Self-deployment of mobile sensors on a ring (ALGOSENSORS 2006).

- Ilcinkas (2008) — Graph exploration by a single agent: Port numbers chosen by the designer (periodic exploration) [introductory]
- Agmon and Peleg (2006) — Gathering: Crash faults, SSYNC [introductory], Byzantine faults, SSYNC [lower bound], Byzantine faults, FSYNC [current best]
- Czyzowicz et al. (2009) — Gathering: Fat robots, few robots, no common orientation [current best]
- Dieudonné et al. (2008) — Uniform circle formation: SSYNC, oblivious [introductory]
- Czyzowicz et al. (2006) — Black hole search: Synchronous agents with a map [introductory]
- Flocchini et al. (2008b) — Uniform deployment on a ring: Oriented ring, asynchronous, oblivious [current best], Oriented ring, asynchronous, oblivious [lower bound]

2007

Treasure hunt is posed in Ta-Shma and Zwick (2014). Ta-Shma and Zwick introduced the deterministic treasure hunt problem at SODA 2007, as a variant of rendezvous in which one agent never moves. In the logical view: subproblem of Graph exploration by a single agent, which was posed earlier (1951).

Leader election in the plane is posed in Dieudonné et al. (2013). Posed for disoriented robots at OPODIS 2007 (Dieudonné and Petit); characterised in (Dieudonné et al. 2013).

k-grouping is posed in Efrima and Peleg (2009). Splitting a swarm into a given number of groups was introduced at SIROCCO 2007, under the name partitioning. No paper titled “k-grouping” was found. In the logical view: special case of Pattern formation, which was posed earlier (1999).

Exploration by oblivious robots is posed in Flocchini et al. (2013). Ring exploration by asynchronous oblivious robots (OPODIS 2007). In the logical view: variant of Graph exploration by a single agent, which was posed earlier (1951).

- Ambühl et al. (2011) — Graph exploration by a single agent: Trees, exploration with stop [current best], Trees, exploration with return [current best]
- Ta-Shma and Zwick (2014) — Treasure hunt: Anonymous finite graphs [introductory]; Rendezvous: Labelled agents, arbitrary finite graphs [current best]
- Xin (2007) — Treasure hunt: Anonymous finite graphs [current best]
- Das et al. (2007) — Map construction: Identical agents with whiteboards [current best], Identical agents with whiteboards [lower bound]

Claude (Anthropic) was used in cataloguing the problems.

- Efrima and Peleg (2009) — k-grouping: k groups of equal size [current best], k equal groups, no common orientation, deterministic [lower bound], k equal groups, randomized, SSYNC [current best]
- Ilcinkas et al. (2009) — Network decontamination: Unknown graphs, monotone strategies [current best], Unknown graphs, monotone strategies [lower bound]
- Flocchini et al. (2013) — Exploration by oblivious robots: Rings, exploration with stop [introductory], Rings, exploration with stop [lower bound]

2008

- Fraigniaud and Pelc (2013) — Rendezvous: Anonymous agents in trees, simultaneous start [current best], Anonymous agents in trees, simultaneous start [lower bound]
- Flocchini et al. (2008a) — Pattern formation: ASYNC, no agreement on coordinates [introductory], ASYNC, no agreement on coordinates [lower bound], ASYNC with one common axis [current best], ASYNC with one common axis [lower bound], ASYNC with a common compass [current best]
- Défago and Souissi (2008) — Circle formation: Circle formation, SSYNC [current best]
- Flocchini et al. (2012b) — Black hole search: Tokens and a map [current best]

2009

Scattering is posed in Dieudonné and Petit (2009). Dieudonné and Petit showed scattering is deterministically impossible and gave a randomized solution (preprint 2007).

- Czyzowicz et al. (2012c) — Graph exploration by a single agent: Port numbers chosen by the designer (periodic exploration) [current best], Port numbers chosen by the designer (periodic exploration) [lower bound]
- Dieudonné and Petit (2009) — Scattering: Randomized, SSYNC, strong multiplicity detection [introductory], Randomized, SSYNC, strong multiplicity detection [lower bound]

2010

Pattern sequences is posed in Das et al. (2015). Introduced at PODC 2010. In the logical view: generalisation of Pattern formation, which was posed earlier (1999).

- Czyzowicz et al. (2012b) — Rendezvous: Anonymous agents: memory [current best], Anonymous agents: memory [lower bound]
- Dieudonné et al. (2010) — Pattern formation: ASYNC with common chirality [current best]
- Chaudhuri and Mukhopadhyaya (2010) — Gathering: Transparent fat robots, no chirality [introductory]
- Chalopin et al. (2010) — Map construction: One agent, no marking [current best]
- Clement et al. (2010) — Scattering: Randomized, SSYNC, strong multiplicity detection [current best]
- Izumi et al. (2018) — Scattering: Limited visibility, connectivity preserving [current best], Limited visibility, connectivity preserving [lower bound]
- Das et al. (2015) — Pattern sequences: Oblivious, SSYNC, chirality [current best]
- Lamani et al. (2010) — Exploration by oblivious robots: Rings, exploration with stop [current best], Rings, exploration with stop [lower bound]

2011

Claude (Anthropic) was used in cataloguing the problems.

Patrolling is posed in Czyzowicz et al. (2011). Boundary patrolling by agents with distinct speeds (ESA 2011).

- Guilbault and Pelc (2011) — Rendezvous: Asynchronous anonymous agents [current best]
- Degener et al. (2011) — Gathering: Limited visibility, FSYNC, disoriented [introductory]
- Katreniak (2011) — Convergence: Limited visibility, asynchronous [introductory]
- Czyzowicz et al. (2011) — Patrolling: Open fence, distinct speeds [introductory]

2012

Gathering in graphs is posed in Dieudonné et al. (2014). Gathering many agents in arbitrary graphs grew out of two-agent rendezvous; the Byzantine version was posed at SODA 2012. In the logical view: generalisation of Rendezvous, which was posed earlier (2003).

Near-gathering is posed in Pagli et al. (2015). Introduced at SIROCCO 2012. In the logical view: relaxation of Gathering, which was posed earlier (1999).

- Dieudonné and Pelc (2012) — Graph exploration by a single agent: Exploration with stop, nothing known [lower bound]
- Das et al. (2016) — Gathering: Rendezvous, full lights (LUMI), ASYNC, non-rigid [introductory]
- Viglietta (2013) — Gathering: Rendezvous, full lights (LUMI), ASYNC, non-rigid [lower bound]
- Chaudhuri and Mukhopadhyaya (2015) — Gathering: Transparent fat robots, no chirality [current best]; Leader election in the plane: Transparent fat robots, no chirality [current best], Transparent fat robots, no chirality [lower bound]
- Balamohan et al. (2016) — Black hole search: Tokens, no map [current best], Tokens, no map [lower bound]
- Dieudonné et al. (2014) — Gathering in graphs: Weakly Byzantine agents [current best], Weakly Byzantine agents [lower bound], Byzantine agents [introductory], Byzantine agents [lower bound]
- Pagli et al. (2015) — Near-gathering: ASYNC, limited visibility, one common axis [current best]
- Kawamura and Kobayashi (2015) — Patrolling: Open fence, distinct speeds [lower bound]
- Devismes et al. (2012) — Exploration by oblivious robots: Grids, exploration with stop [current best], Grids, exploration with stop [lower bound]

2013

- Ilcinkas and Wade (2018) — Graph exploration by a single agent: Dynamic rings [current best], Dynamic rings [lower bound]
- Dereniowski et al. (2015) — Collaborative exploration: Large teams, complete communication [current best], Large teams, complete communication [lower bound], Large teams, local communication [current best], Large teams, local communication [lower bound]
- Dieudonné et al. (2015) — Rendezvous: Asynchronous labelled agents [current best]
- Flocchini et al. (2016) — Gathering: Two robots, SSYNC [lower bound], Rendezvous, external light (FCOM), SSYNC [current best], Rendezvous, external light (FCOM), ASYNC, non-rigid [current best], Rendezvous, external light (FCOM), ASYNC, rigid [introductory], Rendezvous, internal light (FSTA), SSYNC, non-rigid [current best], Rendezvous, internal light (FSTA), SSYNC, rigid [introductory]
- Agathangelou et al. (2013) — Gathering: Fat robots, chirality, any number [current best]

Claude (Anthropic) was used in cataloguing the problems.

- Dobrev et al. (2013) — Black hole search: Tokens, no map [introductory]
- Dieudonné and Pelc (2016) — Gathering in graphs: Anonymous agents [current best], Anonymous agents [lower bound]
- Dieudonné et al. (2013) — Leader election in the plane: Disoriented robots, with or without chirality [current best], Disoriented robots, with or without chirality [lower bound]
- Datta et al. (2013) — Exploration by luminous robots: Rings, FSYNC, terminating, $\varphi = 1$ [lower bound], Rings, oblivious, FSYNC, terminating, $\varphi = 1$ [current best], Rings, oblivious, FSYNC, terminating, $\varphi = 1$ [lower bound], Rings, oblivious, ASYNC, terminating, $\varphi = 1$ [lower bound]

2014

Evacuation is posed in Czyzowicz et al. (2014). Czyzowicz et al. posed evacuation of k robots from a disk through an unknown exit on its boundary at DISC 2014. In the logical view: generalisation of Linear search (cow path), which was posed earlier (1964).

Mutual visibility is posed in Di Luna et al. (2017). Introduced at CCCG 2014 for oblivious robots and at SSS 2014 for robots with lights (Di Luna et al. 2014); journal version (Di Luna et al. 2017). In the logical view: special case of Pattern formation, which was posed earlier (1999).

Coating is posed in Derakhshandeh et al. (2017). Infinite object coating appeared as an arXiv preprint in 2014; universal coating in (Derakhshandeh et al. 2017).

- Miller and Pelc (2015) — Treasure hunt: Treasure hunt with advice [current best], Treasure hunt with advice [lower bound]
- Flocchini et al. (2017) — Circle formation: Circle formation [current best]; Uniform circle formation: ASYNC, oblivious, disoriented [current best], ASYNC, oblivious, disoriented [lower bound]
- Czyzowicz et al. (2014) — Evacuation: Two wireless robots, disk [current best], Two wireless robots, disk [lower bound], Two face-to-face robots, disk [introductory], Three wireless robots, disk [current best], Three wireless robots, disk [lower bound], Three face-to-face robots, disk [current best], Three face-to-face robots, disk [lower bound], Many wireless robots, disk [current best], Many wireless robots, disk [lower bound], Many face-to-face robots, disk [current best], Many face-to-face robots, disk [lower bound]
- Das et al. (2020) — Pattern sequences: Luminous robots, ASYNC, non-rigid, chirality [current best], Luminous robots, ASYNC, non-rigid, chirality [lower bound]

2015

Leader election in programmable matter is posed in Derakhshandeh et al. (2015). Introduced with shape formation at DNA 21 (2015).

Shape formation in programmable matter is posed in Derakhshandeh et al. (2015). Introduced with leader election at DNA 21 (2015).

- Erlebach et al. (2021) — Graph exploration by a single agent: Temporal graphs, schedule known [current best], Temporal graphs, schedule known [lower bound]
- Czyzowicz et al. (2020) — Evacuation: Two face-to-face robots, disk [lower bound]
- Chrobak et al. (2015) — Evacuation: Line, any number of face-to-face robots [current best], Line, any number of wireless robots [current best], Line, any number of wireless robots [lower bound]

Claude (Anthropic) was used in cataloguing the problems.

- Czyzowicz et al. (2015) — Evacuation: Triangle, two wireless robots starting on the boundary [introductory]
- Bouchard et al. (2016) — Gathering in graphs: Byzantine agents [current best]
- Bramas and Tixeuil (2017) — Scattering: Randomized, SSYNC, weak multiplicity detection [current best], Randomized, SSYNC, weak multiplicity detection [lower bound]
- Vaidyanathan et al. (2015) — Mutual visibility: Lights, FSYNC, chirality [current best]
- Derakhshandeh et al. (2015) — Leader election in programmable matter: Randomized, with chirality [introductory]; Shape formation in programmable matter: SILBOT, full orientation [lower bound]
- Czyzowicz et al. (2017b) — Patrolling: Graphs with faulty patrollers [current best], Graphs with faulty patrollers [lower bound]

2016

Exploration of dynamic graphs by teams is posed in Di Luna et al. (2020a). Live exploration of dynamic rings was posed at ICDCS 2016. In the logical view: variant of Collaborative exploration, which was posed earlier (2004).

Compression is posed in Cannon et al. (2016). PODC 2016.

- Disser et al. (2019) — Graph exploration by a single agent: Exploration with stop when n is known [lower bound], Exploration with return and pebbles [current best], Exploration with return and pebbles [lower bound]; Collaborative exploration: Anonymous graphs, a team of constant-memory agents [current best], Anonymous graphs, a team of constant-memory agents [lower bound]
- Czyzowicz et al. (2016b) — Linear search (cow path): k robots on a line, f crash faults [current best]
- Czyzowicz et al. (2021a) — Linear search (cow path): k robots on a line, Byzantine faults [current best]
- Bampas et al. (2019) — Linear search (cow path): Two robots of different speeds, both must reach the target [current best], Two robots of different speeds, both must reach the target [lower bound]; Evacuation: Line, any number of face-to-face robots [lower bound], Line, two face-to-face robots of different speeds [current best], Line, two face-to-face robots of different speeds [lower bound], Line, two wireless robots of different speeds [current best], Line, two wireless robots of different speeds [lower bound]
- Lamprou et al. (2020) — Evacuation: Disk, two wireless robots of different speeds [current best], Disk, two wireless robots of different speeds [lower bound]
- Di Luna et al. (2020a) — Exploration of dynamic graphs by teams: 1-interval-connected rings [introductory], 1-interval-connected rings [lower bound]
- Derakhshandeh et al. (2016) — Shape formation in programmable matter: Universal shapes, sequential, randomized [current best], Universal shapes, sequential, randomized [lower bound]
- Daymude et al. (2018) — Coating: Universal coating [current best], Universal coating [lower bound]
- Cannon et al. (2016) — Compression: Markov chain, sequential [current best]

2017

Bridging is posed in Arroyo et al. (2018). arXiv 2017; Natural Computing 2018. In the logical view: generalisation of Compression, which was posed earlier (2016).

Shape formation by hybrid programmable matter is posed in Gmyr et al. (2020). First presented at EuroCG 2017 (non-archival); DNA 24 (2018); Natural Computing 2020. The work began at a Dagstuhl seminar in 2016.

Claude (Anthropic) was used in cataloguing the problems.

Centralised transformations of modular robots is posed in Michail et al. (2019). arXiv 2017; JCSS 2019.

- Disser et al. (2020) — Collaborative exploration: Trees, complete communication [lower bound], Trees with $k = n$ agents [current best], Trees with $k = n$ agents [lower bound]; Dispersion: Trees [lower bound], Rooted trees [lower bound]
- Pattanayak et al. (2019b) — Gathering: Crash faults, SSYNC [current best], Crash faults, SSYNC [lower bound]
- Poudel and Sharma (2021) — Gathering: Limited visibility, one common axis, ASYNC [current best], Limited visibility, one common axis, ASYNC [lower bound]
- Heriban et al. (2018) — Gathering: Rendezvous, full lights (LUMI), ASYNC, non-rigid [current best]
- Mondal and Chaudhuri (2017) — Uniform circle formation: Transparent fat robots, SSYNC [current best]
- Czyzowicz et al. (2017a) — Evacuation: Three wireless robots, one crash fault [current best], Three wireless robots, one crash fault [lower bound], Three wireless robots, one Byzantine [current best], Three wireless robots, one Byzantine [lower bound]
- Georgiou et al. (2019) — Evacuation: Disk, search-and-fetch, two wireless robots [current best], Disk, search-and-fetch, two face-to-face robots [current best], Disk, search-and-fetch, two face-to-face robots [lower bound]
- Brandt et al. (2020) — Evacuation: m rays, $k = m$ wireless robots at the junction [current best], m rays, $k = m$ wireless robots at the junction [lower bound]
- Di Luna et al. (2017) — Mutual visibility: Lights, SSYNC, rigid [current best], Lights, SSYNC, rigid [lower bound], Lights, SSYNC, non-rigid [current best], Lights, SSYNC, non-rigid [lower bound], Lights, ASYNC, rigid [introductory], Lights, ASYNC, rigid [lower bound], Lights, ASYNC, non-rigid, one axis [current best], Lights, ASYNC, non-rigid, one axis [lower bound]
- Sharma et al. (2021) — Mutual visibility: Lights, ASYNC, rigid [current best], Lights, ASYNC, rigid [lower bound]
- Aljohani and Sharma (2018) — Mutual visibility: Lights, one crash, SSYNC, rigid, compass [current best], Lights, one Byzantine robot [lower bound]
- Daymude et al. (2017) — Leader election in programmable matter: Randomized, with chirality [current best]
- Di Luna et al. (2020c) — Shape formation in programmable matter: Deterministic, simply connected start, no chirality [current best], Deterministic, simply connected start, no chirality [lower bound]
- Derakhshandeh et al. (2017) — Coating: Universal coating [introductory], Universal coating [lower bound]
- Menc et al. (2017) — Exploration of recolourable graphs: Rotor-router: a pointer in every node [lower bound]
- Gmyr et al. (2020) — Shape formation by hybrid programmable matter: Planar [current best], Planar [lower bound]

2018

Dispersion is posed in Augustine and Moses Jr. (2018). Posed at ICDCN 2018, fourteen years after collaborative exploration; the introductory paper already relates dispersion to n-robot collaborative exploration and takes its tree lower bound from it. In the logical view: derived from Collaborative exploration, which was posed earlier (2004).

Convex hull formation is posed in Daymude et al. (2020). arXiv 2018; ICDCN 2020.

Exploration by luminous robots is posed in Ooshita and Tixeuil (2022). Ring exploration by myopic robots with a two-colour light (SSS 2018). In the logical view: variant of Exploration by oblivious robots, which was posed earlier (2007).

Claude (Anthropic) was used in cataloguing the problems.

Separation of coloured particles is posed in Cannon et al. (2019). arXiv 2018; APPROX/RANDOM 2019. In the logical view: generalisation of Compression, which was posed earlier (2016).

- Augustine and Moses Jr. (2018) — Collaborative exploration: Anonymous graphs with $k = n$ agents [introductory]; Dispersion: Arbitrary graph, arbitrary initial configuration [introductory], Arbitrary graph, arbitrary initial configuration [lower bound], Rooted initial configuration [introductory], Rooted initial configuration [lower bound], Asynchronous agents [lower bound], Rings and paths [introductory], Rings and paths [lower bound], Trees [introductory], Trees [lower bound], Rooted trees [current best], Rooted trees [lower bound], Crash faults [lower bound], Weak Byzantine agents [lower bound], Weak Byzantine agents on rings [lower bound], Weak Byzantine agents on grids [lower bound]
- Agarwalla et al. (2018) — Dispersion: Dynamic rings [current best]
- Cicerone et al. (2019) — Pattern formation: ASYNC, no agreement on coordinates [current best]
- Okumura et al. (2023) — Gathering: Rendezvous, external light (FCOM), SSYNC [lower bound]
- Terai et al. (2023) — Gathering: Gathering, full lights, SSYNC, no multiplicity detection [current best], Gathering, full lights, ASYNC, no multiplicity detection [introductory], Gathering, external light (FCOM), SSYNC, rigid [current best], Gathering, internal light (FSTA), SSYNC, non-rigid [introductory], Gathering, external light, sequential scheduler [current best], Gathering, external light, sequential scheduler [lower bound]
- Feletti et al. (2018) — Uniform circle formation: Opaque robots with lights, FSYNC [introductory]
- Kupavskii and Welzl (2018) — Linear search (cow path): k robots on a line, f crash faults [lower bound], k robots on a line, Byzantine faults [lower bound]; Evacuation: Line, $2f + 1$ wireless robots, f crash faults [lower bound]
- Czyzowicz et al. (2018b) — Evacuation: Priority evacuation (queen and servants), disk [current best], Priority evacuation (queen and servants), disk [lower bound]
- Pattanayak et al. (2018) — Evacuation: Two robots, two exits, face-to-face [current best], Two robots, two exits, face-to-face [lower bound], Two robots, two exits, wireless [current best], Two robots, two exits, wireless [lower bound]
- Czyzowicz et al. (2018a) — Evacuation: Disk, priority evacuation, queen and one servant [introductory], Disk, priority evacuation, queen and two servants [current best], Disk, priority evacuation, queen and two servants [lower bound]
- Sharma et al. (2018) — Mutual visibility: Fat robots, FSYNC, rigid [introductory]
- Gastineau et al. (2019) — Leader election in programmable matter: Stationary, chirality, no holes [current best]
- Bazzi and Briones (2019) — Leader election in programmable matter: Stationary, chirality, weak scheduler [current best]
- Ooshita and Tixeuil (2022) — Exploration by luminous robots: Rings, FSYNC, perpetual, $\varphi = 1$ [current best], Rings, FSYNC, perpetual, $\varphi = 1$ [lower bound], Rings, FSYNC, terminating, $\varphi = 1$ [current best], Rings, FSYNC, terminating, $\varphi = 1$ [lower bound], Rings, ASYNC, perpetual, $\varphi = 1$ [current best], Rings, ASYNC, perpetual, $\varphi = 1$ [lower bound], Rings, ASYNC, terminating, $\varphi = 1$ [current best], Rings, ASYNC, terminating, $\varphi = 1$ [lower bound]
- Arroyo et al. (2018) — Bridging: Markov chain, sequential [current best]

2019

Chauffeuring a crashed robot is posed in Pattanayak et al. (2019a). ALGOSENSORS 2019. In the logical view: variant of Evacuation, which was posed earlier (2014).

- Kshemkalyani and Ali (2019) — Collaborative exploration: Anonymous graphs with $k = n$ agents [lower bound]; Dispersion: Arbitrary graph, arbitrary initial configuration [lower bound], Rooted initial configuration [lower bound], Asynchronous agents [introductory], Asynchronous agents [lower bound], Rings and paths [lower bound], Anonymous agents with randomness [lower bound], Crash faults [lower bound], Weak Byzantine agents [lower bound], Strong Byzantine agents [lower bound]

Claude (Anthropic) was used in cataloguing the problems.

- Molla and Moses Jr. (2019) — Dispersion: More agents than nodes [current best], More agents than nodes [lower bound], Anonymous agents with randomness [introductory], Anonymous agents with randomness [lower bound]
- Bose et al. (2021) — Pattern formation: Opaque robots with lights, ASYNC, one axis, rigid [current best], Opaque robots with lights, ASYNC, compass, rigid [current best], Opaque oblivious robots, ASYNC, compass [lower bound]
- Bose et al. (2020) — Pattern formation: Opaque fat robots with lights, ASYNC, one axis [current best]
- Disser and Schmitt (2019) — Evacuation: Two face-to-face robots, disk [current best]
- Gotoh et al. (2021) — Exploration of dynamic graphs by teams: Temporally connected graphs, arbitrary topology [current best], Temporally connected graphs, arbitrary topology [lower bound]
- Emek et al. (2019) — Leader election in programmable matter: Deterministic, no orientation [current best]
- Nagahama et al. (2023) — Exploration by luminous robots: Rings, ASYNC, perpetual, $\varphi \geq 2$ [current best]
- Pattanayak et al. (2019a) — Chauffeuring a crashed robot: e exits [current best], One exit [current best], Two exits [current best]
- Cannon et al. (2019) — Separation of coloured particles: Markov chain, sequential [current best]
- Michail et al. (2019) — Centralised transformations of modular robots: Rotation only [current best], Rotation and sliding [current best], Rotation and sliding [lower bound]

2020

Energy distribution is posed in Daymude et al. (2021). arXiv 2020; ICDCN 2021.

- Kshemkalyani et al. (2020c) — Dispersion: Grids [current best], Grids [lower bound], Crash faults on grids [lower bound], Weak Byzantine agents on grids [lower bound]
- Kshemkalyani et al. (2020a) — Dispersion: Dynamic graphs (1-interval connected) [lower bound], Dynamic graphs with global communication [current best], Dynamic graphs with global communication [lower bound]
- Molla et al. (2020) — Dispersion: Weak Byzantine agents on rings [current best], Weak Byzantine agents on rings [lower bound]
- Mondal et al. (2020) — Uniform circle formation: Opaque fat robots, ASYNC, limited visibility, compass [current best]
- Chuangpishit et al. (2020b) — Evacuation: Two face-to-face robots, equilateral triangle [current best], Two face-to-face robots, equilateral triangle [lower bound], Two face-to-face robots, square [current best], Two face-to-face robots, square [lower bound], Triangle, any number of face-to-face robots [current best], Triangle, any number of face-to-face robots [lower bound]
- D'Angelo et al. (2020) — Leader election in programmable matter: Silent oblivious particles (SILBOT) [current best], SILBOT with holes [current best], SILBOT with holes [lower bound]
- Daymude et al. (2020) — Convex hull formation: No orientation [current best]
- Kawamura and Soejima (2020) — Patrolling: Open fence, distinct speeds [current best], Closed fence, distinct speeds [current best]

2021

Partitioning is posed in Bhagat et al. (2021). The fixed-point version (k-EPF) was posed with k-circle formation in 2021; the earlier partitioning of Efrima and Peleg fixes the number of groups instead and is recorded under k-grouping. In the logical view: special case of Pattern formation, which was posed earlier (1999).

- Das et al. (2021b) — Dispersion: Anonymous agents with randomness [current best]

Claude (Anthropic) was used in cataloguing the problems.

- Pattanayak et al. (2021) — Dispersion: Crash faults [introductory]
- Molla et al. (2021) — Dispersion: Weak Byzantine agents [current best], Weak Byzantine agents [lower bound], Strong Byzantine agents [introductory]
- Bouchard et al. (2023a) — Treasure hunt: Labelled nodes, locally finite graphs [current best], Labelled nodes, locally finite graphs [lower bound]
- Gorain et al. (2022a) — Treasure hunt: Treasure hunt guided by pebbles [current best], Treasure hunt guided by pebbles [lower bound]
- Nakai et al. (2021) — Gathering: Gathering, full lights, ASYNC, no multiplicity detection [current best]
- Kirkpatrick et al. (2024) — Convergence: Limited visibility, asynchronous [current best]
- Czyzowicz et al. (2021b) — Evacuation: Line, a sender and a receiver [current best], Line, a sender and a receiver [lower bound]
- Czyzowicz et al. (2021c) — Evacuation: Line, $2f + 1$ wireless robots, f crash faults [current best]
- Di Luna et al. (2025b) — Black hole search: Dynamic rings [current best], Dynamic rings [lower bound]
- Dufoulon et al. (2021) — Leader election in programmable matter: Deterministic, with chirality [current best]
- Bramas et al. (2023a) — Exploration by luminous robots: Grids, FSYNC, perpetual, chirality, $\varphi = 1$ [current best], Grids, FSYNC, perpetual, chirality, $\varphi = 1$ [lower bound], Grids, FSYNC, perpetual, chirality, $\varphi = 2$ [current best], Grids, FSYNC, perpetual, chirality, $\varphi = 2$ [lower bound], Grids, ASYNC, perpetual, chirality, $\varphi = 2$ [current best], Grids, ASYNC, perpetual, chirality, $\varphi = 2$ [lower bound], Grids, FSYNC, terminating, chirality, $\varphi = 1$ [lower bound], Grids, FSYNC, terminating, no chirality, $\varphi = 1$ [lower bound], Grids, FSYNC, terminating, chirality, $\varphi = 2$ [lower bound], Grids, FSYNC, terminating, no chirality, $\varphi = 2$ [lower bound], Grids, ASYNC, terminating, chirality, $\varphi = 2$ [lower bound], Grids, ASYNC, terminating, no chirality, $\varphi = 2$ [lower bound]
- Rauch et al. (2021) — Exploration by luminous robots: Grids, FSYNC, perpetual, no chirality, $\varphi = 1$ [current best], Grids, FSYNC, perpetual, no chirality, $\varphi = 1$ [lower bound], Grids, ASYNC, perpetual, no chirality, $\varphi = 2$ [lower bound], Grids, ASYNC, perpetual, no chirality, $\varphi = 3$ [lower bound]
- Nagahama et al. (2022) — Exploration by luminous robots: Grids, FSYNC, terminating, chirality, $\varphi = 1$ [current best], Grids, FSYNC, terminating, no chirality, $\varphi = 1$ [current best], Grids, FSYNC, terminating, chirality, $\varphi = 2$ [current best], Grids, FSYNC, terminating, no chirality, $\varphi = 2$ [current best], Grids, ASYNC, terminating, chirality, $\varphi = 1$ [current best], Grids, ASYNC, terminating, chirality, $\varphi = 1$ [lower bound], Grids, ASYNC, terminating, no chirality, $\varphi = 1$ [current best], Grids, ASYNC, terminating, no chirality, $\varphi = 1$ [lower bound], Grids, ASYNC, terminating, chirality, $\varphi = 2$ [current best], Grids, ASYNC, terminating, no chirality, $\varphi = 2$ [current best]
- Bhagat et al. (2021) — Partitioning: Exactly n/k robots at each fixed point [current best]
- Daymude et al. (2021) — Energy distribution: Unfair adversary [current best], Unfair adversary [lower bound]

2022

Alignment is posed in Kedia et al. (2022). APPROX/RANDOM 2022. In the logical view: generalisation of Compression, which was posed earlier (2016).

- Kshemkalyani et al. (2022) — Dispersion: Global communication [introductory], Global communication [lower bound]
- Italiano et al. (2022) — Dispersion: Directed graphs [current best], Directed graphs [lower bound], Directed graphs, 1-hop communication [current best], Directed graphs, 1-hop communication [lower bound]
- Gorain et al. (2022b) — Dispersion: Silent agents (no communication) [current best], Silent agents (no communication) [lower bound], Silent agents on rings [lower bound]
- Castenow et al. (2022) — Gathering: Limited visibility, FSYNC, disoriented [current best], Limited visibility, FSYNC, disoriented [lower bound]; Near-gathering: SSYNC, limited visibility, disoriented [current best], SSYNC, limited visibility, disoriented [lower bound]

Claude (Anthropic) was used in cataloguing the problems.

- Feletti et al. (2023b) — Uniform circle formation: Opaque robots with lights, ASYNC [introductory], Opaque robots with lights, FSYNC [current best]
- Georgiou et al. (2022) — Evacuation: Two robots with asymmetric communication, disk [current best], Two robots with asymmetric communication, disk [lower bound]
- Georgiou and Jang (2026) — Evacuation: Triangle, two wireless robots starting on the boundary [current best], Triangle, two wireless robots starting on the boundary [lower bound]
- Alsaedi et al. (2023) — Mutual visibility: Fat robots, FSYNC, rigid [current best], Fat robots, FSYNC, rigid [lower bound]
- Darwich et al. (2023) — Exploration by luminous robots: Tori, FSYNC, perpetual, chirality, $\varphi = 1$ [current best], Tori, FSYNC, perpetual, chirality, $\varphi = 1$ [lower bound], Tori, FSYNC, perpetual, no chirality, $\varphi = 1$ [current best], Tori, FSYNC, perpetual, no chirality, $\varphi = 1$ [lower bound], Tori, FSYNC, perpetual, chirality, $\varphi = 2$ [current best], Tori, FSYNC, perpetual, chirality, $\varphi = 2$ [lower bound], Tori, FSYNC, perpetual, no chirality, $\varphi = 2$ [current best], Tori, FSYNC, perpetual, no chirality, $\varphi = 2$ [lower bound]
- Bojko et al. (2022) — Exploration of recolourable graphs: Recolourable nodes, oblivious agent, any graph [lower bound], Recolourable nodes on cacti [lower bound], Recolourable nodes on trees [lower bound], Label-guided: colours fixed by a designer [lower bound]
- Kedia et al. (2022) — Alignment: Markov chain, sequential [current best]

2023

Minimal dominating set by agents is posed in Chand et al. (2023a). Introduced at ALGOSENSORS 2023.

Separation is posed in Seike and Yamauchi (2023). Introduced at SSS 2023.

Exploration of recolourable graphs is posed in Böckenhauer et al. (2023). Böckenhauer, Frei, Unger and Wehner posed exploration by an oblivious agent without port numbers that colours nodes and sees its neighbours' colours (SIROCCO 2023; arXiv January 2023), first with each node coloured once and, in a last section, with recolouring. Fixed colours chosen by a designer (2005) and the rotor-router pointer (1996, 2003) are earlier node-side precursors. In the logical view: variant of Graph exploration by a single agent, which was posed earlier (1951).

Scattering of particles is posed in Navarra et al. (2025). Scattering in SILBOT first appeared at AINA 2023 (Navarra, Prencipe, Bonini, Tracolli); that paper's DOI was not verified, so the journal version is cited.

- Cosson et al. (2023) — Collaborative exploration: Trees, asynchronous, complete communication [introductory], Arbitrary graphs, complete communication [current best]
- Pattanayak et al. (2023) — Dispersion: Crash faults [current best]
- Pattanayak and Pelc (2024b) — Treasure hunt: Anonymous graphs with possibly infinite degrees [current best], Anonymous graphs with possibly infinite degrees [lower bound]; Rendezvous: Labelled agents, graphs with infinite degrees [current best], Labelled agents, graphs with infinite degrees [lower bound]
- Alsaedi et al. (2025) — Pattern formation: Fat robots with lights, FSYNC, randomized, no agreement [current best]
- Asahiro and Yamashita (2025) — Gathering: Homonymous robots, SSYNC [current best], Homonymous robots, SSYNC [lower bound]
- Bramas et al. (2023b) — Gathering: Rendezvous, full lights, a crash, SSYNC [lower bound]
- Molla et al. (2023) — Gathering in graphs: Labelled agents [current best]

Claude (Anthropic) was used in cataloguing the problems.

- Chand et al. (2023a) — Minimal dominating set by agents: No knowledge [introductory]
- Seike and Yamauchi (2023) — Separation: ASYNC, one common axis [introductory]
- Briones et al. (2023) — Leader election in programmable matter: Stationary, asynchronous, no orientation, no holes [current best], Stationary, sequential, no orientation, no holes [current best]
- Navarra and Piselli (2023b) — Shape formation in programmable matter: SILBOT, chirality [lower bound], SILBOT, full orientation [current best]
- Navarra and Piselli (2023a) — Coating: SILBOT, chirality, deterministic [current best], SILBOT, chirality, deterministic [lower bound]
- Böckenhauer et al. (2023) — Exploration of recolourable graphs: Recolourable nodes, oblivious agent, any graph [introductory], Once-colourable nodes, any graph [current best], Once-colourable nodes, any graph [lower bound], Once-colourable nodes, n known [current best], Once-colourable nodes, n known [lower bound], Once-colourable nodes on trees [current best], Once-colourable nodes on trees [lower bound]

2024

Leader election by agents is posed in Kshemkalyani et al. (2024). Leader election in the agent-based model, with MST as an application (DISC 2024).

Maximal independent set by agents is posed in Pattanayak et al. (2024). Introduced at ICDCN 2024.

Triangle and butterfly counting by agents is posed in Chand et al. (2026). Agent-based triangle counting (arXiv 2024; IEEE Transactions on Networking 2026).

Minimum spanning tree by agents is posed in Kshemkalyani et al. (2024). MST in the agent-based model appeared together with leader election (DISC 2024).

- Pattanayak and Pelc (2024a) — Graph exploration by a single agent: Memoryless automaton with pebbles [current best], Memoryless automaton with pebbles [lower bound]
- Banerjee et al. (2024) — Dispersion: Crash faults on grids [introductory]
- Moses Jr. and Redlich (2024) — Dispersion: Strong Byzantine agents [current best]
- Hahn et al. (2024) — Pattern formation: Limited visibility [current best]
- Otaka et al. (2026) — Gathering: Rendezvous, external light (FCOM), SSYNC [lower bound], Rendezvous, external light (FCOM), ASYNC, non-rigid [lower bound], Rendezvous, external light (FCOM), ASYNC, rigid [lower bound], Rendezvous, internal light (FSTA), SSYNC, non-rigid [lower bound], Rendezvous, internal light (FSTA), SSYNC, rigid [lower bound], Rendezvous, internal light (FSTA), ASYNC [lower bound], Gathering, external light (FCOM), SSYNC, rigid [lower bound], Gathering, internal light (FSTA), SSYNC, non-rigid [current best], Gathering, internal light (FSTA), SSYNC, non-rigid [lower bound]
- Feletti et al. (2024) — Uniform circle formation: Opaque robots with lights, ASYNC [current best], Opaque robots with lights, ASYNC [lower bound]
- Georgiou and Wang (2025) — Evacuation: Disk, priority evacuation, queen and one servant [current best], Disk, priority evacuation, queen and one servant [lower bound]
- Kshemkalyani et al. (2024) — Leader election by agents: No knowledge [introductory]; Maximal independent set by agents: No knowledge [introductory]; Minimum spanning tree by agents: Dispersed start [introductory]
- Pattanayak et al. (2024) — Maximal independent set by agents: Knowing n and Δ [introductory]
- Bhagat et al. (2024) — Mutual visibility: Fat robots, ASYNC, one axis [current best]

Claude (Anthropic) was used in cataloguing the problems.

- Flocchini et al. (2024) — Separation: ASYNC, one common axis [current best], ASYNC, one common axis [lower bound]
- Chalopin et al. (2026b) — Leader election in programmable matter: Stationary, common direction, no chirality [current best], Stationary, common direction, no chirality [lower bound]
- Ooshita et al. (2025a) — Exploration by luminous robots: Rings, ASYNC, perpetual, $\varphi \geq 2$ [lower bound], Rings, FSYNC, perpetual, up to f crashes [current best], Rings, FSYNC, perpetual, up to f crashes [lower bound], Rings, ASYNC, perpetual, up to f crashes, $\varphi = 1$ [current best], Rings, ASYNC, perpetual, up to f crashes, $\varphi = 1$ [lower bound], Rings, ASYNC, perpetual, up to f crashes, $\varphi \geq 2$ [current best], Rings, ASYNC, perpetual, up to f crashes, $\varphi \geq 2$ [lower bound]
- Bramas et al. (2026a) — Exploration by luminous robots: Grids, ASYNC, perpetual, chirality, $\varphi = 1$ (exclusive) [lower bound], Grids, ASYNC, perpetual, chirality, $\varphi = 2$ [lower bound], Grids, ASYNC, perpetual, no chirality, $\varphi = 2$ [current best], Grids, ASYNC, perpetual, no chirality, $\varphi = 2$ [lower bound], Grids, ASYNC, perpetual, no chirality, $\varphi = 3$ [current best]
- Hinnenthal et al. (2024) — Shape formation by hybrid programmable matter: Three-dimensional [current best]

2025

Universal pattern formation is posed in Flocchini et al. (2026). Introduced at SIROCCO 2025. In the logical view: strengthening of Pattern formation, which was posed earlier (1999).

BFS tree by agents is posed in Chand et al. (2025b). BFS without root knowledge in the agent-based model (SSS 2025).

- Cosson and Massoulié (2025) — Collaborative exploration: Trees, asynchronous, complete communication [lower bound], Trees, asynchronous, whiteboards only [current best]
- Kshemkalyani et al. (2025a) — Collaborative exploration: Anonymous graphs with $k = n$ agents [current best]; Dispersion: Arbitrary graph, arbitrary initial configuration [current best], Rooted initial configuration [current best], Global communication [current best], Rings and paths [current best], Trees [current best]
- Pattanayak et al. (2025) — Dispersion: Asynchronous agents [current best]
- Mondal et al. (2025a) — Dispersion: Dynamic rings [lower bound]
- Banerjee et al. (2025) — Dispersion: Crash faults on grids [current best], Weak Byzantine agents on grids [current best]
- Das et al. (2025b) — Dispersion: Silent agents (no communication) [lower bound], Silent agents on rings [current best], Silent agents on rings [lower bound]
- Di Luna et al. (2025a) — Black hole search: Byzantine companions, with a map [current best], Byzantine companions, whiteboards, no map [current best]
- Flocchini et al. (2026) — Universal pattern formation: Sequential schedulers [current best], Sequential schedulers [lower bound]
- Chand et al. (2025a) — Triangle and butterfly counting by agents: Butterfly counting in bipartite graphs [current best]
- Chand et al. (2025b) — Minimum spanning tree by agents: Dispersed start [current best]; BFS tree by agents: Dispersed start, root unknown [current best]
- Chalopin et al. (2025) — Leader election in programmable matter: Stationary, fair scheduler, no orientation, no holes [current best]
- Takahashi et al. (2026) — Exploration of recolourable graphs: Recolourable nodes, oblivious agent, any graph [current best]
- Navarra et al. (2025) — Scattering of particles: SILBOT [current best]

Claude (Anthropic) was used in cataloguing the problems.

2026

- Cosson and Massoulié (2026) — Collaborative exploration: Trees, complete communication [current best], Trees, asynchronous, complete communication [current best]
- Masuzawa et al. (2026) — Gathering: Rendezvous, external light (FCOM), ASYNC, rigid [current best], Rendezvous, internal light (FSTA), SSYNC, rigid [current best]
- Devismes et al. (2026) — Gathering in graphs: Homonymous agents [current best]
- Chand and Molla (2026) — Leader election by agents: No knowledge [current best]; Minimal dominating set by agents: No knowledge [current best]
- Kshemkalyani et al. (2026) — Leader election by agents: Knowing n and Δ [current best]; Maximal independent set by agents: Knowing n and Δ [current best]
- Kaur et al. (2026b) — Maximal independent set by agents: No knowledge [current best]
- Feletti et al. (2026) — Pattern sequences: Luminous robots, sequential scheduler (universal dancing) [current best], Luminous robots, sequential scheduler (universal dancing) [lower bound]
- Pyati et al. (2026) — Separation: Opaque robots, SSYNC [current best]
- Chand et al. (2026) — Triangle and butterfly counting by agents: Triangle counting [current best]
- Chalopin et al. (2026a) — Leader election in programmable matter: Oblivious, common compass, holes allowed [current best]
- Hiraoka et al. (2026) — Exploration of recolourable graphs: Recolourable nodes, oblivious agent, any graph [lower bound], Recolourable nodes on cacti [current best], Recolourable nodes on cacti [lower bound], Recolourable nodes on trees [current best], Recolourable nodes on trees [lower bound]

Claude (Anthropic) was used in cataloguing the problems.

Bibliography

Sumi Acharjee, Konstantinos Georgiou, Somnath Kundu, Akshaya Srinivasan. Lower Bounds for Shoreline Searching With 2 or More Robots. *OPODIS 2019, LIPIcs 153, 26:1–26:11*, 2020. doi:10.4230/LIPIcs.OPODIS.2019.26, arXiv:2001.04311.

Ankush Agarwalla, John Augustine, William K. Moses Jr., Madhav Sankar K., Arvind Krishna Sridhar. Deterministic Dispersion of Mobile Robots in Dynamic Rings. *ICDCN 2018*, 2018. doi:10.1145/3154273.3154294, arXiv:1707.06391.

Chrysovalandis Agathangelou, Chryssis Georgiou, Marios Mavronicolas. A distributed algorithm for gathering many fat mobile robots in the plane. *PODC 2013, pp. 250–259*, 2013. doi:10.1145/2484239.2484266, arXiv:1209.3904.

Noa Agmon, David Peleg. Fault-Tolerant Gathering Algorithms for Autonomous Mobile Robots. *SIAM Journal on Computing 36(1):56–82*, 2006. doi:10.1137/050645221.

Hugo A. Akitaya, Esther M. Arkin, Mirela Damian, Erik D. Demaine, Vida Dujmović, Robin Flatland, Matias Korman, Belen Palop, Irene Parada, André van Renssen, Vera Sacristán. Universal Reconfiguration of Facet-Connected Modular Robots by Pivots: The O(1) Musketeers. *Algorithmica 83:1316–1351*, 2021. doi:10.1007/s00453-020-00784-6.

Susanne Albers, Monika R. Henzinger. Exploring unknown environments. *SIAM Journal on Computing 29(4):1164–1188*, 2000. doi:10.1137/S009753979732428X.

Romas Aleliunas, Richard M. Karp, Richard J. Lipton, László Lovász, Charles Rackoff. Random walks, universal traversal sequences, and the complexity of maze problems. *FOCS 1979, pp. 218–223*, 1979. doi:10.1109/SFCS.1979.34.

Aisha Aljohani, Gokarna Sharma. Complete Visibility for Mobile Robots with Lights Tolerating Faults. *International Journal of Networking and Computing 8(1):32–52*, 2018. doi:10.15803/ijnc.8.1_32.

Abdullah Almethen, Othon Michail, Igor Potapov. Pushing lines helps: Efficient universal centralised transformations for programmable matter. *Theoretical Computer Science 830–831:43–59*, 2020. doi:10.1016/j.tcs.2020.04.026, arXiv:1904.12777.

Rusul J. Alsaedi, Joachim Gudmundsson, André van Renssen. The Mutual Visibility Problem for Fat Robots. *WADS 2023, LNCS 14079, pp. 15–28*, 2023. doi:10.1007/978-3-031-38906-1_2, arXiv:2206.14423.

Rusul J. Alsaedi, Joachim Gudmundsson, André van Renssen. Pattern formation for fat robots with lights. *Computational Geometry 128:102161*, 2025. doi:10.1016/j.comgeo.2024.102161, arXiv:2306.14440.

Christoph Ambühl, Leszek Gąsieniec, Andrzej Pelc, Tomasz Radzik, Xiaohui Zhang. Tree exploration with logarithmic memory. *ACM Transactions on Algorithms 7(2), article 17 (SODA 2007)*, 2011. doi:10.1145/1921659.1921663.

Hideki Ando, Yoshinobu Oasa, Ichiro Suzuki, Masafumi Yamashita. Distributed memoryless point convergence algorithm for mobile robots with limited visibility. *IEEE Transactions on Robotics and Automation 15(5):818–828*, 1999. doi:10.1109/70.795787.

Marta Andrés Arroyo, Sarah Cannon, Joshua J. Daymude, Dana Randall, Andréa W. Richa. A stochastic approach to shortcut bridging in programmable matter. *Natural Computing 17:723–741*, 2018. doi:10.1007/s11047-018-9714-x, arXiv:1709.02425.

Yuichi Asahiro, Masafumi Yamashita. Minimum algorithm sizes for the gathering and related problems of autonomous mobile robots. *Theoretical Computer Science 1051:115397 (SSS 2023)*, 2025. doi:10.1016/j.tcs.2025.115397, arXiv:2304.02212.

John Augustine, William K. Moses Jr. Dispersion of Mobile Robots: A Study of Memory-Time Trade-offs. *ICDCN 2018*, 2018. doi:10.1145/3154273.3154293, arXiv:1707.05629.

Ricardo A. Baeza-Yates, Joseph C. Culberson, Gregory J. E. Rawlins. Searching in the Plane. *Information and Computation 106(2):234–252*, 1993. doi:10.1006/inco.1993.1054.

Iman Bagheri, Lata Narayanan, Jaroslav Opatrny. Evacuation of Equilateral Triangles by Mobile Agents of Limited Communication Range. *ALGOSENSORS 2019, LNCS, pp. 3–22*, 2019. doi:10.1007/978-3-030-34405-4_1, arXiv:1908.07315.

B. Balamohan, S. Dobrev, P. Flocchini, N. Santoro. Exploring an unknown dangerous graph with a constant number of tokens. *Theoretical Computer Science 610:169–181 (SIROCCO 2012)*, 2016. doi:10.1016/j.tcs.2014.07.013.

Evangelos Bampas, Leszek Gąsieniec, Nicolas Hanusse, David Ilcinkas, Ralf Klasing, Adrian Kosowski. Euler Tour Lock-In Problem in the Rotor-Router Model. *DISC 2009, LNCS, pp. 423–435*, 2009. doi:10.1007/978-3-642-04355-0_44, https://hal.science/hal-00402749.

Evangelos Bampas, Jurek Czyzowicz, Leszek Gąsieniec, David Ilcinkas, Ralf Klasing, Tomasz Kociumaka, Dominik Pająk. Linear Search by a Pair of Distinct-Speed Robots. *Algorithmica 81(1):317–342 (SIROCCO 2016)*, 2019. doi:10.1007/s00453-018-0447-0.

Alberto Bandettini, Fabio Luporini, Giovanni Viglietta. A Survey on Open Problems for Mobile Robots. *arXiv preprint*, 2011. arXiv:1111.2259, https://arxiv.org/abs/1111.2259. Preprint, not peer reviewed.

Rik Banerjee, Manish Kumar, Anisur Rahaman Molla. Optimizing Robot Dispersion on Unoriented Grids: With and Without Fault Tolerance. *ALGOWIN 2024 (LNCS 15026)*, 2024. doi:10.1007/978-3-031-74580-5_3, arXiv:2405.02002.

Rik Banerjee, Manish Kumar, Anisur Rahaman Molla. Optimal Fault-Tolerant Dispersion on Oriented Grids. *ICDCN 2025*, 2025. doi:10.1145/3700838.3700842, arXiv:2410.17813.

Lali Barrière, Paola Flocchini, Pierre Fraigniaud, Nicola Santoro. Capture of an intruder by mobile agents. *SPAA 2002, pp. 200–209*, 2002. doi:10.1145/564870.564906.

Rida A. Bazzi, Joseph L. Briones. Stationary and Deterministic Leader Election in Self-organizing Particle Systems. *SSS 2019, LNCS, pp. 22–37*, 2019. doi:10.1007/978-3-030-34992-9_3.

Anatole Beck. On the linear search problem. *Israel Journal of Mathematics 2(4):221–228*, 1964. doi:10.1007/BF02759737.

Pourandokht Behrouz, Orestis Konstantinidis, Nikos Leonardos, Aris Pagourtzis, Ioannis Papaioannou, Marianna Spyrakou. Byzantine fault-tolerant protocols for (n,f)-evacuation from a circle. *Theoretical Computer Science 1038:115185*, 2025. doi:10.1016/j.tcs.2025.115185.

Richard Bellman. An Optimal Search. *SIAM Review 5(3):274*, 1963. doi:10.1137/1005070.

Michael A. Bender, Antonio Fernández, Dana Ron, Amit Sahai, Salil Vadhan. The power of a pebble: exploring and mapping directed graphs. *Information and Computation 176(1):1–21 (STOC 1998)*, 2002. doi:10.1006/inco.2001.3081.

Subhash Bhagat, Bibhuti Das, Abhinav Chakraborty, Krishnendu Mukhopadhyaya. k-Circle Formation and k-epf by Asynchronous Robots. *Algorithms 14(2):62*, 2021. doi:10.3390/a14020062.

Subhash Bhagat, Andrzej Pelc. How to Meet at a Node of Any Connected Graph. *DISC 2022, LIPIcs 246, 11:1–11:16*, 2022. doi:10.4230/LIPIcs.DISC.2022.11, https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.DISC.2022.11.

Subhash Bhagat, Paulomi Dey, Rajarshi Ray. Collision-Free Linear Time Mutual Visibility for Asynchronous Fat Robots. *ICDCN 2024, pp. 84–93*, 2024. doi:10.1145/3631461.3631544.

Lélia Blin, Pierre Fraigniaud, Nicolas Nisse, Sandrine Vial. Distributed chasing of network intruders. *Theoretical Computer Science 399(1–2):12–37 (SIROCCO 2006)*, 2008. doi:10.1016/j.tcs.2008.02.004.

Manuel Blum, Dexter Kozen. On the power of the compass (or, why mazes are easier to search than graphs). *FOCS 1978, pp. 132–142*, 1978. doi:10.1109/SFCS.1978.30.

Dominik Bojko, Karol Gotfryd, Dariusz R. Kowalski, Dominik Pająk. Tree Exploration in Dual-Memory Model. *MFCS 2022 (LIPIcs 241), 22:1–22:16*, 2022. doi:10.4230/LIPIcs.MFCS.2022.22, arXiv:2112.13449.

Piotr Borowiecki, Shantanu Das, Dariusz Dereniowski, Łukasz Kuszner. Discrete evacuation in graphs with multiple exits. *Theoretical Computer Science 1035, pp. 115141*, 2025. doi:10.1016/j.tcs.2025.115141.

Kaustav Bose, Ranendu Adhikary, Manash Kumar Kundu, Buddhadeb Sau. Arbitrary Pattern Formation by Opaque Fat Robots with Lights. *CALDAM 2020, LNCS 12016, pp. 347–359*, 2020. doi:10.1007/978-3-030-39219-2_28, arXiv:1910.02706.

Kaustav Bose, Manash Kumar Kundu, Ranendu Adhikary, Buddhadeb Sau. Arbitrary pattern formation by asynchronous opaque robots with lights. *Theoretical Computer Science 849:138–158 (SIROCCO 2019)*, 2021. doi:10.1016/j.tcs.2020.10.015, arXiv:1902.04950.

Sébastien Bouchard, Yoann Dieudonné, Bertrand Ducourthial. Byzantine gathering in networks. *Distributed Computing 29(6):435–457 (SIROCCO 2015)*, 2016. doi:10.1007/s00446-016-0276-9.

Sébastien Bouchard, Yoann Dieudonné, Anissa Lamani. Byzantine gathering in polynomial time. *Distributed Computing 35(3):235–263 (ICALP 2018)*, 2022. doi:10.1007/s00446-022-00419-9, arXiv:1801.07656.

Sébastien Bouchard, Yoann Dieudonné, Arnaud Labourel, Andrzej Pelc. Almost-optimal deterministic treasure hunt in unweighted graphs. *ACM Transactions on Algorithms 19(3) (ICALP 2021)*, 2023. doi:10.1145/3588437, arXiv:2010.14916.

Sébastien Bouchard, Yoann Dieudonné, Andrzej Pelc. Want to Gather? No Need to Chatter!. *SIAM Journal on Computing 52(2):358–411 (PODC 2020)*, 2023. doi:10.1137/20M1362899, arXiv:1908.11402.

Zohir Bouzid, Anissa Lamani. Robot Networks with Homonyms: The Case of Patterns Formation. *SSS 2011, pp. 92–107*, 2011. doi:10.1007/978-3-642-24550-3_9, arXiv:1105.5817.

Zohir Bouzid, Shantanu Das, Sebastien Tixeuil. Gathering of Mobile Robots Tolerating Multiple Crash Faults. *ICDCS 2013, pp. 337–346*, 2013. doi:10.1109/ICDCS.2013.27, arXiv:1207.0226.

Quentin Bramas, Sébastien Tixeuil. Wait-Free Gathering Without Chirality. *SIROCCO 2015, pp. 313–327*, 2015. doi:10.1007/978-3-319-25258-2_22.

Quentin Bramas, Sébastien Tixeuil. Probabilistic asynchronous arbitrary pattern formation. *SSS 2016, pp. 88–93 (brief announcement at PODC 2016)*, 2016. doi:10.1007/978-3-319-49259-9_7, arXiv:1508.03714.

Quentin Bramas, Sébastien Tixeuil. The Random Bit Complexity of Mobile Robots Scattering. *International Journal of Foundations of Computer Science 28(2):111–133 (ADHOC-NOW 2015)*, 2017. doi:10.1142/S0129054117500083.

Quentin Bramas, Sébastien Tixeuil. Arbitrary pattern formation with four robots. *SSS 2018, pp. 333–348*, 2018. doi:10.1007/978-3-030-03232-6_22.

Quentin Bramas, Pascal Lafourcade, Stéphane Devismes. Finding water on Poleless using melomaniac myopic chameleon robots. *FUN 2021 (LIPIcs 157), 6:1–6:19*, 2021. doi:10.4230/LIPIcs.FUN.2021.6.

Quentin Bramas, Stéphane Devismes, Pascal Lafourcade. Infinite grid exploration by disoriented robots. *NETYS 2020, LNCS, pp. 129–145 (brief announcement at SIROCCO 2019, pp. 340–344)*, 2021. doi:10.1007/978-3-030-67087-0_9, arXiv:1905.09271.

Quentin Bramas, Stéphane Devismes, Anaïs Durand, Pascal Lafourcade, Anissa Lamani. Beedroids: how luminous autonomous swarms of UAVs can save the world?. *FUN 2022 (LIPIcs 226), 7:1–7:21*, 2022. doi:10.4230/LIPIcs.FUN.2022.7.

Quentin Bramas, Pascal Lafourcade, Stéphane Devismes. Optimal exclusive perpetual grid exploration by luminous myopic opaque robots with common chirality. *Theoretical Computer Science 977:114162 (ICDCN 2021, pp. 76–85)*, 2023. doi:10.1016/j.tcs.2023.114162, https://hal.science/hal-04206069.

Quentin Bramas, Anissa Lamani, Sébastien Tixeuil. Stand up indulgent gathering. *Theoretical Computer Science 939:63–77*, 2023. doi:10.1016/j.tcs.2022.10.015, arXiv:2302.03466.

Quentin Bramas, Hirotsugu Kakugawa, Sayaka Kamei, Anissa Lamani, Fukuhito Ooshita, Masahiro Shibata, Sébastien Tixeuil. A visibility vs. memory trade-off for stand-up indulgent gathering on lines. *SIROCCO 2025, LNCS, pp. 159–175*, 2025. doi:10.1007/978-3-031-91736-3_10.

Quentin Bramas, Stéphane Devismes, Anaïs Durand, Pascal Lafourcade, Anissa Lamani. Optimal asynchronous perpetual finite grid exploration. *Theoretical Computer Science 1082:116072 (SSS 2024, LNCS, pp. 89–105, as "Optimal Asynchronous Perpetual Grid Exploration")*, 2026. doi:10.1016/j.tcs.2026.116072, https://uca.hal.science/hal-04620468.

Quentin Bramas, Stéphane Devismes, Anaïs Durand, Pascal Lafourcade, Anissa Lamani. 77 shades of grey. *Preprint, HAL hal-05504522*, 2026. https://uca.hal.science/hal-05504522. Preprint, not peer reviewed.

Quentin Bramas, Hirotsugu Kakugawa, Sayaka Kamei, Anissa Lamani, Fukuhito Ooshita, Masahiro Shibata, Sébastien Tixeuil. Stand-up indulgent gathering on lines for myopic luminous robots. *The Computer Journal 69(7):1122–1132*, 2026. doi:10.1093/comjnl/bxag015, arXiv:2312.12698.

Sebastian Brandt, Felix Laufenberg, Yuezhou Lv, David Stolz, Roger Wattenhofer. Collaboration Without Communication: Evacuating Two Robots from a Disk. *CIAC 2017, pp. 104–115*, 2017. doi:10.1007/978-3-319-57586-5_10.

Claude (Anthropic) was used in cataloguing the problems.

Sebastian Brandt, Klaus-Tycho Foerster, Benjamin Richner, Roger Wattenhofer. Wireless evacuation on m rays with k searchers. *Theoretical Computer Science 811:56–69*, 2020. doi:10.1016/j.tcs.2018.10.032.

Joseph L. Briones, Tishya Chhabra, Joshua J. Daymude, Andréa W. Richa. Invited Paper: Asynchronous Deterministic Leader Election in Three-Dimensional Programmable Matter. *ICDCN 2023, pp. 38–47*, 2023. doi:10.1145/3571306.3571389, arXiv:2205.15412.

Kevin Buchin, Paola Flocchini, Irina Kostitsyna, Tom Peters, Nicola Santoro, Koichi Wada. On the Computational Power of Energy-Constrained Mobile Robots: Algorithms and Cross-Model Analysis. *SIROCCO 2022, LNCS 13298, pp. 42–61*, 2022. doi:10.1007/978-3-031-09993-9_3, arXiv:2203.06546.

Lothar Budach. Automata and labyrinths. *Mathematische Nachrichten 86:195–282*, 1978. doi:10.1002/mana.19780860120.

Hans-Joachim Böckenhauer, Fabian Frei, Walter Unger, David Wehner. Zero-Memory Graph Exploration with Unknown Inports. *SIROCCO 2023, LNCS 13892, pp. 246–261*, 2023. doi:10.1007/978-3-031-32733-9_11, arXiv:2301.13860.

Sarah Cannon, Joshua J. Daymude, Dana Randall, Andréa W. Richa. A Markov Chain Algorithm for Compression in Self-Organizing Particle Systems. *PODC 2016, pp. 279–288*, 2016. doi:10.1145/2933057.2933107, arXiv:1603.07991.

Sarah Cannon, Joshua J. Daymude, Cem Gökmen, Dana Randall, Andréa W. Richa. A Local Stochastic Algorithm for Separation in Heterogeneous Self-Organizing Particle Systems. *APPROX/RANDOM 2019, LIPIcs 145, 54:1–54:22*, 2019. doi:10.4230/LIPIcs.APPROX-RANDOM.2019.54, arXiv:1805.04599.

Arnaud Casteigts, Paola Flocchini, Walter Quattrociocchi, Nicola Santoro. Time-varying graphs and dynamic networks. *International Journal of Parallel, Emergent and Distributed Systems 27(5):387–408 (ADHOC-NOW 2011)*, 2012. doi:10.1080/17445760.2012.668546, arXiv:1012.0009.

Jannik Castenow, Jonas Harbig, Daniel Jung, Peter Kling, Till Knollmann, Friedhelm Meyer auf der Heide. A unifying approach to efficient (near)-gathering of disoriented robots with limited visibility. *OPODIS 2022 (LIPIcs 253), 15:1–15:25*, 2022. doi:10.4230/LIPIcs.OPODIS.2022.15, arXiv:2206.07567.

Jérémie Chalopin, Shantanu Das, Adrian Kosowski. Constructing a Map of an Anonymous Graph: Applications of Universal Sequences. *OPODIS 2010, pp. 119–134*, 2010. doi:10.1007/978-3-642-17653-1_10.

Jérémie Chalopin, Shantanu Das, Maria Kokkou. Deterministic self-stabilising leader election for programmable matter with constant memory. *Distributed Computing*, 2025. doi:10.1007/s00446-025-00493-9, arXiv:2408.08775.

Jérémie Chalopin, Shantanu Das, Maria Kokkou. Silent Self-stabilising Leader Election in Programmable Matter Systems with Holes. *SIROCCO 2026, LNCS, pp. 151–171*, 2026. doi:10.1007/978-3-032-26465-7_9, arXiv:2507.20201.

Jérémie Chalopin, Shantanu Das, Maria Kokkou. Deterministic leader election for stationary programmable matter with common direction. *Theoretical Computer Science 1080:116045*, 2026. doi:10.1016/j.tcs.2026.116045, arXiv:2402.10582.

Prabhat Kumar Chand, Anisur Rahaman Molla, Sumathi Sivasubramaniam. Run for Cover: Dominating Set via Mobile Agents. *ALGOSENSORS 2023, pp. 133–150*, 2023. doi:10.1007/978-3-031-48882-5_10, arXiv:2309.02200.

Prabhat Kumar Chand, Manish Kumar, Anisur Rahaman Molla, Sumathi Sivasubramaniam. Fault-Tolerant Dispersion of Mobile Robots. *CALDAM 2023 (LNCS 13779)*, 2023. Journal version: *Discrete Applied Mathematics 377:299–313, 2025 (doi:10.1016/j.dam.2025.06.068)*. doi:10.1007/978-3-031-25211-2_3, arXiv:2211.12451.

Prabhat Kumar Chand, Apurba Das, Anisur Rahaman Molla. Brief Announcement: Distributed Butterfly Analysis using Mobile Agents. *SPAA 2025 brief announcement, pp. 623–627*, 2025. doi:10.1145/3694906.3743334, arXiv:2506.17721.

Prabhat Kumar Chand, Manish Kumar, Anisur Rahaman Molla. Computing Tree Structures in Anonymous Graphs via Mobile Agents. *SSS 2025, pp. 111–127*, 2025. doi:10.1007/978-3-032-11127-2_11, arXiv:2506.19365.

Prabhat Kumar Chand, Anisur Rahaman Molla. Improved Linear-Time Construction of Minimal Dominating Set via Mobile Agents. *ICDCIT 2026, pp. 35–51*, 2026. doi:10.1007/978-3-032-16632-6_3, arXiv:2511.19880.

Prabhat Kumar Chand, Apurba Das, Anisur Rahaman Molla. Agent-Based Triangle Counting: Unlocking Truss Decomposition, Triangle Centrality, and Local Clustering Coefficient. *IEEE Transactions on Networking (2026)*, 2026. doi:10.1109/TON.2026.3734259, arXiv:2402.03653.

Sruti Gan Chaudhuri, Krishnendu Mukhopadhyaya. Gathering Asynchronous Transparent Fat Robots. *ICDCIT 2010, LNCS 5966, pp. 170–175*, 2010. doi:10.1007/978-3-642-11659-9_17.

Sruti Gan Chaudhuri, Krishnendu Mukhopadhyaya. Leader election and gathering for asynchronous fat robots without common chirality. *Journal of Discrete Algorithms 33:171–192*, 2015. doi:10.1016/j.jda.2015.04.001, arXiv:1208.4484.

Marek Chrobak, Leszek Gąsieniec, Thomas Gorry, Russell Martin. Group Search on the Line. *SOFSEM 2015, LNCS, pp. 164–176*, 2015. doi:10.1007/978-3-662-46078-8_14.

Huda Chuangpishit, Konstantinos Georgiou, Preeti Sharma. Average Case - Worst Case Tradeoffs for Evacuating 2 Robots from the Disk in the Face-to-Face Model. *ALGOSENSORS 2018, LNCS*, 2019. doi:10.1007/978-3-030-14094-6_5, arXiv:1807.08640.

Huda Chuangpishit, Konstantinos Georgiou, Preeti Sharma. A Multi-Objective Optimization Problem on Evacuating 2 Robots from the Disk in the Face-to-Face Model; Trade-Offs between Worst-Case and Average-Case Analysis. *Information 11, 506*, 2020. doi:10.3390/info11110506.

Huda Chuangpishit, Saeed Mehrabi, Lata Narayanan, Jaroslav Opatrny. Evacuating equilateral triangles and squares in the face-to-face model. *Computational Geometry 89:101624*, 2020. doi:10.1016/j.comgeo.2020.101624, arXiv:1812.10162.

Serafino Cicerone, Gabriele Di Stefano, Alfredo Navarra. Asynchronous arbitrary pattern formation: the effects of a rigorous approach. *Distributed Computing 32(2):91–132*, 2019. doi:10.1007/s00446-018-0325-7, arXiv:1706.02474.

Mark Cieliebak, Paola Flocchini, Giuseppe Prencipe, Nicola Santoro. Distributed Computing by Mobile Robots: Gathering. *SIAM Journal on Computing 41(4):829–879 (ICALP 2003)*, 2012. doi:10.1137/100796534.

Julien Clement, Xavier Défago, Maria Gradinariu Potop-Butucaru, Taisuke Izumi, Stephane Messika. The cost of probabilistic agreement in oblivious robot networks. *Information Processing Letters 110(11):431–438*, 2010. doi:10.1016/j.ipl.2010.04.006, arXiv:0807.1753.

Reuven Cohen, David Peleg. Convergence properties of the gravitational algorithm in asynchronous robot systems. *SIAM Journal on Computing 34(6):1516–1528 (ESA 2004)*, 2005. doi:10.1137/S0097539704446475.

Reuven Cohen, Pierre Fraigniaud, David Ilcinkas, Amos Korman, David Peleg. Label-guided graph exploration by a finite automaton. *ACM Transactions on Algorithms 4(4) (ICALP 2005)*, 2008. doi:10.1145/1383369.1383373.

Reuven Cohen, David Peleg. Local spreading algorithms for autonomous robot systems. *Theoretical Computer Science 399(1–2):71–82 (SIROCCO 2006)*, 2008. doi:10.1016/j.tcs.2008.02.007.

Jared Coleman, Evangelos Kranakis, Danny Krizanc, Oscar Morales-Ponce. Delivery to Safety with Two Cooperating Robots. *Lecture Notes in Computer Science SOFSEM 2023: Theory and Practice of Computer Science, 359-371*, 2023. doi:10.1007/978-3-031-23101-8_24, arXiv:2210.04080.

Jared Coleman, Evangelos Kranakis, Danny Krizanc, Oscar Morales-Ponce. Search and Rescue on the Plane. *arXiv preprint*, 2026. arXiv:2608.12039. Preprint, not peer reviewed.

James Conley, Konstantinos Georgiou. Multi-agent Disk Inspection. *Lecture Notes in Computer Science Structural Information and Communication Complexity, 262-280*, 2025. doi:10.1007/978-3-031-91736-3_16, arXiv:2411.15391.

Andreas Cord-Landwehr, Bastian Degener, Matthias Fischer, Martina Hüllmann, Barbara Kempkes, Alexander Klaas, Peter Kling, Sven Kurras, Marcus Märtens, Friedhelm Meyer auf der Heide, Christoph Raupach, Kamil Swierkot, Daniel Warner, Christoph Weddemann, Daniel Wonisch. A new approach for analyzing convergence algorithms for mobile robots. *ICALP 2011, pp. 650–661*, 2011. doi:10.1007/978-3-642-22012-8_52.

Romain Cosson, Laurent Massoulié, Laurent Viennot. Efficient collaborative tree exploration with breadth-first depth-next. *DISC 2023 (LIPIcs 281), paper 14*, 2023. doi:10.4230/LIPIcs.DISC.2023.14, arXiv:2301.13307.

Romain Cosson. Breaking the k/log k barrier in collective tree exploration via tree-mining. *SODA 2024, pp. 4264–4282*, 2024. doi:10.1137/1.9781611977912.148, arXiv:2309.07011.

Romain Cosson, Laurent Massoulié. Collective tree exploration via potential function method. *ITCS 2024 (LIPIcs), paper 35*, 2024. doi:10.4230/LIPIcs.ITCS.2024.35, arXiv:2311.01354.

Romain Cosson, Laurent Massoulié. Asynchronous collective tree exploration: a distributed algorithm, and a new lower bound. *arXiv preprint*, 2025. arXiv:2507.15658. Preprint, not peer reviewed.

Romain Cosson, Laurent Massoulié. Polylogarithmic collective tree exploration. *arXiv preprint (September 2026)*, 2026. arXiv:2609.26789. Preprint, not peer reviewed.

Jurek Czyzowicz, Dariusz Kowalski, Euripides Markou, Andrzej Pelc. Complexity of Searching for a Black Hole. *Fundamenta Informaticae 71(2–3):229–242*, 2006. doi:10.3233/FUN-2006-712-305.

Claude (Anthropic) was used in cataloguing the problems.

Jurek Czyzowicz, Leszek Gąsieniec, Andrzej Pelc. Gathering few fat mobile robots in the plane. *Theoretical Computer Science 410(6–7):481–499 (OPODIS 2006)*, 2009. doi:10.1016/j.tcs.2008.10.005.

Jurek Czyzowicz, Leszek Gąsieniec, Adrian Kosowski, Evangelos Kranakis. Boundary Patrolling by Mobile Agents with Distinct Maximal Speeds. *ESA 2011, pp. 701–712*, 2011. doi:10.1007/978-3-642-23719-5_59.

Jurek Czyzowicz, Andrzej Pelc, Arnaud Labourel. How to meet asynchronously (almost) everywhere. *ACM Transactions on Algorithms 8(4), article 37 (SODA 2010)*, 2012. doi:10.1145/2344422.2344427.

Jurek Czyzowicz, Adrian Kosowski, Andrzej Pelc. How to meet when you forget: log-space rendezvous in arbitrary graphs. *Distributed Computing 25(2):165–178 (PODC 2010)*, 2012. doi:10.1007/s00446-011-0141-9.

Jurek Czyzowicz, Stefan Dobrev, Leszek Gąsieniec, David Ilcinkas, Jesper Jansson, Ralf Klasing, Ioannis Lignos, Russell Martin, Kunihiko Sadakane, Wing-Kin Sung. More efficient periodic traversal in anonymous undirected graphs. *Theoretical Computer Science 444:60–76 (SIROCCO 2009)*, 2012. doi:10.1016/j.tcs.2012.01.035, arXiv:0905.1737.

Jurek Czyzowicz, Leszek Gąsieniec, Thomas Gorry, Evangelos Kranakis, Russell Martin, Dominik Pajak. Evacuating Robots via Unknown Exit in a Disk. *DISC 2014, pp. 122–136*, 2014. doi:10.1007/978-3-662-45174-8_9.

J. Czyzowicz, E. Kranakis, D. Krizanc, L. Narayanan, J. Opatrny, S. Shende. Wireless Autonomous Robot Evacuation from Equilateral Triangles and Squares. *ADHOC-NOW 2015, LNCS, pp. 181–194*, 2015. doi:10.1007/978-3-319-19662-6_13.

Jurek Czyzowicz, Stefan Dobrev, Konstantinos Georgiou, Evangelos Kranakis, Fraser MacQuarrie. Evacuating two robots from multiple unknown exits in a circle. *ICDCN 2016*, 2016. doi:10.1145/2833312.2833318.

Jurek Czyzowicz, Evangelos Kranakis, Danny Krizanc, Lata Narayanan, Jaroslav Opatrny. Search on a Line with Faulty Robots. *PODC 2016, pp. 405–414; journal: Distributed Computing 32(6):493–504 (2019)*, 2016. doi:10.1145/2933057.2933102.

Jurek Czyzowicz, Konstantinos Georgiou, Maxime Godon, Evangelos Kranakis, Danny Krizanc, Wojciech Rytter, Michał Włodarczyk. Evacuation from a Disc in the Presence of a Faulty Robot. *SIROCCO 2017, pp. 158–173*, 2017. doi:10.1007/978-3-319-72050-0_10.

Jurek Czyzowicz, Leszek Gasieniec, Adrian Kosowski, Evangelos Kranakis, Danny Krizanc, Najmeh Taleb. When Patrolmen Become Corrupted: Monitoring a Graph Using Faulty Mobile Robots. *Algorithmica 79(3):925–940 (ISAAC 2015)*, 2017. doi:10.1007/s00453-016-0233-9.

Jurek Czyzowicz, Konstantinos Georgiou, Ryan Killick, Evangelos Kranakis, Danny Krizanc, Lata Narayanan, Jaroslav Opatrny, Sunil Shende. God Save the Queen. *FUN 2018, LIPIcs 100, 16:1–16:20*, 2018. doi:10.4230/LIPIcs.FUN.2018.16, arXiv:1804.06011.

Jurek Czyzowicz, Konstantinos Georgiou, Ryan Killick, Evangelos Kranakis, Danny Krizanc, Lata Narayanan, Jaroslav Opatrny, Sunil Shende. Priority Evacuation from a Disk Using Mobile Robots. *SIROCCO 2018, pp. 392–407*, 2018. doi:10.1007/978-3-030-01325-7_32, arXiv:1805.03568.

Jurek Czyzowicz, Konstantinos Georgiou, Ryan Killick, Evangelos Kranakis, Danny Krizanc, Manuel Lafond, Lata Narayanan, Jaroslav Opatrny, Sunil Shende. Energy Consumption of Group Search on a Line. *ICALP 2019, LIPIcs 132, 137:1–137:15*, 2019. doi:10.4230/LIPIcs.ICALP.2019.137, arXiv:1904.09714.

Jurek Czyzowicz, Kostantinos Georgiou, Evangelos Kranakis. Group Search and Evacuation. *Lecture Notes in Computer Science Distributed Computing by Mobile Entities, 335-370*, 2019. doi:10.1007/978-3-030-11072-7_14.

Jurek Czyzowicz, Maxime Godon, Evangelos Kranakis, Arnaud Labourel. Group search of the plane with faulty robots. *Theoretical Computer Science 792, pp. 69-84*, 2019. doi:10.1016/j.tcs.2018.09.029.

Jurek Czyzowicz, Konstantinos Georgiou, Evangelos Kranakis, Lata Narayanan, Jaroslav Opatrny, Birgit Vogtenhuber. Evacuating Robots from a Disk Using Face-to-Face Communication. *Discrete Mathematics & Theoretical Computer Science 22(4) (CIAC 2015)*, 2020. doi:10.23638/DMTCS-22-4-4, arXiv:1501.04985.

Jurek Czyzowicz, Konstantinos Georgiou, Evangelos Kranakis, Danny Krizanc, Lata Narayanan, Jaroslav Opatrny, Sunil Shende. Search on a line by Byzantine robots. *International Journal of Foundations of Computer Science 32 (2021) 369–387*, 2021. arXiv:1611.08209.

Jurek Czyzowicz, Ryan Killick, Evangelos Kranakis, Danny Krizanc, Lata Narayanan, Jaroslav Opatrny, Denis Pankratov, Sunil Shende. Group Evacuation on a Line by Agents with Different Communication Abilities. *ISAAC 2021, LIPIcs 212, 57:1–57:24*, 2021. doi:10.4230/LIPIcs.ISAAC.2021.57, arXiv:2109.12676.

Claude (Anthropic) was used in cataloguing the problems.

Jurek Czyzowicz, Ryan Killick, Evangelos Kranakis, Grzegorz Stachowiak. Search and evacuation with a near majority of faulty agents. *SIAM Conference on Applied and Computational Discrete Algorithms (ACDA 2021)*, 2021. doi:10.1137/1.9781611976830.20, arXiv:2605.08355.

Jurek Czyzowicz, Konstantinos Georgiou, Ryan Killick, Evangelos Kranakis, Danny Krizanc, Manuel Lafond, Lata Narayanan, Jaroslav Opatrny, Sunil Shende. Time-energy tradeoffs for evacuation by two robots in the wireless model. *Theoretical Computer Science 852, 61-72*, 2021. doi:10.1016/j.tcs.2020.11.014, arXiv:1905.06783.

Gianlorenzo D'Angelo, Mattia D'Emidio, Shantanu Das, Alfredo Navarra, Giuseppe Prencipe. Asynchronous Silent Programmable Matter Achieves Leader Election and Compaction. *IEEE Access 8:207619–207634 (AAMAS 2020)*, 2020. doi:10.1109/ACCESS.2020.3038174.

Omar Darwich, Ahmet-Sefa Ulucan, Quentin Bramas, Anissa Lamani, Anaïs Durand, Pascal Lafourcade. Perpetual torus exploration by myopic luminous robots. *Theoretical Computer Science 976:114143 (SSS 2022, LNCS, pp. 164–177)*, 2023. doi:10.1016/j.tcs.2023.114143, https://uca.hal.science/hal-04323448.

Shantanu Das, Paola Flocchini, Shay Kutten, Amiya Nayak, Nicola Santoro. Map construction of unknown graphs by multiple agents. *Theoretical Computer Science 385(1–3):34–48*, 2007. doi:10.1016/j.tcs.2007.05.011.

Shantanu Das, Paola Flocchini, Nicola Santoro, Masafumi Yamashita. Forming sequences of geometric patterns with oblivious mobile robots. *Distributed Computing 28(2):131–145 (PODC 2010)*, 2015. doi:10.1007/s00446-014-0220-9.

Shantanu Das, Paola Flocchini, Giuseppe Prencipe, Nicola Santoro, Masafumi Yamashita. Autonomous mobile robots with lights. *Theoretical Computer Science 609:171–184 (ICDCS 2012)*, 2016. doi:10.1016/j.tcs.2015.09.018.

Shantanu Das. Graph Explorations with Mobile Agents. *Distributed Computing by Mobile Entities, LNCS 11340, pp. 403–422*, 2019. doi:10.1007/978-3-030-11072-7_16.

Shantanu Das, Paola Flocchini, Giuseppe Prencipe, Nicola Santoro. Forming Sequences of Patterns With Luminous Robots. *IEEE Access 8:90577–90597*, 2020. doi:10.1109/ACCESS.2020.2994052.

Archak Das, Avisek Sharma, Buddhadeb Sau. Power of Finite Memory and Finite Communication Robots under Asynchronous Scheduler. *arXiv preprint*, 2021. arXiv:2104.12202. Preprint, not peer reviewed.

Archak Das, Kaustav Bose, Buddhadeb Sau. Memory Optimal Dispersion by Anonymous Mobile Robots. *CALDAM 2021*, 2021. Journal version: *Discrete Applied Mathematics 340:171–182, 2023 (doi:10.1016/j.dam.2023.07.005)*. doi:10.1007/978-3-030-67899-9_34, arXiv:2008.00701.

Archak Das, Kaustav Bose, Buddhadeb Sau. Exploring a dynamic ring without landmark. *Theoretical Computer Science 929:191–205 (SSS 2021)*, 2022. doi:10.1016/j.tcs.2022.07.005, arXiv:2107.02769.

Archak Das, Satakshi Ghosh, Avisek Sharma, Pritam Goswami, Buddhadeb Sau. The Computational Landscape of Autonomous Mobile Robots: The Visibility Perspective. *ICDCIT 2024, LNCS, pp. 85–100*, 2024. doi:10.1007/978-3-031-50583-6_6, arXiv:2303.03031.

Raja Das, Avisek Sharma, Buddhadeb Sau. Maximum independent set formation on a finite grid by myopic robots. *Theoretical Computer Science 1031:115077 (Intelligent Computing 2023, LNNS, pp. 1247–1267)*, 2025. doi:10.1016/j.tcs.2025.115077, arXiv:2207.13403.

Bibhuti Das, Barun Gorain, Kaushik Mondal, Krishnendu Mukhopadhyaya, Supantha Pandit. Optimal Dispersion of Silent Robots in a Ring. *SSS 2025 (LNCS)*, 2025. Journal version: *Theoretical Computer Science 1084:116171, 2026 (doi:10.1016/j.tcs.2026.116171)*. doi:10.1007/978-3-032-11127-2_15, arXiv:2408.05491.

Raja Das, Pritam Goswami, Brati Mondal, Buddhadeb Sau. Rectangle enclosed triangular grid exploration with myopic luminous robots. *ICDCN 2025, pp. 249–253*, 2025. doi:10.1145/3700838.3700841.

Ajoy K. Datta, Anissa Lamani, Lawrence L. Larmore, Franck Petit. Ring exploration by oblivious agents with local vision. *ICDCS 2013, pp. 347–356*, 2013. doi:10.1109/ICDCS.2013.55, arXiv:1205.5003.

Joshua J. Daymude, Robert Gmyr, Andréa W. Richa, Christian Scheideler, Thim Strothmann. Improved Leader Election for Self-organizing Programmable Matter. *ALGOSENSORS 2017, pp. 127–140*, 2017. doi:10.1007/978-3-319-72751-6_10, arXiv:1701.03616.

Joshua J. Daymude, Zahra Derakhshandeh, Robert Gmyr, Alexandra Porter, Andréa W. Richa, Christian Scheideler, Thim Strothmann. On the runtime of universal coating for programmable matter. *Natural Computing 17:81–96 (DNA 22, 2016)*, 2018. doi:10.1007/s11047-017-9658-6, arXiv:1606.03642.

Joshua J. Daymude, Robert Gmyr, Kristian Hinnenthal, Irina Kostitsyna, Christian Scheideler, Andréa W. Richa. Convex Hull Formation for Programmable Matter. *ICDCN 2020, 2:1–2:10*, 2020. doi:10.1145/3369740.3372916, arXiv:1805.06149.

Claude (Anthropic) was used in cataloguing the problems.

Joshua J. Daymude, Andréa W. Richa, Jamison W. Weber. Bio-Inspired Energy Distribution for Programmable Matter. *ICDCN 2021, pp. 86–95*, 2021. doi:10.1145/3427796.3427835, arXiv:2007.04377.

Joshua J. Daymude, Andréa W. Richa, Christian Scheideler. The canonical amoebot model: algorithms and concurrency control. *Distributed Computing 36:159–192 (DISC 2021)*, 2023. doi:10.1007/s00446-023-00443-3, arXiv:2105.02420.

Gianluca De Marco, Luisa Gargano, Evangelos Kranakis, Danny Krizanc, Andrzej Pelc, Ugo Vaccaro. Asynchronous deterministic rendezvous in graphs. *Theoretical Computer Science 355(3):315–326 (MFCS 2005)*, 2006. doi:10.1016/j.tcs.2005.12.016.

Bastian Degener, Barbara Kempkes, Tobias Langner, Friedhelm Meyer auf der Heide, Peter Pietrzyk, Roger Wattenhofer. A tight runtime bound for synchronous gathering of autonomous robots with limited visibility. *SPAA 2011, pp. 139–148*, 2011. doi:10.1145/1989493.1989515.

Carole Delporte-Gallet, Hugues Fauconnier, Rachid Guerraoui, Anne-Marie Kermarrec, Eric Ruppert, Hung Tran-The. Byzantine agreement with homonyms. *Distributed Computing 26(5–6):321–340 (PODC 2011)*, 2013. doi:10.1007/s00446-013-0190-3.

Xiaotie Deng, Christos H. Papadimitriou. Exploring an unknown graph. *Journal of Graph Theory 32(3):265–297 (FOCS 1990)*, 1999. doi:10.1002/(SICI)1097-0118(199911)32:3<265::AID-JGT6>3.0.CO;2-8.

Zahra Derakhshandeh, Shlomi Dolev, Robert Gmyr, Andréa W. Richa, Christian Scheideler, Thim Strothmann. Amoebot - a new model for programmable matter. *SPAA 2014 brief announcement, pp. 220–222*, 2014. doi:10.1145/2612669.2612712.

Zahra Derakhshandeh, Robert Gmyr, Thim Strothmann, Rida Bazzi, Andréa W. Richa, Christian Scheideler. Leader Election and Shape Formation with Self-organizing Programmable Matter. *DNA 21 (2015), pp. 117–132*, 2015. doi:10.1007/978-3-319-21999-8_8, arXiv:1503.07991.

Zahra Derakhshandeh, Robert Gmyr, Andrea W. Richa, Christian Scheideler, Thim Strothmann. Universal Shape Formation for Programmable Matter. *SPAA 2016, pp. 289–299*, 2016. doi:10.1145/2935764.2935784.

Zahra Derakhshandeh, Robert Gmyr, Andréa W. Richa, Christian Scheideler, Thim Strothmann. Universal coating for programmable matter. *Theoretical Computer Science 671:56–68*, 2017. doi:10.1016/j.tcs.2016.02.039, arXiv:1601.01008.

Dariusz Dereniowski, Yann Disser, Adrian Kosowski, Dominik Pająk, Przemysław Uznański. Fast collaborative graph exploration. *Information and Computation 243:37–49 (ICALP 2013)*, 2015. doi:10.1016/j.ic.2014.12.005, https://inria.hal.science/hal-00802308.

Anders Dessmark, Pierre Fraigniaud, Dariusz R. Kowalski, Andrzej Pelc. Deterministic rendezvous in graphs. *Algorithmica 46(1):69–96 (ESA 2003, without Kowalski)*, 2006. doi:10.1007/s00453-006-0074-2, https://pelc.uqo.ca/pelc/2006.html.

Henri Devillez, Béni Egressy, Robin Fritsch, Roger Wattenhofer. Two-Agent Tree Evacuation. *SIROCCO 2021, LNCS, pp. 204–221*, 2021. doi:10.1007/978-3-030-79527-6_12.

Stéphane Devismes, Anissa Lamani, Franck Petit, Pascal Raymond, Sébastien Tixeuil. Optimal Grid Exploration by Asynchronous Oblivious Robots. *SSS 2012, pp. 64–76; journal: The Computer Journal 64(1):132–154*, 2012. doi:10.1007/978-3-642-33536-5_7, arXiv:1105.2461.

Stéphane Devismes, Yoann Dieudonné, Arnaud Labourel. Can Like Attract Like? A Study of Homonymous Gathering in Networks. *STOC 2026, pp. 933–942*, 2026. doi:10.1145/3798129.3800807, arXiv:2510.25451.

Giuseppe Antonio Di Luna, Paola Flocchini, Sruti Gan Chaudhuri, Nicola Santoro, Giovanni Viglietta. Robots with Lights: Overcoming Obstructed Visibility Without Colliding. *SSS 2014, LNCS 8756, pp. 150–164*, 2014. doi:10.1007/978-3-319-11764-5_11, arXiv:1405.2430.

Giuseppe A. Di Luna, Paola Flocchini, Sruti Gan Chaudhuri, Federico Poloni, Nicola Santoro, Giovanni Viglietta. Mutual visibility by luminous robots without collisions. *Information and Computation 254:392–418*, 2017. doi:10.1016/j.ic.2016.09.005, arXiv:1503.04347.

Giuseppe A. Di Luna, Stefan Dobrev, Paola Flocchini, Nicola Santoro. Distributed exploration of dynamic rings. *Distributed Computing 33(1):41–67 (ICDCS 2016)*, 2020. doi:10.1007/s00446-018-0339-1, arXiv:1512.05306.

Giuseppe Antonio Di Luna, Paola Flocchini, Nicola Santoro, Giovanni Viglietta, Yukiko Yamauchi. Mobile RAM and Shape Formation by Programmable Particles. *Euro-Par 2020, LNCS, pp. 343–358*, 2020. doi:10.1007/978-3-030-57675-2_22, arXiv:2002.03384.

Claude (Anthropic) was used in cataloguing the problems.

Giuseppe A. Di Luna, Paola Flocchini, Nicola Santoro, Giovanni Viglietta, Yukiko Yamauchi. Shape formation by programmable particles. *Distributed Computing 33:69–101 (OPODIS 2017)*, 2020. doi:10.1007/s00446-019-00350-6, arXiv:1705.03538.

Giuseppe Antonio Di Luna, Paola Flocchini, Debasish Pattanayak, Giuseppe Prencipe, Francesco Piselli, Nicola Santoro. Exploring Dangerous Graphs with Byzantine Companions. *ICDCS 2025, pp. 352–361*, 2025. doi:10.1109/ICDCS63083.2025.00042.

Giuseppe Antonio Di Luna, Paola Flocchini, Giuseppe Prencipe, Nicola Santoro. Locating a black hole in a dynamic ring. *Journal of Parallel and Distributed Computing 196:104998 (ICDCS 2021)*, 2025. doi:10.1016/j.jpdc.2024.104998, arXiv:2005.07453.

Yoann Dieudonné, Ouiddad Labbani-Igbida, Franck Petit. Circle formation of weak mobile robots. *ACM Transactions on Autonomous and Adaptive Systems 3(4), article 16 (SSS 2006)*, 2008. doi:10.1145/1452001.1452006, arXiv:cs/0607060.

Yoann Dieudonné, Franck Petit. Scatter of robots. *Parallel Processing Letters 19(1):175–184*, 2009. doi:10.1142/S0129626409000146, arXiv:cs/0701179.

Yoann Dieudonné, Franck Petit, Vincent Villain. Leader election problem versus pattern formation problem. *DISC 2010, pp. 267–281*, 2010. doi:10.1007/978-3-642-15763-9_26, arXiv:0902.2851.

Yoann Dieudonné, Andrzej Pelc. Deterministic network exploration by a single agent with Byzantine tokens. *Information Processing Letters 112:467–470*, 2012. https://pelc.uqo.ca/pelc/2012.html.

Yoann Dieudonné, Florence Levé, Franck Petit, Vincent Villain. Deterministic geoleader election in disoriented anonymous systems. *Theoretical Computer Science 506:43–54*, 2013. doi:10.1016/j.tcs.2013.07.033, arXiv:1202.4486.

Yoann Dieudonné, Andrzej Pelc, David Peleg. Gathering Despite Mischief. *ACM Transactions on Algorithms 11(1) (SODA 2012)*, 2014. doi:10.1145/2629656.

Yoann Dieudonné, Andrzej Pelc, Vincent Villain. How to meet asynchronously at polynomial cost. *SIAM Journal on Computing 44(3):844–867 (PODC 2013)*, 2015. doi:10.1137/130931990.

Yoann Dieudonné, Andrzej Pelc. Anonymous Meeting in Networks. *Algorithmica 74(2):908–946 (SODA 2013)*, 2016. doi:10.1007/s00453-015-9982-0.

Krzysztof Diks, Pierre Fraigniaud, Evangelos Kranakis, Andrzej Pelc. Tree exploration with little memory. *SODA 2002, pp. 588–597*, 2002. Journal version: *Journal of Algorithms 51(1):38–63, 2004*. https://people.scs.carleton.ca/~canccom/Publications/TreeExploration.pdf.

Yann Disser, Jan Hackfeld, Max Klimm. Tight bounds for undirected graph exploration with pebbles and multiple agents. *Journal of the ACM 66(6) (pebble results first at SODA 2016, pp. 25–39)*, 2019. doi:10.1145/3356883, arXiv:1805.03476.

Yann Disser, Sören Schmitt. Evacuating Two Robots from a Disk: A Second Cut. *SIROCCO 2019, pp. 200–214*, 2019. doi:10.1007/978-3-030-24922-9_14, arXiv:1905.10592.

Yann Disser, Frank Mousset, Andreas Noever, Nemanja Škorić, Angelika Steger. A general lower bound for collaborative tree exploration. *Theoretical Computer Science 811:70–78 (SIROCCO 2017)*, 2020. doi:10.1016/j.tcs.2018.03.006, arXiv:1610.01753.

Stefan Dobrev, Paola Flocchini, Giuseppe Prencipe, Nicola Santoro. Searching for a black hole in arbitrary networks: optimal mobile agents protocols. *Distributed Computing 19(1) (PODC 2002)*, 2006. doi:10.1007/s00446-006-0154-y.

Stefan Dobrev, Paola Flocchini, Giuseppe Prencipe, Nicola Santoro. Mobile Search for a Black Hole in an Anonymous Ring. *Algorithmica 48(1):67–90 (DISC 2001)*, 2007. doi:10.1007/s00453-006-1232-z.

Stefan Dobrev, Paola Flocchini, Rastislav Královič, Nicola Santoro. Exploring an unknown dangerous graph using tokens. *Theoretical Computer Science 472:28–45*, 2013. doi:10.1016/j.tcs.2012.11.022.

Stefan Dobrev, Rastislav Královič, Dana Pardubská. Improved Lower Bounds for Shoreline Search. *Structural Information and Communication Complexity, pp. 80-90*, 2020. doi:10.1007/978-3-030-54921-3_5.

G. Dudek, M. Jenkin, E. Milios, D. Wilkes. Robotic exploration as graph construction. *IEEE Transactions on Robotics and Automation 7(6):859–865*, 1991. doi:10.1109/70.105395.

Fabien Dufoulon, Shay Kutten, William K. Moses Jr. Efficient Deterministic Leader Election for Programmable Matter. *PODC 2021, pp. 103–113*, 2021. doi:10.1145/3465084.3467900, arXiv:2106.01108.

Adrian Dumitrescu, Ichiro Suzuki, Masafumi Yamashita. Motion Planning for Metamorphic Systems: Feasibility, Decidability, and Distributed Reconfiguration. *IEEE Transactions on Robotics and Automation 20(3):409–418*, 2004. doi:10.1109/TRA.2004.824936.

Mirosław Dynia, Jakub Łopuszański, Christian Schindelhauer. Why robots need maps. *SIROCCO 2007, pp. 41–50*, 2007. doi:10.1007/978-3-540-72951-8_5.

Xavier Défago, Akihiko Konagaya. Circle formation for oblivious anonymous mobile robots with no common sense of orientation. *POMC 2002, pp. 97–104*, 2002. doi:10.1145/584490.584509.

Xavier Défago, Maria Gradinariu, Stéphane Messika, Philippe Raipin-Parvédy. Fault-Tolerant and Self-stabilizing Mobile Robots Gathering. *DISC 2006, LNCS, pp. 46–60*, 2006. doi:10.1007/11864219_4.

Xavier Défago, Samia Souissi. Non-uniform circle formation algorithm for oblivious mobile robots with convergence toward uniformity. *Theoretical Computer Science 396(1–3):97–112*, 2008. doi:10.1016/j.tcs.2008.01.050.

Xavier Défago, Adam Heriban, Sébastien Tixeuil, Koichi Wada. Using model checking to formally verify rendezvous algorithms for robots with lights in Euclidean space. *Robotics and Autonomous Systems 163:104378*, 2023. doi:10.1016/j.robot.2023.104378, arXiv:1907.09871.

Asaf Efrima, David Peleg. Distributed Models and Algorithms for Mobile Robot Systems. *SOFSEM 2007, LNCS 4362, pp. 70–87*, 2007. doi:10.1007/978-3-540-69507-3_5.

Asaf Efrima, David Peleg. Distributed algorithms for partitioning a swarm of autonomous mobile robots. *Theoretical Computer Science 410(14):1355–1368 (SIROCCO 2007)*, 2009. doi:10.1016/j.tcs.2008.04.042.

Yuval Emek, Shay Kutten, Ron Lavi, William K. Moses Jr. Deterministic Leader Election in Programmable Matter. *ICALP 2019 (LIPIcs 132), 140*, 2019. doi:10.4230/LIPIcs.ICALP.2019.140, arXiv:1905.00580.

Thomas Erlebach, Michael Hoffmann, Frank Kammer. On temporal graph exploration. *Journal of Computer and System Sciences 119:1–18 (ICALP 2015)*, 2021. doi:10.1016/j.jcss.2021.01.005, arXiv:1504.07976.

Sándor Fekete, Chris Gray, Alexander Kröller. Evacuation of Rectilinear Polygons. *Combinatorial Optimization and Applications, pp. 21-30*, 2010. doi:10.1007/978-3-642-17458-2_3, arXiv:1008.4420.

Caterina Feletti, Carlo Mereghetti, Beatrice Palano. Uniform circle formation for swarms of opaque robots with lights. *SSS 2018, pp. 317–332*, 2018. doi:10.1007/978-3-030-03232-6_21.

Caterina Feletti, Carlo Mereghetti, Beatrice Palano. $O(\log n)$-time uniform circle formation for asynchronous opaque luminous robots. *OPODIS 2023 (LIPIcs 286), 5:1–5:21*, 2023. doi:10.4230/LIPIcs.OPODIS.2023.5.

Caterina Feletti, Carlo Mereghetti, Beatrice Palano. Uniform circle formation for fully, semi-, and asynchronous opaque robots with lights. *Applied Sciences 13(13):7991*, 2023. doi:10.3390/app13137991.

Caterina Feletti, Debasish Pattanayak, Gokarna Sharma. Brief announcement: optimal uniform circle formation by asynchronous luminous robots. *DISC 2024 (LIPIcs 319), 46:1–46:7; full version arXiv 2405.06617; journal version J. Parallel Distrib. Comput. 218:105338 (2026)*, 2024. doi:10.4230/LIPIcs.DISC.2024.46, arXiv:2405.06617.

Caterina Feletti, Lucia Mambretti, Carlo Mereghetti, Beatrice Palano. Computational power of autonomous robots: Transparency vs. opaqueness. *Theoretical Computer Science 1036:115153*, 2025. doi:10.1016/j.tcs.2025.115153, arXiv:2401.16893.

Caterina Feletti, Paola Flocchini, Debasish Pattanayak, Giuseppe Prencipe, Nicola Santoro. Universal Dancing by Luminous Robots Under Sequential Schedulers. *SIROCCO 2026, pp. 292–312*, 2026. doi:10.1007/978-3-032-26465-7_16, arXiv:2508.15484.

Rudolf Fleischer, Gerhard Trippen. Exploring an unknown graph efficiently. *ESA 2005, pp. 11–22*, 2005. doi:10.1007/11561071_4.

Paola Flocchini, Bernard Mans, Nicola Santoro. Sense of direction: Definitions, properties, and classes. *Networks 32(3):165–180*, 1998. doi:10.1002/(SICI)1097-0037(199810)32:3<165::AID-NET1>3.0.CO;2-I.

Paola Flocchini, Giuseppe Prencipe, Nicola Santoro, Peter Widmayer. Hard Tasks for Weak Robots: The Role of Common Knowledge in Pattern Formation by Autonomous Mobile Robots. *ISAAC 1999, LNCS 1741, pp. 93–102*, 1999. doi:10.1007/3-540-46632-0_10.

Paola Flocchini, Giuseppe Prencipe, Nicola Santoro, Peter Widmayer. Gathering of asynchronous robots with limited visibility. *Theoretical Computer Science 337(1–3):147–168 (STACS 2001)*, 2005. doi:10.1016/j.tcs.2005.01.001.

Paola Flocchini, Giuseppe Prencipe, Nicola Santoro, Peter Widmayer. Arbitrary pattern formation by asynchronous, anonymous, oblivious robots. *Theoretical Computer Science 407(1–3):412–447*, 2008. doi:10.1016/j.tcs.2008.07.026.

Claude (Anthropic) was used in cataloguing the problems.

Paola Flocchini, Giuseppe Prencipe, Nicola Santoro. Self-deployment of mobile sensors on a ring. *Theoretical Computer Science 402(1):67–80 (ALGOSENSORS 2006)*, 2008. doi:10.1016/j.tcs.2008.03.006.

Paola Flocchini, Miao Jun Huang, Flaminia L. Luccio. Decontamination of hypercubes by mobile agents. *Networks 52(3):167–178 (IPDPS 2005)*, 2008. doi:10.1002/net.20240.

Paola Flocchini, Giuseppe Prencipe, Nicola Santoro. Distributed Computing by Oblivious Mobile Robots. *Synthesis Lectures on Distributed Computing Theory, Morgan & Claypool*, 2012. doi:10.2200/S00440ED1V01Y201208DCT010.

Paola Flocchini, David Ilcinkas, Nicola Santoro. Ping Pong in Dangerous Graphs: Optimal Black Hole Search with Pebbles. *Algorithmica 62(3–4):1006–1033 (DISC 2008)*, 2012. doi:10.1007/s00453-011-9496-3.

Paola Flocchini, David Ilcinkas, Andrzej Pelc, Nicola Santoro. Computing Without Communicating: Ring Exploration by Asynchronous Oblivious Robots. *Algorithmica 65(3):562–583 (OPODIS 2007)*, 2013. doi:10.1007/s00453-011-9611-5.

Paola Flocchini, Nicola Santoro, Giovanni Viglietta, Masafumi Yamashita. Rendezvous with constant memory. *Theoretical Computer Science 621:57–72 (SIROCCO 2013)*, 2016. doi:10.1016/j.tcs.2016.01.025, arXiv:1306.1956.

Paola Flocchini, Giuseppe Prencipe, Nicola Santoro, Giovanni Viglietta. Distributed computing by mobile robots: uniform circle formation. *Distributed Computing 30(6):413–457 (OPODIS 2014)*, 2017. doi:10.1007/s00446-016-0291-x, arXiv:1407.5917.

Paola Flocchini, Nicola Santoro, Koichi Wada. On Memory, Communication, and Synchronous Schedulers When Moving and Computing. *OPODIS 2019, LIPIcs 153, 25:1–25:17*, 2019. doi:10.4230/LIPIcs.OPODIS.2019.25.

Paola Flocchini, Nicola Santoro, Yuichi Sudo, Koichi Wada. On Asynchrony, Memory, and Communication: Separations and Landscapes. *OPODIS 2023, LIPIcs 286, 28:1–28:23*, 2023. doi:10.4230/LIPIcs.OPODIS.2023.28, arXiv:2311.03328.

Paola Flocchini, Debasish Pattanayak, Francesco Piselli, Nicola Santoro, Yukiko Yamauchi. Asynchronous Separation of Unconscious Colored Robots. *CANDAR Workshops 2024, pp. 183–189; journal: International Journal of Networking and Computing 15(2), 2025*, 2024. doi:10.1109/CANDARW64572.2024.00037.

Paola Flocchini, Alfredo Navarra, Debasish Pattanayak, Francesco Piselli, Nicola Santoro. Universal pattern formation by oblivious robots under sequential schedulers. *Distributed Computing (2026) (SIROCCO 2025)*, 2026. doi:10.1007/s00446-026-00504-3, arXiv:2412.10733.

Pierre Fraigniaud, David Ilcinkas. Digraphs exploration with little memory. *STACS 2004, pp. 246–257*, 2004. doi:10.1007/978-3-540-24749-4_22.

Pierre Fraigniaud, David Ilcinkas, Guy Peer, Andrzej Pelc, David Peleg. Graph exploration by a finite automaton. *Theoretical Computer Science 345(2–3):331–344 (MFCS 2004)*, 2005. https://pelc.uqo.ca/pelc/2005.html.

Pierre Fraigniaud, David Ilcinkas, Sergio Rajsbaum, Sébastien Tixeuil. Space Lower Bounds for Graph Exploration via Reduced Automata. *SIROCCO 2005, LNCS 3499, pp. 140–154*, 2005. doi:10.1007/11429647_13.

Pierre Fraigniaud, Leszek Gąsieniec, Dariusz R. Kowalski, Andrzej Pelc. Collective tree exploration. *Networks 48(3):166–177 (LATIN 2004)*, 2006. doi:10.1002/net.20127.

Pierre Fraigniaud, David Ilcinkas, Andrzej Pelc. Oracle size: a new measure of difficulty for communication tasks. *PODC 2006, pp. 179–187*, 2006. doi:10.1145/1146381.1146410.

Pierre Fraigniaud, David Ilcinkas, Sergio Rajsbaum, Sébastien Tixeuil. The reduced automata technique for graph exploration space lower bounds. *Theoretical Computer Science: Essays in Memory of Shimon Even, LNCS, pp. 1–26 (SIROCCO 2005)*, 2006. doi:10.1007/11685654_1, https://hal.science/hal-00341655v1.

Pierre Fraigniaud, David Ilcinkas, Andrzej Pelc. Tree exploration with advice. *Information and Computation 206(11):1276–1287 (MFCS 2006, as "Tree Exploration with an Oracle")*, 2008. doi:10.1016/j.ic.2008.07.005.

Pierre Fraigniaud, Andrzej Pelc. Delays induce an exponential memory gap for rendezvous in trees. *ACM Transactions on Algorithms 9(2), article 17 (DISC 2008)*, 2013. doi:10.1145/2438645.2438649, arXiv:1102.0467.

Nao Fujinaga, Yukiko Yamauchi, Hirotaka Ono, Shuji Kijima, Masafumi Yamashita. Pattern formation by oblivious asynchronous mobile robots. *SIAM Journal on Computing 44(3):740–785 (DISC 2012)*, 2015. doi:10.1137/140958682.

Nicolas Gastineau, Wahabou Abdou, Nader Mbarek, Olivier Togni. Distributed Leader Election and Computation of Local Identifiers for Programmable Matter. *ALGOSENSORS 2018, LNCS*, 2019. doi:10.1007/978-3-030-14094-6_11, arXiv:1807.10461.

Konstantinos Georgiou, George Karakostas, Evangelos Kranakis. Search-and-Fetch with 2 Robots on a Disk: Wireless and Face-to-Face Communication Models. *Discrete Mathematics & Theoretical Computer Science 21(3)*, 2019. doi:10.23638/DMTCS-21-3-20, arXiv:1611.10208.

Konstantinos Georgiou, George Karakostas, Evangelos Kranakis. Treasure evacuation with one robot on a disk. *Theoretical Computer Science 852:18–28*, 2021. doi:10.1016/j.tcs.2020.11.008.

Konstantinos Georgiou, Nikos Giachoudis, Evangelos Kranakis. Evacuation from a Disk for Robots with Asymmetric Communication. *ISAAC 2022 (LIPIcs 248), 19:1–19:16*, 2022. doi:10.4230/LIPIcs.ISAAC.2022.19.

Konstantinos Georgiou, Sean Leizerovich, Jesse Lucier, Somnath Kundu. Evacuating from ℓ_p unit disks in the wireless model. *Theoretical Computer Science 944:113675*, 2023. doi:10.1016/j.tcs.2022.12.025, arXiv:2108.02367.

Konstantinos Georgiou, Evangelos Kranakis, Nikos Leonardos, Aris Pagourtzis, Ioannis Papaioannou. Optimal circle search despite the presence of faulty robots. *Information Processing Letters 182, 106391*, 2023. doi:10.1016/j.ipl.2023.106391.

Konstantinos Georgiou, Jesse Lucier. Weighted group search on a line & implications to the priority evacuation problem. *Theoretical Computer Science 939, 1-17*, 2023. doi:10.1016/j.tcs.2022.10.013.

Konstantinos Georgiou, Nikos Giachoudis, Evangelos Kranakis. Overcoming probabilistic faults in disoriented linear search. *Theoretical Computer Science 1014:114761*, 2024. doi:10.1016/j.tcs.2024.114761, arXiv:2303.15608.

Konstantinos Georgiou, Caleb Jones, Jesse Lucier. Multi-agent Search-Type Problems on Polygons. *SOFSEM 2025, LNCS, pp. 314–332*, 2025. doi:10.1007/978-3-031-82670-2_23, arXiv:2406.19495.

Konstantinos Georgiou, Caleb Jones, Matthew Madej. Spirals and Beyond: Competitive Plane Search with Multi-Speed Agents. *arXiv preprint*, 2025. arXiv:2508.10793. Preprint, not peer reviewed.

Konstantinos Georgiou, Xin Wang. Weighted group search on the disk & improved lower bounds for priority evacuation. *Journal of Computer and System Sciences 154:103678*, 2025. doi:10.1016/j.jcss.2025.103678, arXiv:2406.19490.

Konstantinos Georgiou. Optimal Average Disk-Inspection via Fermat's Principle. *LIPIcs, Volume 364, STACS 2026 44:1*, 2026. doi:10.4230/LIPIcs.STACS.2026.44, arXiv:2509.06334.

Konstantinos Georgiou, Woojin Jang. Triangle evacuation of 2 agents in the wireless model & the power of choosing a starting point. *Journal of Computer and System Sciences 155:103695*, 2026. doi:10.1016/j.jcss.2025.103695, arXiv:2209.08544.

Vincenzo Gervasi, Giuseppe Prencipe. Coordination without communication: the case of the flocking problem. *Discrete Applied Mathematics 144(3):324–344*, 2004. doi:10.1016/j.dam.2003.11.010.

Robert Gmyr, Kristian Hinnenthal, Irina Kostitsyna, Fabian Kuhn, Dorian Rudolph, Christian Scheideler, Thim Strothmann. Forming tile shapes with simple robots. *Natural Computing 19:375–390*, 2020. doi:10.1007/s11047-019-09774-2.

Barun Gorain, Kaushik Mondal, Himadri Nayak, Supantha Pandit. Pebble guided optimal treasure hunt in anonymous graphs. *Theoretical Computer Science 922:61–80 (SIROCCO 2021)*, 2022. doi:10.1016/j.tcs.2022.04.011, arXiv:2103.05933.

Barun Gorain, Partha Sarathi Mandal, Kaushik Mondal, Supantha Pandit. Collaborative Dispersion by Silent Robots. *SSS 2022 (LNCS 13751)*, 2022. Journal version: *Journal of Parallel and Distributed Computing 188:104852, 2024 (doi:10.1016/j.jpdc.2024.104852)*. doi:10.1007/978-3-031-21017-4_17, arXiv:2202.05710.

Tsuyoshi Gotoh, Paola Flocchini, Toshimitsu Masuzawa, Nicola Santoro. Exploration of dynamic networks: Tight bounds on the number of agents. *Journal of Computer and System Sciences 122:1–18 (OPODIS 2019)*, 2021. doi:10.1016/j.jcss.2021.04.003.

Samuel Guilbault, Andrzej Pelc. Asynchronous rendezvous of anonymous agents in arbitrary graphs. *OPODIS 2011, pp. 421–434*, 2011. doi:10.1007/978-3-642-25873-2_29.

Leszek Gąsieniec, Ralf Klasing, Russell Martin, Alfredo Navarra, Xiaohui Zhang. Fast periodic graph exploration with constant memory. *Journal of Computer and System Sciences 74(5)*, 2008. doi:10.1016/j.jcss.2007.09.004.

Christopher Hahn, Jonas Harbig, Peter Kling. Forming large patterns with local robots in the OBLOT model. *SAND 2024 (LIPIcs 292), 14:1–14:20*, 2024. doi:10.4230/LIPIcs.SAND.2024.14, arXiv:2404.02771.

Adam Heriban, Xavier Défago, Sébastien Tixeuil. Optimally Gathering Two Robots. *ICDCN 2018, pp. 1–10*, 2018. doi:10.1145/3154273.3154323, arXiv:1708.06183.

Attila Hideg, Tamás Lukovszki. Asynchronous filling by myopic luminous robots. *ALGOSENSORS 2020, LNCS, pp. 108–123*, 2020. doi:10.1007/978-3-030-62401-9_8, arXiv:1909.06895.

Yuya Higashikawa, Naoki Katoh, Stefan Langerman, Shin-ichi Tanigawa. Online graph exploration algorithms for cycles and trees by multiple searchers. *Journal of Combinatorial Optimization 28(2):480–495*, 2014. doi:10.1007/s10878-012-9571-y.

Himani, Supantha Pandit. Optimal Dispersion in Triangular Grids: Achieving Efficiency Without Prior Knowledge. *ICDCIT 2025 (LNCS 15507)*, 2025. doi:10.1007/978-3-031-81404-4_7.

Kristian Hinnenthal, David Liedtke, Christian Scheideler. Efficient Shape Formation by 3D Hybrid Programmable Matter: An Algorithm for Low Diameter Intermediate Structures. *SAND 2024, LIPIcs 292, 15*, 2024. doi:10.4230/LIPIcs.SAND.2024.15, arXiv:2401.17734.

Shoma Hiraoka, Shunsuke Imori, Shota Takahashi, Yuichi Sudo. Color Complexity of Recolorable Graph Exploration: Upper and Lower Bounds via Block Structure. *arXiv preprint 2609.14356*, 2026. arXiv:2609.14356, `https://arxiv.org/abs/2609.14356`. Preprint, not peer reviewed.

David Ilcinkas. Setting port numbers for fast graph exploration. *Theoretical Computer Science 401:236–242 (SIROCCO 2006)*, 2008. doi:10.1016/j.tcs.2008.03.035.

David Ilcinkas, Nicolas Nisse, David Soguet. The cost of monotonicity in distributed graph searching. *Distributed Computing 22(2):117–127 (OPODIS 2007)*, 2009. doi:10.1007/s00446-009-0089-1.

David Ilcinkas, Ahmed Mouhamadou Wade. Exploration of the T-interval-connected dynamic graphs: the case of the ring. *Theory of Computing Systems 62(5):1144–1160 (SIROCCO 2013)*, 2018. doi:10.1007/s00224-017-9796-3.

Taichi Inoue, Naoki Kitamura, Taisuke Izumi, Toshimitsu Masuzawa. Computational Power of a Single Oblivious Mobile Agent in Two-Edge-Connected Graphs. *OPODIS 2022 (LIPIcs 253), 11:1–11:18*, 2023. doi:10.4230/LIPIcs.OPODIS.2022.11.

Giuseppe F. Italiano, Debasish Pattanayak, Gokarna Sharma. Dispersion of Mobile Robots on Directed Anonymous Graphs. *SIROCCO 2022 (LNCS 13298)*, 2022. Journal version: *Journal of Parallel and Distributed Computing 204:105139, 2025 (doi:10.1016/j.jpdc.2025.105139), which adds the deficiency-based algorithm.* doi:10.1007/978-3-031-09993-9_11.

Taisuke Izumi, Samia Souissi, Yoshiaki Katayama, Nobuhiro Inuzuka, Xavier Défago, Koichi Wada, Masafumi Yamashita. The gathering problem for two oblivious robots with unreliable compasses. *SIAM Journal on Computing 41(1):26–46*, 2012. doi:10.1137/100797916, arXiv:1111.1492.

Taisuke Izumi, Daichi Kaino, Maria Gradinariu Potop-Butucaru, Sébastien Tixeuil. On time complexity for connectivity-preserving scattering of mobile robots. *Theoretical Computer Science 738:42–52 (SSS 2010)*, 2018. doi:10.1016/j.tcs.2018.04.047.

Khaled Jawhar, Evangelos Kranakis. Bike Assisted Evacuation on a Line. *Lecture Notes in Computer Science SOFSEM 2021: Theory and Practice of Computer Science, 104-118*, 2021. doi:10.1007/978-3-030-67731-2_8.

Khaled Jawhar, Evangelos Kranakis. Bike-assisted evacuation of robots on a line with asymmetric S/R communication. *Theoretical Computer Science 1053:115438*, 2025. doi:10.1016/j.tcs.2025.115438, arXiv:2307.15808.

Bo Jiang, Yuan Liu, Hao Zhang, Xuehou Tan. An On-Line Strategy of Groups Evacuation from a Convex Region in the Plane. *Mathematical Problems in Data Science, pp. 189-199*, 2015. doi:10.1007/978-3-319-25127-1_11.

Sayaka Kamei, Sébastien Tixeuil. An asynchronous maximum independent set algorithm by myopic luminous robots on grids. *The Computer Journal 67(1):57–77 (advance access 2022)*, 2024. doi:10.1093/comjnl/bxac158, arXiv:2012.03399.

Sayaka Kamei, Anissa Lamani, Fukuhito Ooshita, Sébastien Tixeuil, Koichi Wada. Gathering on rings for myopic asynchronous robots with lights. *Theory of Computing Systems 69(1):8 (OPODIS 2019, LIPIcs 153, 27:1–27:17)*, 2025. doi:10.1007/s00224-024-10199-3.

Bogumil Kaminski, Pawel Pralat, Maria Sadza. Going in Circles: Collaborative Multi-Robot Treasure Hunting. *arXiv preprint*, 2026. arXiv:2608.12115. Preprint, not peer reviewed.

Ming-Yang Kao, John H. Reif, Stephen R. Tate. Searching in an Unknown Environment: An Optimal Randomized Algorithm for the Cow-Path Problem. *Information and Computation 131(1):63–79 (SODA 1993)*, 1996. doi:10.1006/inco.1996.0092.

Branislav Katreniak. Convergence with limited visibility by asynchronous mobile robots. *SIROCCO 2011, pp. 125–137*, 2011. doi:10.1007/978-3-642-22212-2_12.

Claude (Anthropic) was used in cataloguing the problems.

Tanvir Kaur, Ashish Saxena, Partha Sarathi Mandal, Kaushik Mondal. Black hole search in dynamic graphs. *Theoretical Computer Science 1086:116237 (ICDCN 2025)*, 2026. doi:10.1016/j.tcs.2026.116237, arXiv:2405.18367.

Tanvir Kaur, Barun Gorain, Kaushik Mondal. Maximal Independent Set via Leader Election by Mobile Agents from Arbitrary Initial Configuration Without Global Knowledge. *SN Computer Science 7, article 125*, 2026. doi:10.1007/s42979-026-04726-x, arXiv:2410.19468.

Akitoshi Kawamura, Yusuke Kobayashi. Fence patrolling by mobile agents with distinct speeds. *Distributed Computing 28:147–154 (ISAAC 2012)*, 2015. doi:10.1007/s00446-014-0226-3, arXiv:1407.8194.

Akitoshi Kawamura, Makoto Soejima. Simple strategies versus optimal schedules in multi-agent patrolling. *Theoretical Computer Science 839:195–206*, 2020. doi:10.1016/j.tcs.2020.07.037, arXiv:1411.6853.

Hridesh Kedia, Shunhao Oh, Dana Randall. Local Stochastic Algorithms for Alignment in Self-Organizing Particle Systems. *APPROX/RANDOM 2022, LIPIcs 245, 14*, 2022. doi:10.4230/LIPIcs.APPROX/RANDOM.2022.14, arXiv:2207.07956.

David Kirkpatrick, Irina Kostitsyna, Alfredo Navarra, Giuseppe Prencipe, Nicola Santoro. On the power of bounded asynchrony: convergence by autonomous robots with limited visibility. *Distributed Computing 37(3):279–308 (PODC 2021, as "Separating bounded and unbounded asynchrony for autonomous robots: point convergence with limited visibility")*, 2024. doi:10.1007/s00446-024-00463-7, arXiv:2105.13042.

Naoki Kitamura, Yuichi Sudo, Koichi Wada. Two-Robot Computational Landscape: A Complete Characterization of Model Power in Minimal Mobile Robot Systems. *arXiv preprint*, 2025. arXiv:2512.22770. Preprint, not peer reviewed.

Ralf Klasing, Euripides Markou, Tomasz Radzik, Fabiano Sarracco. Hardness and approximation results for Black Hole Search in arbitrary networks. *Theoretical Computer Science 384(2–3):201–221 (SIROCCO 2005)*, 2007. doi:10.1016/j.tcs.2007.04.024.

Ralf Klasing, Euripides Markou, Andrzej Pelc. Gathering asynchronous oblivious mobile robots in a ring. *Theoretical Computer Science 390(1):27–39 (ISAAC 2006)*, 2008. doi:10.1016/j.tcs.2007.09.032.

Adrian Kosowski, Alfredo Navarra. Graph decomposition for memoryless periodic exploration. *Algorithmica 63(1–2):26–38 (MFCS 2009)*, 2012. doi:10.1007/s00453-011-9518-1.

Irina Kostitsyna, Tom Peters, Bettina Speckmann. Fast Reconfiguration for Programmable Matter. *DISC 2023, LIPIcs 281, 27*, 2023. doi:10.4230/LIPIcs.DISC.2023.27, arXiv:2202.11663.

Irina Kostitsyna, David Liedtke, Christian Scheideler. Universal Coating by 3D Hybrid Programmable Matter. *SIROCCO 2024, LNCS, pp. 384–401*, 2024. doi:10.1007/978-3-031-60603-8_21, arXiv:2303.16180.

Michal Koucký. Universal traversal sequences with backtracking. *Journal of Computer and System Sciences 65(4):717–726 (CCC 2001)*, 2002. doi:10.1016/S0022-0000(02)00023-5.

Dariusz R. Kowalski, Adam Malinowski. How to meet in anonymous network. *Theoretical Computer Science 399(1–2):141–156 (SIROCCO 2006)*, 2008. doi:10.1016/j.tcs.2008.02.010.

Evangelos Kranakis. Invited Paper: A Survey of the Impact of Knowledge on the Competitive Ratio in Linear Search. *Stabilization, Safety, and Security of Distributed Systems, pp. 23-38*, 2024. doi:10.1007/978-3-031-74498-3_2.

Ajay D. Kshemkalyani, Faizan Ali. Efficient Dispersion of Mobile Robots on Graphs. *ICDCN 2019*, 2019. doi:10.1145/3288599.3288610, arXiv:1805.12242.

Ajay D. Kshemkalyani, Anisur Rahaman Molla, Gokarna Sharma. Fast Dispersion of Mobile Robots on Arbitrary Graphs. *ALGOSENSORS 2019 (LNCS 11931)*, 2019. doi:10.1007/978-3-030-34405-4_2, arXiv:1812.05352. Preprint, not peer reviewed.

Ajay D. Kshemkalyani, Anisur Rahaman Molla, Gokarna Sharma. Efficient Dispersion of Mobile Robots on Dynamic Graphs. *ICDCS 2020*, 2020. doi:10.1109/ICDCS47774.2020.00100, https://www.cs.uic.edu/~ajayk/ext/ICDCS2020.pdf.

Ajay D. Kshemkalyani, Anisur Rahaman Molla, Gokarna Sharma. Dispersion of Mobile Robots in the Global Communication Model. *ICDCN 2020*, 2020. doi:10.1145/3369740.3369775, arXiv:1909.01957.

Ajay D. Kshemkalyani, Anisur Rahaman Molla, Gokarna Sharma. Dispersion of Mobile Robots on Grids. *WALCOM 2020 (LNCS 12049)*, 2020. doi:10.1007/978-3-030-39881-1_16, arXiv:1812.05352. Preprint, not peer reviewed.

Ajay D. Kshemkalyani, Gokarna Sharma. Near-Optimal Dispersion on Arbitrary Anonymous Graphs. *OPODIS 2021 (LIPIcs 217)*, 2021. Journal version: *Journal of Computer and System Sciences 152:103656, 2025 (doi:10.1016/j.jcss.2025.103656)*. doi:10.4230/LIPIcs.OPODIS.2021.8, arXiv:2106.03943.

Ajay D. Kshemkalyani, Anisur Rahaman Molla, Gokarna Sharma. Dispersion of mobile robots using global communication. *Journal of Parallel and Distributed Computing 161:100–117*, 2022. doi:10.1016/j.jpdc.2021.11.007.

Ajay D. Kshemkalyani, Manish Kumar, Anisur Rahaman Molla, Gokarna Sharma. Brief Announcement: Agent-Based Leader Election, MST, and Beyond. *DISC 2024 brief announcement (LIPIcs 319), 50:1–50:7*, 2024. doi:10.4230/LIPIcs.DISC.2024.50, arXiv:2403.13716.

Ajay D. Kshemkalyani, Manish Kumar, Anisur Rahaman Molla, Debasish Pattanayak, Gokarna Sharma. Dispersion is (Almost) Optimal under (A)synchrony. *SPAA 2025*, 2025. doi:10.1145/3694906.3743317, arXiv:2503.16216.

Ajay D. Kshemkalyani. Dispersion of mobile robots on graphs in the asynchronous model. *Theoretical Computer Science 1044:115272*, 2025. doi:10.1016/j.tcs.2025.115272.

Ajay D. Kshemkalyani, Manish Kumar, Anisur Rahaman Molla, Gokarna Sharma. Near-Linear Time Leader Election in Multiagent Networks. *AAMAS 2025, pp. 1218–1226*, 2025. doi:10.65109/BKZJ3357.

Ajay D. Kshemkalyani, Manish Kumar, Anisur Rahaman Molla, Debasish Pattanayak, Gokarna Sharma. Faster leader election via mobile agents and its applications. *Theoretical Computer Science 1059:115578*, 2026. doi:10.1016/j.tcs.2025.115578.

Fabian Kuhn, Nancy Lynch, Rotem Oshman. Distributed computation in dynamic networks. *STOC 2010, pp. 513–522*, 2010. doi:10.1145/1806689.1806760, `https://people.csail.mit.edu/rotem/dynamicgraphs.pdf`.

Andrey Kupavskii, Emo Welzl. Lower bounds for searching robots, some faulty. *PODC 2018*, 2018. arXiv:1707.05077.

Anissa Lamani, Maria Gradinariu Potop-Butucaru, Sébastien Tixeuil. Optimal Deterministic Ring Exploration with Oblivious Asynchronous Robots. *SIROCCO 2010*, 2010. doi:10.1007/978-3-642-13284-1_15, arXiv:0910.0832.

Ioannis Lamprou, Russell Martin, Sven Schewe. Fast two-robot disk evacuation with wireless communication. *Theoretical Computer Science 846:38–60*, 2020. doi:10.1016/j.tcs.2020.09.019, arXiv:1604.04217.

Nikos Leonardos, Aris Pagourtzis, Ioannis Papaioannou. Byzantine fault tolerant symmetric-persistent circle evacuation. *Theoretical Computer Science 956:113839*, 2023. doi:10.1016/j.tcs.2023.113839.

Songhua Li, Yinfeng Xu. Online Strategies for Evacuating from a Convex Region in the Plane. *Frontiers in Algorithmics, pp. 151-162*, 2017. doi:10.1007/978-3-319-59605-1_14.

Yuan Liu, Bo Jiang, Hao Zhang. Online Strategies for Evacuating from a Convex Region by Groups in the Plane. *2015 Ninth International Conference on Frontier of Computer Science and Technology, pp. 178-183*, 2015. doi:10.1109/fcst.2015.70.

Zhiqiang Liu, Yukiko Yamauchi, Shuji Kijima, Masafumi Yamashita. Team assembling problem for asynchronous heterogeneous mobile robots. *Theoretical Computer Science 721:27–41*, 2018. doi:10.1016/j.tcs.2018.01.009.

Luan Liu, Qi Wei, Yan Zhang, Jie Sun, Wenxin Zhang. Online Algorithm for Evacuating Crashed Faulty Robots from A Square. *2022 IEEE 8th International Conference on Computer and Communications (ICCC), 2276-2280*, 2022. doi:10.1109/iccc56324.2022.10065611.

Marcello Mamino, Giovanni Viglietta. Square formation by asynchronous oblivious robots. *CCCG 2016*, 2016. arXiv:1605.06093.

Subhrangsu Mandal, Anisur Rahaman Molla, William K. Moses Jr. Efficient live exploration of a dynamic ring with mobile robots. *Theoretical Computer Science 980:114201*, 2023. doi:10.1016/j.tcs.2023.114201, arXiv:2001.04525.

Toshimitsu Masuzawa, Yuichi Sudo, Koichi Wada. Rendezvous under Variable Disorientation: The Algorithmic Power of Fixed Unit Distance. *arXiv preprint*, 2026. arXiv:2610.09713. Preprint, not peer reviewed.

Artur Menc, Dominik Pająk, Przemysław Uznański. Time and space optimality of rotor-router graph exploration. *Information Processing Letters 127:17–20*, 2017. doi:10.1016/j.ipl.2017.06.010, arXiv:1502.05545.

Othon Michail, Paul G. Spirakis. Traveling salesman problems in temporal graphs. *Theoretical Computer Science 634:1–23*, 2016. doi:10.1016/j.tcs.2016.04.006.

Othon Michail, George Skretas, Paul G. Spirakis. On the transformation capability of feasible mechanisms for programmable matter. *Journal of Computer and System Sciences 102:18–39*, 2019. doi:10.1016/j.jcss.2018.12.001, arXiv:1703.04381.

Avery Miller, Andrzej Pelc. Tradeoffs between cost and information for rendezvous and treasure hunt. *Journal of Parallel and Distributed Computing 83:159–167 (OPODIS 2014)*, 2015. doi:10.1016/j.jpdc.2015.06.004, arXiv:1506.07952.

Anisur Rahaman Molla, William K. Moses Jr. Dispersion of Mobile Robots: The Power of Randomness. *TAMC 2019 (LNCS 11436)*, 2019. doi:10.1007/978-3-030-14812-6_30, arXiv:1902.10489.

Claude (Anthropic) was used in cataloguing the problems.

Anisur Rahaman Molla, Kaushik Mondal, William K. Moses Jr. Efficient Dispersion on an Anonymous Ring in the Presence of Weak Byzantine Robots. *ALGOSENSORS 2020 (LNCS 12503)*, 2020. Journal version: *Optimal dispersion on an anonymous ring in the presence of weak Byzantine robots, Theoretical Computer Science 887:111–121, 2021 (doi:10.1016/j.tcs.2021.07.008)*. doi:10.1007/978-3-030-62401-9_11, arXiv:2004.11439.

Anisur Rahaman Molla, Kaushik Mondal, William K. Moses Jr. Byzantine Dispersion on Graphs. *IPDPS 2021*, 2021. doi:10.1109/IPDPS49936.2021.00103, arXiv:2102.07528.

Anisur Rahaman Molla, Kaushik Mondal, William K. Moses Jr. Fast Deterministic Gathering with Detection on Arbitrary Graphs: The Power of Many Robots. *IPDPS 2023, pp. 47–57*, 2023. doi:10.1109/IPDPS54959.2023.00015.

Moumita Mondal, Sruti Gan Chaudhuri. Uniform Circle Formation by Transparent Fat Robots. *arXiv preprint*, 2017. arXiv:1710.09423. Preprint, not peer reviewed.

Moumita Mondal, Sruti Gan Chaudhuri, Ayan Dutta, Krishnendu Mukhopadhyaya, Punyasha Chatterjee. Uniform Circle Formation By Oblivious Swarm Robots. *arXiv preprint*, 2020. arXiv:2012.07113. Preprint, not peer reviewed.

Brati Mondal, Pritam Goswami, Buddhadeb Sau. Time Optimal Distance-k-Dispersion on Dynamic Ring. *ICDCN 2025*, 2025. doi:10.1145/3700838.3700843, arXiv:2408.12220.

Brati Mondal, Pritam Goswami, Buddhadeb Sau. Simulating Chirality: Solving Distance-k-Dispersion on an 1-Interval Connected Ring. *arXiv preprint*, 2025. arXiv:2507.14723. Preprint, not peer reviewed.

William K. Moses Jr., Amanda Redlich. Dispersion, Capacitated Nodes, and the Power of a Trusted Shepherd. *ICDCN 2024*, 2024. doi:10.1145/3631461.3632310, arXiv:2311.01511.

Shota Nagahama, Fukuhito Ooshita, Michiko Inoue. Terminating grid exploration with myopic luminous robots. *International Journal of Networking and Computing 12(1):73–102 (APDCM 2021, IPDPS Workshops, pp. 586–595)*, 2022. doi:10.15803/ijnc.12.1_73.

Shota Nagahama, Fukuhito Ooshita, Michiko Inoue. Ring exploration of myopic luminous robots with visibility more than one. *Information and Computation 292:105036 (SSS 2019, LNCS, pp. 256–271)*, 2023. doi:10.1016/j.ic.2023.105036.

Shota Naito, Tsukasa Ninomiya, Koichi Wada. Beyond Pairwise Comparisons: Unveiling Structural Landscape of Mobile Robot Models. *SSS 2025, LNCS, pp. 408–424*, 2025. doi:10.1007/978-3-032-11127-2_31, arXiv:2508.19805.

Rikuo Nakai, Yuichi Sudo, Koichi Wada. Asynchronous Gathering Algorithms for Autonomous Mobile Robots with Lights. *SSS 2021, LNCS, pp. 410–424*, 2021. doi:10.1007/978-3-030-91081-5_27, arXiv:2109.12289.

Keita Nakajima, Kaito Takase, Koichi Wada. Efficient Self-stabilizing Simulations of Energy-Restricted Mobile Robots by Asynchronous Luminous Mobile Robots. *SSS 2024, LNCS, pp. 141–155*, 2024. doi:10.1007/978-3-031-74498-3_10, arXiv:2403.05542.

Alfredo Navarra, Francesco Piselli. Silent Programmable Matter: Coating. *OPODIS 2023, LIPIcs 286, 25:1*, 2023. doi:10.4230/LIPIcs.OPODIS.2023.25.

Alfredo Navarra, Francesco Piselli. Asynchronous Silent Programmable Matter: Line Formation. *SSS 2023, LNCS, pp. 598–612*, 2023. doi:10.1007/978-3-031-44274-2_44, arXiv:2307.16731.

Alfredo Navarra, Francesco Piselli, Giuseppe Prencipe. Line formation and scattering in silent programmable matter. *Journal of Parallel and Distributed Computing 204:105129*, 2025. doi:10.1016/j.jpdc.2025.105129.

Shunhao Oh, Joseph L. Briones, Jacob Calvert, Noah Egan, Dana Randall, Andréa W. Richa. Single Bridge Formation in Self-Organizing Particle Systems. *DISC 2024, LIPIcs 319, 34*, 2024. doi:10.4230/LIPIcs.DISC.2024.34, arXiv:2408.10830.

Takashi Okumura, Koichi Wada, Yoshiaki Katayama. Brief Announcement: Optimal Asynchronous Rendezvous for Mobile Robots with Lights. *SSS 2017 (brief announcement), LNCS, pp. 484–488*, 2017. doi:10.1007/978-3-319-69084-1_36, arXiv:1707.04449.

Takashi Okumura, Koichi Wada, Xavier Défago. Optimal L-algorithms for rendezvous of asynchronous mobile robots with external-lights. *Theoretical Computer Science 979:114198*, 2023. doi:10.1016/j.tcs.2023.114198, arXiv:1804.01368.

Fukuhito Ooshita, Sébastien Tixeuil. Ring exploration with myopic luminous robots. *Information and Computation 285:104702 (SSS 2018, LNCS, pp. 301–316)*, 2022. doi:10.1016/j.ic.2021.104702, arXiv:1805.03965.

Fukuhito Ooshita, Naoki Kitamura, Ryota Eguchi, Michiko Inoue, Hirotsugu Kakugawa, Sayaka Kamei, Masahiro Shibata, Yuichi Sudo. Crash-tolerant perpetual exploration with myopic luminous robots on rings. *OPODIS 2024 (LIPIcs 324), 12:1–12:16*, 2025. doi:10.4230/LIPIcs.OPODIS.2024.12.

Fukuhito Ooshita, Hirotsugu Kakugawa, Sayaka Kamei, Masahiro Shibata. Crash-tolerant perpetual exploration of myopic luminous robots with fewer colors on rings. *CANDAR Workshops 2025, pp. 166–172*, 2025. doi:10.1109/CANDARW68385.2025.00036.

Kohei Otaka, Fabian Frei, Koichi Wada. Gathering semi-synchronously scheduled two-state robots. *Theoretical Computer Science 1068:115776*, 2026. doi:10.1016/j.tcs.2026.115776, arXiv:2408.09999.

Andreas Padalkin, Manish Kumar, Christian Scheideler. Reconfiguration and locomotion with joint movements in the amoebot model. *Autonomous Robots 49*, 2025. doi:10.1007/s10514-025-10204-9, arXiv:2305.06146.

Linda Pagli, Giuseppe Prencipe, Giovanni Viglietta. Getting close without touching: near-gathering for autonomous mobile robots. *Distributed Computing 28(5):333–349 (SIROCCO 2012)*, 2015. doi:10.1007/s00446-015-0248-5, arXiv:1505.07168.

Debasish Pattanayak, H. Ramesh, Partha Sarathi Mandal, Stefan Schmid. Evacuating Two Robots from Two Unknown Exits on the Perimeter of a Disk with Wireless Communication. *ICDCN 2018 (short paper)*, 2018. doi:10.1145/3154273.3154313, arXiv:1708.03792.

Debasish Pattanayak, H. Ramesh, Partha Sarathi Mandal. Chauffeuring a Crashed Robot from a Disk. *ALGOSENSORS 2019, pp. 177–191*, 2019. doi:10.1007/978-3-030-34405-4_10, arXiv:1906.03024.

Debasish Pattanayak, Kaushik Mondal, H. Ramesh, Partha Sarathi Mandal. Gathering of mobile robots with weak multiplicity detection in presence of crash-faults. *Journal of Parallel and Distributed Computing 123:145–155 (ICDCN 2017)*, 2019. doi:10.1016/j.jpdc.2018.09.015, arXiv:1608.02432.

Debasish Pattanayak, H. Ramesh, Partha Sarathi Mandal. Collaborative Evacuation of Mobile Robots. *Proceedings of the 21st International Conference on Distributed Computing and Networking, 1-1*, 2020. doi:10.1145/3369740.3373806.

Debasish Pattanayak, Gokarna Sharma, Partha Sarathi Mandal. Dispersion of Mobile Robots Tolerating Faults. *ICDCN 2021 Adjunct (WDALFR workshop)*, 2021. doi:10.1145/3427477.3429464.

Debasish Pattanayak, Gokarna Sharma, Partha Sarathi Mandal. Dispersion of Mobile Robots in Spite of Faults. *SSS 2023 (LNCS 14310)*, 2023. doi:10.1007/978-3-031-44274-2_31.

Debasish Pattanayak, Subhash Bhagat, Sruti Gan Chaudhuri, Anisur Rahaman Molla. Maximal Independent Set via Mobile Agents. *ICDCN 2024, pp. 74–83*, 2024. doi:10.1145/3631461.3631543.

Debasish Pattanayak, Andrzej Pelc. Graph exploration by a deterministic memoryless automaton with pebbles. *Discrete Applied Mathematics 356:149–160*, 2024. doi:10.1016/j.dam.2024.05.024.

Debasish Pattanayak, Gokarna Sharma. Time-color tradeoff on uniform circle formation by asynchronous robots. *IPDPS 2024, pp. 987–997*, 2024. doi:10.1109/IPDPS57955.2024.00092.

Debasish Pattanayak, Andrzej Pelc. Deterministic treasure hunt and rendezvous in arbitrary connected graphs. *Information Processing Letters 185:106455*, 2024. doi:10.1016/j.ipl.2023.106455, arXiv:2310.01136.

Debasish Pattanayak, Ajay D. Kshemkalyani, Manish Kumar, Anisur Rahaman Molla, Gokarna Sharma. Brief Announcement: Optimal Dispersion Under Asynchrony. *DISC 2025, brief announcement (LIPIcs 356)*, 2025. Journal version: *Full version: Asynchronous Dispersion with Optimal Time Complexity, ICPP 2026 (doi:10.1145/3832810.3832921); brief announcement also at SPAA 2026 (doi:10.1145/3816782.3819188).* doi:10.4230/LIPIcs.DISC.2025.63, arXiv:2507.01298.

Andrzej Pelc. Deterministic gathering with crash faults. *Networks 72(2):182–199*, 2018. doi:10.1002/net.21810, arXiv:1704.08880.

Andrzej Pelc. Deterministic rendezvous algorithms. *Distributed Computing by Mobile Entities, LNCS 11340, pp. 423–454*, 2019. doi:10.1007/978-3-030-11072-7_17, arXiv:2303.10391.

David Peleg. Distributed Coordination Algorithms for Mobile Robot Swarms: New Directions and Challenges. *IWDC 2005, LNCS 3741, pp. 1–12*, 2005. doi:10.1007/11603771_1.

Pavan Poudel, Aisha Aljohani, Gokarna Sharma. Fault-tolerant complete visibility for asynchronous robots with lights under one-axis agreement. *Theoretical Computer Science 850:116–134*, 2021. doi:10.1016/j.tcs.2020.10.033.

Pavan Poudel, Gokarna Sharma. Time-Optimal Gathering under Limited Visibility with One-Axis Agreement. *Information 12(11):448 (SSS 2017)*, 2021. doi:10.3390/info12110448.

Subhajit Pramanick, Sai Vamshi Samala, Debasish Pattanayak, Partha Sarathi Mandal. Distributed algorithms for filling MIS vertices of an arbitrary graph by myopic luminous robots. *Theoretical Computer Science*

Claude (Anthropic) was used in cataloguing the problems.

978:114187 (ICDCIT 2023, LNCS, pp. 3–19, as "Filling MIS Vertices of a Graph by Myopic Luminous Robots"), 2023. doi:10.1016/j.tcs.2023.114187, arXiv:2107.04885.

Subhajit Pramanick, Saswata Jana, Partha Sarathi Mandal, Gokarna Sharma. Asynchronous Gathering of Opaque Robots with Mobility Faults. *arXiv preprint*, 2025. arXiv:2509.10711. Preprint, not peer reviewed.

Giuseppe Prencipe. Impossibility of gathering by a set of autonomous mobile robots. *Theoretical Computer Science 384(2–3):222–231 (SIROCCO 2005)*, 2007. doi:10.1016/j.tcs.2007.04.023.

V. B. Priezzhev, Deepak Dhar, Abhishek Dhar, Supriya Krishnamurthy. Eulerian Walkers as a Model of Self-Organized Criticality. *Physical Review Letters 77(25):5079–5082*, 1996. doi:10.1103/PhysRevLett.77.5079.

Prajyot Pyati, Navjot Kaur, Saswata Jana, Adri Bhattacharya, Partha Sarathi Mandal. Separation of Unconscious Robots with Obstructed Visibility. *ICDCIT 2026, pp. 68–84*, 2026. doi:10.1007/978-3-032-16632-6_5.

Arthur Rauch, Quentin Bramas, Stéphane Devismes, Pascal Lafourcade, Anissa Lamani. Optimal exclusive perpetual grid exploration by luminous myopic robots without common chirality. *NETYS 2021, LNCS, pp. 95–110*, 2021. doi:10.1007/978-3-030-91014-3_7, https://inria.hal.science/hal-03547159.

Omer Reingold. Undirected connectivity in log-space. *Journal of the ACM 55(4), article 17 (STOC 2005)*, 2008. doi:10.1145/1391289.1391291.

Hans-Anton Rollik. Automaten in planaren Graphen. *Acta Informatica 13(3):287–298*, 1980. doi:10.1007/BF00288647.

Ashish Saxena, Tanvir Kaur, Kaushik Mondal. Dispersion on Time Varying Graphs. *ICDCN 2025*, 2025. Journal version: *Balanced dispersion on time-varying dynamic graphs, Journal of Computer and System Sciences 161:103829, 2026 (doi:10.1016/j.jcss.2026.103829)*. doi:10.1145/3700838.3700845, arXiv:2410.04050.

Ashish Saxena, Kaushik Mondal. Path Connected Dynamic Graphs with a Study of Efficient Dispersion. *ICDCN 2025*, 2025. Journal version: *Theoretical Computer Science 1050:115390, 2025 (doi:10.1016/j.tcs.2025.115390)*. doi:10.1145/3700838.3700864, arXiv:2504.08474.

Ashish Saxena, Kaushik Mondal. Exploration on highly dynamic graphs. *Distributed Computing 39 (2026)*, 2026. doi:10.1007/s00446-026-00516-z, arXiv:2601.13047.

Hirokazu Seike, Yukiko Yamauchi. Separation of Unconscious Colored Robots. *SSS 2023, pp. 328–343*, 2023. doi:10.1007/978-3-031-44274-2_24.

Claude E. Shannon. Presentation of a maze-solving machine. *Transactions of the 8th Conference on Cybernetics (Macy Conference), pp. 173–180*, 1951. https://www.technologyreview.com/2019/08/21/133555/of-mice-men-and-computers/.

Gokarna Sharma, Costas Busch, Supratik Mukhopadhyay. Mutual Visibility with an Optimal Number of Colors. *ALGOSENSORS 2015, pp. 196–210*, 2015. doi:10.1007/978-3-319-28472-9_15.

Gokarna Sharma, Ramachandran Vaidyanathan, Jerry L. Trahan, Costas Busch, Suresh Rai. Complete Visibility for Robots with Lights in O(1) Time. *SSS 2016, LNCS 10083, pp. 327–345*, 2016. doi:10.1007/978-3-319-49259-9_26.

Gokarna Sharma, Ramachandran Vaidyanathan, Jerry L. Trahan, Costas Busch, Suresh Rai. O(log N)-Time Complete Visibility for Asynchronous Robots with Lights. *IPDPS 2017, pp. 513–522*, 2017. doi:10.1109/IPDPS.2017.51.

Gokarna Sharma, Rusul Alsaedi, Costas Busch, Supratik Mukhopadhyay. The Complete Visibility Problem for Fat Robots with Lights. *ICDCN 2018*, 2018. doi:10.1145/3154273.3154319.

Gokarna Sharma, Ramachandran Vaidyanathan, Jerry L. Trahan. Constant-Time Complete Visibility for Robots with Lights: The Asynchronous Case. *Algorithms 14(2):56*, 2021. doi:10.3390/a14020056.

Avisek Sharma, Pritam Goswami, Buddhadeb Sau. Computational Power of Mobile Robots: Discrete Version. *ICDCN 2025, pp. 231–238*, 2025. doi:10.1145/3700838.3700870, arXiv:2407.05678.

Kohei Shimoyama, Yuichi Sudo, Hirotsugu Kakugawa, Toshimitsu Masuzawa. One Bit Agent Memory is Enough for Snap-Stabilizing Perpetual Exploration of Cactus Graphs with Distinguishable Cycles. *SSS 2022 (invited paper), LNCS 13751, pp. 19–34*, 2022. doi:10.1007/978-3-031-21017-4_2.

Takahiro Shintaku, Yuichi Sudo, Hirotsugu Kakugawa, Toshimitsu Masuzawa. Efficient Dispersion of Mobile Agents without Global Knowledge. *SSS 2020 (LNCS 12514)*, 2020. doi:10.1007/978-3-030-64348-5_22, arXiv:2008.09379.

Yuichi Sudo, Daisuke Baba, Junya Nakamura, Fukuhito Ooshita, Hirotsugu Kakugawa, Toshimitsu Masuzawa. A Single Agent Exploration in Unknown Undirected Graphs with Whiteboards. *IEICE Transactions on Fun-*

damentals of Electronics, Communications and Computer Sciences E98.A(10):2117–2128, 2015. doi:10.1587/transfun.E98.A.2117.

Yuichi Sudo, Masahiro Shibata, Junya Nakamura, Yonghwan Kim, Toshimitsu Masuzawa. Near-Linear Time Dispersion of Mobile Agents. *DISC 2024 (LIPIcs 319)*, 2024. Journal version: *Distributed Computing 39(1), 2026 (doi:10.1007/s00446-025-00494-8)*. doi:10.4230/LIPIcs.DISC.2024.38, arXiv:2310.04376.

Yuichi Sudo, Fukuhito Ooshita, Sayaka Kamei. Self-stabilizing graph exploration by a single agent. *Theoretical Computer Science 1082:116085*, 2026. doi:10.1016/j.tcs.2026.116085, arXiv:2010.08929.

Xiaoming Sun, Yuan Sun, Jialin Zhang. Better Upper Bounds for Searching on a Line with Byzantine Robots. *Complexity and Approximation, LNCS 12000, pp. 151–171*, 2020. doi:10.1007/978-3-030-41672-0_9.

Ichiro Suzuki, Masafumi Yamashita. Distributed anonymous mobile robots: formation of geometric patterns. *SIAM Journal on Computing 28(4):1347–1363*, 1999. doi:10.1137/S009753979628292X.

Amnon Ta-Shma, Uri Zwick. Deterministic rendezvous, treasure hunts, and strongly universal exploration sequences. *ACM Transactions on Algorithms 10(3), article 12 (SODA 2007)*, 2014. doi:10.1145/2601068.

Shota Takahashi, Haruki Kanaya, Shoma Hiraoka, Ryota Eguchi, Yuichi Sudo. Recolorable Graph Exploration by an Oblivious Agent with Fewer Colors. *OPODIS 2025 (LIPIcs 361), 32:1–32:15*, 2026. doi:10.4230/LIPIcs.OPODIS.2025.32, arXiv:2505.02789.

Satoshi Terai, Koichi Wada, Yoshiaki Katayama. Gathering problems for autonomous mobile robots with lights. *Theoretical Computer Science 941:241–261*, 2023. doi:10.1016/j.tcs.2022.11.018, arXiv:1811.12068.

Tommaso Toffoli, Norman Margolus. Programmable matter: Concepts and realization. *Physica D: Nonlinear Phenomena 47(1–2):263–272*, 1991. doi:10.1016/0167-2789(91)90296-L.

Ramachandran Vaidyanathan, Costas Busch, Jerry L. Trahan, Gokarna Sharma, Suresh Rai. Logarithmic-Time Complete Visibility for Robots with Lights. *IPDPS 2015, pp. 375–384*, 2015. doi:10.1109/IPDPS.2015.52.

Giovanni Viglietta. Rendezvous of Two Robots with Visible Bits. *ALGOSENSORS 2013, pp. 291–306*, 2013. doi:10.1007/978-3-642-45346-5_21, arXiv:1211.6039.

Qi WEI, Xuehou TAN, Bo JIANG. Competitive Strategies for Evacuating from an Unknown Affected Area. *IEICE Transactions on Information and Systems E99.D(10):2585–2590*, 2016. doi:10.1587/transinf.2016edp7118.

Jamison W. Weber, Tishya Chhabra, Andréa W. Richa, Joshua J. Daymude. Energy-Constrained Programmable Matter Under Unfair Adversaries. *OPODIS 2023, LIPIcs 286, 7:1–7:21*, 2023. doi:10.4230/LIPIcs.OPODIS.2023.7, arXiv:2309.04898.

Qi Wei, Xuehou Tan, Bo Jiang, Lijuan Wang. On-Line Strategies for Evacuating from a Convex Region in the Plane. *Combinatorial Optimization and Applications, pp. 74-85*, 2014. doi:10.1007/978-3-319-12691-3_7.

Qi Wei, Xuehou Tan, Bo Jiang, Lijuan Wang. Evacuating from an Unknown Affected Area. *International Journal of Pattern Recognition and Artificial Intelligence 30, pp. 1659022*, 2016. doi:10.1142/s0218001416590229.

Qin Xin. Faster treasure hunt and better strongly universal exploration sequences. *ISAAC 2007, pp. 549–560*, 2007. doi:10.1007/978-3-540-77120-3_48, arXiv:1204.5442.

Yinfeng Xu, Lan Qin. Strategies of Groups Evacuation from a Convex Region in the Plane. *Frontiers in Algorithmics and Algorithmic Aspects in Information and Management, pp. 250-260*, 2013. doi:10.1007/978-3-642-38756-2_26.

Yukiko Yamauchi, Masafumi Yamashita. Randomized Pattern Formation Algorithm for Asynchronous Oblivious Mobile Robots. *DISC 2014, LNCS, pp. 137–151*, 2014. doi:10.1007/978-3-662-45174-8_10.

Vladimir Yanovski, Israel A. Wagner, Alfred M. Bruckstein. A distributed ant algorithm for efficiently patrolling a network. *Algorithmica 37(3):165–186*, 2003. doi:10.1007/s00453-003-1030-9.

Meng Zhang, Yi Zhang, Jijun Tang. Label-Guided Graph Exploration with Adjustable Ratio of Labels. *arXiv preprint 1204.2955*, 2012. arXiv:1204.2955, https://arxiv.org/abs/1204.2955. Preprint, not peer reviewed.